รายงานวิชา 01418321

System Analysis and Design

การวิเคราะห์และการออกแบบระบบ

เรื่อง

ระบบจัดการหลังบ้านอัตโนมัติ Colorado Co., Ltd.

เฟส 1: Rakuten Ichiba, Rakuten RMS และ RSL

เสนอ

อาจารย์ สมโชค เรืองอิทธินันท์

จัดทำโดย

นายธนภัทร พูลโภคา

6710450911 หมู่ 200

นายอชิระ ภู่รุ่งเรือง

6710451488 หมู่ 200

นางสาวปิยากร ผลพานิช

6710451054 หมู่ 200

รายงานนี้เป็นส่วนหนึ่งของรายวิชา 01418321 การวิเคราะห์และการออกแบบระบบ

ภาควิชาวิทยาการคอมพิวเตอร์ คณะวิทยาศาสตร์ มหาวิทยาลัยเกษตรศาสตร์

ภาคต้น ปีการศึกษา 2569

บทคัดย่อ

Colorado Co., Ltd. ขายสินค้านำเข้าและสินค้า OEM ผ่าน Rakuten Ichiba ประเทศญี่ปุ่น โดยใช้ Rakuten Super Logistics (RSL) เป็นคลังสินค้าและผู้จัดส่ง งานหลังบ้านทั้งหมด ได้แก่ ตรวจคำสั่งซื้อ เช็กสต๊อก จับคู่คำสั่งซื้อกับ RSL พิมพ์ใบปะสินค้า แจ้งเลขติดตามพัสดุ ตัดสินใจสั่งซื้อเติมสต๊อก และคำนวณต้นทุนต่อหน่วย เจ้าของทำเองด้วยมือคนเดียว จากการประมาณการใช้เวลาประมาณ 1–2 ชั่วโมงต่อวัน และการตัดสินใจสั่งซื้อยังไม่มีเกณฑ์ที่ชัดเจน

รายงานนี้วิเคราะห์และออกแบบระบบจัดการหลังบ้านเฟส 1 สำหรับ Rakuten Ichiba, Rakuten RMS และ RSL ตามขั้นตอนการวิเคราะห์และออกแบบระบบ ตั้งแต่ Business Process เดิมและใหม่ Use Case Diagram, CRUD Table, Use Case Description 17 รายการ พร้อม Sequence Diagram และ Collaboration Diagram ER Diagram, โครงสร้างฐานข้อมูล 17 ตาราง, Class Diagram และ State Diagram ระบบใหม่ทำงานอัตโนมัติ 3 เรื่องหลัก คือ จัดรูปแบบใบปะสินค้า แจ้งเตือนและสั่งซื้อเติมสต๊อกเมื่อต่ำกว่าเกณฑ์ และคำนวณต้นทุนต่อหน่วย โดยยังให้เจ้าของเป็นผู้อนุมัติขั้นตอนที่มีผลทางการเงิน (human-in-the-loop) ผู้จัดทำพัฒนาเว็บแอปพลิเคชันด้วย Next.js และทดสอบตาม Use Case Description ครบทุกกรณีปกติและกรณีผิดพลาด ผลการทดสอบผ่านทุกกรณี

คำสำคัญ: ระบบจัดการหลังบ้าน, Rakuten Ichiba, ใบปะสินค้าอัตโนมัติ, การสั่งซื้อเติมสต๊อก, ต้นทุนต่อหน่วย

Abstract

Colorado Co., Ltd. sells imported and OEM products on Rakuten Ichiba in Japan and uses Rakuten Super Logistics (RSL) as its warehouse and fulfillment partner. The owner handles all back-office work alone and by hand: checking new orders, checking stock, matching orders with RSL, printing shipping labels, sending tracking numbers, deciding when to reorder, and calculating unit cost. We estimate this takes about one to two hours a day, and reorder decisions have no clear rule.

This report analyzes and designs phase 1 of a back-office system for Rakuten Ichiba, Rakuten RMS and RSL. It covers the as-is and to-be business processes, use case diagrams, CRUD tables, 17 use case descriptions with sequence and collaboration diagrams, an ER diagram, a 17-table database, and class and state diagrams. The new system automates three tasks: formatting shipping labels, flagging and placing reorders when stock falls below a threshold, and calculating unit cost. The owner still approves every step that spends money (human-in-the-loop). We built the system as a Next.js web application and tested it against every normal and error case in the use case descriptions. All cases passed.

Keywords: back-office system, Rakuten Ichiba, automatic shipping label, reorder, unit cost

สารบัญ

หน้า

บทคัดย่อ 2
Abstract 3
บทที่ 1 ที่มาและความสำคัญ 20
1.1 ที่มาและความสำคัญของปัญหา 20
1.2 ปัญหาที่พบ 20
1.3 วัตถุประสงค์ 22
1.4 อุปกรณ์ที่ใช้ 23
1.4.1 Next.js 16 และ React 19 ใช้สำหรับพัฒนาหน้าจอระบบ 23
1.4.2 ภาษา TypeScript ใช้สำหรับพัฒนาระบบ 23
1.4.3 Tailwind CSS และ shadcn/ui ใช้สำหรับออกแบบหน้าจอ 23
1.4.4 ฐานข้อมูล PostgreSQL ใช้สำหรับเก็บข้อมูลของระบบ 23
1.4.5 Rakuten RMS API และ RSL ใช้สำหรับเชื่อมคำสั่งซื้อ สต๊อก และการจัดส่ง (เว็บต้นแบบเชื่อมผ่าน adapter ที่จำลองระบบภายนอก) 23
1.4.6 โปรแกรม Visual Studio Code ใช้สำหรับเขียนโปรแกรม 23
1.4.7 Git และ GitHub ใช้สำหรับจัดเก็บโค้ดและทำงานร่วมกัน 23
1.4.8 draw.io และ Mermaid ใช้สำหรับวาด diagram 23
1.4.9 Google Chrome ใช้สำหรับทดสอบระบบ 23
1.5 ขอบเขตการทำงาน 23
1.6 ขั้นตอนการทำงาน 25
1.6.1 ประชุมภายในกลุ่ม เพื่อเลือกหน่วยงานและกำหนดปัญหาที่ต้องการแก้ 25
1.6.2 สัมภาษณ์เจ้าของบริษัท เพื่อเก็บความต้องการ ขั้นตอนการทำงานเดิม และปัญหาที่พบ 25
1.6.3 วิเคราะห์ Business Process เดิม ระบุจุดที่เป็นปัญหา และออกแบบ Business Process ใหม่ 25
1.6.4 แปลง Business Process เป็น Use Case Diagram และ CRUD Table เพื่อหา Use Case ที่ขาด 25
1.6.5 นำเสนอแบบต่ออาจารย์ และปรับแบบตามคำแนะนำ เช่น ให้เจ้าของอนุมัติการสั่งซื้อก่อน (human-in-the-loop) และเพิ่มการลบข้อมูลเก่า 25
1.6.6 เขียน Use Case Description พร้อม Sequence Diagram และ Collaboration Diagram 25
1.6.7 ออกแบบ ER Diagram, โครงสร้างตาราง, Class Diagram และ State Diagram 25
1.6.8 พัฒนาเว็บแอปพลิเคชันด้วย Next.js และทดสอบ SQL ทุกคำสั่งกับข้อมูลตัวอย่าง 25
1.6.9 ทดสอบระบบตาม Use Case Description ทั้งกรณีปกติและกรณีผิดพลาด 25

1.6.10 จัดทำรายงานและเตรียมนำเสนอ 25
1.7 ประโยชน์ที่คาดว่าจะได้รับ 25
บทที่ 2 การวิเคราะห์และออกแบบระบบ 26
2.1 Business Process เดิม (As-Is) 26
2.2 คำอธิบาย Business Process เดิม 27
2.3 ปัญหาของ Business Process เดิม 28
2.4 Business Process ใหม่ (To-Be) 30
2.5 คำอธิบาย Business Process ใหม่ 31
2.6 Business Process ที่สัมพันธ์กับ Use Case 33
2.7 ตารางจับคู่ระหว่าง Business Process และ Use Case Diagram 34
2.8 Use Case Diagram ก่อนปรับปรุง 35
2.9 CRUD Table ก่อนปรับปรุง 36
2.10 คำอธิบาย CRUD Table ก่อนปรับปรุง 37
2.11 Use Case Diagram ที่ทบทวนปรับปรุงแล้ว 38
2.12 CRUD Table ที่ทบทวนแล้ว 39
2.13 Use Case Description, Sequence Diagram, Collaboration Diagram และ User Interface 41
2.13.1 Use Case Description ที่ 1A: เข้าสู่ระบบ 41
2.13.2 Sequence Diagram ของ Use Case Description ที่ 1A 44
2.13.3 Collaboration Diagram ของ Use Case Description ที่ 1A 48
2.13.4 User Interface ของ Use Case Description ที่ 1A 49
2.13.5 Use Case Description ที่ 2A: ตรวจสอบคำสั่งซื้อ 49
2.13.6 Sequence Diagram ของ Use Case Description ที่ 2A 53
2.13.7 Collaboration Diagram ของ Use Case Description ที่ 2A 56
2.13.8 User Interface ของ Use Case Description ที่ 2A 57
2.13.9 Use Case Description ที่ 3A: ตั้งกฎ SKU 57
2.13.10 Sequence Diagram ของ Use Case Description ที่ 3A 61
2.13.11 Collaboration Diagram ของ Use Case Description ที่ 3A 64
2.13.12 User Interface ของ Use Case Description ที่ 3A 65
2.13.13 Use Case Description ที่ 4A: ดึง Order เข้าระบบ 65
2.13.14 Sequence Diagram ของ Use Case Description ที่ 4A 68
2.13.15 Collaboration Diagram ของ Use Case Description ที่ 4A 71
2.13.16 User Interface ของ Use Case Description ที่ 4A 72
2.13.17 Use Case Description ที่ 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock 72
2.13.18 Sequence Diagram ของ Use Case Description ที่ 5A 75

2.13.19 Collaboration Diagram ของ Use Case Description ที่ 5A 77
2.13.20 User Interface ของ Use Case Description ที่ 5A 78
2.13.21 Use Case Description ที่ 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier 78
2.13.22 Sequence Diagram ของ Use Case Description ที่ 6A 83
2.13.23 Collaboration Diagram ของ Use Case Description ที่ 6A 88
2.13.24 User Interface ของ Use Case Description ที่ 6A 89
2.13.25 Use Case Description ที่ 7A: ยกเลิก Order 89
2.13.26 Sequence Diagram ของ Use Case Description ที่ 7A 93
2.13.27 Collaboration Diagram ของ Use Case Description ที่ 7A 95
2.13.28 User Interface ของ Use Case Description ที่ 7A 96
2.13.29 Use Case Description ที่ 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery 96
2.13.30 Sequence Diagram ของ Use Case Description ที่ 8A 100
2.13.31 Collaboration Diagram ของ Use Case Description ที่ 8A 103
2.13.32 User Interface ของ Use Case Description ที่ 8A 104
2.13.33 Use Case Description ที่ 9A: พิมพ์ใบปะสินค้า 104
2.13.34 Sequence Diagram ของ Use Case Description ที่ 9A 107
2.13.35 Collaboration Diagram ของ Use Case Description ที่ 9A 109
2.13.36 User Interface ของ Use Case Description ที่ 9A 110
2.13.37 Use Case Description ที่ 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder) 110
2.13.38 Sequence Diagram ของ Use Case Description ที่ 10A 114
2.13.39 Collaboration Diagram ของ Use Case Description ที่ 10A 116
2.13.40 User Interface ของ Use Case Description ที่ 10A 117
2.13.41 Use Case Description ที่ 1S: จับคู่ Order กับเลข RSL 117
2.13.42 Sequence Diagram ของ Use Case Description ที่ 1S 120
2.13.43 Collaboration Diagram ของ Use Case Description ที่ 1S 123
2.13.44 User Interface ของ Use Case Description ที่ 1S 124
2.13.45 Use Case Description ที่ 2S: จับคู่กฎ SKU 124
2.13.46 Sequence Diagram ของ Use Case Description ที่ 2S 127
2.13.47 Collaboration Diagram ของ Use Case Description ที่ 2S 128
2.13.48 User Interface ของ Use Case Description ที่ 2S 129
2.13.49 Use Case Description ที่ 3S: ตรวจสอบของสินค้าใน stock + RSL 129
2.13.50 Sequence Diagram ของ Use Case Description ที่ 3S 132
2.13.51 Collaboration Diagram ของ Use Case Description ที่ 3S 135
2.13.52 User Interface ของ Use Case Description ที่ 3S 136

2.13.53 Use Case Description ที่ 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง 136
2.13.54 Sequence Diagram ของ Use Case Description ที่ 4S 139
2.13.55 Collaboration Diagram ของ Use Case Description ที่ 4S 142
2.13.56 User Interface ของ Use Case Description ที่ 4S 143
2.13.57 Use Case Description ที่ 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace 143
2.13.58 Sequence Diagram ของ Use Case Description ที่ 5S 146
2.13.59 Collaboration Diagram ของ Use Case Description ที่ 5S 148
2.13.60 User Interface ของ Use Case Description ที่ 5S 149
2.13.61 Use Case Description ที่ 6S: จัดรูปแบบใบปะสินค้า 149
2.13.62 Sequence Diagram ของ Use Case Description ที่ 6S 152
2.13.63 Collaboration Diagram ของ Use Case Description ที่ 6S 155
2.13.64 User Interface ของ Use Case Description ที่ 6S 156
2.13.65 Use Case Description ที่ 7S: ตัดสต๊อกสินค้า 156
2.13.66 Sequence Diagram ของ Use Case Description ที่ 7S 160
2.13.67 Collaboration Diagram ของ Use Case Description ที่ 7S 163
2.13.68 User Interface ของ Use Case Description ที่ 7S 164
2.14 ER Diagram 165
2.15 ER Diagram พร้อม Query 171
2.16 Query Table 175
2.17 Table Structure 187
2.17.1 ตารางผู้ดูแลระบบ (ADMINS) 187
2.17.2 ตารางซัพพลายเออร์ (SUPPLIERS) 188
2.17.3 ตารางสินค้าและกฎ SKU (SKUS) 189
2.17.4 ตารางสต๊อกสินค้า (STOCK) 190
2.17.5 ตารางคำสั่งซื้อ (ORDERS) 191
2.17.6 ตารางรายการสินค้าในคำสั่งซื้อ (ORDER_ITEMS) 195
2.17.7 ตารางต้นทุนสินค้า (COSTS) 196
2.17.8 ตารางคำสั่งซื้อเติมสต๊อก (REORDERS) 198
2.17.9 ตารางใบปะสินค้า (LABELS) 199
2.17.10 ตารางเซสชันเข้าสู่ระบบ (SESSIONS) 201
2.17.11 ตารางบันทึกข้อผิดพลาดของงานอัตโนมัติ (AUTO_ERROR_LOGS) 202
2.17.12 ตารางประวัติการลบข้อมูล (DELETION_HISTORY) 203
2.17.13 ตารางประวัติการแก้ไขต้นทุน (COST_HISTORY) 203
2.17.14 ตารางบันทึกการแจ้งเลขติดตาม (TRACKING_NOTIFICATION_LOGS) 205

2.17.15 ตารางสถานะซิงก์สต๊อกกับ Rakuten (RAKUTEN_STOCK_SYNC_STATES) 206
2.17.16 ตารางประวัติการปรับสต๊อก (STOCK_ADJUSTMENT_HISTORY) 207
2.17.17 ตารางประวัติการพิมพ์ใบปะ (LABEL_PRINT_HISTORY) 208
2.18 Class Diagram 210
2.19 State Diagram 226
2.20 Software Development Platform 234
2.21 Site Map / UI Structure 236
2.21.1 User Interface Design 237
2.22 Output Design (Report) 245
2.22.1 ใบปะสินค้า (Shipping Label) 245
2.22.2 รายงานต้นทุนต่อหน่วย (Unit Cost Report) 247
2.23 SQL Statement ใน Use Case Description 249
2.23.1 Use Case 1A: เข้าสู่ระบบ 249
2.23.2 Use Case 2A: ตรวจสอบคำสั่งซื้อ 250
2.23.3 Use Case 3A: ตั้งกฎ SKU 253
2.23.4 Use Case 4A: ดึง Order เข้าระบบ 255
2.23.5 Use Case 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock 256
2.23.6 Use Case 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier 257
2.23.7 Use Case 7A: ยกเลิก Order 259
2.23.8 Use Case 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery 260
2.23.9 Use Case 9A: พิมพ์ใบปะสินค้า 261
2.23.10 Use Case 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder) 262
2.23.11 Use Case 1S: จับคู่ Order กับเลข RSL 263
2.23.12 Use Case 2S: จับคู่กฎ SKU 264
2.23.13 Use Case 3S: ตรวจสอบของสินค้าใน stock + RSL 265
2.23.14 Use Case 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง 267
2.23.15 Use Case 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace 268
2.23.16 Use Case 6S: จัดรูปแบบใบปะสินค้า 269
2.23.17 Use Case 7S: ตัดสต๊อกสินค้า 270
2.24 การทดสอบโปรแกรมทำงานตาม Use Case Description 273
2.24.1 การทดสอบ Use Case 1A: เข้าสู่ระบบ 273
2.24.2 การทดสอบ Use Case 4A: ดึง Order เข้าระบบ 274
2.24.3 การทดสอบ Use Case 2A: ตรวจสอบคำสั่งซื้อ 275
2.24.4 การทดสอบ Use Case 2S: จับคู่กฎ SKU 276

2.24.5 การทดสอบ Use Case 3S: ตรวจสอบของสินค้าใน stock + RSL....277
2.24.6 การทดสอบ Use Case 1S: จับคู่ Order กับเลข RSL....279
2.24.7 การทดสอบ Use Case 6S: จัดรูปแบบใบปะสินค้า....280
2.24.8 การทดสอบ Use Case 3A: ตั้งกฎ SKU....281
2.24.9 การทดสอบ Use Case 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง....282
2.24.10 การทดสอบ Use Case 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock....283
2.24.11 การทดสอบ Use Case 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier....284
2.24.12 การทดสอบ Use Case 7A: ยกเลิก Order....286
2.24.13 การทดสอบ Use Case 9A: พิมพ์ใบปะสินค้า....287
2.24.14 การทดสอบ Use Case 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery....288
2.24.15 การทดสอบ Use Case 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace....289
2.24.16 การทดสอบ Use Case 7S: ตัดสต๊อกสินค้า....290
2.24.17 การทดสอบ Use Case 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว....291
2.24.18 การทดสอบการออกรายงาน....292
2.25 หน้าที่รับผิดชอบ....294
2.25.1 คะแนนการทำงานของทีม....297

สารบัญรูปภาพ

หน้า

รูปที่ 2-1 Business Process เดิม (As-Is)....26
รูปที่ 2-2 จุดที่เป็นปัญหาใน Business Process เดิม....28
รูปที่ 2-3 Business Process ใหม่ (To-Be)....30
รูปที่ 2-4 Business Process ที่สัมพันธ์กับ Use Case....33
รูปที่ 2-5 Use Case Diagram ก่อนปรับปรุง....35
รูปที่ 2-6 Use Case Diagram ที่ทบทวนปรับปรุงแล้ว....38
รูปที่ 2-7 Sequence Diagram ของ Use Case Description ที่ 1A (1/4)....44
รูปที่ 2-8 Sequence Diagram ของ Use Case Description ที่ 1A (2/4)....45
รูปที่ 2-9 Sequence Diagram ของ Use Case Description ที่ 1A (3/4)....46
รูปที่ 2-10 Sequence Diagram ของ Use Case Description ที่ 1A (4/4)....47
รูปที่ 2-11 Collaboration Diagram ของ Use Case Description ที่ 1A....48
รูปที่ 2-12 User Interface ของ Use Case Description ที่ 1A (LoginScreen)....49
รูปที่ 2-13 Sequence Diagram ของ Use Case Description ที่ 2A (1/3)....53
รูปที่ 2-14 Sequence Diagram ของ Use Case Description ที่ 2A (2/3)....54
รูปที่ 2-15 Sequence Diagram ของ Use Case Description ที่ 2A (3/3)....55
รูปที่ 2-16 Collaboration Diagram ของ Use Case Description ที่ 2A....56
รูปที่ 2-17 User Interface ของ Use Case Description ที่ 2A (OrderVerifyScreen)....57
รูปที่ 2-18 Sequence Diagram ของ Use Case Description ที่ 3A (1/3)....61
รูปที่ 2-19 Sequence Diagram ของ Use Case Description ที่ 3A (2/3)....62
รูปที่ 2-20 Sequence Diagram ของ Use Case Description ที่ 3A (3/3)....63
รูปที่ 2-21 Collaboration Diagram ของ Use Case Description ที่ 3A....64
รูปที่ 2-22 User Interface ของ Use Case Description ที่ 3A (ProductScreen)....65
รูปที่ 2-23 Sequence Diagram ของ Use Case Description ที่ 4A (1/3)....68
รูปที่ 2-24 Sequence Diagram ของ Use Case Description ที่ 4A (2/3)....69
รูปที่ 2-25 Sequence Diagram ของ Use Case Description ที่ 4A (3/3)....70
รูปที่ 2-26 Collaboration Diagram ของ Use Case Description ที่ 4A....71
รูปที่ 2-27 User Interface ของ Use Case Description ที่ 4A (DashboardScreen)....72
รูปที่ 2-28 Sequence Diagram ของ Use Case Description ที่ 5A (1/2)....75
รูปที่ 2-29 Sequence Diagram ของ Use Case Description ที่ 5A (2/2)....76
รูปที่ 2-30 Collaboration Diagram ของ Use Case Description ที่ 5A....77
รูปที่ 2-31 User Interface ของ Use Case Description ที่ 5A (ReorderScreen)....78
รูปที่ 2-32 Sequence Diagram ของ Use Case Description ที่ 6A (1/5)....83

รูปที่ 2-33 Sequence Diagram ของ Use Case Description ที่ 6A (2/5) 84
รูปที่ 2-34 Sequence Diagram ของ Use Case Description ที่ 6A (3/5) 85
รูปที่ 2-35 Sequence Diagram ของ Use Case Description ที่ 6A (4/5) 86
รูปที่ 2-36 Sequence Diagram ของ Use Case Description ที่ 6A (5/5) 87
รูปที่ 2-37 Collaboration Diagram ของ Use Case Description ที่ 6A 88
รูปที่ 2-38 User Interface ของ Use Case Description ที่ 6A (ReorderScreen หลังยืนยันการสั่งซื้อ) 89
รูปที่ 2-39 Sequence Diagram ของ Use Case Description ที่ 7A (1/2) 93
รูปที่ 2-40 Sequence Diagram ของ Use Case Description ที่ 7A (2/2) 94
รูปที่ 2-41 Collaboration Diagram ของ Use Case Description ที่ 7A 95
รูปที่ 2-42 User Interface ของ Use Case Description ที่ 7A (CancelOrderScreen) 96
รูปที่ 2-43 Sequence Diagram ของ Use Case Description ที่ 8A (1/3) 100
รูปที่ 2-44 Sequence Diagram ของ Use Case Description ที่ 8A (2/3) 101
รูปที่ 2-45 Sequence Diagram ของ Use Case Description ที่ 8A (3/3) 102
รูปที่ 2-46 Collaboration Diagram ของ Use Case Description ที่ 8A 103
รูปที่ 2-47 User Interface ของ Use Case Description ที่ 8A (ShippingScreen) 104
รูปที่ 2-48 Sequence Diagram ของ Use Case Description ที่ 9A (1/2) 107
รูปที่ 2-49 Sequence Diagram ของ Use Case Description ที่ 9A (2/2) 108
รูปที่ 2-50 Collaboration Diagram ของ Use Case Description ที่ 9A 109
รูปที่ 2-51 User Interface ของ Use Case Description ที่ 9A (LabelPrintScreen) 110
รูปที่ 2-52 Sequence Diagram ของ Use Case Description ที่ 10A (1/2) 114
รูปที่ 2-53 Sequence Diagram ของ Use Case Description ที่ 10A (2/2) 115
รูปที่ 2-54 Collaboration Diagram ของ Use Case Description ที่ 10A 116
รูปที่ 2-55 User Interface ของ Use Case Description ที่ 10A (CleanupScreen) 117
รูปที่ 2-56 Sequence Diagram ของ Use Case Description ที่ 1S (1/3) 120
รูปที่ 2-57 Sequence Diagram ของ Use Case Description ที่ 1S (2/3) 121
รูปที่ 2-58 Sequence Diagram ของ Use Case Description ที่ 1S (3/3) 122
รูปที่ 2-59 Collaboration Diagram ของ Use Case Description ที่ 1S 123
รูปที่ 2-60 User Interface ของ Use Case Description ที่ 1S (RslMatchScreen) 124
รูปที่ 2-61 Sequence Diagram ของ Use Case Description ที่ 2S 127
รูปที่ 2-62 Collaboration Diagram ของ Use Case Description ที่ 2S 128
รูปที่ 2-63 Sequence Diagram ของ Use Case Description ที่ 3S (1/3) 132
รูปที่ 2-64 Sequence Diagram ของ Use Case Description ที่ 3S (2/3) 133
รูปที่ 2-65 Sequence Diagram ของ Use Case Description ที่ 3S (3/3) 134
รูปที่ 2-66 Collaboration Diagram ของ Use Case Description ที่ 3S 135
รูปที่ 2-67 User Interface ของ Use Case Description ที่ 3S (StockCheckScreen) 136

รูปที่ 2-68 Sequence Diagram ของ Use Case Description ที่ 4S (1/3).....139
รูปที่ 2-69 Sequence Diagram ของ Use Case Description ที่ 4S (2/3).....140
รูปที่ 2-70 Sequence Diagram ของ Use Case Description ที่ 4S (3/3).....141
รูปที่ 2-71 Collaboration Diagram ของ Use Case Description ที่ 4S.....142
รูปที่ 2-72 User Interface ของ Use Case Description ที่ 4S (CostScreen).....143
รูปที่ 2-73 Sequence Diagram ของ Use Case Description ที่ 5S (1/2).....146
รูปที่ 2-74 Sequence Diagram ของ Use Case Description ที่ 5S (2/2).....147
รูปที่ 2-75 Collaboration Diagram ของ Use Case Description ที่ 5S.....148
รูปที่ 2-76 User Interface ของ Use Case Description ที่ 5S (ShippingScreen หลังแจ้งเลขติดตาม).149
รูปที่ 2-77 Sequence Diagram ของ Use Case Description ที่ 6S (1/3).....152
รูปที่ 2-78 Sequence Diagram ของ Use Case Description ที่ 6S (2/3).....153
รูปที่ 2-79 Sequence Diagram ของ Use Case Description ที่ 6S (3/3).....154
รูปที่ 2-80 Collaboration Diagram ของ Use Case Description ที่ 6S.....155
รูปที่ 2-81 Sequence Diagram ของ Use Case Description ที่ 7S (1/3).....160
รูปที่ 2-82 Sequence Diagram ของ Use Case Description ที่ 7S (2/3).....161
รูปที่ 2-83 Sequence Diagram ของ Use Case Description ที่ 7S (3/3).....162
รูปที่ 2-84 Collaboration Diagram ของ Use Case Description ที่ 7S.....163
รูปที่ 2-85 ER Diagram ภาพรวมของระบบ (17 ตาราง).....166
รูปที่ 2-86 ER Diagram ส่วนคำสั่งซื้อ (ORDERS) (1/4).....167
รูปที่ 2-87 ER Diagram ส่วนรายการสินค้า ใบปะ และการแจ้งเลขติดตาม (2/4).....168
รูปที่ 2-88 ER Diagram ส่วนสินค้า สต๊อก ต้นทุน และการสั่งซื้อเติม (3/4).....169
รูปที่ 2-89 ER Diagram ส่วนผู้ดูแลระบบและประวัติการทำงาน (4/4).....170
รูปที่ 2-90 ER Diagram พร้อม Query ส่วนคำสั่งซื้อ (ORDERS) (1/4).....171
รูปที่ 2-91 ER Diagram พร้อม Query ส่วนรายการสินค้า ใบปะ และการแจ้งเลขติดตาม (2/4).....172
รูปที่ 2-92 ER Diagram พร้อม Query ส่วนสินค้า สต๊อก ต้นทุน และการสั่งซื้อเติม (3/4).....173
รูปที่ 2-93 ER Diagram พร้อม Query ส่วนผู้ดูแลระบบและประวัติการทำงาน (4/4).....174
รูปที่ 2-94 Class Diagram กลุ่มเข้าสู่ระบบ (1/15).....211
รูปที่ 2-95 Class Diagram กลุ่มรับคำสั่งซื้อและจับคู่ SKU (2/15).....212
รูปที่ 2-96 Class Diagram กลุ่มตรวจสอบและยกเลิกคำสั่งซื้อ (3/15).....213
รูปที่ 2-97 Class Diagram กลุ่มจัดการสินค้า (4/15).....214
รูปที่ 2-98 Class Diagram กลุ่มตรวจและตัดสต๊อก (5/15).....215
รูปที่ 2-99 Class Diagram กลุ่มคำนวณต้นทุน (6/15).....216
รูปที่ 2-100 Class Diagram กลุ่มตัดสินใจสั่งซื้อเติม stock (7/15).....217
รูปที่ 2-101 Class Diagram กลุ่มสั่งซื้อเติม stock จาก supplier (8/15).....218
รูปที่ 2-102 Class Diagram กลุ่มส่ง Order เข้า RSL (9/15).....219

รูปที่ 2-103 Class Diagram กลุ่มจัดการใบปะ (10/15) 220
รูปที่ 2-104 Class Diagram กลุ่มจัดส่งและแจ้งเลขติดตาม (11/15) 221
รูปที่ 2-105 Class Diagram กลุ่มลบข้อมูลเก่า (12/15) 222
รูปที่ 2-106 Class Diagram กลุ่มคลาสข้อมูลคำสั่งซื้อและการจัดส่ง (Entity) (13/15) 223
รูปที่ 2-107 Class Diagram กลุ่มคลาสข้อมูลผู้ดูแลระบบและประวัติ (Entity) (14/15) 224
รูปที่ 2-108 Class Diagram กลุ่มคลาสข้อมูลสินค้าและสต๊อก (Entity) (15/15) 225
รูปที่ 2-109 State Diagram ของ Order (order_status) ก่อนจัดส่ง (1/2) 227
รูปที่ 2-110 State Diagram ของ Order (order_status) ช่วงจัดส่ง (2/2) 228
รูปที่ 2-111 State Diagram ของ Reorder (reorder_status) 229
รูปที่ 2-112 State Diagram ของ Label (label_status) 230
รูปที่ 2-113 State Diagram ของ SKU (sku_status) 231
รูปที่ 2-114 State Diagram ของ Order (delivery_status) 232
รูปที่ 2-115 State Diagram ของ RakutenStockSyncState (sync_status) 233
รูปที่ 2-116 เครื่องมือที่ใช้พัฒนาระบบ (Stack of Tools) 234
รูปที่ 2-117 โครงสร้างหน้าจอของระบบ (Site Map) 236
รูปที่ 2-118 LoginScreen (1A เข้าสู่ระบบ) 238
รูปที่ 2-119 DashboardScreen (4A ดึง Order เข้าระบบ (ปุ่มนำเข้า Order)) 238
รูปที่ 2-120 OrderVerifyScreen (2A ตรวจสอบคำสั่งซื้อ) 239
รูปที่ 2-121 RslMatchScreen (1S จับคู่ Order กับเลข RSL) 239
รูปที่ 2-122 ProductScreen (3A ตั้งกฎ SKU) 240
รูปที่ 2-123 StockCheckScreen (3S ตรวจสอบของสินค้าใน stock + RSL) 240
รูปที่ 2-124 CostScreen (4S คำนวณ Cost ในการสั่งสินค้าเติม stock เอง) 241
รูปที่ 2-125 ReorderScreen (5A ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock, 6A สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier) 241
รูปที่ 2-126 ReorderScreen (รับสินค้า) (6A สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier — บันทึกการรับสินค้าจาก Supplier) 242
รูปที่ 2-127 CancelOrderScreen (7A ยกเลิก Order) 242
รูปที่ 2-128 ShippingScreen (8A จัดส่งสินค้าให้ลูกค้าผ่าน Delivery, 5S ส่งเลขติดตาม) 243
รูปที่ 2-129 LabelPrintScreen (9A พิมพ์ใบปะสินค้า) 243
รูปที่ 2-130 CleanupScreen (10A ลบข้อมูลเก่าที่ไม่ใช้แล้ว) 244
รูปที่ 2-131 ใบปะสินค้าของ ORD-24178 (RSL จัดส่ง, RSL-STD-A6) 245
รูปที่ 2-132 ใบปะสินค้าของ ORD-24177 (บริษัทจัดส่งเอง, INHOUSE-YAMATO-A6) 246
รูปที่ 2-133 รายงานต้นทุนต่อหน่วยของ FSH-MLR-240 247
รูปที่ 2-134 ผลการทดสอบ Use Case 1A กรณีปกติ 273
รูปที่ 2-135 ผลการทดสอบ Use Case 1A กรณีผิดพลาด 274

รูปที่ 2-136 ผลการทดสอบ Use Case 4A กรณีปกติ 274
รูปที่ 2-137 ผลการทดสอบ Use Case 4A กรณีผิดพลาด 275
รูปที่ 2-138 ผลการทดสอบ Use Case 2A กรณีปกติ 275
รูปที่ 2-139 ผลการทดสอบ Use Case 2A กรณีผิดพลาด 276
รูปที่ 2-140 ผลการทดสอบ Use Case 2S กรณีปกติ....276
รูปที่ 2-141 ผลการทดสอบ Use Case 2S กรณีผิดพลาด....277
รูปที่ 2-142 ผลการทดสอบ Use Case 3S กรณีปกติ....278
รูปที่ 2-143 ผลการทดสอบ Use Case 3S กรณีบริษัทจัดส่งเอง (in-house)....278
รูปที่ 2-144 ผลการทดสอบ Use Case 3S กรณีผิดพลาด....279
รูปที่ 2-145 ผลการทดสอบ Use Case 1S กรณีปกติ....279
รูปที่ 2-146 ผลการทดสอบ Use Case 1S กรณีผิดพลาด....280
รูปที่ 2-147 ผลการทดสอบ Use Case 6S กรณีปกติ....280
รูปที่ 2-148 ผลการทดสอบ Use Case 6S กรณีผิดพลาด....281
รูปที่ 2-149 ผลการทดสอบ Use Case 3A กรณีปกติ 281
รูปที่ 2-150 ผลการทดสอบ Use Case 3A กรณีผิดพลาด 282
รูปที่ 2-151 ผลการทดสอบ Use Case 4S กรณีปกติ....282
รูปที่ 2-152 ผลการทดสอบ Use Case 4S กรณีผิดพลาด....283
รูปที่ 2-153 ผลการทดสอบ Use Case 5A กรณีปกติ 283
รูปที่ 2-154 ผลการทดสอบ Use Case 5A กรณีผิดพลาด 284
รูปที่ 2-155 ผลการทดสอบ Use Case 6A กรณีปกติ 285
รูปที่ 2-156 ผลการทดสอบ Use Case 6A กรณีผิดพลาด 285
รูปที่ 2-157 ผลการทดสอบ Use Case 6A กรณีปกติ 286
รูปที่ 2-158 ผลการทดสอบ Use Case 7A กรณีปกติ 286
รูปที่ 2-159 ผลการทดสอบ Use Case 7A กรณีผิดพลาด 287
รูปที่ 2-160 ผลการทดสอบ Use Case 9A กรณีปกติ 287
รูปที่ 2-161 ผลการทดสอบ Use Case 9A กรณีผิดพลาด 288
รูปที่ 2-162 ผลการทดสอบ Use Case 8A กรณีปกติ 288
รูปที่ 2-163 ผลการทดสอบ Use Case 8A กรณีผิดพลาด 289
รูปที่ 2-164 ผลการทดสอบ Use Case 5S กรณีปกติ....289
รูปที่ 2-165 ผลการทดสอบ Use Case 5S กรณีผิดพลาด....290
รูปที่ 2-166 ผลการทดสอบ Use Case 7S กรณีปกติ....290
รูปที่ 2-167 ผลการทดสอบ Use Case 7S กรณีผิดพลาด....291
รูปที่ 2-168 ผลการทดสอบ Use Case 10A กรณีปกติ....291
รูปที่ 2-169 ผลการทดสอบ Use Case 10A กรณีผิดพลาด 292
รูปที่ 2-170 ผลการออกรายงานใบปะสินค้า 292

รูปที่ 2-171 ผลการออกรายงานต้นทุนต่อหน่วย 293

สารบัญตาราง

หน้า

ตารางที่ 1-1 ค่าประมาณเวลาที่ใช้กับงานหลังบ้านต่อวัน....21
ตารางที่ 2-1 ขั้นตอนของ Business Process ใหม่ ผู้ทำ และปัญหาที่แก้....31
ตารางที่ 2-2 ตารางจับคู่ระหว่าง Business Process และ Use Case Diagram....34
ตารางที่ 2-3 CRUD Table ก่อนปรับปรุง....36
ตารางที่ 2-4 CRUD Table ที่ทบทวนแล้ว (สร้างจาก SQL ใน Use Case Description)....40
ตารางที่ 2-5 Use Case Description ที่ 1A: เข้าสู่ระบบ....41
ตารางที่ 2-6 Use Case Description ที่ 2A: ตรวจสอบคำสั่งซื้อ....49
ตารางที่ 2-7 Use Case Description ที่ 3A: ตั้งกฎ SKU....57
ตารางที่ 2-8 Use Case Description ที่ 4A: ดึง Order เข้าระบบ....65
ตารางที่ 2-9 Use Case Description ที่ 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock....73
ตารางที่ 2-10 Use Case Description ที่ 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier....79
ตารางที่ 2-11 Use Case Description ที่ 7A: ยกเลิก Order....90
ตารางที่ 2-12 Use Case Description ที่ 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery....97
ตารางที่ 2-13 Use Case Description ที่ 9A: พิมพ์ใบปะสินค้า....105
ตารางที่ 2-14 Use Case Description ที่ 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)....111
ตารางที่ 2-15 Use Case Description ที่ 1S: จับคู่ Order กับเลข RSL....118
ตารางที่ 2-16 Use Case Description ที่ 2S: จับคู่กฎ SKU....125
ตารางที่ 2-17 Use Case Description ที่ 3S: ตรวจสอบของสินค้าใน stock + RSL....129
ตารางที่ 2-18 Use Case Description ที่ 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง....136
ตารางที่ 2-19 Use Case Description ที่ 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace....144
ตารางที่ 2-20 Use Case Description ที่ 6S: จัดรูปแบบใบปะสินค้า....150
ตารางที่ 2-21 Use Case Description ที่ 7S: ตัดสต๊อกสินค้า....156
ตารางที่ 2-22 Query Table....175
ตารางที่ 2-23 Data Dictionary ของตารางผู้ดูแลระบบ (ADMINS)....187
ตารางที่ 2-24 ตัวอย่างข้อมูลของตารางผู้ดูแลระบบ (ADMINS) (คอลัมน์ 1-6)....187
ตารางที่ 2-25 ตัวอย่างข้อมูลของตารางผู้ดูแลระบบ (ADMINS) (คอลัมน์ 7-7)....188
ตารางที่ 2-26 Data Dictionary ของตารางซัพพลายเออร์ (SUPPLIERS)....188
ตารางที่ 2-27 ตัวอย่างข้อมูลของตารางซัพพลายเออร์ (SUPPLIERS)....189
ตารางที่ 2-28 Data Dictionary ของตารางสินค้าและกฎ SKU (SKUS)....189
ตารางที่ 2-29 ตัวอย่างข้อมูลของตารางสินค้าและกฎ SKU (SKUS) (คอลัมน์ 1-6)....190

ตารางที่ 2-30 ตัวอย่างข้อมูลของตารางสินค้าและกฎ SKU (SKUS) (คอลัมน์ 7-12).....190
ตารางที่ 2-31 Data Dictionary ของตารางสต๊อกสินค้า (STOCK) 190
ตารางที่ 2-32 ตัวอย่างข้อมูลของตารางสต๊อกสินค้า (STOCK) 191
ตารางที่ 2-33 Data Dictionary ของตารางคำสั่งซื้อ (ORDERS).....191
ตารางที่ 2-34 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 1-6) 193
ตารางที่ 2-35 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 7-12) 194
ตารางที่ 2-36 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 13-18).....194
ตารางที่ 2-37 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 19-24).....194
ตารางที่ 2-38 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 25-26).....194
ตารางที่ 2-39 Data Dictionary ของตารางรายการสินค้าในคำสั่งซื้อ (ORDER_ITEMS).....195
ตารางที่ 2-40 ตัวอย่างข้อมูลของตารางรายการสินค้าในคำสั่งซื้อ (ORDER_ITEMS) (คอลัมน์ 1-6) 195
ตารางที่ 2-41 ตัวอย่างข้อมูลของตารางรายการสินค้าในคำสั่งซื้อ (ORDER_ITEMS) (คอลัมน์ 7-9) 196
ตารางที่ 2-42 Data Dictionary ของตารางต้นทุนสินค้า (COSTS).....196
ตารางที่ 2-43 ตัวอย่างข้อมูลของตารางต้นทุนสินค้า (COSTS) (คอลัมน์ 1-6).....197
ตารางที่ 2-44 ตัวอย่างข้อมูลของตารางต้นทุนสินค้า (COSTS) (คอลัมน์ 7-12) 197
ตารางที่ 2-45 ตัวอย่างข้อมูลของตารางต้นทุนสินค้า (COSTS) (คอลัมน์ 13-15).....197
ตารางที่ 2-46 Data Dictionary ของตารางคำสั่งซื้อเติมสต๊อก (REORDERS) 198
ตารางที่ 2-47 ตัวอย่างข้อมูลของตารางคำสั่งซื้อเติมสต๊อก (REORDERS) (คอลัมน์ 1-6) 199
ตารางที่ 2-48 ตัวอย่างข้อมูลของตารางคำสั่งซื้อเติมสต๊อก (REORDERS) (คอลัมน์ 7-12).....199
ตารางที่ 2-49 ตัวอย่างข้อมูลของตารางคำสั่งซื้อเติมสต๊อก (REORDERS) (คอลัมน์ 13-16).....199
ตารางที่ 2-50 Data Dictionary ของตารางใบปะสินค้า (LABELS).....199
ตารางที่ 2-51 ตัวอย่างข้อมูลของตารางใบปะสินค้า (LABELS) (คอลัมน์ 1-6).....200
ตารางที่ 2-52 ตัวอย่างข้อมูลของตารางใบปะสินค้า (LABELS) (คอลัมน์ 7-9).....200
ตารางที่ 2-53 Data Dictionary ของตารางเซสชันเข้าสู่ระบบ (SESSIONS).....201
ตารางที่ 2-54 ตัวอย่างข้อมูลของตารางเซสชันเข้าสู่ระบบ (SESSIONS).....201
ตารางที่ 2-55 Data Dictionary ของตารางบันทึกข้อผิดพลาดของงานอัตโนมัติ (AUTO_ERROR_LOGS) 202
ตารางที่ 2-56 ตัวอย่างข้อมูลของตารางบันทึกข้อผิดพลาดของงานอัตโนมัติ (AUTO_ERROR_LOGS) 202
ตารางที่ 2-57 Data Dictionary ของตารางประวัติการลบข้อมูล (DELETION_HISTORY) 203
ตารางที่ 2-58 ตัวอย่างข้อมูลของตารางประวัติการลบข้อมูล (DELETION_HISTORY) 203
ตารางที่ 2-59 Data Dictionary ของตารางประวัติการแก้ไขต้นทุน (COST_HISTORY) 203
ตารางที่ 2-60 ตัวอย่างข้อมูลของตารางประวัติการแก้ไขต้นทุน (COST_HISTORY) (คอลัมน์ 1-6).....204
ตารางที่ 2-61 ตัวอย่างข้อมูลของตารางประวัติการแก้ไขต้นทุน (COST_HISTORY) (คอลัมน์ 7-7).....204
ตารางที่ 2-62 Data Dictionary ของตารางบันทึกการแจ้งเลขติดตาม (TRACKING_NOTIFICATION_LOGS)
.....205

ตารางที่ 2-63 ตัวอย่างข้อมูลของตารางบันทึกการแจ้งเลขติดตาม (TRACKING_NOTIFICATION_LOGS) (คอลัมน์ 1-6) 205
ตารางที่ 2-64 ตัวอย่างข้อมูลของตารางบันทึกการแจ้งเลขติดตาม (TRACKING_NOTIFICATION_LOGS) (คอลัมน์ 7-8) 206
ตารางที่ 2-65 Data Dictionary ของตารางสถานะซิงก์สต๊อกกับ Rakuten (RAKUTEN_STOCK_SYNC_STATES) 206
ตารางที่ 2-66 ตัวอย่างข้อมูลของตารางสถานะซิงก์สต๊อกกับ Rakuten (RAKUTEN_STOCK_SYNC_STATES) 206
ตารางที่ 2-67 Data Dictionary ของตารางประวัติการปรับสต๊อก (STOCK_ADJUSTMENT_HISTORY) ..207
ตารางที่ 2-68 ตัวอย่างข้อมูลของตารางประวัติการปรับสต๊อก (STOCK_ADJUSTMENT_HISTORY) (คอลัมน์ 1-6) 207
ตารางที่ 2-69 ตัวอย่างข้อมูลของตารางประวัติการปรับสต๊อก (STOCK_ADJUSTMENT_HISTORY) (คอลัมน์ 7-9) 208
ตารางที่ 2-70 Data Dictionary ของตารางประวัติการพิมพ์ใบปะ (LABEL_PRINT_HISTORY) 208
ตารางที่ 2-71 ตัวอย่างข้อมูลของตารางประวัติการพิมพ์ใบปะ (LABEL_PRINT_HISTORY) (คอลัมน์ 1-6) .209
ตารางที่ 2-72 ตัวอย่างข้อมูลของตารางประวัติการพิมพ์ใบปะ (LABEL_PRINT_HISTORY) (คอลัมน์ 7-8) .209
ตารางที่ 2-73 เครื่องมือที่ใช้พัฒนาระบบ 234
ตารางที่ 2-74 หน้าจอของระบบและ Use Case ที่เกี่ยวข้อง 237
ตารางที่ 2-75 โครงสร้างของใบปะสินค้า 246
ตารางที่ 2-76 โครงสร้างของรายงานต้นทุนต่อหน่วย 248
ตารางที่ 2-77 SQL Statement และผลการทำงานของ Use Case 1A 249
ตารางที่ 2-78 SQL Statement และผลการทำงานของ Use Case 2A 250
ตารางที่ 2-79 SQL Statement และผลการทำงานของ Use Case 3A 253
ตารางที่ 2-80 SQL Statement และผลการทำงานของ Use Case 4A 255
ตารางที่ 2-81 SQL Statement และผลการทำงานของ Use Case 5A 256
ตารางที่ 2-82 SQL Statement และผลการทำงานของ Use Case 6A 257
ตารางที่ 2-83 SQL Statement และผลการทำงานของ Use Case 7A 259
ตารางที่ 2-84 SQL Statement และผลการทำงานของ Use Case 8A 260
ตารางที่ 2-85 SQL Statement และผลการทำงานของ Use Case 9A 261
ตารางที่ 2-86 SQL Statement และผลการทำงานของ Use Case 10A 262
ตารางที่ 2-87 SQL Statement และผลการทำงานของ Use Case 1S 263
ตารางที่ 2-88 SQL Statement และผลการทำงานของ Use Case 2S 264
ตารางที่ 2-89 SQL Statement และผลการทำงานของ Use Case 3S 265
ตารางที่ 2-90 SQL Statement และผลการทำงานของ Use Case 4S 267
ตารางที่ 2-91 SQL Statement และผลการทำงานของ Use Case 5S 268

ตารางที่ 2-92 SQL Statement และผลการทำงานของ Use Case 6S 269
ตารางที่ 2-93 SQL Statement และผลการทำงานของ Use Case 7S 270
ตารางที่ 2-94 การทดสอบ Use Case 1A: เข้าสู่ระบบ 273
ตารางที่ 2-95 การทดสอบ Use Case 4A: ดึง Order เข้าระบบ 274
ตารางที่ 2-96 การทดสอบ Use Case 2A: ตรวจสอบคำสั่งซื้อ 275
ตารางที่ 2-97 การทดสอบ Use Case 2S: จับคู่กฎ SKU 276
ตารางที่ 2-98 การทดสอบ Use Case 3S: ตรวจสอบของสินค้าใน stock + RSL 277
ตารางที่ 2-99 การทดสอบ Use Case 1S: จับคู่ Order กับเลข RSL 279
ตารางที่ 2-100 การทดสอบ Use Case 6S: จัดรูปแบบใบปะสินค้า 280
ตารางที่ 2-101 การทดสอบ Use Case 3A: ตั้งกฎ SKU 281
ตารางที่ 2-102 การทดสอบ Use Case 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง 282
ตารางที่ 2-103 การทดสอบ Use Case 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock 283
ตารางที่ 2-104 การทดสอบ Use Case 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier 284
ตารางที่ 2-105 การทดสอบ Use Case 7A: ยกเลิก Order 286
ตารางที่ 2-106 การทดสอบ Use Case 9A: พิมพ์ใบปะสินค้า 287
ตารางที่ 2-107 การทดสอบ Use Case 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery 288
ตารางที่ 2-108 การทดสอบ Use Case 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace 289
ตารางที่ 2-109 การทดสอบ Use Case 7S: ตัดสต๊อกสินค้า 290
ตารางที่ 2-110 การทดสอบ Use Case 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว 291
ตารางที่ 2-111 การทดสอบการออกรายงาน 292
ตารางที่ 2-112 จำนวนงานของสมาชิกแต่ละคน 294
ตารางที่ 2-113 หน้าที่รับผิดชอบ 294
ตารางที่ 2-114 คะแนนการทำงานของทีม 297

บทที่ 1 ที่มาและความสำคัญ

1.1 ที่มาและความสำคัญของปัญหา

Colorado Co., Ltd. เป็นธุรกิจจำหน่ายสินค้านำเข้าและสินค้า OEM ผ่านมาร์เก็ตเพลสอีคอมเมิร์ซในประเทศญี่ปุ่น กลุ่มสินค้าหลักได้แก่ สินค้าอุปโภคทั่วไปนำเข้า อุปกรณ์ตกปลาและเหยื่อล่อ สินค้ากลางแจ้งและยาดมสมุนไพร นอกจากนี้ยังอยู่ระหว่างพัฒนาสินค้าของตนเอง ได้แก่ อุปกรณ์ตกปลา สินค้าฟิตเนส กรอบรูปไม้ และสินค้าสำหรับงานฉลองครบ 100 วันของทารก

ธุรกิจเริ่มเป็นธุรกิจเสริมตั้งแต่ปี 2009 และแยกเป็นธุรกิจอิสระในภายหลัง เจ้าของดูแลเองทั้งหมด ตั้งแต่วางแผนสินค้า จัดหา เจรจากับโรงงานต่างประเทศ ขาย จัดส่ง ไปจนถึงบริการลูกค้า ปัจจุบันใช้ RSL (Rakuten Super Logistics) เป็นผู้ให้บริการคลังสินค้าและจัดส่งหลัก ลูกค้าเป็นผู้บริโภครายบุคคลที่ซื้อผ่าน Rakuten Ichiba เป็นหลัก และจะขยายไป Yahoo! Auctions กับ Amazon ในเฟสถัดไป

งานหลังบ้านทุกขั้นตอน ได้แก่ พิมพ์ใบปะสินค้า เช็กสต๊อก สั่งซื้อเพิ่ม และคำนวณต้นทุน ยังทำด้วยมือทั้งหมด และเจ้าของเป็นผู้ดูแลระบบเพียงคนเดียว โครงงานนี้จึงพัฒนาระบบจัดการหลังบ้านเพื่อทำงานที่ซ้ำและเสียเวลาให้เป็นอัตโนมัติ โดยเลือก **Rakuten เป็นแพลตฟอร์มนำร่อง (เฟส 1)** ด้วยเหตุผลดังนี้

- เป็นช่องทางขายหลักที่มีข้อมูลคำสั่งซื้อจริงให้ทดสอบระบบได้ทันที
- ใช้ RSL เป็นคลังสินค้าหลักอยู่แล้ว ซึ่งอยู่ในระบบเดียวกับ Rakuten ทำให้เชื่อมข้อมูลคำสั่งซื้อ สต๊อก และใบปะสินค้าได้ต่อเนื่อง
- Rakuten RMS มี API สำหรับดึงคำสั่งซื้อ จัดการรูปแบบใบปะสินค้า และเชื่อมสต๊อก ครอบคลุม 3 งานหลักที่ต้องการ
- ลดความเสี่ยงจากการเชื่อมหลายระบบพร้อมกันในรอบแรก

1.2 ปัญหาที่พบ

จากการสอบถามเจ้าของธุรกิจ พบปัญหาหลักในการทำงานประจำวัน 4 ข้อ ดังนี้

1. **ต้องพิมพ์ใบปะสินค้า (shipping label) ด้วยมือ** หลังได้รับคำสั่งซื้อทุกครั้ง
2. **ต้องเช็กสต๊อกข้ามหลายคลังด้วยตนเอง** ทั้งคลังบริษัทและคลัง RSL

3. **ตัดสินใจสั่งซื้อเพิ่ม (reorder) ด้วยความรู้สึก** ไม่มีจุดสั่งซื้อที่ชัดเจน
4. **คำนวณต้นทุนที่แท้จริงต่อชิ้นได้ยาก** เพราะต้องรวมราคาซื้อ อัตราแลกเปลี่ยน ค่าขนส่งระหว่างประเทศ ภาษีนำเข้า ค่าธรรมเนียมมาร์เก็ตเพลส ค่าส่งในประเทศ และค่าธรรมเนียม RSL

ร้านขายสินค้ากลุ่มหลักประมาณ 5–10 ชิ้นต่อวัน ทุกคำสั่งซื้อต้องผ่านงานมือชุดเดียวกัน คือ ตรวจคำสั่งซื้อใหม่ เช็กสต๊อก จับคู่กับ RSL พิมพ์ใบปะสินค้า และแจ้งเลขติดตาม ผู้จัดทำประมาณการเวลาที่ใช้ไว้ในตารางที่ 1-1 (เป็น**ค่าประมาณ** จากขั้นตอนที่เจ้าของเล่า ยังไม่ได้จับเวลาจริง)

ตารางที่ 1-1 ค่าประมาณเวลาที่ใช้กับงานหลังบ้านต่อวัน

งาน	เวลาต่อOrder (นาที)	ต่อวัน ที่ 5 Order (นาที)	ต่อวัน ที่ 10 Order (นาที)
ตรวจคำสั่งซื้อใหม่ใน Rakuten RMS	2	10	20
เช็กสต๊อกในคลังบริษัทและ RSL	3	15	30
จับคู่คำสั่งซื้อกับเลข RSL	2	10	20
จัดรูปแบบและพิมพ์ใบปะสินค้า	3	15	30
ติดตามสถานะพัสดุและแจ้งเลขติดตามให้ลูกค้า	2	10	20
รวม	12	60	120

งานเหล่านี้ใช้เวลาประมาณ 1–2 ชั่วโมงต่อวัน หรือ 30–60 ชั่วโมงต่อเดือน ยังไม่รวมเวลาคำนวณต้นทุนเพื่อตัดสินใจสั่งซื้อ ซึ่งต้องรวมค่าใช้จ่าย 8 รายการต่อสินค้าหนึ่งรายการ (ประมาณ 20–30 นาทีต่อครั้งเป็นค่าประมาณ)

ปัญหาเหล่านี้เคยส่งผลกระทบจริงต่อธุรกิจ เช่น

- Binder เก็บสติกเกอร์ล็อตแรก 500 ชิ้น มีของเสียประมาณ 20 ชิ้น (4%) ต้องแก้สเปกความหนาปก ระบบล็อก มาตรฐานรู และสี
- ค่าส่งเมื่อจัดส่งจากบริษัทเองประมาณ 600 เยนต่อชิ้น เทียบกับคู่แข่งประมาณ 200 เยน ต่างกัน 400 เยนต่อชิ้น หรือประมาณ 2,000–4,000 เยนต่อวันที่ 5–10 ชิ้น ขณะที่ RSL คิดค่าหยิบสินค้าประมาณ 60 เยนต่อชิ้น จึงต้องคำนวณต้นทุนเทียบกันก่อนเลือกวิธีจัดส่งและก่อนสั่งซื้อ

1.3 วัตถุประสงค์

โครงงานนี้มีวัตถุประสงค์เพื่อสร้างระบบอัตโนมัติ 3 เรื่องสำหรับคำสั่งซื้อจาก Rakuten ดังนี้

1. **จัดรูปแบบใบปะสินค้าอัตโนมัติ** ทันทีที่คำสั่งซื้อจับคู่กับ RSL แล้ว เหลือเพียงกดพิมพ์
2. **แจ้งเตือนและสั่งซื้อเติมสต๊อก** เมื่อสต๊อกรวมต่ำกว่าจุดสั่งซื้อ โดยให้เจ้าของเป็นผู้อนุมัติก่อนส่งคำสั่งซื้อ
3. **คำนวณต้นทุนที่แท้จริงต่อหน่วยของแต่ละ SKU** ด้วยสูตรเดียวกันทุกครั้ง

ตัวชี้วัดความสำเร็จภายใน 3 เดือนหลังเปิดใช้งาน ได้แก่

- คำสั่งซื้อ Rakuten ใหม่ทุกรายการได้ใบปะสินค้าที่ถูกต้องโดยไม่ต้องจัดรูปแบบเอง
- สต๊อกที่ต่ำกว่าเกณฑ์ถูกแจ้งเตือนครบทุก SKU
- คำนวณต้นทุนต่อหน่วยได้ภายใน 1 นาที แทน 20–30 นาที
- ลดเวลางานหลังบ้านต่อวันลงอย่างน้อยครึ่งหนึ่งจากค่าประมาณในตารางที่ 1-1

1.4 อุปกรณ์ที่ใช้

1.4.1 Next.js 16 และ React 19 ใช้สำหรับพัฒนาหน้าจอระบบ

1.4.2 ภาษา TypeScript ใช้สำหรับพัฒนาระบบ

1.4.3 Tailwind CSS และ shadcn/ui ใช้สำหรับออกแบบหน้าจอ

1.4.4 ฐานข้อมูล PostgreSQL ใช้สำหรับเก็บข้อมูลของระบบ

1.4.5 Rakuten RMS API และ RSL ใช้สำหรับเชื่อมคำสั่งซื้อ สต๊อก และการจัดส่ง (เว็บต้นแบบเชื่อมผ่าน adapter ที่จำลองระบบภายนอก)

1.4.6 โปรแกรม Visual Studio Code ใช้สำหรับเขียนโปรแกรม

1.4.7 Git และ GitHub ใช้สำหรับจัดเก็บโค้ดและทำงานร่วมกัน

1.4.8 draw.io และ Mermaid ใช้สำหรับวาด diagram

1.4.9 Google Chrome ใช้สำหรับทดสอบระบบ

1.5 ขอบเขตการทำงาน

ขอบเขตของเฟส 1: ครอบคลุมเฉพาะ Rakuten Ichiba, Rakuten RMS และ RSL เท่านั้น แพลตฟอร์มอื่น (Yahoo! Auctions, Amazon) จะพัฒนาในเฟสถัดไป

สิ่งที่อยู่นอกขอบเขตของเฟส 1

- การเชื่อมต่อ Yahoo! Auctions และ Amazon (SP-API)
- ระบบสื่อสารกับลูกค้าโดยตรง ยังใช้ช่องทางแจ้งเตือนของ Rakuten ตามเดิม
- การรับชำระเงินในระบบ ลูกค้าชำระผ่าน Rakuten ทั้งหมด
- การยืนยันตัวตนสองชั้น (2FA) ของบัญชี Admin เฟส 1 ใช้อีเมลและรหัสผ่านตาม Use Case 1A
- Mercari Shops ซึ่งเลิกใช้งานแล้ว

สิ่งที่ยังทำด้วยมือ: วางแผนสินค้า จัดหา เจรจาโรงงาน ตัดสินใจด้านคุณภาพ อนุมัติขั้นสุดท้าย และกรณีลูกค้าพิเศษ เจ้าของเป็นผู้กำหนดกฎทั้งหมด (กฎ SKU, จุดสั่งซื้อ, ซัพพลายเออร์, จำนวนสั่งซื้อ, องค์ประกอบต้นทุน) และหยุดหรือแก้ไขงานอัตโนมัติได้ทุกเมื่อ

ผู้ใช้งาน: ผู้ใช้หลักคือเจ้าของในฐานะ Admin ระบบมีบัญชี Super Admin สำหรับเจ้าของเพียงบัญชีเดียว ไม่มีบัญชีพนักงาน RSL และซัพพลายเออร์ยังติดต่อผ่านช่องทางเดิม ลูกค้าสั่งซื้อผ่าน Rakuten Ichiba ตามเดิม ระบบนี้ไม่มีหน้าร้านสำหรับลูกค้า

1.6 ขั้นตอนการทำงาน

1.6.1 ประชุมภายในกลุ่ม เพื่อเลือกหน่วยงานและกำหนดปัญหาที่ต้องการแก้

1.6.2 สัมภาษณ์เจ้าของบริษัท เพื่อเก็บความต้องการ ขั้นตอนการทำงานเดิม และปัญหาที่พบ

1.6.3 วิเคราะห์ Business Process เดิม ระบุจุดที่เป็นปัญหา และออกแบบ Business Process ใหม่

1.6.4 แปลง Business Process เป็น Use Case Diagram และ CRUD Table เพื่อหา Use Case ที่ขาด

1.6.5 นำเสนอแบบต่ออาจารย์ และปรับแบบตามคำแนะนำ เช่น ให้เจ้าของอนุมัติการสั่งซื้อก่อน (human-in-the-loop) และเพิ่มการลบข้อมูลเก่า

1.6.6 เขียน Use Case Description พร้อม Sequence Diagram และ Collaboration Diagram

1.6.7 ออกแบบ ER Diagram, โครงสร้างตาราง, Class Diagram และ State Diagram

1.6.8 พัฒนาเว็บแอปพลิเคชันด้วย Next.js และทดสอบ SQL ทุกคำสั่งกับข้อมูลตัวอย่าง

1.6.9 ทดสอบระบบตาม Use Case Description ทั้งกรณีปกติและกรณีผิดพลาด

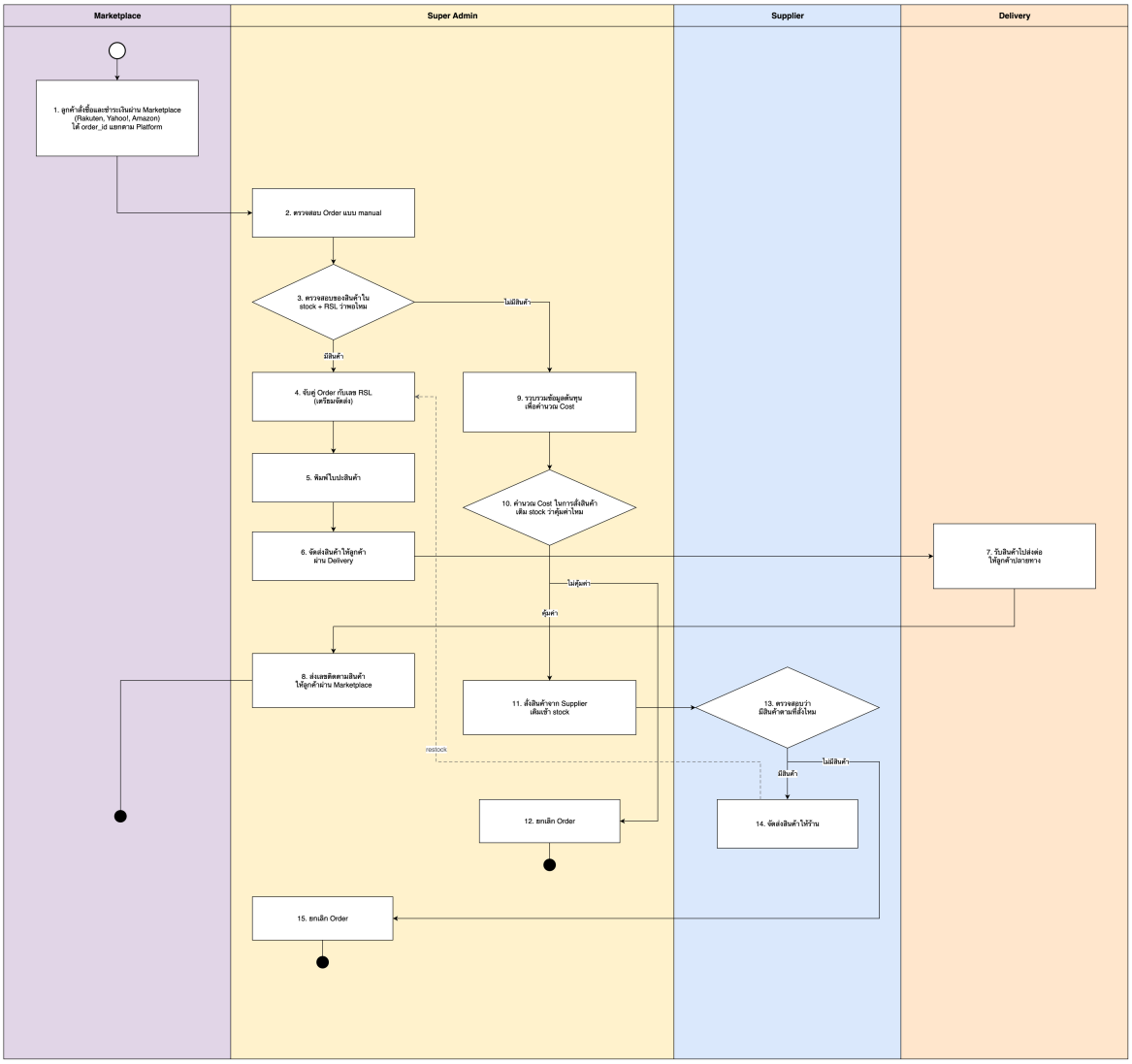
1.6.10 จัดทำรายงานและเตรียมนำเสนอ

1.7 ประโยชน์ที่คาดว่าจะได้รับ

- คำสั่งซื้อ Rakuten ทุกรายการได้ใบปะสินค้าที่ถูกต้องอัตโนมัติ ลดเวลาและความผิดพลาด
- ระบบแจ้งเตือนเมื่อสต๊อกต่ำกว่าเกณฑ์และเตรียมคำสั่งซื้อให้ ลดปัญหาสินค้าขาดและลดการตัดสินใจด้วยความรู้สึก
- คำนวณต้นทุนและกำไรต่อหน่วยได้รวดเร็วและสม่ำเสมอ ช่วยให้ตัดสินใจเรื่องราคาและการสั่งซื้อได้แม่นยำขึ้น
- ลดงานมือประจำวัน เจ้าของมีเวลาไปพัฒนาสินค้าใหม่และเจรจากับโรงงาน
- มีโครงสร้างพร้อมขยายไป Yahoo! Auctions และ Amazon ในเฟสถัดไป

บทที่ 2 การวิเคราะห์และออกแบบระบบ

2.1 Business Process เดิม (As-Is)



รูปที่ 2-1 Business Process เดิม (As-Is)

2.2 คำอธิบาย Business Process เดิม

2.2.1 ลูกค้าสั่งซื้อและชำระเงินผ่าน Marketplace ที่ต้องการ เช่น Rakuten, Yahoo!, Amazon โดยแต่ละ Platform จะสร้าง order_id แยกตาม Platform นั้นๆ

2.2.2 Super Admin ตรวจทาน order_id ที่เข้ามาด้วยตนเองแบบ Manual

2.2.3 Super Admin เช็กว่ามีจำนวนสินค้าที่ลูกค้าต้องการเพียงพอใน Stock หรือไม่

2.2.4 หากมีสินค้าเพียงพอ Super Admin จะจับคู่เลข RSL กับ order_id ด้วยตนเองเพื่อเตรียมจัดส่ง

2.2.5 Super Admin พิมพ์ Label (ใบปะสินค้า) ด้วยตนเอง

2.2.6 Super Admin จัดส่งสินค้าให้บริษัท Delivery ที่ติดต่อไว้เอง

2.2.7 Delivery รับสินค้าไปเพื่อนำไปส่งต่อให้ลูกค้าปลายทาง

2.2.8 Super Admin ส่งเลขติดตามการจัดส่งกลับไปให้ลูกค้าผ่าน Marketplace ด้วยตนเอง เพื่อจบกระบวนการ

2.2.9 หากสินค้าใน Stock ไม่เพียงพอ Super Admin จะเตรียมข้อมูลเพื่อคำนวณต้นทุน (Cost) ว่าคุ้มค่าที่จะสั่งสินค้าเพิ่มหรือไม่

2.2.10 Super Admin คำนวณต้นทุนต่อหน่วยด้วยตนเอง เพื่อตัดสินใจว่าควรสั่งสินค้าเพิ่มหรือไม่

2.2.11 หากคุ้มค่า Super Admin จะสั่งซื้อสินค้าเพิ่มเข้า Stock จาก Supplier

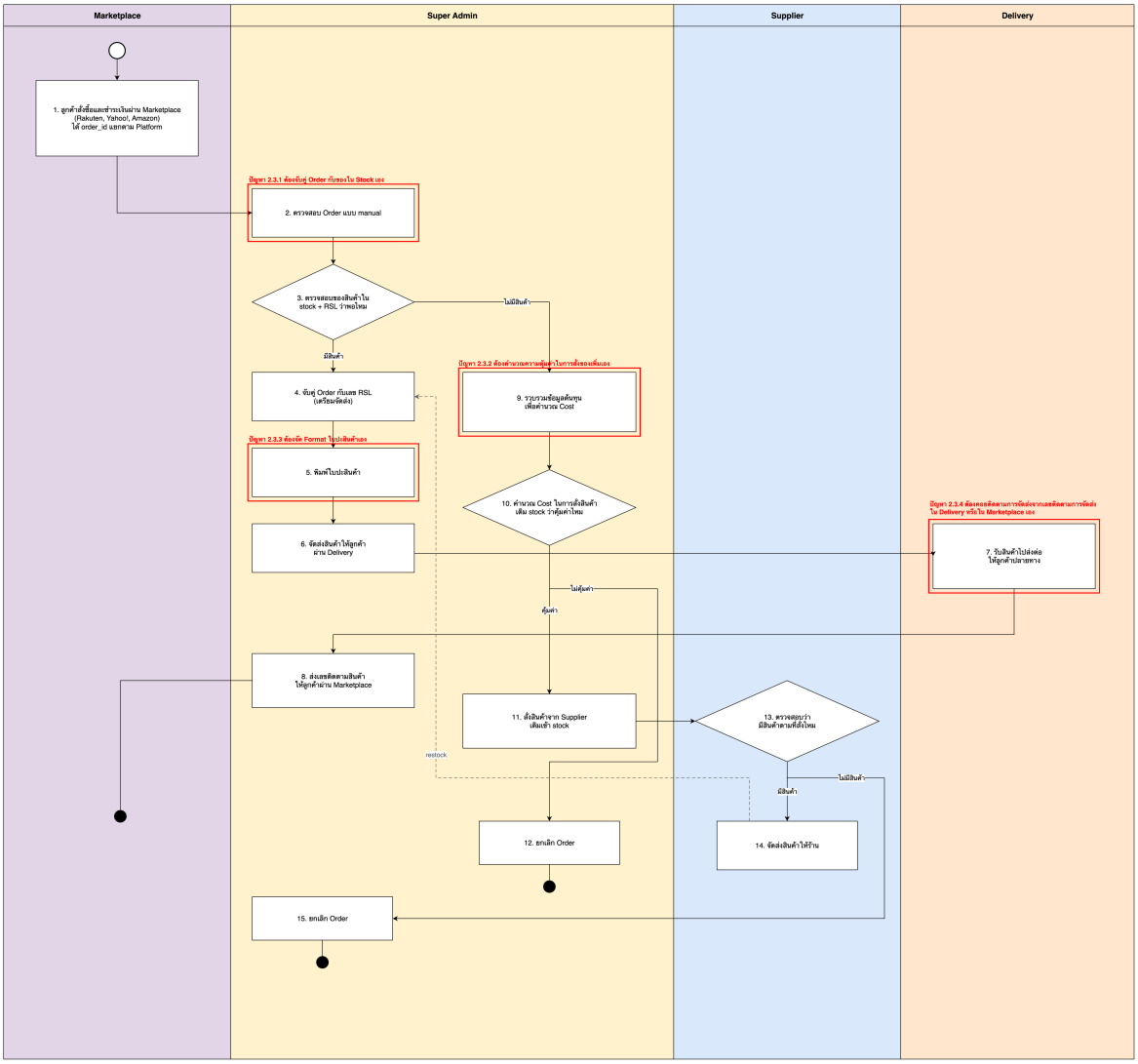
2.2.12 หากไม่คุ้มค่า Super Admin จะยกเลิก Order

2.2.13 Supplier ตรวจสอบว่ามีสินค้าตามที่สั่งหรือไม่

2.2.14 หากมีสินค้า Supplier จะจัดส่งสินค้าเข้าร้าน ทำให้กระบวนการวนกลับไปจับคู่เลข RSL อีกครั้ง (restock)

2.2.15 หาก Supplier ไม่มีสินค้า Super Admin จะยกเลิก Order

2.3 ปัญหาของ Business Process เดิม



รูปที่ 2-2 จุดที่เป็นปัญหาใน Business Process เดิม

จากการวิเคราะห์ Business Process เดิม พบปัญหาดังนี้

2.3.1 **ต้องจับคู่ Order กับสินค้าใน Stock ด้วยตนเอง** — Super Admin ต้องตรวจสอบและจับคู่ order_id กับจำนวนสินค้าคงเหลือใน Stock ทุกครั้งด้วยตนเอง ทำให้ใช้เวลานานและมีโอกาสจับคู่ผิดพลาด เมื่อมี Order เข้ามาพร้อมกันจำนวนมาก ประมาณ 5 นาทีต่อ Order (เช็กสต๊อก 3 นาที และจับคู่ RSL 2 นาที) หรือ 25–50 นาทีต่อวัน (ค่าประมาณ ดูตารางที่ 1-1)

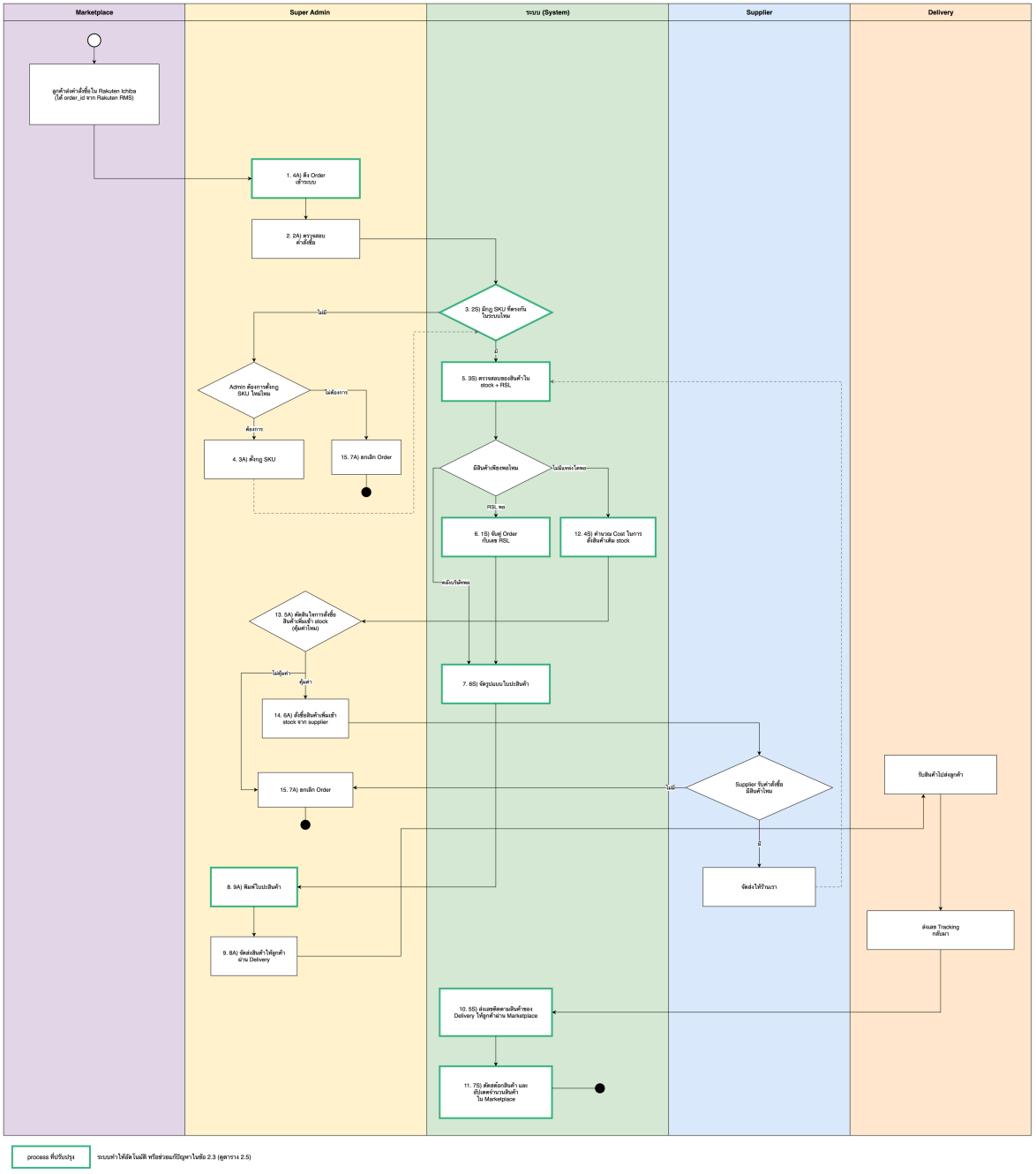
2.3.2 **ต้องคำนวณความคุ้มค่าในการสั่งของเพิ่มด้วยตนเอง** — เมื่อสินค้าใน Stock ไม่พอ Super Admin ต้องคำนวณต้นทุนต่อหน่วยเองเพื่อตัดสินใจว่าควรสั่งเพิ่มหรือไม่ ทำให้การตัดสินใจล่าช้าและ

ขึ้นอยู่กับดุลยพินิจส่วนบุคคล ไม่มีมาตรฐานที่แน่นอน ต้องรวมค่าใช้จ่าย 8 รายการต่อสินค้าหนึ่งรายการ ใช้เวลาประมาณ 20–30 นาทีต่อครั้ง (ค่าประมาณ)

2.3.3 **ต้องจัด Format ใบปะสินค้าเอง** — Super Admin ต้องจัดรูปแบบและพิมพ์ใบปะสินค้าด้วยตนเองทุก Order ทำให้เสียเวลาและเสี่ยงต่อความผิดพลาดของข้อมูลบนใบปะสินค้า ประมาณ 3 นาทีต่อ Order หรือ 15–30 นาทีต่อวัน (ค่าประมาณ)

2.3.4 **ต้องคอยติดตามการจัดส่งจากเลขติดตามการจัดส่งใน Delivery หรือ Marketplace เอง** — Super Admin ต้องคอยเข้าไปเช็กสถานะพัสดุเองทั้งฝั่ง Delivery และ Marketplace แล้วค่อยแจ้งลูกค้า ทำให้ลูกค้าทราบสถานะล่าช้า และเพิ่มภาระงานที่ไม่จำเป็นให้ Super Admin ประมาณ 2 นาทีต่อ Order หรือ 10–20 นาทีต่อวัน (ค่าประมาณ)

2.4 Business Process ใหม่ (To-Be)



รูปที่ 2-3 Business Process ใหม่ (To-Be)

2.5 คำอธิบาย Business Process ใหม่

ออกแบบใหม่โดยเพิ่ม Actor "ระบบ (System)" เข้ามารับผิดชอบขั้นตอนที่ทำงานอัตโนมัติได้ แยกออกจาก Super Admin ที่ยังต้องทำ/อนุมัติด้วยตนเองในบางขั้นตอน ดังตาราง

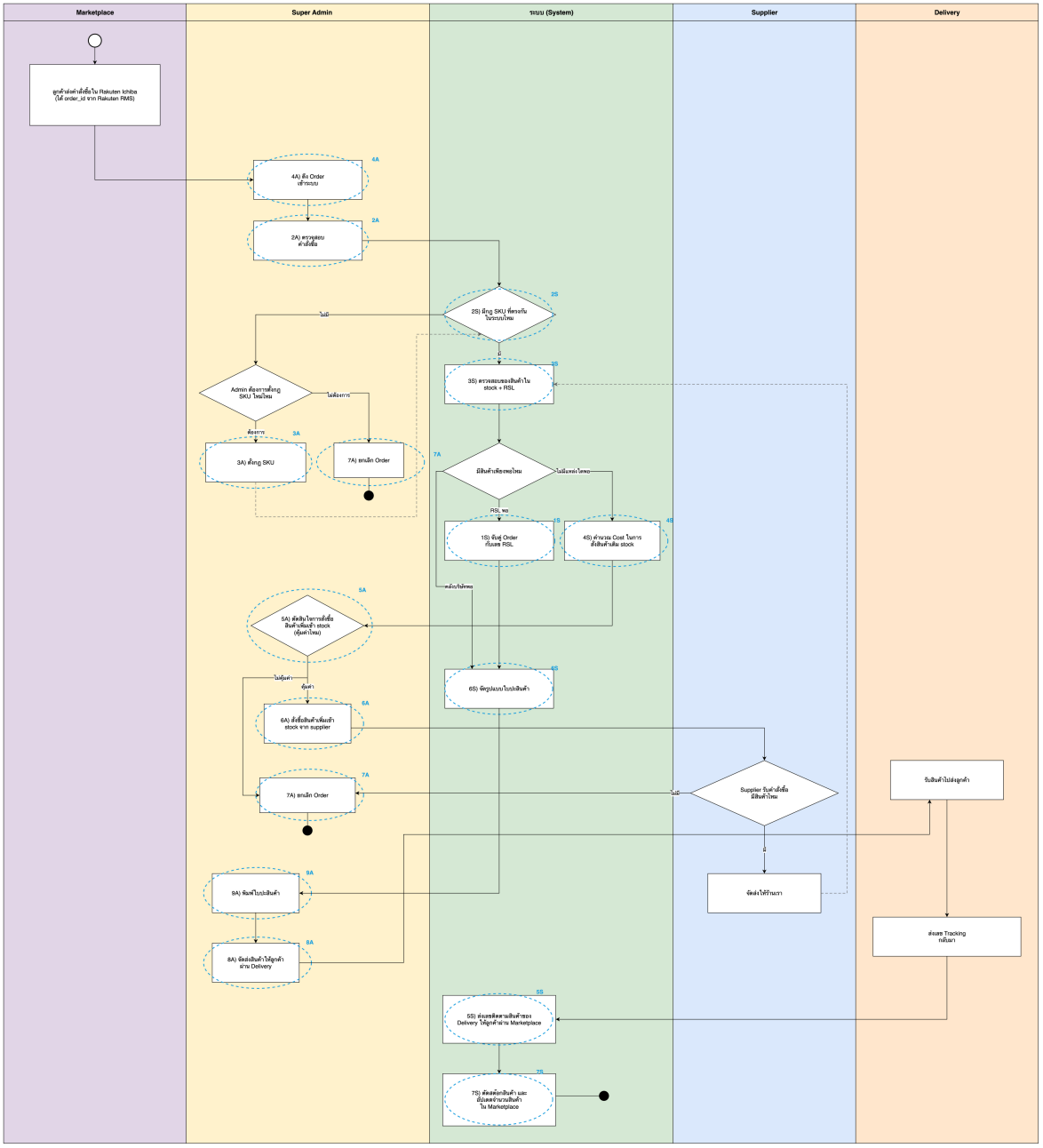
ตารางที่ 2-1 ขั้นตอนของ Business Process ใหม่ ผู้ทำ และปัญหาที่แก้

#	ขั้นตอน (Process)	ผู้ทำ	แก้ปัญหาข้อ (2.3)
1	4A) ดึง Order เข้าระบบ	Manual — Super Admin กดนำเข้า Order จาก Marketplace ที่หน้า Dashboard (เว็บต้นแบบยังไม่มีการดึงตามรอบเวลา)	—
2	2A) ตรวจสอบคำสั่งซื้อ	Manual — Super Admin ตรวจทาน Order ที่ดึงเข้ามา	—
3	2S) จับคู่กฎ SKU	Automatic — ระบบเช็กว่ามีกฎแปลง SKU ของ Order นี้อยู่แล้วหรือไม่	—
4	3A) ตั้งกฎ SKU (เฉพาะกรณีไม่พบ)	Manual — Super Admin ตั้งกฎ SKU ใหม่ เมื่อบันทึกแล้วระบบส่ง Order ที่ค้างเพราะกฎ SKU กลับไปผ่าน 2S และ 3S ต่อทันที	—
5	3S) ตรวจสอบของสินค้าใน stock + RSL	Automatic — ระบบเช็กสต๊อกให้เอง เลือก RSL ก่อน ถ้า RSL ไม่พอแต่คลังบริษัทพอให้บริษัทจัดส่งเอง (ข้ามขั้นตอนที่ 6 ไปขั้นตอนที่ 7) ถ้าไม่มีแหล่งใดพอไปขั้นตอนที่ 12	2.3.1
6	1S) จับคู่ Order กับเลข RSL	Automatic — ระบบจับคู่ให้เอง ไม่ต้องจับคู่ด้วยมือ (เฉพาะ Order ที่ RSL จัดส่ง)	2.3.1
7	6S) จัดรูปแบบใบปะสินค้า	Automatic — ระบบจัดรูปแบบใบปะสินค้าให้เอง เหลือแค่ Super Admin กดพิมพ์	2.3.3
8	9A) พิมพ์ใบปะสินค้า	Manual — ขั้นตอนทางกายภาพ (พิมพ์ออกกระดาษจริง) จึงยังต้องมีคนกด	2.3.3 (บางส่วน)
9	8A) จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	Manual — Super Admin ส่งมอบพัสดุให้บริษัทขนส่ง	—
10	5S) ส่งเลขติดตามสินค้าให้ลูกค้าผ่าน Marketplace	Automatic — ระบบส่งเลขติดตามให้ลูกค้าเอง ทันทีที่ Delivery แจ้งกลับมา	2.3.4
11	7S) ตัดสต๊อกสินค้า และอัปเดตจำนวนใน Marketplace	Automatic — ระบบตัดสต๊อกและอัปเดตยอดคงเหลือให้เอง Order ที่ยังตัดสต๊อกไม่สำเร็จถูกหยิบมาทำซ้ำในการทำงานครั้งถัดไป	2.3.4

#	ขั้นตอน (Process)	ผู้ทำ	แก้ปัญหาข้อ (2.3)
12	4S) คำนวณ Cost ในการสั่งสินค้าเติม stock	Automatic — ระบบคำนวณต้นทุนต่อหน่วยให้เอง	2.3.2
13	5A) ตัดสินใจการสั่งซื้อสินค้าเพิ่ม (คุ้มค่าไหม)	Manual โดยตั้งใจ — แม้ระบบคำนวณต้นทุนให้แล้ว แต่การอนุมัติสั่งซื้อที่มีผลผูกพันทางการเงินยังต้องให้ Super Admin เป็นผู้ตัดสินใจ (human-in-the-loop) เพื่อป้องกันระบบสั่งซื้อผิดพลาดโดยไม่มีคนตรวจสอบก่อน	2.3.2 (ช่วยคำนวณ แต่ยังให้คนอนุมัติ)
14	6A) สั่งซื้อสินค้าเพิ่มจาก Supplier	Manual — Super Admin ยืนยันการสั่งซื้อต่อจาก 5A ระบบสร้างใบสั่งซื้อ ผูก Order ที่รอสินค้าทุกรายการ และส่งให้ Supplier เมื่อสินค้ามาถึง Super Admin กดรับสินค้า ระบบเพิ่มสต๊อกในบริษัทแล้วส่ง Order เหล่านั้นกลับไปขั้นตอนที่ 5 (3S)	—
15	7A) ยกเลิก Order	Manual — Super Admin ยกเลิก Order พร้อมระบุเหตุผล ระบบคืนสต๊อกให้ถ้าตัดไปแล้ว	—
16	10A) ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น/Cost/Label/Reorder)	Manual — Super Admin สั่งล้างข้อมูลที่ใช้งานเสร็จแล้วเป็นระยะ เพื่อไม่ให้ฐานข้อมูลโตขึ้นเรื่อยๆ โดยไม่มีประโยชน์	ข้อมูลเก่าสะสมจนฐานข้อมูลใหญ่ขึ้น (data retention)

ขั้นตอนที่ 13 เป็นจุดออกแบบสำคัญ — แม้จะมี Actor "ระบบ" ทำงานอัตโนมัติได้เกือบทั้งหมด แต่จงใจ**ไม่ให้ระบบสั่งซื้อสินค้าไปเองโดยไม่มีคนอนุมัติ** เพราะเป็นขั้นตอนที่ผูกกับเงินจริง (ดู Use Case Diagram: มีเส้นประสีแดง "ต้องอนุมัติ (5A) ก่อน" กำกับไว้ระหว่าง 5A ⟶ 6A)

2.6 Business Process ที่สัมพันธ์กับ Use Case



รูปที่ 2-4 Business Process ที่สัมพันธ์กับ Use Case

2.7 ตารางจับคู่ระหว่าง Business Process และ Use Case Diagram

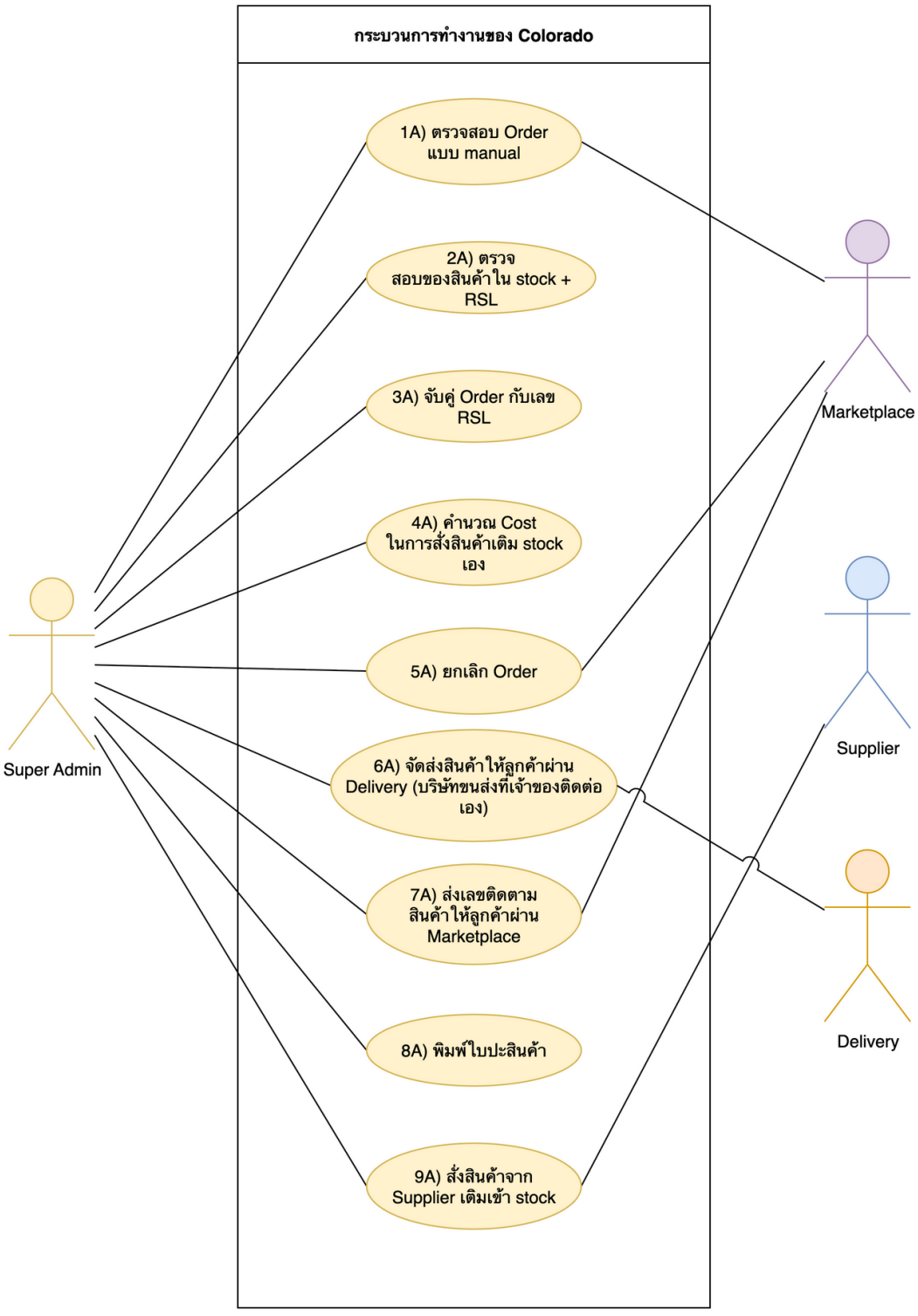
ตารางที่ 2-2 ตารางจับคู่ระหว่าง Business Process และ Use Case Diagram

#	Business Process	Use Case Diagram
1	ดึง Order เข้าระบบ	4A
2	ตรวจสอบคำสั่งซื้อ	2A
3	มีกฎ SKU ที่ตรงกันในระบบไหม	2S
4	ตั้งกฎ SKU	3A
5	ยกเลิก Order (กรณีไม่ตั้งกฎ SKU)	7A
6	ตรวจสอบของสินค้าใน stock + RSL	3S
7	จับคู่ Order กับเลข RSL	1S
8	จัดรูปแบบใบปะสินค้า	6S
9	พิมพ์ใบปะสินค้า	9A
10	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	8A
11	ส่งเลขติดตามสินค้าให้ลูกค้า	5S
12	ตัดสต๊อกสินค้า	7S
13	คำนวณ Cost (กรณีสินค้าไม่พอ)	4S
14	ตัดสินใจสั่งซื้อเพิ่ม	5A
15	สั่งซื้อสินค้าเพิ่มจาก Supplier	6A
16	ยกเลิก Order (กรณีไม่คุ้มค่า / Supplier ไม่มีของ)	7A

1A) เข้าสู่ระบบ ไม่ปรากฏเป็นกล่องแยกใน Business Process เพราะเป็น precondition ก่อนเริ่มกระบวนการ (ต้อง login ก่อนถึงจะเริ่มทำงานได้)

10A) ลบข้อมูลเก่าที่ไม่ใช้แล้ว ไม่ปรากฏใน Business Process หลัก เพราะเป็น maintenance Use Case ที่ Super Admin ทำเป็นระยะ ไม่ได้ผูกกับขั้นตอนการรับ-ส่ง Order โดยตรง

2.8 Use Case Diagram ก่อนปรับปรุง



รูปที่ 2-5 Use Case Diagram ก่อนปรับปรุง

2.9 CRUD Table ก่อนปรับปรุง

ตารางที่ 2-3 CRUD Table ก่อนปรับปรุง

Use Case No.	Use Case	Order	Stock	Label	Cost	Reorder	Supplier
1A	ตรวจสอบ Order แบบ manual	RU					
2A	ตรวจสอบของสินค้าใน stock + RSL	R	R				
3A	จับคู่ Order กับเลข RSL	RU	RU				
4A	คำนวณ Cost ในการสั่งสินค้าเติม stock เอง	R	R		CR		
5A	ยกเลิก Order	RUD					
6A	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	RU		RU			
7A	ส่งเลขติดตามสินค้าให้ลูกค้าผ่าน Marketplace	RU		R			
8A	พิมพ์ใบปะสินค้า	R		C			
9A	สั่งสินค้าจาก Supplier เติมเข้า stock		R		R	C	R

หมายเหตุ: ไม่มี Use Case ใดมีสิทธิ์ Delete ข้อมูลเก่า (นอกจาก 5A ยกเลิก Order) ดูข้อ 2.10.4

2.10 คำอธิบาย CRUD Table ก่อนปรับปรุง

จากการวิเคราะห์ CRUD Table ก่อนปรับปรุง (2.9) ซึ่งมี Use Case ทั้งหมด 9 รายการ ผูกอยู่กับ Super Admin เกือบทั้งหมดในลักษณะ Manual พบว่ายังขาด Use Case ต่อไปนี้ ซึ่งเป็นเหตุผลรองรับการออกแบบ Use Case ใหม่ในข้อ 2.11

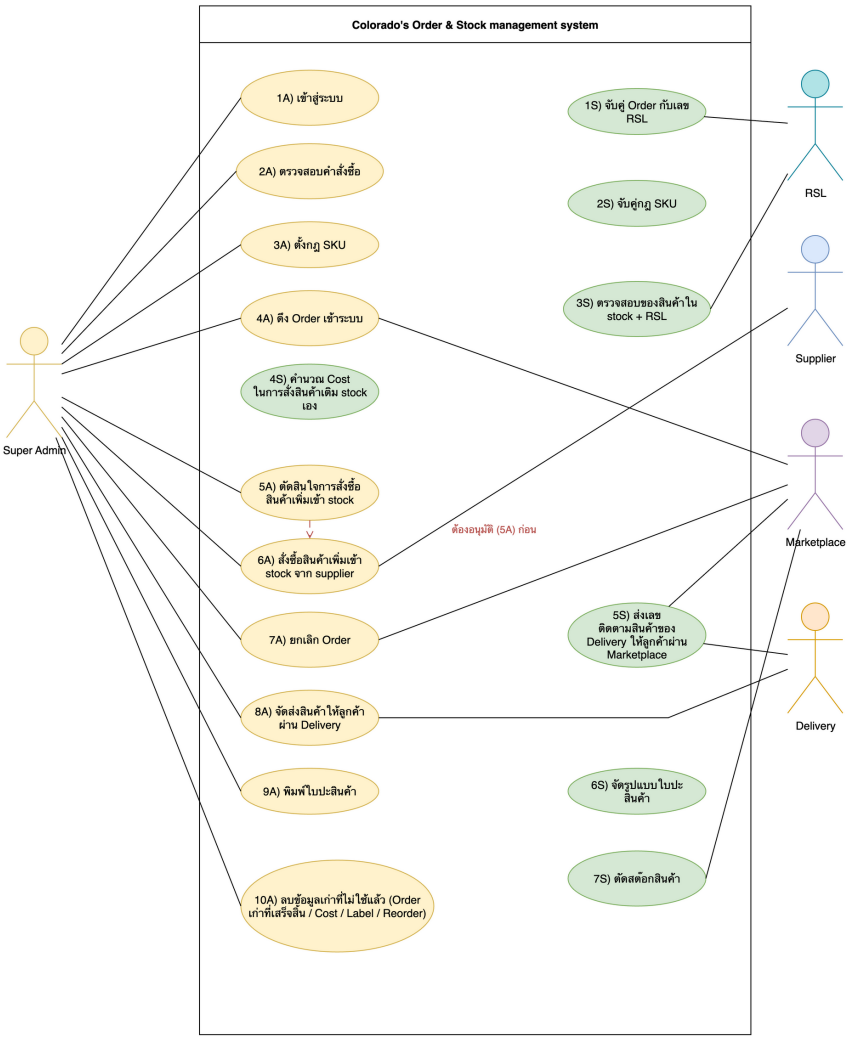
2.10.1 **ไม่มี Use Case สำหรับตั้งกฎ SKU และจับคู่กฎ SKU** — Business Process เดิมไม่มีแนวคิดเรื่อง SKU rule เลย ทำให้เมื่อสินค้าใหม่เข้ามาไม่มีขั้นตอนที่ชัดเจนว่าจะจับคู่สินค้ากับ order_id อย่างไรหากยังไม่เคยเจอ SKU นั้นมาก่อน — To-Be จึงต้องเพิ่ม 2S) จับคู่กฎ SKU และ 3A) ตั้งกฎ SKU

2.10.2 **งานอัตโนมัติยังผูกกับ Super Admin** — CRUD เดิมผูกทุก Use Case ไว้กับ Super Admin คนเดียว ทั้งที่หลายขั้นตอน (จับคู่ RSL, คำนวณต้นทุน, จัดรูปแบบใบปะสินค้า, ส่งเลขติดตาม, ตัดสต๊อก) ควรเป็นงานที่ระบบทำอัตโนมัติได้ ไม่จำเป็นต้องให้คนกดทุกครั้ง — To-Be จึงแยกเป็น Use Case ที่ระบบทำเอง (1S ถึง 7S)

2.10.3 **ไม่มี Use Case สำหรับตัดสต๊อกหลังส่งของสำเร็จ** — ไม่มี Use Case ใดใน CRUD เดิมที่ปรับปรุง (U) ยอด Stock หลังส่งของ และอัปเดตจำนวนคงเหลือในตลาดออนไลน์ Super Admin ต้องทำเองโดยไม่มีขั้นตอนที่ชัดเจน — To-Be จึงแยกเป็น Use Case ชัดเจน คือ 7S) ตัดสต๊อกสินค้า

2.10.4 **ไม่มี Use Case สำหรับลบ/ล้างข้อมูลเก่า (Delete)** — ทุกแถวใน CRUD เดิมมีแค่ Create/Read/Update ยกเว้น 5A ยกเลิก Order ที่มี D อยู่แล้ว แต่ไม่มี Use Case ใดที่ลบข้อมูลเก่าที่ใช้งานเสร็จสิ้นแล้ว (เช่น Order เก่า, ข้อมูลคำนวณต้นทุนเก่า) ทำให้ข้อมูลสะสมไปเรื่อยๆ โดยไม่มีประโยชน์เปลืองพื้นที่จัดเก็บ — To-Be จึงเพิ่ม 10A) ลบข้อมูลเก่าที่ไม่ใช้แล้ว

2.11 Use Case Diagram ที่ทบทวนปรับปรุงแล้ว



รูปที่ 2-6 Use Case Diagram ที่ทบทวนปรับปรุงแล้ว

- **Actor: Super Admin (เจ้าของร้าน) (10 Use Case)** — 1A, 2A, 3A, 4A, 5A, 6A, 7A, 8A, 9A, 10A
- **Use Case ที่ระบบทำอัตโนมัติ (7 Use Case)** — 1S, 2S, 3S, 4S, 5S, 6S, 7S ระบบทำต่อเองโดยไม่ต้องมีคนกด จึงไม่มีเส้นจาก actor ในรูป ในเว็บต้นแบบ 2S, 3S, 6S และ 7S ทำงานใน executeWorkflow (execute-workflow.ts) ต่อจากการกระทำของ Admin ใน transaction เดียวกัน ไม่มีตัวตั้งเวลา (Scheduler) แยก ส่วน 1S, 4S และ 5S Super Admin กดปุ่มสั่งให้เริ่ม แต่ขั้นตอนทั้งหมดระบบทำเอง จึงไม่วาดเส้นจาก Super Admin
- **Actor ภายนอก:** Marketplace, Supplier, Delivery, RSL

2.12 CRUD Table ที่ทบทวนแล้ว

CRUD Table ที่ทบทวนแล้วสร้างจากคำสั่ง SQL ทุกข้อใน Use Case Description (2.13) ชื่อ entity จึงตรงกับตารางใน ER Diagram (2.14) ทุกตัว ตัวอักษร C, R, U, D หมายถึง Create, Read, Update และ Delete ตามคำสั่ง INSERT, SELECT, UPDATE และ DELETE ของ Use Case นั้น เมื่อเทียบกับตารางก่อนปรับปรุง มี Use Case ที่เพิ่มเข้ามาดังนี้

- **2A) ตรวจสอบคำสั่งซื้อ** — อ่านและปรับสถานะ Order (R, U) ก่อนส่งเข้ากระบวนการจับคู่กฎ SKU
- **2S) จับคู่กฎ SKU** และ **3A) ตั้งกฎ SKU** — ใช้คอลัมน์ skus.channel_sku จับคู่รหัสสินค้าของช่องทางขายกับ SKU ภายใน (ไม่มีตารางกฎแยก)
- **9A) พิมพ์ใบปะสินค้า** — แยกจาก 6S ที่จัดรูปแบบใบปะ ให้ Admin เป็นผู้กดพิมพ์ (U ที่ labels และ orders, C ที่ label_print_history)
- **5S) ส่งเลขติดตามสินค้าให้ลูกค้าผ่าน Marketplace** และ **7S) ตัดสต๊อกสินค้า** — แยกงานหลังส่งมอบพัสดุให้ระบบทำเอง
- **10A) ลบข้อมูลเก่าที่ไม่ใช้แล้ว** — เป็น Use Case เดียวที่มี D ลบ Order ที่ปิดแล้ว ประวัติต้นทุน ใบปะ และคำสั่งซื้อเติมสต๊อกที่ปิดแล้ว (แบบลบตามช่วงวันที่ต้องเก่ากว่า 12 เดือน) และบันทึก snapshot ลง deletion_history

ตารางที่ 2-4 CRUD Table ที่ทบทวนแล้ว (สร้างจาก SQL ใน Use Case Description)

Use Case	admins	suppliers	skus	stock	orders	order_items	costs	reorders	labels	sessions	auto_error_logs	deletion_history	cost_history	tracking_notification_logs	rakuten_stock_sync_states	stock_adjustment_history	label_print_history
1A เข้าสู่ระบบ	RU									CRU							
2A ตรวจสอบคำสั่งซื้อ			R	R	RU	RU		R	R					R	R		
3A ตั้งกฎ SKU		R	CRUD	CD	U	RU	D	R					D				
4A ดึง Order เข้าระบบ			R		CRU	C											
5A ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock			R		RU	R	R						R				
6A สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier		R	R	RU	U	R		CRU							CU	C	
7A ยกเลิก Order			R	U	RU	RU		RU	R								
8A จัดส่งสินค้าให้ลูกค้าผ่าน Delivery				U	RU	RU			R								
9A พิมพ์ใบปะสินค้า					RU				RU								C
10A ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)			R		RD	R	RD	D	RD			CR	UD				
1S จับคู่ Order กับเลข RSL			R		RU	R		R	RD								
2S จับคู่กฎ SKU			R		RU	RU		R	R		C						
3S ตรวจสอบของสินค้าใน stock + RSL			R	RU	RU	RU					C				CU	C	
4S คำนวณ Cost ในการสั่งสินค้าเติม stock เอง			R				CRU						CR				
5S ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace					RU	R								CRU			
6S จัดรูปแบบใบปะสินค้า			R		RU	R		R	CR		C						
7S ตัดสต๊อกสินค้า			R	RU	RU	RU					C				CRU		

2.13 Use Case Description, Sequence Diagram, Collaboration Diagram และ User Interface

Sequence Diagram และ Collaboration Diagram ใช้รูปแบบ MVC ตามโครงสร้างโค้ดจริง ได้แก่ Actor ⟶ Screen (หน้าเว็บ page.tsx ซึ่งเป็นชั้น View ของ MVC) ⟶ Controller (Server Action ใน actions.ts) ⟶ Model (UseCase และ Queries ใน src/lib) ⟶ DBConnector (db.ts) ส่วนระบบภายนอก (Rakuten, RSL, ขนส่ง, Supplier) เป็น object แยกที่ Model เรียกผ่าน Adapter Screen และ Controller เป็นชื่อบทบาทของไฟล์จริง ไม่ใช่คลาสที่เพิ่มขึ้นในโค้ด object ที่อยู่ติดกับ Actor ตั้งชื่อลงท้ายว่า Screen ไม่ใช้ชื่อ View ตามเกณฑ์ข้อ 24

ไม่มีลูกศรย้อนกลับไปที่ Actor ผลลัพธ์ส่งกลับจาก Controller มาที่ Screen แล้ว Screen แสดงผลเองด้วย self message (เช่น showError(), showSuccess()) และ Screen ไม่เรียก Model หรือฐานข้อมูลตรง การอ่านข้อมูลตอนเปิดหน้าทำผ่าน Controller loadAppState() Use Case ที่ระบบทำเอง (S) ไม่มี Actor เริ่มจาก Use Case ที่เรียกหรือ ExecuteWorkflow แทน Sequence Diagram ที่ยาวแบ่งเป็นหลายรูปตามลำดับ (1/n) โดยแสดงหัว object ซ้ำทุกรูป

2.13.1 Use Case Description ที่ 1A: เข้าสู่ระบบ

Goal: ให้ Super Admin สามารถเข้าใช้งานระบบได้ และออกจากระบบเมื่อเลิกใช้งาน

Pre-Condition: Super Admin มีบัญชีอยู่ในตาราง `admins` แล้ว และอยู่ที่หน้า Login (`/login`)

Post-Condition: Super Admin เข้าสู่ระบบสำเร็จ ระบบสร้าง session ในตาราง `sessions` และ redirect ไปหน้า Dashboard (`/dashboard`) / เมื่อออกจากระบบ session ถูกยกเลิก (`revoked_at`) cookie ถูกลบ และกลับมาที่หน้า Login

Actor: Super Admin

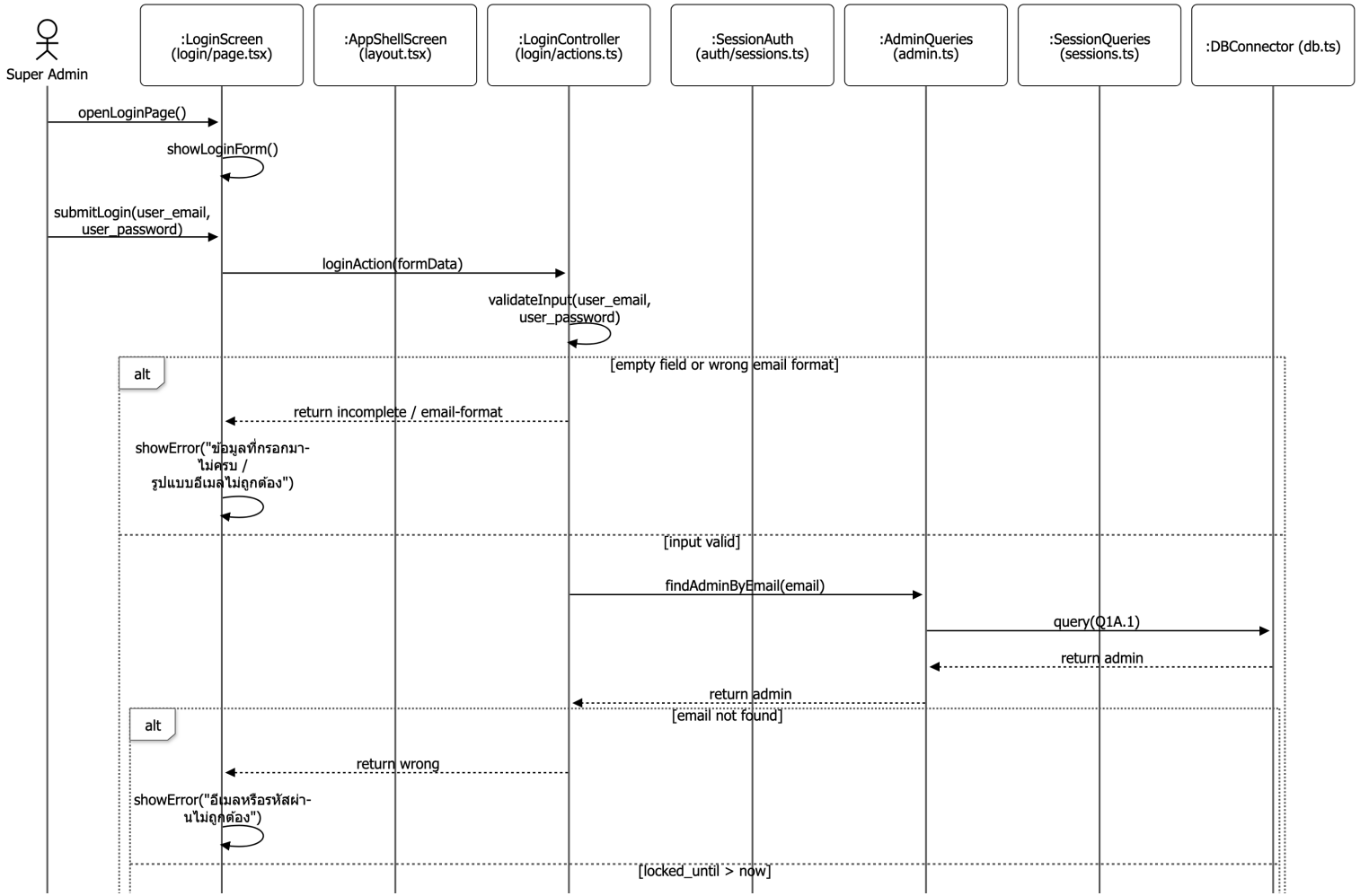
ตารางที่ 2-5 Use Case Description ที่ 1A: เข้าสู่ระบบ

ลำดับ	User Action	System Response
1		แสดงฟอร์มเข้าสู่ระบบ โดยต้องกรอกอีเมล และ รหัสผ่าน
2	กรอกอีเมล (`user_email`) และรหัสผ่าน (`user_password`)	

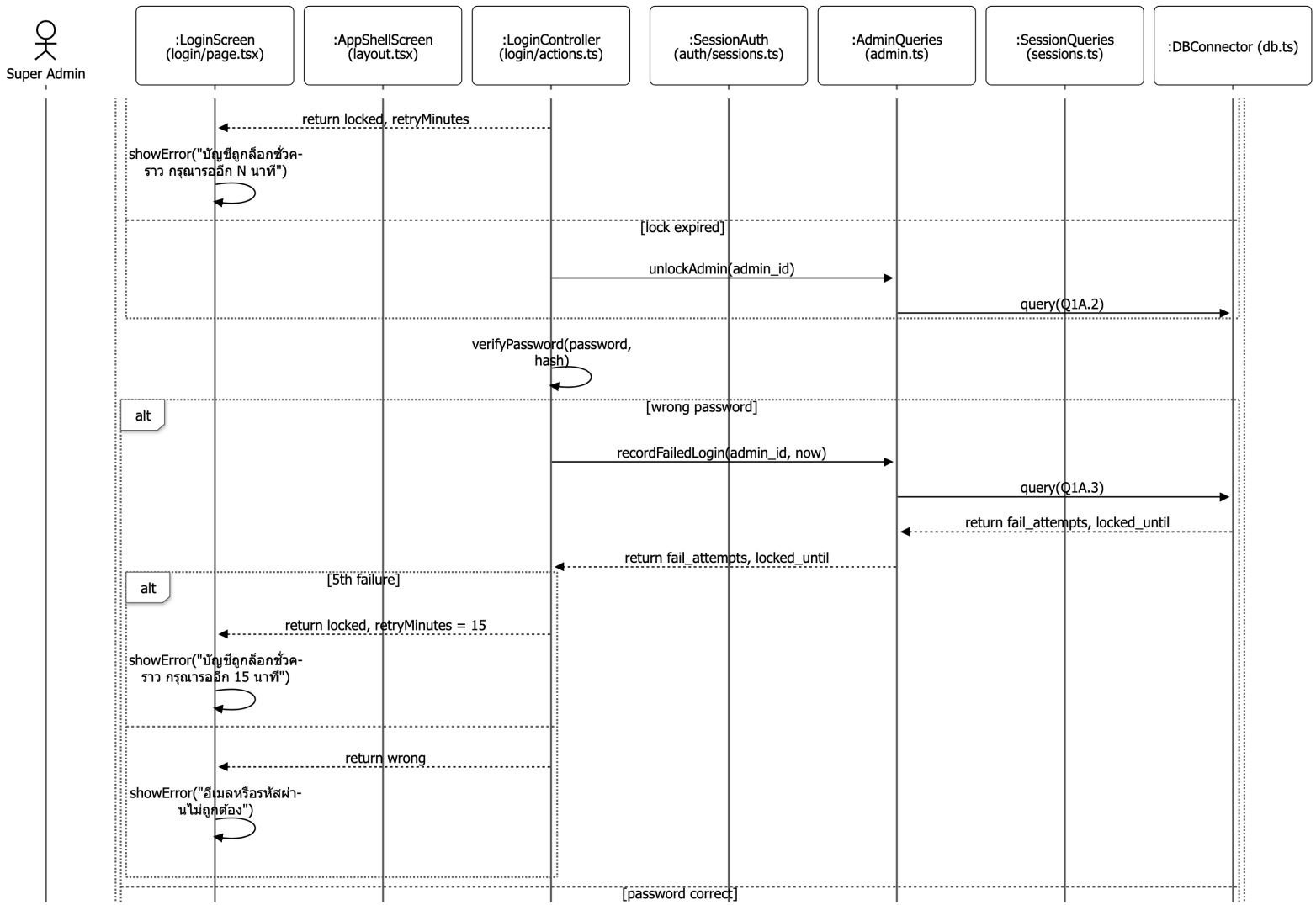
ลำดับ	User Action	System Response
3	กด "เข้าสู่ระบบ"	ระบบเรียก Server Action `loginAction` (`src/app/login/actions.ts`) **1. ตรวจสอบค่าที่กรอกมา** (`user_email` & `user_password`) — ต้องไม่เป็นช่องว่าง หากเป็นช่องว่าง จะแจ้งเตือนว่า "ข้อมูลที่กรอกมาไม่ครบ" **2. ตรวจสอบรูปแบบอีเมล** — ต้องอยู่ในรูปแบบ `[A-Za-z0-9]+@[A-Za-z0-9]+\.[A-Za-z0-9]+` เท่านั้น หากไม่ตรงจะแจ้งเตือนว่า "รูปแบบของอีเมลที่กรอกมาไม่ถูกต้อง" **3. ตรวจสอบอีเมลและรหัสผ่าน** (ทำใน transaction เดียว) — Q1A.1: `SELECT admin_id, user_email, user_password, fail_attempts, locked_until, last_login FROM admins WHERE user_email = $1` — หากไม่พบอีเมลในระบบ จะแจ้งเตือนว่า "อีเมลหรือรหัสผ่านไม่ถูกต้อง" (ไม่ระบุว่าผิดที่ฟิลด์ใด เพื่อความปลอดภัย) และไม่นับจำนวนครั้ง เพราะไม่มีบัญชีให้นับ — หากพบอีเมลและบัญชียังถูกล็อกอยู่ (`locked_until` มากกว่าเวลาปัจจุบัน) จะแจ้งเตือนว่า "คุณพยายามเข้าสู่ระบบบ่อยเกินไป บัญชีถูกล็อกชั่วคราว กรุณารออีก N นาทีก่อนลองใหม่อีกครั้ง" (N = จำนวนนาทีที่เหลือปัดขึ้น อย่างน้อย 1) และไม่ตรวจรหัสผ่าน — หากเคยถูกล็อกแต่ครบเวลาแล้ว (`locked_until` ไม่ว่าง หรือ `fail_attempts` ≥ 5) ระบบปลดล็อกให้อัตโนมัติก่อนตรวจรหัสผ่าน — Q1A.2: `UPDATE admins SET fail_attempts = 0, locked_until = NULL WHERE admin_id = $1` — หากรหัสผ่านไม่ตรงกับค่า hash (scrypt) ที่เก็บไว้ จะแจ้งเตือนว่า "อีเมลหรือรหัสผ่านไม่ถูกต้อง" เช่นกัน พร้อม +1 ให้ `fail_attempts` ถ้าครบ 5 ครั้ง ตั้ง `locked_until` = เวลาปัจจุบัน + 15 นาที และแจ้งเตือนว่าบัญชีถูกล็อกทันทีตั้งแต่ครั้งที่ 5 — Q1A.3: `UPDATE admins SET fail_attempts = fail_attempts + 1, locked_until = CASE WHEN fail_attempts + 1 >= $3 THEN $2::timestamptz + make_interval(mins => $4::integer) ELSE locked_until END WHERE admin_id = $1 RETURNING fail_attempts, locked_until` ($2 = เวลาปัจจุบัน, $3 = 5 ครั้ง, $4 = 15 นาที) — หากรหัสผ่านถูกต้อง (เข้าสู่ระบบสำเร็จ): 1. set `fail_attempts` = 0, `locked_until` = NULL และ `last_login` = เวลาปัจจุบัน — Q1A.4: `UPDATE admins SET fail_attempts = 0, locked_until = NULL, last_login = $2 WHERE admin_id = $1` 2. สร้าง session ใหม่ เก็บเฉพาะค่า hash ของ token และเวลาหมดอายุ — Q1A.5: `INSERT INTO sessions ( session_id, admin_id, token_hash, created_at, expires_at, revoked_at ) VALUES ($1, $2, $3, $4, $5, NULL)` 3. ตั้ง cookie แบบ HttpOnly ที่เก็บ token แล้ว redirect ไปหน้า Dashboard

ลำดับ	User Action	System Response
4	เปิดหน้าใดก็ได้ในระบบหลังเข้าสู่ระบบ	**ตรวจสอบ session ทุกครั้งที่เปิดหน้า** (`requireSession` ใน layout และทุก Server Action) — นำ hash ของ token ใน cookie ไปค้นหา session ที่ยังไม่ถูกยกเลิกและยังไม่หมดอายุ — **Q1A.6:** `SELECT s.session_id, s.admin_id, a.user_email, s.created_at, s.expires_at FROM sessions AS s INNER JOIN admins AS a ON a.admin_id = s.admin_id WHERE s.token_hash = $1 AND s.revoked_at IS NULL AND s.expires_at > $2` — หากไม่พบ session ที่ใช้งานได้ ระบบ redirect กลับไปหน้า Login
5	กด "ออกจากระบบ" ที่แถบเมนูด้านข้าง	ระบบถามยืนยันว่า "ต้องการออกจากระบบหรือไม่" เมื่อยืนยันแล้วเรียก Server Action `logoutAction` **ยกเลิก session** — **Q1A.7:** `UPDATE sessions SET revoked_at = $2 WHERE token_hash = $1 AND revoked_at IS NULL` ระบบลบ cookie แล้ว redirect กลับไปหน้า Login พร้อมแจ้งว่า "ออกจากระบบแล้ว"
ทางเลือก #1	ฐานข้อมูลไม่พร้อมใช้งานระหว่างเข้าสู่ระบบ	แจ้งเตือนว่า "ไม่สามารถเชื่อมต่อฐานข้อมูลได้ กรุณาลองใหม่ภายหลัง" (หรือ "ไม่สามารถดำเนินการกับฐานข้อมูลได้ กรุณาลองใหม่ภายหลัง") และ rollback transaction ทั้งหมด ไม่มีการเปลี่ยน `fail_attempts`
ทางเลือก #2	ฐานข้อมูลไม่พร้อมใช้งานขณะออกจากระบบ	ระบบยังลบ cookie และพากลับไปหน้า Login เพื่อไม่ให้ผู้ใช้ค้างอยู่ในหน้าที่ต้องเข้าสู่ระบบ
ทางเลือก #3	บัญชีถูกล็อกและ Super Admin ต้องการทดสอบใหม่ทันที (เฉพาะสภาพแวดล้อมพัฒนา)	หน้า `/dev` มีปุ่ม "ปลดล็อกบัญชี" ล้าง `fail_attempts` และ `locked_until` ของทุกบัญชี (เครื่องมือทดสอบ ไม่ใช่ขั้นตอนของ Use Case) ในการใช้งานจริงบัญชีปลดล็อกเองเมื่อครบ 15 นาที

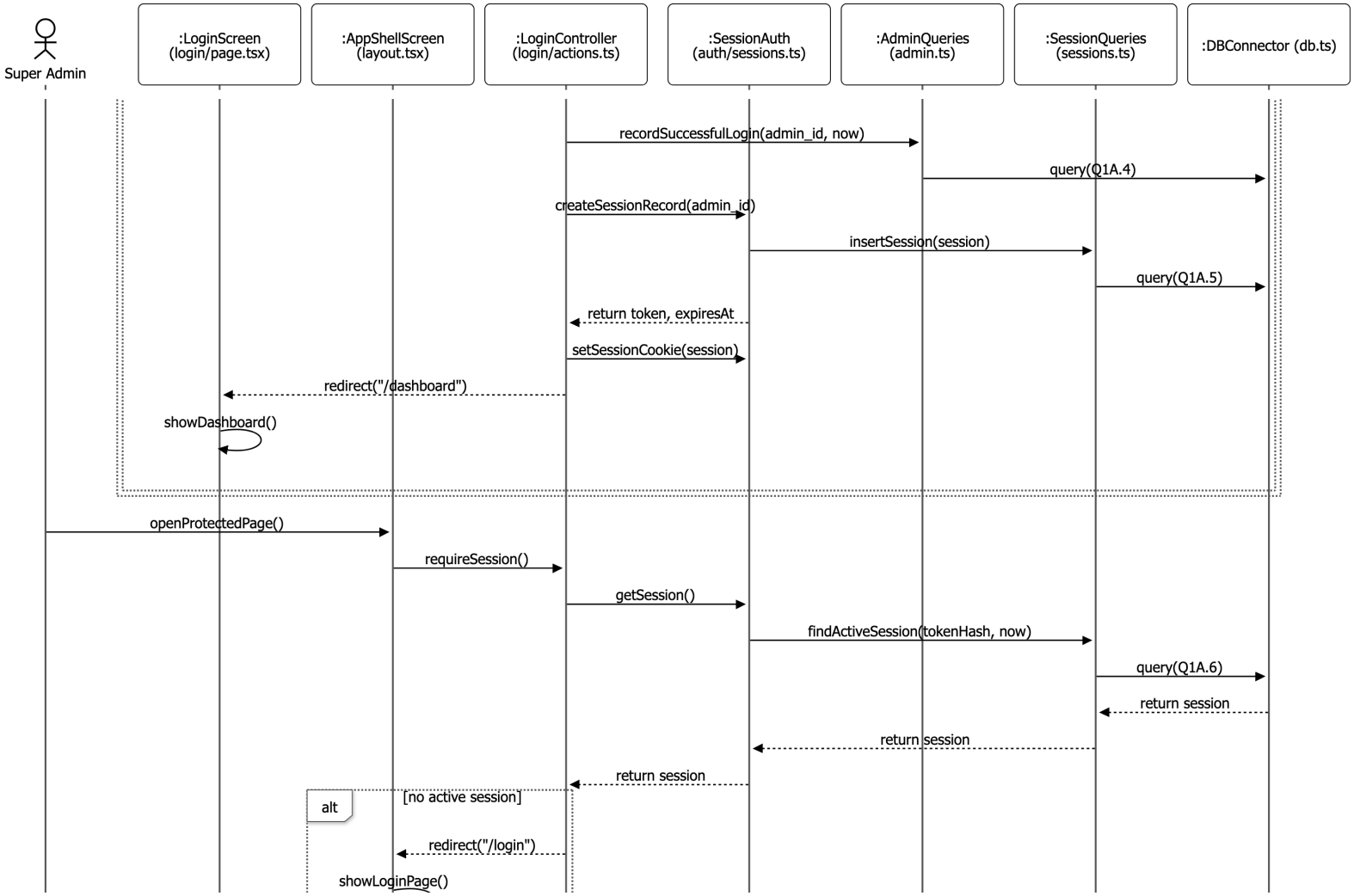
2.13.2 Sequence Diagram ของ Use Case Description ที่ 1A



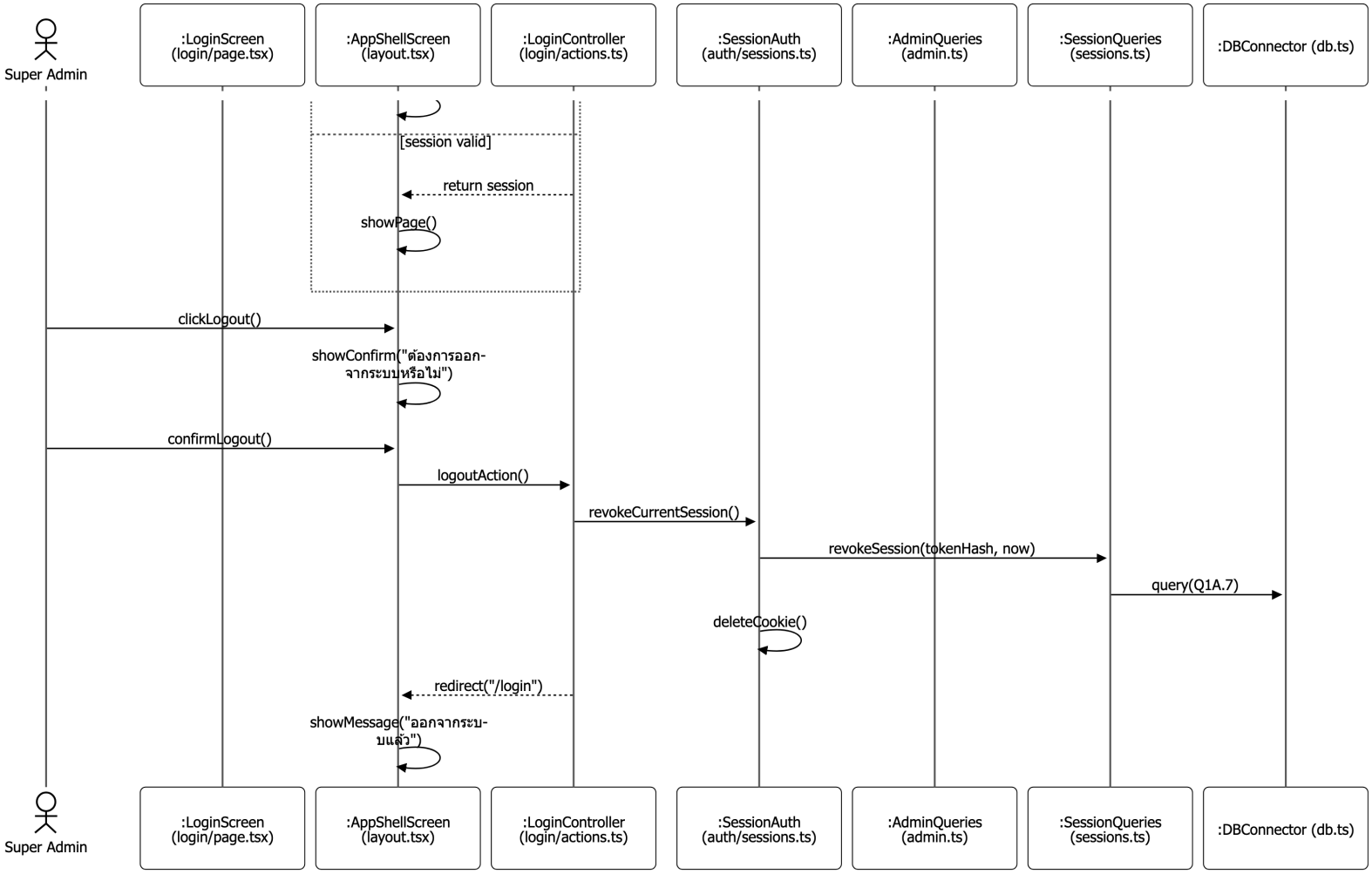
รูปที่ 2-7 Sequence Diagram ของ Use Case Description ที่ 1A (1/4)



รูปที่ 2-8 Sequence Diagram ของ Use Case Description ที่ 1A (2/4)

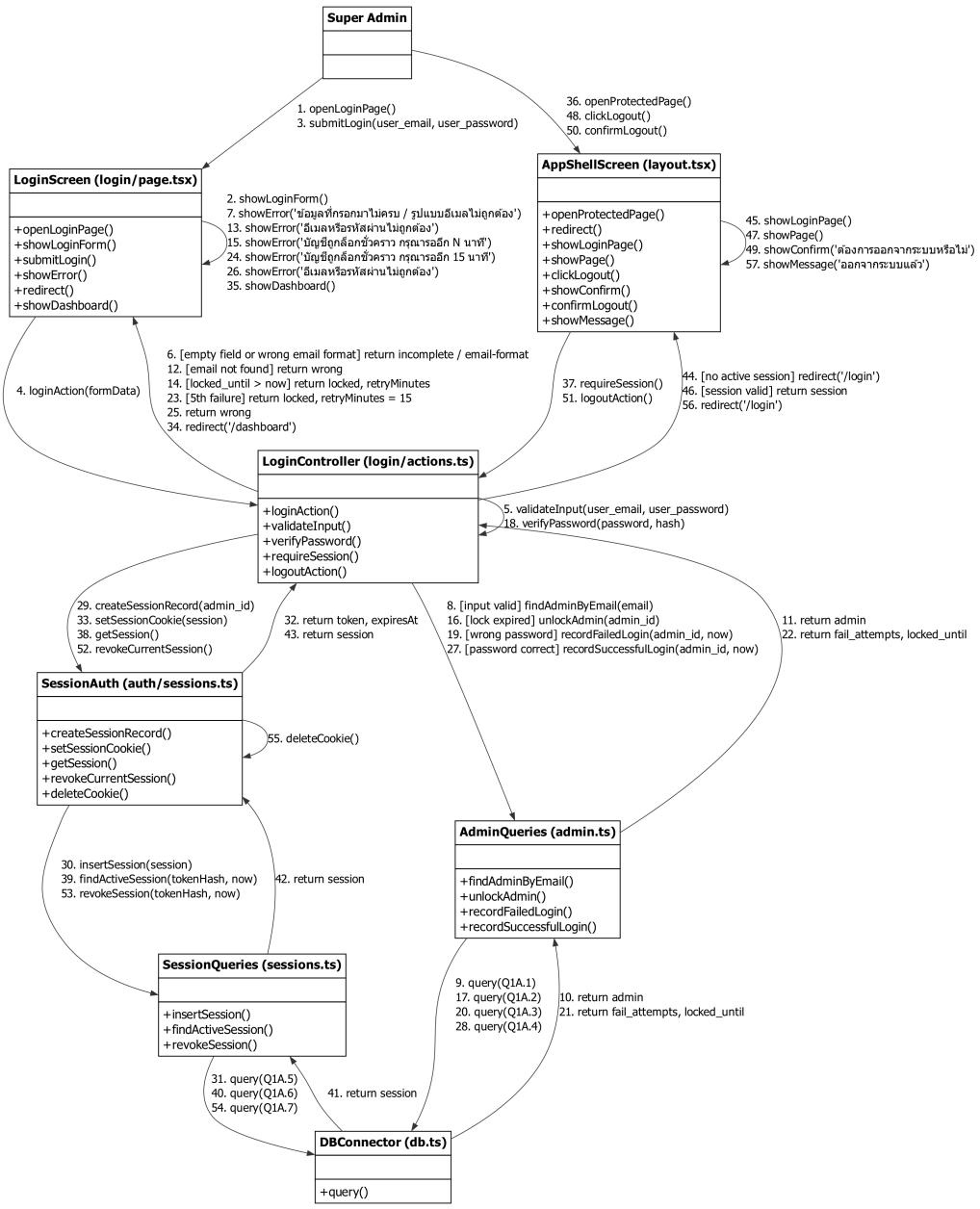


รูปที่ 2-9 Sequence Diagram ของ Use Case Description ที่ 1A (3/4)



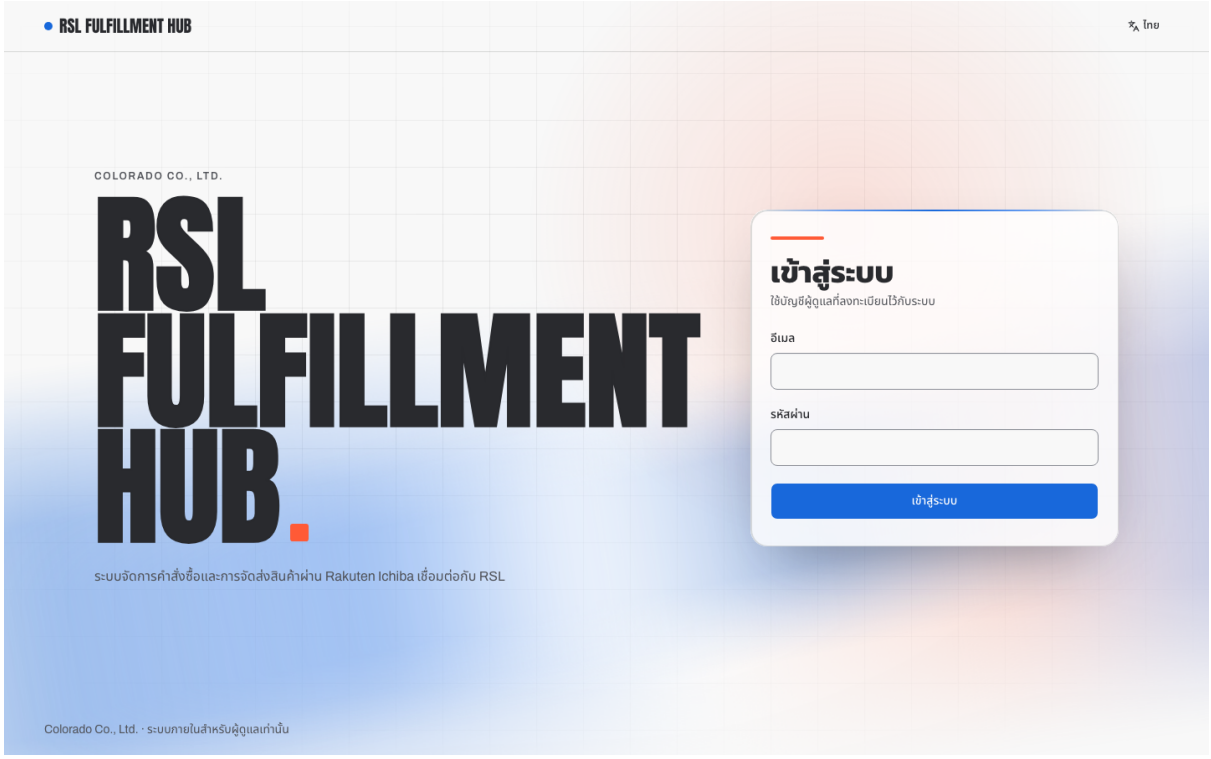
รูปที่ 2-10 Sequence Diagram ของ Use Case Description ที่ 1A (4/4)

2.13.3 Collaboration Diagram ของ Use Case Description ที่ 1A



รูปที่ 2-11 Collaboration Diagram ของ Use Case Description ที่ 1A

2.13.4 User Interface ของ Use Case Description ที่ 1A



รูปที่ 2-12 User Interface ของ Use Case Description ที่ 1A (LoginScreen)

2.13.5 Use Case Description ที่ 2A: ตรวจสอบคำสั่งซื้อ

Goal: ให้ Super Admin ตรวจสอบความถูกต้องของ Order ที่ดึงเข้ามา ก่อนส่งเข้าสู่กระบวนการจับคู่กฎ SKU

Pre-Condition: Super Admin เข้าสู่ระบบแล้ว และ Order อยู่ในสถานะ "รอตรวจสอบคำสั่งซื้อ"

Post-Condition: Order ผ่านการตรวจสอบ บันทึก `verified_at` และเปลี่ยนสถานะเป็น "รอจับคู่กฎ SKU" จากนั้น `executeWorkflow` ทำ UC 2S ต่อทันทีใน transaction เดียวกัน (หรือ Order ไปที่ "รอดำเนินการด้วยตนเอง" / "ยกเลิกแล้ว" ตามทางเลือก)

Actor: Super Admin

ตารางที่ 2-6 Use Case Description ที่ 2A: ตรวจสอบคำสั่งซื้อ

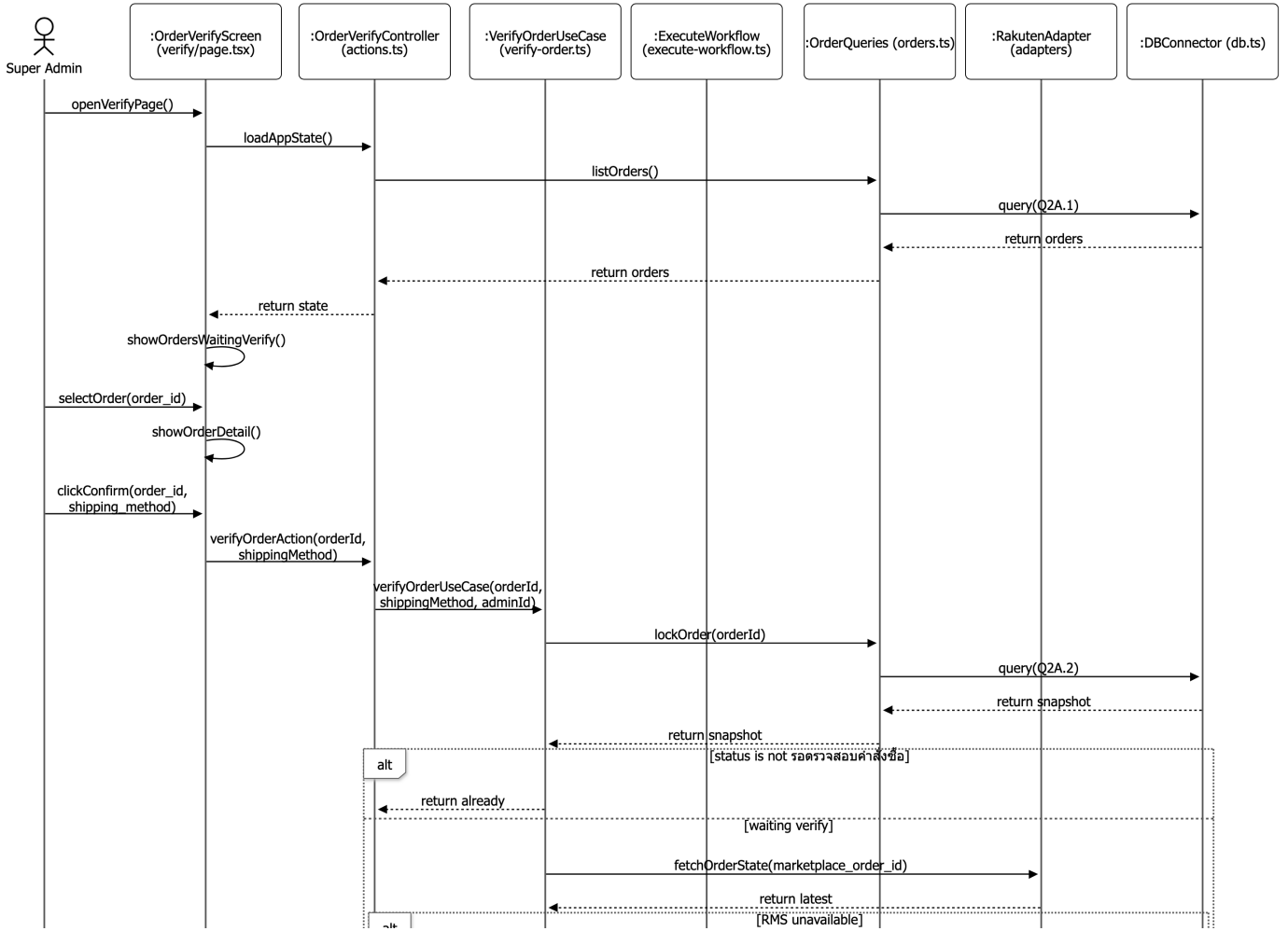
ลำดับ	User Action	System Response
1		แสดงหน้าตรวจสอบคำสั่งซื้อ (`/orders/verify`) รายการ Order ที่อยู่ในสถานะ "รอตรวจสอบคำสั่งซื้อ" พร้อมช่องค้นหาตาม Order ID, SKU และช่องทางขาย (ข้อมูลมาจาก query รวมของ Order ที่ layout โหลด แล้ว

ลำดับ	User Action	System Response
		กรองสถานะที่หน้าจอ) — Q2A.1: SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_date, o.shipping_address, o.shipping_method, o.order_status, o.rsl_no, o.tracking_number, o.carrier_name, o.dispatched_at, o.delivery_status, o.tracking_notified, o.tracking_notified_at, o.cancel_reason, o.cancelled_at, o.verified_at, o.manual_reason, o.customer_cancel_requested, o.edit_log, o.reorder_decision, o.decided_qty, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.deducted_at, item.sku_id, sku.sku, sku.product_name, sku.selling_price, o.reorder_id, label.label_template, label.parcel_total, label.label_printed_at, sync.sync_status, notification.notify_log FROM orders o LEFT JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN stock stock_row ON stock_row.sku_id = sku.sku_id LEFT JOIN rakuten_stock_sync_states sync ON sync.stock_id = stock_row.stock_id LEFT JOIN LATERAL (SELECT MAX(l.label_template) AS label_template, MAX(l.parcel_total) AS parcel_total, MAX(l.printed_at) AS label_printed_at FROM labels l WHERE l.order_id = o.order_id) label ON TRUE LEFT JOIN LATERAL (SELECT jsonb_agg(jsonb_build_object('by', COALESCE(t.admin_id, 'system'), 'at', t.sent_at) ORDER BY t.sent_at) AS notify_log FROM tracking_notification_logs t WHERE t.order_id = o.order_id) notification ON TRUE ORDER BY o.order_date DESC, o.order_id DESC
2	เลือก Order ที่ต้องการตรวจสอบ	แสดงรายละเอียด Order (เลขคำสั่งซื้อใน Rakuten, รหัสสินค้าใน Rakuten, Variation, จำนวน, ที่อยู่, วิธีจัดส่ง) และป้าย "ลูกค้าขอยกเลิก" หาก customer_cancel_requested เป็นจริง ให้ Super Admin ตรวจทาน และแก้ shipping_method ได้ก่อนยืนยัน
3	กด "ยืนยันคำสั่งซื้อ"	ระบบเรียก Server Action verifyOrderAction ⟶ verifyOrderUseCase 1. ล็อกและตรวจสถานะ Order — Q2A.2: SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id

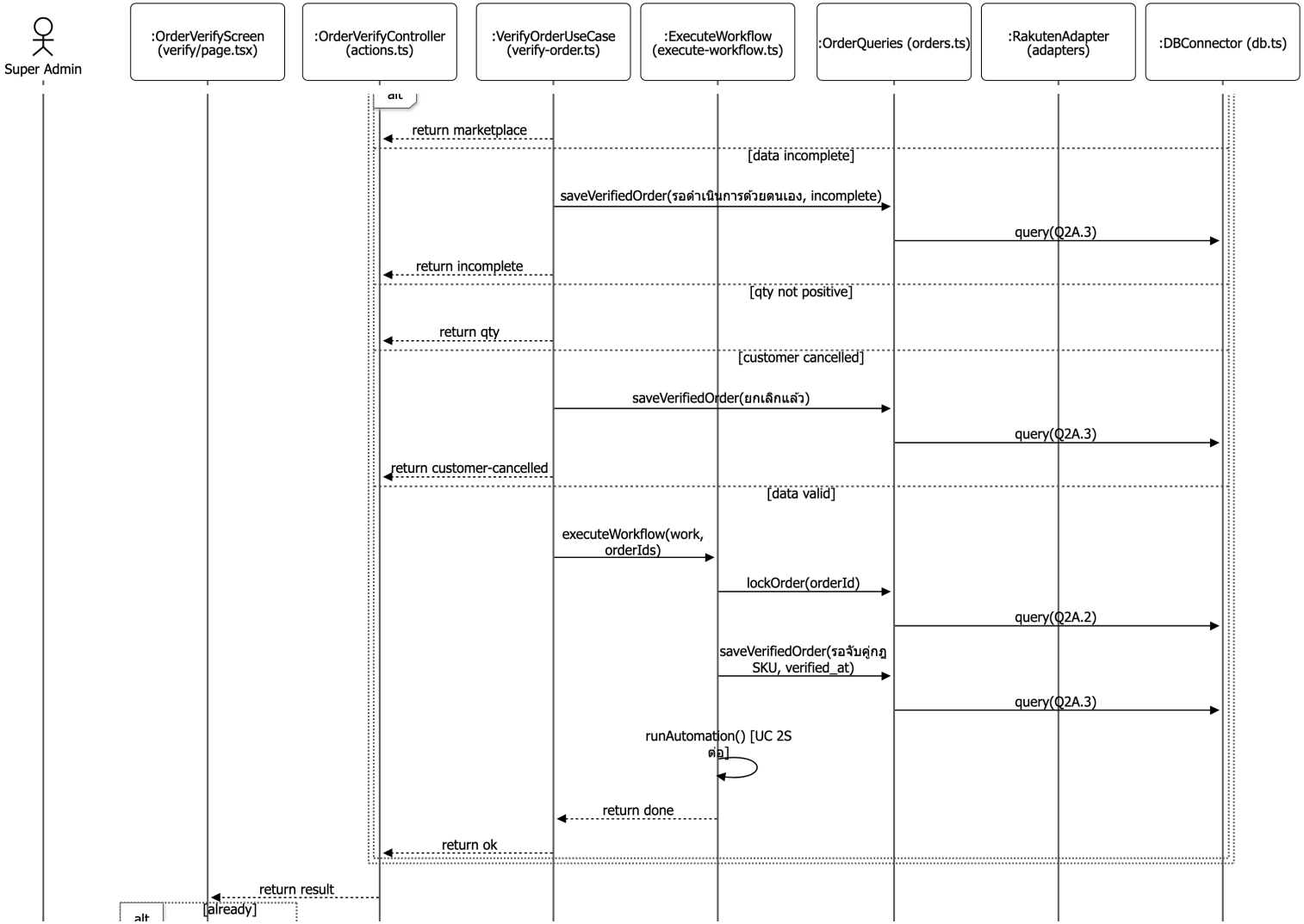
ลำดับ	User Action	System Response
		`= o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o` — หาก Order ไม่อยู่ในสถานะ "รอตรวจสอบคำสั่งซื้อ" แล้ว จะแจ้งเตือนว่า "Order นี้ผ่านการตรวจสอบไปแล้ว" และไม่ดำเนินการซ้ำ **2. ดึงข้อมูลล่าสุดจาก Rakuten RMS** (`RakutenAdapter.fetchOrderState`) มาแทนค่าที่เปลี่ยนไป **3. ตรวจสอบความครบถ้วน** — `channel_sku`, `shipping_address` และ `shipping_method` ต้องไม่ว่าง และ `qty` ต้องเป็นจำนวนเต็มบวก **4. บันทึกผลการตรวจสอบ** — ล็อก Order ซ้ำด้วย Q2A.2 แล้วบันทึกข้อมูลล่าสุดลง `order_items` และ `orders` พร้อมเปลี่ยนสถานะเป็น "รอจับคู่กฎ SKU", `verified_at` = เวลาปัจจุบัน และหากแก้ `shipping_method` จะเพิ่มรายการใน `edit_log` (ผู้แก้ไข/เวลา) — Q2A.3: `WITH updated_item AS (UPDATE order_items SET channel_sku = $2, variation = $3, qty = $4 WHERE order_id = $1) UPDATE orders SET shipping_address = $5, shipping_method = $6, order_status = $7, manual_reason = $8, customer_cancel_requested = $9, verified_at = $10, edit_log = CASE WHEN $11::boolean THEN edit_log || jsonb_build_array(jsonb_build_object('field', 'shipping_method', 'by', $12::text, 'at', $10::timestamptz)) ELSE edit_log END, cancel_reason = $13, cancelled_at = $14, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1` **5. ทำงานอัตโนมัติต่อ** — ขั้นตอน 4 ทำใน `executeWorkflow` ซึ่งเรียก UC 2S (จับคู่กฎ SKU) และ UC ถัดไปต่อทันทีใน transaction เดียวกัน แล้วแสดงข้อความ "ยืนยันคำสั่งซื้อสำเร็จ"
ทางเลือก #1	ข้อมูลคำสั่งซื้อไม่ครบ (`channel_sku`, ที่อยู่ หรือวิธีจัดส่งว่าง)	แจ้งเตือนว่า "ข้อมูลคำสั่งซื้อไม่ครบถ้วน กรุณาตรวจสอบ" ระบบล็อก Order (Q2A.2) และบันทึกด้วย Q2A.3 ให้สถานะเป็น "รอดำเนินการด้วยตนเอง" `manual_reason` = `incomplete` ให้ Super Admin แก้ข้อมูลใน Rakuten RMS แล้วนำเข้าใหม่ หรือยกเลิก Order (UC 7A)
ทางเลือก #2	จำนวนสินค้า (`qty`) ไม่ใช่จำนวนเต็มบวก	แจ้งเตือนว่า "จำนวนสินค้าต้องมากกว่า 0" และไม่เปลี่ยนแปลงข้อมูล
ทางเลือก #3	ลูกค้ายกเลิก Order ผ่าน Rakuten ก่อน Super Admin กดยืนยัน (`customer_cancel_requested` หรือข้อมูลล่าสุดจาก RMS)	แจ้งเตือนว่า "Order นี้ถูกยกเลิกจากฝั่งลูกค้าแล้ว" ระบบบันทึกด้วย Q2A.3 ให้สถานะเป็น "ยกเลิกแล้ว", `cancel_reason` = "ลูกค้ายกเลิกผ่าน Rakuten" และ `cancelled_at` = เวลาปัจจุบัน โดยไม่ส่งเข้ากระบวนการต่อ

ลำดับ	User Action	System Response
ทางเลือก #4	ดึงข้อมูลล่าสุดจาก Rakuten RMS ไม่ได้	แจ้งเตือนว่า "ไม่สามารถตรวจสอบข้อมูลล่าสุดจาก Marketplace ได้ กรุณาลองใหม่ภายหลัง" และคงสถานะ Order ไว้เท่าเดิม ให้ Super Admin กดยืนยันใหม่ภายหลัง

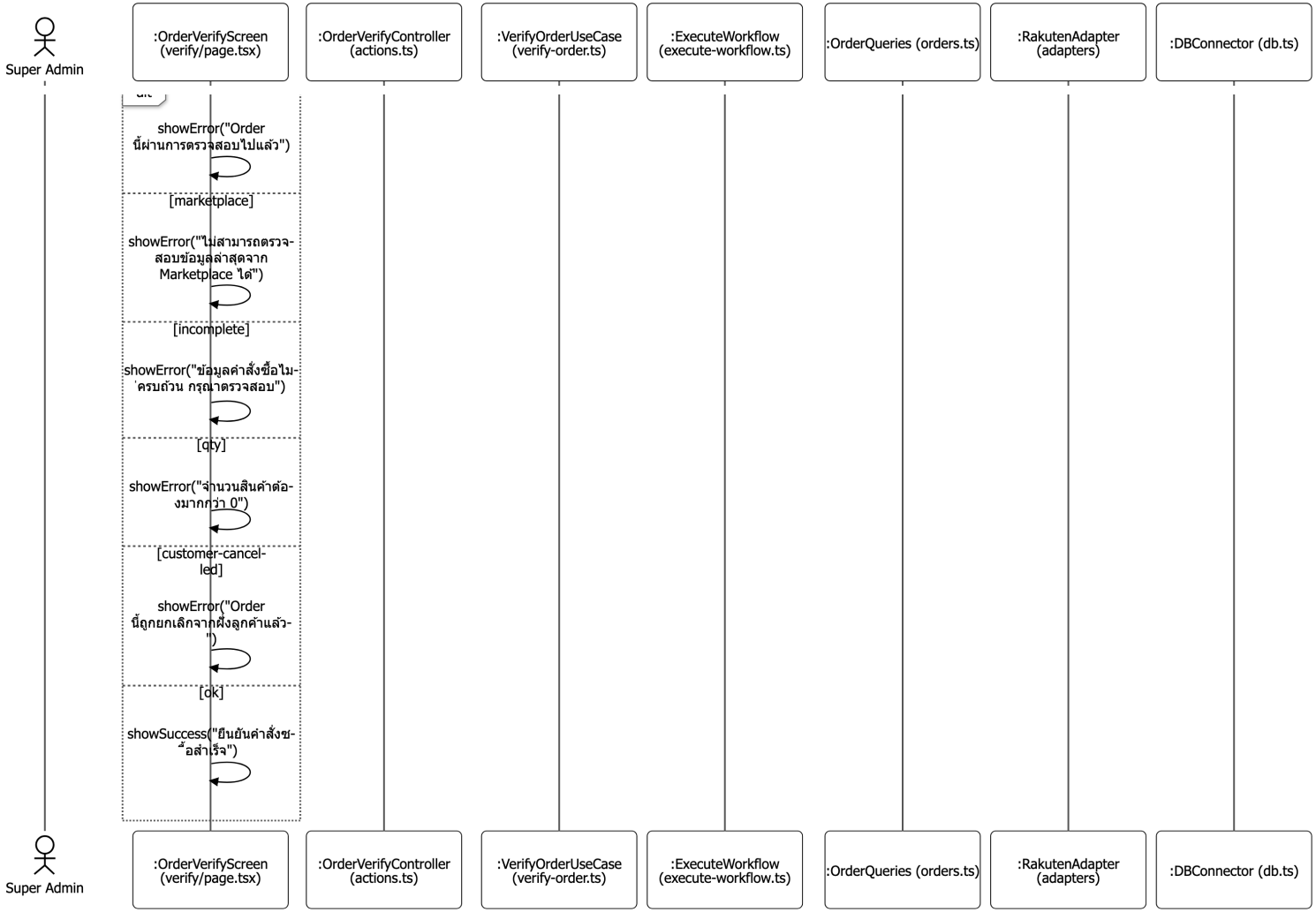
2.13.6 Sequence Diagram ของ Use Case Description ที่ 2A



รูปที่ 2-13 Sequence Diagram ของ Use Case Description ที่ 2A (1/3)

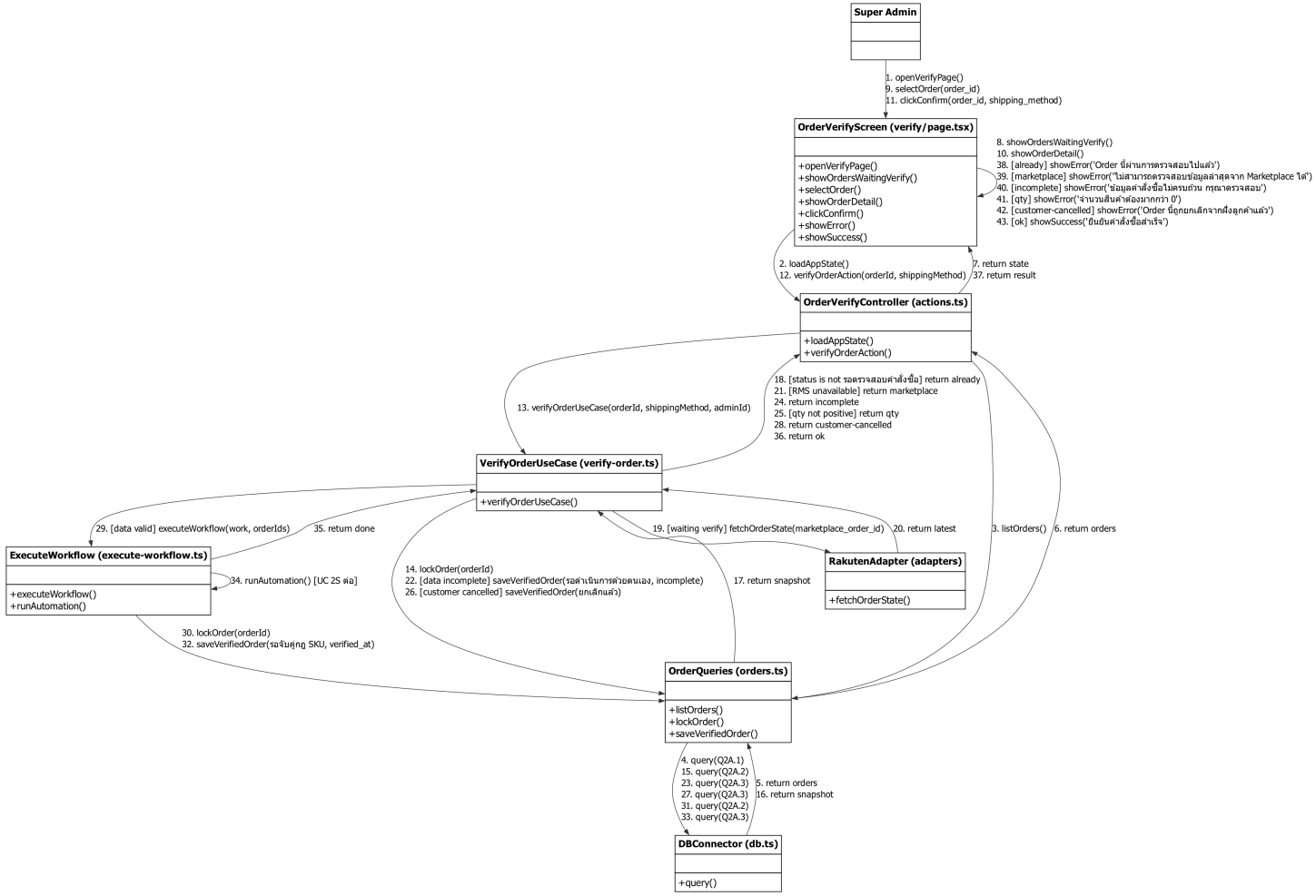


รูปที่ 2-14 Sequence Diagram ของ Use Case Description ที่ 2A (2/3)



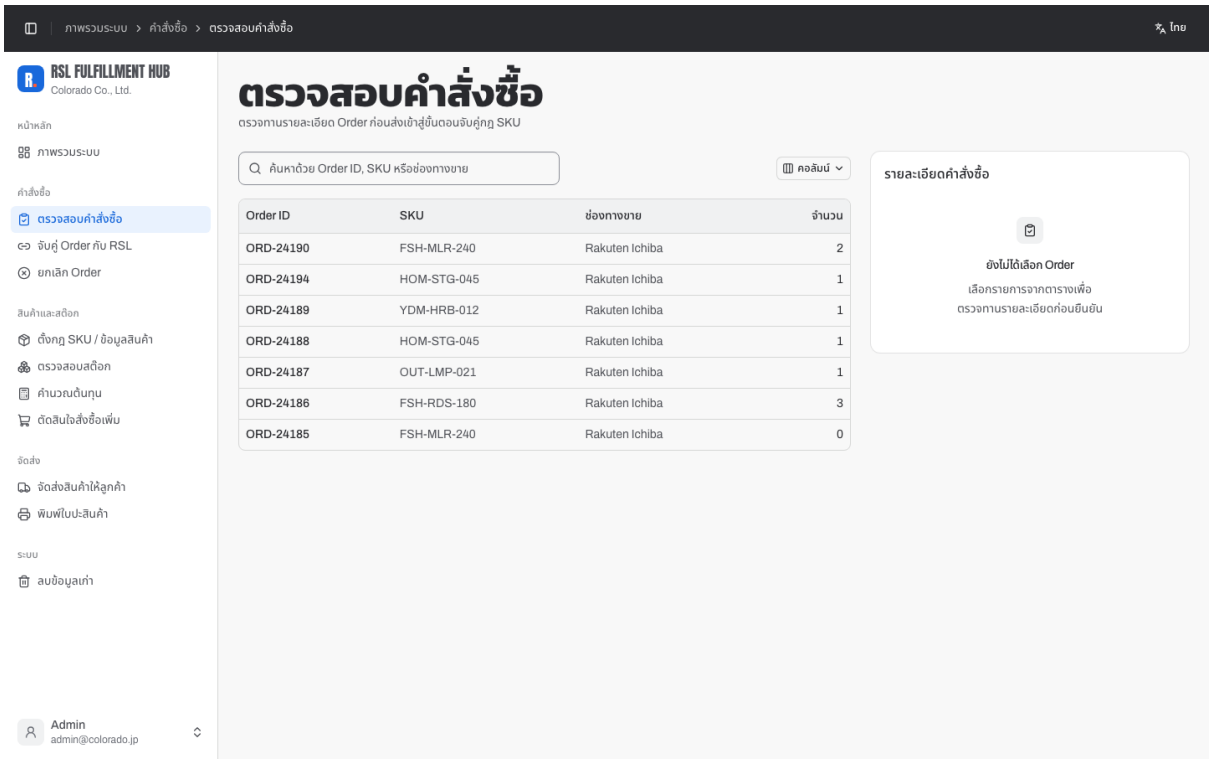
รูปที่ 2-15 Sequence Diagram ของ Use Case Description ที่ 2A (3/3)

2.13.7 Collaboration Diagram ของ Use Case Description ที่ 2A



รูปที่ 2-16 Collaboration Diagram ของ Use Case Description ที่ 2A

2.13.8 User Interface ของ Use Case Description ที่ 2A



รูปที่ 2-17 User Interface ของ Use Case Description ที่ 2A (OrderVerifyScreen)

2.13.9 Use Case Description ที่ 3A: ตั้งกฎ SKU

Goal: ให้ Super Admin เพิ่ม แก้ไข เปิด/ปิดการขาย และลบข้อมูลสินค้า (SKU) รวมถึงกฎ SKU คือ รหัสสินค้าใน Rakuten (`channel_sku`) ที่ใช้จับคู่ Order เพื่อให้ Order, สต๊อก และต้นทุนอ้างอิง SKU เดียวกัน

Pre-Condition: Super Admin เข้าสู่ระบบแล้ว และอยู่ที่หน้าจัดการสินค้า (`/products`)

Post-Condition: ข้อมูลสินค้าและกฎ SKU ถูกบันทึกในตาราง `skus` (สินค้าใหม่มีแถวใน `stock` เริ่มที่ 0) และ Order ที่ค้างอยู่ที่ "รอดำเนินการด้วยตนเอง" เพราะ SKU นี้ถูกส่งกลับเข้ากระบวนการอัตโนมัติ (Order ที่ค้างเพราะกฎ SKU ผ่าน 2S และ 3S ต่อทันทีใน transaction เดียวกัน)

Actor: Super Admin

ตารางที่ 2-7 Use Case Description ที่ 3A: ตั้งกฎ SKU

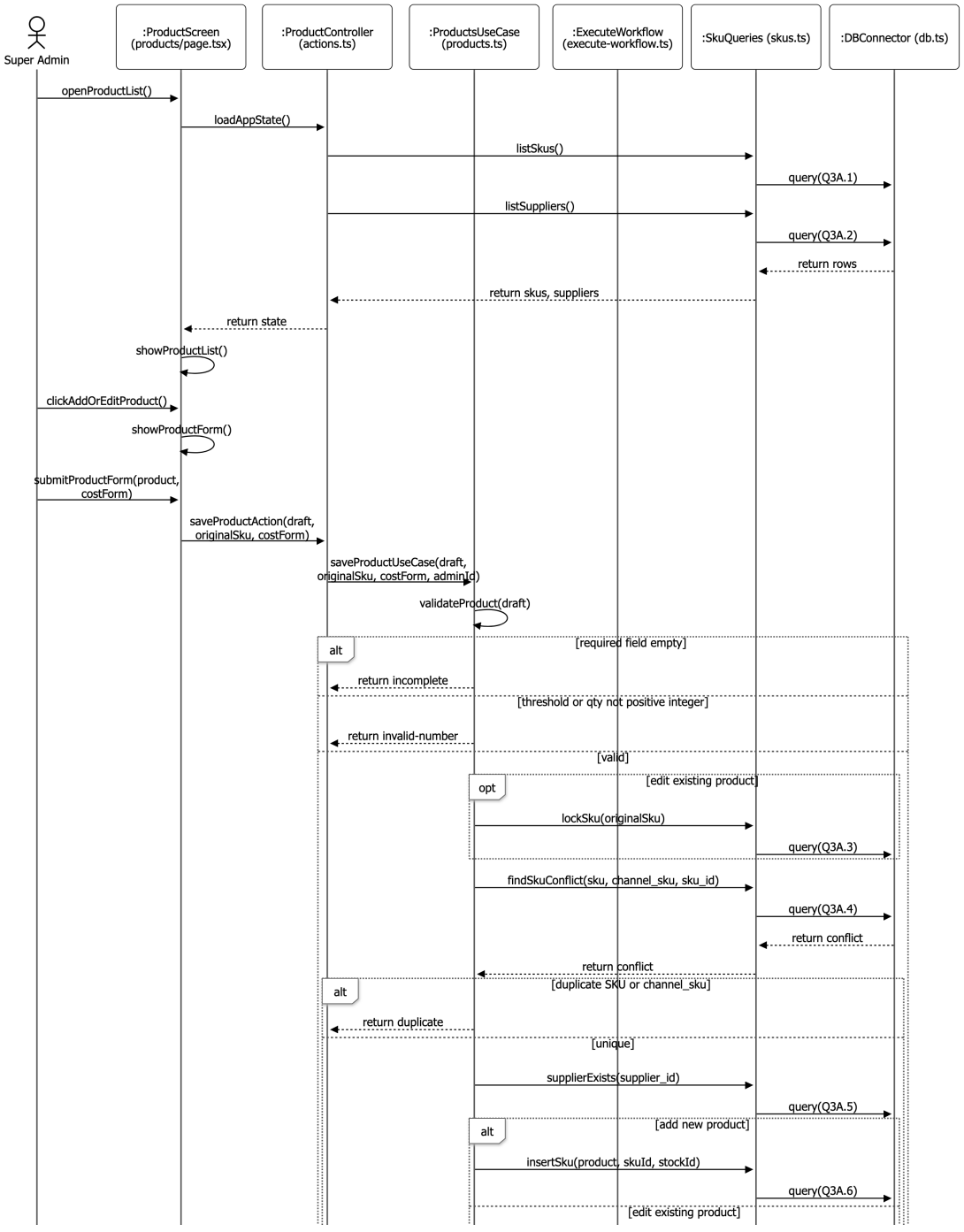
ลำดับ	User Action	System Response
1		แสดงรายการสินค้าทั้งหมด — Q3A.1: `SELECT sku, product_name, variation, sales_channel,`

ลำดับ	User Action	System Response
		`channel_sku, supplier_id, reorder_threshold, reorder_qty, selling_price, sku_status FROM skus ORDER BY sku` พร้อมรายชื่อ Supplier สำหรับเลือกในฟอร์ม — **Q3A.2:** `SELECT supplier_id, supplier_name, contact_info, lead_time_days FROM suppliers ORDER BY supplier_name, supplier_id` มีปุ่ม "เพิ่มสินค้าใหม่", "นำเข้าจากไฟล์ CSV" และตัวเลือกแก้ไข / เปิด-ปิดการขาย / ลบ
2	กด "เพิ่มสินค้าใหม่" หรือเลือกสินค้าที่ต้องการแก้ไข	แสดงฟอร์มข้อมูลสินค้า ได้แก่ SKU, ชื่อสินค้า, Variation, รหัสสินค้าใน Rakuten (`channel_sku` = กฎ SKU), Supplier, Reorder Threshold, Reorder Quantity, ราคาขาย, สถานะเปิดขาย และค่าเริ่มต้นของต้นทุน
3	กรอกข้อมูลแล้วกด "บันทึก"	ระบบเรียก Server Action `saveProductAction` ⟶ `saveProductUseCase` (ทำทั้งหมดใน transaction เดียว) **1. ตรวจสอบค่าที่กรอก** — SKU, ชื่อสินค้า, Variation และ `channel_sku` ต้องไม่ว่าง หากว่างจะแจ้งเตือนว่า "กรุณากรอกข้อมูลให้ครบถ้วน"; Reorder Threshold และ Reorder Quantity ต้องเป็นจำนวนเต็มบวก และราคาขาย (ถ้ามี) ต้องไม่ติดลบ หากไม่ถูกต้องจะแจ้งเตือนว่า "Reorder Threshold และ Reorder Quantity ต้องเป็นตัวเลขจำนวนเต็มบวก" **2. ล็อกสินค้าเดิม** (กรณีแก้ไข) — **Q3A.3:** `SELECT sku_id, sku, channel_sku FROM skus WHERE sku = $1 FOR UPDATE` **3. ตรวจ SKU / `channel_sku` ซ้ำ** — **Q3A.4:** `SELECT sku_id, sku, channel_sku FROM skus WHERE (sku = $1 OR channel_sku = $2) AND ($3::text IS NULL OR sku_id <> $3) LIMIT 1` หากซ้ำกับสินค้าอื่นจะแจ้งเตือนว่า "SKU นี้มีอยู่ในระบบแล้ว" **4. ตรวจ Supplier** (ถ้าเลือก) — **Q3A.5:** `SELECT EXISTS ( SELECT 1 FROM suppliers WHERE supplier_id = $1 ) AS found` หากไม่พบจะแจ้งเตือนว่า "กรุณากรอกข้อมูลให้ครบถ้วน" **5. บันทึกข้อมูลสินค้า** — กรณีเพิ่มสินค้าใหม่ สร้างแถวใน `skus` และแถว `stock` เริ่มต้นที่ 0 — **Q3A.6:** `WITH inserted_sku AS (INSERT INTO skus (sku_id, sku, product_name, variation, sales_channel, channel_sku, selling_price, reorder_threshold, reorder_qty, sku_status, supplier_id) VALUES ($1, $2, $3, $4, 'Rakuten Ichiba', $5, $6, $7, $8, $9, $10) RETURNING sku_id) INSERT INTO stock (stock_id, in_house_qty, rsl_qty, updated_at, sku_id) SELECT $11, 0, 0, CURRENT_TIMESTAMP, sku_id FROM inserted_sku` กรณีแก้ไขสินค้าเดิม (คง `sku_id` เดิมไว้) — **Q3A.7:** `UPDATE skus SET sku = $2, product_name = $3, variation = $4, channel_sku = $5, selling_price = $6, reorder_threshold = $7, reorder_qty = $8, sku_status = $9, supplier_id = $10 WHERE sku_id = $1`

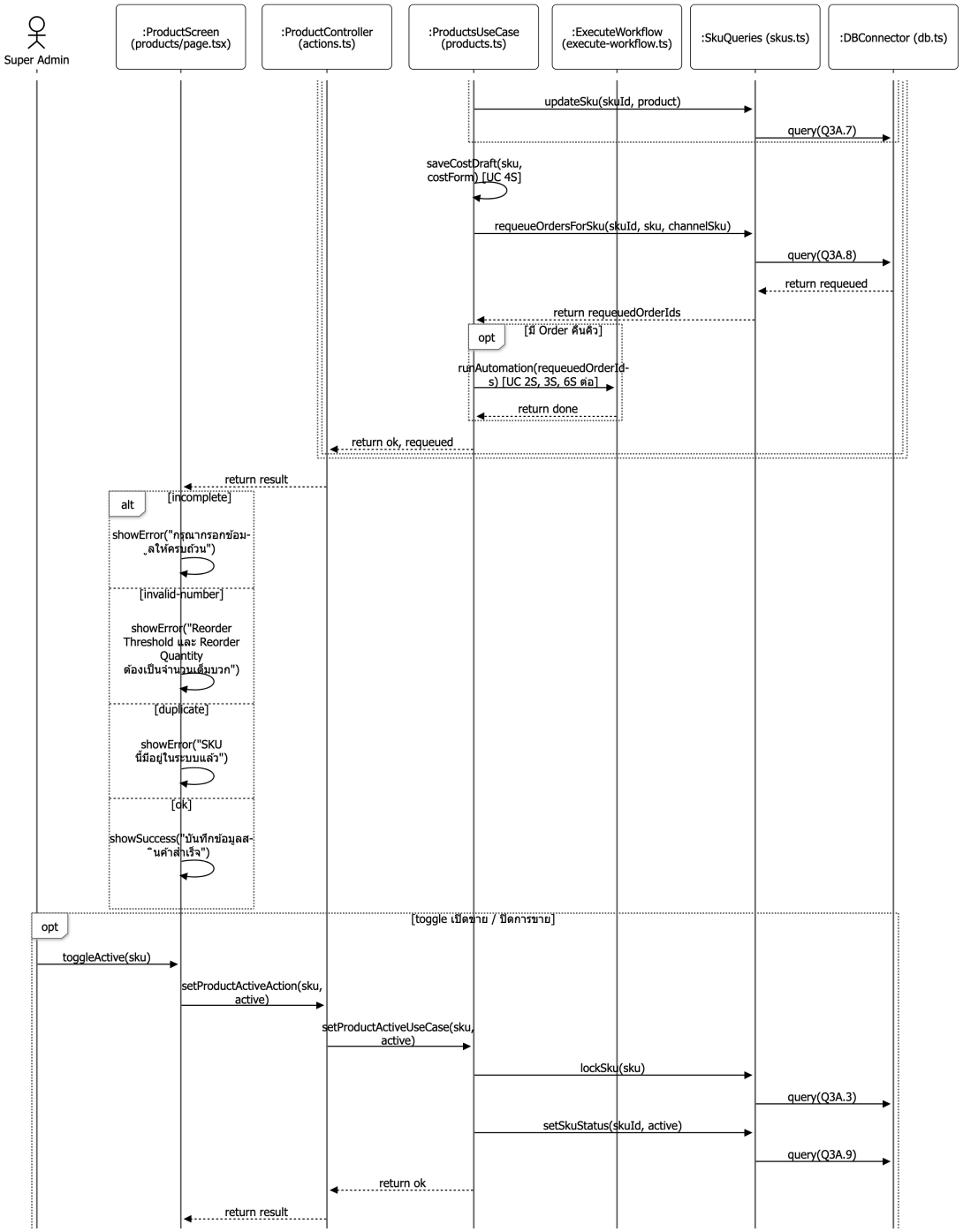
ลำดับ	User Action	System Response
		6. บันทึกค่าเริ่มต้นของต้นทุน — เรียก `saveCostDraft` ตามขั้นตอนของ UC 4S **7. นำ Order ที่ค้างกลับเข้ากระบวนการ** — Order ที่ "รอดำเนินการด้วยตนเอง" ด้วย `manual_reason` = `sku-rule` กลับไป "รอจับคู่กฎ SKU", `sku-unregistered` ผูก `sku_id` แล้วกลับไป "รอตรวจสอบคำสั่งซื้อ" และ `price` (เมื่อมีราคาขายแล้ว) กลับไป "รอ Admin ตัดสินใจสั่งซื้อ" — Q3A.8: `WITH registered_items AS (UPDATE order_items SET sku_id = $2 WHERE sku_id IS NULL AND (channel_sku = $1 OR channel_sku = $4) RETURNING order_id) UPDATE orders o SET order_status = CASE WHEN o.manual_reason = 'sku-rule' THEN 'รอจับคู่กฎ SKU' WHEN o.manual_reason = 'sku-unregistered' THEN 'รอตรวจสอบคำสั่งซื้อ' ELSE 'รอ Admin ตัดสินใจสั่งซื้อ' END, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP FROM order_items item WHERE item.order_id = o.order_id AND o.order_status = 'รอดำเนินการด้วยตนเอง' AND ((o.manual_reason = 'sku-rule' AND item.channel_sku = $1) OR (o.manual_reason = 'sku-unregistered' AND item.order_id IN (SELECT order_id FROM registered_items)) OR (o.manual_reason = 'price' AND item.sku_id = $2 AND $3::boolean)) RETURNING o.order_id` **8. ส่ง Order ที่คืนคิวเข้ากระบวนการอัตโนมัติทันที** — `saveProductUseCase` ทำงานใน `executeWorkflow` จึงรัน 2S, 3S (และ 6S สำหรับ Order ที่ส่งจากคลังบริษัท) ต่อกับ Order ที่ได้จาก `RETURNING o.order_id` ใน transaction เดียวกัน Order ที่กลับไป "รอตรวจสอบคำสั่งซื้อ" (รอ 2A) หรือ "รอ Admin ตัดสินใจสั่งซื้อ" (รอ 5A) รอ Super Admin ตามเดิม แสดงข้อความ "บันทึกข้อมูลสินค้าสำเร็จ" และ "นำ Order ที่ค้างอยู่ n รายการกลับเข้าสู่กระบวนการอัตโนมัติ"
ทางเลือก #1	กดเปลี่ยนสถานะ "เปิดขาย" / "ปิดการขาย"	เรียก `setProductActiveAction` ล็อกสินค้าด้วย Q3A.3 แล้วเปลี่ยน `sku_status` เป็น `active` / `inactive` — Q3A.9: `UPDATE skus SET sku_status = $2 WHERE sku_id = $1` (สินค้าที่ปิดการขายจะไม่ถูกใช้จับคู่ใน UC 2S)
ทางเลือก #2	กด "ลบสินค้า"	เรียก `deleteProductAction` ล็อกสินค้าด้วย Q3A.3 แล้วตรวจว่ายังมี Order หรือคำสั่งซื้อเติมสต๊อกอ้างอิงอยู่หรือไม่ — Q3A.10: `SELECT EXISTS ( SELECT 1 FROM order_items WHERE sku_id = $1 UNION ALL SELECT 1 FROM reorders WHERE sku_id = $1 ) AS has_related` — หากมี จะแจ้งเตือนว่า "ไม่สามารถลบสินค้านี้ได้ เนื่องจากยังมีรายการที่เกี่ยวข้องอยู่" และเสนอปุ่ม "เปลี่ยนเป็น ปิดการขาย แทน" — หากไม่มี ลบประวัติต้นทุน ต้นทุน สต๊อก และสินค้า — Q3A.11: `WITH removed_history AS ( DELETE FROM cost_history`

ลำดับ	User Action	System Response
		`WHERE sku_id = $1 ), removed_costs AS ( DELETE FROM costs WHERE sku_id = $1 ), removed_stock AS ( DELETE FROM stock WHERE sku_id = $1 ) DELETE FROM skus WHERE sku_id = $1` แล้วแจ้งว่า "ลบสินค้า [SKU] แล้ว"
ทางเลือก #3	นำเข้าข้อมูลสินค้าจากไฟล์ CSV (หัวตาราง `sku,product_name,variation,channel_sku,supplier_id,reorder_threshold,reorder_qty,selling_price`)	เรียก `importProductsAction` ⟶ `importProductsUseCase` ตรวจและบันทึกทีละแถวด้วยขั้นตอนเดียวกับขั้นที่ 3 (Q3A.3–Q3A.8 แถวที่มี SKU อยู่แล้วถือเป็นการแก้ไข) ทั้งไฟล์อยู่ใน transaction เดียว (`executeWorkflow`) และรัน 2S/3S/6S กับ Order ที่คืนคิวต่อทันที เหมือนขั้นตอนที่ 3 แล้วแสดงผลรายแถวพร้อมเหตุผลของแถวที่นำเข้าไม่สำเร็จ

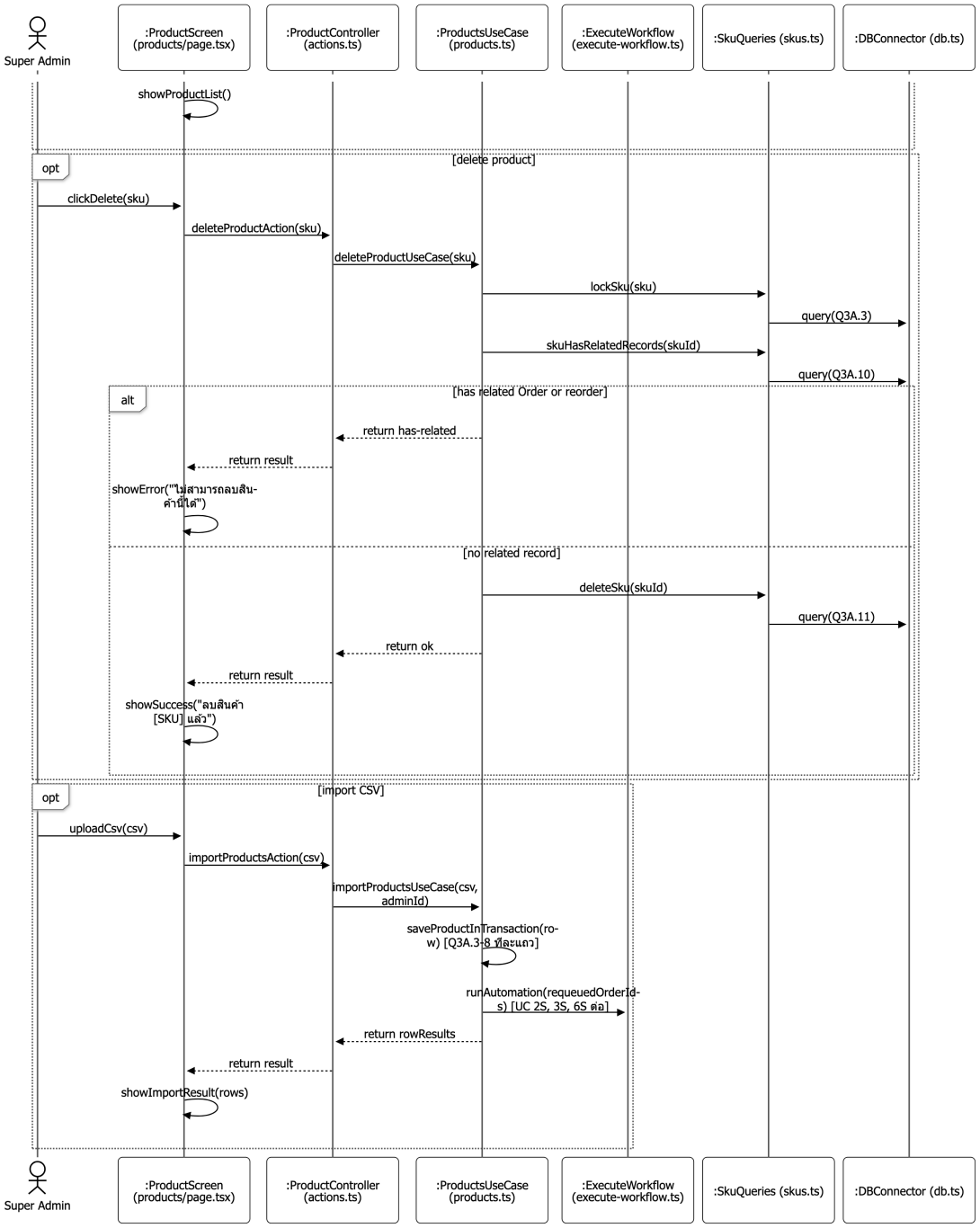
2.13.10 Sequence Diagram ของ Use Case Description ที่ 3A



รูปที่ 2-18 Sequence Diagram ของ Use Case Description ที่ 3A (1/3)

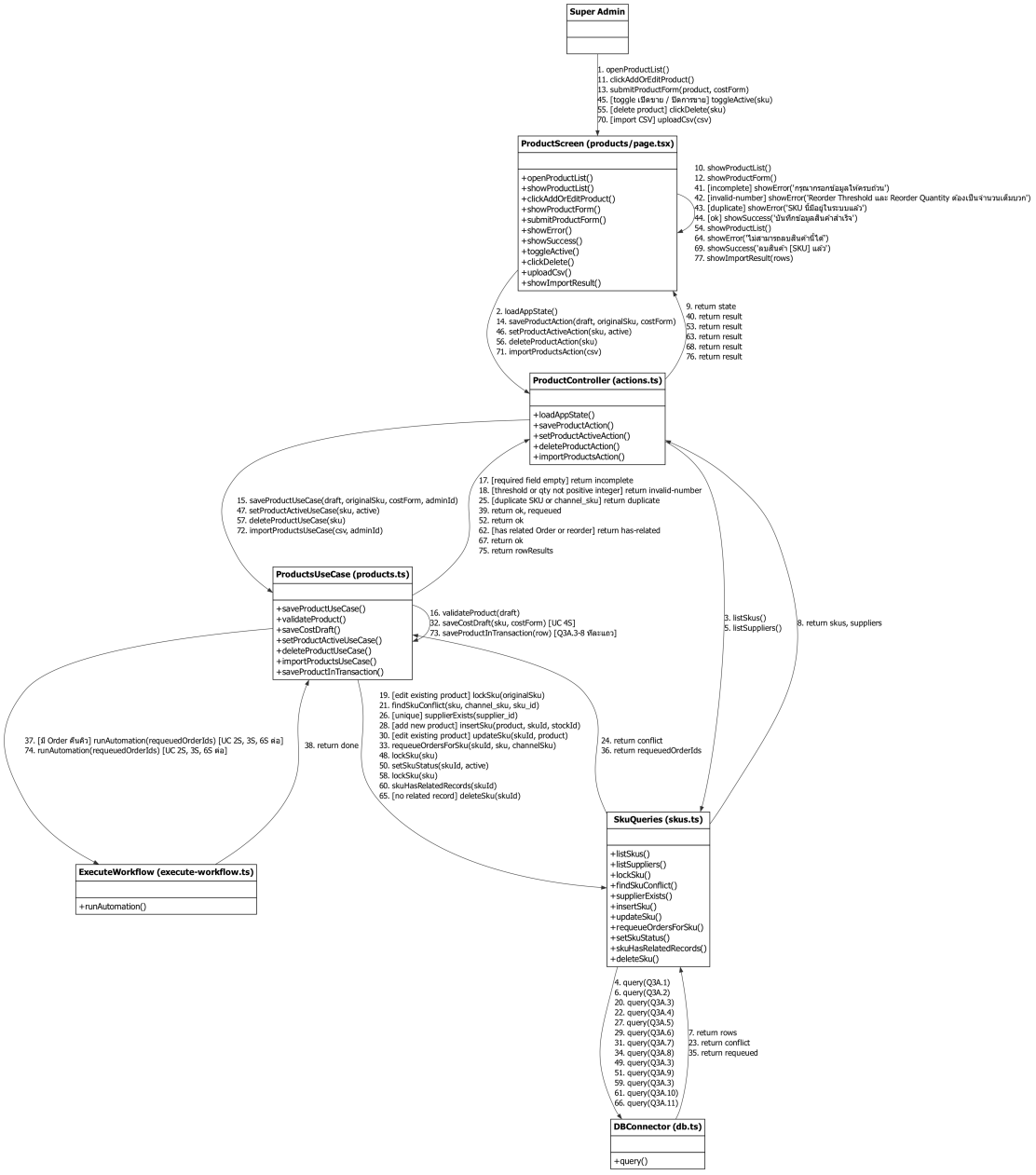


รูปที่ 2-19 Sequence Diagram ของ Use Case Description ที่ 3A (2/3)



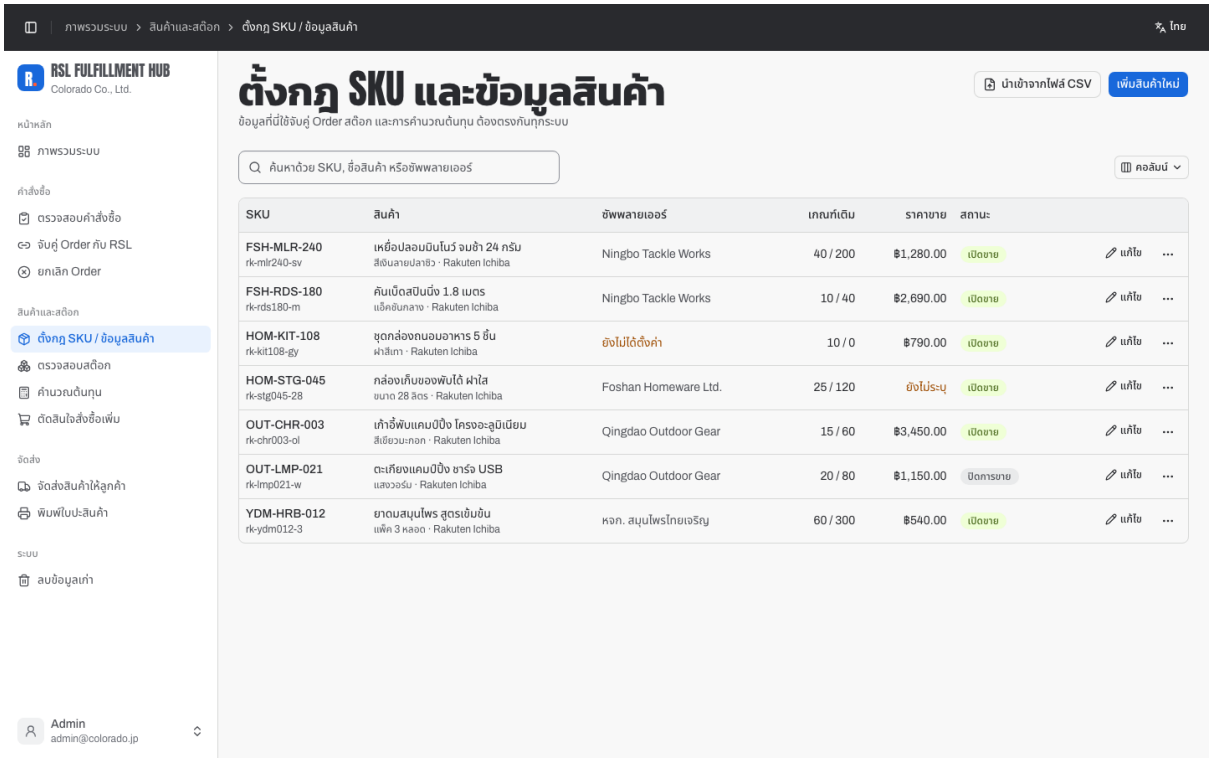
รูปที่ 2-20 Sequence Diagram ของ Use Case Description ที่ 3A (3/3)

2.13.11 Collaboration Diagram ของ Use Case Description ที่ 3A



รูปที่ 2-21 Collaboration Diagram ของ Use Case Description ที่ 3A

2.13.12 User Interface ของ Use Case Description ที่ 3A



รูปที่ 2-22 User Interface ของ Use Case Description ที่ 3A (ProductScreen)

2.13.13 Use Case Description ที่ 4A: ดึง Order เข้าระบบ

Goal: ให้ Super Admin กดดึง Order ใหม่จาก Rakuten Ichiba (ผ่าน Rakuten RMS) เข้าสู่ระบบ เพื่อเป็นจุดเริ่มต้นของกระบวนการจัดส่ง

Pre-Condition: Super Admin เข้าสู่ระบบแล้วและอยู่ที่หน้าภาพรวมระบบ (`/dashboard`) และระบบเชื่อมต่อ Rakuten RMS ผ่าน `RakutenAdapter` ได้

Post-Condition: Order ใหม่ถูกบันทึกลง `orders` และ `order_items` สถานะ "รอตรวจสอบคำสั่งซื้อ" (ส่งต่อให้ 2A) หรือ "รอดำเนินการด้วยตนเอง" หากยังไม่มีกฎ SKU และ Order ที่ลูกค้าขอยกเลิกถูกตั้ง `customer_cancel_requested`

Actor: Super Admin (Marketplace เป็นแหล่งข้อมูล Order)

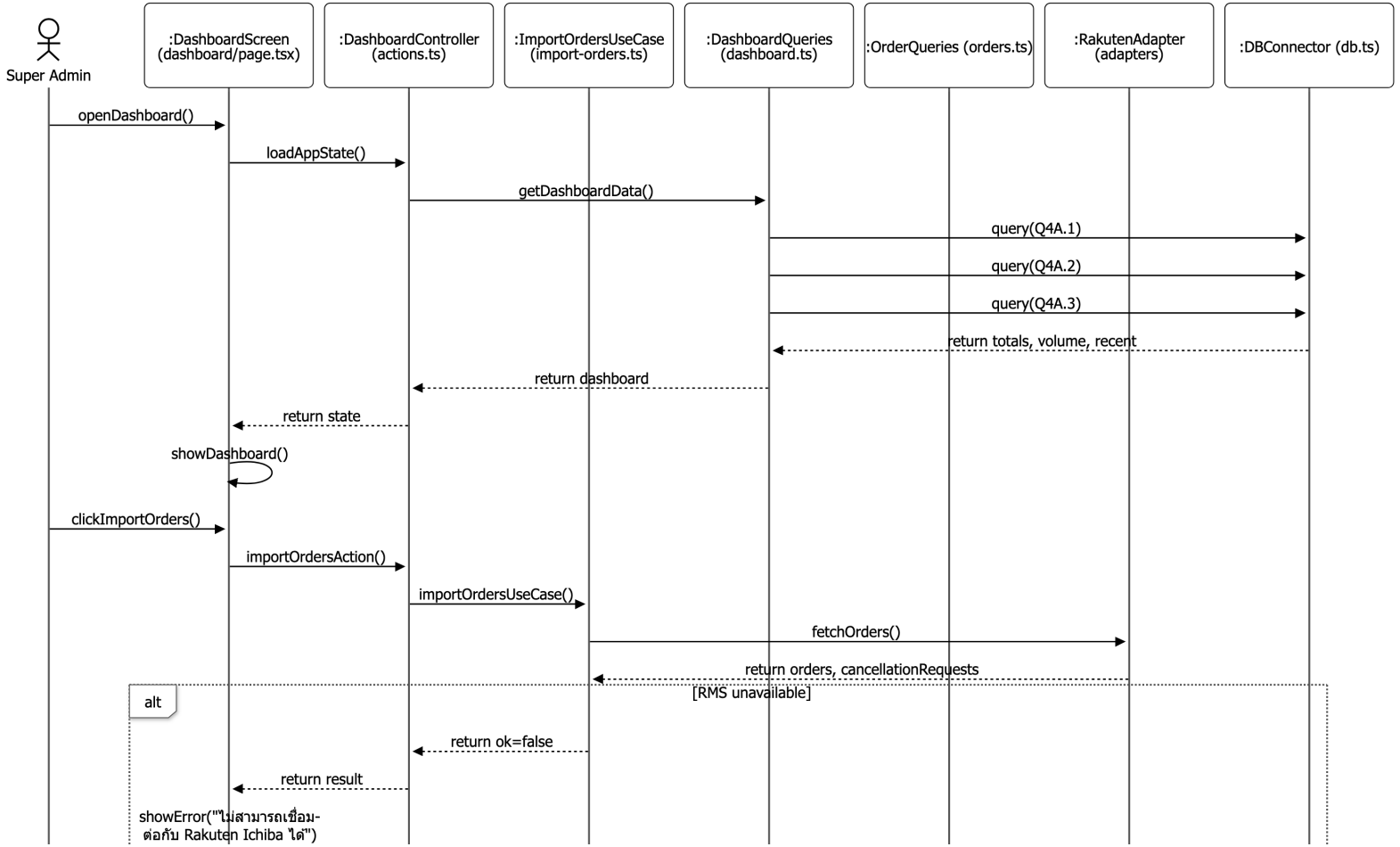
ตารางที่ 2-8 Use Case Description ที่ 4A: ดึง Order เข้าระบบ

ลำดับ	User Action	System Response
1		แสดงหน้าภาพรวมระบบ: จำนวน Order แยกตามสถานะ — **Q4A.1:** `SELECT order_status, COUNT(*)::integer AS count FROM orders GROUP BY order_status ORDER BY order_status`

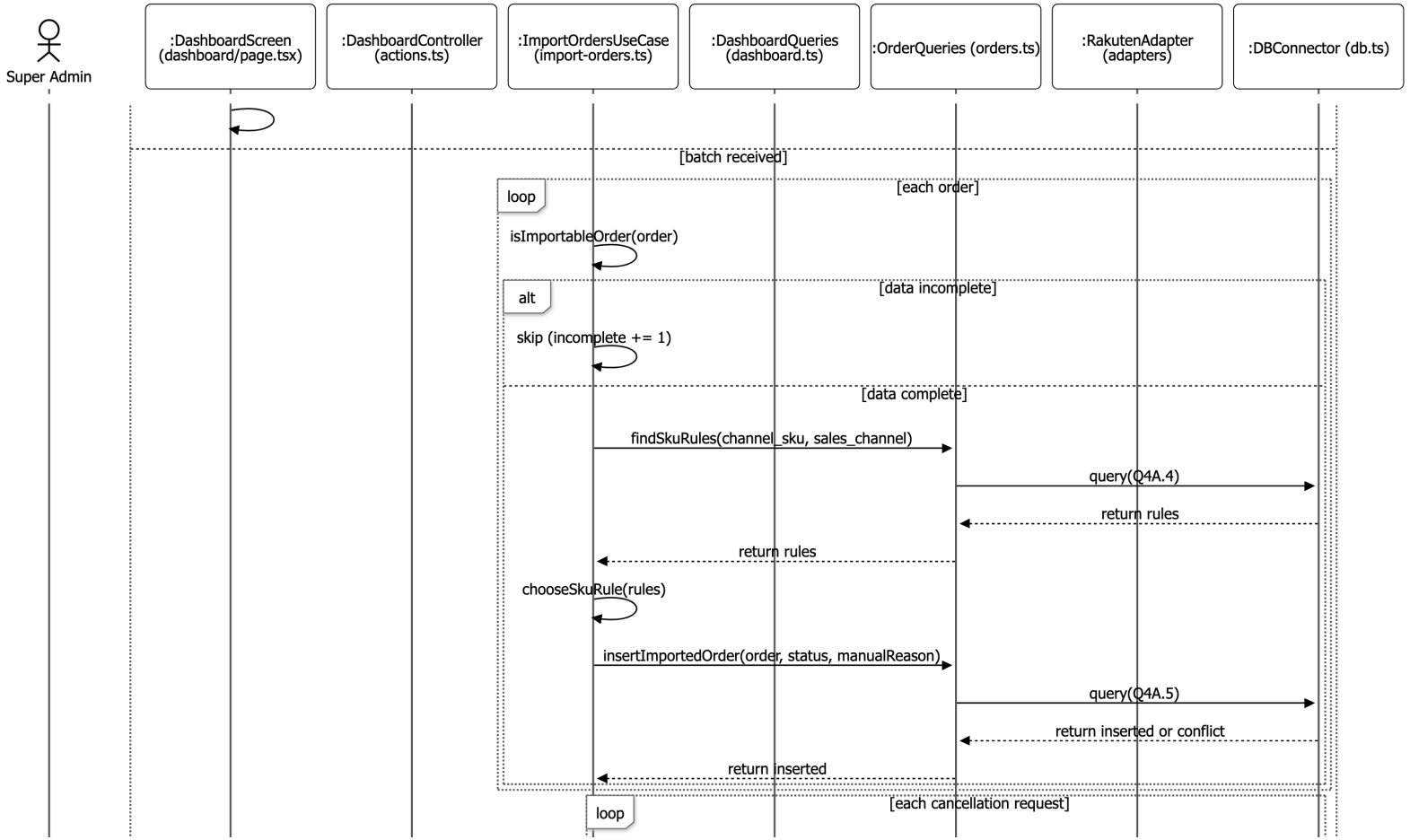
ลำดับ	User Action	System Response
		จำนวน Order 7 วันล่าสุด — Q4A.2: `SELECT TO_CHAR(order_date::date, 'YYYY-MM-DD') AS date, COUNT(*)::integer AS count FROM orders WHERE order_date >= ( SELECT MAX(order_date)::date - INTERVAL '6 days' FROM orders ) GROUP BY order_date::date ORDER BY order_date::date` และ Order ล่าสุด — Q4A.3: `SELECT order_id, order_date FROM orders ORDER BY order_date DESC, order_id DESC LIMIT 6` พร้อมปุ่ม "นำเข้า Order"
2	กด "นำเข้า Order"	ระบบเรียก Server Action `importOrdersAction` ⟶ `importOrdersUseCase` ซึ่งดึง Order ใหม่และรายการขอยกเลิกจาก Rakuten RMS (`RakutenAdapter.fetchOrders`) (ระบบไม่มีการดึงตามรอบเวลา ดึงเมื่อกดปุ่มเท่านั้น)
3		ทำทุก Order ใน transaction เดียว: **1. ตรวจสอบข้อมูล** — ต้องมี `marketplace_order_id`, `channel_sku`, `shipping_address`, `shipping_method` และ `qty` เป็นจำนวนเต็มบวก หากไม่ครบจะข้ามรายการนั้นและแจ้งเตือนว่า "ข้อมูล Order ไม่ครบถ้วน กรุณาตรวจสอบ" **2. หากฎ SKU** จาก `channel_sku` (สินค้าที่เปิดขาย) — Q4A.4: `SELECT sku_id, sku, product_name, variation FROM skus WHERE channel_sku = $1 AND sales_channel = $2 AND sku_status = 'active' ORDER BY sku_id` **3. บันทึก Order และรายการสินค้า** — `order_id` สร้างจาก hash ของ `marketplace_order_id`; หาก `marketplace_order_id` ซ้ำกับที่มีอยู่แล้ว `ON CONFLICT DO NOTHING` จะข้ามรายการนั้นเพื่อป้องกันข้อมูลซ้ำ — Q4A.5: `WITH inserted_order AS (INSERT INTO orders (order_id, marketplace_order_id, sales_channel, order_date, shipping_address, shipping_method, order_status, manual_reason, created_at, updated_at) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP) ON CONFLICT (marketplace_order_id) DO NOTHING RETURNING order_id), inserted_item AS (INSERT INTO order_items (order_item_id, channel_sku, variation, qty, fulfill_source, order_id, sku_id) SELECT $9, $10, $11, $12, $13, order_id, $14 FROM inserted_order RETURNING order_id) SELECT order_id FROM inserted_order` — พบกฎ SKU 1 รายการ: สถานะ "รอตรวจสอบคำสั่งซื้อ" และผูก `sku_id` (ค่า `fulfill_source` = `rsl` เป็นค่าตั้งต้น UC 3S จะกำหนดค่าจริง) **4. บันทึกคำขอยกเลิกจากลูกค้า** — Order ที่ยังไม่จบจะถูกตั้ง `customer_cancel_requested` — Q4A.6: `UPDATE orders SET customer_cancel_requested = TRUE, updated_at = CURRENT_TIMESTAMP WHERE marketplace_order_id = $1 AND order_status NOT IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') RETURNING order_id, order_status`

ลำดับ	User Action	System Response
		แสดงข้อความ "นำเข้า Order สำเร็จ n รายการ" (หรือ "ไม่มี Order ใหม่ใน Rakuten RMS") และ "ข้ามรายการที่ซ้ำกับที่มีอยู่แล้ว n รายการ"
ทางเลือก #1	เชื่อมต่อ Rakuten RMS ไม่ได้	แจ้งเตือนว่า "ไม่สามารถเชื่อมต่อกับ Rakuten Ichiba ได้ กรุณาตรวจสอบการตั้งค่า" ไม่มีการบันทึกข้อมูล ให้ Super Admin กดนำเข้าใหม่ภายหลัง
ทางเลือก #2	Order มี `channel_sku` ที่ยังไม่มีกฎ SKU หรือพบกฎมากกว่า 1 รายการ	บันทึก Order ด้วย Q4A.5 ในสถานะ "รอดำเนินการด้วยตนเอง" `manual_reason` = `sku-unregistered` (`sku_id` ว่าง) และแจ้งเตือนว่า "พบ SKU ที่ยังไม่ได้ลงทะเบียนในระบบ" เมื่อ Super Admin เพิ่มสินค้าใน UC 3A ระบบจะนำ Order กลับไป "รอตรวจสอบคำสั่งซื้อ"
ทางเลือก #3	ลูกค้ายกเลิก Order ผ่าน Rakuten หลังนำเข้าแล้ว	Q4A.6 ตั้ง `customer_cancel_requested` = TRUE หาก Order ผ่านการตรวจสอบไปแล้ว (ไม่ใช่ "รอตรวจสอบคำสั่งซื้อ") ระบบแจ้งเตือนว่า "ลูกค้ายกเลิก [Order ID] ผ่าน Rakuten ระหว่างที่ Order อยู่ระหว่างดำเนินการ กรุณาตรวจสอบที่หน้ายกเลิก Order" (UC 7A)

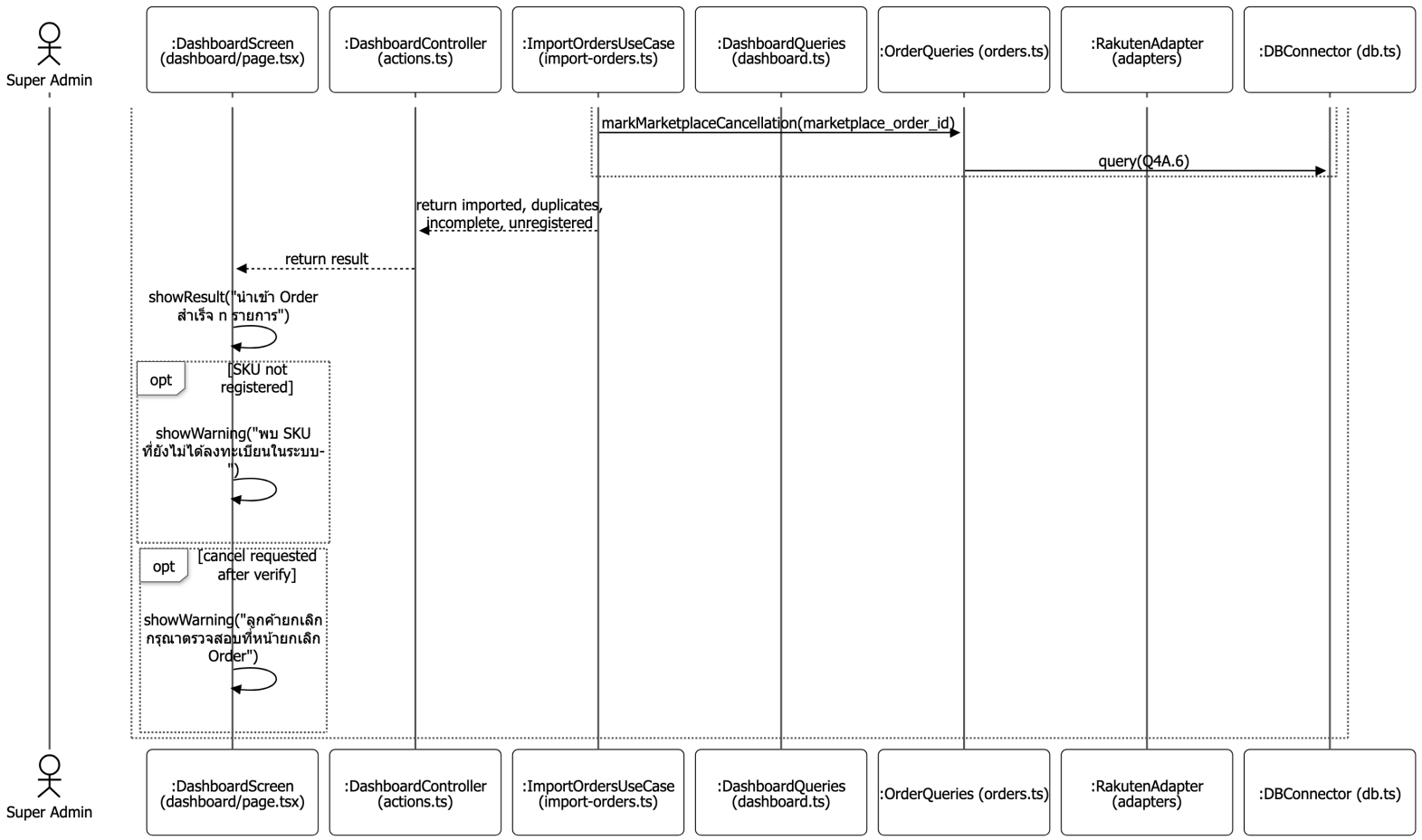
2.13.14 Sequence Diagram ของ Use Case Description ที่ 4A



รูปที่ 2-23 Sequence Diagram ของ Use Case Description ที่ 4A (1/3)

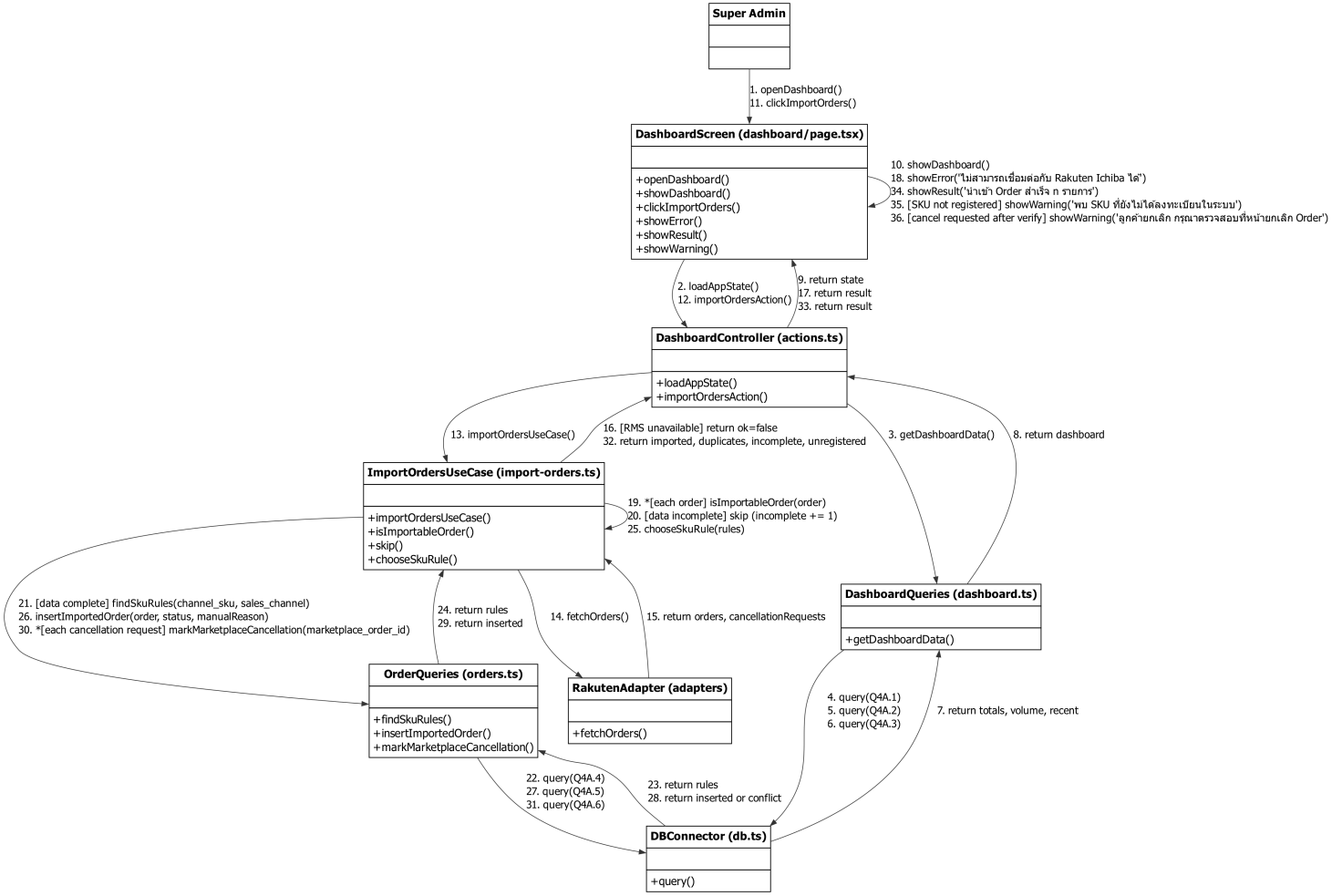


รูปที่ 2-24 Sequence Diagram ของ Use Case Description ที่ 4A (2/3)



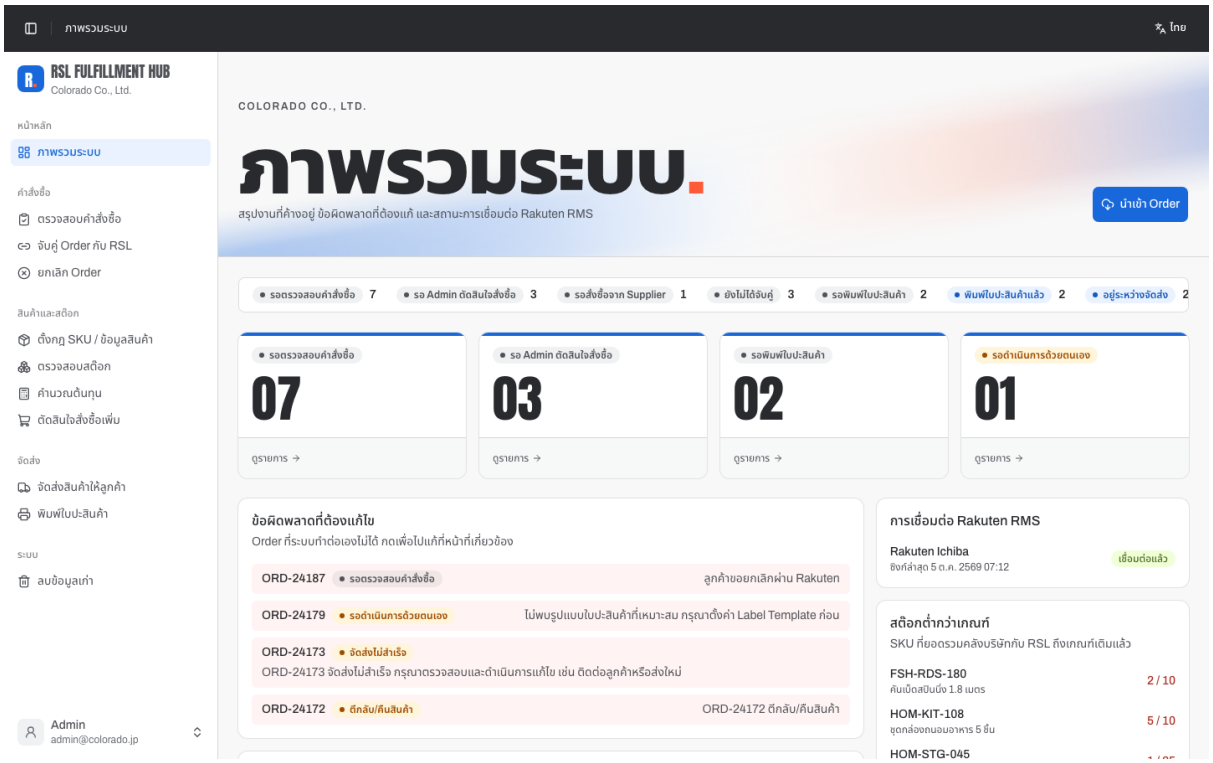
รูปที่ 2-25 Sequence Diagram ของ Use Case Description ที่ 4A (3/3)

2.13.15 Collaboration Diagram ของ Use Case Description ที่ 4A



รูปที่ 2-26 Collaboration Diagram ของ Use Case Description ที่ 4A

2.13.16 User Interface ของ Use Case Description ที่ 4A



รูปที่ 2-27 User Interface ของ Use Case Description ที่ 4A (DashboardScreen)

2.13.17 Use Case Description ที่ 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock

Goal: ให้ Super Admin ตัดสินใจจากต้นทุนที่คำนวณได้ว่าคุ้มค่าที่จะสั่งสินค้าเพิ่มเข้าสต๊อกหรือไม่

Pre-Condition: Order อยู่ในสถานะ "รอ Admin ตัดสินใจสั่งซื้อ" (มาจาก 3S เมื่อสต๊อก RSL และสต๊อกในบ้านไม่พอจำนวนใน Order) และ SKU มีต้นทุนต่อหน่วย (`costs.unit_cost`) จาก 4S แล้ว

Post-Condition: Order ถูกแยกเส้นทาง — คุ้มค่า: บันทึก `reorder_decision`='คุ้มค่า' และ `decided_qty` แล้วไปสู่ "รอสั่งซื้อจาก Supplier" (6A) / ไม่คุ้มค่า: ไปสู่ "รอยกเลิก Order" (7A) / ไม่มีราคาขาย: "รอดำเนินการด้วยตนเอง" (`manual_reason`='price') / Order ที่ผูกกับคำสั่งซื้อเติมสต๊อกเก็บเลขคำสั่งซื้อไว้ที่ `orders.reorder_id` (กำหนดใน 6A) คำสั่งซื้อหนึ่งรายการครอบคลุมได้หลาย Order ของ SKU เดียวกัน

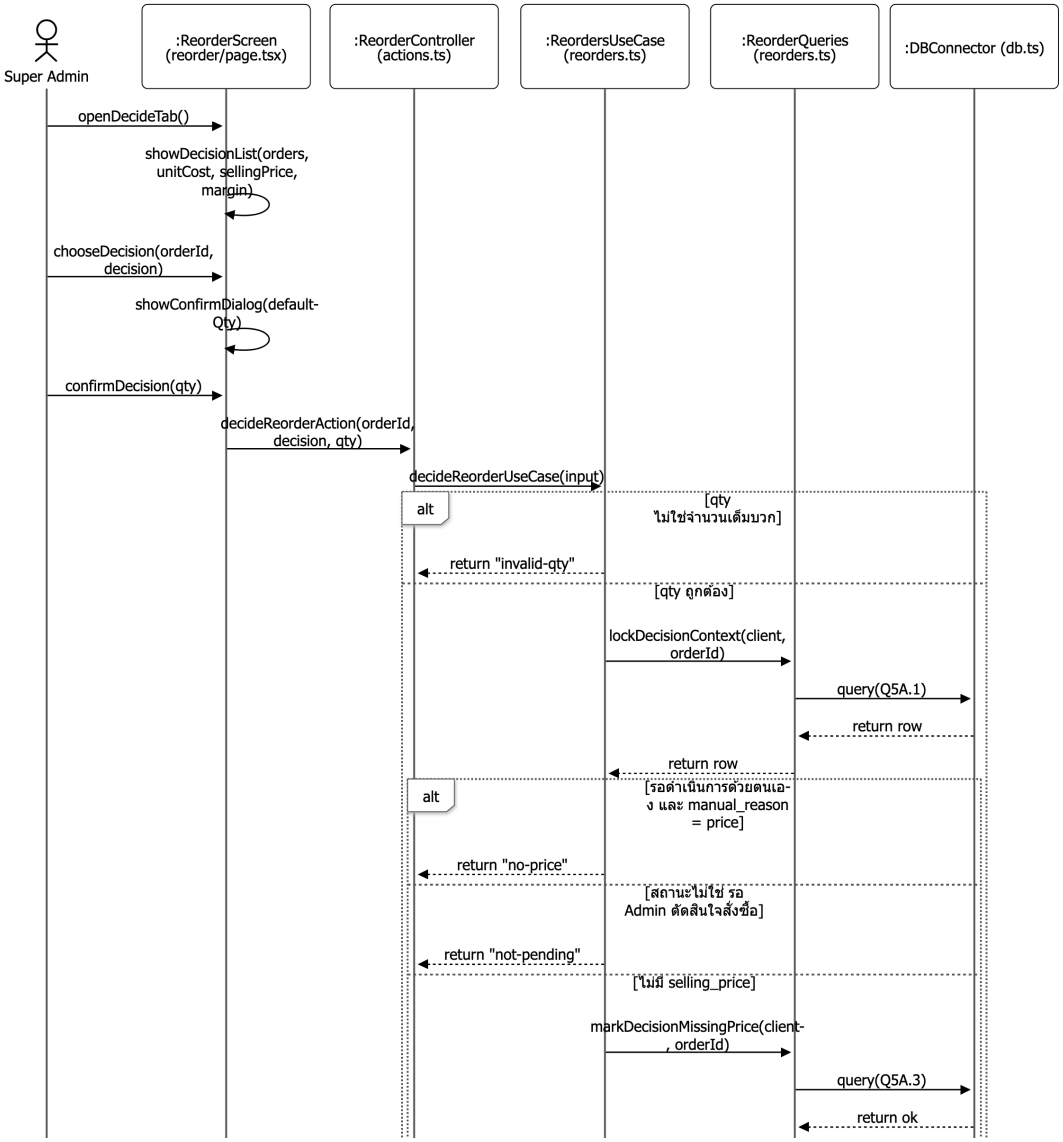
Actor: Super Admin

ตารางที่ 2-9 Use Case Description ที่ 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock

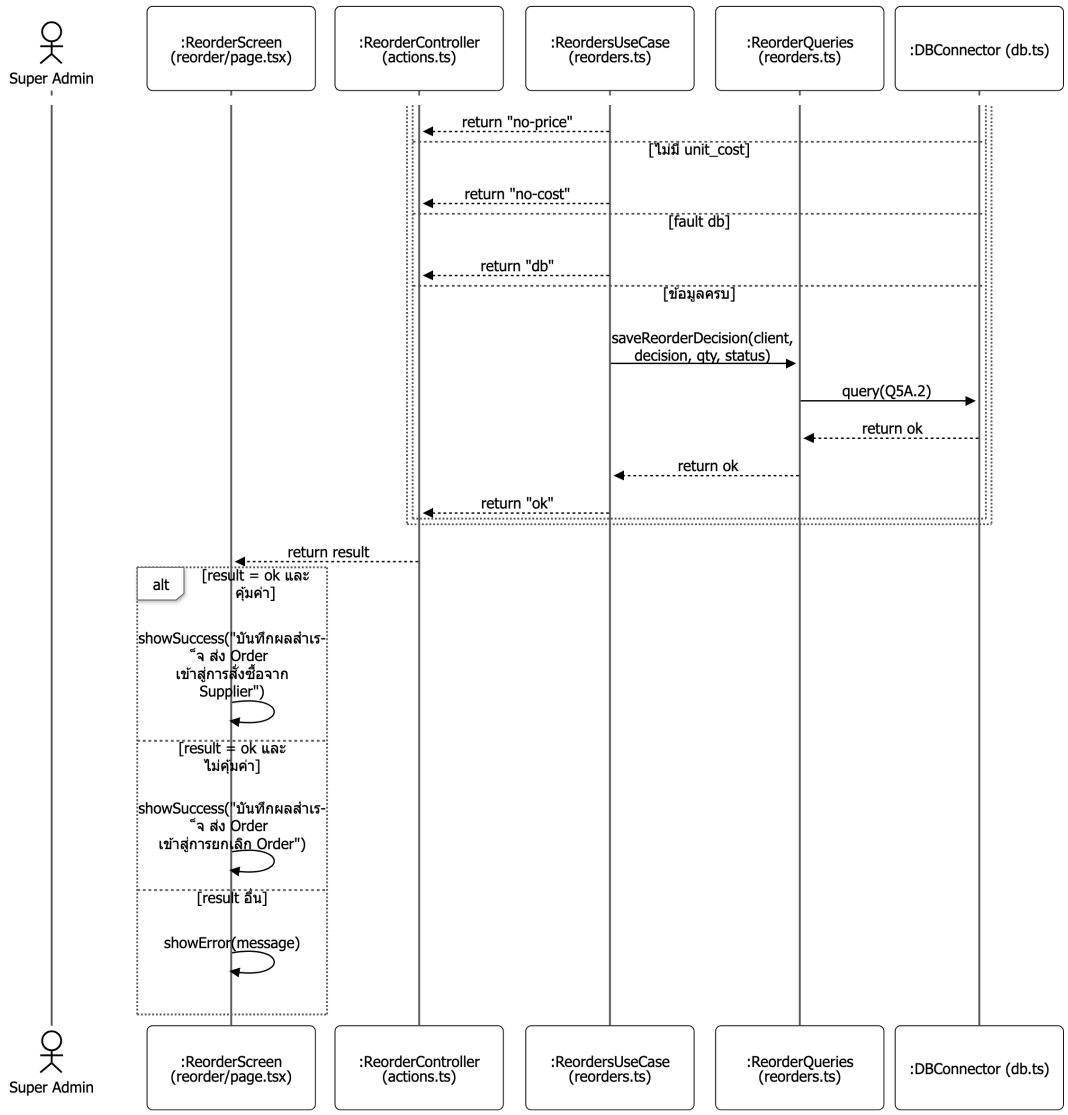
ลำดับ	User Action	System Response
1		หน้า `/products/reorder` แท็บ "รอตัดสินใจ" แสดง Order ที่อยู่ในสถานะ "รอ Admin ตัดสินใจสั่งซื้อ" และ Order ที่ "รอดำเนินการด้วยตนเอง" เพราะไม่มีราคาขาย (`manual_reason`='price') พร้อมต้นทุนต่อหน่วย (`unit_cost`), ราคาขายปัจจุบัน (`selling_price`), กำไรต่อหน่วยที่คาดการณ์ (`selling_price − unit_cost`) และระยะเวลารอของ (`lead_time_days`) ของ Supplier — ข้อมูลมาจาก read model ที่ layout โหลดผ่าน `loadAppState` (รายการ Order, SKU, ต้นทุน, Supplier) หากยังไม่มี `unit_cost` จะแสดง "คำนวณไม่ได้" พร้อมลิงก์ "ไปหน้าคำนวณต้นทุน" (4S) หากไม่มีราคาขายจะแสดง "ไม่พบราคาขาย" พร้อมลิงก์ "ไปหน้าตั้งกฎ SKU" (3A)
2	เลือก Order แล้วกด "คุ้มค่า - สั่งซื้อเพิ่ม" หรือ "ไม่คุ้มค่า - ยกเลิก"	เปิดหน้าต่าง "ยืนยันผลการตัดสินใจ: [คุ้มค่า/ไม่คุ้มค่า]" กรณีคุ้มค่าแสดงช่อง "จำนวนที่จะสั่งซื้อ" ค่าเริ่มต้น = `reorder_qty` ของ SKU แก้ได้เฉพาะครั้งนี้ (ไม่กระทบ Reorder Quantity ในกฎ SKU) และแสดงต้นทุนต่อหน่วยที่คำนวณใหม่ตามจำนวนนี้
3	กด "ยืนยัน"	ระบบเรียก Server Action `decideReorderAction` ⟶ `decideReorderUseCase` ทำงานใน transaction เดียว **1. ตรวจสอบจำนวน** — จำนวนต้องเป็นจำนวนเต็มบวก หากไม่ใช่ จะแจ้งเตือนว่า "กรุณากรอกจำนวนที่จะสั่งซื้อเป็นจำนวนเต็มบวก" (แสดงในหน้าต่างยืนยัน) **2. ล็อก Order และอ่านราคาขาย/ต้นทุนล่าสุด** — Q5A.1: SELECT `o.order_id, o.order_status, o.manual_reason, item.sku_id, sku.sku, sku.selling_price, COALESCE(cost.unit_cost, NULLIF(history.new_value ->> 'unit_cost', '')::numeric) AS unit_cost FROM orders o INNER JOIN LATERAL (SELECT * FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN LATERAL (SELECT unit_cost FROM costs WHERE sku_id = item.sku_id ORDER BY calculated_at DESC NULLS LAST, cost_id DESC LIMIT 1) cost ON TRUE LEFT JOIN LATERAL (SELECT new_value FROM cost_history WHERE sku_id = item.sku_id AND new_value ->> 'unit_cost' IS NOT NULL ORDER BY changed_at DESC, cost_history_id DESC LIMIT 1) history ON TRUE WHERE o.order_id = $1 FOR UPDATE OF o` (`unit_cost` ใช้ค่าล่าสุดจาก `costs` ถ้าไม่มีใช้ค่าล่าสุดใน `cost_history`) — หาก Order อยู่ที่ "รอดำเนินการด้วยตนเอง" เพราะไม่มีราคาขาย (`manual_reason`='price') จะแจ้งเตือนว่า "ไม่พบราคาขายของสินค้านี้ กรุณาตรวจสอบ" พร้อมปุ่ม "ไปหน้าตั้งกฎ SKU" (3A) และไม่บันทึก (หน้าจอซ่อนปุ่มตัดสินใจของ Order เหล่านี้อยู่แล้ว)

ลำดับ	User Action	System Response
		— หาก Order ไม่อยู่ในสถานะ "รอ Admin ตัดสินใจสั่งซื้อ" จะแจ้งเตือนว่า "Order นี้ได้รับการตัดสินใจไปแล้ว" และไม่บันทึกซ้ำ — หากไม่มีราคาขาย (`selling_price` เป็น NULL) ดูทางเลือก #3 — หากไม่มี `unit_cost` จะแจ้งเตือนว่า "กรุณากรอกข้อมูลต้นทุนให้ครบถ้วนและถูกต้อง" และคงสถานะเดิม **3. บันทึกผลการตัดสินใจ** — Q5A.2: `UPDATE orders SET reorder_decision = $2, decided_qty = CASE WHEN $2 = 'คุ้มค่า' THEN $3::integer ELSE NULL END, order_status = $4, manual_reason = $5, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1` ($2 = ผลการตัดสินใจ, $3 = จำนวนที่จะสั่งซื้อ เก็บเฉพาะกรณีคุ้มค่า, $4 = สถานะใหม่ ตามทางเลือก #1 / #2, $5 = NULL) — บันทึกสำเร็จ แสดงข้อความ "บันทึกผลการตัดสินใจสำเร็จ"
ทางเลือก #1	ตัดสินใจ "คุ้มค่า"	Q5A.2 ตั้ง `reorder_decision`='คุ้มค่า', `decided_qty` = จำนวนที่ยืนยัน และ `order_status`='รอสั่งซื้อจาก Supplier' แสดงข้อความ "ส่ง Order เข้าสู่การสั่งซื้อจาก Supplier" — SKU นี้จะไปอยู่ในแท็บ "รอสั่งซื้อเติมสต๊อก" ของ 6A โดยใช้ `decided_qty` เป็นจำนวนเริ่มต้น
ทางเลือก #2	ตัดสินใจ "ไม่คุ้มค่า"	Q5A.2 ตั้ง `reorder_decision`='ไม่คุ้มค่า', `decided_qty` = NULL และ `order_status`='รอยกเลิก Order' แสดงข้อความ "ส่ง Order เข้าสู่การยกเลิก Order" (ไป 7A)
ทางเลือก #3	SKU ไม่มีราคาขายปัจจุบัน	ตั้งสถานะ "รอดำเนินการด้วยตนเอง" พร้อม `manual_reason`='price' — Q5A.3: `UPDATE orders SET order_status = 'รอดำเนินการด้วยตนเอง', manual_reason = 'price', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1` แล้วแจ้งเตือนว่า "ไม่พบราคาขายของสินค้านี้ กรุณาตรวจสอบ" เมื่อ Super Admin ใส่ราคาขายใน 3A ระบบจะคืน Order กลับเป็น "รอ Admin ตัดสินใจสั่งซื้อ"
ทางเลือก #4	บันทึกลงฐานข้อมูลไม่สำเร็จ (จำลองด้วย fault `db` ในหน้า `/dev`)	แจ้งเตือนว่า "ไม่สามารถบันทึกผลการตัดสินใจได้ กรุณาลองใหม่ภายหลัง" ไม่ทำ Q5A.2 และคงสถานะ Order ไว้เท่าเดิมเพื่อให้กดยืนยันซ้ำได้

2.13.18 Sequence Diagram ของ Use Case Description ที่ 5A

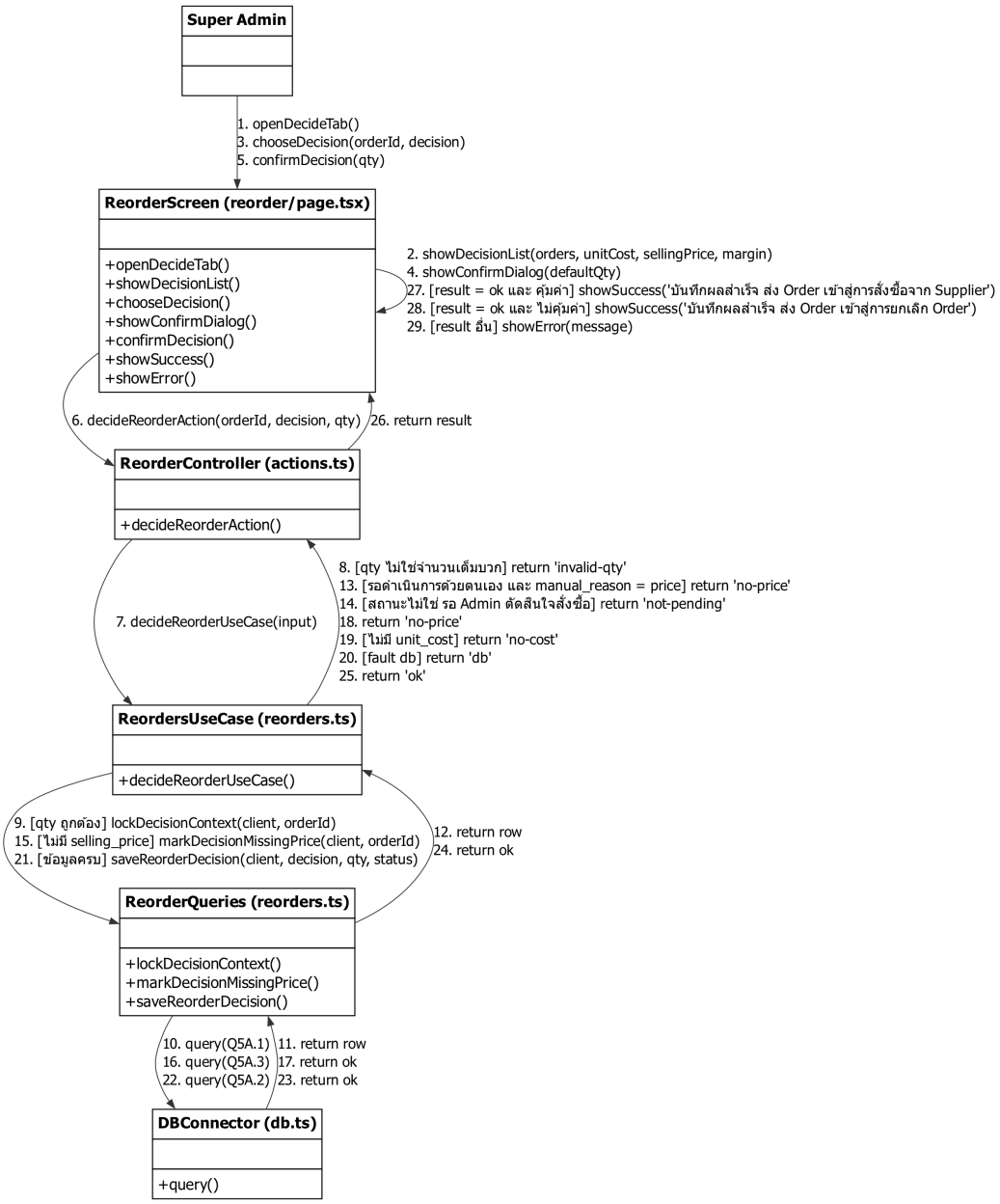


รูปที่ 2-28 Sequence Diagram ของ Use Case Description ที่ 5A (1/2)



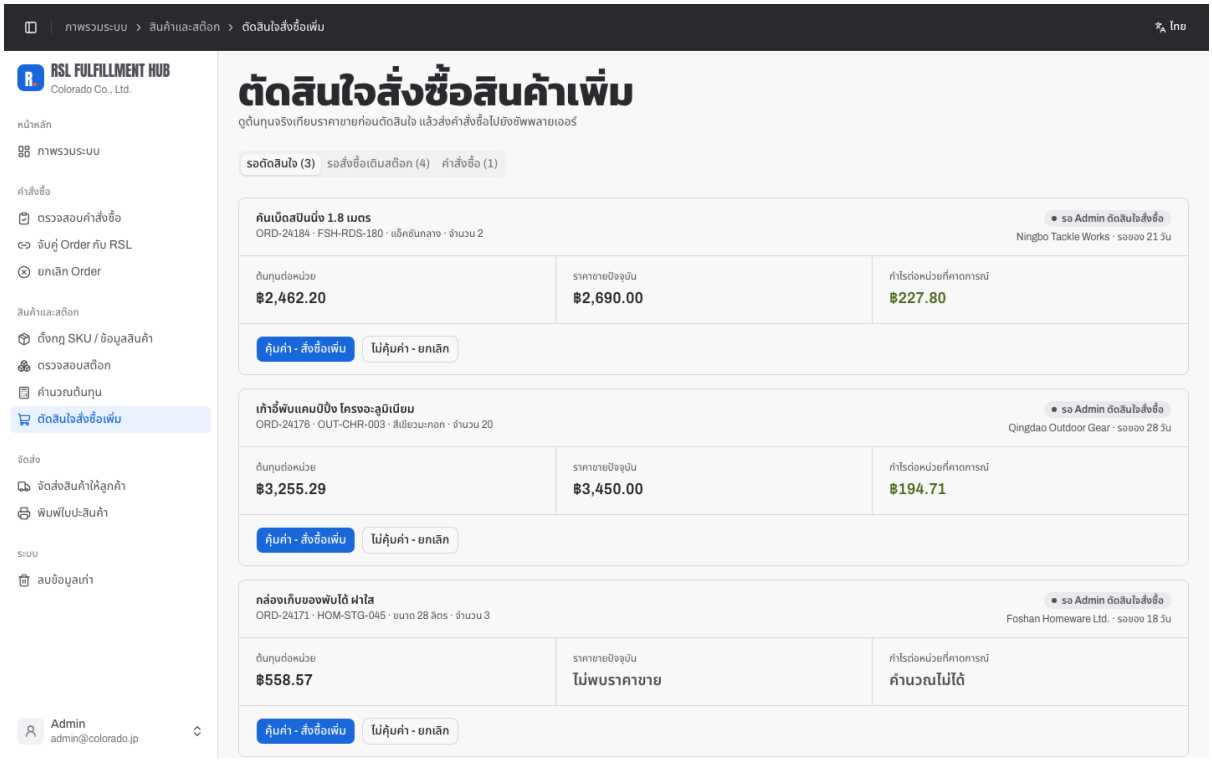
รูปที่ 2-29 Sequence Diagram ของ Use Case Description ที่ 5A (2/2)

2.13.19 Collaboration Diagram ของ Use Case Description ที่ 5A



รูปที่ 2-30 Collaboration Diagram ของ Use Case Description ที่ 5A

2.13.20 User Interface ของ Use Case Description ที่ 5A



รูปที่ 2-31 User Interface ของ Use Case Description ที่ 5A (ReorderScreen)

2.13.21 Use Case Description ที่ 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier

Goal: ให้ Super Admin ส่งคำสั่งซื้อเติมสต๊อกไปยัง Supplier สำหรับ SKU ที่สต๊อกรวมต่ำกว่าเกณฑ์ (Reorder Threshold) หรือ SKU ที่มี Order ตัดสินใจว่าคุ้มค่าใน 5A และบันทึกการรับสินค้าเมื่อ Supplier ส่งของมาถึง

Pre-Condition: SKU มี Supplier ที่มี Lead Time มากกว่า 0 และมี Reorder Quantity (หรือ `decided_qty` จาก 5A)

Post-Condition: คำสั่งซื้อ (`reorders`) อยู่ในสถานะ "สั่งซื้อแล้ว" พร้อม `expected_date` = วันนี้ + Lead Time และ Order ทุกรายการที่ผูกกับคำสั่งซื้อ (`orders.reorder_id`) อยู่ในสถานะ "สั่งซื้อแล้ว" จนกว่า Super Admin จะบันทึกการรับสินค้า / หากส่งไม่สำเร็จ คำสั่งซื้อและ Order คงสถานะ "รอส่งคำสั่งซื้อ" เพื่อส่งซ้ำ / เมื่อบันทึกรับสินค้า คำสั่งซื้อเป็น "ได้รับสินค้าแล้ว" (`received_at`, `received_qty`) สต๊อกในบริษัท (`in_house_qty`) เพิ่มตามจำนวนที่ได้รับ มีประวัติใน `stock_adjustment_history` และ Order ที่ผูกไว้กลับไปผ่าน 3S ทันที

Actor: Super Admin, Supplier

ตารางที่ 2-10 Use Case Description ที่ 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier

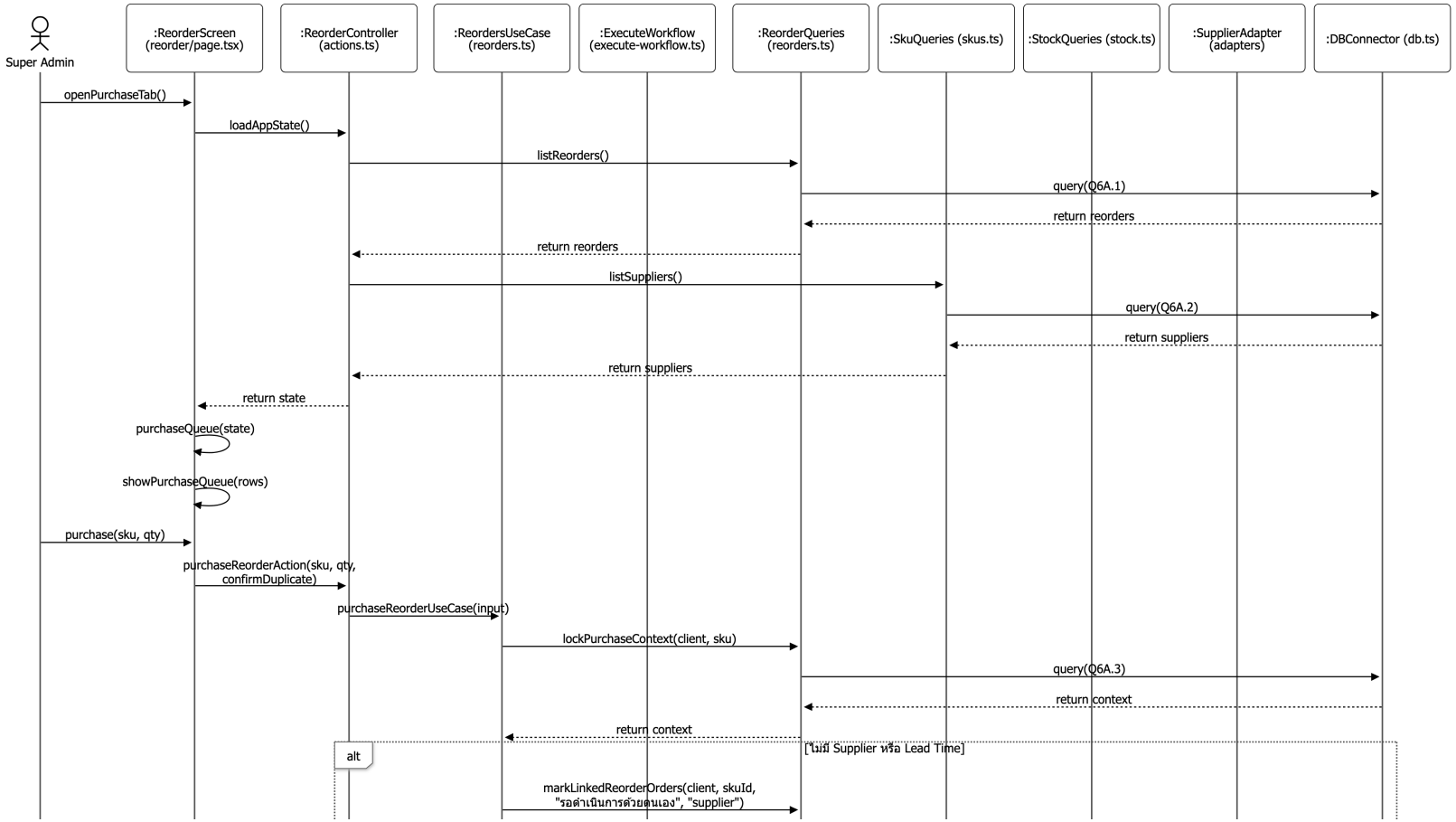
ลำดับ	User Action	System Response
1		หน้า `/products/reorder` โหลดรายการคำสั่งซื้อเติมสต๊อกทั้งหมด (แท็บ "คำสั่งซื้อ") — Q6A.1: `SELECT reorders.reorder_id, sku.sku, reorders.supplier_id, reorders.order_qty, COALESCE(reorders.ordered_at, reorders.created_at) AS order_date, reorders.expected_date, reorders.reorder_status, reorders.received_at, reorders.received_qty FROM reorders INNER JOIN skus sku ON sku.sku_id = reorders.sku_id ORDER BY COALESCE(reorders.ordered_at, reorders.created_at) DESC, reorders.reorder_id DESC` และรายการ Supplier พร้อม Lead Time — Q6A.2: `SELECT supplier_id, supplier_name, contact_info, lead_time_days FROM suppliers ORDER BY supplier_name, supplier_id`
2		แท็บ "รอสั่งซื้อเติมสต๊อก" แสดง SKU ที่ต้องสั่ง (คำนวณจาก read model ด้วย `purchaseQueue`): (ก) SKU active ที่สต๊อกรวม `in_house_qty + rsl_qty` ≤ `reorder_threshold` และไม่มีคำสั่งซื้อค้าง (ข) SKU ของ Order ที่ "รอสั่งซื้อจาก Supplier" หรือ "รอส่งคำสั่งซื้อ" (ค) SKU ที่มีคำสั่งซื้อ "รอส่งคำสั่งซื้อ" (แสดงป้าย "ส่งไม่สำเร็จครั้งก่อน") แต่ละแถวแสดงคงเหลือ/เกณฑ์, จำนวนที่จะสั่ง (`decided_qty` ถ้ามี ไม่เช่นนั้น `reorder_qty`), Order ที่รอ และ Supplier (หรือ "ยังไม่ได้ตั้งค่า")
3	กด "สั่งซื้อ" ที่ SKU แก้ "จำนวนที่จะสั่ง" ได้ (ไม่กระทบ Reorder Quantity) แล้วกดยืนยัน	ระบบเรียก Server Action `purchaseReorderAction` ⟶ `purchaseReorderUseCase` **1. ตรวจสอบจำนวนและล็อก SKU** — จำนวนต้องเป็นจำนวนเต็มบวก แล้วล็อกแถว SKU และอ่าน Supplier — Q6A.3: `SELECT sku.sku_id, sku.sku, sku.product_name, sku.variation, supplier.supplier_id, supplier.supplier_name, supplier.lead_time_days FROM skus sku LEFT JOIN suppliers supplier ON supplier.supplier_id = sku.supplier_id WHERE sku.sku = $1 FOR UPDATE OF sku` — หากไม่พบ SKU หรือไม่มี Supplier/Lead Time ดูทางเลือก #1 **2. ตรวจคำสั่งซื้อค้าง** — Q6A.4: `SELECT reorder_id, order_qty, supplier_id, supplier_request_id, expected_date, created_at FROM reorders WHERE sku_id = $1 AND reorder_status = 'สั่งซื้อแล้ว' AND expected_date >= CURRENT_DATE - 1 ORDER BY ordered_at DESC NULLS LAST, created_at DESC LIMIT 1` หากมีและยังไม่ยืนยันสั่งซ้ำ ดูทางเลือก #3 **3. เตรียมคำสั่งซื้อ** — หาคำสั่งซื้อที่ส่งไม่สำเร็จครั้งก่อนเพื่อใช้ `supplier_request_id` เดิม — Q6A.5: `SELECT reorder_id, order_qty, supplier_id, supplier_request_id, expected_date, created_at FROM reorders WHERE sku_id = $1 AND`

ลำดับ	User Action	System Response
		`reorder_status = 'รอส่งคำสั่งซื้อ' ORDER BY created_at DESC LIMIT 1` — หากไม่มี: ขอเลขคำสั่งซื้อถัดไปแบบเลขรัน (PO-0092 ต่อจาก PO-0091 และ `supplier_request_id` = SUPPLIER-req-0092) จาก sequence `reorder_no_seq` (migration 008) — **Q6A.6:** `SELECT nextval('reorder_no_seq') AS reorder_no` — แล้วสร้างคำสั่งซื้อใหม่สถานะ "รอส่งคำสั่งซื้อ" (`expected_date` = วันนี้ + `lead_time_days`) — **Q6A.7:** `INSERT INTO reorders ( reorder_id, order_qty, reorder_status, expected_date, created_at, sku_id, supplier_id, supplier_request_id ) VALUES ($1, $2, 'รอส่งคำสั่งซื้อ', $3, CURRENT_TIMESTAMP, $4, $5, $6)` — ผูก Order ทุกรายการของ SKU นี้ที่อยู่ในสถานะ "รอสั่งซื้อจาก Supplier" หรือ "รอส่งคำสั่งซื้อ" (รวม Order ที่รอดำเนินการด้วยตนเอง เพราะไม่มีข้อมูล Supplier) กับคำสั่งซื้อนี้ โดยบันทึกเลขคำสั่งซื้อลงใน `orders.reorder_id` และเปลี่ยนเป็น "รอส่งคำสั่งซื้อ" — **Q6A.8:** `UPDATE orders o SET order_status = $2, manual_reason = $3, reorder_id = COALESCE($4, o.reorder_id), updated_at = CURRENT_TIMESTAMP WHERE EXISTS (SELECT 1 FROM order_items item WHERE item.order_id = o.order_id AND item.sku_id = $1) AND (o.order_status IN ('รอสั่งซื้อจาก Supplier', 'รอส่งคำสั่งซื้อ') OR (o.order_status = 'รอดำเนินการด้วยตนเอง' AND o.manual_reason = 'supplier'))` ($2='รอส่งคำสั่งซื้อ', $3=NULL, $4=เลขคำสั่งซื้อ) แล้ว commit ก่อนติดต่อภายนอก **4. ส่งคำสั่งซื้อไปยัง Supplier** — เรียก `SupplierAdapter.sendPurchaseOrder(po, supplier_request_id)` (ส่งทางอีเมล) **5. บันทึกผล** (transaction ใหม่) — **Q6A.9:** `UPDATE reorders SET reorder_status = $2, ordered_at = CASE WHEN $2 = 'สั่งซื้อแล้ว' THEN CURRENT_TIMESTAMP ELSE ordered_at END, supplier_reference = $3, last_error_code = $4 WHERE reorder_id = $1 AND supplier_request_id = $5 AND ($2 = 'สั่งซื้อแล้ว' OR reorder_status <> 'สั่งซื้อแล้ว') RETURNING reorder_id` ($2='สั่งซื้อแล้ว', $3=`supplier_reference`) และ **Q6A.8** ($2='สั่งซื้อแล้ว', $4=เลขคำสั่งซื้อ) Order ที่ผูกไว้ทุกรายการเปลี่ยนเป็น "สั่งซื้อแล้ว" แล้วแสดงข้อความ "ส่งคำสั่งซื้อ [SKU] ไปยัง [Supplier] แล้ว" และ "สถานะ สั่งซื้อแล้ว · คาดว่าจะได้รับสินค้า [วันที่]"
4	เมื่อสินค้ามาถึง กด "รับสินค้า" ที่คำสั่งซื้อสถานะ "สั่งซื้อแล้ว" ในแท็บ "คำสั่งซื้อ"	แสดงหน้าต่าง "บันทึกการรับสินค้า [เลขที่]" พร้อมข้อความ "สั่ง [SKU] ไว้ [จำนวน] ชิ้น สินค้าที่ได้รับจะถูกเพิ่มเข้าสต๊อกในบริษัท และ Order ที่รอสินค้านี้จะกลับไปตรวจสอบสต๊อก" และช่อง "จำนวนที่ได้รับ" (ค่าเริ่มต้น = จำนวนที่สั่ง แก้ได้หากได้รับไม่ครบหรือเกิน บันทึกได้ครั้งเดียวต่อคำสั่งซื้อ)

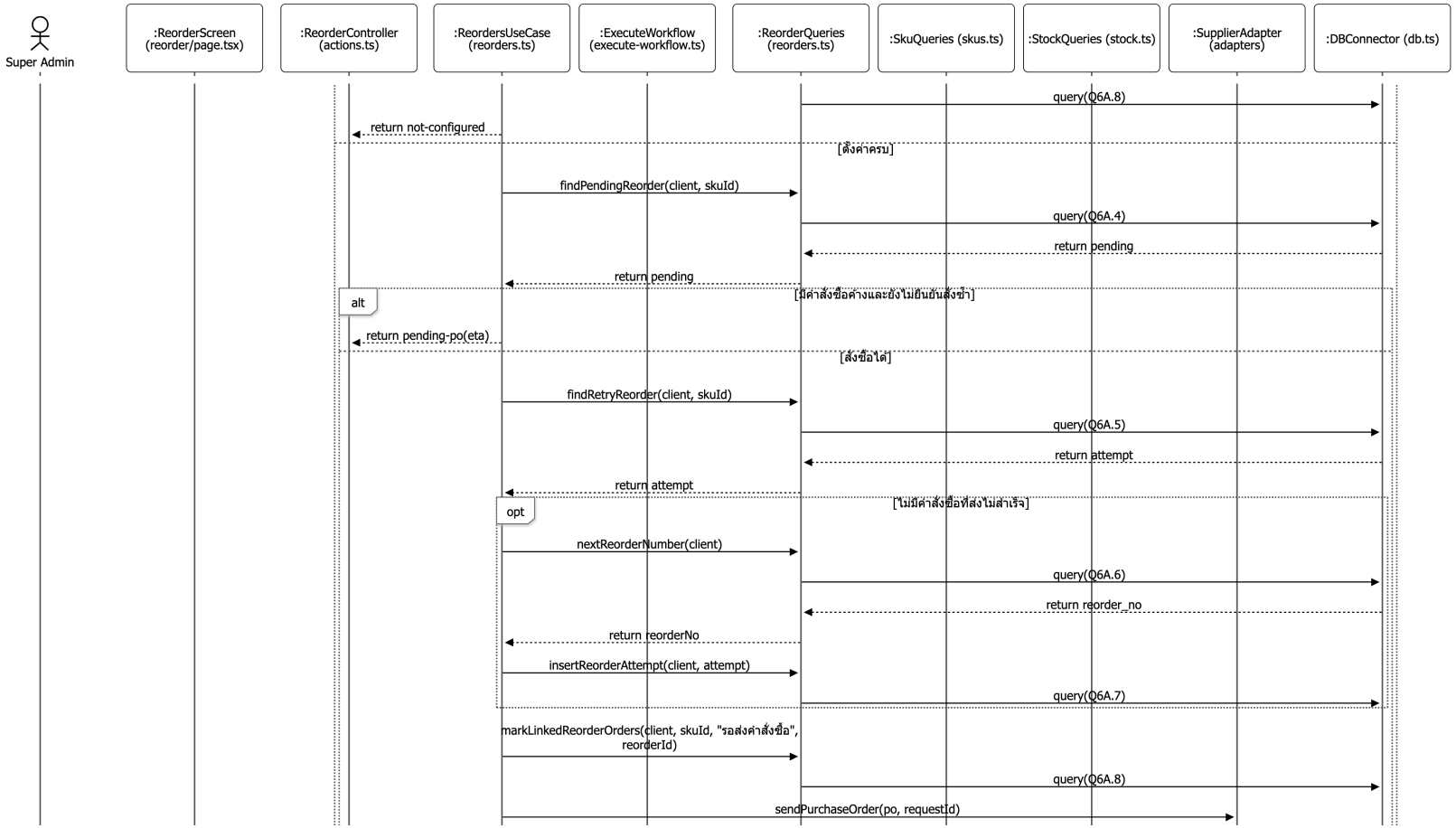
ลำดับ	User Action	System Response
5	กรอกจำนวนที่ได้รับแล้วกด "รับสินค้า"	ระบบเรียก Server Action `receiveReorderAction` ⟶ `receiveReorderUseCase` ทำงานใน `executeWorkflow` (transaction เดียว) **1. ตรวจสอบจำนวน** — ต้องเป็นจำนวนเต็มบวก หากไม่ใช่ จะแจ้งเตือนว่า "กรุณากรอกจำนวนที่ได้รับเป็นจำนวนเต็มบวก" **2. ล็อกคำสั่งซื้อและแถวสต๊อกของ SKU** — Q6A.10: `SELECT r.reorder_id, r.reorder_status, r.order_qty, sku.sku, stock.stock_id, stock.in_house_qty, stock.rsl_qty FROM reorders r INNER JOIN skus sku ON sku.sku_id = r.sku_id INNER JOIN stock ON stock.sku_id = r.sku_id WHERE r.reorder_id = $1 FOR UPDATE OF r, stock` — หากเป็น "ได้รับสินค้าแล้ว" ดูทางเลือก #5 หากไม่ใช่ "สั่งซื้อแล้ว" (เช่น ยกเลิกแล้ว) ดูทางเลือก #6 **3. ปิดคำสั่งซื้อ** — เปลี่ยนสถานะเป็น "ได้รับสินค้าแล้ว" พร้อม `received_at` และ `received_qty` — Q6A.11: `UPDATE reorders SET reorder_status = 'ได้รับสินค้าแล้ว', received_at = CURRENT_TIMESTAMP, received_qty = $2 WHERE reorder_id = $1 AND reorder_status = 'สั่งซื้อแล้ว' RETURNING reorder_id` (เงื่อนไข `reorder_status = 'สั่งซื้อแล้ว'` ทำให้กดซ้ำหรือกดพร้อมกันไม่บวกสต๊อกซ้ำ) **4. เพิ่มสต๊อกในบริษัท** — บวกจำนวนที่ได้รับเข้า `in_house_qty` (`rsl_qty` คงเดิม) และตั้งสถานะซิงก์สต๊อก Rakuten เป็น 'pending' — Q6A.12: `WITH adjusted AS (UPDATE stock SET in_house_qty = $2, rsl_qty = $3, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 RETURNING stock_id) INSERT INTO rakuten_stock_sync_states (stock_id, sync_status) SELECT stock_id, 'pending' FROM adjusted ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL` แล้วบันทึกประวัติการปรับสต๊อก (เหตุผล "รับสินค้าจาก Supplier ตามคำสั่งซื้อ [เลขที่]") — Q6A.13: `INSERT INTO stock_adjustment_history (stock_adjustment_history_id, stock_id, previous_in_house_qty, new_in_house_qty, previous_rsl_qty, new_rsl_qty, reason, admin_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)` **5. ส่ง Order ที่รอสินค้ากลับไปตรวจสต๊อก** — Q6A.14: `UPDATE orders SET order_status = 'รอตรวจสอบสต๊อก', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE reorder_id = $1 AND order_status = 'สั่งซื้อแล้ว' RETURNING order_id` แล้ว `runAutomation` รัน 3S กับ Order เหล่านั้น (และ 6S สำหรับ Order ที่ส่งจากคลังบริษัท) และ 7S ซิงก์ยอดคงเหลือไป Rakuten ก่อน commit

ลำดับ	User Action	System Response
		— แสดงข้อความ "เพิ่ม [SKU] เข้าสต๊อกในบริษัท [จำนวน] ชิ้นแล้ว" และ "Order ที่รอสินค้านี้ [n] รายการกลับไปตรวจสอบสต๊อกแล้ว" แถวคำสั่งซื้อแสดง "ได้รับ [จำนวน] ชิ้น · [วันที่]"
ทางเลือก #1	SKU ไม่มี Supplier หรือ Lead Time หรือจำนวนสั่งซื้อไม่ถูกต้อง	แจ้งเตือนว่า "ไม่พบข้อมูล Supplier หรือจำนวนสั่งซื้อ กรุณาตั้งค่าก่อนสั่งซื้อ" หากพบ SKU จะตั้ง Order ที่ผูกอยู่เป็น "รอดำเนินการด้วยตนเอง" ด้วย **Q6A.8** ($2='รอดำเนินการด้วยตนเอง', $3='supplier') ให้ Super Admin ไปตั้งค่าใน 3A
ทางเลือก #2	Super Admin แก้จำนวนสั่งซื้อก่อนยืนยัน	ใช้จำนวนที่แก้ใน **Q6A.7** (`order_qty`) เฉพาะคำสั่งซื้อนี้ ไม่แก้ `reorder_qty` ของ SKU
ทางเลือก #3	SKU มีคำสั่งซื้อ "สั่งซื้อแล้ว" ที่ยังไม่ถึงกำหนดรับของ (`expected_date` ≥ เมื่อวาน)	แจ้งเตือนว่า "มีคำสั่งซื้อ SKU นี้ค้างอยู่แล้ว ต้องการสั่งซื้อเพิ่มหรือไม่" พร้อม "ล็อตเดิมคาดว่าจะได้รับ [วันที่]" หาก Super Admin ยืนยัน ระบบเรียกซ้ำโดยส่ง `confirmDuplicate`=true แล้วทำขั้นตอนที่ 3 ต่อ
ทางเลือก #4	ส่งคำสั่งซื้อไปยัง Supplier ไม่สำเร็จ (จำลองด้วย fault `supplierSend`)	บันทึก **Q6A.9** ($2='รอส่งคำสั่งซื้อ', `last_error_code` = รหัสข้อผิดพลาด) และ **Q6A.8** ($2='รอส่งคำสั่งซื้อ') แจ้งเตือนว่า "ส่งคำสั่งซื้อไม่สำเร็จ กรุณาตรวจสอบและลองใหม่" เมื่อกดสั่งซื้ออีกครั้ง **Q6A.5** จะหยิบคำสั่งซื้อเดิมและ `supplier_request_id` เดิมมาส่งซ้ำ (ไม่สร้างคำสั่งซื้อซ้ำ)
ทางเลือก #5	กดรับสินค้าซ้ำกับคำสั่งซื้อที่บันทึกรับไปแล้ว	แจ้งเตือนว่า "คำสั่งซื้อนี้บันทึกการรับสินค้าไปแล้ว" ไม่เปลี่ยนสต๊อกหรือสถานะใดๆ
ทางเลือก #6	คำสั่งซื้อไม่อยู่ในสถานะ "สั่งซื้อแล้ว" (เช่น ถูกยกเลิกใน 7A)	แจ้งเตือนว่า "รับสินค้าได้เฉพาะคำสั่งซื้อที่อยู่ในสถานะ สั่งซื้อแล้ว" ไม่เปลี่ยนข้อมูล
ทางเลือก #7	ได้รับสินค้าไม่ครบหรือเกินจำนวนที่สั่ง	กรอกจำนวนที่ได้รับจริง ระบบบันทึก `received_qty` ตามนั้นและปิดคำสั่งซื้อ (รับได้ครั้งเดียว) หากสต๊อกยังไม่พอ 3S จะส่ง Order กลับไป "รอ Admin ตัดสินใจสั่งซื้อ" (5A) เพื่อสั่งเพิ่ม

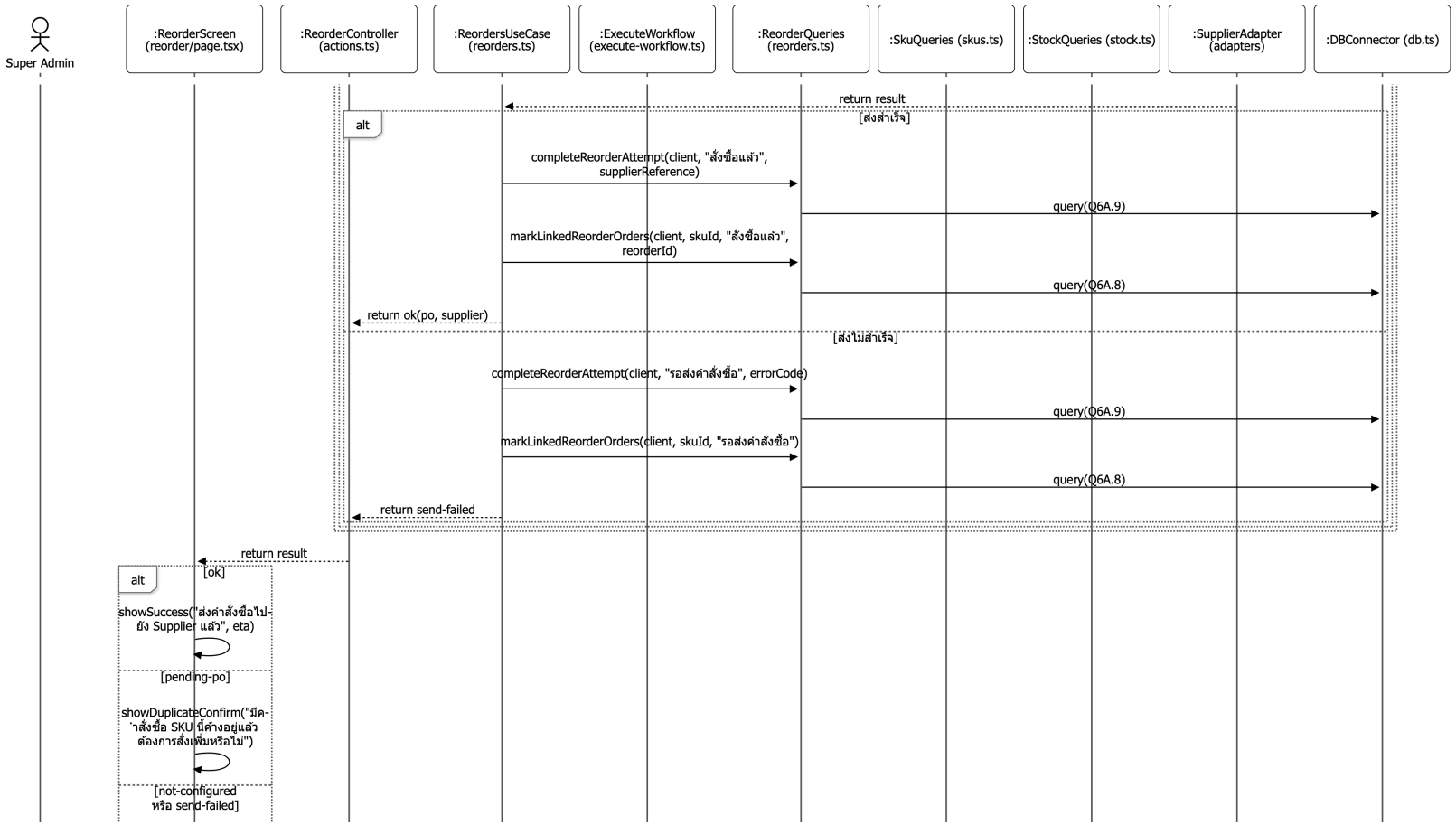
2.13.22 Sequence Diagram ของ Use Case Description ที่ 6A



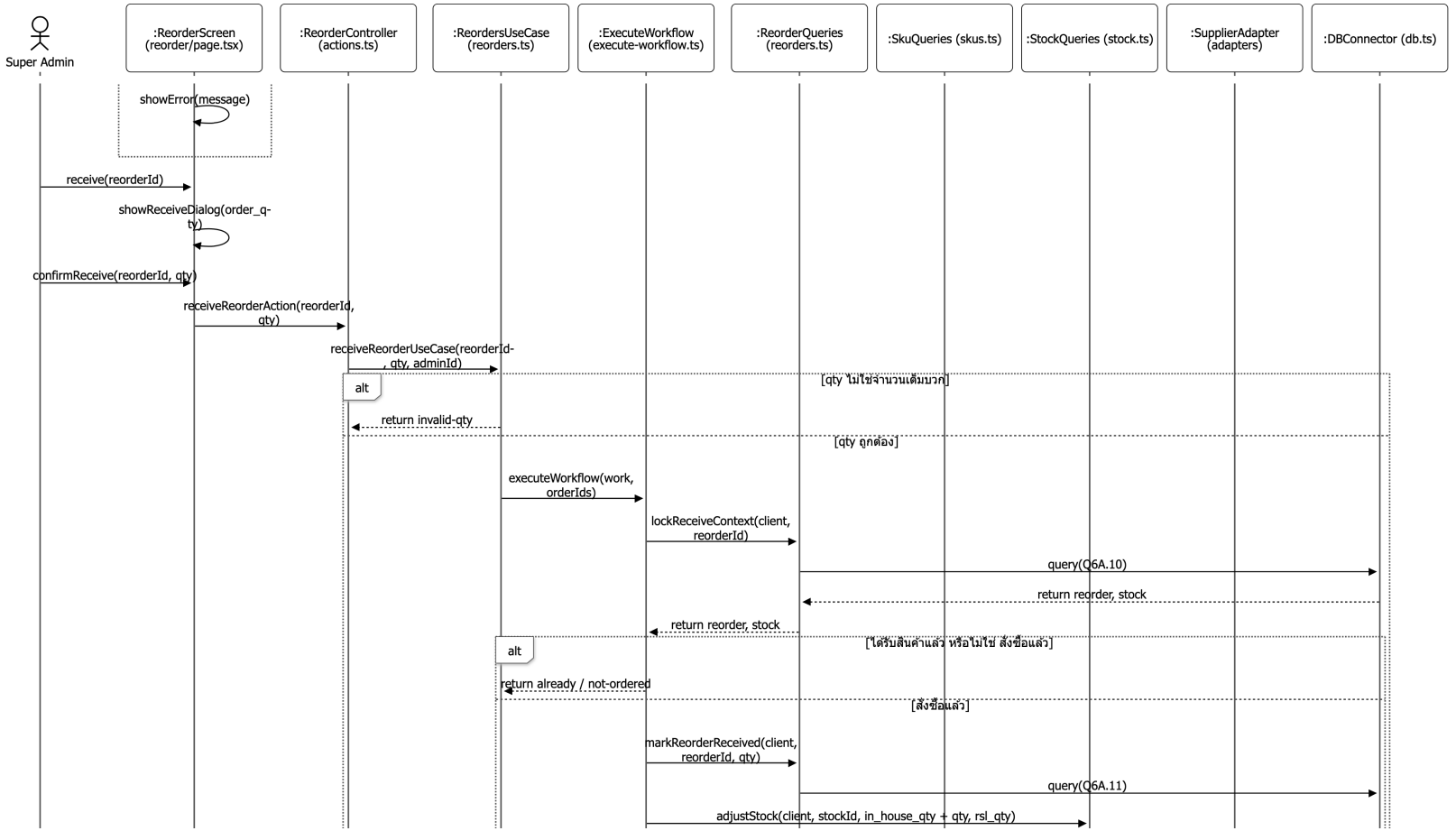
รูปที่ 2-32 Sequence Diagram ของ Use Case Description ที่ 6A (1/5)



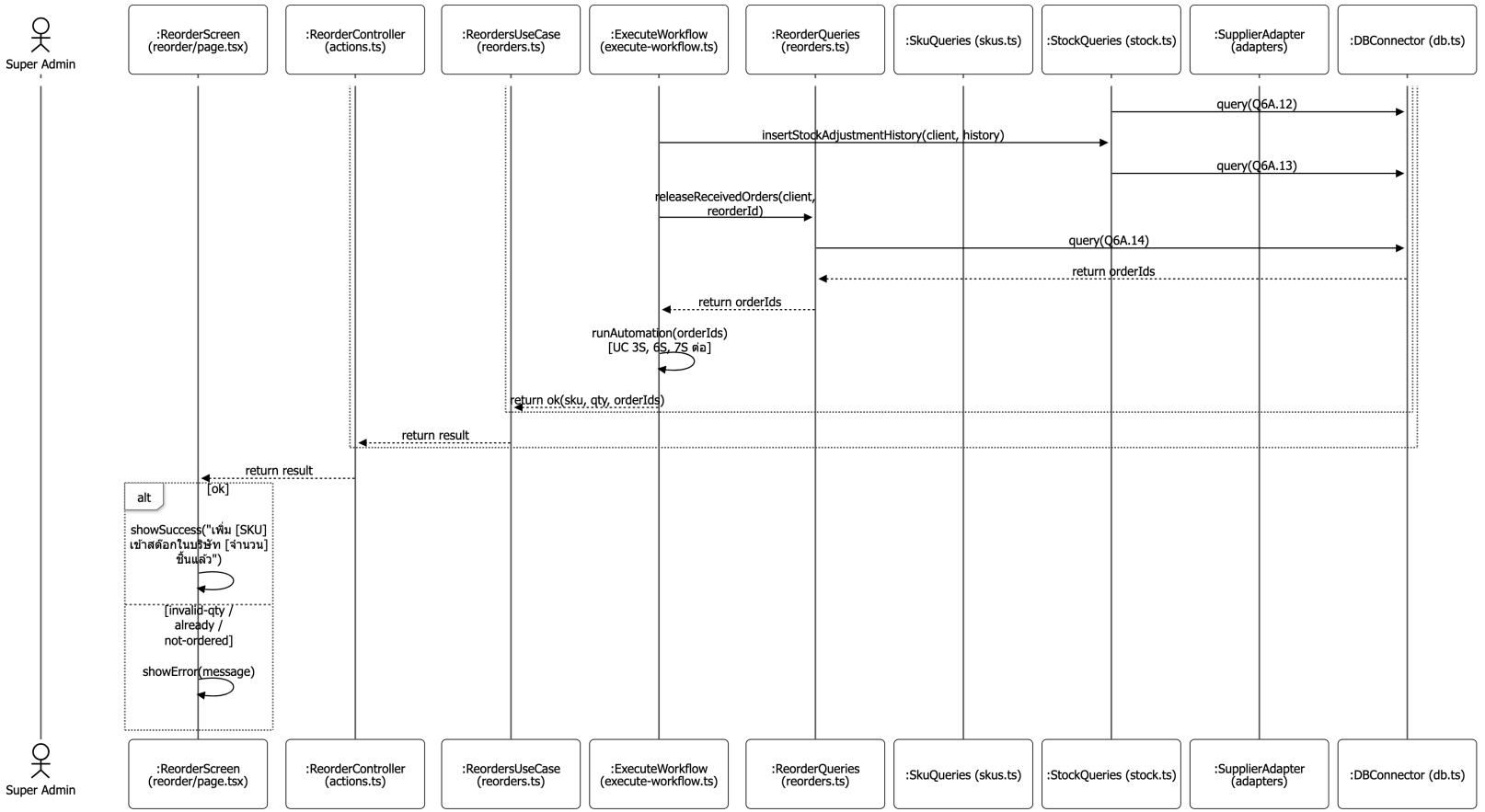
รูปที่ 2-33 Sequence Diagram ของ Use Case Description ที่ 6A (2/5)



รูปที่ 2-34 Sequence Diagram ของ Use Case Description ที่ 6A (3/5)

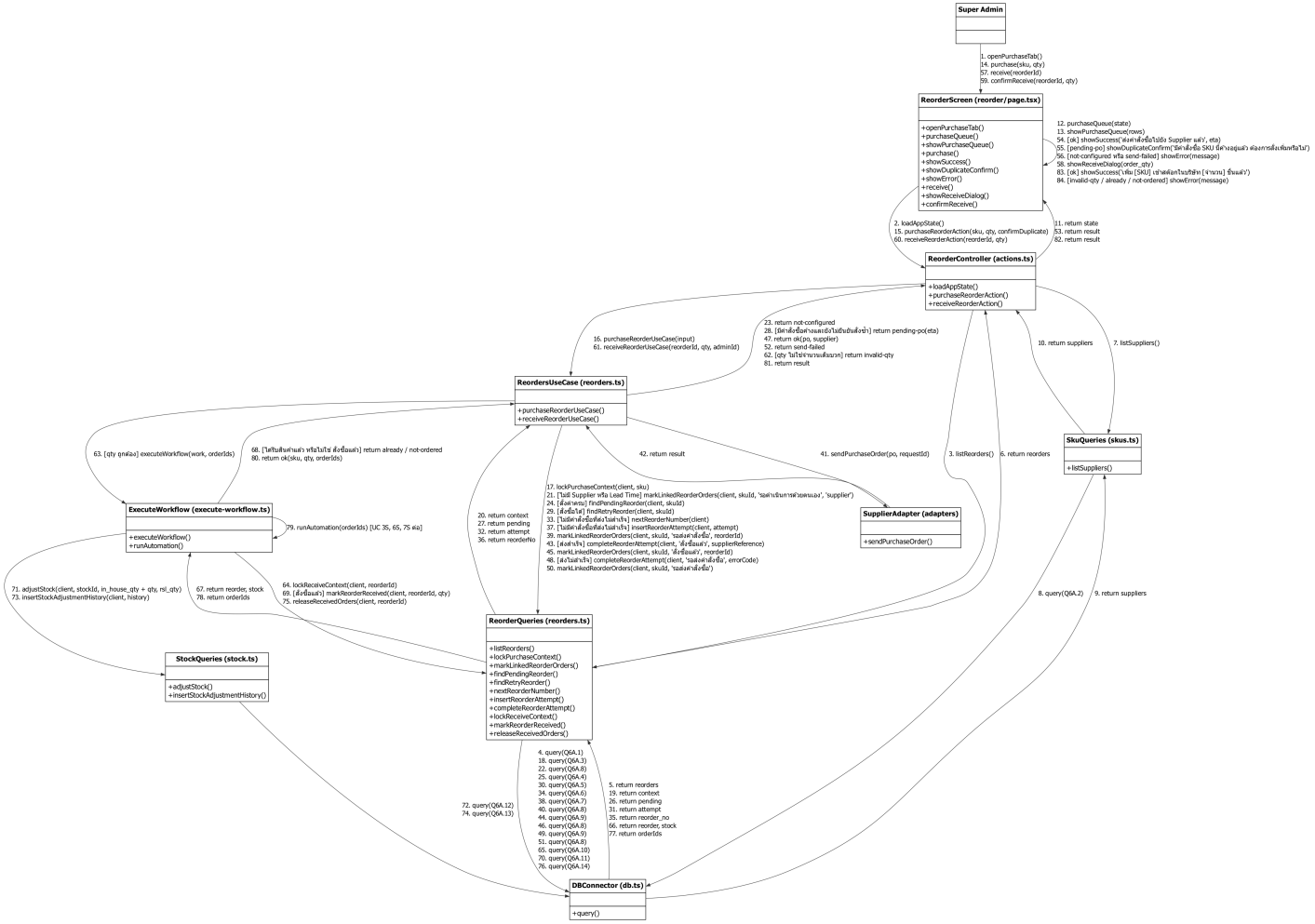


รูปที่ 2-35 Sequence Diagram ของ Use Case Description ที่ 6A (4/5)



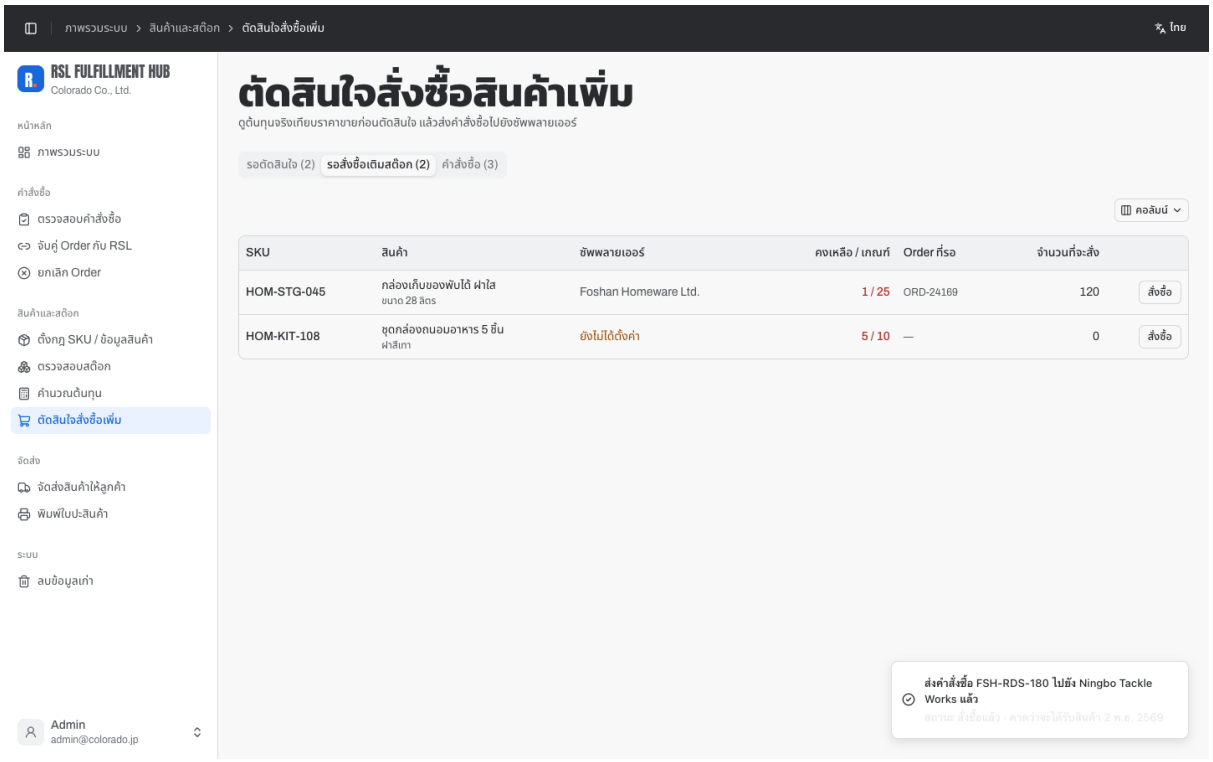
รูปที่ 2-36 Sequence Diagram ของ Use Case Description ที่ 6A (5/5)

2.13.23 Collaboration Diagram ของ Use Case Description ที่ 6A



รูปที่ 2-37 Collaboration Diagram ของ Use Case Description ที่ 6A

2.13.24 User Interface ของ Use Case Description ที่ 6A



รูปที่ 2-38 User Interface ของ Use Case Description ที่ 6A (ReorderScreen หลังยืนยันการสั่งซื้อ)

2.13.25 Use Case Description ที่ 7A: ยกเลิก Order

Goal: ให้ Super Admin สามารถยกเลิก Order ที่มีปัญหาหรือถูกลูกค้ายกเลิกได้ พร้อมคืนสต๊อกและยกเลิกกระบวนการที่เกี่ยวข้องอย่างถูกต้อง

Pre-Condition: Order มีอยู่ในระบบและยังไม่อยู่ในสถานะ "จัดส่งสำเร็จ" หรือ "ยกเลิกแล้ว"

Post-Condition: Order มีสถานะ "ยกเลิกแล้ว" พร้อม `cancel_reason` และ `cancelled_at` คืนสต๊อกตามแหล่งที่จัดส่ง (`fulfill_source`: คลัง RSL หรือคลังบริษัท) เฉพาะ Order ที่ถูกตัดสต๊อกแล้ว (`order_items.stock_deducted` = true) / Order ที่ส่งมอบ Delivery แล้วถูกบันทึกเป็น "รอดำเนินการพิเศษ" แทน

Actor: Super Admin, Marketplace (แหล่งคำขอยกเลิกจากลูกค้าผ่าน Rakuten ที่ 4A บันทึกไว้)

ตารางที่ 2-11 Use Case Description ที่ 7A: ยกเลิก Order

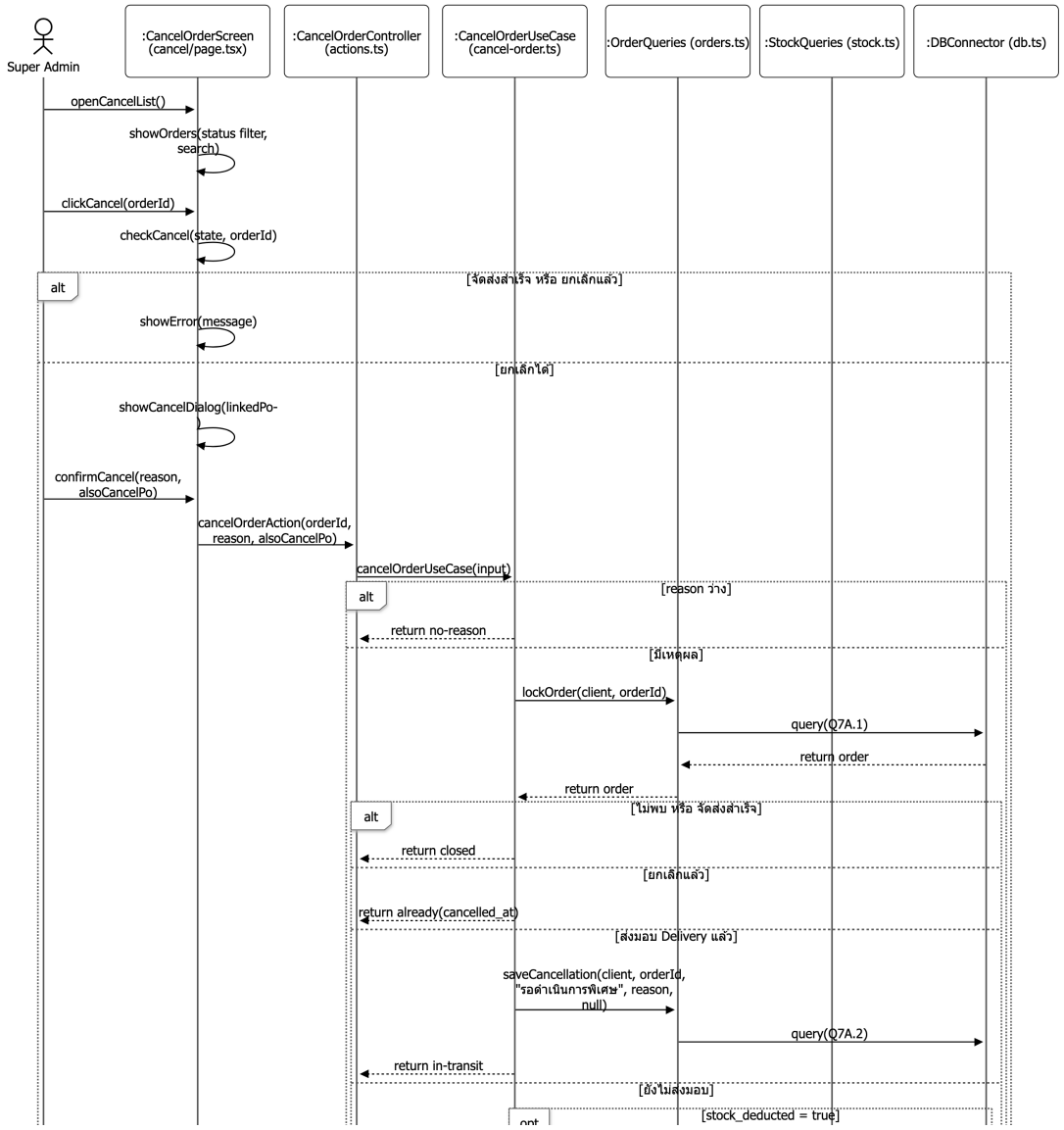
ลำดับ	User Action	System Response
1		หน้า `/orders/cancel` แสดงรายการ Order พร้อมสถานะปัจจุบัน เรียงใหม่สุดก่อน กรองตามสถานะและค้นหาด้วย Order ID, SKU หรือชื่อสินค้าได้ (ข้อมูลจาก read model ที่ layout โหลดผ่าน `loadAppState`) Order ที่ลูกค้าขอยกเลิกผ่าน Rakuten (`customer_cancel_requested` = true ซึ่ง 4A ตั้งไว้ตอนดึง Order) มีหมายเหตุ "ลูกค้าขอยกเลิกผ่าน Rakuten"
2	เลือก Order แล้วกด "ยกเลิก Order"	หน้าจอตรวจสถานะจากข้อมูลที่โหลดไว้ก่อนเปิดหน้าต่าง: หาก "จัดส่งสำเร็จ" จะแจ้งเตือนว่า "ไม่สามารถยกเลิก Order นี้ได้ เนื่องจากจัดส่งสำเร็จแล้ว/ถูกยกเลิกไปแล้ว" หาก "ยกเลิกแล้ว" ดูทางเลือก #4 นอกนั้นเปิดหน้าต่าง "ยกเลิก [Order ID]" ให้กรอก "เหตุผลการยกเลิก" และหากมีคำสั่งซื้อเติมสต๊อกผูกอยู่ แสดงตัวเลือก "ยกเลิกคำสั่งซื้อเติมสต๊อกนี้ด้วย" (ทางเลือก #2)

3	กรอกเหตุผลแล้วกด "ยืนยันการยกเลิก"	ระบบเรียก Server Action `cancelOrderAction` ⟶ `cancelOrderUseCase` ทำงานใน transaction เดียว
		1. ตรวจเหตุผล — ต้องไม่เป็นค่าว่าง หากว่างจะแจ้งเตือนว่า "กรุณาระบุเหตุผลการยกเลิก"
		2. ล็อก Order และอ่านสถานะล่าสุด — Q7A.1: SELECT
		`o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o`
		— หากไม่พบ หรือ "จัดส่งสำเร็จ" จะแจ้งเตือนว่า "ไม่สามารถยกเลิก Order นี้ได้ เนื่องจากจัดส่งสำเร็จแล้ว/ถูกยกเลิกไปแล้ว"
		— หาก "ยกเลิกแล้ว" ดูทางเลือก #4
		— หากส่งมอบ Delivery แล้ว (อยู่ระหว่างจัดส่ง, รอแจ้งเลขติดตาม, แจ้งเลขติดตามแล้ว, จัดส่งไม่สำเร็จ, รอดำเนินการพิเศษ) ดูทางเลือก #1 — Q7A.2:
		`UPDATE orders SET order_status = $2, cancel_reason = $3, cancelled_at = $4, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1`
		3. คืนสต๊อก (เฉพาะ `stock_deducted` = true) — คืนจำนวนเข้าสต๊อกตาม `fulfill_source` ('in_house' หรือ 'rsl') แล้วตั้ง

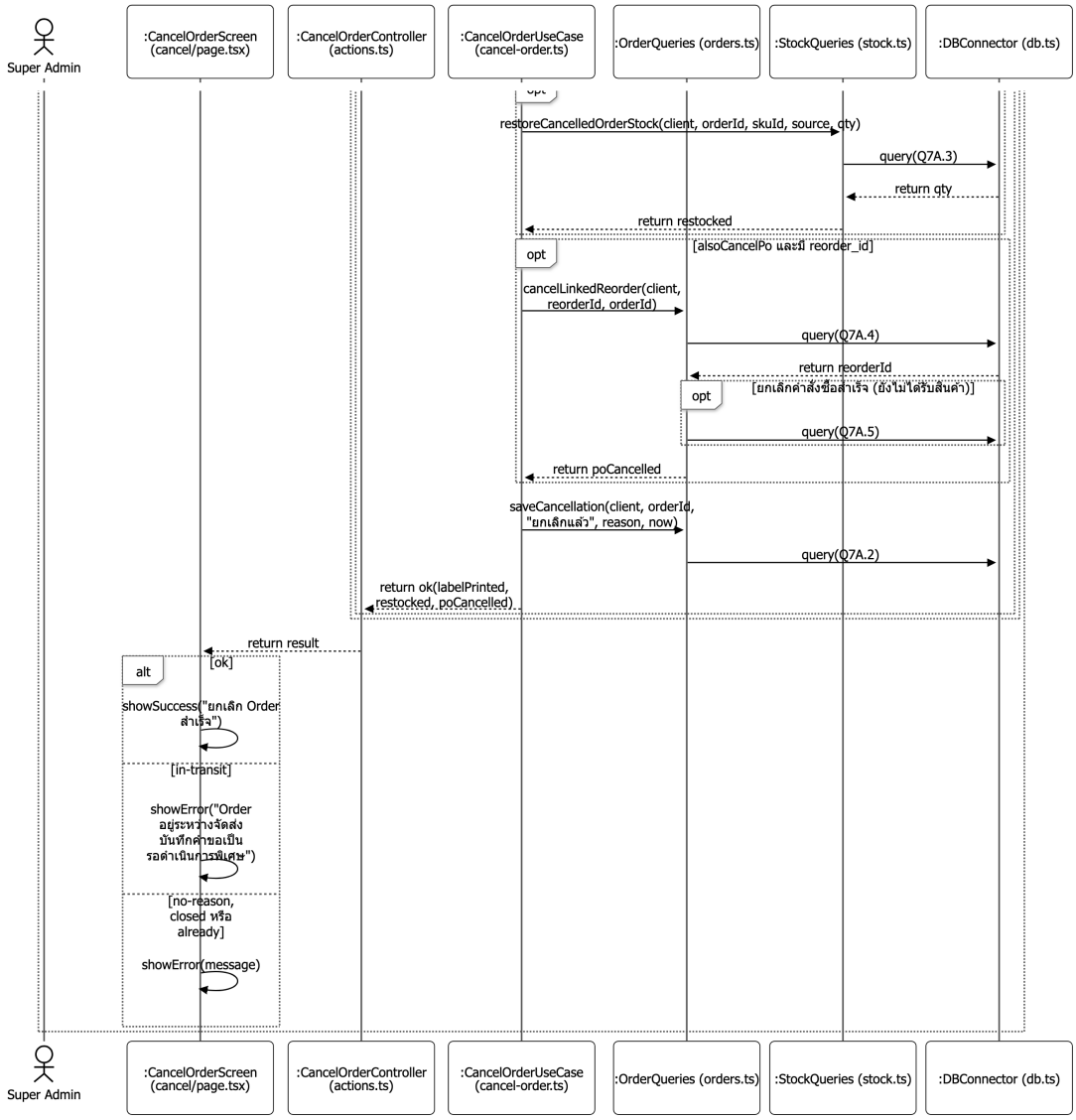
ลำดับ	User Action	System Response
		`stock_deducted` = false — **Q7A.3:** `WITH restored AS (UPDATE stock SET in_house_qty = in_house_qty + CASE WHEN $3 = 'in_house' THEN $4 ELSE 0 END, rsl_qty = CASE WHEN $3 = 'rsl' THEN COALESCE(rsl_qty, 0) + $4 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE sku_id = $2 AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = TRUE) RETURNING stock_id) UPDATE order_items SET stock_deducted = FALSE, deducted_at = NULL WHERE order_id = $1 AND EXISTS (SELECT 1 FROM restored) RETURNING qty` **4. ยกเลิกคำสั่งซื้อเติมสต๊อก** (เมื่อเลือกทางเลือก #2 ยกเลิกได้เฉพาะคำสั่งซื้อที่ยังไม่ได้รับสินค้า) — **Q7A.4:** `UPDATE reorders SET reorder_status = 'ยกเลิกแล้ว' WHERE reorder_id = $1 AND reorder_status IN ('รอส่งคำสั่งซื้อ', 'สั่งซื้อแล้ว') RETURNING reorder_id` แล้วส่ง Order อื่นที่ผูกกับคำสั่งซื้อเดียวกันกลับไป \"รอ Admin ตัดสินใจสั่งซื้อ\" (5A) พร้อมล้างการผูก — **Q7A.5:** `UPDATE orders SET order_status = 'รอ Admin ตัดสินใจสั่งซื้อ', manual_reason = NULL, reorder_id = NULL, reorder_decision = NULL, decided_qty = NULL, updated_at = CURRENT_TIMESTAMP WHERE reorder_id = $1 AND order_id <> $2 AND order_status IN ('รอส่งคำสั่งซื้อ', 'สั่งซื้อแล้ว')` **5. บันทึกการยกเลิก** — **Q7A.2** ($2='ยกเลิกแล้ว', $3=เหตุผล, $4=เวลาปัจจุบัน) แล้วแสดงข้อความ "ยกเลิก Order สำเร็จ" พร้อมรายละเอียด "คืนสต๊อก [n] ชิ้นกลับเข้าคลังแล้ว" / "ยกเลิกคำสั่งซื้อเติมสต๊อกที่เกี่ยวข้องแล้ว" ตามที่เกิดขึ้น หาก Order พิมพ์ใบปะสินค้าแล้วแต่ยังไม่ส่งมอบ แจ้งว่า "Order นี้พิมพ์ใบปะสินค้าแล้วแต่ยังไม่ส่งมอบ Delivery กรุณายกเลิกใบปะสินค้ากับผู้ให้บริการขนส่งด้วย"
ทางเลือก #1	Order ส่งมอบ Delivery ไปแล้ว	**Q7A.2** ($2='รอดำเนินการพิเศษ', $3=เหตุผล, $4=NULL) ไม่คืนสต๊อก แจ้งเตือนว่า "Order นี้อยู่ระหว่างการจัดส่งแล้ว ไม่สามารถยกเลิกในระบบได้ทันที กรุณาประสานงานกับผู้ให้บริการขนส่งเพื่อเรียกพัสดุคืน" และ "บันทึกคำขอยกเลิกเป็น รอดำเนินการพิเศษ แล้ว"
ทางเลือก #2	Order มีคำสั่งซื้อเติมสต๊อก (`orders.reorder_id`) ผูกอยู่	หน้าต่างแสดง "มีคำสั่งซื้อเติมสต๊อกที่เกี่ยวข้องกับ Order นี้ ([เลขที่])" หาก Super Admin ติ๊ก "ยกเลิกคำสั่งซื้อเติมสต๊อกนี้ด้วย" ระบบทำ **Q7A.4** ตั้ง `reorder_status`='ยกเลิกแล้ว' ในขั้นตอนที่ 3 ยกเลิกได้เฉพาะคำสั่งซื้อที่ "รอส่งคำสั่งซื้อ" หรือ "สั่งซื้อแล้ว" (ได้รับสินค้าแล้วยกเลิกไม่ได้) Order อื่นที่ผูกกับคำสั่งซื้อเดียวกันกลับไปสถานะ "รอ Admin ตัดสินใจสั่งซื้อ" เพื่อให้ Super Admin ตัดสินใจใหม่ (5A) ด้วย **Q7A.5** ถ้าไม่ติ๊ก คำสั่งซื้อคงเดิม
ทางเลือก #3	ลูกค้าขอยกเลิกผ่าน Rakuten	4A ตั้ง `customer_cancel_requested` = true ให้ Order นั้น หน้าจอแสดงหมายเหตุ "ลูกค้าขอยกเลิกผ่าน Rakuten" ระบบไม่ยกเลิกเอง Super Admin ต้องยืนยันตามขั้นตอนที่ 2-3

ลำดับ	User Action	System Response
ทางเลือก #4	กดยกเลิกซ้ำ Order ที่ยกเลิกไปแล้ว	แจ้งเตือนว่า "Order นี้ถูกยกเลิกไปแล้วเมื่อ [วันที่/เวลา]" (ใช้ `cancelled_at`) และไม่บันทึกซ้ำ

2.13.26 Sequence Diagram ของ Use Case Description ที่ 7A



รูปที่ 2-39 Sequence Diagram ของ Use Case Description ที่ 7A (1/2)

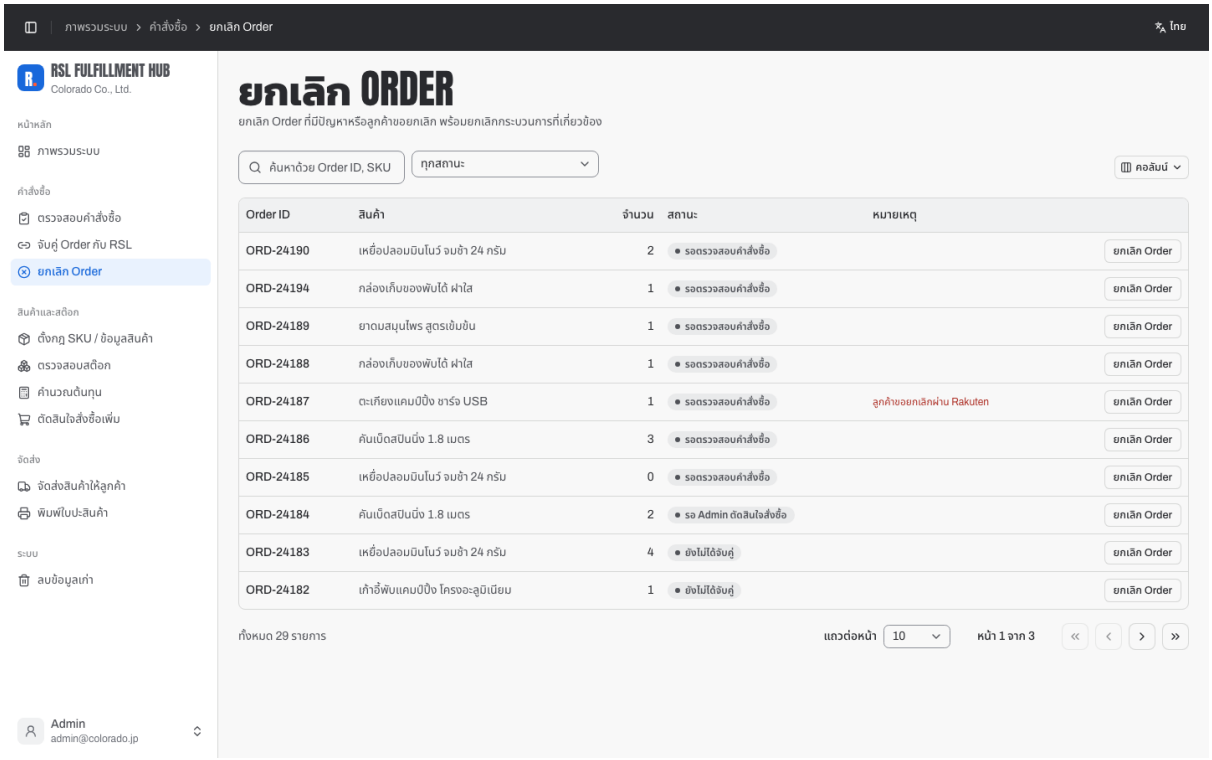


รูปที่ 2-40 Sequence Diagram ของ Use Case Description ที่ 7A (2/2)

2.13.27 Collaboration Diagram ของ Use Case Description ที่ 7A

รูปที่ 2-41 Collaboration Diagram ของ Use Case Description ที่ 7A

2.13.28 User Interface ของ Use Case Description ที่ 7A



รูปที่ 2-42 User Interface ของ Use Case Description ที่ 7A (CancelOrderScreen)

2.13.29 Use Case Description ที่ 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery

Goal: ให้ Order ที่พิมพ์ใบปะสินค้าแล้วถูกส่งมอบให้บริษัทขนส่ง (Delivery) พร้อมบันทึกหมายเลขติดตามพัสดุ และติดตามสถานะจากผู้ขนส่งจนปิดงานเป็น "จัดส่งสำเร็จ"

Pre-Condition: Order มีสถานะ "พิมพ์ใบปะสินค้าแล้ว" หรือ "รอส่งมอบ" และใบปะสินค้าทุกใบของ Order พิมพ์แล้ว (`labels.printed_at` ไม่ว่าง)

Post-Condition: Order มีสถานะ "อยู่ระหว่างจัดส่ง" พร้อม `tracking_number`, `carrier_name`, `dispatched_at` และ `delivery_status`='in_transit' จากนั้นระบบอัปเดต `delivery_status` ตามผู้ขนส่ง Order จะเป็น "จัดส่งสำเร็จ" เมื่อครบ 3 เงื่อนไขเท่านั้น คือ ผู้ขนส่งยืนยันส่งถึง (`delivered`) แจ้งเลขติดตามให้ลูกค้าแล้ว (5S) และตัดสต๊อกแล้ว (7S)

Actor: Super Admin, Delivery

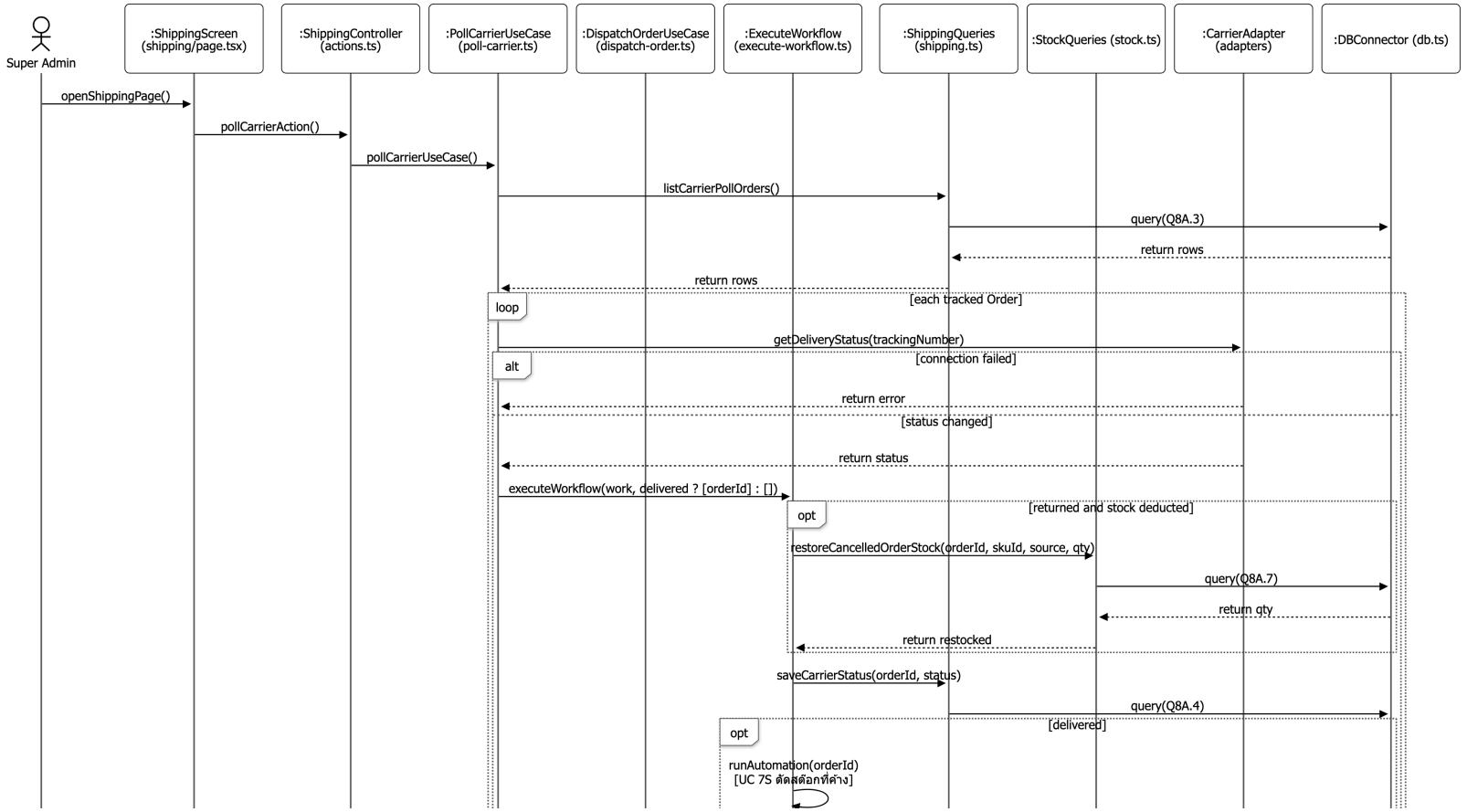
ตารางที่ 2-12 Use Case Description ที่ 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery

ลำดับ	User Action	System Response
1		หน้า "จัดส่งสินค้าให้ลูกค้า" (`/shipping`) แสดง 2 รายการ คือ "รอส่งมอบให้ Delivery" (Order สถานะ "พิมพ์ใบปะสินค้าแล้ว" และ "รอส่งมอบ") และ "ส่งมอบแล้ว" (แสดงหมายเลขติดตาม ผู้ขนส่ง สถานะจากขนส่ง สถานะแจ้งลูกค้า และการตัดสต๊อก) เมื่อเปิดหน้า ระบบดึงสถานะจากผู้ให้บริการขนส่งทันที (ขั้นตอนที่ 5)
2	Super Admin รวบรวมพัสดุ เลือก Order แล้วกด "ส่งมอบให้ Delivery"	แสดงหน้าต่าง "ยืนยันรายการพัสดุที่ส่งมอบ" พร้อมข้อความ "ตรวจนับพัสดุให้ตรงกับรายการก่อนยืนยัน"
3	กดยืนยันรายการพัสดุที่ส่งมอบ	ระบบเรียก `dispatchOrdersAction` ⟶ `dispatchOrdersUseCase` ทำทีละ Order **1. ล็อกและตรวจสอบ Order** — Q8A.1: `SELECT o.order_id, o.order_status, o.tracking_number, o.carrier_name, o.delivery_status, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id) AND NOT EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NULL) AS labels_printed FROM orders o WHERE o.order_id = $1 FOR UPDATE OF o` — Order ต้องมีสถานะ "พิมพ์ใบปะสินค้าแล้ว" หรือ "รอส่งมอบ" และ `labels_printed` เป็นจริง หากไม่ผ่านจะแจ้งเตือนว่า "Order นี้ยังไม่ได้พิมพ์ใบปะสินค้า ไม่สามารถส่งมอบได้" **2. ขอหมายเลขติดตามพัสดุจาก Delivery** — เรียก `CarrierAdapter.getTrackingInfo(orderId)` ต้องได้ `trackingNumber` และ `carrierName` ที่ไม่ว่าง **3. บันทึกการส่งมอบ** — ล็อก Order ซ้ำ (Q8A.1) ใน transaction ใหม่ แล้วบันทึก Q8A.2: `UPDATE orders SET tracking_number = $2, carrier_name = $3, dispatched_at = CURRENT_TIMESTAMP, delivery_status = 'in_transit', tracking_notified = FALSE, tracking_notified_at = NULL, order_status = 'อยู่ระหว่างจัดส่ง', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status IN ('พิมพ์ใบปะสินค้าแล้ว', 'รอส่งมอบ')` ระบบเปลี่ยนสถานะเป็น "อยู่ระหว่างจัดส่ง" ตั้ง `delivery_status`='in_transit' และ `tracking_notified`=false แล้วแจ้งว่า "ส่งมอบ [Order ID] ให้ [ผู้ขนส่ง] แล้ว" พร้อม "หมายเลขติดตามพัสดุ [เลข]"
4		Order ที่ส่งมอบแล้วย้ายไปอยู่ในรายการ "ส่งมอบแล้ว" รอ Super Admin กด "ส่งเลขติดตามให้ลูกค้า" (**Use Case 5S**)

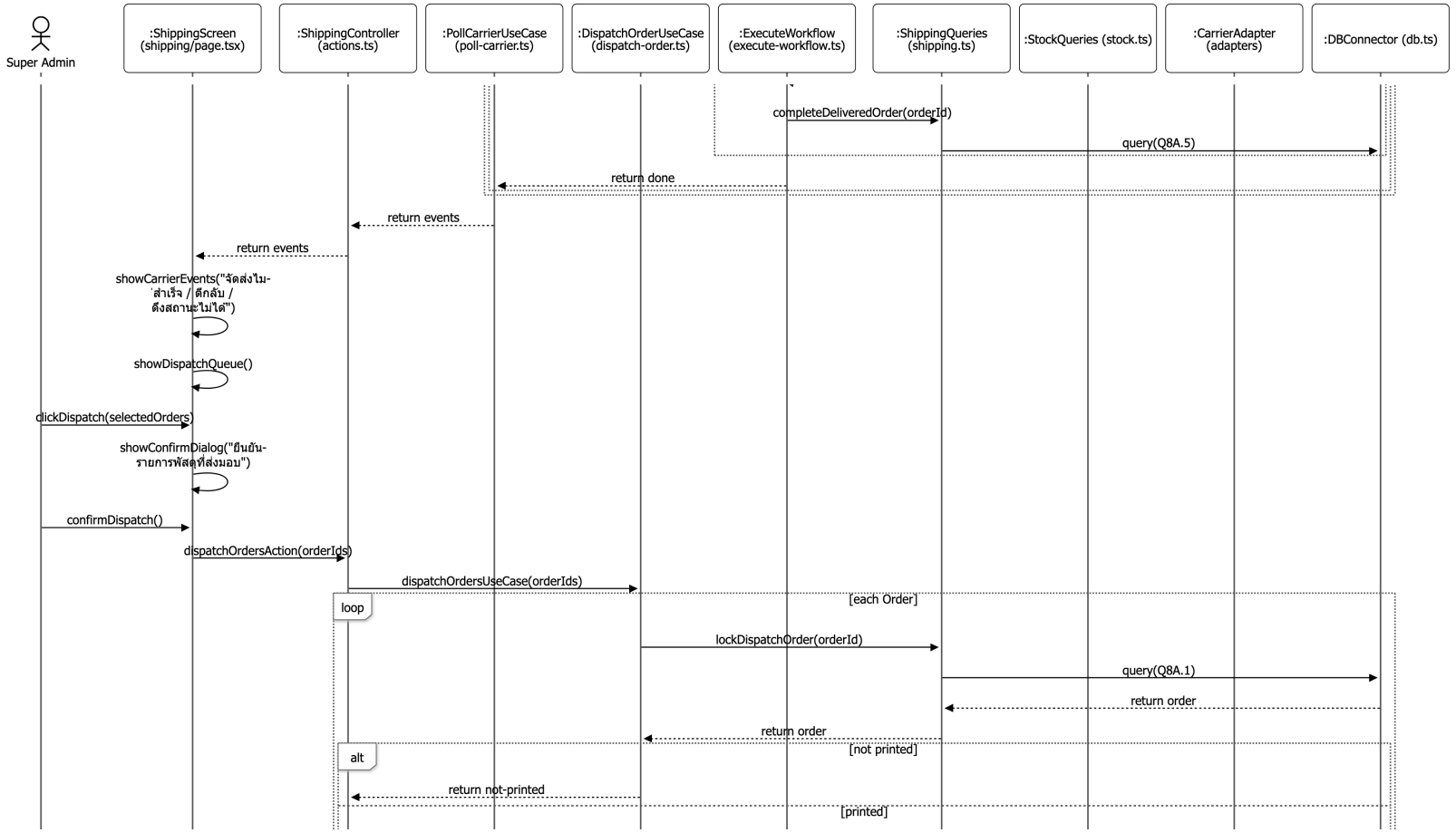
ลำดับ	User Action	System Response
5	เปิดหน้า `/shipping` (ระบบเรียก `pollCarrierAction` ให้อัตโนมัติ)	**ติดตามสถานะจากผู้ขนส่ง** — `pollCarrierUseCase` ดึง Order ที่มีหมายเลขติดตามและยังต้องติดตาม Q8A.3: `SELECT o.order_id, o.order_status, o.tracking_number, o.delivery_status, o.tracking_notified, item.stock_deducted, item.sku_id, item.fulfill_source, item.qty FROM orders o INNER JOIN LATERAL (SELECT * FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE WHERE o.tracking_number IS NOT NULL AND o.order_status IN ('อยู่ระหว่างจัดส่ง', 'รอแจ้งเลขติดตาม', 'แจ้งเลขติดตามแล้ว', 'จัดส่งสำเร็จ', 'รอดำเนินการพิเศษ') ORDER BY o.order_id` สำหรับแต่ละ Order เรียก `CarrierAdapter.getDeliveryStatus(tracking_number)` หากสถานะเท่าเดิม (หรือเคย `delivered` แล้วได้ `in_transit` กลับมา) ระบบข้ามไป หากสถานะเปลี่ยน บันทึก Q8A.4: `UPDATE orders SET delivery_status = $2, order_status = CASE WHEN $2 = 'failed' THEN 'จัดส่งไม่สำเร็จ' WHEN $2 = 'returned' THEN 'ตีกลับ/คืนสินค้า' WHEN $2 = 'delivered' AND tracking_notified = TRUE AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE) THEN 'จัดส่งสำเร็จ' ELSE order_status END, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND (delivery_status IS DISTINCT FROM 'delivered' OR $2 <> 'in_transit')` — `failed` ⟶ สถานะ Order "จัดส่งไม่สำเร็จ" — `returned` ⟶ สถานะ Order "ตีกลับ/คืนสินค้า" — `delivered` ⟶ เก็บ `delivery_status`='delivered' และเปลี่ยนเป็น "จัดส่งสำเร็จ" เฉพาะเมื่อ `tracking_notified`=true และ `order_items.stock_deducted`=true ครบทุกรายการแล้ว มิฉะนั้นคงสถานะเดิมไว้รอ 5S/7S การบันทึกสถานะแต่ละ Order ทำผ่าน `executeWorkflow` เมื่อได้ `delivered` ระบบรัน 7S (ตัดสต๊อกที่ค้าง) และปิด Order เป็น "จัดส่งสำเร็จ" ด้วย Q8A.5 ใน transaction เดียวกัน
6		**ปิดงานหลังตัดสต๊อก** — ถ้าผู้ขนส่งยืนยันส่งถึงก่อนตัดสต๊อก เมื่อ Super Admin แจ้งเลขติดตาม (5S) แล้ว `executeWorkflow` จะรัน 7S ใน transaction เดียวกัน และเมื่อตัดสต๊อกได้จะปิด Order Q8A.5: `UPDATE orders SET order_status = 'จัดส่งสำเร็จ', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'แจ้งเลขติดตามแล้ว' AND tracking_notified = TRUE AND delivery_status = 'delivered' AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE)` เป็น "จัดส่งสำเร็จ"

ลำดับ	User Action	System Response
ทางเลือก #1	ไม่พบหมายเลขติดตามพัสดุจากผู้ให้บริการขนส่ง หรือเรียก Delivery ไม่สำเร็จ	บันทึก Q8A.6: `UPDATE orders SET order_status = 'รอส่งมอบ', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'พิมพ์ใบปะสินค้าแล้ว'` เปลี่ยนสถานะเป็น "รอส่งมอบ" และแจ้งเตือนว่า "ไม่พบหมายเลขติดตามพัสดุ กรุณาตรวจสอบกับผู้ให้บริการขนส่ง" Order ยังอยู่ในรายการรอส่งมอบ ให้กดส่งมอบใหม่ได้
ทางเลือก #2	ผู้ขนส่งแจ้งว่าจัดส่งไม่สำเร็จ (ที่อยู่ไม่ถูกต้อง / ติดต่อลูกค้าไม่ได้)	บันทึกตามขั้นตอนที่ 5 (Q8A.4) สถานะเป็น "จัดส่งไม่สำเร็จ" และแจ้งเตือนว่า "[Order ID] จัดส่งไม่สำเร็จ กรุณาตรวจสอบและดำเนินการแก้ไข เช่น ติดต่อลูกค้าหรือส่งใหม่"
ทางเลือก #3	ลูกค้าปฏิเสธรับพัสดุหรือขอคืนสินค้า (ผู้ขนส่งแจ้ง `returned`)	หาก Order ตัดสต๊อกไปแล้ว ระบบคืนสต๊อกตาม `fulfill_source` ใน transaction เดียวกัน Q8A.7: `WITH restored AS (UPDATE stock SET in_house_qty = in_house_qty + CASE WHEN $3 = 'in_house' THEN $4 ELSE 0 END, rsl_qty = CASE WHEN $3 = 'rsl' THEN COALESCE(rsl_qty, 0) + $4 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE sku_id = $2 AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = TRUE) RETURNING stock_id) UPDATE order_items SET stock_deducted = FALSE, deducted_at = NULL WHERE order_id = $1 AND EXISTS (SELECT 1 FROM restored) RETURNING qty` แล้วบันทึกสถานะ "ตีกลับ/คืนสินค้า" (Q8A.4) แจ้งเตือนว่า "[Order ID] ตีกลับ/คืนสินค้า · คืนสต๊อก [จำนวน] ชิ้นแล้ว"
ทางเลือก #4	การเชื่อมต่อกับระบบติดตามของผู้ให้บริการขนส่งล้มเหลว	แจ้งเตือนครั้งเดียวว่า "ไม่สามารถดึงสถานะจากผู้ให้บริการขนส่งได้ กรุณาลองใหม่ภายหลัง" คงสถานะล่าสุดไว้ และดึงใหม่เมื่อเปิดหน้า `/shipping` ครั้งถัดไป
ทางเลือก #5	Order ถูกขอยกเลิกหลังส่งมอบให้ Delivery แล้ว (7A)	Order อยู่ในสถานะ "รอดำเนินการพิเศษ" ระบบยังติดตามสถานะจากผู้ขนส่งต่อ (อยู่ในเงื่อนไขของ Q8A.3) จนได้ผลการจัดส่ง

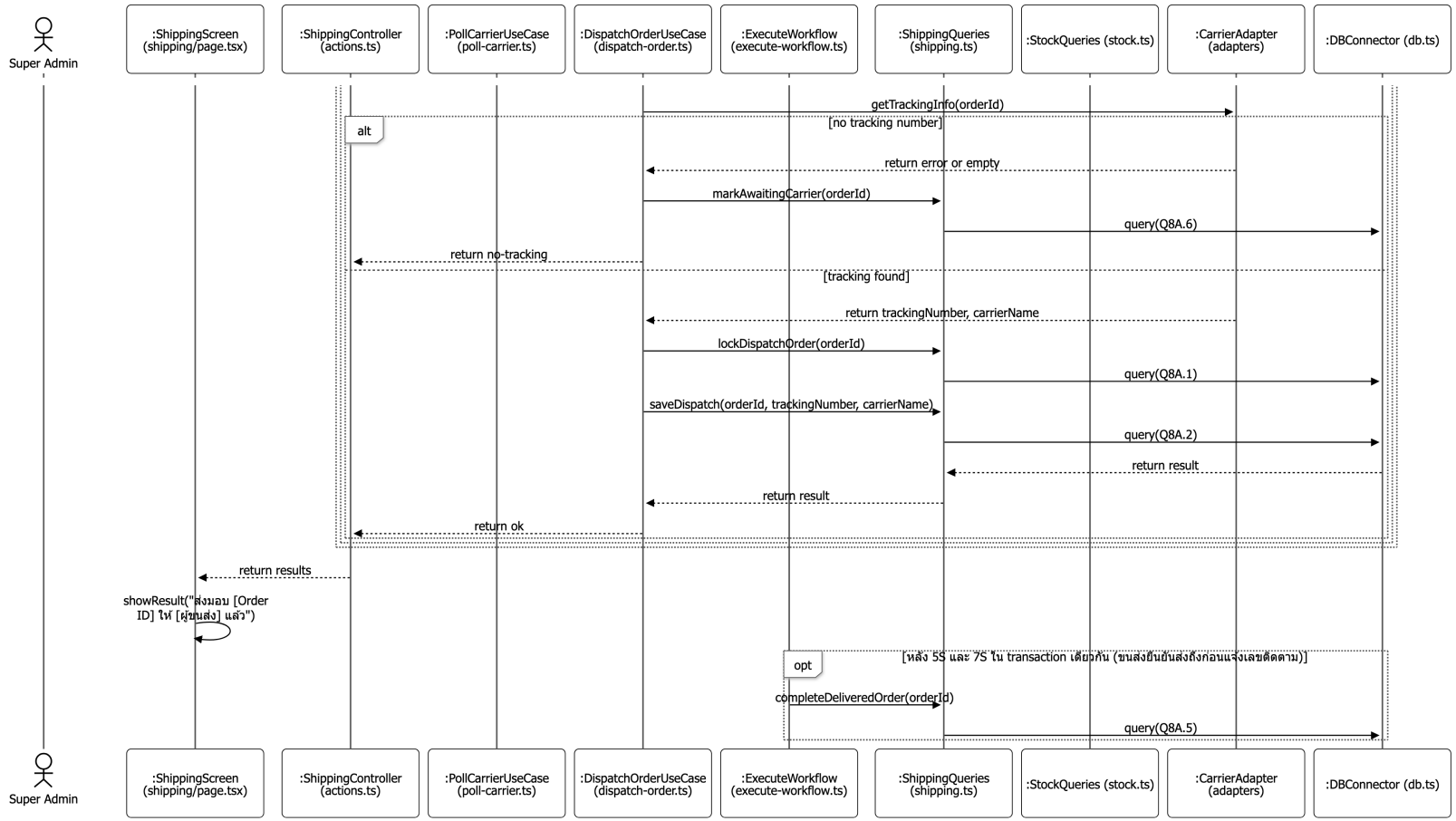
2.13.30 Sequence Diagram ของ Use Case Description ที่ 8A



รูปที่ 2-43 Sequence Diagram ของ Use Case Description ที่ 8A (1/3)

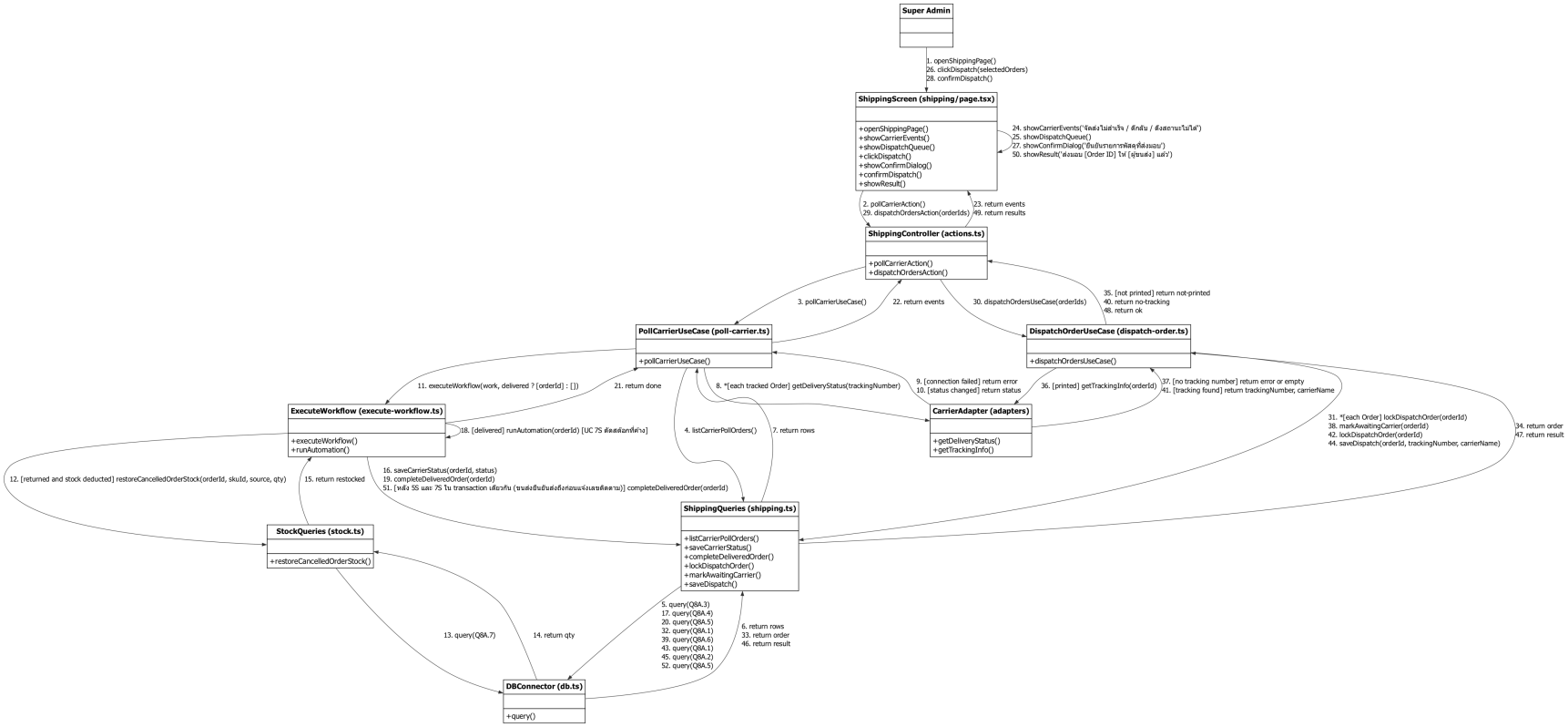


รูปที่ 2-44 Sequence Diagram ของ Use Case Description ที่ 8A (2/3)



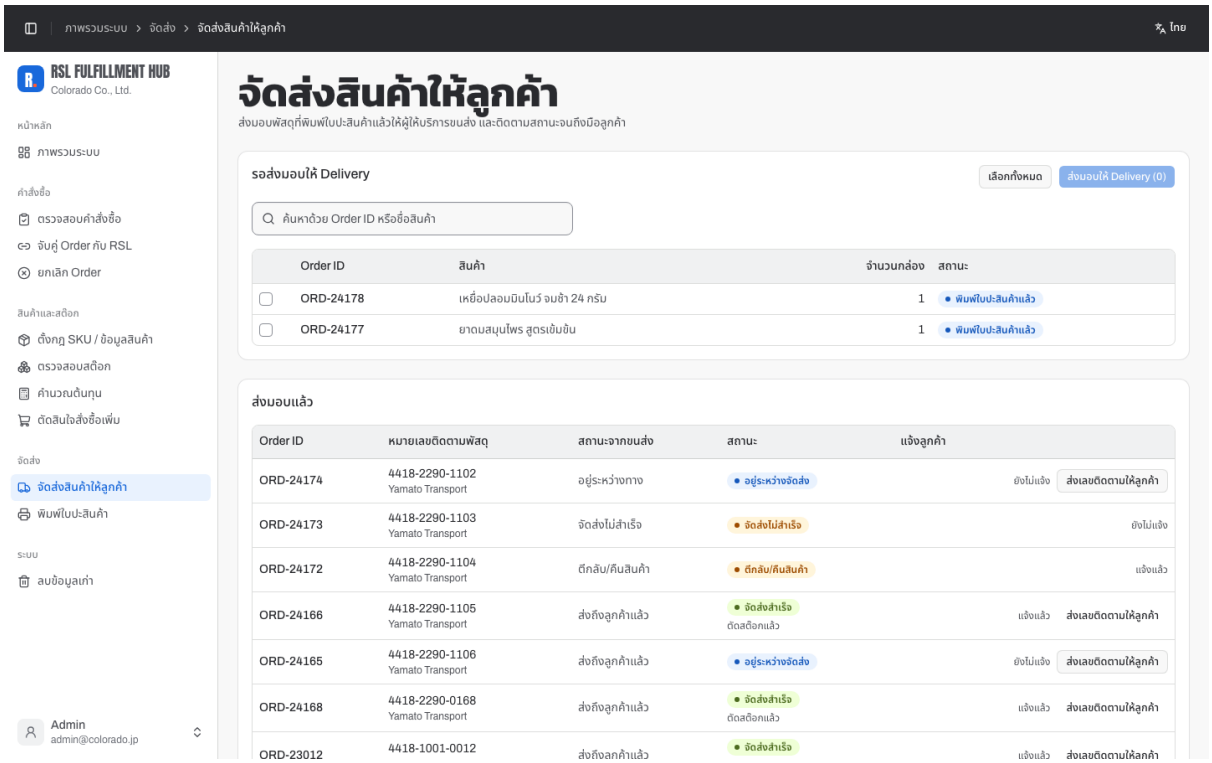
รูปที่ 2-45 Sequence Diagram ของ Use Case Description ที่ 8A (3/3)

2.13.31 Collaboration Diagram ของ Use Case Description ที่ 8A



รูปที่ 2-46 Collaboration Diagram ของ Use Case Description ที่ 8A

2.13.32 User Interface ของ Use Case Description ที่ 8A



รูปที่ 2-47 User Interface ของ Use Case Description ที่ 8A (ShippingScreen)

2.13.33 Use Case Description ที่ 9A: พิมพ์ใบปะสินค้า

Goal: ให้ Super Admin พิมพ์ใบปะสินค้า (Shipping Label) ที่ระบบจัดรูปแบบไว้แล้ว (6S) ออกเครื่องพิมพ์ และบันทึกประวัติการพิมพ์ทุกครั้ง

Pre-Condition: Order มีสถานะ "รอพิมพ์ใบปะสินค้า" และมีใบปะสินค้าในตาราง `labels` (สถานะ `ready`) ที่ 6S สร้างไว้แล้ว

Post-Condition: ใบปะสินค้าทุกใบของ Order มี `label_status`='printed' และ `printed_at` และ Order มีสถานะ "พิมพ์ใบปะสินค้าแล้ว" ทุกการพิมพ์ (รวมพิมพ์ซ้ำและพิมพ์ไม่สำเร็จ) ถูกบันทึกใน `label_print_history` (ใบปะของ Order ที่บริษัทจัดส่งเองไม่มีเลข RSL รายงานใบปะแสดง "—" ที่ช่อง RSL และใช้ [Order ID]-[กล่อง] เป็นรหัสสแกน)

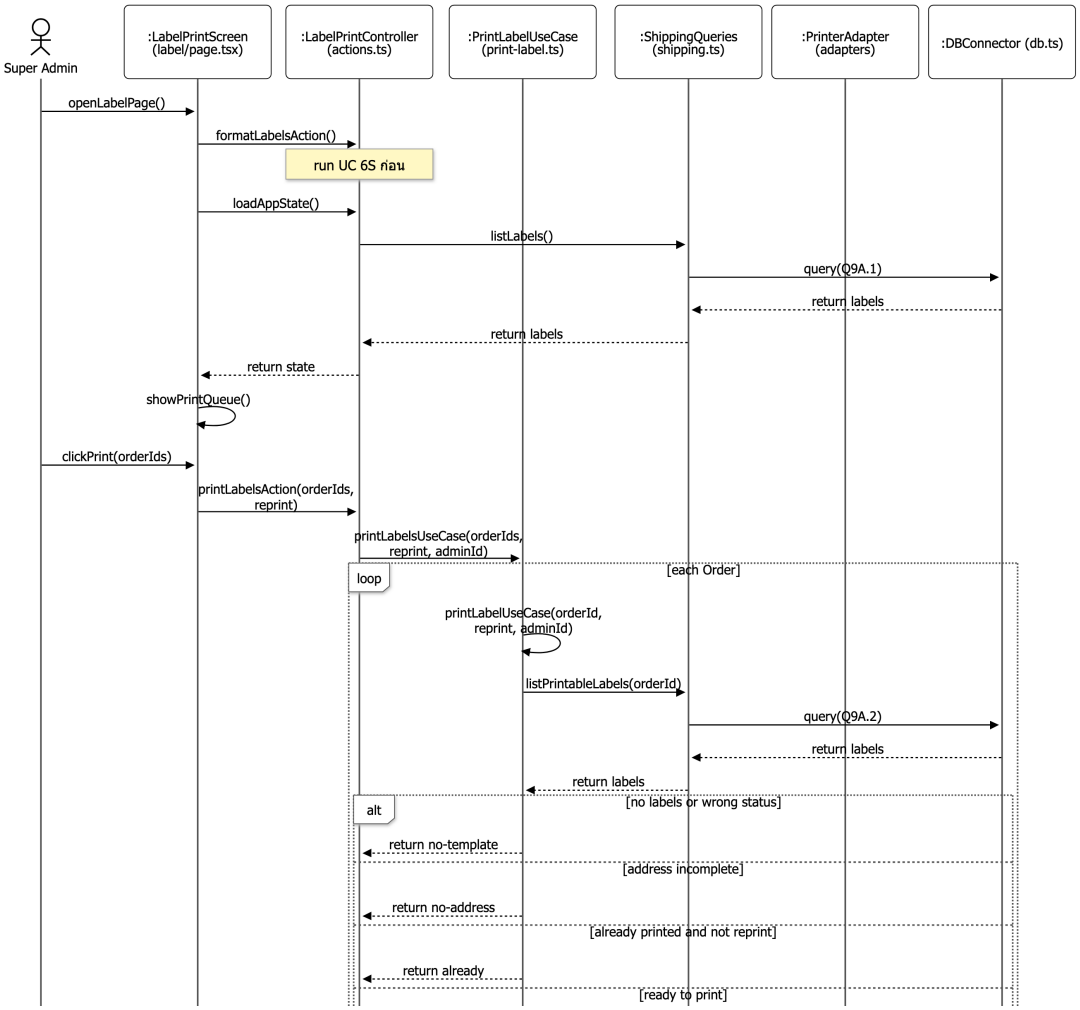
Actor: Super Admin

ตารางที่ 2-13 Use Case Description ที่ 9A: พิมพ์ใบปะสินค้า

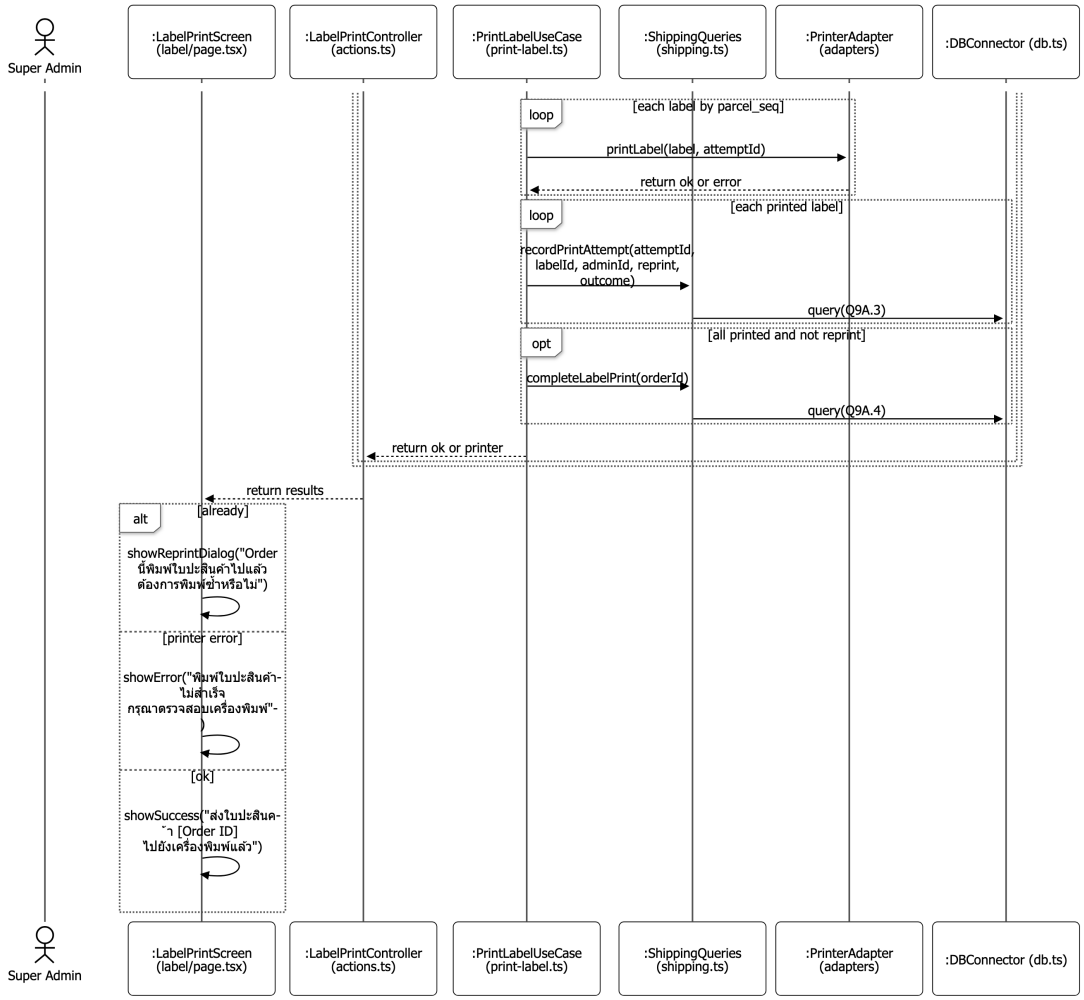
ลำดับ	User Action	System Response
1		เมื่อเปิดหน้า "พิมพ์ใบปะสินค้า" (`/shipping/label`) ระบบเรียก `formatLabelsAction` ให้ 6S สร้างใบปะของ Order ที่รอจัดรูปแบบก่อน แล้วแสดงรายการ "รอพิมพ์" และ "พิมพ์แล้ว" พร้อมรูปแบบใบปะ จำนวนกล่อง และเวลาพิมพ์ **Q9A.1:** `SELECT order_id, MAX(label_template) AS label_template, MAX(parcel_total)::integer AS parcel_total, MAX(printed_at) AS printed_at FROM labels GROUP BY order_id ORDER BY order_id`
2	Super Admin กด "พิมพ์ใบปะสินค้า" ที่ Order หรือกด "พิมพ์ใบปะสินค้าทั้งหมด"	ระบบเรียก `printLabelsAction(orderIds, reprint=false)` ⟶ `printLabelsUseCase` ซึ่งเรียก `printLabelUseCase` ทีละ Order
3		**1. ตรวจสอบใบปะสินค้าของ Order** — **Q9A.2:** `SELECT l.label_id, l.order_id, l.label_template, l.rsl_no, l.parcel_seq, l.parcel_total, o.order_status, o.shipping_address, o.shipping_method FROM labels l INNER JOIN orders o ON o.order_id = l.order_id WHERE l.order_id = $1 ORDER BY l.parcel_seq` — หากไม่พบใบปะสินค้า หรือ Order ไม่อยู่ในสถานะ "รอพิมพ์ใบปะสินค้า" จะแจ้งเตือนว่า "ไม่พบรูปแบบใบปะสินค้าที่เหมาะสม กรุณาตั้งค่า Label Template ก่อน" — หาก `shipping_address` หรือ `shipping_method` ว่าง จะแจ้งเตือนว่า "ข้อมูลที่อยู่จัดส่งไม่ครบถ้วน ไม่สามารถพิมพ์ใบปะสินค้าได้" — หาก Order มีสถานะ "พิมพ์ใบปะสินค้าแล้ว" ระบบถามว่า "Order นี้พิมพ์ใบปะสินค้าไปแล้ว ต้องการพิมพ์ซ้ำหรือไม่" (ทางเลือก #3) **2. ส่งพิมพ์** — สร้าง `print_attempt_id` แล้วเรียก `PrinterAdapter.printLabel(label, attemptId)` ทีละใบตาม `parcel_seq` หยุดทันทีเมื่อใบใดพิมพ์ไม่สำเร็จ **3. บันทึกผล** (transaction เดียว) — บันทึกประวัติทุกใบที่สั่งพิมพ์ **Q9A.3:** `INSERT INTO label_print_history ( label_print_history_id, print_attempt_id, label_id, admin_id, is_reprint, outcome, error_code ) VALUES ($1, $2, $3, $4, $5, $6, $7) ON CONFLICT (print_attempt_id, label_id) DO NOTHING` เมื่อพิมพ์ครบทุกใบ ระบบอัปเดตใบปะเป็น `printed` และสถานะ Order เป็น "พิมพ์ใบปะสินค้าแล้ว" **Q9A.4:** `WITH updated_labels AS ( UPDATE labels SET label_status = 'printed', printed_at = CURRENT_TIMESTAMP WHERE order_id = $1 ) UPDATE orders SET order_status = 'พิมพ์ใบปะสินค้าแล้ว', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอพิมพ์ใบปะสินค้า'`

ลำดับ	User Action	System Response
		แจ้งว่า "ส่งใบปะสินค้า [Order ID] ไปยังเครื่องพิมพ์แล้ว" พร้อม "รูปแบบ [template] · [จำนวน] ใบ" กรณีพิมพ์ทั้งหมดแสดงสรุป "พิมพ์สำเร็จ [n] · ไม่สำเร็จ [m]" แล้ว Order ไปรอส่งมอบที่ **Use Case 8A**
ทางเลือก #1	ไม่พบใบปะสินค้า/Label Template ของ Order	แจ้งเตือนว่า "ไม่พบรูปแบบใบปะสินค้าที่เหมาะสม กรุณาตั้งค่า Label Template ก่อน" ไม่เปลี่ยนแปลงข้อมูล (Order ที่ไม่มี template ถูก 6S ส่งเข้า "รอดำเนินการด้วยตนเอง" เหตุผล `template` ไว้แล้ว)
ทางเลือก #2	เครื่องพิมพ์ไม่พร้อมใช้งาน หรือพิมพ์ไม่สำเร็จ	บันทึกประวัติใบที่ล้มเหลวด้วย `outcome`='failed' และ `error_code` (**Q9A.3**) ไม่เรียก **Q9A.4** แจ้งเตือนว่า "พิมพ์ใบปะสินค้าไม่สำเร็จ กรุณาตรวจสอบเครื่องพิมพ์" Order คงสถานะ "รอพิมพ์ใบปะสินค้า" ให้กดพิมพ์ใหม่ได้
ทางเลือก #3	Super Admin ต้องการพิมพ์ใบปะสินค้าซ้ำ (เช่น ใบเดิมเสียหาย)	ระบบแสดงหน้าต่างยืนยัน "ระบบจะบันทึก Log การพิมพ์ซ้ำพร้อมเวลาและผู้ดำเนินการ" เมื่อกด "ยืนยันพิมพ์ซ้ำ" ระบบเรียก `printLabelsAction(orderIds, reprint=true)` พิมพ์ทุกใบอีกครั้ง บันทึกประวัติด้วย `is_reprint`=true และ `admin_id` (**Q9A.3**) ไม่เปลี่ยนสถานะ Order แล้วแจ้งว่า "พิมพ์ใบปะสินค้า [Order ID] ซ้ำแล้ว"
ทางเลือก #4	Order มีหลายกล่อง (Multi-parcel)	6S สร้างใบปะแยกต่อกล่อง (`parcel_seq/parcel_total` เช่น 1/2, 2/2) ระบบพิมพ์ทุกใบตามลำดับและบันทึกประวัติแยกทีละใบ Order จะเป็น "พิมพ์ใบปะสินค้าแล้ว" เมื่อพิมพ์ครบทุกใบเท่านั้น

2.13.34 Sequence Diagram ของ Use Case Description ที่ 9A

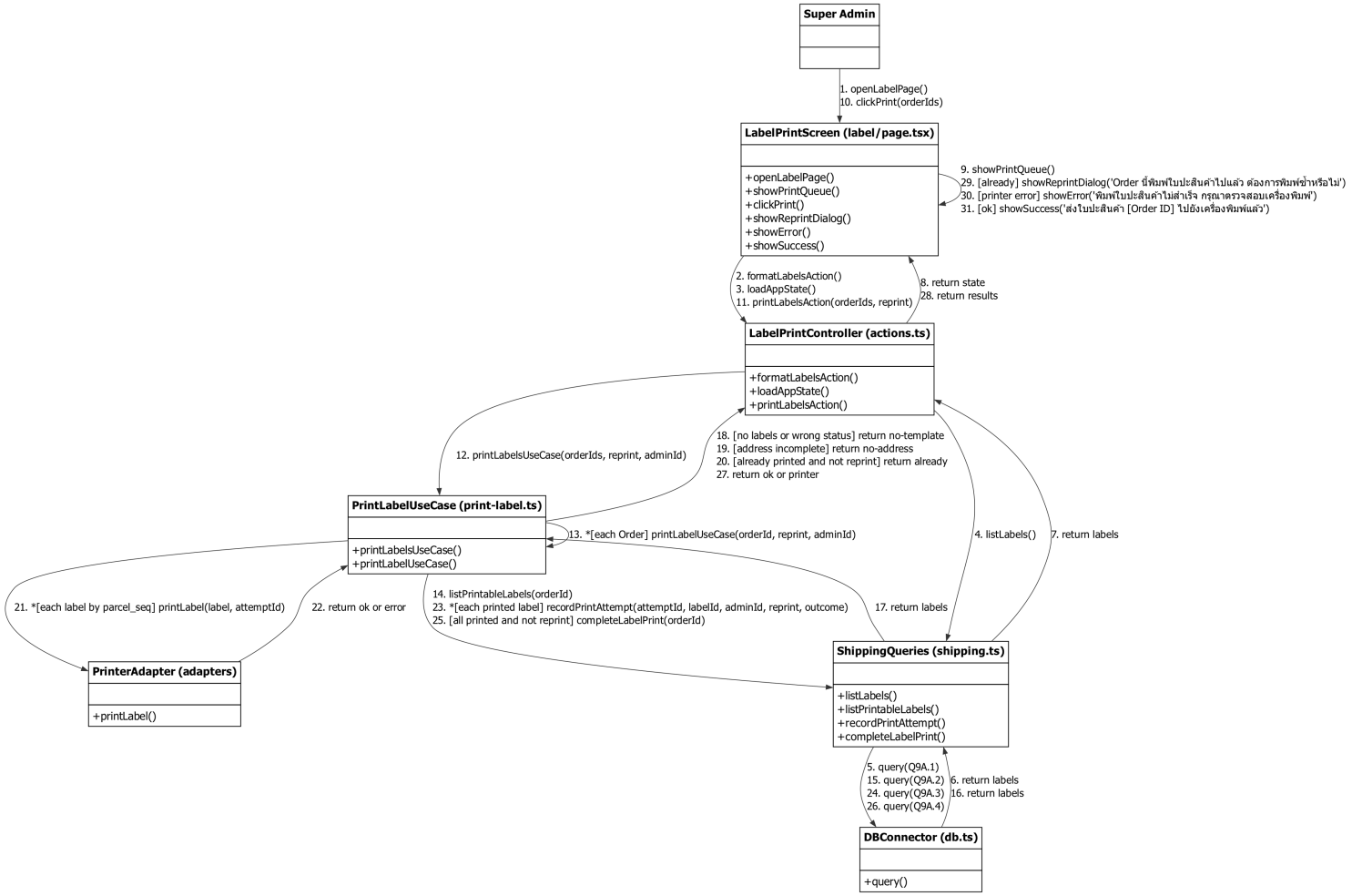


รูปที่ 2-48 Sequence Diagram ของ Use Case Description ที่ 9A (1/2)



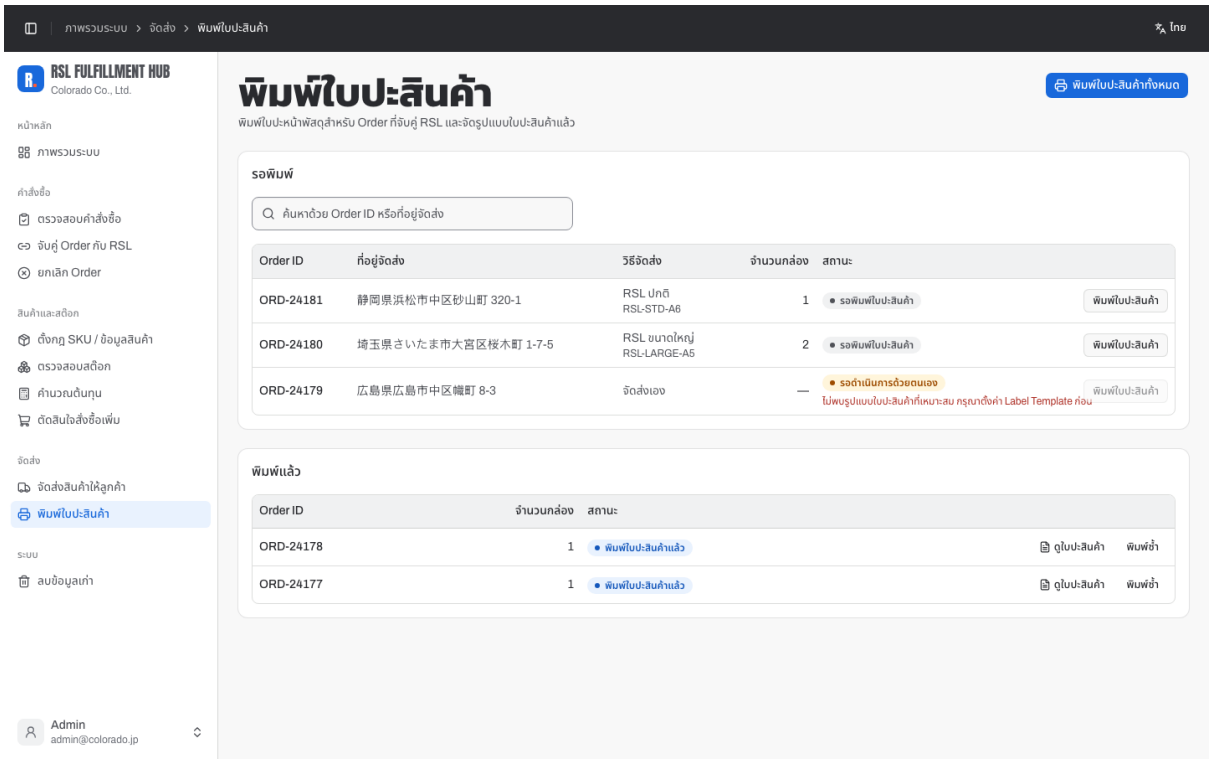
รูปที่ 2-49 Sequence Diagram ของ Use Case Description ที่ 9A (2/2)

2.13.35 Collaboration Diagram ของ Use Case Description ที่ 9A



รูปที่ 2-50 Collaboration Diagram ของ Use Case Description ที่ 9A

2.13.36 User Interface ของ Use Case Description ที่ 9A



รูปที่ 2-51 User Interface ของ Use Case Description ที่ 9A (LabelPrintScreen)

2.13.37 Use Case Description ที่ 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)

Goal: ให้ Super Admin ลบข้อมูลเก่าที่เสร็จสิ้นแล้วออกจากระบบ เพื่อลดปริมาณข้อมูลสะสมและรักษาประสิทธิภาพของระบบ

Pre-Condition: Super Admin เข้าสู่ระบบแล้ว ข้อมูลที่จะลบต้องสิ้นสุดแล้ว ได้แก่ Order "จัดส่งสำเร็จ" หรือ "ยกเลิกแล้ว" และ Reorder "ยกเลิกแล้ว" "ได้รับสินค้าแล้ว" หรือ "สั่งซื้อแล้ว" ที่เลยวันรับสินค้าและไม่ผูกกับ Order ที่ยังไม่ปิด

Post-Condition: ข้อมูล Order, Cost (ประวัติต้นทุน), Label และ Reorder ที่เลือกถูกลบใน transaction เดียว Order ที่ลบถูกเก็บ snapshot (Order + รายการสินค้า + ใบปะ) ไว้ในตาราง `deletion_history`

Actor: Super Admin

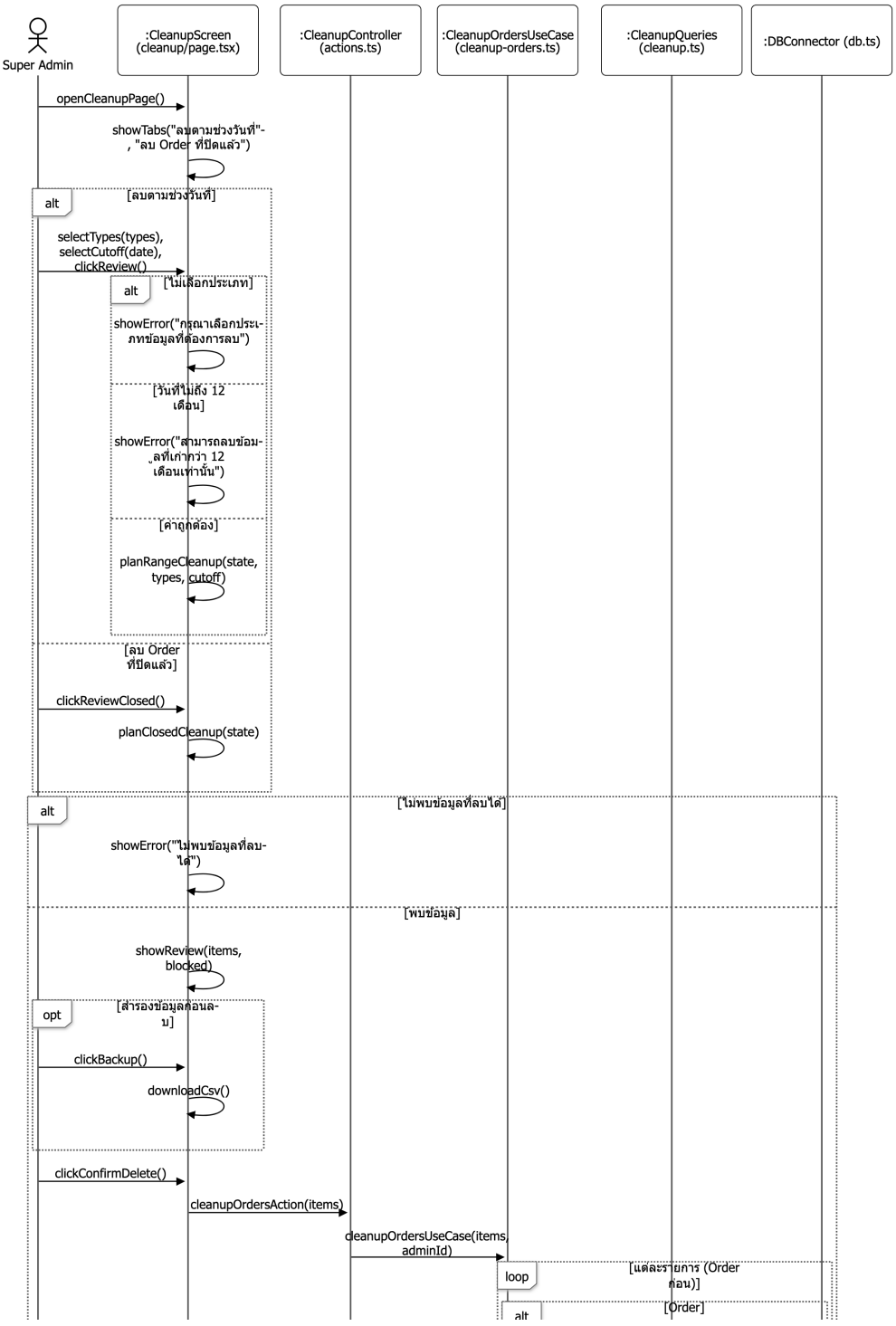
ตารางที่ 2-14 Use Case Description ที่ 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)

ลำดับ	User Action	System Response
1	เปิดหน้า "ลบข้อมูลเก่า"	แสดง 2 แท็บ ได้แก่ "ลบตามช่วงวันที่" และ "ลบ Order ที่ปิดแล้ว" รายการที่ลบได้คำนวณจากข้อมูลที่โหลดไว้แล้วในหน้า (`loadAppState`: Order, ประวัติต้นทุน, Reorder) จึงไม่มี query แยกในขั้นนี้
2	เลือกวิธีการลบ แล้วกด "ตรวจทานรายการ"	แสดงรายการที่จะลบแยกตามประเภท และ "รายการที่ลบไม่ได้" (ดูทางเลือก #1 และ #2) พร้อมคำเตือน "การลบนี้ย้อนกลับไม่ได้"
3	กด "ดาวน์โหลดไฟล์สำรองก่อนลบ" (ไม่บังคับ)	สร้างไฟล์ CSV ของรายการที่จะลบให้ดาวน์โหลด และแจ้งว่า "ดาวน์โหลดไฟล์สำรองแล้ว"
4	กด "ยืนยันการลบข้อมูล"	`cleanupOrdersAction` ตรวจ session และกรองรายการ (ประเภท Order/Cost/Label/Reorder, ไม่เกิน 1,000 รายการ) แล้ว `cleanupOrdersUseCase` ลบทั้งชุดใน transaction เดียว (`withTransaction`) โดยตรวจเงื่อนไขซ้ำใน SQL ทุกข้อ และลบ Order ก่อน **1. Order:** บันทึก snapshot ลง `deletion_history` ปลด `cost_history.cost_id` ที่อ้างถึง (`orders.reorder_id` เป็น FK จาก Order ไปคำสั่งซื้อ จึงไม่ต้องปลด) ลบ `costs` ของ Order แล้วลบ Order (`order_items` และ `labels` ถูกลบตาม ON DELETE CASCADE) **Q10A.1:** `WITH eligible AS MATERIALIZED (SELECT o.order_id, jsonb_build_object('order', to_jsonb(o), 'items', COALESCE((SELECT jsonb_agg(to_jsonb(i)) FROM order_items i WHERE i.order_id = o.order_id), '[]'::jsonb), 'labels', COALESCE((SELECT jsonb_agg(to_jsonb(l)) FROM labels l WHERE l.order_id = o.order_id), '[]'::jsonb)) AS snapshot FROM orders o WHERE o.order_id = $1 AND o.order_status IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') FOR UPDATE), logged AS (INSERT INTO deletion_history (deletion_history_id, order_reference, order_snapshot, admin_id) SELECT $2, order_id, snapshot, $3 FROM eligible RETURNING order_reference), detached_cost_history AS (UPDATE cost_history SET cost_id = NULL WHERE cost_id IN (SELECT cost_id FROM costs WHERE order_id IN (SELECT order_reference FROM logged)) RETURNING cost_history_id), deleted_costs AS (DELETE FROM costs WHERE order_id IN (SELECT order_reference FROM logged) AND (SELECT COUNT() FROM detached_cost_history) >= 0 RETURNING cost_id), deleted AS (DELETE FROM orders WHERE order_id IN (SELECT order_reference FROM logged) AND (SELECT COUNT() FROM deleted_costs) >= 0 RETURNING order_id) SELECT order_id FROM deleted`

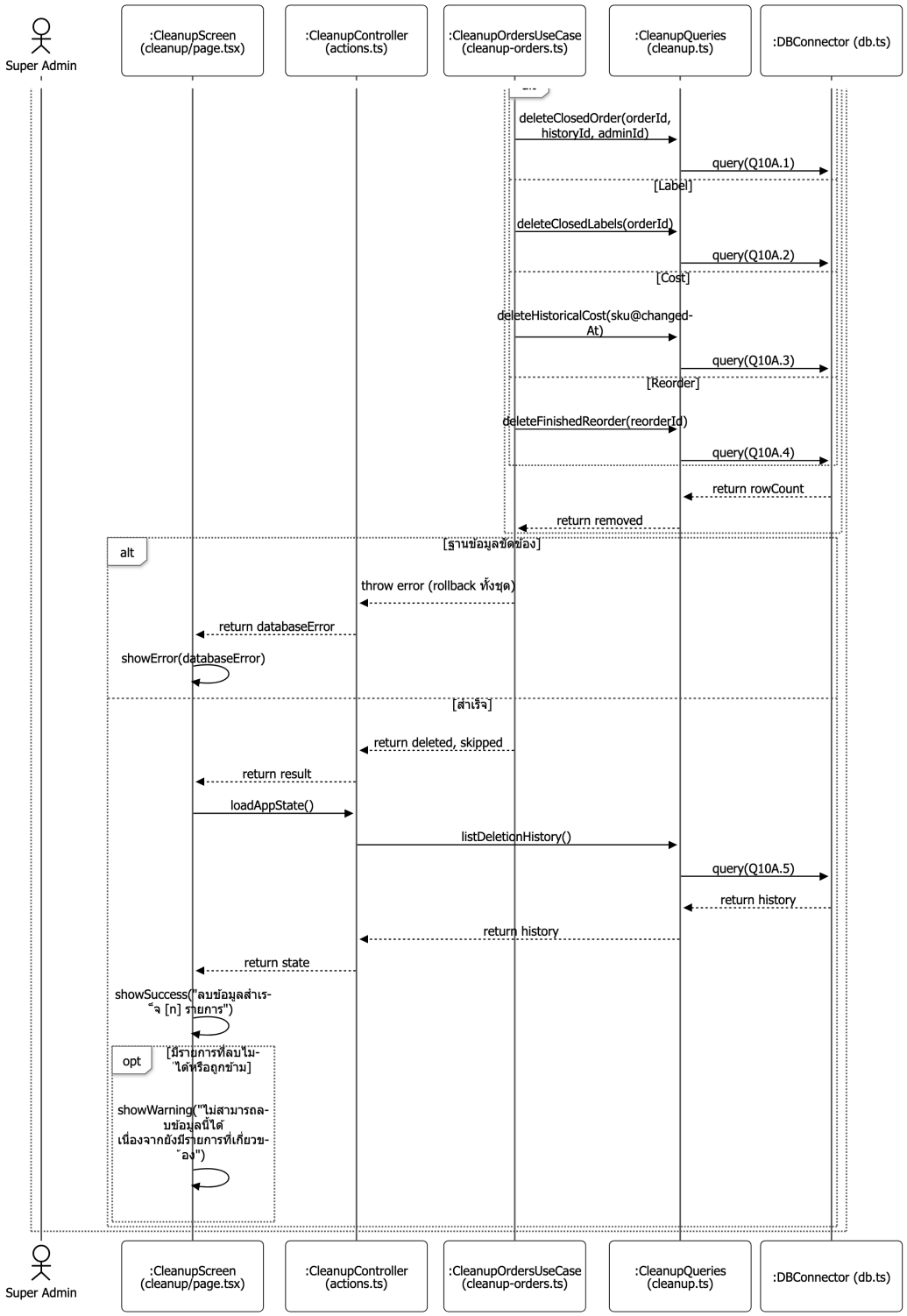
ลำดับ	User Action	System Response
		2. Label: ลบใบปะของ Order ที่ปิดแล้ว (ถ้า Order ถูกลบในรอบเดียวกันนับว่าลบแล้ว) **Q10A.2:** `DELETE FROM labels l USING orders o WHERE l.order_id = $1 AND o.order_id = l.order_id AND o.order_status IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') RETURNING l.label_id` **3. Cost:** ลบแถวประวัติต้นทุนของ SKU ตามเวลาที่บันทึก (`record_id` = `sku@changed_at`) **Q10A.3:** `WITH sku_row AS ( SELECT sku_id FROM skus WHERE sku = $1 ), deleted_history AS ( DELETE FROM cost_history h USING sku_row s WHERE h.sku_id = s.sku_id AND h.changed_at = $2::timestamptz RETURNING h.cost_history_id ) SELECT 'history' AS kind FROM deleted_history` **4. Reorder:** **Q10A.4:** `DELETE FROM reorders r WHERE r.reorder_id = $1 AND (r.reorder_status IN ('ยกเลิกแล้ว', 'ได้รับสินค้าแล้ว') OR (r.reorder_status = 'สั่งซื้อแล้ว' AND r.expected_date < CURRENT_DATE)) AND NOT EXISTS (SELECT 1 FROM orders o WHERE o.reorder_id = r.reorder_id AND o.order_status NOT IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว')) RETURNING r.reorder_id` — รายการที่ไม่เข้าเงื่อนไขแล้วตอนลบจริงจะถูกข้าม (skipped) — สำเร็จ: แสดง "ลบข้อมูลสำเร็จ [จำนวน] รายการ" พร้อมสรุปตามประเภท หรือ "ลบ Order ที่ปิดแล้วสำเร็จ [จำนวน] รายการ" สำหรับทางเลือก #2
5		หลังลบ ระบบโหลดข้อมูลหน้าใหม่ (`revalidatePath`) รวมถึงประวัติการลบ **Q10A.5:** `SELECT order_reference, admin_id, deleted_at FROM deletion_history ORDER BY deleted_at DESC`
ทางเลือก #1	**ลบตามช่วงวันที่:** เลือกประเภทข้อมูล (Order/Cost/Label/Reorder) และวันที่ "ลบข้อมูลที่เก่ากว่าวันที่"	**ตรวจสอบค่า** — ไม่เลือกประเภท: "กรุณาเลือกประเภทข้อมูลที่ต้องการลบ"; วันที่ไม่ย้อนหลังอย่างน้อย 12 เดือน: "สามารถลบข้อมูลที่เก่ากว่า 12 เดือนเท่านั้น" **เลือกรายการ** (`planRangeCleanup`) — Order ที่ปิดแล้วและ `order_date` ก่อนวันที่เลือก; Label ของ Order ดังกล่าว; Cost = แถวประวัติต้นทุนก่อนวันที่เลือก ยกเว้นแถวล่าสุดของ SKU ที่ยังมี Order ไม่ปิด (ยังใช้ใน 5A); Reorder ที่ "ยกเลิกแล้ว", "ได้รับสินค้าแล้ว" หรือ "สั่งซื้อแล้ว" ที่เลยวันคาดว่าจะได้รับ และไม่มี Order ที่ยังไม่ปิดผูกอยู่ (`orders.reorder_id`) — ไม่พบข้อมูล: "ไม่พบข้อมูลที่ลบได้ในช่วงเวลาที่เลือก" และแนะนำ "ลองปรับช่วงวันที่ใหม่"
ทางเลือก #2	**ลบ Order ที่ปิดแล้ว:** เลือกแท็บ "ลบ Order ที่ปิดแล้ว"	เลือก Order ที่ "จัดส่งสำเร็จ" หรือ "ยกเลิกแล้ว" ทั้งหมดพร้อมใบปะที่เกี่ยวข้อง (`planClosedCleanup`) โดยไม่ต้องกำหนดวันที่ แสดง "Order ที่ปิดแล้ว [n] รายการ · ใบปะสินค้าที่เกี่ยวข้อง [m] รายการ" แล้วลบด้วย **Q10A.1** และ **Q10A.2** ตามขั้นตอนที่ 4 — ไม่พบ: "ไม่พบ Order ที่ปิดแล้วให้ลบ"

ลำดับ	User Action	System Response
ทางเลือก #3	(ระบบตรวจพบเอง) มีรายการที่ลบไม่ได้หรือถูกข้าม	แจ้งเตือนว่า "ไม่สามารถลบข้อมูลนี้ได้ เนื่องจากยังมีรายการที่เกี่ยวข้องอยู่" พร้อมรายการที่ลบไม่ได้และจำนวนที่ถูกข้าม
ทางเลือก #4	(ระบบตรวจพบเอง) ฐานข้อมูลขัดข้องระหว่างลบ (จำลองด้วย fault **db**)	ยกเลิกการลบทั้งชุด (rollback) คงข้อมูลเดิมไว้ทั้งหมด และแจ้งข้อผิดพลาดของฐานข้อมูลบนหน้าจอให้ลองใหม่

2.13.38 Sequence Diagram ของ Use Case Description ที่ 10A



รูปที่ 2-52 Sequence Diagram ของ Use Case Description ที่ 10A (1/2)

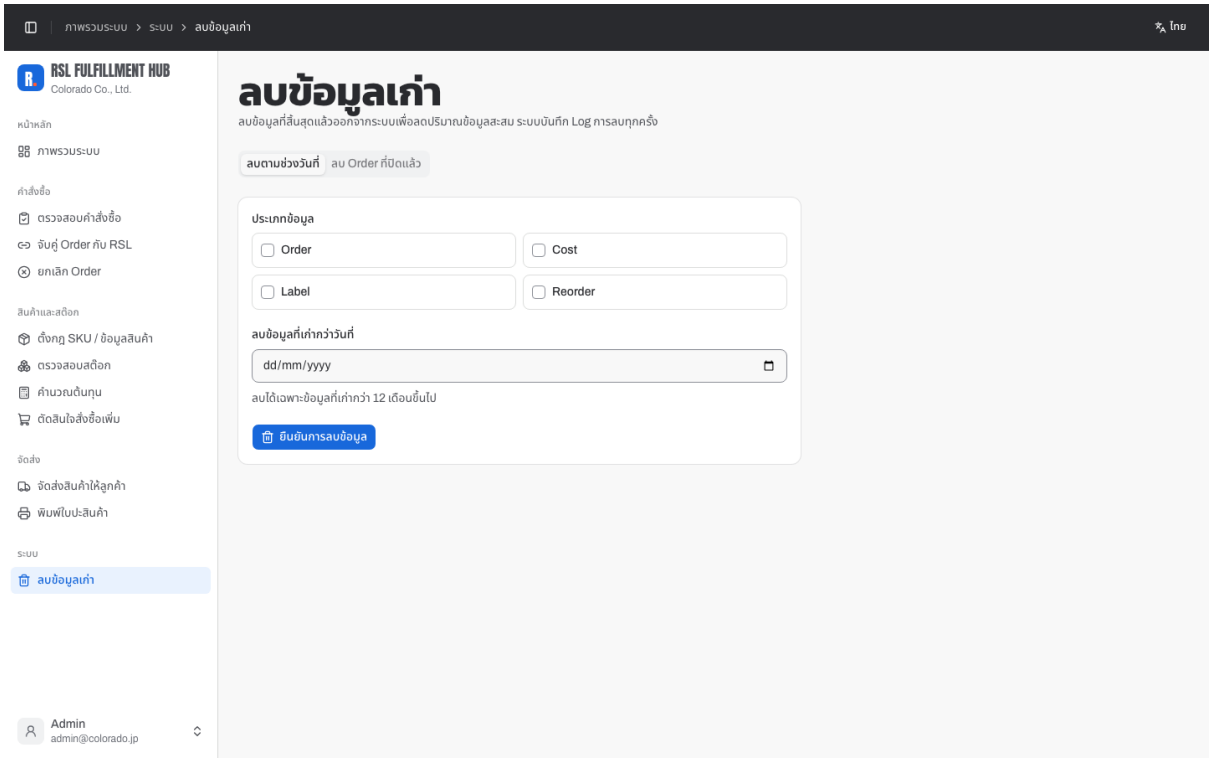


รูปที่ 2-53 Sequence Diagram ของ Use Case Description ที่ 10A (2/2)

2.13.39 Collaboration Diagram ของ Use Case Description ที่ 10A

รูปที่ 2-54 Collaboration Diagram ของ Use Case Description ที่ 10A

2.13.40 User Interface ของ Use Case Description ที่ 10A



รูปที่ 2-55 User Interface ของ Use Case Description ที่ 10A (CleanupScreen)

2.13.41 Use Case Description ที่ 1S: จับคู่ Order กับเลข RSL

Goal: ให้ Order ที่ตรวจสต๊อกแล้วถูกจับคู่กับข้อมูลในคลัง RSL (เลข `rsl_no`) ได้ถูกต้อง เพื่อใช้จัดรูปแบบใบปะสินค้าและจัดส่งต่อไป

Pre-Condition: Order จับคู่กฎ SKU แล้ว (2S) ผ่าน 3S โดยให้ RSL จัดส่ง (`order_items.fulfill_source`='rsl') และมีสถานะ "ยังไม่ได้จับคู่" หรือ "รอดำเนินการด้วยตนเอง" ที่ `manual_reason`='rsl' (Order ที่บริษัทจัดส่งเองจากคลังบริษัทไม่ผ่าน Use Case นี้)

Post-Condition: Order มี `rsl_no` และ `rsl_matched_at` สถานะ "รอจัดรูปแบบใบปะสินค้า" และ `executeWorkflow` รัน 6S ต่อใน transaction เดียวกันทันที (สำเร็จแล้วเป็น "รอพิมพ์ใบปะสินค้า")

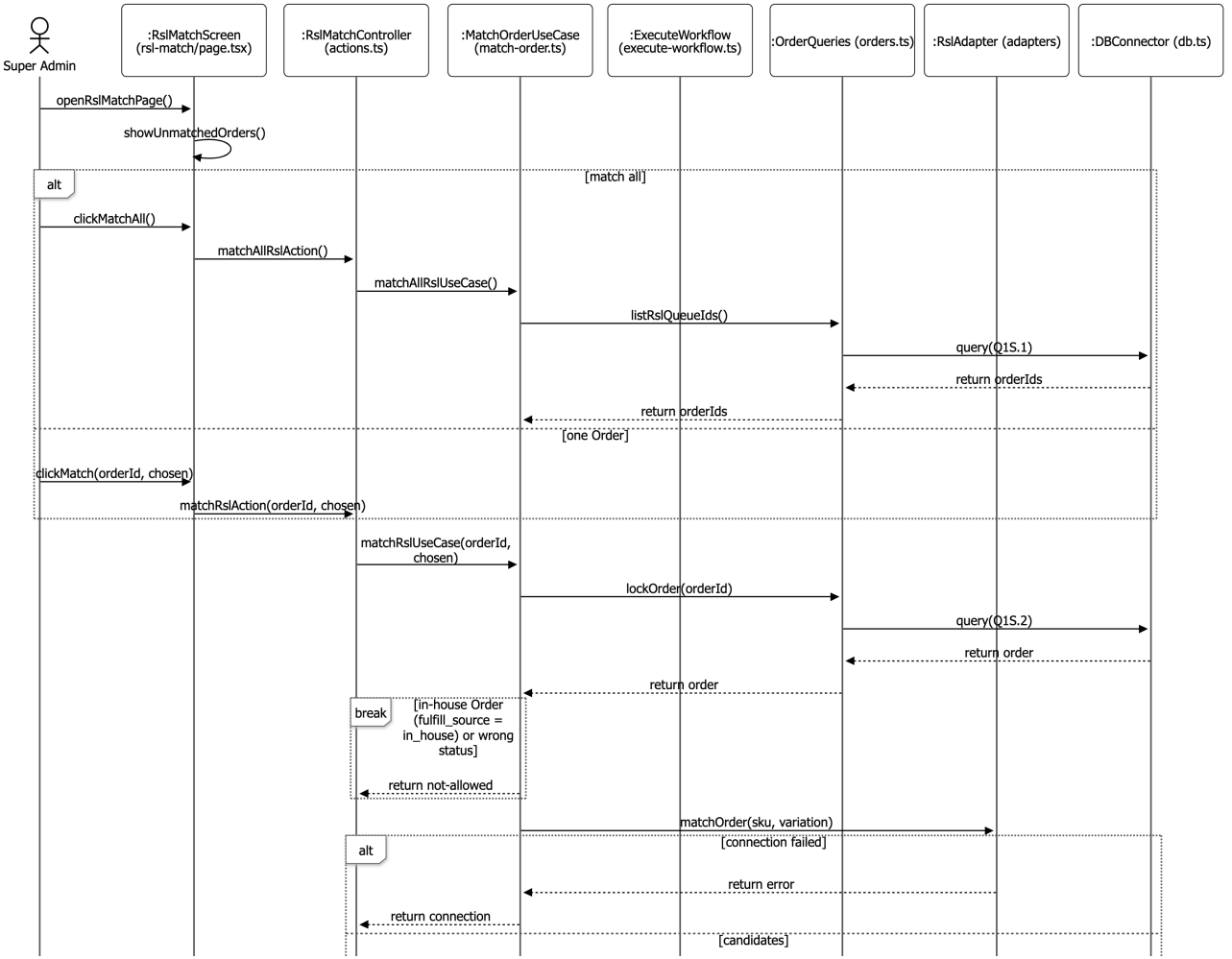
Actor: Super Admin (กดสั่งจับคู่จากหน้าจอ), RSL

ตารางที่ 2-15 Use Case Description ที่ 1S: จับคู่ Order กับเลข RSL

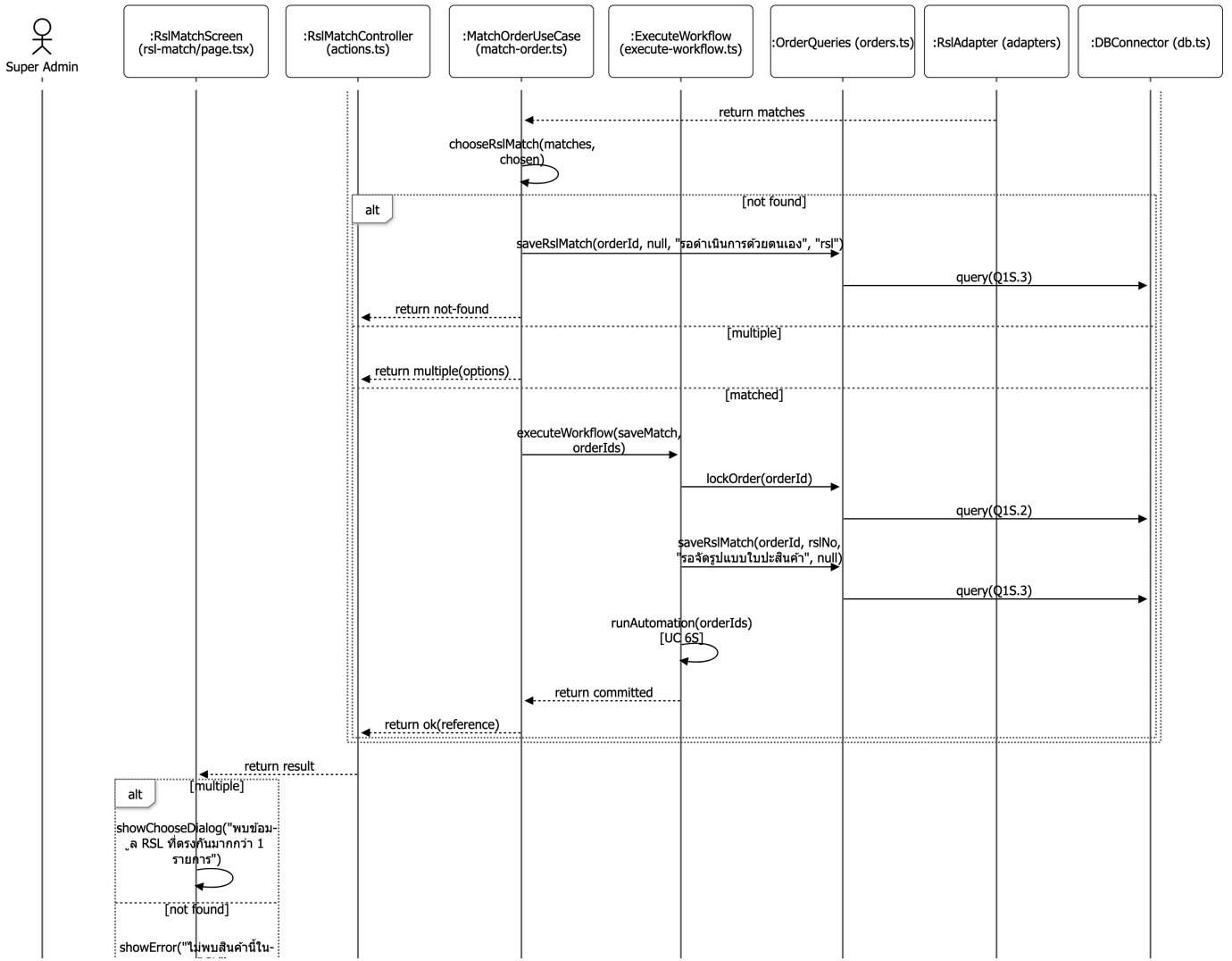
ลำดับ	User Action	System Response
1		หน้า "จับคู่ Order กับ RSL" (`/orders/rsl-match`) แสดงรายการ "ยังไม่ได้จับคู่" และ "จับคู่แล้ว รอพิมพ์ใบปะสินค้า" เฉพาะ Order ที่ RSL จัดส่ง (`fulfill_source='rsl'`)
2	Super Admin กด "จับคู่กับ RSL" ที่ Order หรือกด "จับคู่อัตโนมัติทั้งหมด"	ปุ่มรายตัวเรียก `matchRslAction(orderId)` ⟶ `matchRslUseCase` ปุ่มทั้งหมดเรียก `matchAllRslAction` ⟶ `matchAllRslUseCase` ดึงคิว Order ที่จับคู่ได้ **Q1S.1:** `SELECT o.order_id FROM orders o WHERE ( o.order_status = 'ยังไม่ได้จับคู่' OR (o.order_status = 'รอดำเนินการด้วยตนเอง' AND o.manual_reason = 'rsl') ) AND EXISTS ( SELECT 1 FROM order_items oi WHERE oi.order_id = o.order_id AND oi.fulfill_source = 'rsl' ) ORDER BY o.order_date, o.order_id` แล้วเรียก `matchRslUseCase` ทีละ Order
3		**1. ล็อกและตรวจสอบ Order** — **Q1S.2:** `SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o` Order ต้องอยู่ในสถานะตามเงื่อนไขก่อนเริ่ม มี SKU แล้ว และ `fulfill_source='rsl'` (Order ที่บริษัทจัดส่งเองถูกปฏิเสธ) **2. ค้นหาใน RSL** — เรียก `RslAdapter.matchOrder({sku, variation})` แล้วเลือกผลด้วย `chooseRslMatch`: ไม่พบ = ทางเลือก #1, พบมากกว่า 1 รายการและยังไม่ได้เลือก = ทางเลือก #2, พบ 1 รายการ (หรือ Admin เลือกแล้ว) = จับคู่ได้ **3. บันทึกผลการจับคู่** — ภายใน `executeWorkflow` ล็อก Order ซ้ำ (**Q1S.2**) แล้วบันทึก `rsl_no` และสถานะ "รอจัดรูปแบบใบปะสินค้า" **Q1S.3:** `UPDATE orders SET rsl_no = $2, rsl_matched_at = CASE WHEN $2::text IS NULL THEN NULL ELSE CURRENT_TIMESTAMP END, order_status = $3, manual_reason = $4, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1`

ลำดับ	User Action	System Response
		ใน transaction เดียวกัน `executeWorkflow` รัน **Use Case 6S** สร้างใบปะสินค้าต่อทันที แล้วแจ้งว่า "จับคู่ Order กับ RSL สำเร็จ" กรณีกดทั้งหมดแสดงสรุป "จับคู่สำเร็จ [n] · ต้องเลือกเอง [m] · ไม่พบใน RSL [k]"
ทางเลือก #1	ไม่พบ SKU ที่ตรงกันใน RSL	บันทึก (**Q1S.3**) ด้วย `rsl_no`=NULL สถานะ "รอดำเนินการด้วยตนเอง" และ `manual_reason`='rsl' แจ้งเตือนว่า "ไม่พบสินค้านี้ในระบบ RSL" Order ยังอยู่ในคิวให้กดจับคู่ใหม่ได้
ทางเลือก #2	พบข้อมูล RSL ที่ตรงกันมากกว่า 1 รายการ	แจ้งเตือนว่า "พบข้อมูล RSL ที่ตรงกันมากกว่า 1 รายการ กรุณาเลือกด้วยตนเอง" และแสดงหน้าต่าง "เลือกรายการ RSL สำหรับ [Order ID] · [SKU]" พร้อมจำนวนคงเหลือในคลัง RSL เมื่อ Admin กด "เลือกรายการนี้" ระบบเรียก `matchRslAction(orderId, chosen)` และบันทึกตามขั้นตอนที่ 3
ทางเลือก #3	Super Admin กด "ยกเลิกการจับคู่" (จับคู่ผิด)	`unmatchRslAction` ⟶ `unmatchRslUseCase` ล็อก Order (**Q1S.2**) แล้ว **Q1S.4:** `WITH eligible AS (SELECT order_id FROM orders WHERE order_id = $1 AND rsl_no IS NOT NULL AND (order_status IN ('รอจัดรูปแบบใบปะสินค้า', 'รอพิมพ์ใบปะสินค้า') OR (order_status = 'รอดำเนินการด้วยตนเอง' AND rsl_no IS NOT NULL))), removed_labels AS (DELETE FROM labels WHERE order_id IN (SELECT order_id FROM eligible)) UPDATE orders SET rsl_no = NULL, rsl_matched_at = NULL, order_status = 'ยังไม่ได้จับคู่', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id IN (SELECT order_id FROM eligible) RETURNING order_id` ทำได้เฉพาะ Order ที่มี `rsl_no` (Order ที่บริษัทจัดส่งเองไม่เคยจับคู่จึงยกเลิกไม่ได้) และอยู่ในสถานะ "รอจัดรูปแบบใบปะสินค้า", "รอพิมพ์ใบปะสินค้า" หรือ "รอดำเนินการด้วยตนเอง" ที่มี `rsl_no` ระบบลบใบปะที่สร้างไว้ ล้าง `rsl_no` และแจ้งว่า "ยกเลิกการจับคู่ [Order ID] แล้ว กลับไปอยู่ในสถานะยังไม่ได้จับคู่"
ทางเลือก #4	การเชื่อมต่อกับระบบ RSL ล้มเหลวขณะจับคู่	แจ้งเตือนว่า "ไม่สามารถเชื่อมต่อกับระบบ RSL ได้ กรุณาลองใหม่ภายหลัง" ไม่เปลี่ยนแปลงข้อมูล Order คงสถานะเดิม กรณีกดทั้งหมดระบบหยุดที่ Order แรกที่เชื่อมต่อไม่ได้ ให้ Super Admin กดใหม่ภายหลัง

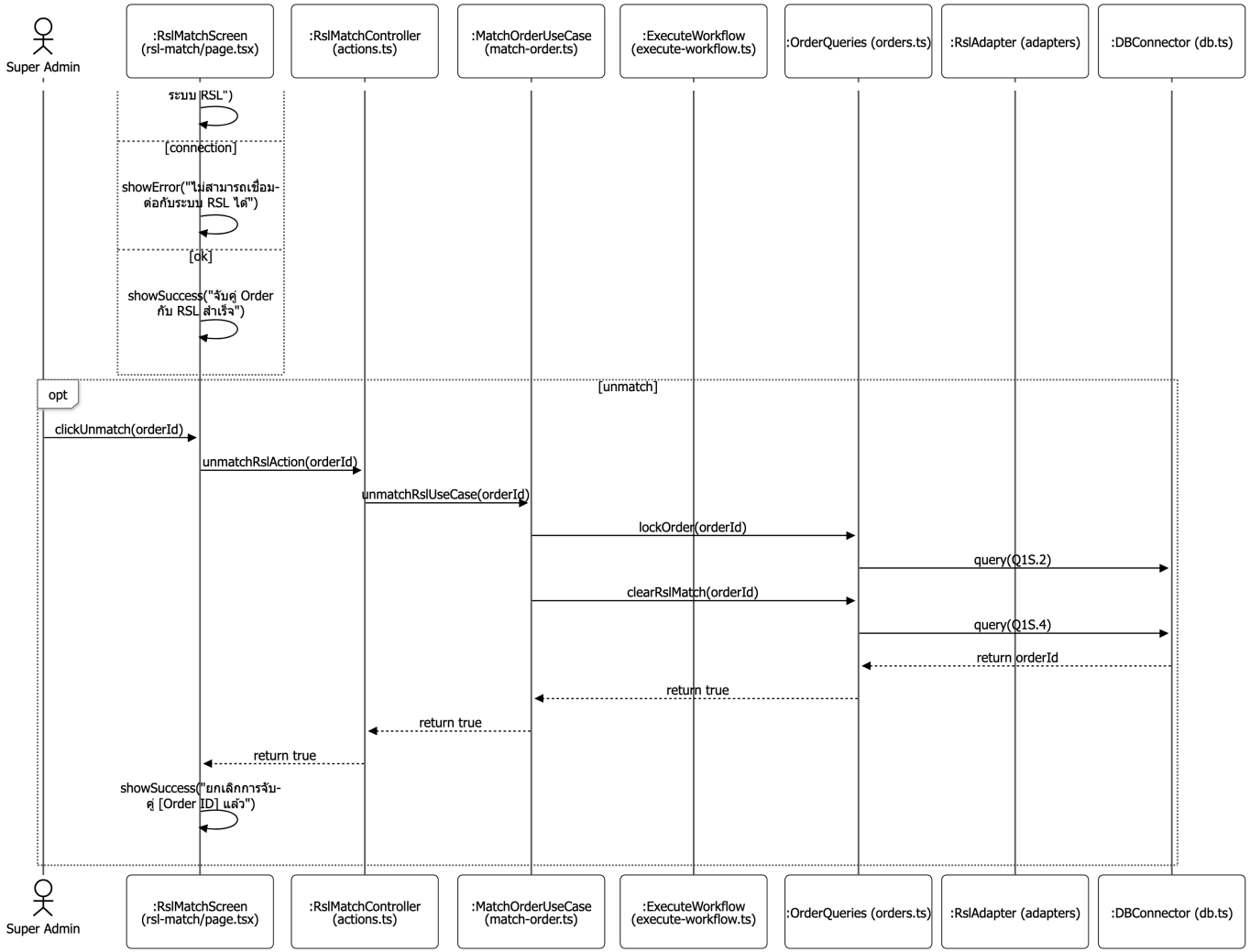
2.13.42 Sequence Diagram ของ Use Case Description ที่ 1S



รูปที่ 2-56 Sequence Diagram ของ Use Case Description ที่ 1S (1/3)

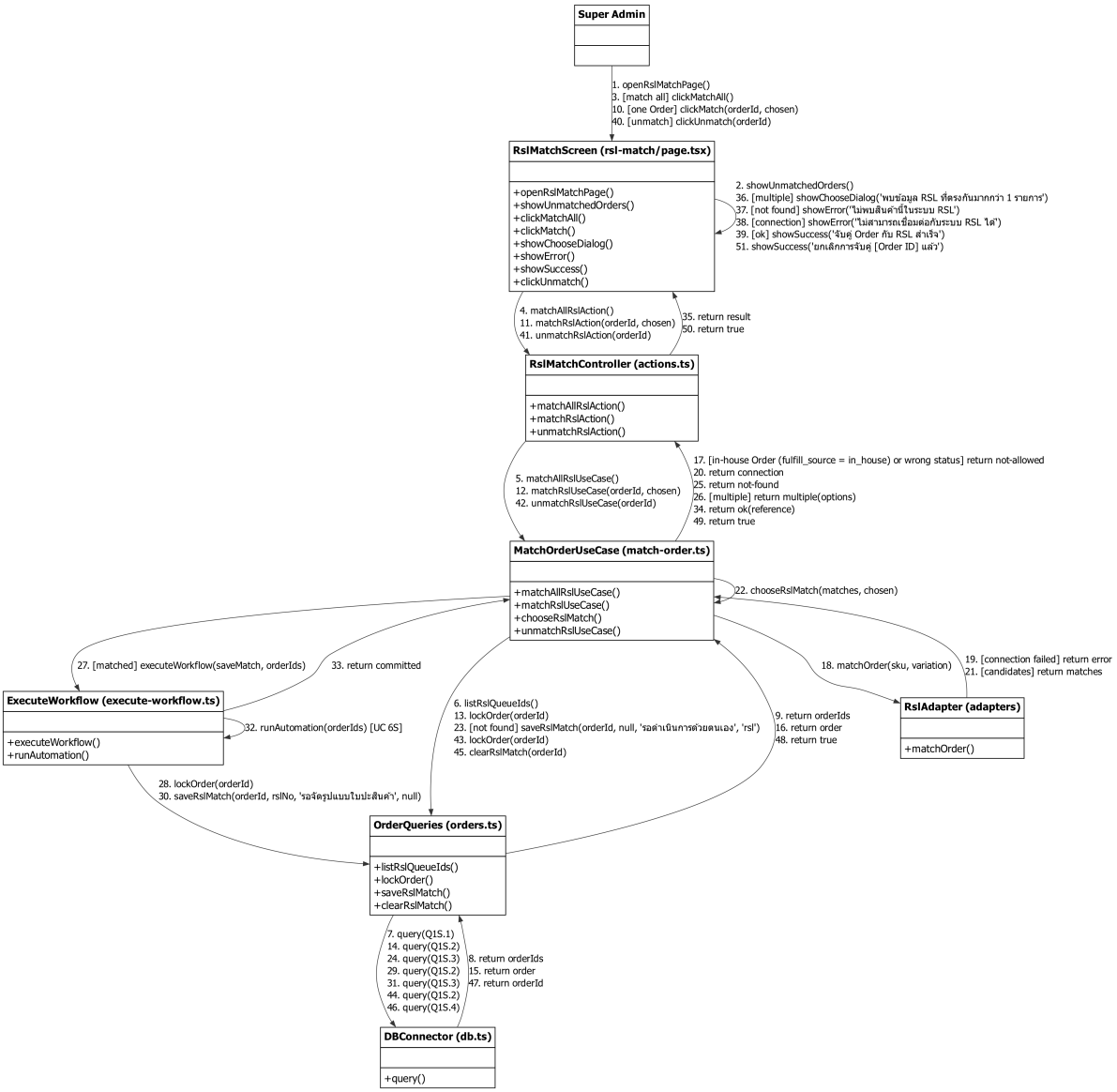


รูปที่ 2-57 Sequence Diagram ของ Use Case Description ที่ 1S (2/3)



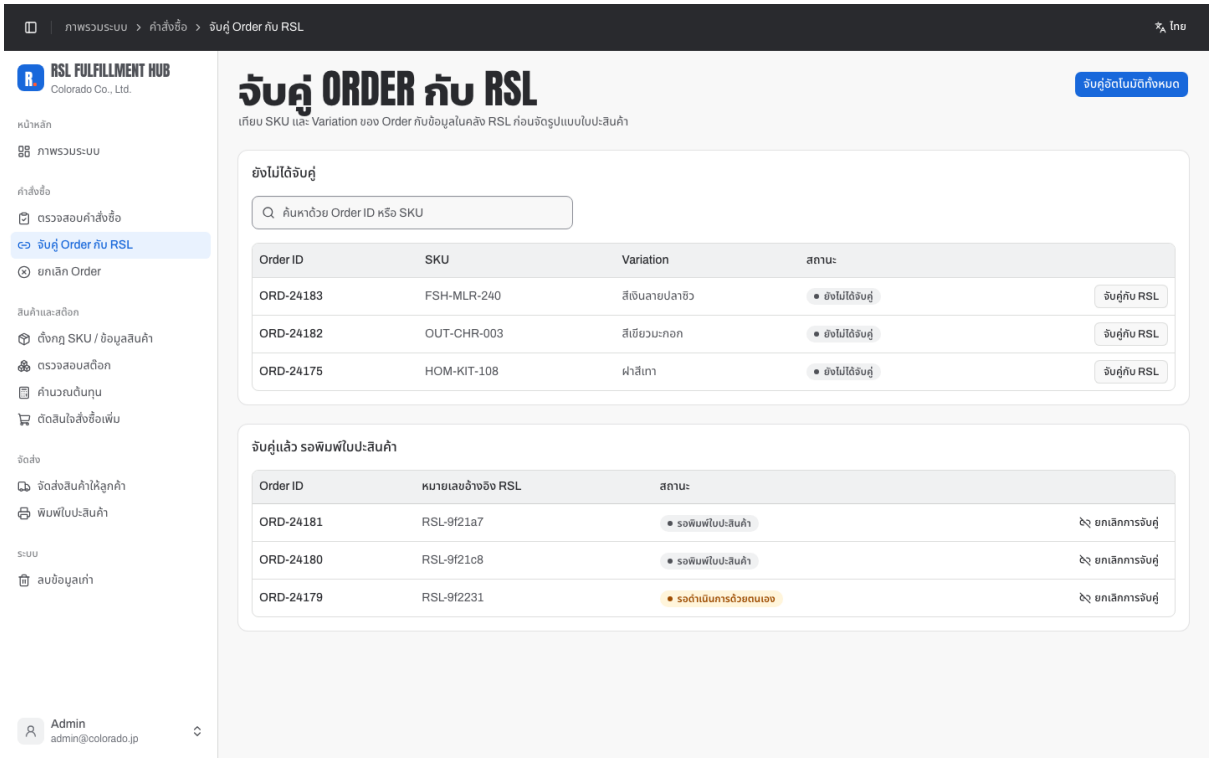
รูปที่ 2-58 Sequence Diagram ของ Use Case Description ที่ 1S (3/3)

2.13.43 Collaboration Diagram ของ Use Case Description ที่ 1S



รูปที่ 2-59 Collaboration Diagram ของ Use Case Description ที่ 1S

2.13.44 User Interface ของ Use Case Description ที่ 1S



รูปที่ 2-60 User Interface ของ Use Case Description ที่ 1S (RslMatchScreen)

2.13.45 Use Case Description ที่ 2S: จับคู่กฎ SKU

Goal: ให้ระบบจับคู่รหัสสินค้าใน Marketplace (`channel_sku`) ของ Order เข้ากับ SKU ภายในระบบ (`skus.channel_sku`) โดยอัตโนมัติ

Pre-Condition: Order อยู่ในสถานะ "รอจับคู่กฎ SKU" (Super Admin ยืนยัน Order แล้วใน 2A) และ SKU ที่ใช้งาน (`sku_status`='active') มี `channel_sku` ตั้งไว้แล้ว (3A)

Post-Condition: `order_items.sku_id` ถูกผูกกับ SKU ที่ตรงกันและ Order เปลี่ยนเป็น "รอตรวจสอบสต๊อก" แล้วทำ 3S ต่อทันทีใน transaction เดียวกัน / หากไม่พบหรือพบมากกว่า 1 รายการ Order เปลี่ยนเป็น "รอดำเนินการด้วยตนเอง" (`manual_reason`='sku-rule')

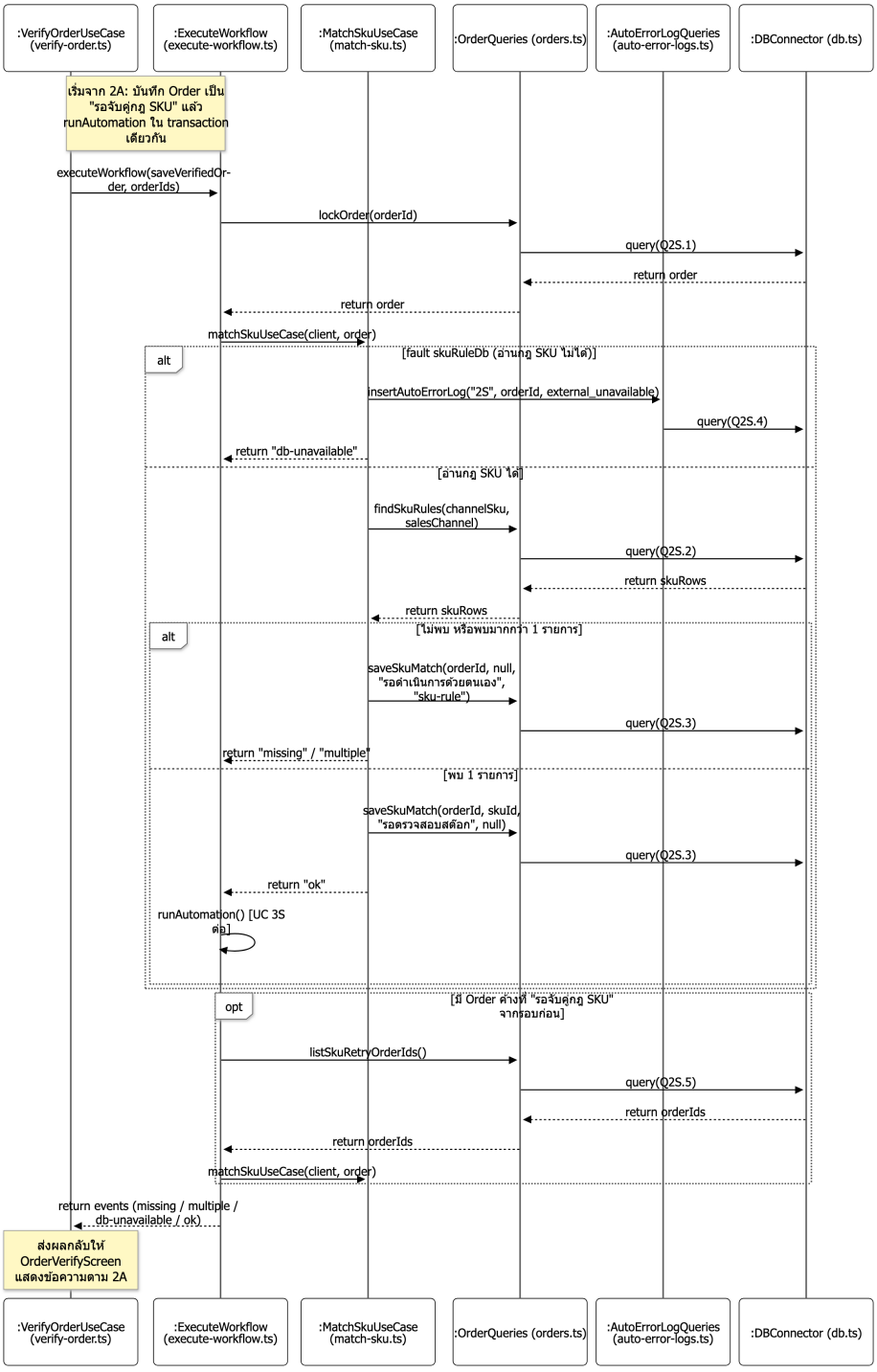
Actor: ระบบทำงานอัตโนมัติ (ทำงานใน `executeWorkflow` ต่อจาก action ของ Super Admin)

ตารางที่ 2-16 Use Case Description ที่ 2S: จับคู่กฎ SKU

ลำดับ	User Action	System Response
1	Super Admin กด "ยืนยัน" Order ในหน้าตรวจสอบคำสั่งซื้อ (2A)	**เริ่มทำงาน** — `verifyOrderAction` เรียก `verifyOrderUseCase` ซึ่งบันทึก Order เป็น "รอจับคู่กฎ SKU" ภายใน `executeWorkflow` จากนั้น `runAutomation` ทำงานต่อใน transaction เดียวกัน (ไม่มี Scheduler แยก) โดยล็อก Order ก่อนทุกขั้น Q2S.1: `SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o`
2		**1. ค้นหา SKU ที่ตรงกัน** — `matchSkuUseCase` เทียบ `channel_sku` และ `sales_channel` ของรายการสินค้าใน Order กับ SKU ที่เปิดใช้งาน Q2S.2: `SELECT sku_id, sku, product_name, variation FROM skus WHERE channel_sku = $1 AND sales_channel = $2 AND sku_status = 'active' ORDER BY sku_id` — หากไม่พบ SKU ที่ตรงกัน จะแจ้งเตือนว่า "ไม่พบกฎ SKU ที่ตรงกับสินค้านี้ กรุณาตั้งกฎ SKU ก่อน" และตั้ง Order เป็น "รอดำเนินการด้วยตนเอง" (`manual_reason`='sku-rule', `sku_id`=NULL) ด้วย **Q2S.3** เพื่อให้ Super Admin ไปตั้ง `channel_sku` ใน 3A หรือยกเลิก Order (7A) — หากพบมากกว่า 1 รายการ จะแจ้งเตือนว่า "พบกฎ SKU ที่ตรงกันมากกว่า 1 รายการ กรุณาเลือกด้วยตนเอง" และตั้งสถานะแบบเดียวกัน (`manual_reason`='sku-rule') ด้วย **Q2S.3** ให้ Super Admin แก้ `channel_sku` ใน 3A ให้เหลือรายการเดียว
3		**2. ผูก SKU และเปลี่ยนสถานะ** Q2S.3: `WITH matched_item AS ( UPDATE order_items SET sku_id = $2 WHERE order_id = $1 ) UPDATE orders SET order_status = $3, manual_reason = $4, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจับคู่กฎ SKU'` โดย $2=`sku_id` ที่พบ, $3='รอตรวจสอบสต๊อก', $4=NULL — ระบบแสดงว่า "จับคู่กฎ SKU สำเร็จ [จำนวน] รายการ" แล้ว `executeWorkflow` วนกลับไปทำ 3S กับ Order นี้ทันที

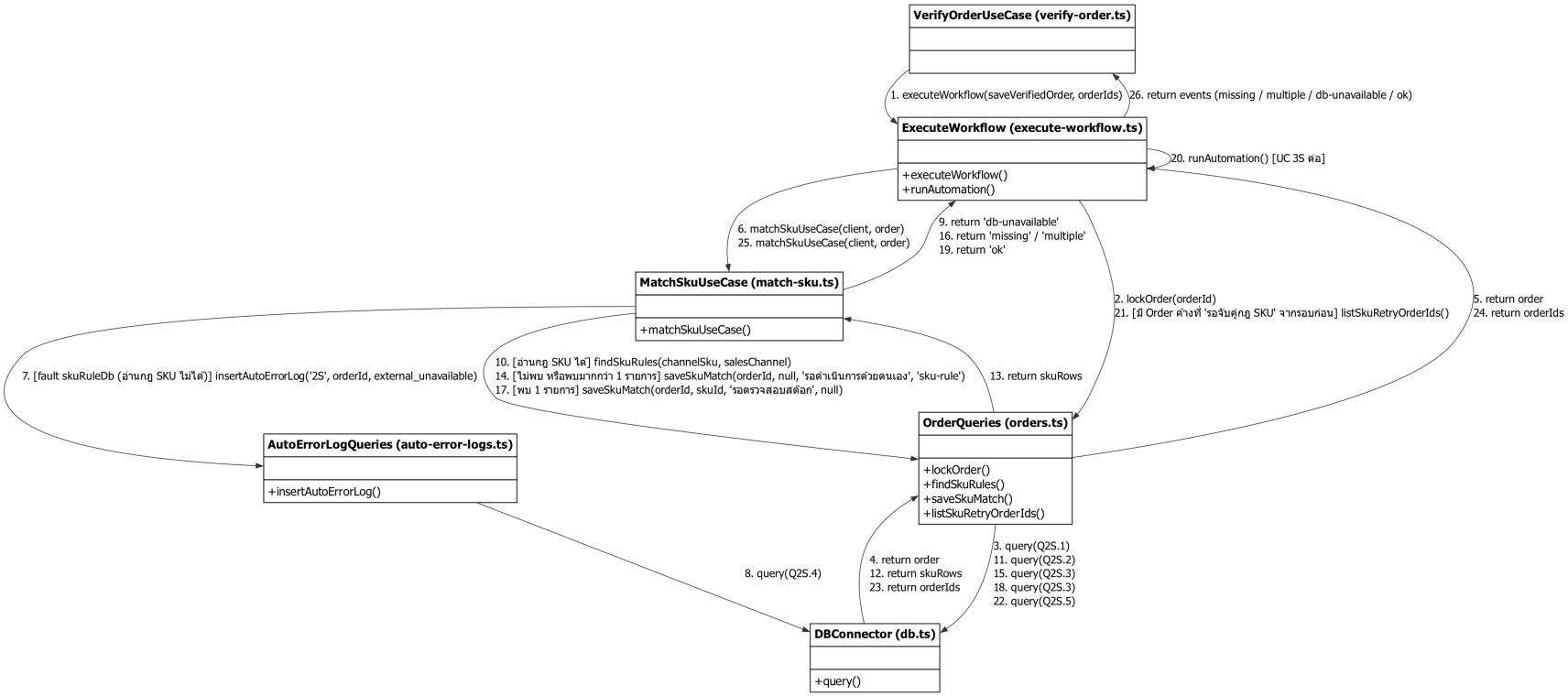
ลำดับ	User Action	System Response
ทางเลือก #1	(ระบบตรวจพบเอง) อ่านข้อมูลกฎ SKU ไม่ได้ (จำลองด้วย fault `skuRuleDb` ในหน้า /dev)	บันทึกข้อผิดพลาด **Q2S.4:** `INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, $2, $3, $4, $5)` (`use_case_id`='2S', `error_code`='external_unavailable') แจ้งเตือนว่า "ไม่สามารถอ่านข้อมูลกฎ SKU ได้ กรุณาลองใหม่ภายหลัง" และคงสถานะ Order เป็น "รอจับคู่กฎ SKU" (action 2A ยังสำเร็จ)
ทางเลือก #2	(ระบบตรวจพบเอง) มี Order ค้างที่ "รอจับคู่กฎ SKU" จากรอบก่อน	ทุกครั้งที่มี action ใดเรียก `executeWorkflow` ระบบดึง Order ที่ค้างอยู่ (สูงสุด 100 รายการ ข้ามรายการที่ถูกล็อก) **Q2S.5:** `SELECT order_id FROM orders WHERE order_status = 'รอจับคู่กฎ SKU' ORDER BY order_date, order_id LIMIT 100 FOR UPDATE SKIP LOCKED` แล้วทำขั้นตอนที่ 1–3 ใหม่ หากยังอ่านกฎ SKU ไม่ได้จะบันทึก log ด้วย **Q2S.4** อย่างเดียว ไม่แจ้งเตือนบน action ที่ไม่เกี่ยวข้อง เมื่อ Super Admin แก้ `channel_sku` ใน 3A แล้ว Order ที่ `manual_reason`='sku-rule' จะถูกส่งกลับเป็น "รอจับคู่กฎ SKU" และถูกจับคู่ใหม่ในรอบถัดไปด้วยวิธีเดียวกัน
ทางเลือก #3	(ระบบตรวจพบเอง) ฐานข้อมูลผิดพลาดระหว่างจับคู่	ยกเลิกทั้ง transaction (รวมการยืนยัน Order ของ 2A) แล้วบันทึกข้อผิดพลาดใน transaction ใหม่ด้วย **Q2S.4** และแจ้งข้อผิดพลาดของฐานข้อมูลบนหน้าจอ

2.13.46 Sequence Diagram ของ Use Case Description ที่ 2S



รูปที่ 2-61 Sequence Diagram ของ Use Case Description ที่ 2S

2.13.47 Collaboration Diagram ของ Use Case Description ที่ 2S



รูปที่ 2-62 Collaboration Diagram ของ Use Case Description ที่ 2S

2.13.48 User Interface ของ Use Case Description ที่ 2S

Use Case นี้ระบบทำเองโดยไม่มีหน้าจอของตัวเอง ผลการทำงานแสดงเป็นข้อความแจ้งเตือนในหน้าที่ Super Admin เปิดอยู่

2.13.49 Use Case Description ที่ 3S: ตรวจสอบของสินค้าใน stock + RSL

Goal: ให้ระบบเลือกแหล่งส่งสินค้าของ Order (คลัง RSL หรือคลังบริษัท) จากยอดสต๊อกจริง หรือส่งต่อไปตัดสินใจสั่งซื้อเพิ่มเมื่อสต๊อกไม่พอ และให้ Super Admin ดูและปรับยอดสต๊อกได้

Pre-Condition: Order อยู่ในสถานะ "รอตรวจสอบสต๊อก" (ผ่าน 2S แล้ว) และ SKU มีแถวในตาราง `stock`

Post-Condition: Order ที่ RSL มียอดใช้ได้พอทั้งจำนวน เปลี่ยนเป็น "ยังไม่ได้จับคู่" (`fulfill_source`='rsl' ไป 1S) / ถ้า RSL ไม่พอแต่คลังบริษัทพอ เปลี่ยนเป็น "รอจัดรูปแบบใบปะสินค้า" (`fulfill_source`='in_house' ข้าม 1S ไป 6S) / ไม่มีแหล่งใดพอ เปลี่ยนเป็น "รอ Admin ตัดสินใจสั่งซื้อ" (ไป 4S และ 5A) / Super Admin เห็นยอดสต๊อกล่าสุดเทียบกับ Reorder Threshold

Actor: ระบบทำงานอัตโนมัติ (ใน `executeWorkflow` ต่อจาก 2S), RSL, Super Admin (ดูและปรับยอดสต๊อก)

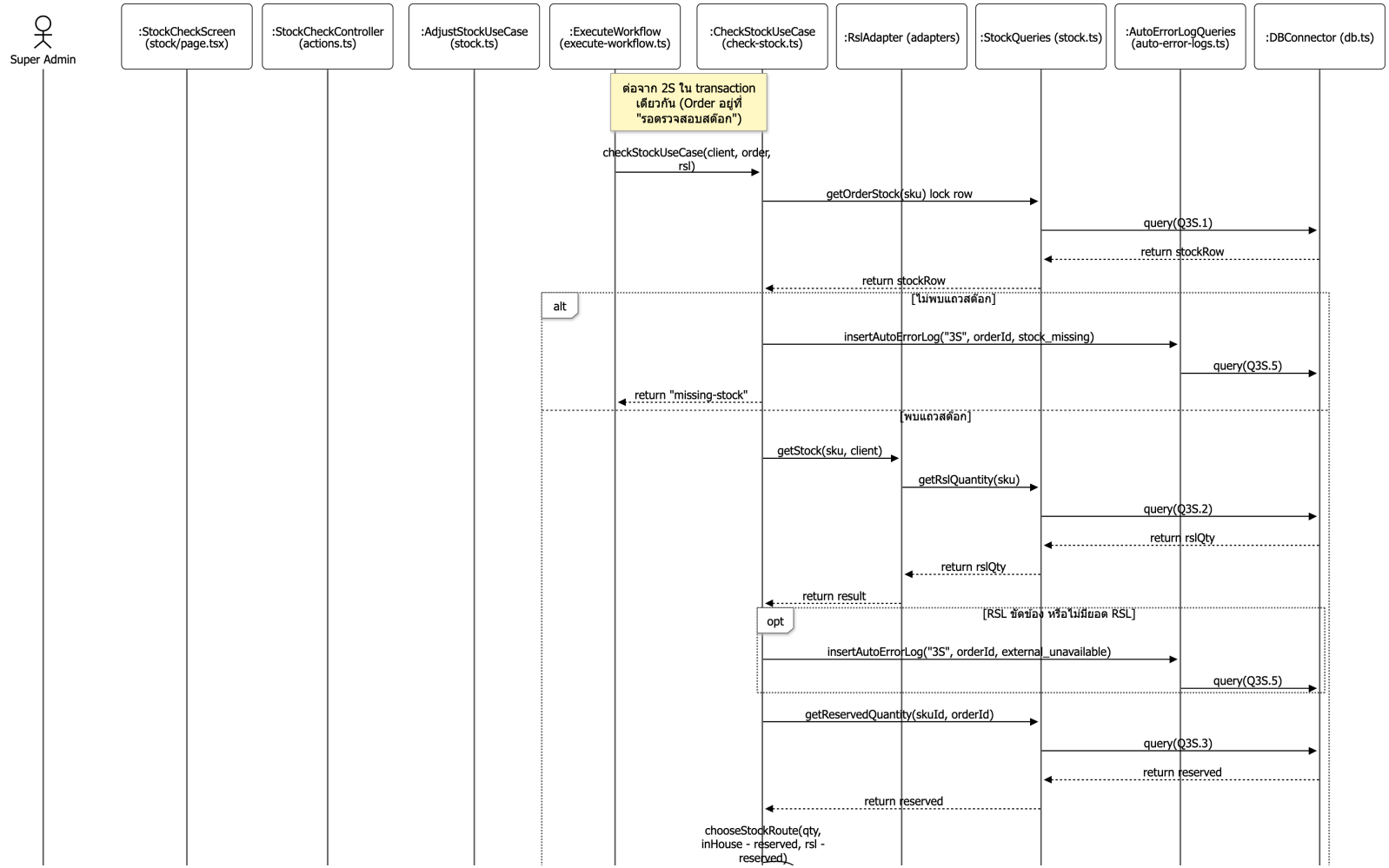
ตารางที่ 2-17 Use Case Description ที่ 3S: ตรวจสอบของสินค้าใน stock + RSL

ลำดับ	User Action	System Response
1		**เริ่มทำงาน** — ต่อจาก 2S ใน `executeWorkflow` เดียวกัน เมื่อ Order ที่ล็อกไว้อยู่ในสถานะ "รอตรวจสอบสต๊อก" `checkStockUseCase` ล็อกและอ่านแถวสต๊อกของ SKU **Q3S.1:** `SELECT stock.stock_id, stock.sku_id, stock.in_house_qty, stock.rsl_qty FROM stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1 FOR UPDATE OF stock` การล็อกแถว (`FOR UPDATE`) ทำให้ Order อื่นของ SKU เดียวกันต้องรอตรวจสต๊อกทีละ Order — หากไม่พบแถวสต๊อก ระบบบันทึกข้อผิดพลาดด้วย **Q3S.5** (`error_code`='stock_missing') และคงสถานะ Order ไว้ให้ Super Admin ตรวจสอบ
2		**อ่านยอดจาก RSL** — เรียก `RslAdapter.getStock(sku, client)` (mock adapter อ่านยอด RSL ที่เก็บในฐานข้อมูลภายใน transaction เดียวกัน **Q3S.2:** `SELECT stock.rsl_qty FROM`

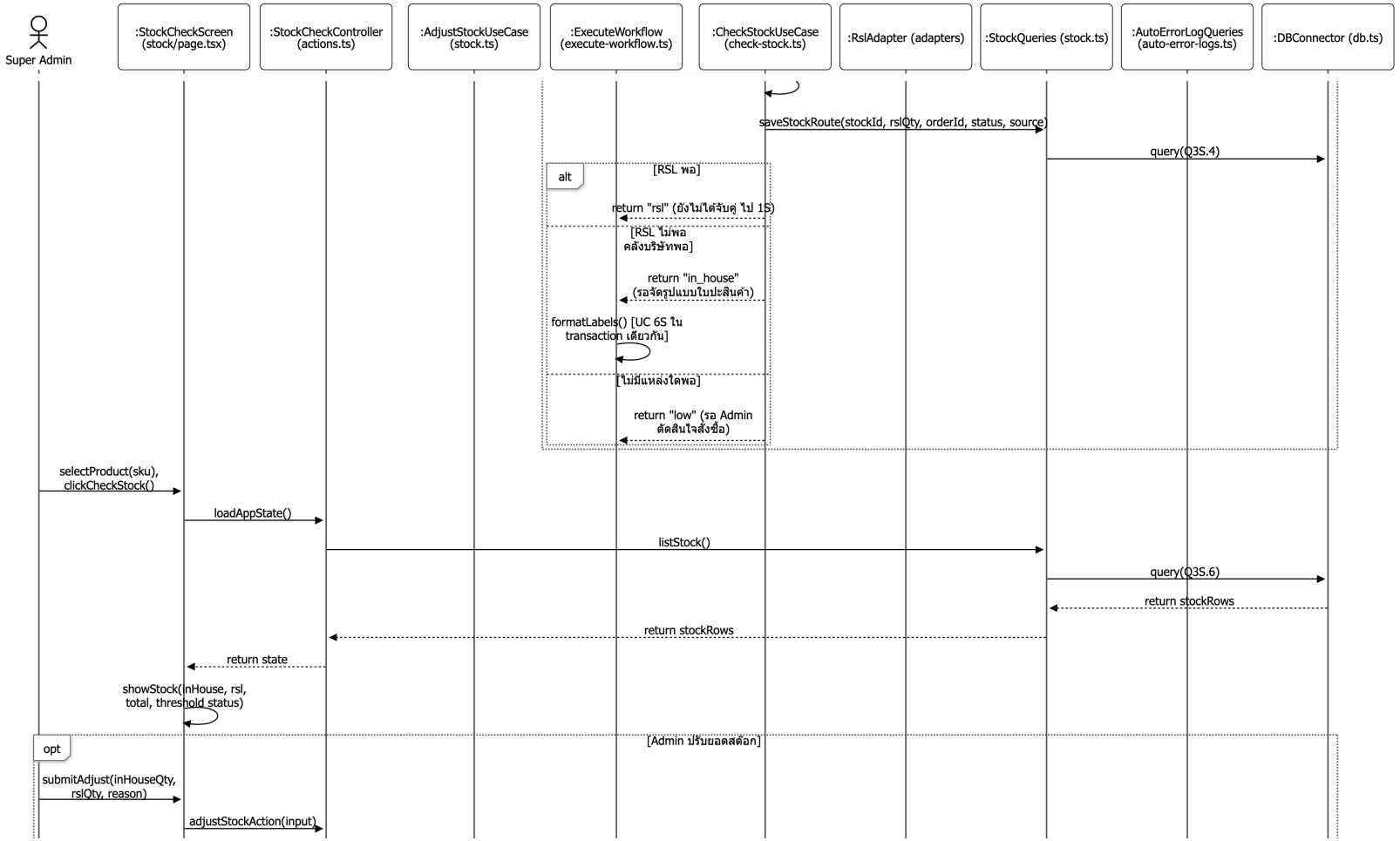
ลำดับ	User Action	System Response
		`stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1`)
3		**หักยอดที่ถูกจองไว้** — รวมจำนวนสินค้าของ Order อื่นที่เป็น SKU เดียวกันผ่าน 3S แล้ว ยังไม่ถูกตัดสต๊อก และยังอยู่ระหว่างดำเนินการ (สถานะตั้งแต่ "ยังไม่ได้จับคู่" ถึง "แจ้งเลขติดตามแล้ว" ตามค่าคงที่ `STOCK_RESERVING_STATUSES`) แยกตามแหล่ง **Q3S.3:** `SELECT COALESCE(SUM(item.qty) FILTER (WHERE item.fulfill_source = 'rsl'), 0) AS rsl_reserved, COALESCE(SUM(item.qty) FILTER (WHERE item.fulfill_source = 'in_house'), 0) AS in_house_reserved FROM order_items item INNER JOIN orders ON orders.order_id = item.order_id WHERE item.sku_id = $1 AND item.stock_deducted = FALSE AND orders.order_status = ANY($2::text[]) AND orders.order_id <> $3` ยอดที่ใช้ได้ของแต่ละแหล่ง = ยอดในแหล่งนั้น − ยอดที่ถูกจอง (Order ที่ยกเลิก ปิดแล้ว ส่งไม่สำเร็จ/ตีกลับ หรือรอดำเนินการด้วยตนเอง ไม่นับ ยอดจองจึงหายไปเองโดยไม่ต้องมีตารางจองแยก)
4		**เลือกแหล่งส่งสินค้า** (`chooseStockRoute`) — 1) ถ้ายอด RSL ที่ใช้ได้ ≥ `qty` เลือก 'rsl' 2) ถ้าไม่ใช่ และยอดคลังบริษัทที่ใช้ได้ ≥ `qty` เลือก 'in_house' 3) ถ้าไม่มีแหล่งใดพอทั้งจำนวน (แม้รวมสองแหล่งแล้วพอ) ถือว่าสต๊อกไม่พอ เพราะ 7S ตัดสต๊อกจากแหล่งเดียว บันทึกผล **Q3S.4:** `WITH latest_stock AS (UPDATE stock SET rsl_qty = COALESCE($2, rsl_qty), updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1), item_source AS (UPDATE order_items SET fulfill_source = COALESCE($5, fulfill_source) WHERE order_id = $3 AND EXISTS (SELECT 1 FROM orders WHERE order_id = $3 AND order_status = 'รอตรวจสอบสต๊อก')) UPDATE orders SET order_status = $4, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $3 AND order_status = 'รอตรวจสอบสต๊อก'` โดย `$2`=ยอด RSL ที่อ่านได้, `$4`=สถานะใหม่, `$5`='rsl' / 'in_house' / NULL — 'rsl': สถานะ "ยังไม่ได้จับคู่" ส่งต่อไปจับคู่กับ RSL (**Use Case 1S**) แสดง "[Order] สต๊อกเพียงพอ ส่งต่อไปจับคู่กับ RSL" — 'in_house': บริษัทจัดส่งเองจากคลังบริษัท สถานะ "รอจัดรูปแบบใบปะสินค้า" ข้าม 1S (ไม่มี `rsl_no`) และ `executeWorkflow` รัน **Use Case 6S** ต่อใน transaction เดียวกันทันที — ไม่พอ: สถานะ "รอ Admin ตัดสินใจสั่งซื้อ" แสดง "[Order] สต๊อกไม่เพียงพอ ส่งต่อไปคำนวณต้นทุนและรอ Admin ตัดสินใจสั่งซื้อ"
ทางเลือก #1	(ระบบตรวจพบเอง) RSL ไม่ตอบ (fault `rsl`) หรือไม่มียอด RSL	บันทึกข้อผิดพลาด **Q3S.5:** `INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, $2, $3, $4, $5)` (`use_case_id`='3S',

ลำดับ	User Action	System Response
		`error_code`='external_unavailable') ถือว่า RSL ส่งไม่ได้ ไม่เขียนทับ `rsl_qty` เดิม และเลือกจากคลังบริษัทอย่างเดียวตามขั้นตอนที่ 3–4
ทางเลือก #2	Super Admin เปิดหน้า "ตรวจสอบสต๊อก" เลือกสินค้า แล้วกด "เช็คสต๊อก"	ระบบอ่านสต๊อกทุก SKU **Q3S.6:** `SELECT sku.sku, stock.in_house_qty, stock.rsl_qty, stock.updated_at FROM stock INNER JOIN skus sku ON sku.sku_id = stock.sku_id ORDER BY sku.sku` (โหลดผ่าน `loadAppState`) แสดงคลังบริษัท คลัง RSL และยอดรวม เทียบกับเกณฑ์เติมสต๊อก: ยอดรวม ≤ `reorder_threshold` แสดง "สต๊อกต่ำกว่าเกณฑ์" มิฉะนั้นแสดง "สต๊อกปกติ" — ไม่พบสินค้า: "ไม่พบข้อมูลสินค้านี้ในระบบ" — `rsl_qty` เป็น NULL: ระบุ "ข้อมูลไม่สมบูรณ์" และแจ้งว่า "ข้อมูลสต๊อกไม่ครบถ้วน กรุณาตรวจสอบแหล่งข้อมูล" — RSL ขัดข้อง: "ไม่สามารถดึงข้อมูลสต๊อกจาก RSL ได้ กรุณาลองใหม่ภายหลัง" และแสดงเฉพาะข้อมูลสต๊อกในคลังบริษัท — ตาราง "ภาพรวมสต๊อกทุกสินค้า" เรียงรายการที่ต่ำกว่าเกณฑ์ไว้บนสุด
ทางเลือก #3	Super Admin กรอกยอดคลังบริษัท ยอดคลัง RSL (เว้นว่างได้) และเหตุผลที่ปรับ แล้วกด "บันทึกยอดสต๊อก"	`adjustStockAction` เรียก `adjustStockUseCase` ตรวจว่ายอดเป็นจำนวนเต็มตั้งแต่ 0 และมีเหตุผล (ไม่เกิน 500 ตัวอักษร) หากไม่ผ่านแจ้งว่า "กรอกจำนวนเต็มตั้งแต่ 0 ขึ้นไปและระบุเหตุผล" จากนั้นใน `executeWorkflow`: ล็อกแถวสต๊อก **Q3S.7:** `SELECT stock.stock_id, stock.sku_id, stock.in_house_qty, stock.rsl_qty FROM stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1 FOR UPDATE OF stock` บันทึกยอดใหม่และตั้งสถานะซิงก์ Rakuten เป็น 'pending' **Q3S.8:** `WITH adjusted AS (UPDATE stock SET in_house_qty = $2, rsl_qty = $3, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 RETURNING stock_id) INSERT INTO rakuten_stock_sync_states (stock_id, sync_status) SELECT stock_id, 'pending' FROM adjusted ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL` บันทึกประวัติการปรับ **Q3S.9:** `INSERT INTO stock_adjustment_history (stock_adjustment_history_id, stock_id, previous_in_house_qty, new_in_house_qty, previous_rsl_qty, new_rsl_qty, reason, admin_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)` แล้วซิงก์ยอดไป Rakuten Ichiba ตาม 7S ขั้นตอนที่ 5 แสดง "บันทึกยอดสต๊อกและประวัติแล้ว"

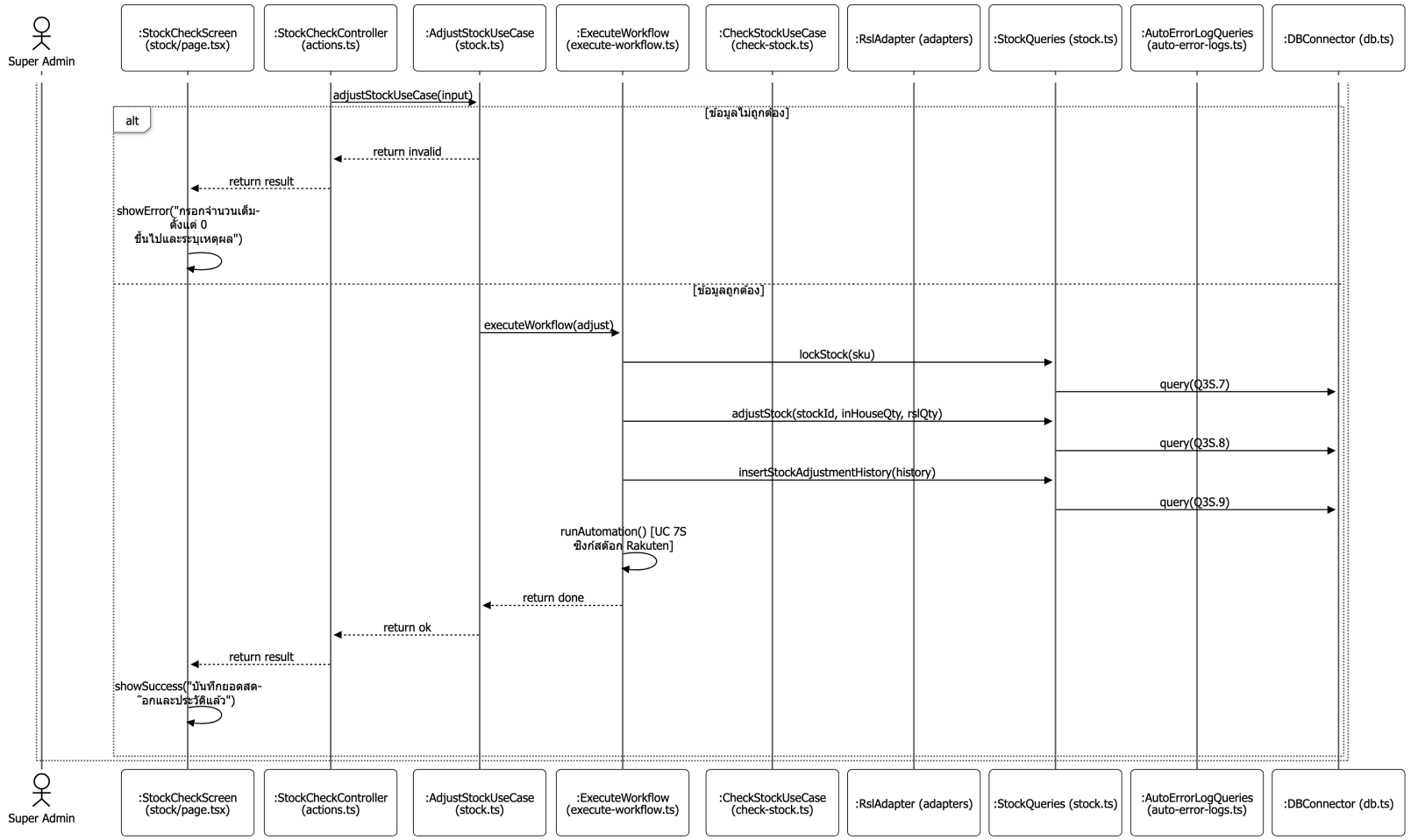
2.13.50 Sequence Diagram ของ Use Case Description ที่ 3S



รูปที่ 2-63 Sequence Diagram ของ Use Case Description ที่ 3S (1/3)

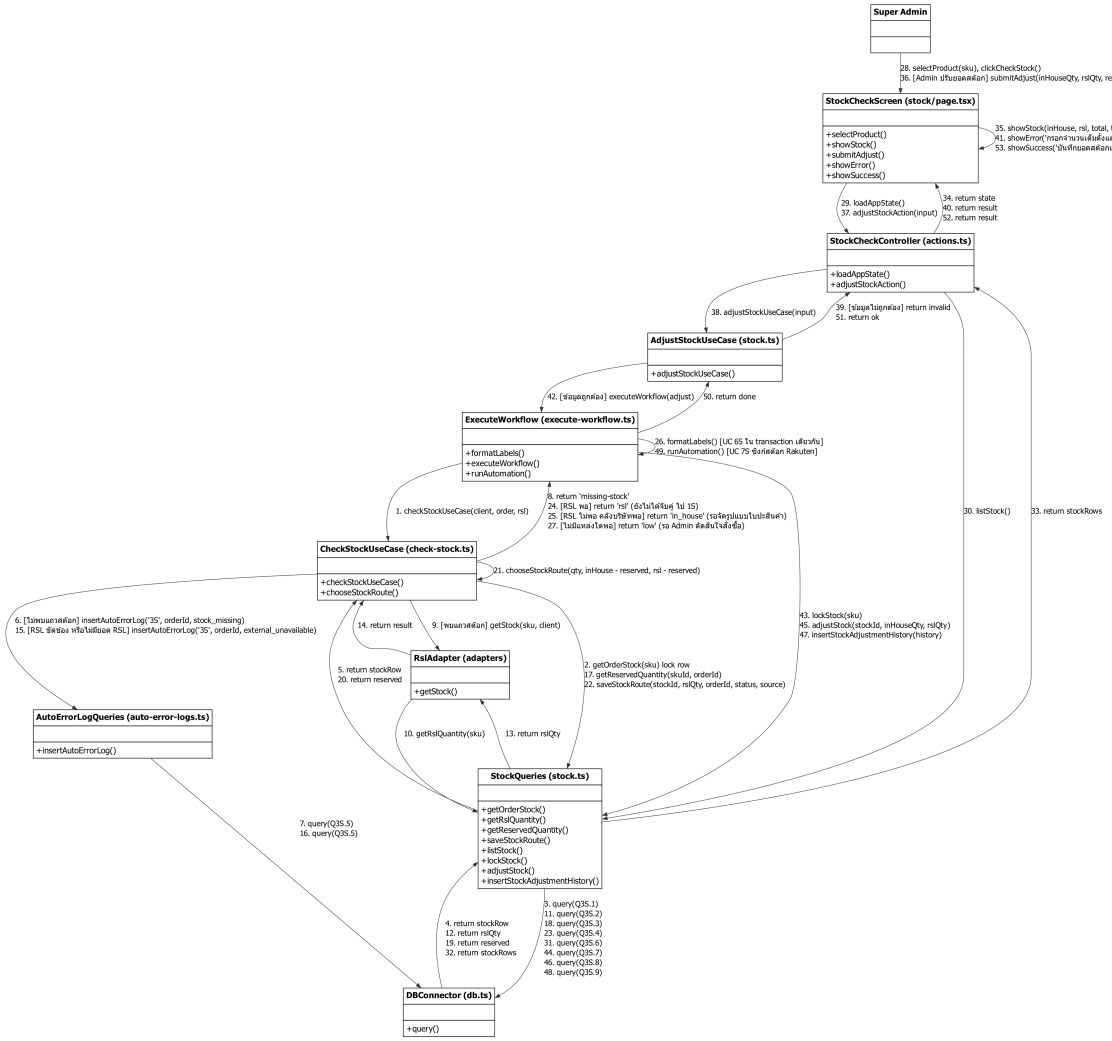


รูปที่ 2-64 Sequence Diagram ของ Use Case Description ที่ 3S (2/3)



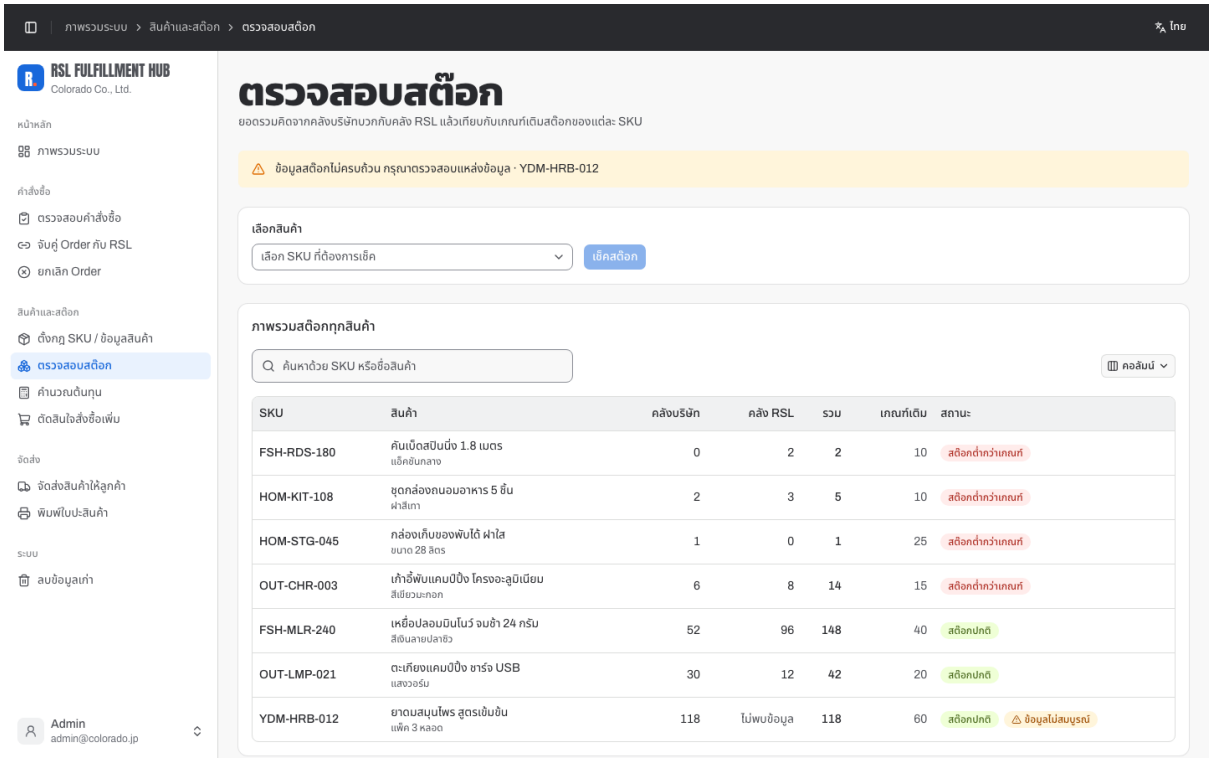
รูปที่ 2-65 Sequence Diagram ของ Use Case Description ที่ 3S (3/3)

2.13.51 Collaboration Diagram ของ Use Case Description ที่ 3S



รูปที่ 2-66 Collaboration Diagram ของ Use Case Description ที่ 3S

2.13.52 User Interface ของ Use Case Description ที่ 3S



รูปที่ 2-67 User Interface ของ Use Case Description ที่ 3S (StockCheckScreen)

2.13.53 Use Case Description ที่ 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง

Goal: ให้ Super Admin ทราบต้นทุนที่แท้จริงต่อหน่วยของสินค้า เพื่อใช้ประกอบการตัดสินใจสั่งซื้อเติมสต๊อกและกำหนดราคาขาย

Pre-Condition: SKU มีอยู่ในระบบ (3A) Super Admin เข้าสู่ระบบแล้ว

Post-Condition: ระบบแสดงต้นทุนต่อหน่วย (Unit Cost) ทุกครั้งที่คำนวณจะบันทึกประวัติใน `cost_history` และหากเลือก "บันทึกค่าที่กรอกไว้ใช้ครั้งถัดไป" จะบันทึกองค์ประกอบและ `unit_cost` ล่าสุดลง `costs` (ใช้ต่อใน 5A)

Actor: ระบบทำงานอัตโนมัติ (คำนวณเมื่อ Super Admin กด "คำนวณ" ที่หน้า `/products/cost` และเมื่อบันทึกสินค้าพร้อมองค์ประกอบต้นทุนใน 3A)

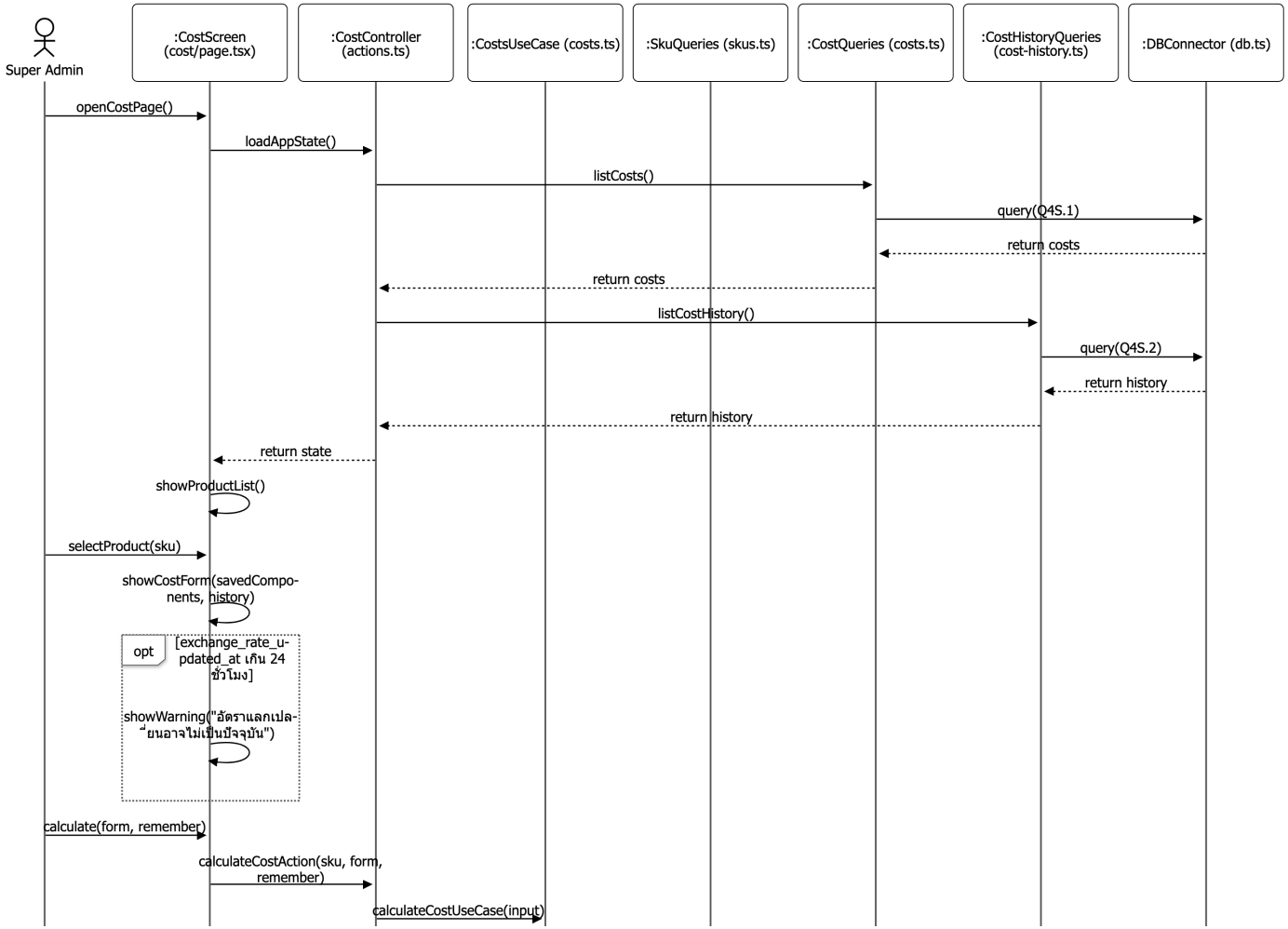
ตารางที่ 2-18 Use Case Description ที่ 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง

ลำดับ	User Action	System Response
1		หน้า `/products/cost` แสดงรายการสินค้าพร้อมช่องค้นหาด้วย SKU หรือชื่อสินค้า ข้อมูลองค์ประกอบต้นทุนที่บันทึกไว้ — **Q4S.1: SELECT**

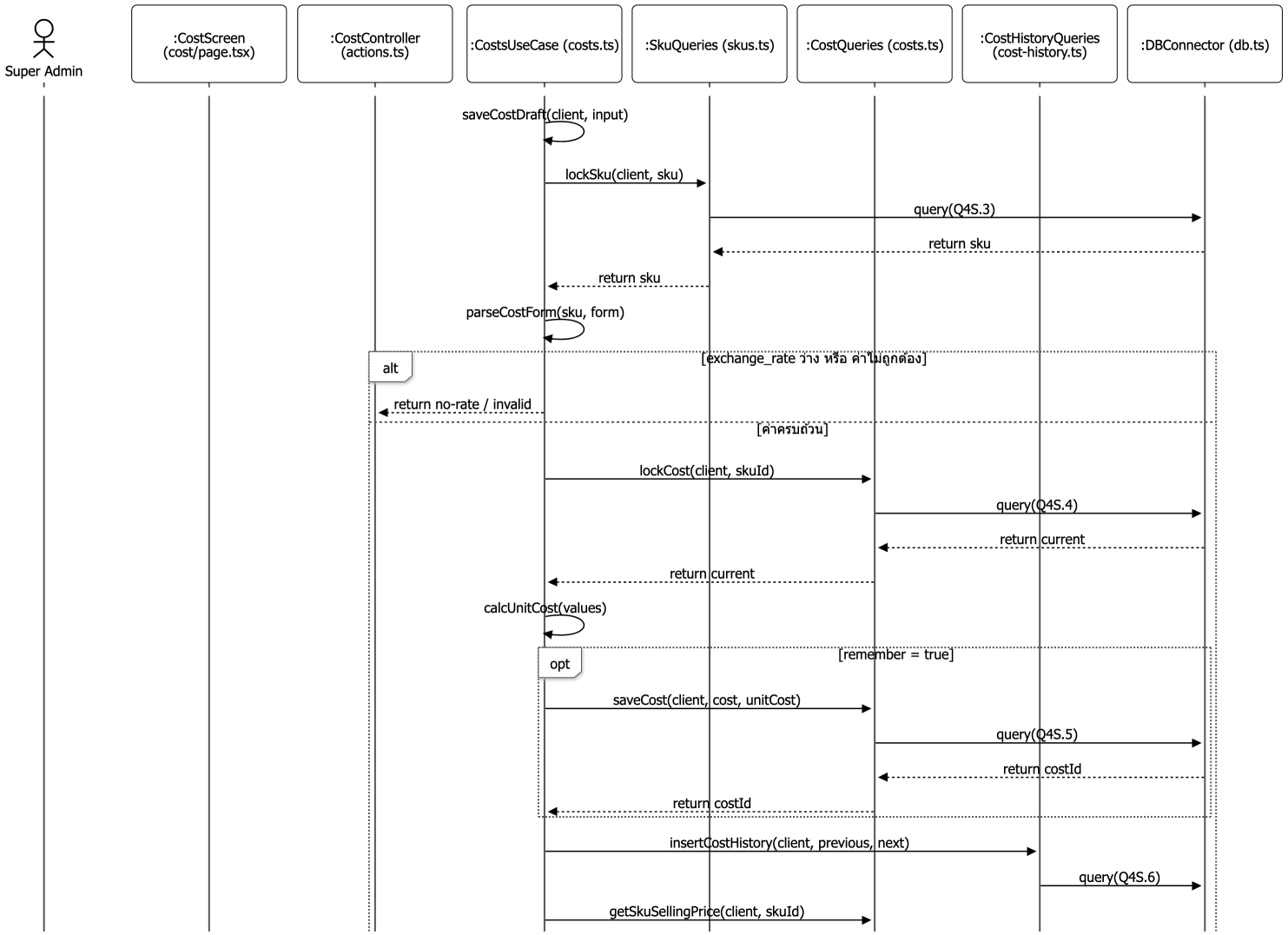
ลำดับ	User Action	System Response
		sku.sku, costs.purchase_price, costs.currency, costs.exchange_rate, costs.exchange_rate_updated_at, costs.intl_freight, costs.duty_fee, costs.marketplace_fee, costs.domestic_shipping, costs.rsl_charge, costs.order_qty, costs.unit_cost, costs.calculated_at FROM costs INNER JOIN skus sku ON sku.sku_id = costs.sku_id ORDER BY sku.sku, costs.calculated_at NULLS LAST, costs.cost_id และประวัติการคำนวณ — Q4S.2: SELECT sku.sku, history.new_value, history.admin_id, history.changed_at FROM cost_history history INNER JOIN skus sku ON sku.sku_id = history.sku_id ORDER BY history.changed_at, history.cost_history_id
2	เลือกสินค้า หรือกด "คำนวณต้นทุนเติมสต๊อก"	แสดงฟอร์ม "องค์ประกอบต้นทุน · [SKU]" เติมค่าที่บันทึกไว้ (สกุลเงินที่ซื้อ, ราคาซื้อทั้งล็อต, อัตราแลกเปลี่ยน, ค่าขนส่งระหว่างประเทศ, ภาษีนำเข้า, จำนวนที่สั่งต่อล็อต, ค่าธรรมเนียมมาร์เก็ตเพลส, ค่าส่งในประเทศ, ค่าธรรมเนียม RSL) และประวัติการคำนวณของ SKU นั้น
3	กรอก/แก้องค์ประกอบต้นทุน เลือก "บันทึกค่าที่กรอกไว้ใช้ครั้งถัดไป" (ค่าเริ่มต้นเลือกไว้) แล้วกด "คำนวณ"	ระบบเรียก Server Action calculateCostAction ⟶ calculateCostUseCase ⟶ saveCostDraft ใน transaction เดียว 1. ล็อก SKU — Q4S.3: SELECT sku_id, sku, channel_sku FROM skus WHERE sku = $1 FOR UPDATE 2. ตรวจสอบค่าที่กรอก — หากอัตราแลกเปลี่ยนว่าง จะแจ้งเตือนว่า "ไม่พบอัตราแลกเปลี่ยนล่าสุด กรุณาระบุด้วยตนเอง" หากช่องใดว่าง ไม่ใช่ตัวเลข ติดลบ หรือ order_qty/exchange_rate ≤ 0 จะแจ้งเตือนว่า "กรุณากรอกข้อมูลต้นทุนให้ครบถ้วนและถูกต้อง" 3. ล็อกต้นทุนเดิม — Q4S.4: SELECT costs.cost_id, to_jsonb(costs) AS snapshot, costs.exchange_rate, costs.exchange_rate_updated_at FROM costs WHERE costs.sku_id = $1 ORDER BY costs.calculated_at DESC NULLS LAST, costs.cost_id LIMIT 1 FOR UPDATE (ถ้าอัตราแลกเปลี่ยนไม่เปลี่ยน ใช้ exchange_rate_updated_at เดิม ถ้าเปลี่ยน ตั้งเป็นเวลาปัจจุบัน) 4. คำนวณต้นทุนต่อหน่วย (calcUnitCost) — unit_cost = ((purchase_price × exchange_rate) + intl_freight + duty_fee) / order_qty + marketplace_fee + domestic_shipping + rsl_charge 5. บันทึกองค์ประกอบ (เมื่อเลือกบันทึกค่า) — Q4S.5: INSERT INTO costs (cost_id, purchase_price, currency, exchange_rate, exchange_rate_updated_at, intl_freight, duty_fee, marketplace_fee, domestic_shipping, rsl_charge, order_qty, unit_cost, calculated_at, sku_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, CURRENT_TIMESTAMP, $13) ON CONFLICT (cost_id) DO UPDATE SET purchase_price =

ลำดับ	User Action	System Response
		`EXCLUDED.purchase_price, currency = EXCLUDED.currency, exchange_rate = EXCLUDED.exchange_rate, exchange_rate_updated_at = EXCLUDED.exchange_rate_updated_at, intl_freight = EXCLUDED.intl_freight, duty_fee = EXCLUDED.duty_fee, marketplace_fee = EXCLUDED.marketplace_fee, domestic_shipping = EXCLUDED.domestic_shipping, rsl_charge = EXCLUDED.rsl_charge, order_qty = EXCLUDED.order_qty, unit_cost = EXCLUDED.unit_cost, calculated_at = CURRENT_TIMESTAMP RETURNING cost_id` **6. บันทึกประวัติ** (ทุกครั้ง เก็บค่าก่อน/หลังเป็น JSON) — **Q4S.6:** `INSERT INTO cost_history (cost_history_id, sku_id, cost_id, previous_value, new_value, admin_id) VALUES ($1, $2, $3, $4::jsonb, $5::jsonb, $6)` **7. เทียบราคาขาย** — Q4S.7: `SELECT selling_price FROM skus WHERE sku_id = $1` แสดง "คำนวณต้นทุนต่อหน่วยแล้ว" พร้อมผลการคำนวณ ราคาซื้อคิดเป็นเงินบาท การหารด้วยจำนวนต่อล็อต ต้นทุนต่อหน่วย ราคาขายปัจจุบัน และกำไรต่อหน่วย
ทางเลือก #1	องค์ประกอบบางรายการยังไม่เคยตั้งค่า (เช่น `rsl_charge` เป็น NULL)	ฟอร์มเว้นช่องนั้นว่างและไฮไลต์ให้กรอก หากกดคำนวณโดยยังว่าง ระบบแจ้งเตือนตามขั้นตอนที่ 3 ข้อ 2 เมื่อกรอกครบและเลือก "บันทึกค่าที่กรอกไว้ใช้ครั้งถัดไป" ค่าจะถูกบันทึกด้วย **Q4S.5**
ทางเลือก #2	ต้นทุนต่อหน่วยสูงกว่าราคาขายปัจจุบัน (ผลจาก **Q4S.7**)	แสดงคำเตือนสีแดง "ต้นทุนต่อหน่วยสูงกว่าราคาขาย กรุณาตรวจสอบราคาขายหรือองค์ประกอบต้นทุน"
ทางเลือก #3	ต้องการเปรียบเทียบต้นทุนจากซัพพลายเออร์หลายราย	ตาราง "เปรียบเทียบซัพพลายเออร์" คำนวณ `unit_cost` ด้วยสูตรเดียวกัน ใช้องค์ประกอบอื่นเท่าเดิม เปลี่ยนเฉพาะราคาซื้อ อัตราแลกเปลี่ยน และค่าขนส่งระหว่างประเทศ ตามราคาเสนอของซัพพลายเออร์ทางเลือก (ข้อมูลตัวอย่างในโค้ด ไม่ได้เก็บในฐานข้อมูล) หากไม่มีแสดง "ยังไม่มีราคาจากซัพพลายเออร์ทางเลือกสำหรับสินค้านี้"
ทางเลือก #4	อัตราแลกเปลี่ยนที่บันทึกไว้อัปเดตเกิน 24 ชั่วโมง (`exchange_rate_updated_at`)	แสดงคำเตือน "อัตราแลกเปลี่ยนอาจไม่เป็นปัจจุบัน กรุณาตรวจสอบก่อนคำนวณ" ก่อนกดคำนวณ
ทางเลือก #5	บันทึกสินค้าใน 3A พร้อมองค์ประกอบต้นทุน	`saveProductUseCase` เรียก `saveCostDraft` ใน transaction เดียวกับการบันทึกสินค้า ยอมให้บางช่องว่างได้ (**Q4S.3**-**Q4S.6**) ถ้าครบทุกช่องจึงคำนวณ `unit_cost` ถ้าไม่ครบเก็บ `unit_cost` เป็น NULL

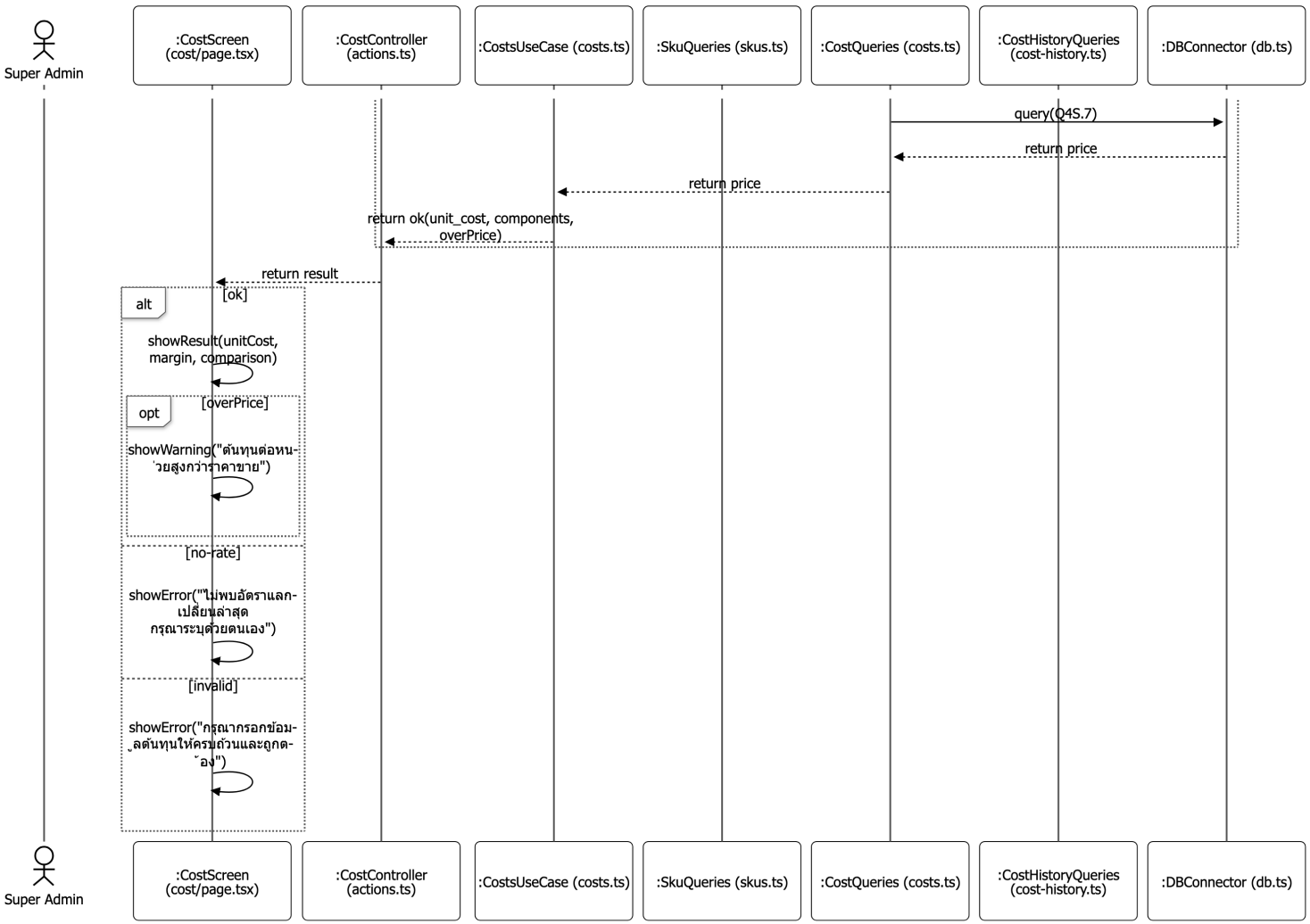
2.13.54 Sequence Diagram ของ Use Case Description ที่ 4S



รูปที่ 2-68 Sequence Diagram ของ Use Case Description ที่ 4S (1/3)

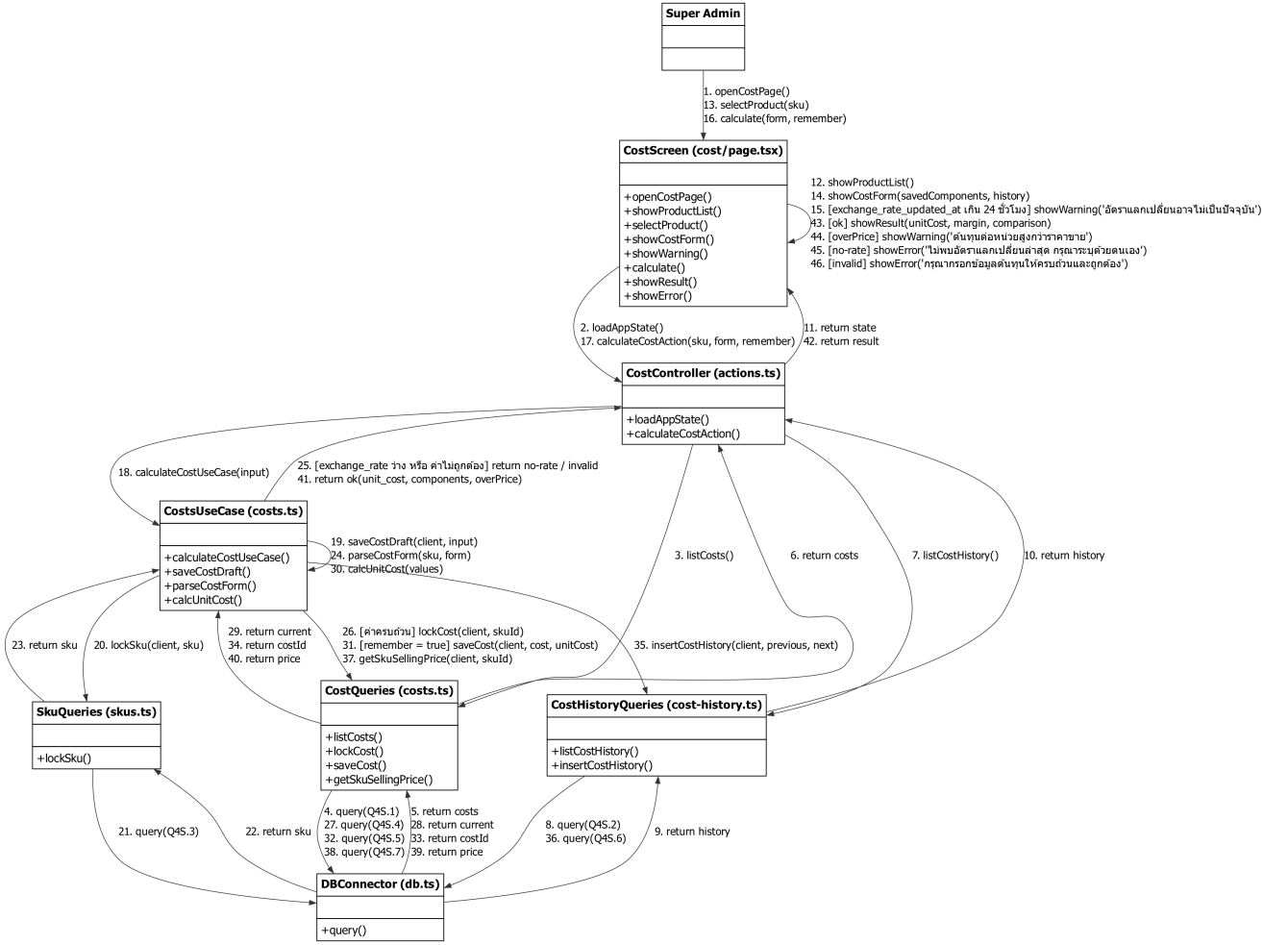


รูปที่ 2-69 Sequence Diagram ของ Use Case Description ที่ 4S (2/3)



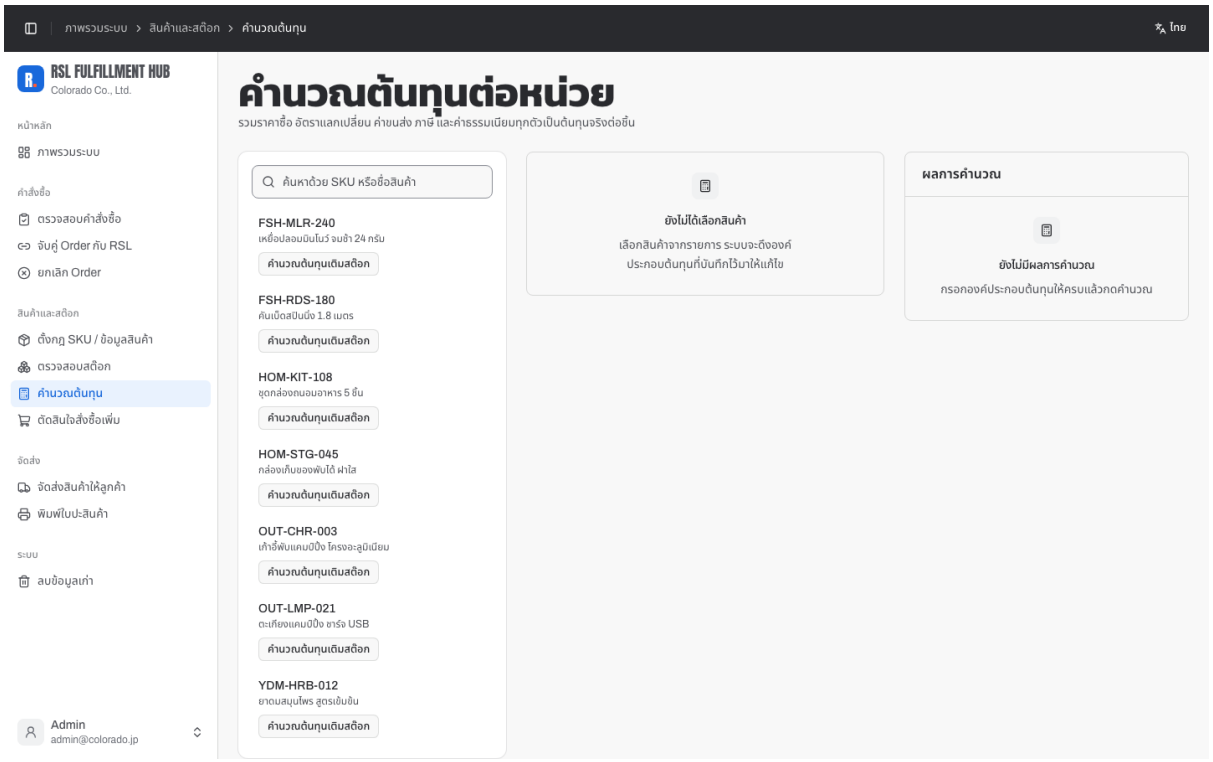
รูปที่ 2-70 Sequence Diagram ของ Use Case Description ที่ 4S (3/3)

2.13.55 Collaboration Diagram ของ Use Case Description ที่ 4S



รูปที่ 2-71 Collaboration Diagram ของ Use Case Description ที่ 4S

2.13.56 User Interface ของ Use Case Description ที่ 4S



รูปที่ 2-72 User Interface ของ Use Case Description ที่ 4S (CostScreen)

2.13.57 Use Case Description ที่ 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace

Goal: ให้ลูกค้าได้รับหมายเลขติดตามพัสดุ (Tracking Number) ผ่าน Rakuten Ichiba (RMS) พร้อมเก็บประวัติการส่งทุกครั้ง และส่งต่อให้ระบบตัดสต๊อก (7S) อัตโนมัติ

Pre-Condition: Order มี `tracking_number` และ `carrier_name` จากการส่งมอบ (8A) แล้ว (สถานะ "อยู่ระหว่างจัดส่ง" หรือ "รอแจ้งเลขติดตาม")

Post-Condition: `tracking_notification_logs` มีรายการผล `sent`, Order มี `tracking_notified`=true และสถานะ "แจ้งเลขติดตามแล้ว" (หรือ "จัดส่งสำเร็จ" ถ้าผู้ขนส่งยืนยันส่งถึงและตัดสต๊อกครบทุกรายการแล้ว) จากนั้น `executeWorkflow` รัน 7S ใน transaction เดียวกัน

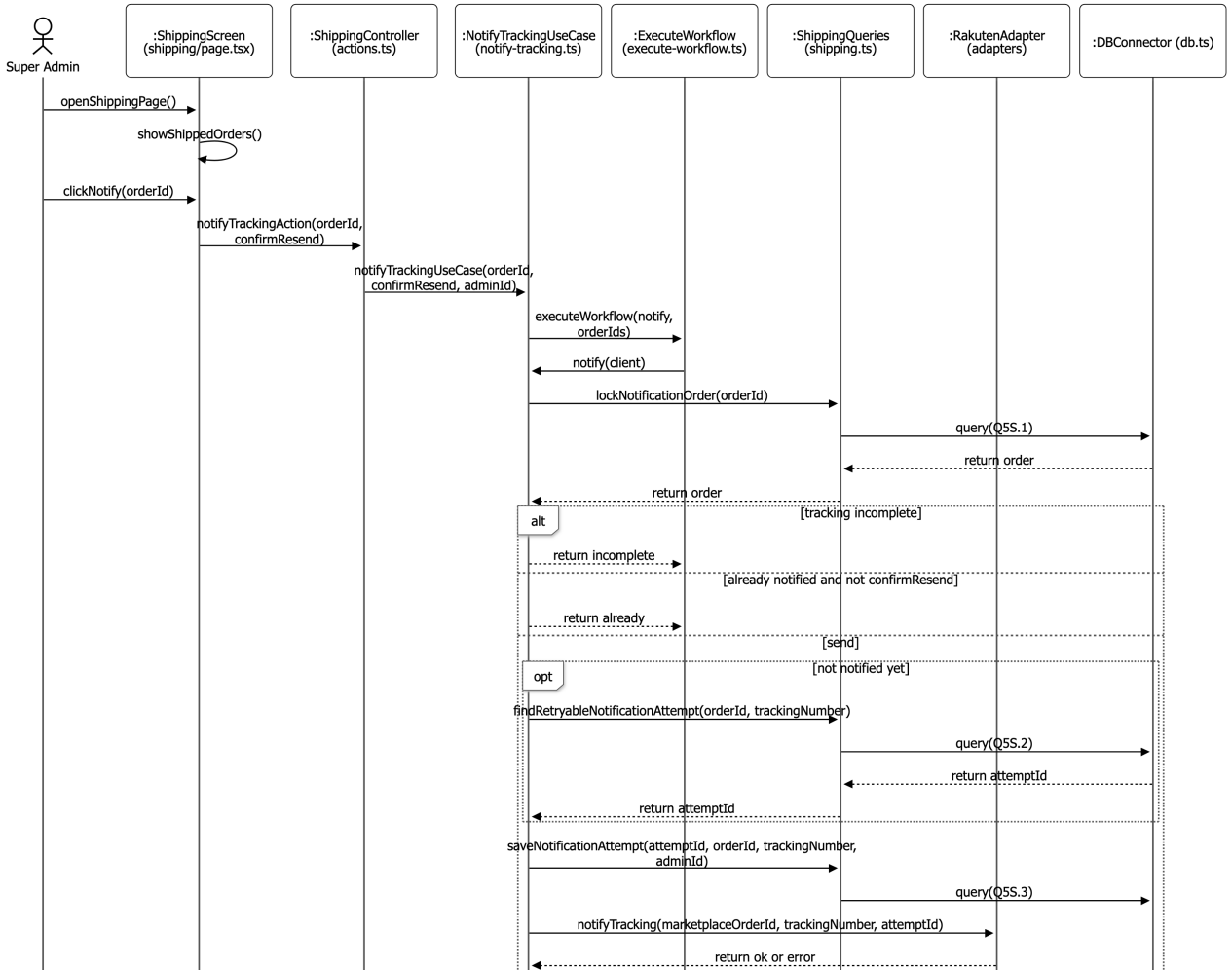
Actor: Super Admin (กดส่งจากหน้าจอ), Marketplace, Delivery

ตารางที่ 2-19 Use Case Description ที่ 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace

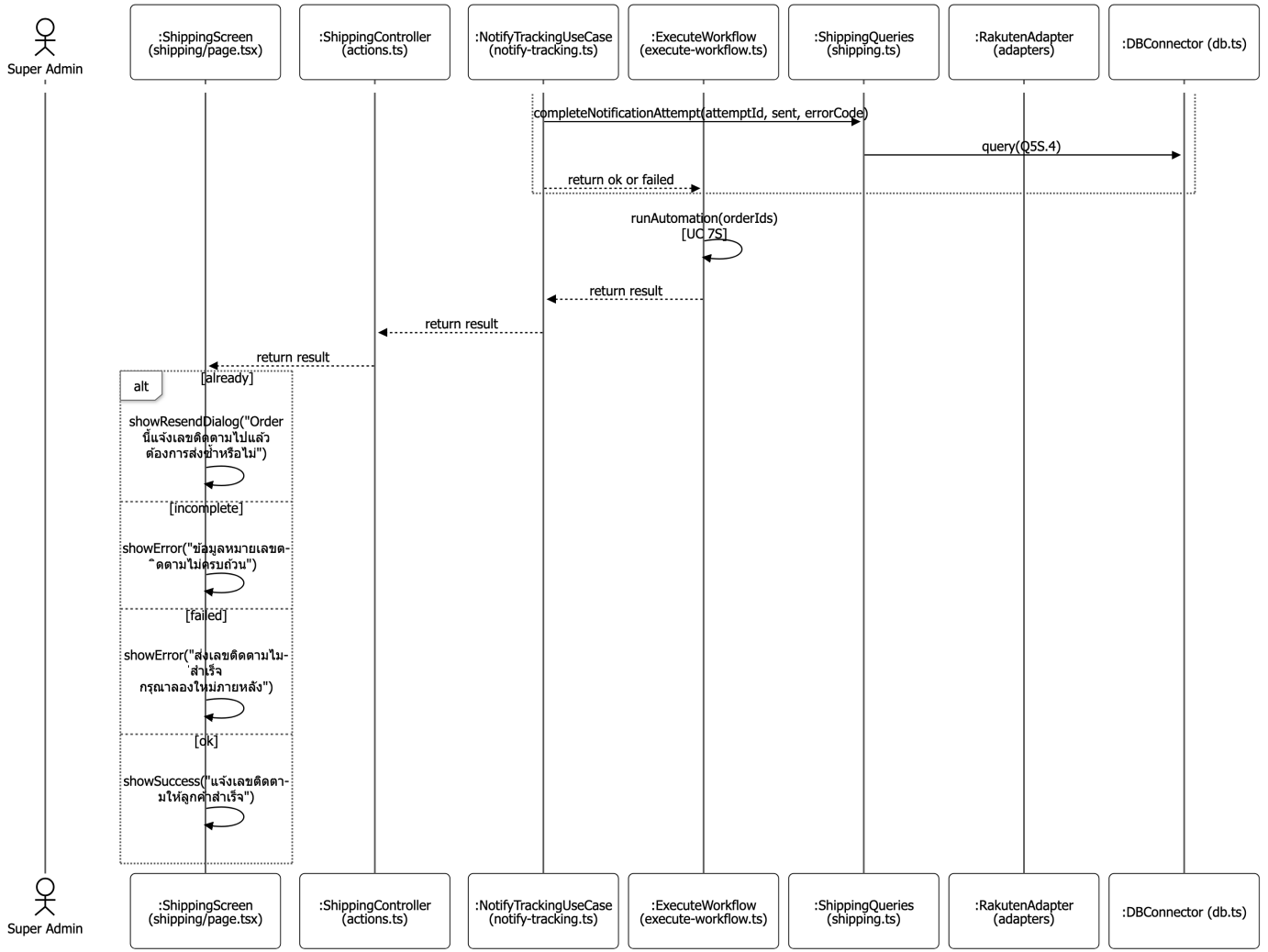
ลำดับ	User Action	System Response
1		หน้า "จัดส่งสินค้าให้ลูกค้า" (`/shipping`) ส่วน "ส่งมอบแล้ว" แสดงหมายเลขติดตาม ผู้ขนส่ง สถานะจากขนส่ง และสถานะแจ้งลูกค้า ("แจ้งแล้ว"/"ยังไม่แจ้ง")
2	Super Admin กด "ส่งเลขติดตามให้ลูกค้า"	ระบบเรียก `notifyTrackingAction(orderId, false)` ⟶ `notifyTrackingUseCase` ซึ่งทำงานภายใน `executeWorkflow`
3		**1. ล็อกและตรวจสอบ Order** — Q5S.1: `SELECT order_id, order_status, marketplace_order_id, tracking_number, carrier_name, tracking_notified, delivery_status FROM orders WHERE order_id = $1 FOR UPDATE` — หากไม่มี `tracking_number` หรือ `carrier_name` จะแจ้งเตือนว่า "ข้อมูลหมายเลขติดตามไม่ครบถ้วน ไม่สามารถแจ้งลูกค้าได้" — หาก `tracking_notified`=true และยังไม่ยืนยันส่งซ้ำ ไปทางเลือก #2 **2. เตรียม attempt** — ค้นหา attempt เดิมที่ค้างหรือล้มเหลวของเลขเดียวกันเพื่อใช้ `attempt_id` เดิม (กันส่งซ้ำ) Q5S.2: `SELECT attempt_id FROM tracking_notification_logs WHERE order_id = $1 AND tracking_number = $2 AND outcome IN ('pending', 'failed') ORDER BY sent_at DESC LIMIT 1` ถ้าไม่มีจึงสร้าง `attempt_id` ใหม่ แล้วบันทึกสถานะ `pending` พร้อม `admin_id` Q5S.3: `INSERT INTO tracking_notification_logs (tracking_notification_log_id, attempt_id, order_id, tracking_number, outcome, sent_at, admin_id, error_code) VALUES ($1, $2, $3, $4, 'pending', CURRENT_TIMESTAMP, $5, NULL) ON CONFLICT (attempt_id) DO UPDATE SET outcome = 'pending', sent_at = CURRENT_TIMESTAMP, admin_id = EXCLUDED.admin_id, error_code = NULL` **3. ส่งเลขติดตามผ่าน Marketplace** — เรียก `RakutenAdapter.notifyTracking(marketplace_order_id, tracking_number, attemptId)` **4. บันทึกผล** — Q5S.4: `WITH finished AS (UPDATE tracking_notification_logs SET outcome = $2, error_code = $3, sent_at = CURRENT_TIMESTAMP WHERE attempt_id = $1 RETURNING order_id) UPDATE orders o SET tracking_notified = CASE WHEN $2 = 'sent' THEN TRUE ELSE tracking_notified END, tracking_notified_at = CASE WHEN $2 = 'sent' THEN CURRENT_TIMESTAMP ELSE tracking_notified_at END, order_status = CASE WHEN $2 <> 'sent' AND tracking_notified = FALSE THEN 'รอแจ้งเลขติดตาม' WHEN $2 = 'sent' AND delivery_status = 'delivered' AND EXISTS (SELECT 1 FROM order_items WHERE order_id = o.order_id) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = o.order_id AND stock_deducted = FALSE`

ลำดับ	User Action	System Response
		) THEN 'จัดส่งสำเร็จ' WHEN $2 = 'sent' AND tracking_notified = FALSE THEN 'แจ้งเลขติดตามแล้ว' ELSE order_status END, updated_at = CURRENT_TIMESTAMP FROM finished WHERE o.order_id = finished.order_id ถ้าส่งสำเร็จ ตั้ง tracking_notified=true สถานะ "แจ้งเลขติดตามแล้ว" (หรือ "จัดส่งสำเร็จ" ถ้า delivery_status='delivered' และตัดสต๊อกแล้ว) แจ้งว่า "แจ้งเลขติดตามให้ลูกค้าสำเร็จ"
4		executeWorkflow รันงานอัตโนมัติต่อใน transaction เดียวกัน: Order สถานะ "แจ้งเลขติดตามแล้ว" เข้าสู่ **Use Case 7S** (ตัดสต๊อก) และถ้าผู้ขนส่งยืนยันส่งถึงแล้วจะปิดงานเป็น "จัดส่งสำเร็จ" (ดู 8A)
ทางเลือก #1	การส่งข้อมูลไปยัง Marketplace ล้มเหลว (API Error)	บันทึกผล failed พร้อม error_code (**Q5S.4**) Order ที่ยังไม่เคยแจ้งเปลี่ยนเป็น "รอแจ้งเลขติดตาม" แจ้งเตือนว่า "ส่งเลขติดตามไม่สำเร็จ กรุณาลองใหม่ภายหลัง" เมื่อกดส่งใหม่ ระบบใช้ attempt_id เดิม (**Q5S.2**)
ทางเลือก #2	Super Admin ต้องการส่งเลขติดตามซ้ำ (เช่น ลูกค้าแจ้งว่ายังไม่ได้รับ)	ระบบถามว่า "Order นี้แจ้งเลขติดตามไปแล้ว ต้องการส่งซ้ำหรือไม่" และ "ระบบจะบันทึก Log การส่งซ้ำพร้อมเวลาและผู้ดำเนินการ" เมื่อกด "ยืนยันส่งซ้ำ" ระบบเรียก notifyTrackingAction(orderId, true) สร้าง attempt ใหม่พร้อม admin_id (**Q5S.3**) ส่งอีกครั้ง และบันทึกผล (**Q5S.4**) โดยไม่เปลี่ยนสถานะ Order
ทางเลือก #3	ผู้ขนส่งยืนยันส่งถึง (delivered) และตัดสต๊อกครบทุกรายการแล้วก่อนแจ้งเลขติดตาม	เมื่อส่งเลขสำเร็จ **Q5S.4** เปลี่ยนสถานะเป็น "จัดส่งสำเร็จ" ทันที เพราะครบทั้ง 3 เงื่อนไข

2.13.58 Sequence Diagram ของ Use Case Description ที่ 5S

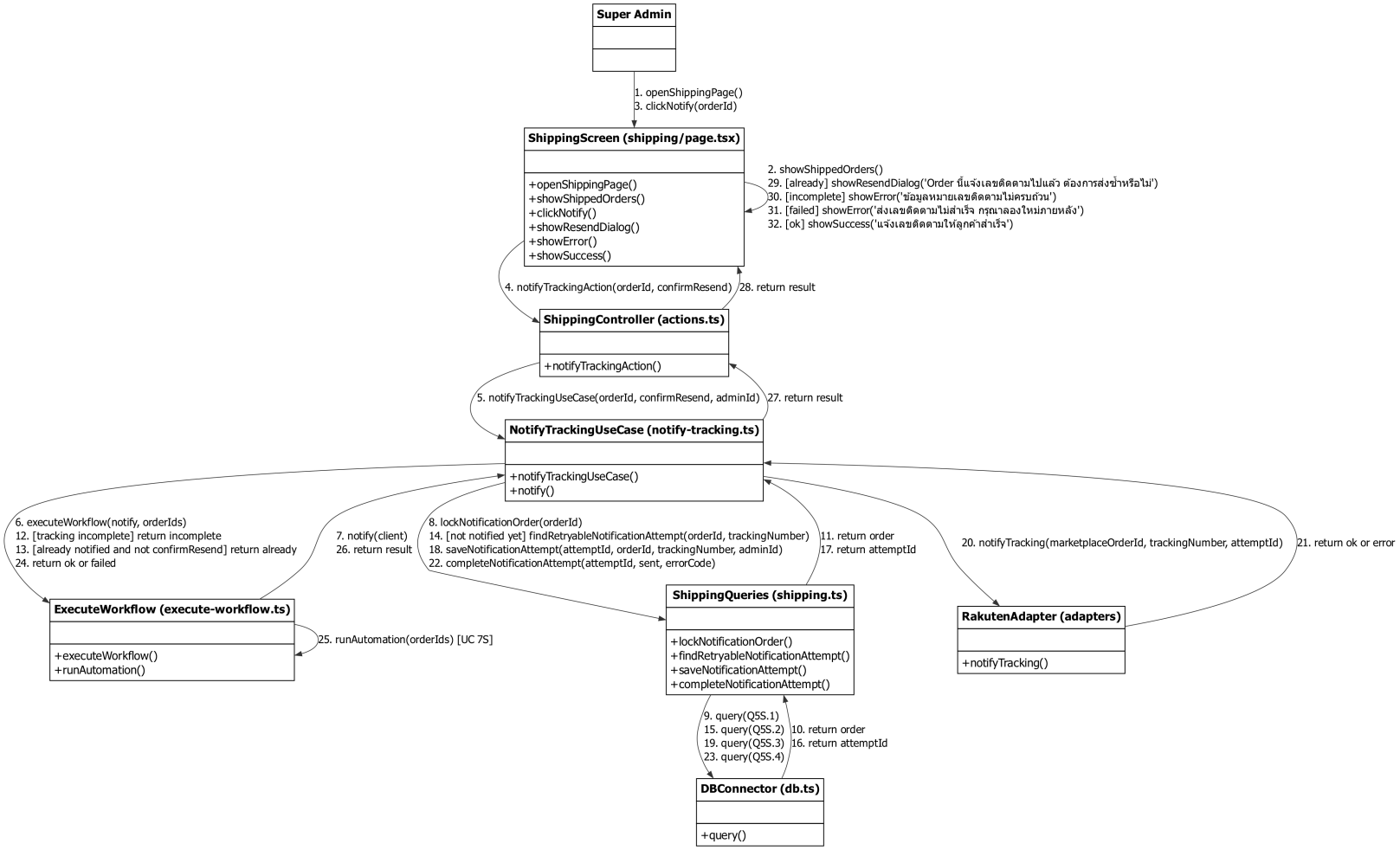


รูปที่ 2-73 Sequence Diagram ของ Use Case Description ที่ 5S (1/2)



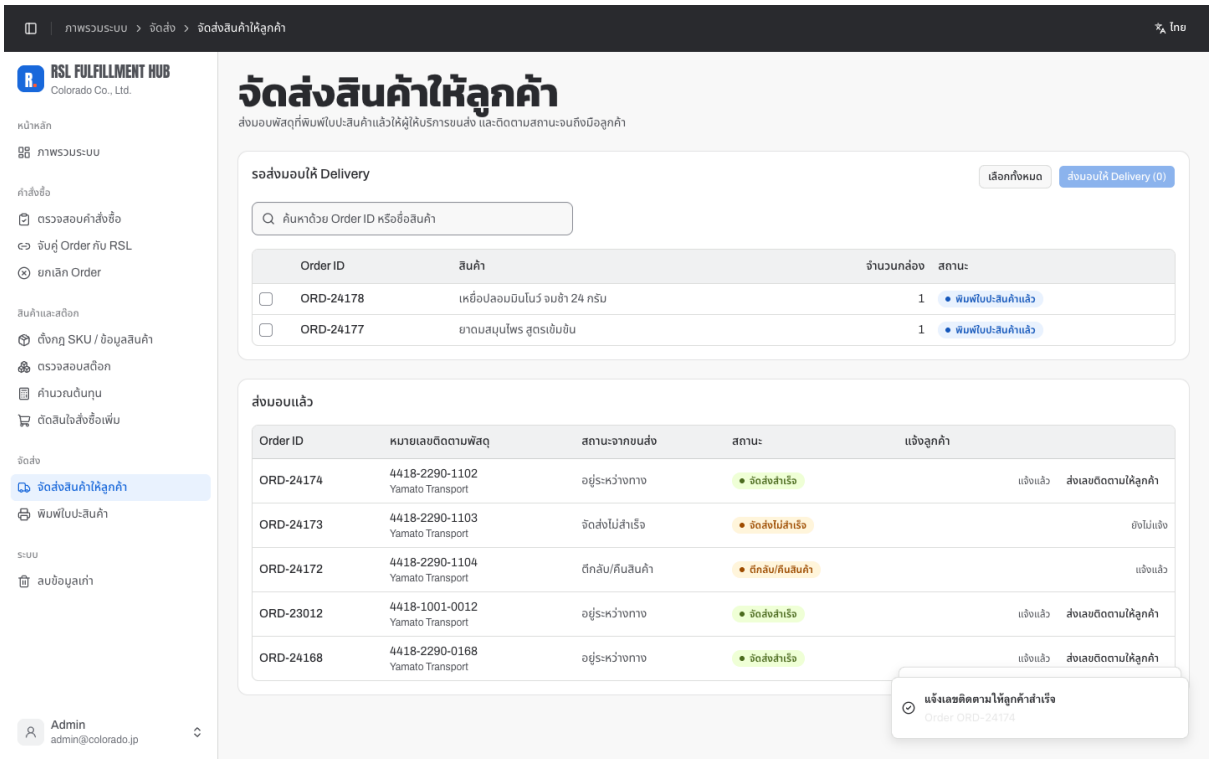
รูปที่ 2-74 Sequence Diagram ของ Use Case Description ที่ 5S (2/2)

2.13.59 Collaboration Diagram ของ Use Case Description ที่ 5S



รูปที่ 2-75 Collaboration Diagram ของ Use Case Description ที่ 5S

2.13.60 User Interface ของ Use Case Description ที่ 5S



รูปที่ 2-76 User Interface ของ Use Case Description ที่ 5S (ShippingScreen หลังแจ้งเลขติดตาม)

2.13.61 Use Case Description ที่ 6S: จัดรูปแบบใบปะสินค้า

Goal: ให้ระบบสร้างใบปะสินค้า (Shipping Label) ที่มีรูปแบบถูกต้องตามวิธีจัดส่งและช่องทางขายของแต่ละ Order โดยอัตโนมัติ

Pre-Condition: Order อยู่ในสถานะ "รอจัดรูปแบบใบปะสินค้า" โดยมาจาก 1S (RSL จัดส่ง มี `rsl_no` แล้ว) หรือมาจาก 3S โดยตรง (บริษัทจัดส่งเองจากคลังบริษัท `fulfill_source`='in_house' ไม่มี `rsl_no`)

Post-Condition: ตาราง `labels` มีใบปะสินค้าของ Order ครบทุกกล่อง (`label_status`='ready') และ Order มีสถานะ "รอพิมพ์ใบปะสินค้า" (9A) หรือ "รอดำเนินการด้วยตนเอง" ถ้าข้อมูลไม่พอ

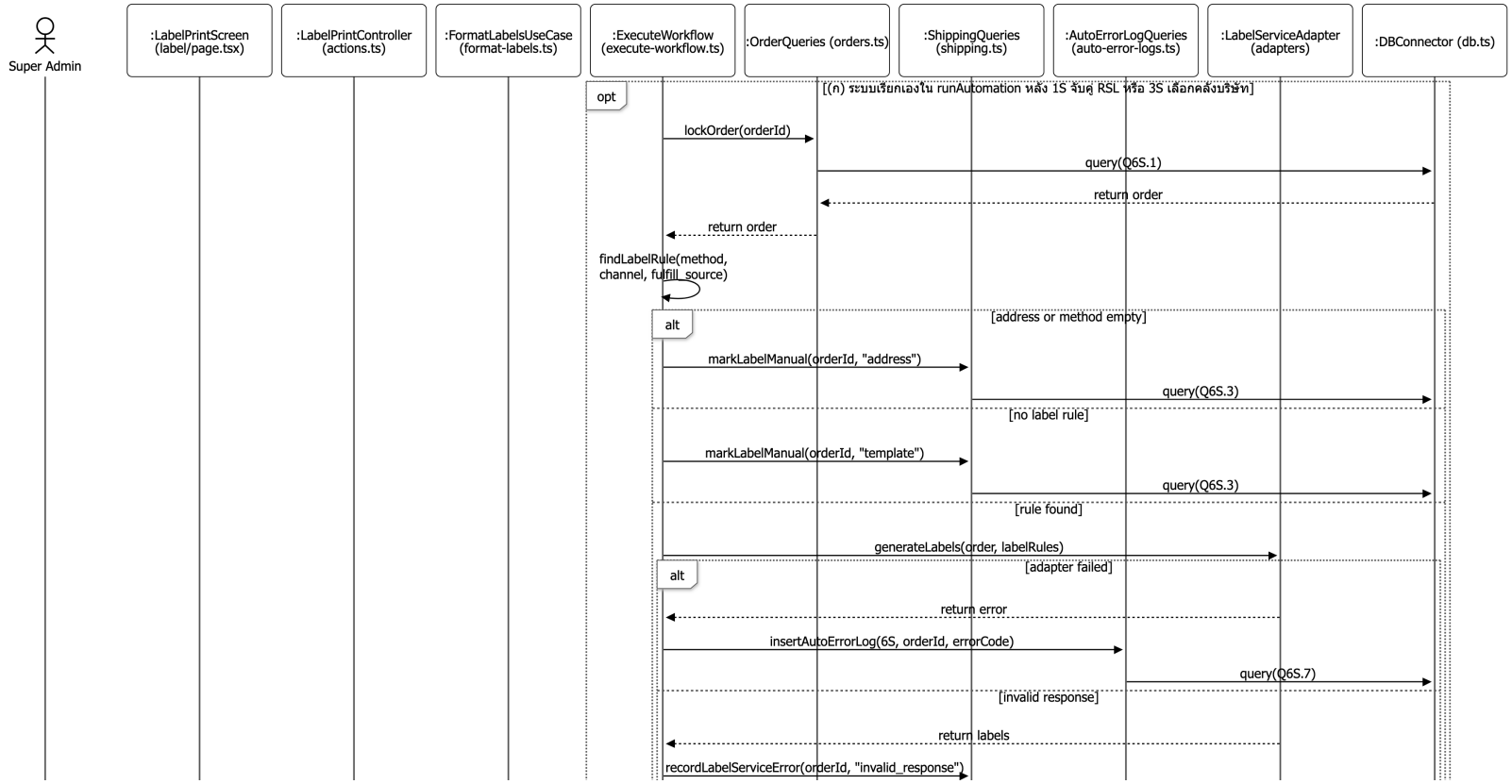
Actor: ระบบทำงานอัตโนมัติ (Label Service)

ตารางที่ 2-20 Use Case Description ที่ 6S: จัดรูปแบบใบปะสินค้า

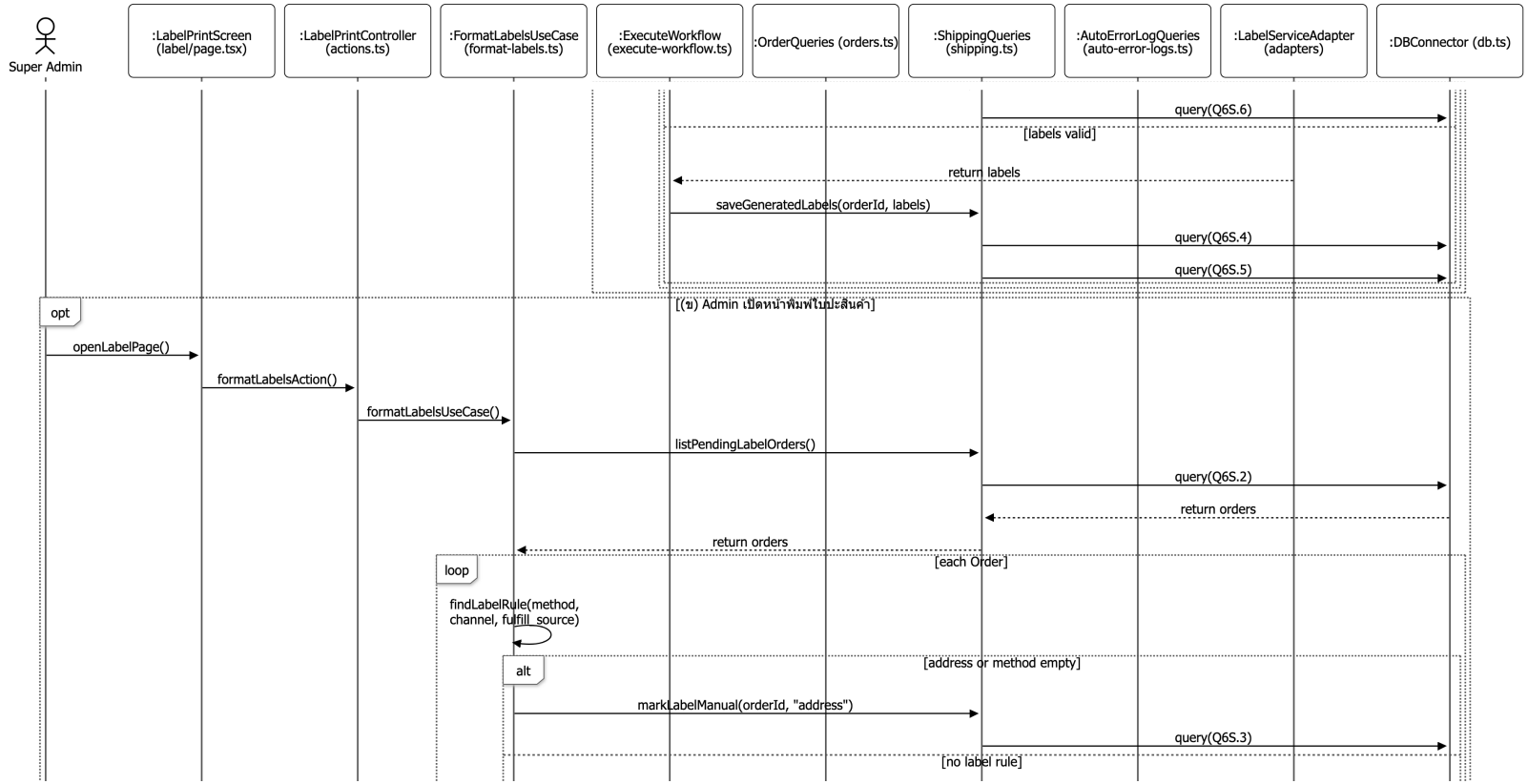
ลำดับ	User Action	System Response
1		**จุดเริ่มทำงาน** มี 2 ทาง (ก) ทันทีหลัง 1S บันทึกผลจับคู่ (Order ที่ RSL จัดส่ง) หรือหลัง 3S เลือกคลังบริษัท (Order ที่บริษัทจัดส่งเอง) `executeWorkflow` ล็อก Order Q6S.1: `SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o` แล้วทำ 6S ใน transaction เดียวกัน (ข) เมื่อ Super Admin เปิดหน้า `/shipping/label` ระบบเรียก `formatLabelsAction` ⟶ `formatLabelsUseCase` ดึง Order ที่รอจัดรูปแบบทั้งหมด Q6S.2: `SELECT o.order_id, o.sales_channel, o.rsl_no, o.shipping_address, o.shipping_method, item.qty, item.fulfill_source, o.order_status FROM orders o INNER JOIN LATERAL (SELECT qty, fulfill_source FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE WHERE o.order_status = 'รอจัดรูปแบบใบปะสินค้า' ORDER BY o.order_date, o.order_id` แล้วทำทีละ Order (transaction แยกต่อ Order)
2		**1. ตรวจสอบข้อมูล Order และเลือกรูปแบบใบปะสินค้า** — หาก `shipping_address` หรือ `shipping_method` ว่าง ระบบบันทึก Q6S.3: `UPDATE orders SET order_status = 'รอดำเนินการด้วยตนเอง', manual_reason = $2, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจัดรูปแบบใบปะสินค้า'` ด้วย `manual_reason`='address' และแจ้งเตือนว่า "ข้อมูลที่อยู่จัดส่งไม่ครบถ้วน ไม่สามารถสร้างใบปะสินค้าได้" — ระบบหา label rule จากกฎการจัดส่ง (`SHIPPING_RULES` ใน `src/mock/delivery.ts`) ตามแหล่งที่จัดส่ง (`fulfill_source` ที่ 3S บันทึก, `findLabelRule`): Order ที่ RSL จัดส่งใช้รูปแบบตามวิธีจัดส่ง (RSL ปกติ: RSL-STD-A6 กล่องละ 5 ชิ้น, RSL ขนาดใหญ่: RSL-LARGE-A5 กล่องละ 1 ชิ้น) และพิมพ์เลข RSL บนใบปะ Order ที่บริษัทจัดส่งเองใช้รูปแบบของบริษัท INHOUSE-YAMATO-A6 (กล่องละ 5 ชิ้น) โดย

ลำดับ	User Action	System Response
		`labels.rsl_no` เป็น NULL หากไม่พบ บันทึก (**Q6S.3**) ด้วย `manual_reason`='template' และแจ้งเตือนว่า "ไม่พบรูปแบบใบปะสินค้าที่เหมาะสม กรุณาตั้งค่า Label Template ก่อน"
3		**2. สร้างใบปะสินค้าและอัปเดตสถานะ** เรียก `LabelServiceAdapter.generateLabels(order, rules)` แล้วตรวจผลด้วย `validGeneratedLabels` บันทึกใบปะทีละกล่อง (`label_id` = `[Order ID]-LABEL-[ลำดับ]`, ซ้ำไม่ได้ จึงรันซ้ำได้ปลอดภัย) **Q6S.4:** `INSERT INTO labels (label_id, label_template, rsl_no, parcel_seq, parcel_total, label_status, created_at, order_id) SELECT $1, $2, $3, $4, $5, 'ready', CURRENT_TIMESTAMP, o.order_id FROM orders o WHERE o.order_id = $6 AND o.order_status = 'รอจัดรูปแบบใบปะสินค้า' ON CONFLICT (order_id, parcel_seq) DO NOTHING` เลื่อนสถานะเป็น "รอพิมพ์ใบปะสินค้า" **Q6S.5:** `UPDATE orders SET order_status = 'รอพิมพ์ใบปะสินค้า', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจัดรูปแบบใบปะสินค้า' AND EXISTS (SELECT 1 FROM labels WHERE order_id = $1)` — แจ้งว่า "สร้างใบปะสินค้าสำเร็จ [จำนวน] ใบ" และส่ง Order ไปที่ **Use Case 9A** — Order หลายกล่อง (Multi-parcel): จำนวนกล่องคำนวณจาก `max_items_per_parcel` ของ rule ใบปะแต่ละใบมี `parcel_seq/parcel_total` เช่น 1/2, 2/2
ทางเลือก #1		บริการสร้าง Label ล้มเหลวหรือส่งผลที่ไม่ถูกต้อง: แจ้งเตือนว่า "ไม่สามารถสร้างใบปะสินค้าได้ กรุณาลองใหม่ภายหลัง" คงสถานะ "รอจัดรูปแบบใบปะสินค้า" และบันทึก error — ทาง (ข) และกรณีผลไม่ถูกต้อง (`invalid_response`): **Q6S.6:** `INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, '6S', $2, $3, 'Label service could not generate labels')` — ทาง (ก) เมื่อ adapter ล้มเหลว: `logAdapterFailure` บันทึก **Q6S.7:** `INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, $2, $3, $4, $5)` ระบบลองใหม่เมื่อเปิดหน้า `/shipping/label` ครั้งถัดไป
ทางเลือก #2		เกิด error ของฐานข้อมูลระหว่าง 6S ในทาง (ก): ทั้ง transaction (รวมผลจับคู่ 1S) ถูก rollback แล้ว `executeWorkflow` บันทึก error ใน transaction ใหม่ (**Q6S.7**) ด้วย `use_case_id`='6S'

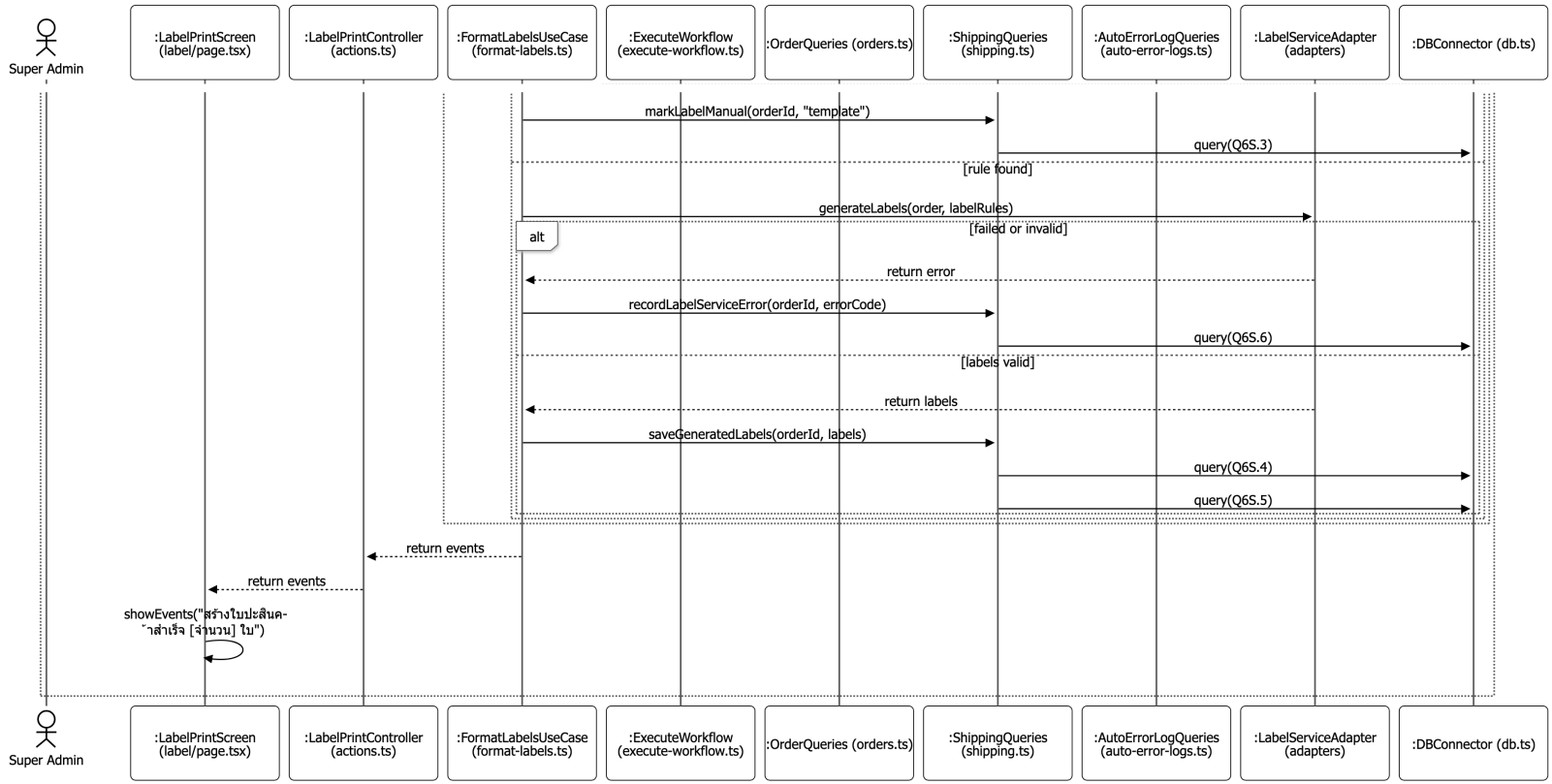
2.13.62 Sequence Diagram ของ Use Case Description ที่ 6S



รูปที่ 2-77 Sequence Diagram ของ Use Case Description ที่ 6S (1/3)

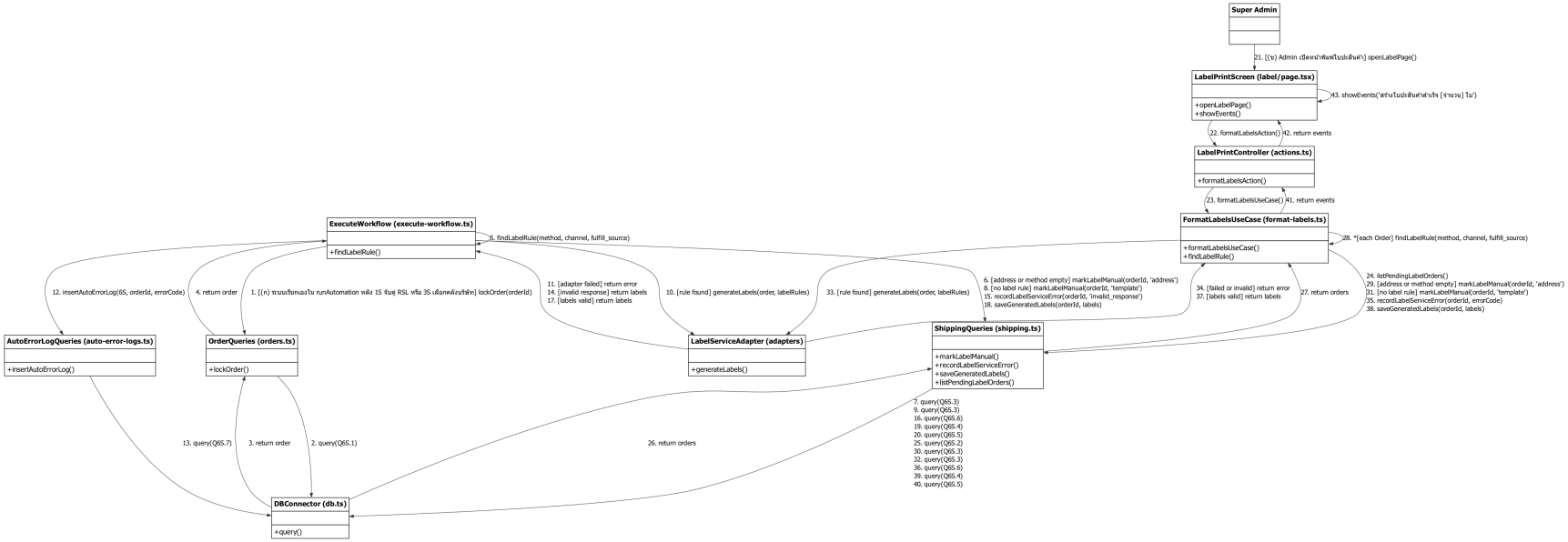


รูปที่ 2-78 Sequence Diagram ของ Use Case Description ที่ 6S (2/3)



รูปที่ 2-79 Sequence Diagram ของ Use Case Description ที่ 6S (3/3)

2.13.63 Collaboration Diagram ของ Use Case Description ที่ 6S



รูปที่ 2-80 Collaboration Diagram ของ Use Case Description ที่ 6S

2.13.64 User Interface ของ Use Case Description ที่ 6S

Use Case นี้ระบบทำเองโดยไม่มีหน้าจอของตัวเอง ผลการทำงานแสดงเป็นข้อความแจ้งเตือนในหน้าที่ Super Admin เปิดอยู่

2.13.65 Use Case Description ที่ 7S: ตัดสต๊อกสินค้า

Goal: ให้ยอดสต๊อกในระบบถูกตัดตามจำนวนที่จัดส่งจริง และจำนวนสินค้าคงเหลือใน Marketplace ตรงกับความเป็นจริง

Pre-Condition: Order อยู่ในสถานะ "แจ้งเลขติดตามแล้ว" (5S ส่งเลขติดตามสำเร็จ) รายการสินค้ายังไม่ถูกตัดสต๊อก และมี `fulfill_source` จาก 3S

Post-Condition: ยอด `in_house_qty` หรือ `rsl_qty` ถูกหักตามแหล่งที่ส่ง `order_items.stock_deducted`=TRUE และยอดคงเหลือรวมถูกซิงก์ไป Rakuten Ichiba (`rakuten_stock_sync_states`='synced') / Order เปลี่ยนเป็น "จัดส่งสำเร็จ" ก็ต่อเมื่อขนส่งยืนยันว่าส่งถึงแล้ว แจ้งเลขติดตามแล้ว และตัดสต๊อกครบทุกรายการแล้ว ครบทั้งสามข้อ / Order ที่ค้างตัดสต๊อกถูกหยิบมาทำซ้ำใน action ถัดไป (ยกเว้นสต๊อกไม่พอซึ่งไปที่ "รอดำเนินการด้วยตนเอง")

Actor: ระบบทำงานอัตโนมัติ (ใน `executeWorkflow` ต่อจาก 5S และรอบ retry ของทุก action), Marketplace

ตารางที่ 2-21 Use Case Description ที่ 7S: ตัดสต๊อกสินค้า

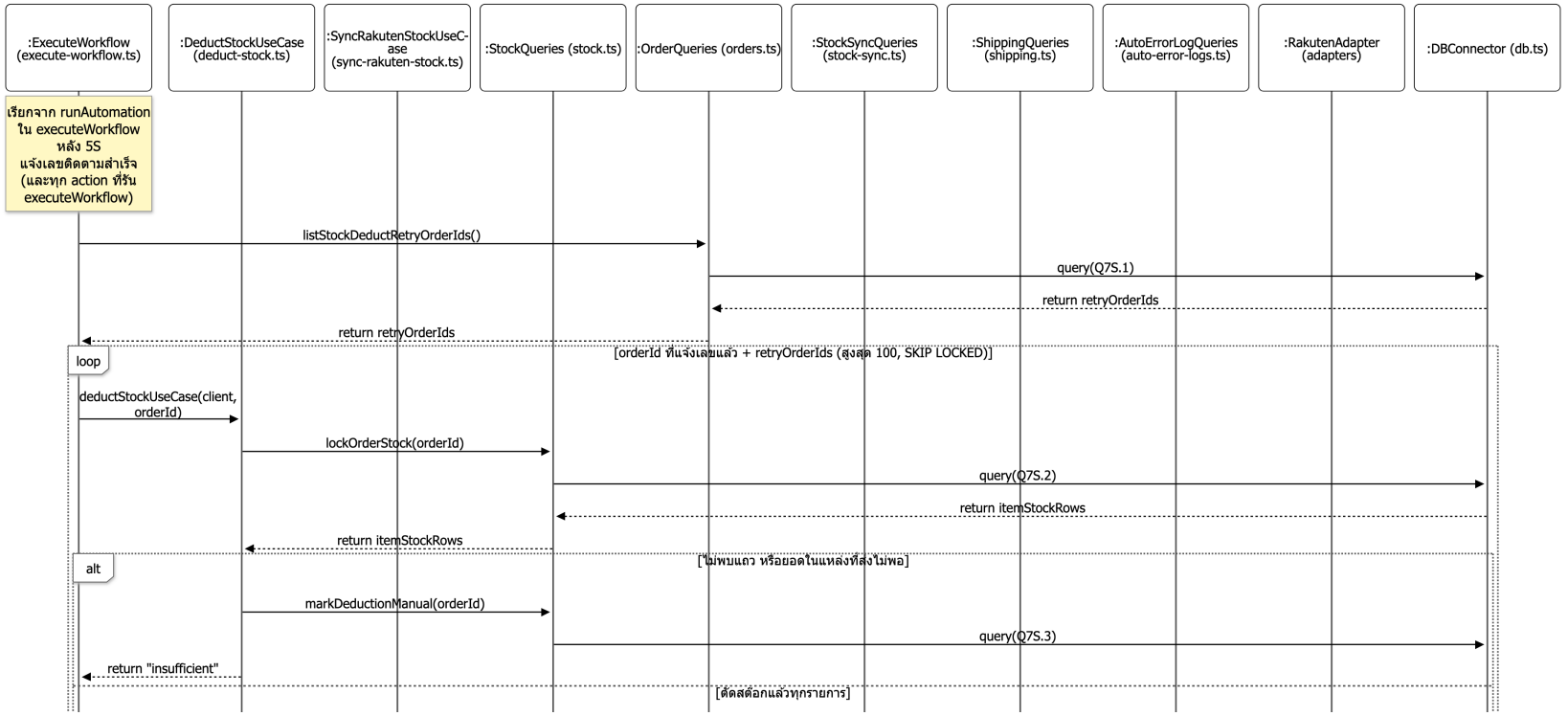
ลำดับ	User Action	System Response
1	Super Admin กด "แจ้งเลขติดตาม" ในหน้าจัดส่ง (5S)	**เริ่มทำงาน** — `notifyTrackingAction` เรียก `notifyTrackingUseCase` ภายใน `executeWorkflow` เมื่อ Rakuten Ichiba รับเลขติดตามสำเร็จ Order เป็น "แจ้งเลขติดตามแล้ว" แล้ว `runAutomation` เรียก `deductStockUseCase` ต่อใน transaction เดียวกัน **ลองตัดสต๊อกซ้ำ (retry)** — ทุก action ที่รัน `executeWorkflow` (เช่น ยืนยัน Order, ปรับสต๊อก, รับสินค้า, บันทึก SKU, ดึงสถานะขนส่ง) ค้นหา Order ที่ค้างอยู่ "แจ้งเลขติดตามแล้ว" แต่ยังตัดสต๊อกไม่ครบ หรือขนส่งยืนยันส่งถึงแล้วแต่ยังไม่ปิด — Q7S.1: `SELECT o.order_id FROM orders o WHERE o.order_status = 'แจ้งเลขติดตามแล้ว' AND ( EXISTS ( SELECT 1 FROM order_items item WHERE item.order_id = o.order_id AND item.stock_deducted = FALSE ) OR (o.delivery_status = 'delivered' AND`

ลำดับ	User Action	System Response
		o.tracking_notified = TRUE)) ORDER BY o.order_date, o.order_id LIMIT 100 FOR UPDATE OF o SKIP LOCKED (ครั้งละไม่เกิน 100 รายการ ข้ามแถวที่ถูกล็อก) แล้วทำขั้นตอนที่ 2–4 กับแต่ละ Order
2		**1. ล็อกรายการสินค้าและสต๊อก** Q7S.2: SELECT item.order_item_id, item.sku_id, item.channel_sku, item.fulfill_source, item.qty, item.stock_deducted, stock.stock_id, stock.in_house_qty, stock.rsl_qty FROM order_items item INNER JOIN stock ON stock.sku_id = item.sku_id WHERE item.order_id = $1 ORDER BY stock.stock_id, item.order_item_id FOR UPDATE OF item, stock (ล็อกทั้ง order_items และ stock ทีละแถว เรียงตาม stock_id เพื่อกันการตัดซ้ำและ deadlock) — ไม่พบรายการหรือแถวสต๊อก: แจ้งว่า "ยอดสต๊อกไม่เพียงพอสำหรับการตัดสต๊อก กรุณาตรวจสอบ" และตั้ง Order เป็น "รอดำเนินการด้วยตนเอง" (manual_reason='stock-deduct') Q7S.3: UPDATE orders SET order_status = **'รอดำเนินการด้วยตนเอง'**, manual_reason = 'stock-deduct', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = **'แจ้งเลขติดตามแล้ว'** — ทุกรายการ stock_deducted=TRUE แล้ว: ข้าม ไม่ตัดซ้ำ — รวมจำนวนตามแถวสต๊อกและแหล่งที่ส่ง หากยอดในแหล่งนั้นน้อยกว่าจำนวนที่ต้องตัด (หรือ rsl_qty เป็น NULL) ใช้ Q7S.3 เช่นกัน และไม่ตัดสต๊อกรายการใดเลย
3		**2. ตัดสต๊อกตามแหล่งที่ส่ง** Q7S.4: UPDATE stock SET in_house_qty = CASE WHEN $2 = 'in_house' THEN in_house_qty - $3 ELSE in_house_qty END, rsl_qty = CASE WHEN $2 = 'rsl' THEN rsl_qty - $3 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 AND CASE WHEN $2 = 'in_house' THEN in_house_qty >= $3 ELSE rsl_qty IS NOT NULL AND rsl_qty >= $3 END RETURNING stock_id, sku_id, in_house_qty, rsl_qty (หัก in_house_qty หรือ rsl_qty ตาม fulfill_source และมีเงื่อนไขกันยอดติดลบอีกชั้น หากไม่สำเร็จจะยกเลิกทั้ง transaction) ตั้ง stock_deducted=TRUE ให้รายการสินค้า Q7S.5: UPDATE order_items SET stock_deducted = TRUE, deducted_at = CURRENT_TIMESTAMP WHERE order_item_id = ANY($1::text[]) AND stock_deducted = FALSE ตั้งสถานะซิงก์ Rakuten ของแถวสต๊อกเป็น 'pending' Q7S.6: INSERT INTO rakuten_stock_sync_states (stock_id, sync_status, last_error_code) VALUES ($1, 'pending', NULL) ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL — แสดงข้อความ "ตัดสต๊อกและอัปเดตจำนวนสินค้าสำเร็จ"

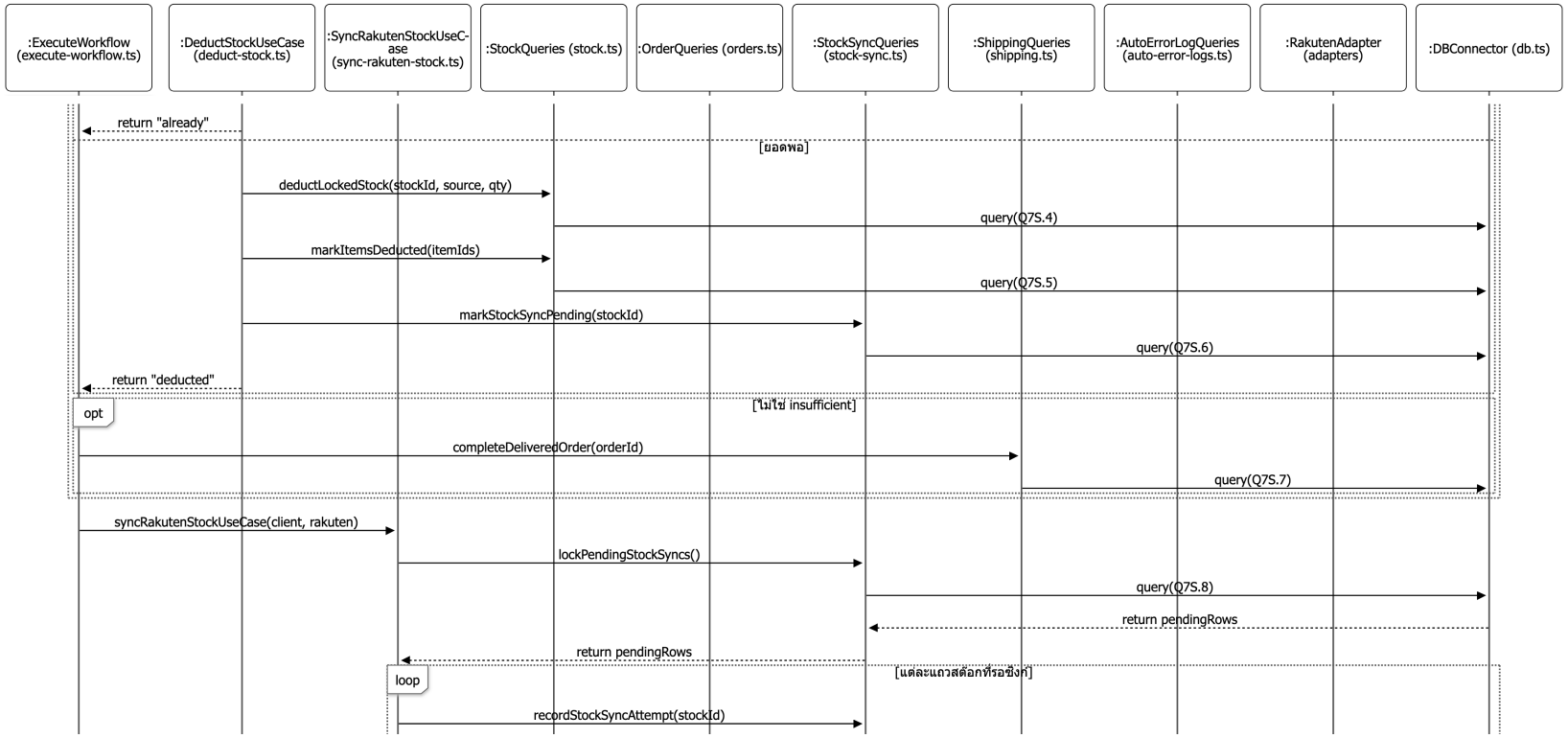
ลำดับ	User Action	System Response
4		**3. ปิด Order หากส่งถึงแล้ว** (กติกาเดียวกับ 8A) Q7S.7: `UPDATE orders SET order_status = 'จัดส่งสำเร็จ', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'แจ้งเลขติดตามแล้ว' AND tracking_notified = TRUE AND delivery_status = 'delivered' AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE)` — เปลี่ยนเป็น "จัดส่งสำเร็จ" เมื่อ `delivery_status`='delivered' และ `tracking_notified`=TRUE และตัดสต๊อกครบทุกรายการ หากขนส่งยังไม่ยืนยัน Order คงเป็น "แจ้งเลขติดตามแล้ว" จนกว่าการ poll ขนส่งใน 8A จะยืนยัน (`deductStockUseCase` เองไม่ตั้ง "จัดส่งสำเร็จ")
5		**4. ซิงก์ยอดคงเหลือไป Rakuten Ichiba** — ท้าย `runAutomation` ทุกครั้ง `syncRakutenStockUseCase` ล็อกแถวที่รอซิงก์พร้อมคำนวณ Available Qty = `in_house_qty` + `rsl_qty` Q7S.8: `SELECT sync.stock_id, sku.channel_sku, (stock.in_house_qty + COALESCE(stock.rsl_qty, 0)) AS available_qty FROM rakuten_stock_sync_states sync INNER JOIN stock ON stock.stock_id = sync.stock_id INNER JOIN skus sku ON sku.sku_id = stock.sku_id WHERE sync.sync_status = 'pending' AND ($1::text[] IS NULL OR sync.stock_id = ANY($1::text[])) ORDER BY sync.stock_id FOR UPDATE OF sync, stock` บันทึกเวลาที่พยายาม Q7S.9: `UPDATE rakuten_stock_sync_states SET last_attempt_at = CURRENT_TIMESTAMP WHERE stock_id = $1 AND sync_status = 'pending'` แล้วเรียก `RakutenAdapter.syncStock(channel_sku, qty)` — สำเร็จ: Q7S.10: `UPDATE rakuten_stock_sync_states sync SET sync_status = 'synced', last_synced_qty = $2, last_synced_at = CURRENT_TIMESTAMP, last_error_code = NULL FROM stock WHERE sync.stock_id = $1 AND stock.stock_id = sync.stock_id AND stock.in_house_qty + COALESCE(stock.rsl_qty, 0) = $2` ตั้งเป็น 'synced' เฉพาะเมื่อยอดในระบบยังเท่ากับที่ส่งไป หากยอดเปลี่ยนระหว่างนั้นจะคง 'pending' ไว้ซิงก์รอบถัดไป
ทางเลือก #1	(ระบบตรวจพบเอง) อัปเดตจำนวนคงเหลือใน Rakuten Ichiba ไม่สำเร็จ (fault `mpStockSync`)	สต๊อกในระบบถูกตัดไว้แล้ว คงสถานะซิงก์เป็น 'pending' พร้อม error code Q7S.11: `UPDATE rakuten_stock_sync_states SET sync_status = 'pending', last_error_code = $2 WHERE stock_id = $1` และบันทึกข้อผิดพลาด Q7S.12: `INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, $2, $3, $4, $5)` (`use_case_id`='7S', `context_key`=`stock_id`) ระบบลองซิงก์ใหม่อัตโนมัติทุกครั้งที่มี action เรียก `executeWorkflow`

ลำดับ	User Action	System Response
ทางเลือก #2	(ระบบตรวจพบเอง) ฐานข้อมูลผิดพลาดระหว่างตัดสต๊อก	ยกเลิกทั้ง transaction (รวมผลของ 5S ในฐานข้อมูล) แล้วบันทึกข้อผิดพลาดใน transaction ใหม่ด้วย **Q7S.12**
ทางเลือก #3	Order ถูกยกเลิกหรือคืนสินค้าหลังตัดสต๊อกแล้ว	การคืนยอดสต๊อกทำใน UC 7A (ยกเลิก Order) ไม่ใช่ใน 7S

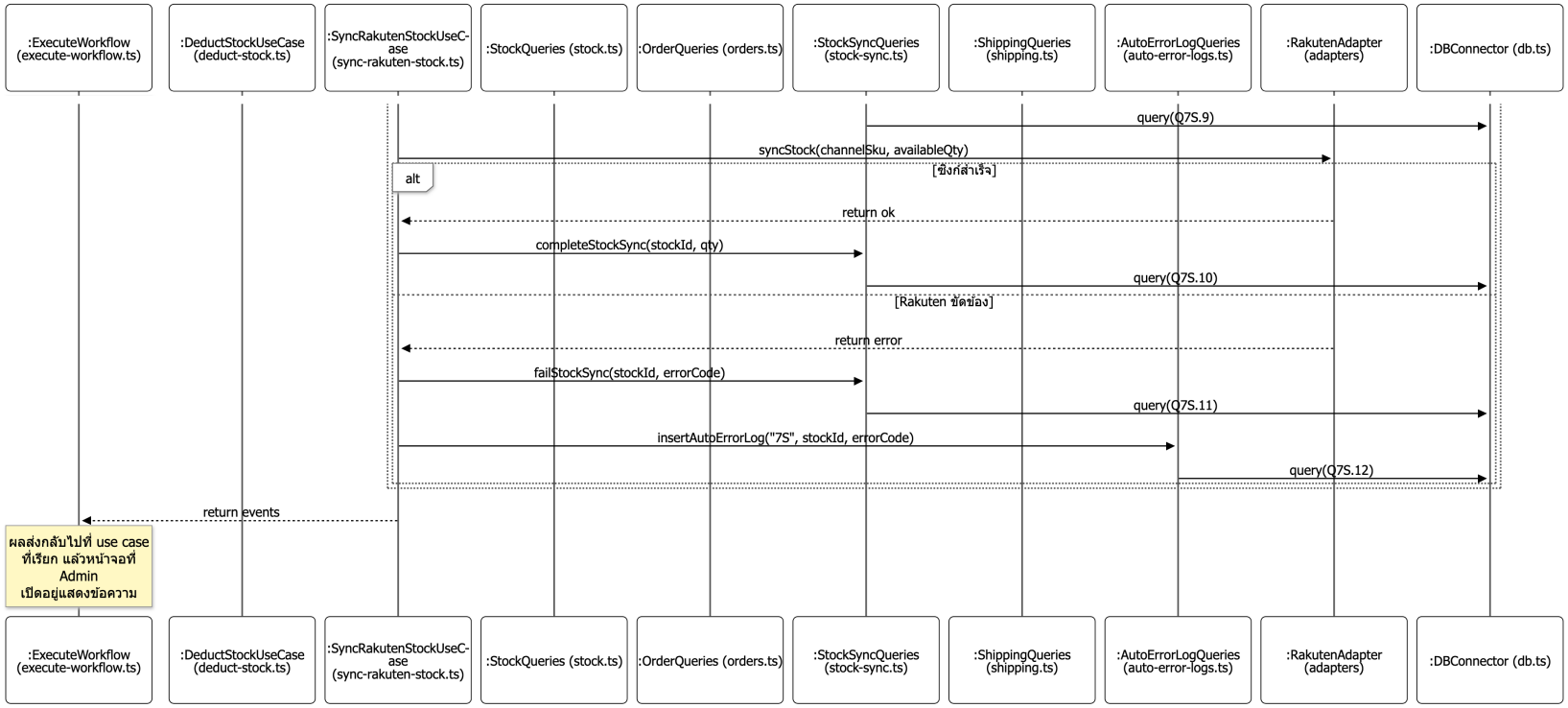
2.13.66 Sequence Diagram ของ Use Case Description ที่ 7S



รูปที่ 2-81 Sequence Diagram ของ Use Case Description ที่ 7S (1/3)

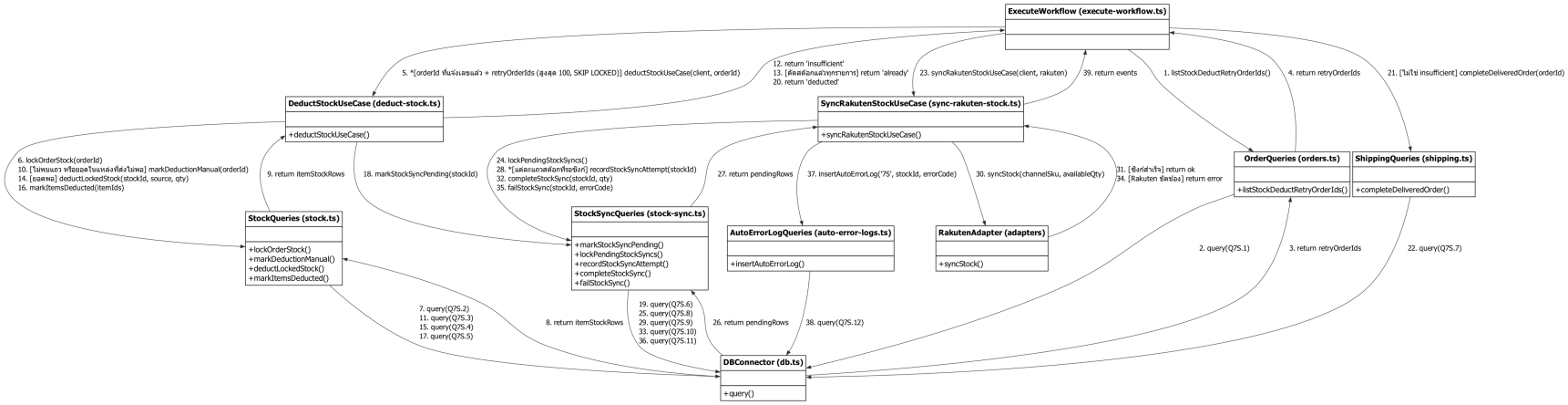


รูปที่ 2-82 Sequence Diagram ของ Use Case Description ที่ 7S (2/3)



รูปที่ 2-83 Sequence Diagram ของ Use Case Description ที่ 7S (3/3)

2.13.67 Collaboration Diagram ของ Use Case Description ที่ 7S



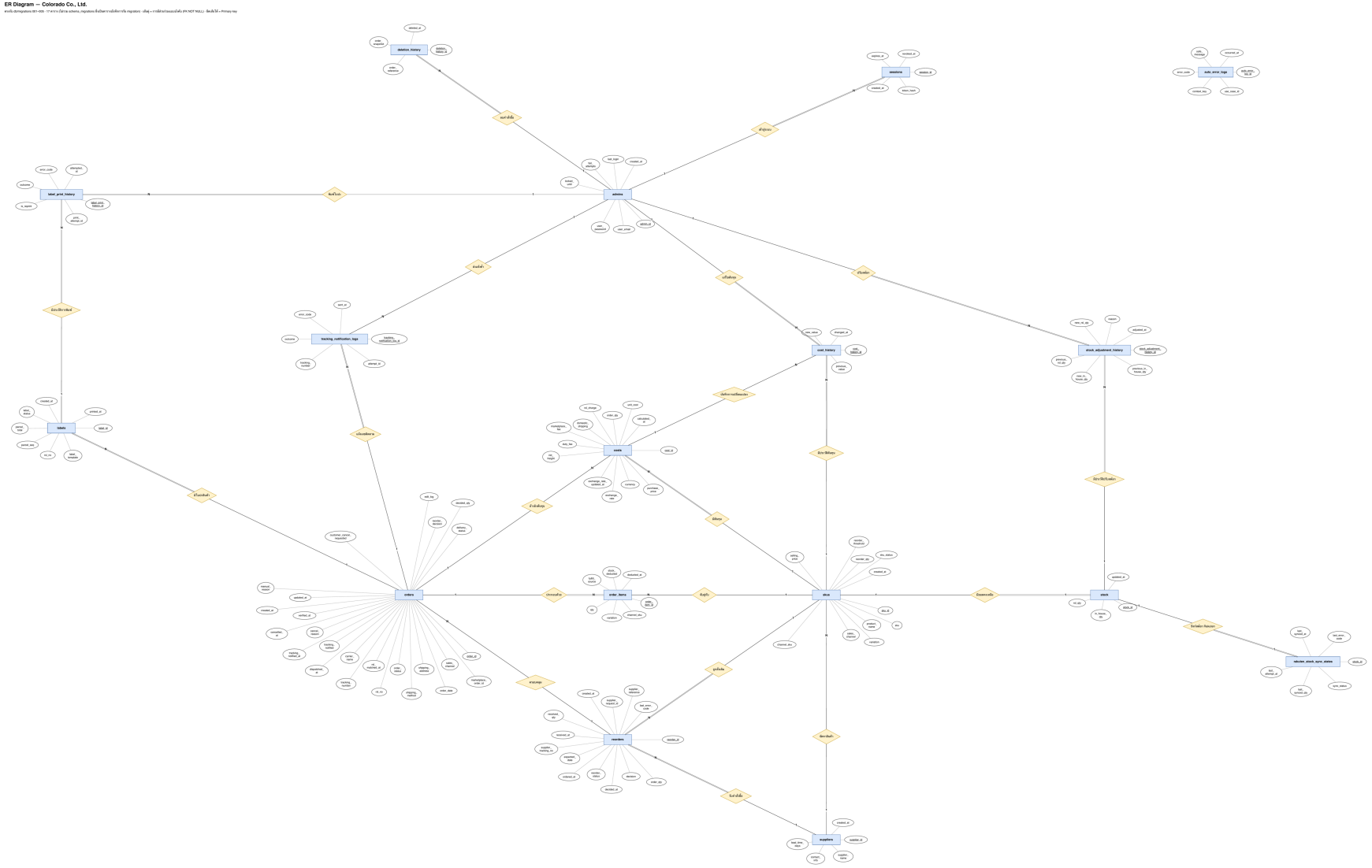
รูปที่ 2-84 Collaboration Diagram ของ Use Case Description ที่ 7S

2.13.68 User Interface ของ Use Case Description ที่ 7S

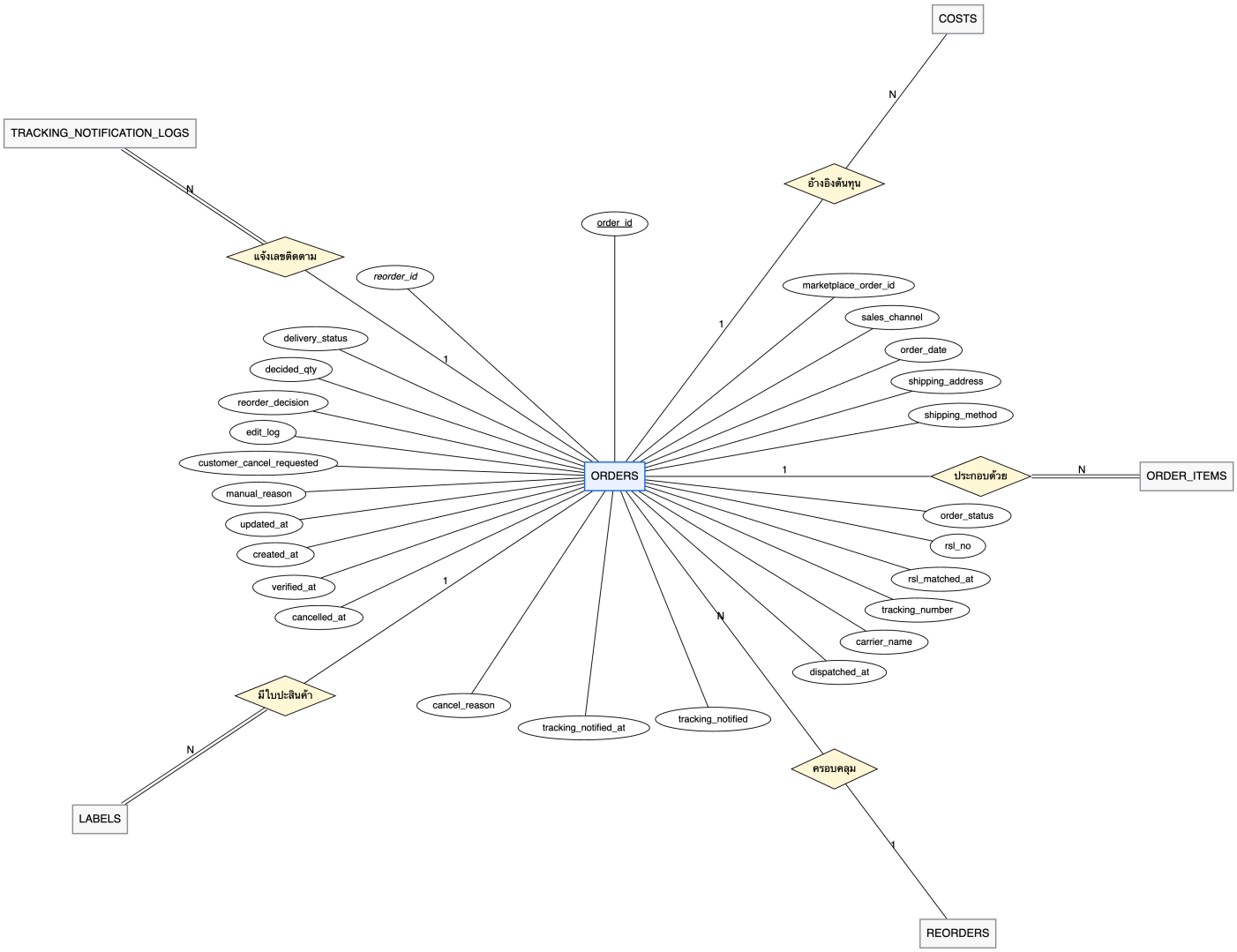
Use Case นี้ระบบทำเองโดยไม่มีหน้าจอของตัวเอง ผลการทำงานแสดงเป็นข้อความแจ้งเตือนในหน้าที่ Super Admin เปิดอยู่

2.14 ER Diagram

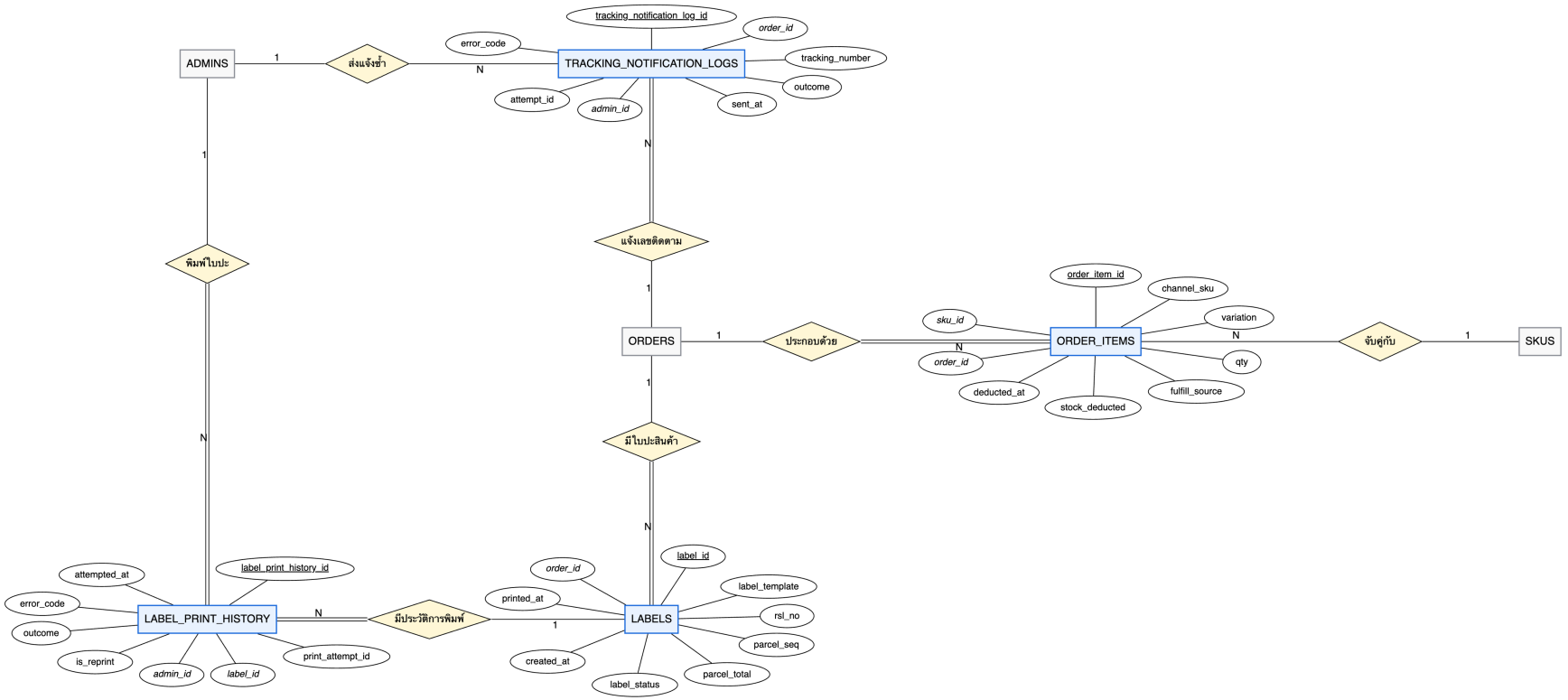
ER Diagram เขียนแบบ Chen มี 17 entity ตรงกับตารางในฐานข้อมูลจริงของระบบ (db/migrations 001-008 โดย 008 เพิ่มเฉพาะ sequence reorder_no_seq สำหรับเลขคำสั่งซื้อ ไม่ได้เพิ่มตาราง) และ CRUD Table (2.12) ไม่นับตาราง schema_migrations ซึ่งตัวรัน migration ใช้จดว่ารันไฟล์ใดแล้ว ไม่ใช่ข้อมูลทางธุรกิจ มี relationship 22 เส้น ทุกเส้นเป็นแบบ 1:1 หรือ 1:N จึงไม่มีความสัมพันธ์ M:N เส้นคู่หมายถึง total participation (foreign key เป็น NOT NULL) เส้นเดี่ยวหมายถึง partial participation attribute ที่ขีดเส้นใต้คือ primary key และตัวเอียงคือ foreign key ความสัมพันธ์ reorders ครอบคลุม orders เป็น 1:N เพราะคำสั่งซื้อเติมสต๊อกหนึ่งรายการครอบคลุมได้หลาย Order ของ SKU เดียวกัน แต่ Order หนึ่งรอคำสั่งซื้อได้ครั้งละหนึ่งรายการ จึงเก็บเป็น orders.reorder_id ไม่ต้องมีตารางเชื่อม



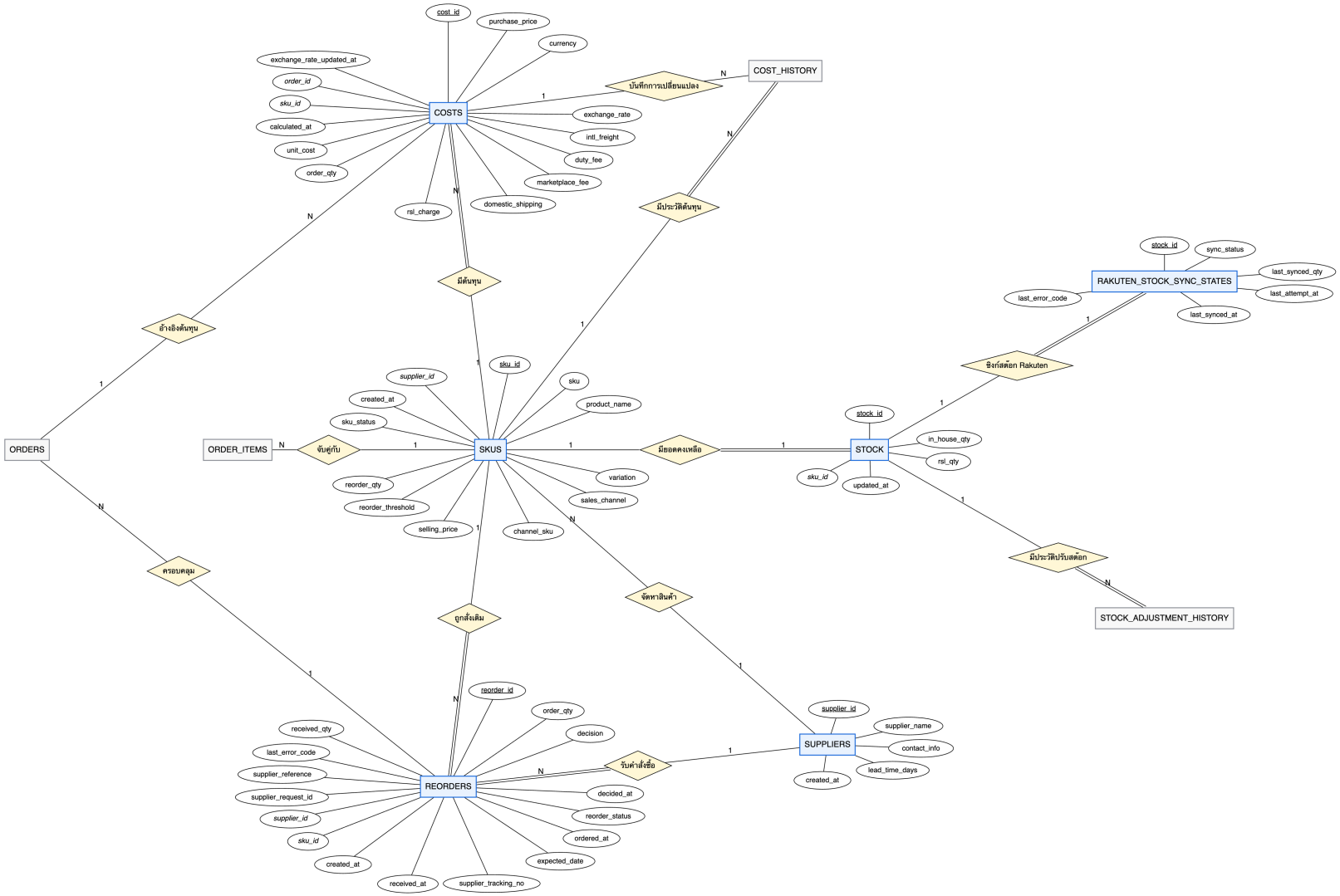
รูปที่ 2-85 ER Diagram ภาพรวมของระบบ (17 ตาราง)



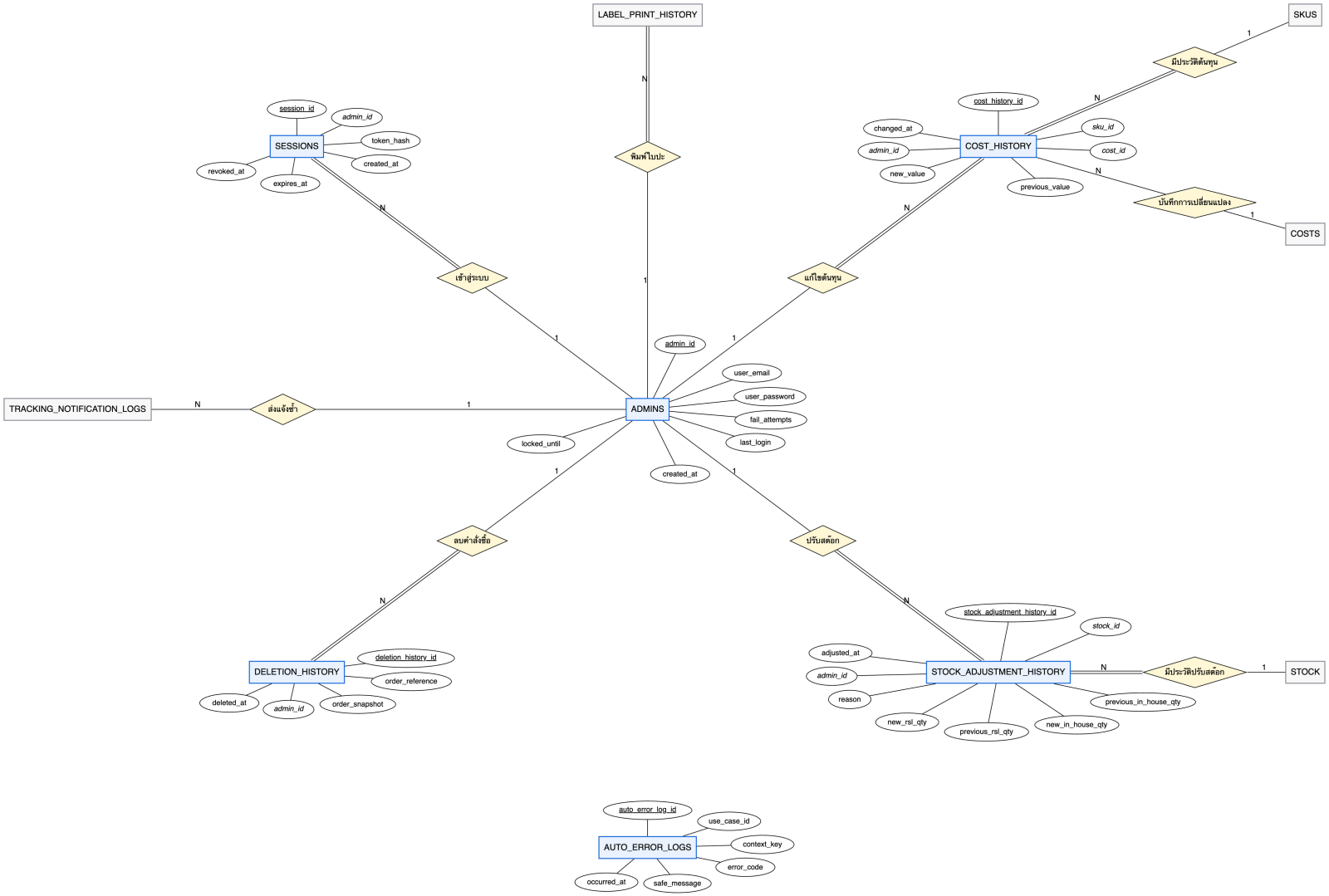
รูปที่ 2-86 ER Diagram ส่วนคำสั่งซื้อ (ORDERS) (1/4)



รูปที่ 2-87 ER Diagram ส่วนรายการสินค้า ใบปะ และการแจ้งเลขติดตาม (2/4)



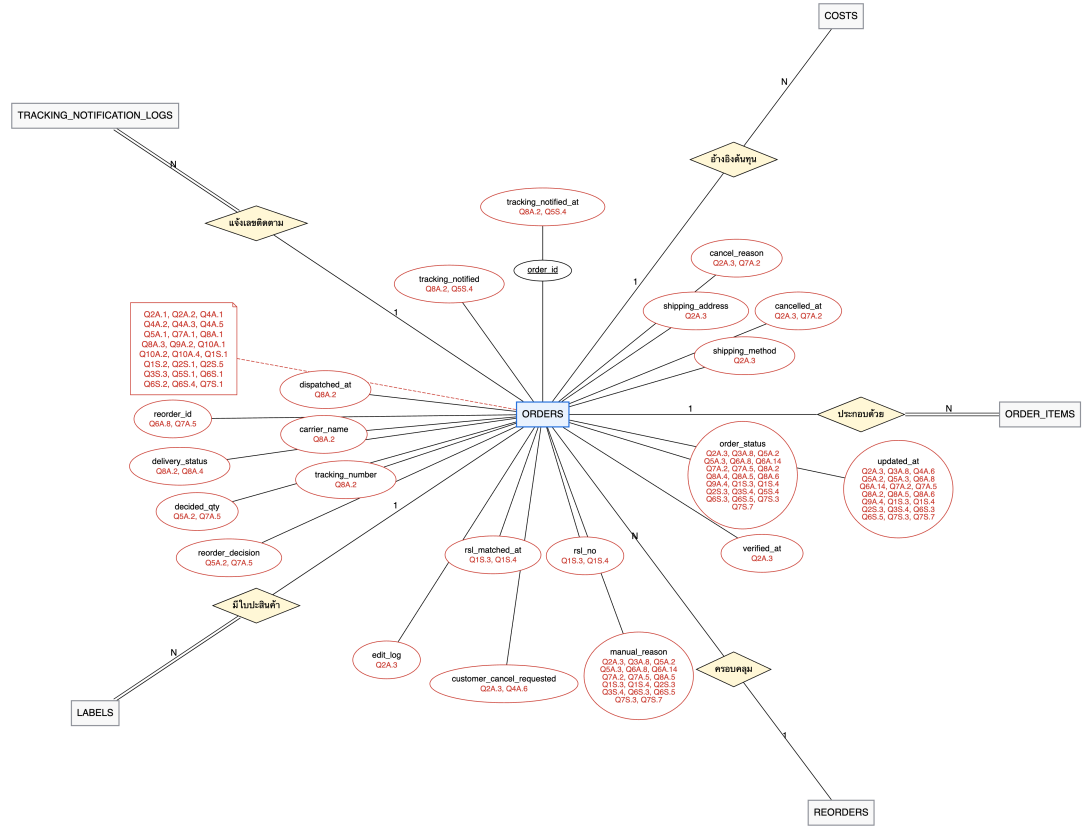
รูปที่ 2-88 ER Diagram ส่วนสินค้า สต๊อก ต้นทุน และการสั่งซื้อเติม (3/4)



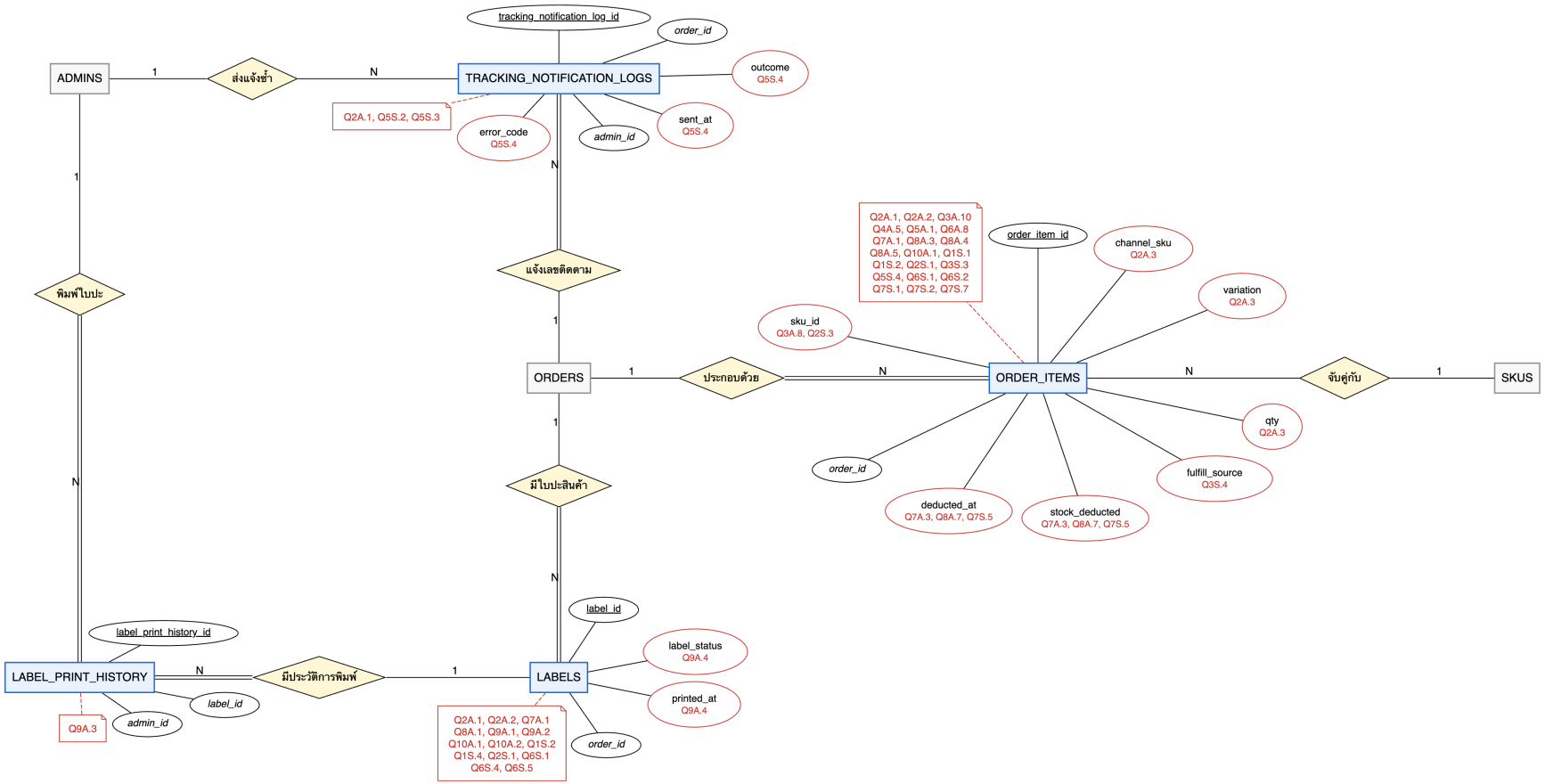
รูปที่ 2-89 ER Diagram ส่วนผู้ดูแลระบบและประวัติการทำงาน (4/4)

2.15 ER Diagram พร้อม Query

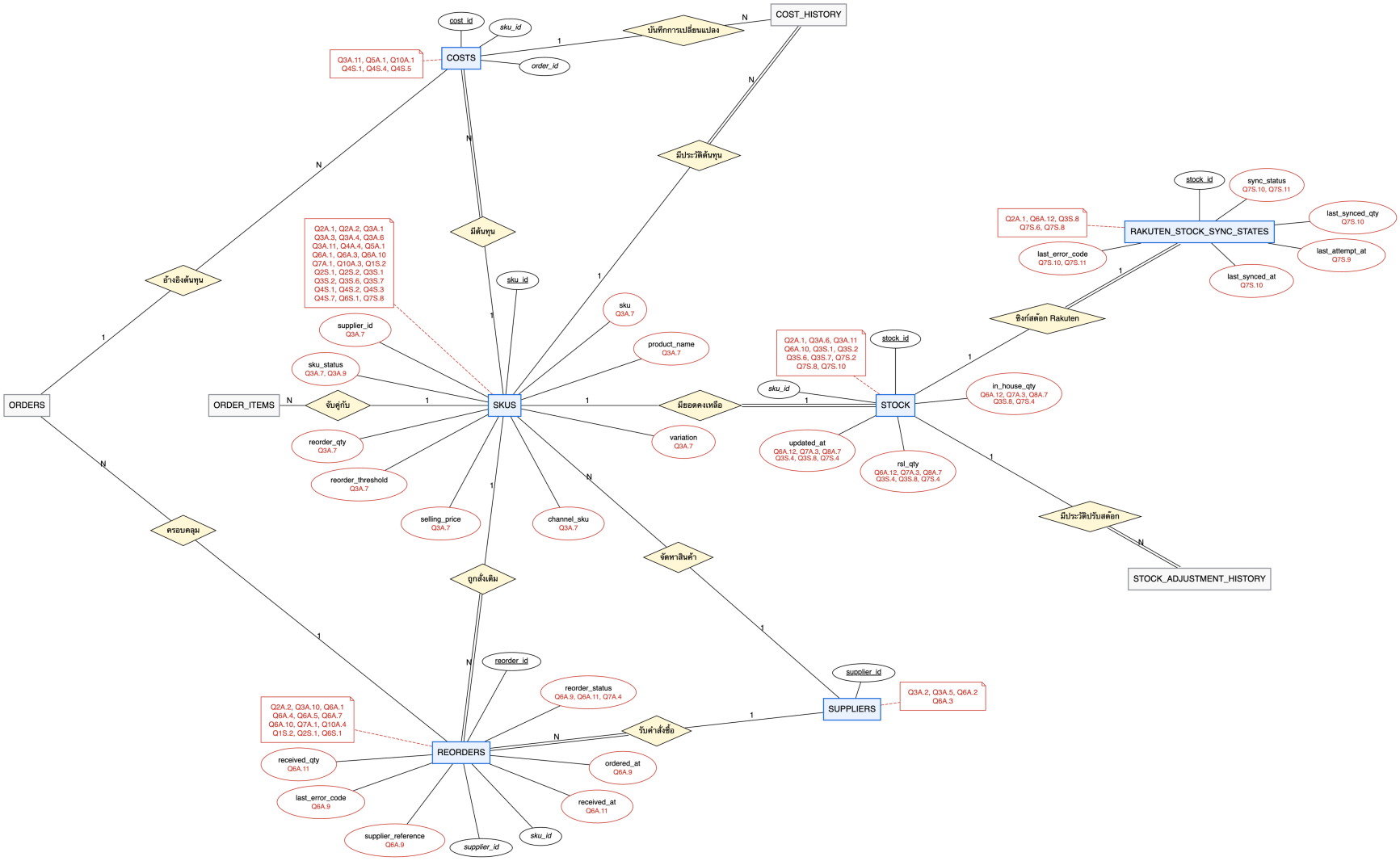
แบ่ง 4 ส่วนเหมือน 2.14 แสดงเฉพาะ entity พร้อม primary key และ foreign key ป้ายสีแดงบอกเลข query ที่กระทำกับ entity นั้น ส่วน query ประเภท UPDATE แสดง attribute ที่ถูกแก้ไขพร้อมเลข query กำกับ



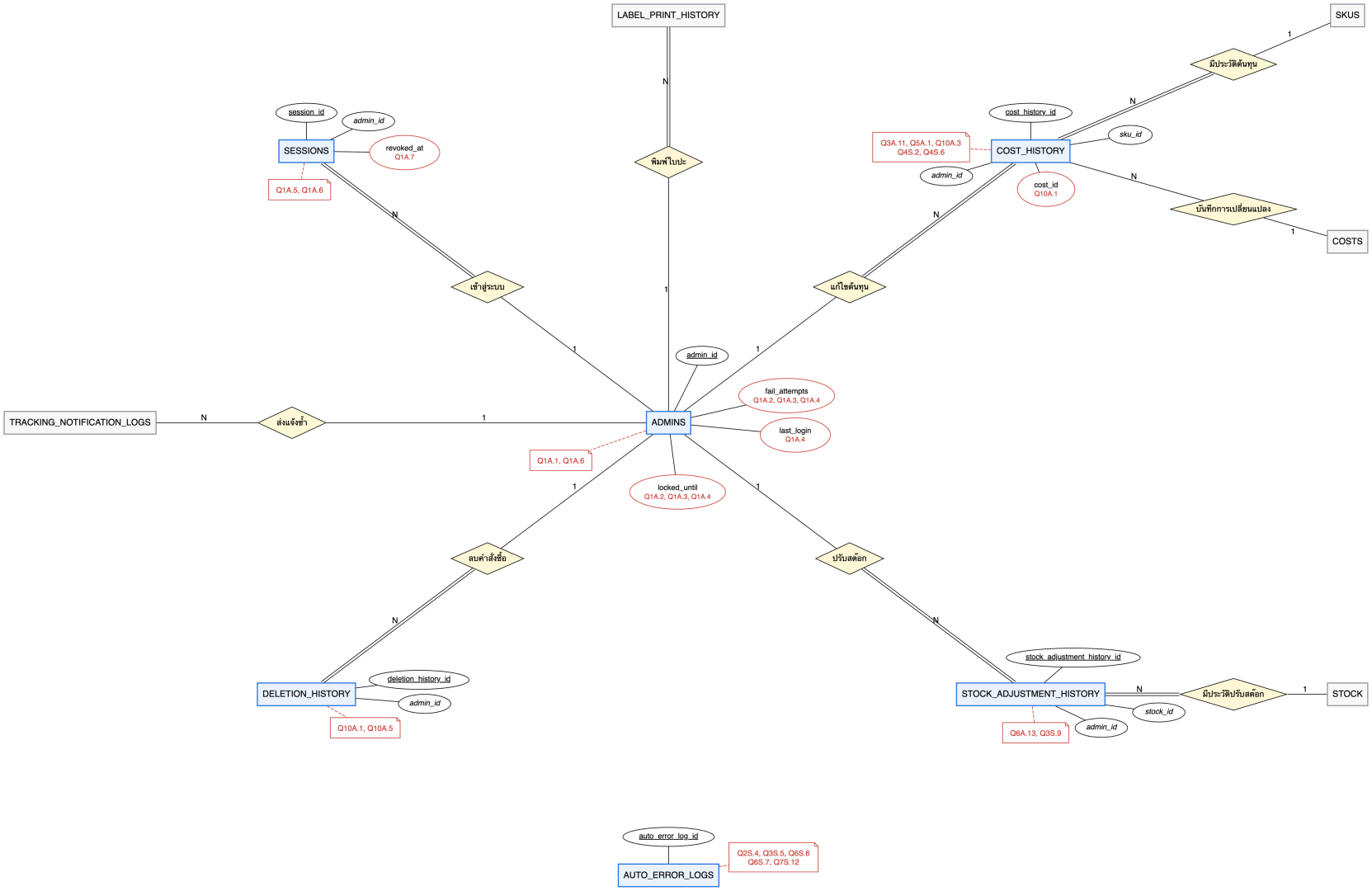
รูปที่ 2-90 ER Diagram พร้อม Query ส่วนคำสั่งซื้อ (ORDERS) (1/4)



รูปที่ 2-91 ER Diagram พร้อม Query ส่วนรายการสินค้า ใบปะ และการแจ้งเลขติดตาม (2/4)



รูปที่ 2-92 ER Diagram พร้อม Query ส่วนสินค้า สต๊อก ต้นทุน และการสั่งซื้อเติม (3/4)



รูปที่ 2-93 ER Diagram พร้อม Query ส่วนผู้ดูแลระบบและประวัติการทำงาน (4/4)

2.16 Query Table

ตารางแสดง query ทั้งหมดในระบบ แต่ละแถวมี Use Case ID, Use Case Name, Query ID และ SQL Command เรียงตาม Use Case เลข Q<UC>.n เรียงตามลำดับขั้นตอนใน Use Case Description (2.13) ส่วนคอลัมน์ค่าคงที่ในโค้ดบอกไฟล์และชื่อค่าคงที่ SQL ใน src/lib/queries ของเว็บที่รันคำสั่งนั้นจริง เช่น Q6A.1 คือ reorders.ts ค่าคงที่ Q5A_1 ชื่อค่าคงที่ในโค้ดไม่จำเป็นต้องตรงกับเลขในเล่ม เพราะ query เดียวกันถูกใช้ซ้ำในหลาย Use Case และ SQL ทุกข้อคัดลอกจากโค้ดโดยตรง

ตารางที่ 2-22 Query Table

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
1A	เข้าสู่ระบบ	Q1A.1	admin.ts Q1A_1	`SELECT admin_id, user_email, user_password, fail_attempts, locked_until, last_login FROM admins WHERE user_email = $1`
1A	เข้าสู่ระบบ	Q1A.2	admin.ts Q1A_2_UNLOCK	`UPDATE admins SET fail_attempts = 0, locked_until = NULL WHERE admin_id = $1`
1A	เข้าสู่ระบบ	Q1A.3	admin.ts Q1A_2	`UPDATE admins SET fail_attempts = fail_attempts + 1, locked_until = CASE WHEN fail_attempts + 1 >= $3 THEN $2::timestamptz + make_interval(mins => $4::integer) ELSE locked_until END WHERE admin_id = $1 RETURNING fail_attempts, locked_until`
1A	เข้าสู่ระบบ	Q1A.4	admin.ts Q1A_3	`UPDATE admins SET fail_attempts = 0, locked_until = NULL, last_login = $2 WHERE admin_id = $1`
1A	เข้าสู่ระบบ	Q1A.5	sessions.ts Q1A_4	`INSERT INTO sessions ( session_id, admin_id, token_hash, created_at, expires_at, revoked_at ) VALUES ($1, $2, $3, $4, $5, NULL)`
1A	เข้าสู่ระบบ	Q1A.6	sessions.ts Q1A_5	`SELECT s.session_id, s.admin_id, a.user_email, s.created_at, s.expires_at FROM sessions AS s INNER JOIN admins AS a ON a.admin_id = s.admin_id WHERE s.token_hash = $1 AND s.revoked_at IS NULL AND s.expires_at > $2`
1A	เข้าสู่ระบบ	Q1A.7	sessions.ts Q1A_6	`UPDATE sessions SET revoked_at = $2 WHERE token_hash = $1 AND revoked_at IS NULL`
2A	ตรวจสอบคำสั่งซื้อ	Q2A.1	orders.ts Q2A_1	`SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_date, o.shipping_address, o.shipping_method, o.order_status, o.rsl_no, o.tracking_number, o.carrier_name, o.dispatched_at, o.delivery_status, o.tracking_notified, o.tracking_notified_at, o.cancel_reason, o.cancelled_at, o.verified_at, o.manual_reason, o.customer_cancel_requested, o.edit_log, o.reorder_decision, o.decided_qty, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.deducted_at, item.sku_id, sku.sku, sku.product_name, sku.selling_price, o.reorder_id, label.label_template, label.parcel_total, label.label_printed_at, sync.sync_status, notification.notify_log FROM orders o LEFT JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN stock stock_row ON stock_row.sku_id = sku.sku_id LEFT JOIN rakuten_stock_sync_states`

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
				sync ON sync.stock_id = stock_row.stock_id LEFT JOIN LATERAL (SELECT MAX(l.label_template) AS label_template, MAX(l.parcel_total) AS parcel_total, MAX(l.printed_at) AS label_printed_at FROM labels l WHERE l.order_id = o.order_id) label ON TRUE LEFT JOIN LATERAL (SELECT jsonb_agg(jsonb_build_object('by', COALESCE(t.admin_id, 'system'), 'at', t.sent_at) ORDER BY t.sent_at) AS notify_log FROM tracking_notification_logs t WHERE t.order_id = o.order_id) notification ON TRUE ORDER BY o.order_date DESC, o.order_id DESC
2A	ตรวจสอบคำสั่งซื้อ	Q2A.2	orders.ts Q2A_1_FOR_UPDATE	SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o
2A	ตรวจสอบคำสั่งซื้อ	Q2A.3	orders.ts Q2A_2	WITH updated_item AS (UPDATE order_items SET channel_sku = $2, variation = $3, qty = $4 WHERE order_id = $1) UPDATE orders SET shipping_address = $5, shipping_method = $6, order_status = $7, manual_reason = $8, customer_cancel_requested = $9, verified_at = $10, edit_log = CASE WHEN $11::boolean THEN edit_log || jsonb_build_array(jsonb_build_object('field', 'shipping_method', 'by', $12::text, 'at', $10::timestamptz)) ELSE edit_log END, cancel_reason = $13, cancelled_at = $14, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1
3A	ตั้งกฎ SKU	Q3A.1	skus.ts Q3A_1	SELECT sku, product_name, variation, sales_channel, channel_sku, supplier_id, reorder_threshold, reorder_qty, selling_price, sku_status FROM skus ORDER BY sku
3A	ตั้งกฎ SKU	Q3A.2	skus.ts Q6A_2	SELECT supplier_id, supplier_name, contact_info, lead_time_days FROM suppliers ORDER BY supplier_name, supplier_id
3A	ตั้งกฎ SKU	Q3A.3	skus.ts Q3A_1_FOR_UPDATE	SELECT sku_id, sku, channel_sku FROM skus WHERE sku = $1 FOR UPDATE
3A	ตั้งกฎ SKU	Q3A.4	skus.ts Q3A_1_FIND_CONFLICT	SELECT sku_id, sku, channel_sku FROM skus WHERE (sku = $1 OR channel_sku = $2) AND ($3::text IS NULL OR sku_id <> $3) LIMIT 1
3A	ตั้งกฎ SKU	Q3A.5	skus.ts Q6A_2_SUPPLIER_EXISTS	SELECT EXISTS (SELECT 1 FROM suppliers WHERE supplier_id = $1) AS found
3A	ตั้งกฎ SKU	Q3A.6	skus.ts Q3A_1_INSERT	WITH inserted_sku AS (INSERT INTO skus (sku_id, sku, product_name, variation, sales_channel, channel_sku, selling_price, reorder_threshold, reorder_qty, sku_status, supplier_id) VALUES ($1, $2, $3, $4, 'Rakuten Ichiba', $5, $6, $7, $8, $9, $10) RETURNING sku_id) INSERT INTO stock (stock_id, in_house_qty, rsl_qty, updated_at, sku_id) SELECT $11, 0, 0, CURRENT_TIMESTAMP, sku_id FROM inserted_sku

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
3A	ตั้งกฎ SKU	Q3A.7	skus.ts Q3A_1_UPDATE	UPDATE skus SET sku = $2, product_name = $3, variation = $4, channel_sku = $5, selling_price = $6, reorder_threshold = $7, reorder_qty = $8, sku_status = $9, supplier_id = $10 WHERE sku_id = $1
3A	ตั้งกฎ SKU	Q3A.8	skus.ts Q3A_1_REQUEUE	WITH registered_items AS (UPDATE order_items SET sku_id = $2 WHERE sku_id IS NULL AND (channel_sku = $1 OR channel_sku = $4) RETURNING order_id) UPDATE orders o SET order_status = CASE WHEN o.manual_reason = 'sku-rule' THEN 'รอจับคู่กฎ SKU' WHEN o.manual_reason = 'sku-unregistered' THEN 'รอตรวจสอบค่าสั่งซื้อ' ELSE 'รอ Admin ตัดสินใจสั่งซื้อ' END, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP FROM order_items item WHERE item.order_id = o.order_id AND o.order_status = 'รอดำเนินการด้วยตนเอง' AND ((o.manual_reason = 'sku-rule' AND item.channel_sku = $1) OR (o.manual_reason = 'sku-unregistered' AND item.order_id IN (SELECT order_id FROM registered_items)) OR (o.manual_reason = 'price' AND item.sku_id = $2 AND $3::boolean)) RETURNING o.order_id
3A	ตั้งกฎ SKU	Q3A.9	skus.ts Q3A_1_SET_STATUS	UPDATE skus SET sku_status = $2 WHERE sku_id = $1
3A	ตั้งกฎ SKU	Q3A.10	skus.ts Q3A_1_HAS_RELATED	SELECT EXISTS (SELECT 1 FROM order_items WHERE sku_id = $1 UNION ALL SELECT 1 FROM reorders WHERE sku_id = $1) AS has_related
3A	ตั้งกฎ SKU	Q3A.11	skus.ts Q3A_1_DELETE	WITH removed_history AS (DELETE FROM cost_history WHERE sku_id = $1), removed_costs AS (DELETE FROM costs WHERE sku_id = $1), removed_stock AS (DELETE FROM stock WHERE sku_id = $1) DELETE FROM skus WHERE sku_id = $1
4A	ดึง Order เข้าระบบ	Q4A.1	dashboard.ts Q2A_1_DASHBOARD_TOTALS	SELECT order_status, COUNT(*)::integer AS count FROM orders GROUP BY order_status ORDER BY order_status
4A	ดึง Order เข้าระบบ	Q4A.2	dashboard.ts Q2A_1_DASHBOARD_VOLUME	SELECT TO_CHAR(order_date::date, 'YYYY-MM-DD') AS date, COUNT(*)::integer AS count FROM orders WHERE order_date >= (SELECT MAX(order_date)::date - INTERVAL '6 days' FROM orders) GROUP BY order_date::date ORDER BY order_date::date
4A	ดึง Order เข้าระบบ	Q4A.3	dashboard.ts Q2A_1_DASHBOARD_RECENT	SELECT order_id, order_date FROM orders ORDER BY order_date DESC, order_id DESC LIMIT 6
4A	ดึง Order เข้าระบบ	Q4A.4	orders.ts Q2S_1	SELECT sku_id, sku, product_name, variation FROM skus WHERE channel_sku = $1 AND sales_channel = $2 AND sku_status = 'active' ORDER BY sku_id
4A	ดึง Order เข้าระบบ	Q4A.5	orders.ts Q4A_1	WITH inserted_order AS (INSERT INTO orders (order_id, marketplace_order_id, sales_channel, order_date, shipping_address, shipping_method, order_status, manual_reason, created_at, updated_at) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP) ON CONFLICT (marketplace_order_id) DO NOTHING RETURNING order_id), inserted_item AS (INSERT INTO order_items (order_item_id, channel_sku, variation, qty, fulfill_source, order_id, sku_id) SELECT $9, $10, $11, $12, $13, order_id, $14 FROM inserted_order RETURNING order_id) SELECT order_id FROM inserted_order
4A	ดึง Order เข้าระบบ	Q4A.6	orders.ts Q4A_1_MARK_CANCELLATION	UPDATE orders SET customer_cancel_requested = TRUE, updated_at = CURRENT_TIMESTAMP WHERE marketplace_order_id = $1 AND order_status NOT IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') RETURNING order_id, order_status

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
5A	ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock	Q5A.1	reorders.ts Q5A_1_FOR_UPDATE	`SELECT o.order_id, o.order_status, o.manual_reason, item.sku_id, sku.sku, sku.selling_price, COALESCE(cost.unit_cost, NULLIF(history.new_value ->> 'unit_cost', '')::numeric) AS unit_cost FROM orders o INNER JOIN LATERAL (SELECT * FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN LATERAL (SELECT unit_cost FROM costs WHERE sku_id = item.sku_id ORDER BY calculated_at DESC NULLS LAST, cost_id DESC LIMIT 1) cost ON TRUE LEFT JOIN LATERAL (SELECT new_value FROM cost_history WHERE sku_id = item.sku_id AND new_value ->> 'unit_cost' IS NOT NULL ORDER BY changed_at DESC, cost_history_id DESC LIMIT 1) history ON TRUE WHERE o.order_id = $1 FOR UPDATE OF o`
5A	ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock	Q5A.2	reorders.ts Q5A_4	`UPDATE orders SET reorder_decision = $2, decided_qty = CASE WHEN $2 = 'คุ้มค่า' THEN $3::integer ELSE NULL END, order_status = $4, manual_reason = $5, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1`
5A	ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock	Q5A.3	reorders.ts Q5A_4_MANUAL_PRICE	`UPDATE orders SET order_status = 'รอดำเนินการด้วยตนเอง', manual_reason = 'price', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1`
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.1	reorders.ts Q5A_1	`SELECT reorders.reorder_id, sku.sku, reorders.supplier_id, reorders.order_qty, COALESCE(reorders.ordered_at, reorders.created_at) AS order_date, reorders.expected_date, reorders.reorder_status, reorders.received_at, reorders.received_qty FROM reorders INNER JOIN skus sku ON sku.sku_id = reorders.sku_id ORDER BY COALESCE(reorders.ordered_at, reorders.created_at) DESC, reorders.reorder_id DESC`
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.2	skus.ts Q6A_2	`SELECT supplier_id, supplier_name, contact_info, lead_time_days FROM suppliers ORDER BY supplier_name, supplier_id`
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.3	reorders.ts Q6A_2_FOR_UPDATE	`SELECT sku.sku_id, sku.sku, sku.product_name, sku.variation, supplier.supplier_id, supplier.supplier_name, supplier.lead_time_days FROM skus sku LEFT JOIN suppliers supplier ON supplier.supplier_id = sku.supplier_id WHERE sku.sku = $1 FOR UPDATE OF sku`
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.4	reorders.ts Q6A_1_PENDING	`SELECT reorder_id, order_qty, supplier_id, supplier_request_id, expected_date, created_at FROM reorders WHERE sku_id = $1 AND reorder_status = 'สั่งซื้อแล้ว' AND expected_date >= CURRENT_DATE - 1 ORDER BY ordered_at DESC NULLS LAST, created_at DESC LIMIT 1`
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.5	reorders.ts Q6A_1_RETRY	`SELECT reorder_id, order_qty, supplier_id, supplier_request_id, expected_date, created_at FROM reorders WHERE sku_id = $1 AND reorder_status = 'รอส่งคำสั่งซื้อ' ORDER BY created_at DESC LIMIT 1`
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.6	reorders.ts Q6A_1_NEXT_NUMBER	`SELECT nextval('reorder_no_seq') AS reorder_no`
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.7	reorders.ts Q6A_1_INSERT	`INSERT INTO reorders ( reorder_id, order_qty, reorder_status, expected_date, created_at, sku_id, supplier_id, supplier_request_id ) VALUES ($1, $2, 'รอส่งคำสั่งซื้อ', $3, CURRENT_TIMESTAMP, $4, $5, $6)`
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.8	reorders.ts Q6A_1_MARK_ORDERS	`UPDATE orders o SET order_status = $2, manual_reason = $3, reorder_id = COALESCE($4, o.reorder_id), updated_at = CURRENT_TIMESTAMP WHERE EXISTS ( SELECT 1 FROM order_items item`

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
				WHERE item.order_id = o.order_id AND item.sku_id = $1) AND (o.order_status IN ('รอสั่งซื้อจาก Supplier', 'รอส่งคำสั่งซื้อ') OR (o.order_status = 'รอดำเนินการด้วยตนเอง' AND o.manual_reason = 'supplier'))
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.9	reorders.ts Q6A_1_COMPLETE	UPDATE reorders SET reorder_status = $2, ordered_at = CASE WHEN $2 = 'สั่งซื้อแล้ว' THEN CURRENT_TIMESTAMP ELSE ordered_at END, supplier_reference = $3, last_error_code = $4 WHERE reorder_id = $1 AND supplier_request_id = $5 AND ($2 = 'สั่งซื้อแล้ว' OR reorder_status <> 'สั่งซื้อแล้ว') RETURNING reorder_id
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.10	reorders.ts Q6A_10_FOR_UPDATE	SELECT r.reorder_id, r.reorder_status, r.order_qty, sku.sku, stock.stock_id, stock.in_house_qty, stock.rsl_qty FROM reorders r INNER JOIN skus sku ON sku.sku_id = r.sku_id INNER JOIN stock ON stock.sku_id = r.sku_id WHERE r.reorder_id = $1 FOR UPDATE OF r, stock
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.11	reorders.ts Q6A_11_RECEIVE	UPDATE reorders SET reorder_status = 'ได้รับสินค้าแล้ว', received_at = CURRENT_TIMESTAMP, received_qty = $2 WHERE reorder_id = $1 AND reorder_status = 'สั่งซื้อแล้ว' RETURNING reorder_id
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.12	stock.ts Q3S_1_ADJUST	WITH adjusted AS (UPDATE stock SET in_house_qty = $2, rsl_qty = $3, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 RETURNING stock_id) INSERT INTO rakuten_stock_sync_states (stock_id, sync_status) SELECT stock_id, 'pending' FROM adjusted ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.13	stock.ts Q3S_1_INSERT_HISTORY	INSERT INTO stock_adjustment_history (stock_adjustment_history_id, stock_id, previous_in_house_qty, new_in_house_qty, previous_rsl_qty, new_rsl_qty, reason, admin_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
6A	สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier	Q6A.14	reorders.ts Q6A_12_RELEASE_ORDERS	UPDATE orders SET order_status = 'รอตรวจสอบสต๊อก', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE reorder_id = $1 AND order_status = 'สั่งซื้อแล้ว' RETURNING order_id
7A	ยกเลิก Order	Q7A.1	orders.ts Q2A_1_FOR_UPDATE	SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o
7A	ยกเลิก Order	Q7A.2	orders.ts Q7A_1	UPDATE orders SET order_status = $2, cancel_reason = $3, cancelled_at = $4, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1
7A	ยกเลิก Order	Q7A.3	stock.ts Q7A_2	WITH restored AS (UPDATE stock SET in_house_qty = in_house_qty + CASE WHEN $3 = 'in_house' THEN $4 ELSE 0 END, rsl_qty = CASE WHEN $3 = 'rsl' THEN COALESCE(rsl_qty, 0) + $4 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE sku_id = $2 AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = TRUE) RETURNING stock_id) UPDATE order_items SET stock_deducted = FALSE, deducted_at = NULL WHERE order_id = $1 AND EXISTS (SELECT 1 FROM restored) RETURNING qty

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
7A	ยกเลิก Order	Q7A.4	orders.ts Q7A_1_CANCEL_REORDER	`UPDATE reorders SET reorder_status = 'ยกเลิกแล้ว' WHERE reorder_id = $1 AND reorder_status IN ('รอส่งค่าสั่งซื้อ', 'สั่งซื้อแล้ว') RETURNING reorder_id`
7A	ยกเลิก Order	Q7A.5	orders.ts Q7A_1_RELEASE_ORDERS	`UPDATE orders SET order_status = 'รอ Admin ตัดสินใจสั่งซื้อ', manual_reason = NULL, reorder_id = NULL, reorder_decision = NULL, decided_qty = NULL, updated_at = CURRENT_TIMESTAMP WHERE reorder_id = $1 AND order_id <> $2 AND order_status IN ('รอส่งค่าสั่งซื้อ', 'สั่งซื้อแล้ว')`
8A	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	Q8A.1	shipping.ts Q8A_1	`SELECT o.order_id, o.order_status, o.tracking_number, o.carrier_name, o.delivery_status, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id) AND NOT EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NULL) AS labels_printed FROM orders o WHERE o.order_id = $1 FOR UPDATE OF o`
8A	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	Q8A.2	shipping.ts Q8A_1_SAVE	`UPDATE orders SET tracking_number = $2, carrier_name = $3, dispatched_at = CURRENT_TIMESTAMP, delivery_status = 'in_transit', tracking_notified = FALSE, tracking_notified_at = NULL, order_status = 'อยู่ระหว่างจัดส่ง', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status IN ('พิมพ์ใบปะสินค้าแล้ว', 'รอส่งมอบ')`
8A	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	Q8A.3	shipping.ts Q8A_1_POLL	`SELECT o.order_id, o.order_status, o.tracking_number, o.delivery_status, o.tracking_notified, item.stock_deducted, item.sku_id, item.fulfill_source, item.qty FROM orders o INNER JOIN LATERAL (SELECT * FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE WHERE o.tracking_number IS NOT NULL AND o.order_status IN ('อยู่ระหว่างจัดส่ง', 'รอแจ้งเลขติดตาม', 'แจ้งเลขติดตามแล้ว', 'จัดส่งสำเร็จ', 'รอดำเนินการพิเศษ') ORDER BY o.order_id`
8A	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	Q8A.4	shipping.ts Q8A_1_STATUS	`UPDATE orders SET delivery_status = $2, order_status = CASE WHEN $2 = 'failed' THEN 'จัดส่งไม่สำเร็จ' WHEN $2 = 'returned' THEN 'ตีกลับ/คืนสินค้า' WHEN $2 = 'delivered' AND tracking_notified = TRUE AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE) THEN 'จัดส่งสำเร็จ' ELSE order_status END, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND (delivery_status IS DISTINCT FROM 'delivered' OR $2 <> 'in_transit')`
8A	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	Q8A.5	shipping.ts Q8A_1_DELIVERED	`UPDATE orders SET order_status = 'จัดส่งสำเร็จ', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'แจ้งเลขติดตามแล้ว' AND tracking_notified = TRUE AND delivery_status = 'delivered' AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE)`
8A	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	Q8A.6	shipping.ts Q8A_1_WAIT	`UPDATE orders SET order_status = 'รอส่งมอบ', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'พิมพ์ใบปะสินค้าแล้ว'`
8A	จัดส่งสินค้าให้ลูกค้าผ่าน Delivery	Q8A.7	stock.ts Q7A_2	`WITH restored AS (UPDATE stock SET in_house_qty = in_house_qty + CASE WHEN $3 = 'in_house' THEN $4 ELSE 0 END, rsl_qty = CASE WHEN $3 = 'rsl' THEN COALESCE(rsl_qty, 0) + $4 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE sku_id = $2 AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = TRUE) RETURNING stock_id) UPDATE order_items SET stock_deducted = FALSE, deducted_at = NULL WHERE order_id = $1 AND EXISTS (SELECT 1 FROM restored) RETURNING qty`

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
9A	พิมพ์ใบปะสินค้า	Q9A.1	shipping.ts Q9A_1	SELECT order_id, MAX(label_template) AS label_template, MAX(parcel_total)::integer AS parcel_total, MAX(printed_at) AS printed_at FROM labels GROUP BY order_id ORDER BY order_id
9A	พิมพ์ใบปะสินค้า	Q9A.2	shipping.ts Q9A_1_PRINT	SELECT l.label_id, l.order_id, l.label_template, l.rsl_no, l.parcel_seq, l.parcel_total, o.order_status, o.shipping_address, o.shipping_method FROM labels l INNER JOIN orders o ON o.order_id = l.order_id WHERE l.order_id = $1 ORDER BY l.parcel_seq
9A	พิมพ์ใบปะสินค้า	Q9A.3	shipping.ts Q9A_2_HISTORY	INSERT INTO label_print_history (label_print_history_id, print_attempt_id, label_id, admin_id, is_reprint, outcome, error_code) VALUES ($1, $2, $3, $4, $5, $6, $7) ON CONFLICT (print_attempt_id, label_id) DO NOTHING
9A	พิมพ์ใบปะสินค้า	Q9A.4	shipping.ts Q9A_2_COMPLETE	WITH updated_labels AS (UPDATE labels SET label_status = 'printed', printed_at = CURRENT_TIMESTAMP WHERE order_id = $1) UPDATE orders SET order_status = 'พิมพ์ใบปะสินค้าแล้ว', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอพิมพ์ใบปะสินค้า'
10A	ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)	Q10A.1	cleanup.ts Q10A_5_DELETE_ORDER	WITH eligible AS MATERIALIZED (SELECT o.order_id, jsonb_build_object('order', to_jsonb(o), 'items', COALESCE((SELECT jsonb_agg(to_jsonb(i)) FROM order_items i WHERE i.order_id = o.order_id), '[]'::jsonb), 'labels', COALESCE((SELECT jsonb_agg(to_jsonb(l)) FROM labels l WHERE l.order_id = o.order_id), '[]'::jsonb)) AS snapshot FROM orders o WHERE o.order_id = $1 AND o.order_status IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') FOR UPDATE), logged AS (INSERT INTO deletion_history (deletion_history_id, order_reference, order_snapshot, admin_id) SELECT $2, order_id, snapshot, $3 FROM eligible RETURNING order_reference), detached_cost_history AS (UPDATE cost_history SET cost_id = NULL WHERE cost_id IN (SELECT cost_id FROM costs WHERE order_id IN (SELECT order_reference FROM logged)) RETURNING cost_history_id), deleted_costs AS (DELETE FROM costs WHERE order_id IN (SELECT order_reference FROM logged) AND (SELECT COUNT() FROM detached_cost_history) >= 0 RETURNING cost_id), deleted AS (DELETE FROM orders WHERE order_id IN (SELECT order_reference FROM logged) AND (SELECT COUNT() FROM deleted_costs) >= 0 RETURNING order_id) SELECT order_id FROM deleted
10A	ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)	Q10A.2	cleanup.ts Q10A_3_DELETE_LABEL	DELETE FROM labels l USING orders o WHERE l.order_id = $1 AND o.order_id = l.order_id AND o.order_status IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') RETURNING l.label_id
10A	ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)	Q10A.3	cleanup.ts Q10A_2_DELETE_COST	WITH sku_row AS (SELECT sku_id FROM skus WHERE sku = $1), deleted_history AS (DELETE FROM cost_history h USING sku_row s WHERE h.sku_id = s.sku_id AND h.changed_at = $2::timestamptz RETURNING h.cost_history_id) SELECT 'history' AS kind FROM deleted_history
10A	ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)	Q10A.4	cleanup.ts Q10A_4_DELETE_REORDER	DELETE FROM reorders r WHERE r.reorder_id = $1 AND (r.reorder_status IN ('ยกเลิกแล้ว', 'ได้รับสินค้าแล้ว') OR (r.reorder_status = 'สั่งซื้อแล้ว' AND r.expected_date < CURRENT_DATE)) AND NOT EXISTS (SELECT 1 FROM orders o WHERE o.reorder_id = r.reorder_id AND o.order_status NOT IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว')) RETURNING r.reorder_id
10A	ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น /	Q10A.5	cleanup.ts Q10A_6	SELECT order_reference, admin_id, deleted_at FROM deletion_history ORDER BY deleted_at DESC

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
	Cost / Label / Reorder)			
1S	จับคู่ Order กับเลข RSL	Q1S.1	orders.ts Q1S_1_QUEUE	SELECT o.order_id FROM orders o WHERE (o.order_status = 'ยังไม่ได้จับคู่' OR (o.order_status = 'รอดำเนินการด้วยตนเอง' AND o.manual_reason = 'rsl')) AND EXISTS (SELECT 1 FROM order_items oi WHERE oi.order_id = o.order_id AND oi.fulfill_source = 'rsl') ORDER BY o.order_date, o.order_id
1S	จับคู่ Order กับเลข RSL	Q1S.2	orders.ts Q2A_1_FOR_UPDATE	SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o
1S	จับคู่ Order กับเลข RSL	Q1S.3	orders.ts Q1S_1	UPDATE orders SET rsl_no = $2, rsl_matched_at = CASE WHEN $2::text IS NULL THEN NULL ELSE CURRENT_TIMESTAMP END, order_status = $3, manual_reason = $4, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1
1S	จับคู่ Order กับเลข RSL	Q1S.4	orders.ts Q1S_1_UNMATCH	WITH eligible AS (SELECT order_id FROM orders WHERE order_id = $1 AND rsl_no IS NOT NULL AND (order_status IN ('รอจัดรูปแบบใบปะสินค้า', 'รอพิมพ์ใบปะสินค้า') OR (order_status = 'รอดำเนินการด้วยตนเอง' AND rsl_no IS NOT NULL))), removed_labels AS (DELETE FROM labels WHERE order_id IN (SELECT order_id FROM eligible)) UPDATE orders SET rsl_no = NULL, rsl_matched_at = NULL, order_status = 'ยังไม่ได้จับคู่', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id IN (SELECT order_id FROM eligible) RETURNING order_id
2S	จับคู่กฎ SKU	Q2S.1	orders.ts Q2A_1_FOR_UPDATE	SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o
2S	จับคู่กฎ SKU	Q2S.2	orders.ts Q2S_1	SELECT sku_id, sku, product_name, variation FROM skus WHERE channel_sku = $1 AND sales_channel = $2 AND sku_status = 'active' ORDER BY sku_id
2S	จับคู่กฎ SKU	Q2S.3	orders.ts Q2S_2	WITH matched_item AS (UPDATE order_items SET sku_id = $2 WHERE order_id = $1) UPDATE orders SET order_status = $3, manual_reason = $4, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจับคู่กฎ SKU'
2S	จับคู่กฎ SKU	Q2S.4	auto-error-logs.ts	INSERT INTO auto_error_logs (auto_error_log_id, use_case_id, context_key, error_code, safe_message) VALUES ($1, $2, $3, $4, $5)

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
			Q_AUTO_ERROR_LOG	
2S	จับคู่กฎ SKU	Q2S.5	orders.ts Q2S_1_RETRY	SELECT order_id FROM orders WHERE order_status = 'รอจับคู่กฎ SKU' ORDER BY order_date, order_id LIMIT 100 FOR UPDATE SKIP LOCKED
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.1	stock.ts Q3S_1_ORDER	SELECT stock.stock_id, stock.sku_id, stock.in_house_qty, stock.rsl_qty FROM stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1 FOR UPDATE OF stock
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.2	stock.ts Q3S_1_RSL	SELECT stock.rsl_qty FROM stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.3	stock.ts Q3S_1_RESERVED	SELECT COALESCE(SUM(item.qty) FILTER (WHERE item.fulfill_source = 'rsl'), 0) AS rsl_reserved, COALESCE(SUM(item.qty) FILTER (WHERE item.fulfill_source = 'in_house'), 0) AS in_house_reserved FROM order_items item INNER JOIN orders ON orders.order_id = item.order_id WHERE item.sku_id = $1 AND item.stock_deducted = FALSE AND orders.order_status = ANY($2::text[]) AND orders.order_id <> $3
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.4	stock.ts Q3S_1_ROUTE	WITH latest_stock AS (UPDATE stock SET rsl_qty = COALESCE($2, rsl_qty), updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1), item_source AS (UPDATE order_items SET fulfill_source = COALESCE($5, fulfill_source) WHERE order_id = $3 AND EXISTS (SELECT 1 FROM orders WHERE order_id = $3 AND order_status = 'รอตรวจสอบสต๊อก')) UPDATE orders SET order_status = $4, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $3 AND order_status = 'รอตรวจสอบสต๊อก'
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.5	auto-error-logs.ts Q_AUTO_ERROR_LOG	INSERT INTO auto_error_logs (auto_error_log_id, use_case_id, context_key, error_code, safe_message) VALUES ($1, $2, $3, $4, $5)
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.6	stock.ts Q3S_1	SELECT sku.sku, stock.in_house_qty, stock.rsl_qty, stock.updated_at FROM stock INNER JOIN skus sku ON sku.sku_id = stock.sku_id ORDER BY sku.sku
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.7	stock.ts Q3S_1_FOR_UPDATE	SELECT stock.stock_id, stock.sku_id, stock.in_house_qty, stock.rsl_qty FROM stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1 FOR UPDATE OF stock
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.8	stock.ts Q3S_1_ADJUST	WITH adjusted AS (UPDATE stock SET in_house_qty = $2, rsl_qty = $3, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 RETURNING stock_id) INSERT INTO rakuten_stock_sync_states (stock_id, sync_status) SELECT stock_id, 'pending' FROM adjusted ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL
3S	ตรวจสอบของสินค้าใน stock + RSL	Q3S.9	stock.ts Q3S_1_INSERT_HISTORY	INSERT INTO stock_adjustment_history (stock_adjustment_history_id, stock_id, previous_in_house_qty, new_in_house_qty, previous_rsl_qty, new_rsl_qty, reason, admin_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)
4S	คำนวณ Cost ในการสั่งสินค้าเติม stock เอง	Q4S.1	costs.ts Q4S_1	SELECT sku.sku, costs.purchase_price, costs.currency, costs.exchange_rate, costs.exchange_rate_updated_at, costs.intl_freight, costs.duty_fee, costs.marketplace_fee, costs.domestic_shipping, costs.rsl_charge, costs.order_qty, costs.unit_cost, costs.calculated_at FROM costs INNER JOIN skus sku ON sku.sku_id = costs.sku_id ORDER BY sku.sku, costs.calculated_at NULLS LAST, costs.cost_id

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
4S	คำนวณ Cost ในการสั่งสินค้าเติม stock เอง	Q4S.2	cost-history.ts Q4S_2_HISTORY	`SELECT sku.sku, history.new_value, history.admin_id, history.changed_at FROM cost_history history INNER JOIN skus sku ON sku.sku_id = history.sku_id ORDER BY history.changed_at, history.cost_history_id`
4S	คำนวณ Cost ในการสั่งสินค้าเติม stock เอง	Q4S.3	skus.ts Q3A_1_FOR_UPDATE	`SELECT sku_id, sku, channel_sku FROM skus WHERE sku = $1 FOR UPDATE`
4S	คำนวณ Cost ในการสั่งสินค้าเติม stock เอง	Q4S.4	costs.ts Q4S_1_FOR_UPDATE	`SELECT costs.cost_id, to_jsonb(costs) AS snapshot, costs.exchange_rate, costs.exchange_rate_updated_at FROM costs WHERE costs.sku_id = $1 ORDER BY costs.calculated_at DESC NULLS LAST, costs.cost_id LIMIT 1 FOR UPDATE`
4S	คำนวณ Cost ในการสั่งสินค้าเติม stock เอง	Q4S.5	costs.ts Q4S_2_SAVE	`INSERT INTO costs (cost_id, purchase_price, currency, exchange_rate, exchange_rate_updated_at, intl_freight, duty_fee, marketplace_fee, domestic_shipping, rsl_charge, order_qty, unit_cost, calculated_at, sku_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, CURRENT_TIMESTAMP, $13) ON CONFLICT (cost_id) DO UPDATE SET purchase_price = EXCLUDED.purchase_price, currency = EXCLUDED.currency, exchange_rate = EXCLUDED.exchange_rate, exchange_rate_updated_at = EXCLUDED.exchange_rate_updated_at, intl_freight = EXCLUDED.intl_freight, duty_fee = EXCLUDED.duty_fee, marketplace_fee = EXCLUDED.marketplace_fee, domestic_shipping = EXCLUDED.domestic_shipping, rsl_charge = EXCLUDED.rsl_charge, order_qty = EXCLUDED.order_qty, unit_cost = EXCLUDED.unit_cost, calculated_at = CURRENT_TIMESTAMP RETURNING cost_id`
4S	คำนวณ Cost ในการสั่งสินค้าเติม stock เอง	Q4S.6	cost-history.ts Q4S_2_INSERT_HISTORY	`INSERT INTO cost_history ( cost_history_id, sku_id, cost_id, previous_value, new_value, admin_id ) VALUES ($1, $2, $3, $4::jsonb, $5::jsonb, $6)`
4S	คำนวณ Cost ในการสั่งสินค้าเติม stock เอง	Q4S.7	costs.ts Q4S_1_SELLING_PRICE	`SELECT selling_price FROM skus WHERE sku_id = $1`
5S	ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace	Q5S.1	shipping.ts Q5S_1	`SELECT order_id, order_status, marketplace_order_id, tracking_number, carrier_name, tracking_notified, delivery_status FROM orders WHERE order_id = $1 FOR UPDATE`
5S	ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace	Q5S.2	shipping.ts Q5S_1_RETRY	`SELECT attempt_id FROM tracking_notification_logs WHERE order_id = $1 AND tracking_number = $2 AND outcome IN ('pending', 'failed') ORDER BY sent_at DESC LIMIT 1`
5S	ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace	Q5S.3	shipping.ts Q5S_1_ATTEMPT	`INSERT INTO tracking_notification_logs (tracking_notification_log_id, attempt_id, order_id, tracking_number, outcome, sent_at, admin_id, error_code) VALUES ($1, $2, $3, $4, 'pending', CURRENT_TIMESTAMP, $5, NULL) ON CONFLICT (attempt_id) DO UPDATE SET outcome = 'pending', sent_at = CURRENT_TIMESTAMP, admin_id = EXCLUDED.admin_id, error_code = NULL`
5S	ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace	Q5S.4	shipping.ts Q5S_2	`WITH finished AS (UPDATE tracking_notification_logs SET outcome = $2, error_code = $3, sent_at = CURRENT_TIMESTAMP WHERE attempt_id = $1 RETURNING order_id) UPDATE orders o SET tracking_notified = CASE WHEN $2 = 'sent' THEN TRUE ELSE tracking_notified END, tracking_notified_at = CASE WHEN $2 = 'sent' THEN CURRENT_TIMESTAMP ELSE tracking_notified_at END, order_status = CASE WHEN $2 <> 'sent' AND tracking_notified = FALSE THEN 'รอแจ้งเลขติดตาม' WHEN $2 = 'sent' AND delivery_status = 'delivered' AND EXISTS (SELECT`

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
				1 FROM order_items WHERE order_id = o.order_id) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = o.order_id AND stock_deducted = FALSE) THEN 'จัดส่งสำเร็จ' WHEN $2 = 'sent' AND tracking_notified = FALSE THEN 'แจ้งเลขติดตามแล้ว' ELSE order_status END, updated_at = CURRENT_TIMESTAMP FROM finished WHERE o.order_id = finished.order_id
6S	จัดรูปแบบใบปะสินค้า	Q6S.1	orders.ts Q2A_1_FOR_UPDATE	SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o
6S	จัดรูปแบบใบปะสินค้า	Q6S.2	shipping.ts Q6S_1	SELECT o.order_id, o.sales_channel, o.rsl_no, o.shipping_address, o.shipping_method, item.qty, item.fulfill_source, o.order_status FROM orders o INNER JOIN LATERAL (SELECT qty, fulfill_source FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE WHERE o.order_status = 'รอจัดรูปแบบใบปะสินค้า' ORDER BY o.order_date, o.order_id
6S	จัดรูปแบบใบปะสินค้า	Q6S.3	shipping.ts Q6S_2_MANUAL	UPDATE orders SET order_status = 'รอดำเนินการด้วยตนเอง', manual_reason = $2, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจัดรูปแบบใบปะสินค้า'
6S	จัดรูปแบบใบปะสินค้า	Q6S.4	shipping.ts Q6S_2_INSERT	INSERT INTO labels (label_id, label_template, rsl_no, parcel_seq, parcel_total, label_status, created_at, order_id) SELECT $1, $2, $3, $4, $5, 'ready', CURRENT_TIMESTAMP, o.order_id FROM orders o WHERE o.order_id = $6 AND o.order_status = 'รอจัดรูปแบบใบปะสินค้า' ON CONFLICT (order_id, parcel_seq) DO NOTHING
6S	จัดรูปแบบใบปะสินค้า	Q6S.5	shipping.ts Q6S_2_ADVANCE	UPDATE orders SET order_status = 'รอพิมพ์ใบปะสินค้า', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจัดรูปแบบใบปะสินค้า' AND EXISTS (SELECT 1 FROM labels WHERE order_id = $1)
6S	จัดรูปแบบใบปะสินค้า	Q6S.6	shipping.ts Q6S_2_ERROR	INSERT INTO auto_error_logs (auto_error_log_id, use_case_id, context_key, error_code, safe_message) VALUES ($1, '6S', $2, $3, 'Label service could not generate labels')
6S	จัดรูปแบบใบปะสินค้า	Q6S.7	auto-error-logs.ts Q_AUTO_ERROR_LOG	INSERT INTO auto_error_logs (auto_error_log_id, use_case_id, context_key, error_code, safe_message) VALUES ($1, $2, $3, $4, $5)
7S	ตัดสต๊อกสินค้า	Q7S.1	orders.ts Q7S_1_RETRY	SELECT o.order_id FROM orders o WHERE o.order_status = 'แจ้งเลขติดตามแล้ว' AND (EXISTS (SELECT 1 FROM order_items item WHERE item.order_id = o.order_id AND item.stock_deducted = FALSE) OR (o.delivery_status = 'delivered' AND o.tracking_notified = TRUE)) ORDER BY o.order_date, o.order_id LIMIT 100 FOR UPDATE OF o SKIP LOCKED
7S	ตัดสต๊อกสินค้า	Q7S.2	stock.ts Q7S_1	SELECT item.order_item_id, item.sku_id, item.channel_sku, item.fulfill_source, item.qty, item.stock_deducted, stock.stock_id, stock.in_house_qty, stock.rsl_qty FROM

Use Case ID	Use Case Name	Query ID	ค่าคงที่ในโค้ด (src/lib/queries)	SQL Command
				order_items item INNER JOIN stock ON stock.sku_id = item.sku_id WHERE item.order_id = $1 ORDER BY stock.stock_id, item.order_item_id FOR UPDATE OF item, stock
7S	ตัดสต๊อกสินค้า	Q7S.3	stock.ts Q7S_1_MANUAL	UPDATE orders SET order_status = 'รอดำเนินการด้วยตนเอง', manual_reason = 'stock-deduct', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'แจ้งเลขติดตามแล้ว'
7S	ตัดสต๊อกสินค้า	Q7S.4	stock.ts Q7S_2	UPDATE stock SET in_house_qty = CASE WHEN $2 = 'in_house' THEN in_house_qty - $3 ELSE in_house_qty END, rsl_qty = CASE WHEN $2 = 'rsl' THEN rsl_qty - $3 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 AND CASE WHEN $2 = 'in_house' THEN in_house_qty >= $3 ELSE rsl_qty IS NOT NULL AND rsl_qty >= $3 END RETURNING stock_id, sku_id, in_house_qty, rsl_qty
7S	ตัดสต๊อกสินค้า	Q7S.5	stock.ts Q7S_3	UPDATE order_items SET stock_deducted = TRUE, deducted_at = CURRENT_TIMESTAMP WHERE order_item_id = ANY($1::text[]) AND stock_deducted = FALSE
7S	ตัดสต๊อกสินค้า	Q7S.6	stock-sync.ts Q7S_4_PENDING	INSERT INTO rakuten_stock_sync_states (stock_id, sync_status, last_error_code) VALUES ($1, 'pending', NULL) ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL
7S	ตัดสต๊อกสินค้า	Q7S.7	shipping.ts Q8A_1_DELIVERED	UPDATE orders SET order_status = 'จัดส่งสำเร็จ', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'แจ้งเลขติดตามแล้ว' AND tracking_notified = TRUE AND delivery_status = 'delivered' AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE)
7S	ตัดสต๊อกสินค้า	Q7S.8	stock-sync.ts Q7S_4_LOCK_PENDING	SELECT sync.stock_id, sku.channel_sku, (stock.in_house_qty + COALESCE(stock.rsl_qty, 0)) AS available_qty FROM rakuten_stock_sync_states sync INNER JOIN stock ON stock.stock_id = sync.stock_id INNER JOIN skus sku ON sku.sku_id = stock.sku_id WHERE sync.sync_status = 'pending' AND ($1::text[] IS NULL OR sync.stock_id = ANY($1::text[])) ORDER BY sync.stock_id FOR UPDATE OF sync, stock
7S	ตัดสต๊อกสินค้า	Q7S.9	stock-sync.ts Q7S_5_ATTEMPT	UPDATE rakuten_stock_sync_states SET last_attempt_at = CURRENT_TIMESTAMP WHERE stock_id = $1 AND sync_status = 'pending'
7S	ตัดสต๊อกสินค้า	Q7S.10	stock-sync.ts Q7S_5_SUCCESS	UPDATE rakuten_stock_sync_states sync SET sync_status = 'synced', last_synced_qty = $2, last_synced_at = CURRENT_TIMESTAMP, last_error_code = NULL FROM stock WHERE sync.stock_id = $1 AND stock.stock_id = sync.stock_id AND stock.in_house_qty + COALESCE(stock.rsl_qty, 0) = $2
7S	ตัดสต๊อกสินค้า	Q7S.11	stock-sync.ts Q7S_5_FAILURE	UPDATE rakuten_stock_sync_states SET sync_status = 'pending', last_error_code = $2 WHERE stock_id = $1
7S	ตัดสต๊อกสินค้า	Q7S.12	auto-error-logs.ts Q_AUTO_ERROR_LOG	INSERT INTO auto_error_logs (auto_error_log_id, use_case_id, context_key, error_code, safe_message) VALUES ($1, $2, $3, $4, $5)

2.17 Table Structure

โครงสร้างตารางทั้ง 17 ตาราง ชื่อคอลัมน์ ชนิดข้อมูล key และเงื่อนไข CHECK ตรงกับ db/migrations ของระบบ แต่ละตารางแสดง data dictionary และตัวอย่างข้อมูล 5 รายการจากข้อมูลตั้งต้นของเว็บ (npm run db:reset) ยกเว้นตาราง admins ซึ่งระบบมีผู้ใช้เพียง Super Admin ของเจ้าของ จึงมีตัวอย่าง 2 แถว คือบัญชีหลัก และบัญชีที่ใช้ทดสอบการล็อก ตารางที่ข้อมูลตั้งต้นมีไม่ถึง 5 แถว เช่น sessions เติมแถวตัวอย่างที่ key อ้างถึงกันถูกต้องเฉพาะในตารางตัวอย่างนี้ ส่วนผลรัน SQL ใน 2.23 ใช้ข้อมูลตั้งต้นเดิม

2.17.1 ตารางผู้ดูแลระบบ (ADMINS)

ตารางที่ 2-23 Data Dictionary ของตารางผู้ดูแลระบบ (ADMINS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	admin_id	รหัสผู้ดูแลระบบ	text	PK	Not Null	
2	user_email	อีเมลที่ใช้เข้าสู่ระบบ	text		Not Null	ห้ามซ้ำ (UNIQUE)
3	user_password	รหัสผ่านที่เข้ารหัสแบบ hash (scrypt) แล้ว	text		Not Null	
4	fail_attempts	จำนวนครั้งที่เข้าสู่ระบบผิดติดกัน ครบ 5 ครั้งบัญชีถูกล็อก 15 นาที	INTEGER		Not Null	fail_attempts >= 0; ค่าเริ่มต้น 0
5	last_login	วันเวลาที่เข้าสู่ระบบสำเร็จครั้งล่าสุด	TIMESTAMPTZ			
6	created_at	วันเวลาที่สร้างบัญชี	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP
7	locked_until	เวลาสิ้นสุดการล็อกบัญชีหลังเข้าสู่ระบบผิด 5 ครั้ง (NULL = ไม่ถูกล็อก) ครบเวลาแล้วปลดล็อกเอง	TIMESTAMPTZ			

ตารางที่ 2-24 ตัวอย่างข้อมูลของตารางผู้ดูแลระบบ (ADMINS) (คอลัมน์ 1-6)

admin_id	user_email	user_password	fail_attempts	last_login	created_at
U001	admin@colorado.jp	scrypt$16384$8$1$yFu3Dhgi6N9Az8G5pC-	0	NULL	2026-11-06 03:07

admin_id	user_email	user_password	fail_attempts	last_login	created_at
		K5g$2vNbQ3dPbBot WW7GtOH7okBthgVV wKV9synZ7JfHMjp8f RmqUFR_Ncu6EQrm moJ1- vaToeuO2VoeUplGp M7inw			
U002	locked@colorado.jp	scrypt$16384$8$1$y Fu3Dhgi6N9Az8G5pC - K5g$2vNbQ3dPbBot WW7GtOH7okBthgVV wKV9synZ7JfHMjp8f RmqUFR_Ncu6EQrm moJ1- vaToeuO2VoeUplGp M7inw	5	NULL	2026-11-06 03:07

ตารางที่ 2-25 ตัวอย่างข้อมูลของตารางผู้ดูแลระบบ (ADMINS) (คอลัมน์ 7-7)

locked_until
NULL
2026-11-06 03:22

2.17.2 ตารางซัพพลายเออร์ (SUPPLIERS)

ตารางที่ 2-26 Data Dictionary ของตารางซัพพลายเออร์ (SUPPLIERS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	supplier_id	รหัสซัพพลายเออร์	text	PK	Not Null	
2	supplier_name	ชื่อซัพพลายเออร์	text		Not Null	
3	contact_info	ช่องทางติดต่อสำหรับส่งคำสั่งซื้อ	text		Not Null	
4	lead_time_days	จำนวนวันตั้งแต่สั่งซื้อจนได้รับสินค้า	INTEGER		Not Null	lead_time_days >= 0
5	created_at	วันเวลาที่เพิ่มซัพพลายเออร์	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP

ตารางที่ 2-27 ตัวอย่างข้อมูลของตารางซัพพลายเออร์ (SUPPLIERS)

supplier_id	supplier_name	contact_info	lead_time_days	created_at
SUP-01	Ningbo Tackle Works	order@ningbo-tackle.example.cn	21	2026-11-06 03:07
SUP-02	หจก. สมุนไพรไทยเจริญ	sales@thaiherb.example.co.th	10	2026-11-06 03:07
SUP-03	Qingdao Outdoor Gear	po@qingdao-outdoor.example.cn	28	2026-11-06 03:07
SUP-04	Foshan Homeware Ltd.	trade@foshan-home.example.cn	18	2026-11-06 03:07
SUP-05	Yiwu Fishing Supply Co.	sales@yiwu-fishing.example.cn	25	2026-11-06 03:07

2.17.3 ตารางสินค้าและกฎ SKU (SKUS)

ตารางที่ 2-28 Data Dictionary ของตารางสินค้าและกฎ SKU (SKUS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	sku_id	รหัสรายการสินค้าในระบบ	text	PK	Not Null	
2	sku	รหัสสินค้าภายในร้าน ใช้เชื่อมข้อมูลทุกระบบ	text		Not Null	ห้ามซ้ำ (UNIQUE)
3	product_name	ชื่อสินค้า	text		Not Null	
4	variation	ตัวเลือกของสินค้า เช่น สี ขนาด	text		Not Null	
5	sales_channel	ช่องทางขาย (เฟส 1 มีเฉพาะ Rakuten Ichiba)	text		Not Null	sales_channel = 'Rakuten Ichiba'
6	channel_sku	รหัสสินค้าในช่องทางขาย ใช้เป็นกฎจับคู่ SKU (2S) หนึ่งสินค้ามีหนึ่งกฎ	text		Not Null	ห้ามซ้ำ (UNIQUE)
7	selling_price	ราคาขายปัจจุบัน	NUMERIC(14,2)			
8	reorder_threshold	จุดสั่งซื้อ ถ้าสต๊อกไม่เกินค่านี้ต้องสั่งเพิ่ม	INTEGER		Not Null	reorder_threshold >= 0
9	reorder_qty	จำนวนที่สั่งซื้อต่อครั้ง	INTEGER		Not Null	reorder_qty >= 0
10	sku_status	สถานะการขาย (active, inactive)	text		Not Null	

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
11	created_at	วันเวลาที่เพิ่มสินค้า	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP
12	supplier_id	ซัพพลายเออร์ที่สั่งซื้อสินค้านี้	text	FK ⟶ suppliers		

ตารางที่ 2-29 ตัวอย่างข้อมูลของตารางสินค้าและกฎ SKU (SKUS) (คอลัมน์ 1-6)

sku_id	sku	product_name	variation	sales_channel	channel_sku
SKU-FSH-MLR-240	FSH-MLR-240	เหยื่อปลอมมินโนว์ จมช้า 24 กรัม	สีเงินลายปลาชิว	Rakuten Ichiba	rk-mlr240-sv
SKU-YDM-HRB-012	YDM-HRB-012	ยาดมสมุนไพร สูตรเข้มข้น	แพ็ค 3 หลอด	Rakuten Ichiba	rk-ydm012-3
SKU-OUT-CHR-003	OUT-CHR-003	เก้าอี้พับแคมป์ปิ้ง โครงอะลูมิเนียม	สีเขียวมะกอก	Rakuten Ichiba	rk-chr003-ol
SKU-HOM-STG-045	HOM-STG-045	กล่องเก็บของพับได้ ฝาใส	ขนาด 28 ลิตร	Rakuten Ichiba	rk-stg045-28
SKU-FSH-RDS-180	FSH-RDS-180	คันเบ็ดสปินนิ่ง 1.8 เมตร	แอ็คชันกลาง	Rakuten Ichiba	rk-rds180-m

ตารางที่ 2-30 ตัวอย่างข้อมูลของตารางสินค้าและกฎ SKU (SKUS) (คอลัมน์ 7-12)

selling_price	reorder_threshold	reorder_qty	sku_status	created_at	supplier_id
1280.00	40	200	active	2026-11-06 03:07	SUP-01
540.00	60	300	active	2026-11-06 03:07	SUP-02
3450.00	15	60	active	2026-11-06 03:07	SUP-03
NULL	25	120	active	2026-11-06 03:07	SUP-04
2690.00	10	40	active	2026-11-06 03:07	SUP-01

2.17.4 ตารางสต๊อกสินค้า (STOCK)

ตารางที่ 2-31 Data Dictionary ของตารางสต๊อกสินค้า (STOCK)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	stock_id	รหัสรายการสต๊อก	text	PK	Not Null	

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
2	in_house_qty	จำนวนคงเหลือในคลังบริษัท	INTEGER		Not Null	in_house_qty >= 0
3	rsl_qty	จำนวนคงเหลือในคลัง RSL (ว่างได้เมื่ออ่านข้อมูลจาก RSL ไม่ได้)	INTEGER			(rsl_qty IS NULL) OR (rsl_qty >= 0)
4	updated_at	วันเวลาที่ปรับปรุงยอดล่าสุด	TIMESTAMPTZ		Not Null	
5	sku_id	สินค้าของสต๊อกนี้ (หนึ่งสินค้ามีหนึ่งรายการ)	text	FK ⟶ skus	Not Null	ห้ามซ้ำ (UNIQUE)

ตารางที่ 2-32 ตัวอย่างข้อมูลของตารางสต๊อกสินค้า (STOCK)

stock_id	in_house_qty	rsl_qty	updated_at	sku_id
STK-FSH-MLR-240	52	96	2026-10-04 21:00	SKU-FSH-MLR-240
STK-YDM-HRB-012	118	NULL	2026-10-04 21:00	SKU-YDM-HRB-012
STK-OUT-CHR-003	6	8	2026-10-04 21:00	SKU-OUT-CHR-003
STK-HOM-STG-045	1	0	2026-10-04 21:00	SKU-HOM-STG-045
STK-FSH-RDS-180	0	1	2026-10-04 21:00	SKU-FSH-RDS-180

2.17.5 ตารางคำสั่งซื้อ (ORDERS)

ตารางที่ 2-33 Data Dictionary ของตารางคำสั่งซื้อ (ORDERS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	order_id	รหัส Order ในระบบ	text	PK	Not Null	
2	marketplace_order_id	เลขคำสั่งซื้อใน Rakuten Ichiba	text		Not Null	ห้ามซ้ำ (UNIQUE)
3	sales_channel	ช่องทางขาย	text		Not Null	sales_channel = 'Rakuten Ichiba'
4	order_date	วันเวลาที่ลูกค้าสั่งซื้อ	TIMESTAMPTZ		Not Null	
5	shipping_address	ที่อยู่จัดส่ง	text		Not Null	

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
6	shipping_method	วิธีจัดส่ง	text		Not Null	
7	order_status	สถานะของ Order (ดู State Diagram)	text		Not Null	order_status IN 'รอตรวจสอบคำสั่งซื้อ', 'รอจับคู่กฎ SKU', 'รอตรวจสอบสต๊อก', 'รอ Admin ตัดสินใจสั่งซื้อ', 'รอสั่งซื้อจาก Supplier', 'สั่งซื้อแล้ว', 'ยังไม่ได้จับคู่', 'รอจัดรูปแบบใบปะสินค้า', 'รอพิมพ์ใบปะสินค้า', 'พิมพ์ใบปะสินค้าแล้ว', 'รอส่งคำสั่งซื้อ', 'รอส่งมอบ', 'อยู่ระหว่างจัดส่ง', 'รอแจ้งเลขติดตาม', 'แจ้งเลขติดตามแล้ว', 'จัดส่งสำเร็จ', 'จัดส่งไม่สำเร็จ', 'ตีกลับ/คืนสินค้า', 'รอยกเลิก Order', 'ยกเลิกแล้ว', 'รอดำเนินการด้วยตนเอง', 'รอดำเนินการพิเศษ'
8	rsl_no	เลขอ้างอิง RSL หลังจับคู่ (1S)	text			
9	rsl_matched_at	วันเวลาที่จับคู่ RSL	TIMESTAMPTZ			
10	tracking_number	หมายเลขติดตามพัสดุจากขนส่ง (8A)	text			
11	carrier_name	ชื่อบริษัทขนส่ง	text			
12	dispatched_at	วันเวลาที่ส่งมอบให้ขนส่ง	TIMESTAMPTZ			
13	tracking_notified	แจ้งเลขติดตามไป Rakuten แล้วหรือยัง (5S)	BOOLEAN		Not Null	ค่าเริ่มต้น false
14	tracking_notified_at	วันเวลาที่แจ้งเลขติดตามสำเร็จ	TIMESTAMPTZ			
15	cancel_reason	เหตุผลที่ยกเลิก (7A)	text			
16	cancelled_at	วันเวลาที่ยกเลิก	TIMESTAMPTZ			
17	verified_at	วันเวลาที่ Admin ยืนยันคำสั่งซื้อ (2A)	TIMESTAMPTZ			
18	created_at	วันเวลาที่นำเข้า Order	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP
19	updated_at	วันเวลาที่แก้ไขล่าสุด	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
20	manual_reason	สาเหตุที่ต้องดำเนินการด้วยตนเอง เช่น ข้อมูลไม่ครบ ไม่พบ SKU ไม่พบ template	text			manual_reason IN 'incomplete', 'sku-unregistered', 'sku-rule', 'rsl', 'template', 'address', 'price', 'supplier', 'stock-deduct'
21	customer_cancel_requested	ลูกค้าขอยกเลิกผ่าน Rakuten หรือไม่	BOOLEAN		Not Null	ค่าเริ่มต้น false
22	edit_log	ประวัติการแก้ไขข้อมูล Order (JSON)	jsonb		Not Null	ค่าเริ่มต้น '[]'
23	reorder_decision	ผลการตัดสินใจสั่งซื้อเพิ่ม (5A) คุ้มค่า หรือ ไม่คุ้มค่า	text			(reorder_decision IS NULL) OR (reorder_decision IN 'คุ้มค่า', 'ไม่คุ้มค่า')
24	decided_qty	จำนวนที่ตัดสินใจสั่งซื้อ	INTEGER			(decided_qty IS NULL) OR (decided_qty > 0)
25	delivery_status	สถานะพัสดุล่าสุดจากขนส่ง (in_transit, delivered, failed, returned)	text			(delivery_status IS NULL) OR (delivery_status IN 'in_transit', 'delivered', 'failed', 'returned')
26	reorder_id	คำสั่งซื้อเติมสต๊อกที่ Order นี้รออยู่ (6A ผูก, ว่างได้) คำสั่งซื้อหนึ่งรายการครอบคลุมได้หลาย Order	text	FK ⟶ reorders		

ตารางที่ 2-34 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 1-6)

order_id	marketplace_order_id	sales_channel	order_date	shipping_address	shipping_method
ORD-24190	377841-20261005-0000190	Rakuten Ichiba	2026-10-05 00:12	東京都杉並区高円寺南 3-14-7 サンハイツ 402	RSL ปกติ
ORD-24194	377841-20261005-0000194	Rakuten Ichiba	2026-10-04 23:47	長野県松本市中央 2-5-11	จัดส่งเอง
ORD-24189	377841-20261005-0000189	Rakuten Ichiba	2026-10-04 23:47	大阪府吹田市江坂町 1-23-5	RSL ปกติ
ORD-24188	377841-20261005-0000188	Rakuten Ichiba	2026-10-04 23:02		RSL ปกติ
ORD-24187	377841-20261005-0000187	Rakuten Ichiba	2026-10-04 22:31	北海道札幌市中央区南九条西 4-2-18	RSL ปกติ

ตารางที่ 2-35 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 7-12)

order_status	rsl_no	rsl_matched_at	tracking_number	carrier_name	dispatched_at
รอตรวจสอบคำสั่งซื้อ	NULL	NULL	NULL	NULL	NULL
รอตรวจสอบคำสั่งซื้อ	NULL	NULL	NULL	NULL	NULL
รอตรวจสอบคำสั่งซื้อ	NULL	NULL	NULL	NULL	NULL
รอตรวจสอบคำสั่งซื้อ	NULL	NULL	NULL	NULL	NULL
รอตรวจสอบคำสั่งซื้อ	NULL	NULL	NULL	NULL	NULL

ตารางที่ 2-36 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 13-18)

tracking_notified	tracking_notified_at	cancel_reason	cancelled_at	verified_at	created_at
false	NULL	NULL	NULL	NULL	2026-10-05 00:12
false	NULL	NULL	NULL	NULL	2026-10-04 23:47
false	NULL	NULL	NULL	NULL	2026-10-04 23:47
false	NULL	NULL	NULL	NULL	2026-10-04 23:02
false	NULL	NULL	NULL	NULL	2026-10-04 22:31

ตารางที่ 2-37 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 19-24)

updated_at	manual_reason	customer_cancel_requested	edit_log	reorder_decision	decided_qty
2026-11-06 03:07	NULL	false		NULL	NULL
2026-11-06 03:07	NULL	false		NULL	NULL
2026-11-06 03:07	NULL	false		NULL	NULL
2026-11-06 03:07	NULL	false		NULL	NULL
2026-11-06 03:07	NULL	true		NULL	NULL

ตารางที่ 2-38 ตัวอย่างข้อมูลของตารางคำสั่งซื้อ (ORDERS) (คอลัมน์ 25-26)

delivery_status	reorder_id
NULL	NULL
NULL	NULL
NULL	NULL
NULL	NULL

delivery_status	reorder_id
NULL	NULL

2.17.6 ตารางรายการสินค้าในคำสั่งซื้อ (ORDER_ITEMS)

ตารางที่ 2-39 Data Dictionary ของตารางรายการสินค้าในคำสั่งซื้อ (ORDER_ITEMS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	order_item_id	รหัสรายการสินค้าใน Order	text	PK	Not Null	
2	channel_sku	รหัสสินค้าตามช่องทางขายที่มากับ Order	text		Not Null	
3	variation	ตัวเลือกสินค้า	text		Not Null	
4	qty	จำนวนที่สั่ง	INTEGER		Not Null	qty >= 0
5	fulfill_source	คลังที่ใช้ส่งสินค้า (rsl, in_house) กำหนดตอนตรวจสต๊อก (3S)	text		Not Null	fulfill_source IN 'in_house', 'rsl'
6	stock_deducted	ตัดสต๊อกแล้วหรือยัง (7S)	BOOLEAN		Not Null	ค่าเริ่มต้น false
7	deducted_at	วันเวลาที่ตัดสต๊อก	TIMESTAMPTZ			
8	order_id	Order ที่รายการนี้อยู่	text	FK ⟶ orders	Not Null	
9	sku_id	สินค้าในระบบหลังจับคู่กฎ SKU (2S) ว่างได้ถ้ายังไม่จับคู่	text	FK ⟶ skus		

ตารางที่ 2-40 ตัวอย่างข้อมูลของตารางรายการสินค้าในคำสั่งซื้อ (ORDER_ITEMS) (คอลัมน์ 1-6)

order_item_id	channel_sku	variation	qty	fulfill_source	stock_deducted
ORD-24190-ITEM-1	rk-mlr240-sv	สีเงินลายปลาซิว	2	rsl	false
ORD-24194-ITEM-1	rk-stg045-28	ขนาด 28 ลิตร	1	rsl	false
ORD-24189-ITEM-1	rk-ydm012-3-new	แพ็ค 3 หลอด	1	rsl	false
ORD-24188-ITEM-1	rk-stg045-28	ขนาด 28 ลิตร	1	rsl	false
ORD-24187-ITEM-1	rk-lmp021-w	แสงวอร์ม	1	rsl	false

ตารางที่ 2-41 ตัวอย่างข้อมูลของตารางรายการสินค้าในคำสั่งซื้อ (ORDER_ITEMS) (คอลัมน์ 7-9)

deducted_at	order_id	sku_id
NULL	ORD-24190	SKU-FSH-MLR-240
NULL	ORD-24194	SKU-HOM-STG-045
NULL	ORD-24189	SKU-YDM-HRB-012
NULL	ORD-24188	SKU-HOM-STG-045
NULL	ORD-24187	SKU-OUT-LMP-021

2.17.7 ตารางต้นทุนสินค้า (COSTS)

ตารางที่ 2-42 Data Dictionary ของตารางต้นทุนสินค้า (COSTS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	cost_id	รหัสรายการต้นทุน	text	PK	Not Null	
2	purchase_price	ราคาซื้อทั้งล็อตในสกุลเงินของซัพพลายเออร์	NUMERIC(14,4)			
3	currency	สกุลเงินที่ซื้อ	text		Not Null	
4	exchange_rate	อัตราแลกเปลี่ยนที่ใช้แปลงราคาซื้อ	NUMERIC(14,6)			
5	intl_freight	ค่าขนส่งระหว่างประเทศทั้งล็อต	NUMERIC(14,4)			
6	duty_fee	ภาษีนำเข้าทั้งล็อต	NUMERIC(14,4)			
7	marketplace_fee	ค่าธรรมเนียม Marketplace ต่อชิ้น	NUMERIC(14,4)			
8	domestic_shipping	ค่าส่งในประเทศต่อชิ้น	NUMERIC(14,4)			
9	rsl_charge	ค่าธรรมเนียม RSL ต่อชิ้น	NUMERIC(14,4)			
10	order_qty	จำนวนชิ้นในล็อตที่ใช้หารต้นทุน	INTEGER			(order_qty IS NULL) OR (order_qty > 0)
11	unit_cost	ต้นทุนต่อหน่วยที่คำนวณได้ (4S)	NUMERIC(14,4)			
12	calculated_at	วันเวลาที่คำนวณล่าสุด	TIMESTAMPTZ			
13	sku_id	สินค้าของต้นทุนนี้	text	FK ⟶ skus	Not Null	

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
14	order_id	Order ที่อ้างอิงต้นทุนนี้ (ว่างได้)	text	FK ⟶ orders		
15	exchange_rate_updated_at	วันเวลาที่อัปเดตอัตราแลกเปลี่ยน ใช้ตรวจว่าอัตราเก่าเกินไปหรือไม่	TIMESTAMPTZ			

ตารางที่ 2-43 ตัวอย่างข้อมูลของตารางต้นทุนสินค้า (COSTS) (คอลัมน์ 1-6)

cost_id	purchase_price	currency	exchange_rate	intl_freight	duty_fee
COST-FSH-MLR-240	9100.0000	CNY	4.920000	8600.0000	3150.0000
COST-FSH-RDS-180	16400.0000	CNY	4.920000	5400.0000	2080.0000
COST-OUT-CHR-003	32280.0000	CNY	4.920000	11200.0000	4360.0000
COST-HOM-STG-045	8400.0000	CNY	4.920000	6800.0000	2100.0000
COST-HOM-KIT-108	3000.0000	CNY	4.920000	3000.0000	900.0000

ตารางที่ 2-44 ตัวอย่างข้อมูลของตารางต้นทุนสินค้า (COSTS) (คอลัมน์ 7-12)

marketplace_fee	domestic_shipping	rsl_charge	order_qty	unit_cost	calculated_at
72.0000	46.0000	18.0000	200	418.6100	2026-11-06 03:07
148.0000	92.0000	18.0000	40	2462.2000	2026-11-06 03:07
190.0000	135.0000	24.0000	60	3255.2933	2026-11-06 03:07
64.0000	58.0000	18.0000	120	558.5667	2026-11-06 03:07
60.0000	45.0000	18.0000	20	1056.0000	2026-11-06 03:07

ตารางที่ 2-45 ตัวอย่างข้อมูลของตารางต้นทุนสินค้า (COSTS) (คอลัมน์ 13-15)

sku_id	order_id	exchange_rate_updated_at
SKU-FSH-MLR-240	NULL	2026-11-06 01:07
SKU-FSH-RDS-180	NULL	2026-11-06 01:07
SKU-OUT-CHR-003	NULL	2026-11-03 03:07
SKU-HOM-STG-045	NULL	2026-11-06 01:07
SKU-HOM-KIT-108	NULL	2026-11-06 01:07

2.17.8 ตารางคำสั่งซื้อเติมสต๊อก (REORDERS)

ตารางที่ 2-46 Data Dictionary ของตารางคำสั่งซื้อเติมสต๊อก (REORDERS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	reorder_id	เลขที่คำสั่งซื้อเติมสต๊อก	text	PK	Not Null	
2	order_qty	จำนวนที่สั่งซื้อ	INTEGER		Not Null	order_qty > 0
3	decision	ผลการตัดสินใจที่มาของคำสั่งซื้อ	text			
4	decided_at	วันเวลาที่ตัดสินใจ	TIMESTAMPTZ			
5	reorder_status	สถานะคำสั่งซื้อ (รอส่งคำสั่งซื้อ, สั่งซื้อแล้ว, ได้รับสินค้าแล้ว, ยกเลิกแล้ว)	text		Not Null	
6	ordered_at	วันเวลาที่ส่งคำสั่งซื้อให้ซัพพลายเออร์ (6A)	TIMESTAMPTZ			
7	expected_date	วันที่คาดว่าจะได้รับสินค้า	DATE			
8	supplier_tracking_no	เลขติดตามพัสดุจากซัพพลายเออร์	text			
9	received_at	วันเวลาที่ Admin บันทึกการรับสินค้า (6A)	TIMESTAMPTZ			
10	created_at	วันเวลาที่สร้างคำสั่งซื้อ	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP
11	sku_id	สินค้าที่สั่งซื้อ	text	FK ⟶ skus	Not Null	
12	supplier_id	ซัพพลายเออร์ที่สั่งซื้อ	text	FK ⟶ suppliers	Not Null	
13	supplier_request_id	รหัสคำขอที่ส่งให้ซัพพลายเออร์ ใช้ซ้ำเมื่อส่งใหม่เพื่อไม่ให้สั่งซ้ำ	text			ห้ามซ้ำ (UNIQUE)
14	supplier_reference	เลขอ้างอิงที่ซัพพลายเออร์ตอบกลับ	text			
15	last_error_code	รหัสข้อผิดพลาดล่าสุดตอนส่งคำสั่งซื้อ	text			
16	received_qty	จำนวนที่ได้รับจริงจากซัพพลายเออร์ (6A รับสินค้า) ว่างได้จนกว่าจะรับสินค้า	INTEGER			(received_qty IS NULL) OR (received_qty > 0)

ตารางที่ 2-47 ตัวอย่างข้อมูลของตารางคำสั่งซื้อเติมสต๊อก (REORDERS) (คอลัมน์ 1-6)

reorder_id	order_qty	decision	decided_at	reorder_status	ordered_at
PO-0091	60	NULL	NULL	สั่งซื้อแล้ว	2026-09-24 01:00
PO-0087	40	คุ้มค่า	2026-08-28 02:00	ได้รับสินค้าแล้ว	2026-08-28 02:05
PO-0088	30	คุ้มค่า	2026-09-02 03:00	ได้รับสินค้าแล้ว	2026-09-02 03:05
PO-0089	25	คุ้มค่า	2026-09-10 02:00	ยกเลิกแล้ว	2026-09-10 02:05
PO-0090	50	คุ้มค่า	2026-10-05 01:30	รอส่งคำสั่งซื้อ	NULL

ตารางที่ 2-48 ตัวอย่างข้อมูลของตารางคำสั่งซื้อเติมสต๊อก (REORDERS) (คอลัมน์ 7-12)

expected_date	supplier_tracking_no	received_at	created_at	sku_id	supplier_id
2026-10-22 00:00	NULL	NULL	2026-09-24 01:00	SKU-OUT-CHR-003	SUP-03
2026-09-18 00:00	SF1029384756	2026-09-17 04:00	2026-08-28 02:00	SKU-FSH-MLR-240	SUP-01
2026-09-12 00:00	NULL	2026-09-11 05:00	2026-09-02 03:00	SKU-YDM-HRB-012	SUP-02
2026-09-28 00:00	NULL	NULL	2026-09-10 02:00	SKU-HOM-STG-045	SUP-04
NULL	NULL	NULL	2026-10-05 01:30	SKU-FSH-RDS-180	SUP-01

ตารางที่ 2-49 ตัวอย่างข้อมูลของตารางคำสั่งซื้อเติมสต๊อก (REORDERS) (คอลัมน์ 13-16)

supplier_request_id	supplier_reference	last_error_code	received_qty
NULL	NULL	NULL	NULL
REQ-PO-0087	NB-77120	NULL	40
REQ-PO-0088	TH-5521	NULL	30
REQ-PO-0089	FS-3310	NULL	NULL
REQ-PO-0090	NULL	external_unavailable	NULL

2.17.9 ตารางใบปะสินค้า (LABELS)

ตารางที่ 2-50 Data Dictionary ของตารางใบปะสินค้า (LABELS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	label_id	รหัสใบปะสินค้า	text	PK	Not Null	

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
2	label_template	รูปแบบใบปะสินค้าตามวิธีจัดส่ง	text		Not Null	
3	rsl_no	เลขอ้างอิง RSL ที่พิมพ์บนใบปะ	text			
4	parcel_seq	ลำดับกล่อง	INTEGER		Not Null	parcel_seq > 0
5	parcel_total	จำนวนกล่องทั้งหมดของ Order	INTEGER		Not Null	parcel_total > 0
6	label_status	สถานะใบปะ (ready, printed)	text		Not Null	
7	created_at	วันเวลาที่สร้างใบปะ (6S)	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP
8	printed_at	วันเวลาที่พิมพ์สำเร็จ (9A)	TIMESTAMPTZ			
9	order_id	Order ที่ใบปะนี้ใช้	text	FK ⟶ orders	Not Null	

ตารางที่ 2-51 ตัวอย่างข้อมูลของตารางใบปะสินค้า (LABELS) (คอลัมน์ 1-6)

label_id	label_template	rsl_no	parcel_seq	parcel_total	label_status
ORD-24181-LABEL-1	RSL-STD-A6	RSL-9f21a7	1	1	ready
ORD-24180-LABEL-1	RSL-LARGE-A5	RSL-9f21c8	1	2	ready
ORD-24180-LABEL-2	RSL-LARGE-A5	RSL-9f21c8	2	2	ready
ORD-24178-LABEL-1	RSL-STD-A6	RSL-9f21a7	1	1	printed
ORD-24177-LABEL-1	INHOUSE-YAMATO-A6	NULL	1	1	printed

ตารางที่ 2-52 ตัวอย่างข้อมูลของตารางใบปะสินค้า (LABELS) (คอลัมน์ 7-9)

created_at	printed_at	order_id
2026-10-03 10:44	NULL	ORD-24181
2026-10-03 02:02	NULL	ORD-24180
2026-10-03 02:02	NULL	ORD-24180
2026-10-02 01:19	2026-10-04 01:30	ORD-24178
2026-10-02 00:03	2026-10-04 01:31	ORD-24177

2.17.10 ตารางเซสชันเข้าสู่ระบบ (SESSIONS)

ตารางที่ 2-53 Data Dictionary ของตารางเซสชันเข้าสู่ระบบ (SESSIONS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	session_id	รหัสเซสชัน	text	PK	Not Null	
2	admin_id	ผู้ดูแลระบบเจ้าของเซสชัน	text	FK ⟶ admins	Not Null	
3	token_hash	ค่า hash ของ token ในคุกกี้ (ไม่เก็บ token จริง)	text		Not Null	ห้ามซ้ำ (UNIQUE)
4	created_at	วันเวลาที่เข้าสู่ระบบ (1A)	TIMESTAMPTZ		Not Null	expires_at > created_at; ค่าเริ่มต้น CURRENT_TIMESTAMP
5	expires_at	วันเวลาที่เซสชันหมดอายุ	TIMESTAMPTZ		Not Null	expires_at > created_at
6	revoked_at	วันเวลาที่ออกจากระบบ (ว่างถ้ายังใช้งาน)	TIMESTAMPTZ			

ตารางที่ 2-54 ตัวอย่างข้อมูลของตารางเซสชันเข้าสู่ระบบ (SESSIONS)

session_id	admin_id	token_hash	created_at	expires_at	revoked_at
SES-0001	U001	sha256:7f3c9a1e5b2d4c8f0a6e9b3d1c7f5a2e	2026-10-04 23:30	2026-10-05 11:30	NULL
SES-0002	U001	sha256:2b8e4f6a1c9d3e7b5f0a8c2d6e4b1f9a	2026-10-03 01:10	2026-10-03 13:10	2026-10-03 09:00
SES-0003	U001	sha256:9c1d7e3f5a2b8c4d6e0f1a3b5c7d9e2f	2026-10-04 00:05	2026-10-04 12:05	NULL
SES-0004	U001	sha256:4e6a8c0b2d4f6a8c1e3b5d7f9a1c3e5b	2026-10-04 08:15	2026-10-04 20:15	2026-10-04 10:00
SES-0005	U001	sha256:1a3c5e7b9d2f4a6c8e0b1d3f5a7c9e2d	2026-10-03 01:00	2026-10-03 13:00	NULL

2.17.11 ตารางบันทึกข้อผิดพลาดของงานอัตโนมัติ (AUTO_ERROR_LOGS)

ตารางที่ 2-55 Data Dictionary ของตารางบันทึกข้อผิดพลาดของงานอัตโนมัติ (AUTO_ERROR_LOGS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	auto_error_log_id	รหัสบันทึก	text	PK	Not Null	
2	use_case_id	Use Case อัตโนมัติที่เกิดข้อผิดพลาด เช่น 2S, 3S, 6S, 7S	text		Not Null	
3	context_key	รายการที่เกี่ยวข้อง เช่น รหัส Order	text		Not Null	
4	error_code	รหัสข้อผิดพลาด เช่น external_unavailable	text		Not Null	
5	safe_message	ข้อความที่แสดงได้โดยไม่เปิดเผยรายละเอียดภายใน	text		Not Null	
6	occurred_at	วันเวลาที่เกิดข้อผิดพลาด	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP

ตารางที่ 2-56 ตัวอย่างข้อมูลของตารางบันทึกข้อผิดพลาดของงานอัตโนมัติ (AUTO_ERROR_LOGS)

auto_error_log_id	use_case_id	context_key	error_code	safe_message	occurred_at
AEL-0001	2S	ORD-24185	external_unavailable	externalUnavailable	2026-10-05 01:05
AEL-0002	6S	ORD-24179	external_unavailable	Label service could not generate labels	2026-10-03 03:12
AEL-0003	3S	ORD-24176	stock_missing	Stock record is unavailable	2026-10-02 08:40
AEL-0004	7S	ORD-24172	external_unavailable	Rakuten stock sync failed, will retry	2026-10-01 06:05
AEL-0005	1S	ORD-24174	external_unavailable	RSL could not be reached	2026-10-02 02:30

2.17.12 ตารางประวัติการลบข้อมูล (DELETION_HISTORY)

ตารางที่ 2-57 Data Dictionary ของตารางประวัติการลบข้อมูล (DELETION_HISTORY)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	deletion_history_id	รหัสบันทึกการลบ	text	PK	Not Null	
2	order_reference	รหัส Order ที่ถูกลบ	text		Not Null	
3	order_snapshot	สำเนาข้อมูล Order ก่อนลบ (JSON)	jsonb		Not Null	
4	admin_id	ผู้ดูแลระบบที่สั่งลบ (10A)	text	FK ⟶ admins	Not Null	
5	deleted_at	วันเวลาที่ลบ	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP

ตารางที่ 2-58 ตัวอย่างข้อมูลของตารางประวัติการลบข้อมูล (DELETION_HISTORY)

deletion_history_id	order_reference	order_snapshot	admin_id	deleted_at
DEL-0001	ORD-22950	[object Object]	U001	2026-10-01 02:00
DEL-0002	ORD-22951	[object Object]	U001	2026-10-01 02:00
DEL-0003	ORD-22952	[object Object]	U001	2026-10-01 02:00
DEL-0004	ORD-22953	[object Object]	U001	2026-10-01 02:00
DEL-0005	PO-0071	[object Object]	U001	2026-10-01 02:01

2.17.13 ตารางประวัติการแก้ไขต้นทุน (COST_HISTORY)

ตารางที่ 2-59 Data Dictionary ของตารางประวัติการแก้ไขต้นทุน (COST_HISTORY)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	cost_history_id	รหัสบันทึก	text	PK	Not Null	
2	sku_id	สินค้าที่แก้ต้นทุน	text	FK ⟶ skus	Not Null	
3	cost_id	รายการต้นทุนที่ถูกแก้	text	FK ⟶ costs		

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
4	previous_value	ค่าก่อนแก้ (JSON) ว่างถ้าเป็นการสร้างครั้งแรก	jsonb			
5	new_value	ค่าหลังแก้ (JSON)	jsonb		Not Null	
6	admin_id	ผู้ดูแลระบบที่แก้ (4S)	text	FK ⟶ admins	Not Null	
7	changed_at	วันเวลาที่แก้	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP

ตารางที่ 2-60 ตัวอย่างข้อมูลของตารางประวัติการแก้ไขต้นทุน (COST_HISTORY) (คอลัมน์ 1-6)

cost_history_id	sku_id	cost_id	previous_value	new_value	admin_id
CH-0001	SKU-FSH-MLR-240	COST-FSH-MLR-240	[object Object]	[object Object]	U001
CH-0002	SKU-OUT-LMP-021	COST-OUT-LMP-021	NULL	[object Object]	U001
CH-0003	SKU-OUT-CHR-003	COST-OUT-CHR-003	[object Object]	[object Object]	U001
CH-0004	SKU-HOM-KIT-108	COST-HOM-KIT-108	[object Object]	[object Object]	U001
CH-0005	SKU-YDM-HRB-012	COST-YDM-HRB-012	[object Object]	[object Object]	U001

ตารางที่ 2-61 ตัวอย่างข้อมูลของตารางประวัติการแก้ไขต้นทุน (COST_HISTORY) (คอลัมน์ 7-7)

changed_at
2026-10-02 04:20
2026-10-03 05:00
2026-10-03 06:00
2026-10-04 02:10
2026-10-04 03:40

2.17.14 ตารางบันทึกการแจ้งเลขติดตาม (TRACKING_NOTIFICATION_LOGS)

ตารางที่ 2-62 Data Dictionary ของตารางบันทึกการแจ้งเลขติดตาม (TRACKING_NOTIFICATION_LOGS)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	tracking_notification_log_id	รหัสบันทึก	text	PK	Not Null	
2	order_id	Order ที่แจ้งเลขติดตาม	text	FK ⟶ orders	Not Null	
3	tracking_number	หมายเลขติดตามที่แจ้ง	text		Not Null	
4	outcome	ผลการแจ้ง (pending, sent, failed)	text		Not Null	
5	sent_at	วันเวลาที่ส่งแจ้งล่าสุด	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP
6	admin_id	ผู้ดูแลระบบที่สั่งแจ้งซ้ำ (ว่างถ้าระบบแจ้งเองใน 5S)	text	FK ⟶ admins		
7	attempt_id	รหัสความพยายามส่ง ใช้ซ้ำตอน retry เพื่อไม่ให้แจ้งซ้ำ	text		Not Null	
8	error_code	รหัสข้อผิดพลาดเมื่อแจ้งไม่สำเร็จ	text			

ตารางที่ 2-63 ตัวอย่างข้อมูลของตารางบันทึกการแจ้งเลขติดตาม (TRACKING_NOTIFICATION_LOGS) (คอลัมน์ 1-6)

tracking_notification_log_id	order_id	tracking_number	outcome	sent_at	admin_id
ORD-24172-NOTIFY-1	ORD-24172	4418-2290-1104	sent	2026-10-01 06:00	U001
ORD-24166-NOTIFY-1	ORD-24166	4418-2290-1105	sent	2026-10-01 06:00	U001
ORD-23012-NOTIFY-1	ORD-23012	4418-1001-0012	sent	2026-11-06 03:07	U001
ORD-24168-NOTIFY-1	ORD-24168	4418-2290-0168	sent	2026-11-06 03:07	U001
ORD-24165-NOTIFY-1	ORD-24165	4418-2290-1106	failed	2026-10-01 05:55	U001

ตารางที่ 2-64 ตัวอย่างข้อมูลของตารางบันทึกการแจ้งเลขติดตาม (TRACKING_NOTIFICATION_LOGS) (คอลัมน์ 7-8)

attempt_id	error_code
ORD-24172-NOTIFY-1	NULL
ORD-24166-NOTIFY-1	NULL
ORD-23012-NOTIFY-1	NULL
ORD-24168-NOTIFY-1	NULL
ORD-24165-NOTIFY-1	external_unavailable

2.17.15 ตารางสถานะซิงก์สต๊อกกับ Rakuten (RAKUTEN_STOCK_SYNC_STATES)

ตารางที่ 2-65 Data Dictionary ของตารางสถานะซิงก์สต๊อกกับ Rakuten (RAKUTEN_STOCK_SYNC_STATES)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	stock_id	รายการสต๊อกที่ซิงก์ (เป็นทั้ง PK และ FK)	text	PK, FK ⟶ stock	Not Null	
2	sync_status	สถานะการซิงก์ (pending, synced)	text		Not Null	sync_status IN 'pending', 'synced'
3	last_synced_qty	จำนวนที่ส่งไป Rakuten สำเร็จครั้งล่าสุด	INTEGER			(last_synced_qty IS NULL) OR (last_synced_qty >= 0)
4	last_attempt_at	วันเวลาที่พยายามซิงก์ล่าสุด	TIMESTAMPTZ			
5	last_synced_at	วันเวลาที่ซิงก์สำเร็จล่าสุด	TIMESTAMPTZ			
6	last_error_code	รหัสข้อผิดพลาดล่าสุดตอนซิงก์ (7S)	text			

ตารางที่ 2-66 ตัวอย่างข้อมูลของตารางสถานะซิงก์สต๊อกกับ Rakuten (RAKUTEN_STOCK_SYNC_STATES)

stock_id	sync_status	last_synced_qty	last_attempt_at	last_synced_at	last_error_code
STK-FSH-MLR-240	synced	148	NULL	2026-11-06 03:07	NULL
STK-YDM-HRB-012	synced	118	NULL	2026-11-06 03:07	NULL
STK-OUT-CHR-003	synced	14	NULL	2026-11-06 03:07	NULL
STK-HOM-STG-045	synced	1	NULL	2026-11-06 03:07	NULL
STK-FSH-RDS-180	synced	1	NULL	2026-11-06 03:07	NULL

2.17.16 ตารางประวัติการปรับสต๊อก (STOCK_ADJUSTMENT_HISTORY)

ตารางที่ 2-67 Data Dictionary ของตารางประวัติการปรับสต๊อก (STOCK_ADJUSTMENT_HISTORY)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	stock_adjustment_history_id	รหัสบันทึก	text	PK	Not Null	
2	stock_id	รายการสต๊อกที่ปรับ	text	FK ⟶ stock	Not Null	
3	previous_in_house_qty	จำนวนในคลังบริษัทก่อนปรับ	INTEGER		Not Null	previous_in_house_qty >= 0
4	new_in_house_qty	จำนวนในคลังบริษัทหลังปรับ	INTEGER		Not Null	new_in_house_qty >= 0
5	previous_rsl_qty	จำนวนในคลัง RSL ก่อนปรับ	INTEGER			(previous_rsl_qty IS NULL) OR (previous_rsl_qty >= 0)
6	new_rsl_qty	จำนวนในคลัง RSL หลังปรับ	INTEGER			(new_rsl_qty IS NULL) OR (new_rsl_qty >= 0)
7	reason	เหตุผลที่ปรับสต๊อก	text		Not Null	
8	admin_id	ผู้ดูแลระบบที่ปรับ	text	FK ⟶ admins	Not Null	
9	adjusted_at	วันเวลาที่ปรับ	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP

ตารางที่ 2-68 ตัวอย่างข้อมูลของตารางประวัติการปรับสต๊อก (STOCK_ADJUSTMENT_HISTORY) (คอลัมน์ 1-6)

stock_adjustment_history_id	stock_id	previous_in_house_qty	new_in_house_qty	previous_rsl_qty	new_rsl_qty
SAH-0001	STK-OUT-CHR-003	4	6	8	8
SAH-0002	STK-HOM-STG-045	3	1	0	0
SAH-0003	STK-FSH-MLR-240	54	52	96	96
SAH-0004	STK-YDM-HRB-012	120	118	NULL	NULL
SAH-0005	STK-HOM-KIT-108	0	2	3	3

ตารางที่ 2-69 ตัวอย่างข้อมูลของตารางประวัติการปรับสต๊อก (STOCK_ADJUSTMENT_HISTORY) (คอลัมน์ 7-9)

reason	admin_id	adjusted_at
นับสต๊อกจริงพบเพิ่ม 2 ตัว	U001	2026-10-04 06:00
สินค้าชำรุด 2 ชิ้น	U001	2026-10-04 06:10
ส่งตัวอย่างให้ลูกค้า 2 ชิ้น	U001	2026-10-04 06:20
นับสต๊อกจริงพบขาด 2 ชิ้น	U001	2026-10-04 06:30
รับคืนสินค้าจากลูกค้า 2 ชิ้น	U001	2026-10-04 06:40

2.17.17 ตารางประวัติการพิมพ์ใบปะ (LABEL_PRINT_HISTORY)

ตารางที่ 2-70 Data Dictionary ของตารางประวัติการพิมพ์ใบปะ (LABEL_PRINT_HISTORY)

No.	Attribute	Description	Data Type	Key	Not Null	ช่วงข้อมูล / การตรวจสอบ
1	label_print_history_id	รหัสบันทึก	text	PK	Not Null	
2	print_attempt_id	รหัสการสั่งพิมพ์หนึ่งครั้ง (หนึ่งครั้งพิมพ์ได้หลายใบ)	text		Not Null	
3	label_id	ใบปะที่พิมพ์	text	FK ⟶ labels	Not Null	
4	admin_id	ผู้ดูแลระบบที่สั่งพิมพ์ (9A)	text	FK ⟶ admins	Not Null	
5	is_reprint	เป็นการพิมพ์ซ้ำหรือไม่	BOOLEAN		Not Null	ค่าเริ่มต้น false
6	outcome	ผลการพิมพ์ (printed, failed)	text		Not Null	outcome IN 'printed', 'failed'
7	error_code	รหัสข้อผิดพลาดเมื่อพิมพ์ไม่สำเร็จ	text			
8	attempted_at	วันเวลาที่สั่งพิมพ์	TIMESTAMPTZ		Not Null	ค่าเริ่มต้น CURRENT_TIMESTAMP

ตารางที่ 2-71 ตัวอย่างข้อมูลของตารางประวัติการพิมพ์ใบปะ (LABEL_PRINT_HISTORY) (คอลัมน์ 1-6)

label_print_history_id	print_attempt_id	label_id	admin_id	is_reprint	outcome
LPH-0001	PRINT-0001	ORD-24178-LABEL-1	U001	false	failed
LPH-0002	PRINT-0002	ORD-24178-LABEL-1	U001	false	printed
LPH-0003	PRINT-0003	ORD-24177-LABEL-1	U001	false	printed
LPH-0004	PRINT-0004	ORD-24177-LABEL-1	U001	true	printed
LPH-0005	PRINT-0005	ORD-24173-LABEL-1	U001	false	printed

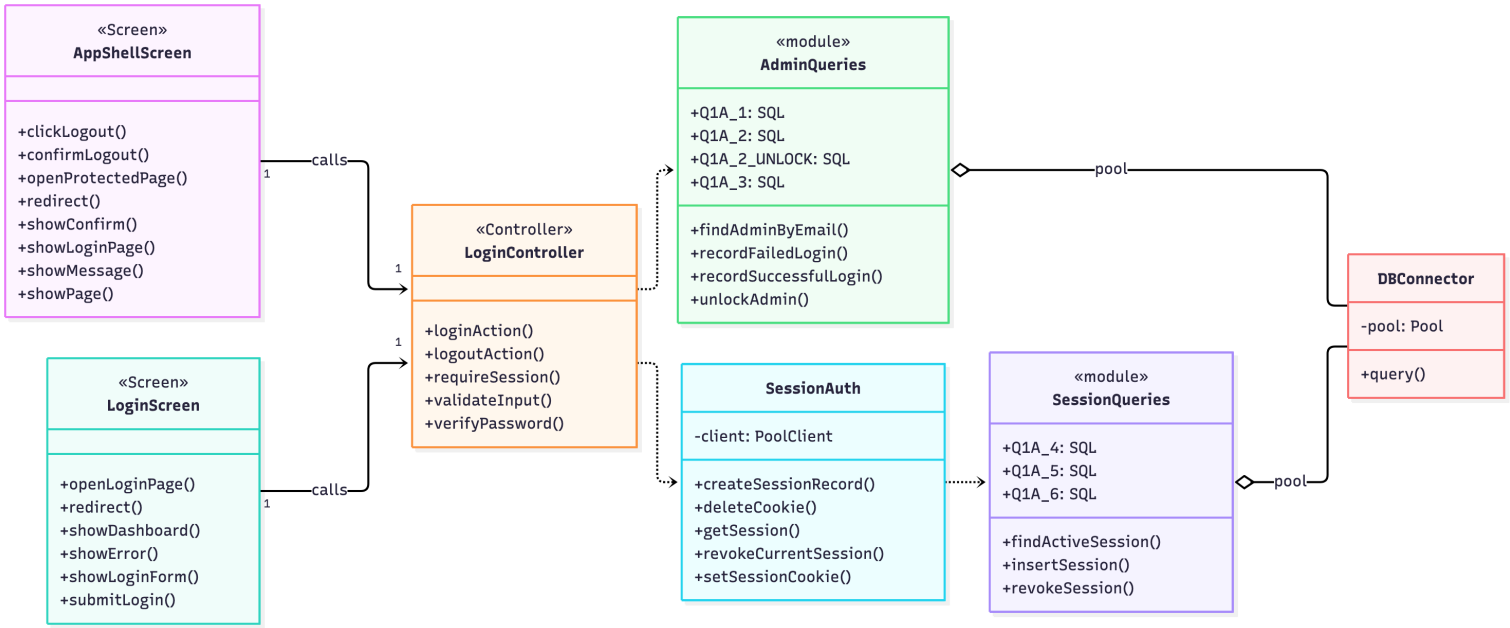
ตารางที่ 2-72 ตัวอย่างข้อมูลของตารางประวัติการพิมพ์ใบปะ (LABEL_PRINT_HISTORY) (คอลัมน์ 7-8)

error_code	attempted_at
external_unavailable	2026-10-04 01:25
NULL	2026-10-04 01:30
NULL	2026-10-04 01:31
NULL	2026-10-04 01:40
NULL	2026-10-04 01:45

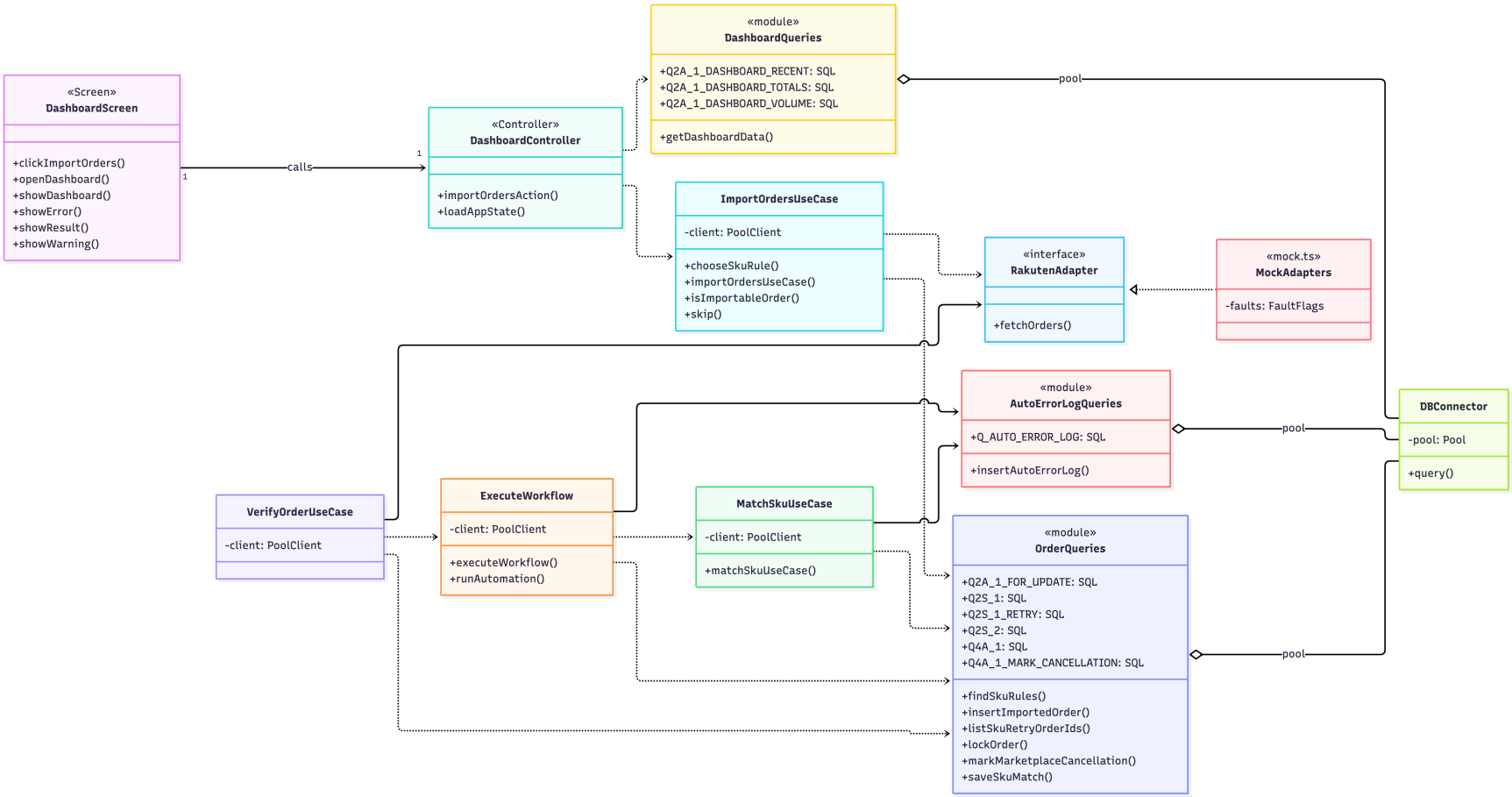
2.18 Class Diagram

Class Diagram ได้จากการนำ object ทุกตัวใน Sequence Diagram และ Collaboration Diagram ของทั้ง 17 Use Case มาทำ abstraction เป็นคลาส ตรงกับโครงสร้างโค้ดจริงของเว็บ แบ่งเป็นชั้นตามแบบ MVC ได้แก่ Screen (หน้าเว็บ page.tsx ของ Next.js ซึ่งเป็นชั้น View ของ MVC), Controller (Server Action ใน actions.ts และ login/actions.ts), Model คือ UseCase (ไฟล์ใน src/lib/use-cases รวม ExecuteWorkflow ที่รัน Use Case อัตโนมัติ), Queries (โมดูลใน src/lib/queries ที่เก็บ SQL เป็นค่าคงที่ชื่อ Q<UC>_n), DBConnector (db.ts) และ Adapter ของระบบภายนอก ส่วนคลาสข้อมูล (Entity) ตรงกับตารางในฐานข้อมูล

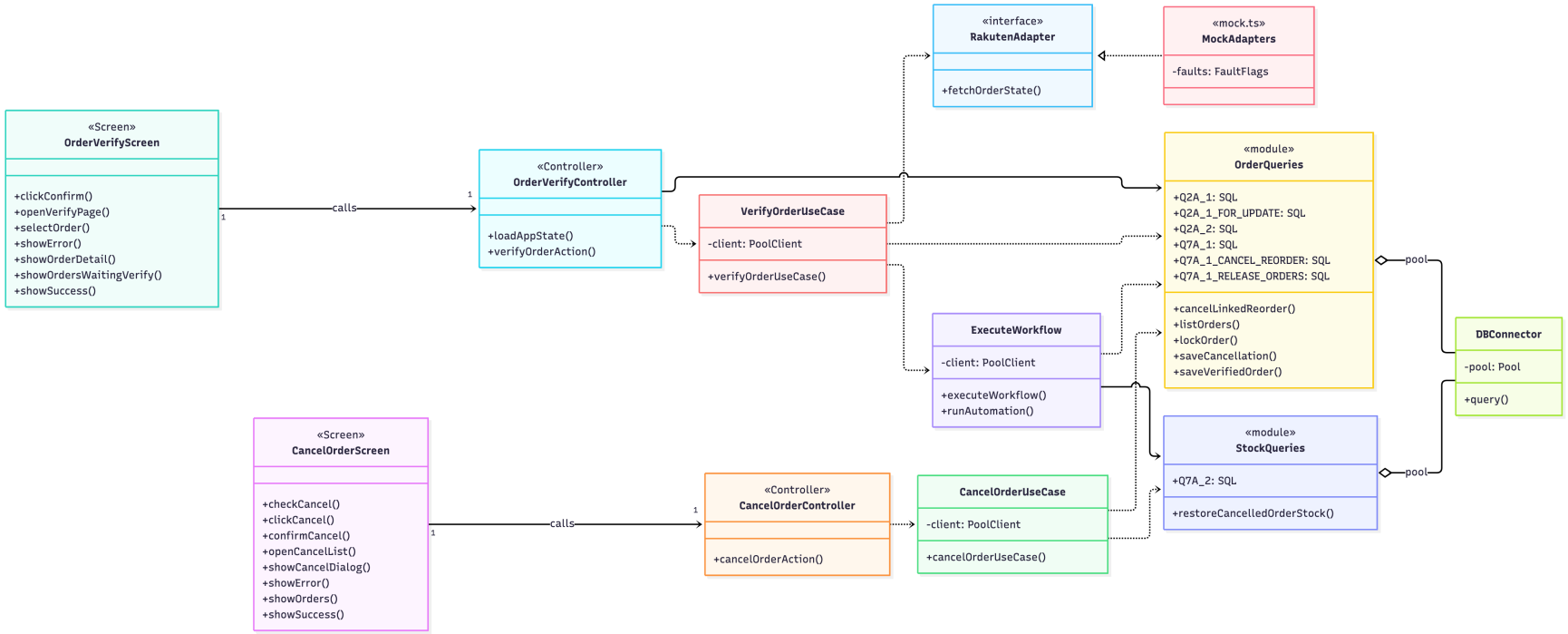
ความสัมพันธ์ที่ใช้ ได้แก่ association (Screen เรียก Controller), dependency (Controller เรียก UseCase และ UseCase เรียก Queries และ Adapter), aggregation (Queries ทุกโมดูลใช้ pool ของ DBConnector ตัวเดียวกัน) และ realization (MockAdapters ใน mock.ts implement interface ของ Adapter ทุกตัว) Screen และ Controller เป็นชื่อบทบาท (stereotype) ของไฟล์จริง โค้ดไม่มีคลาส Controller, Repository หรือ Scheduler แยก



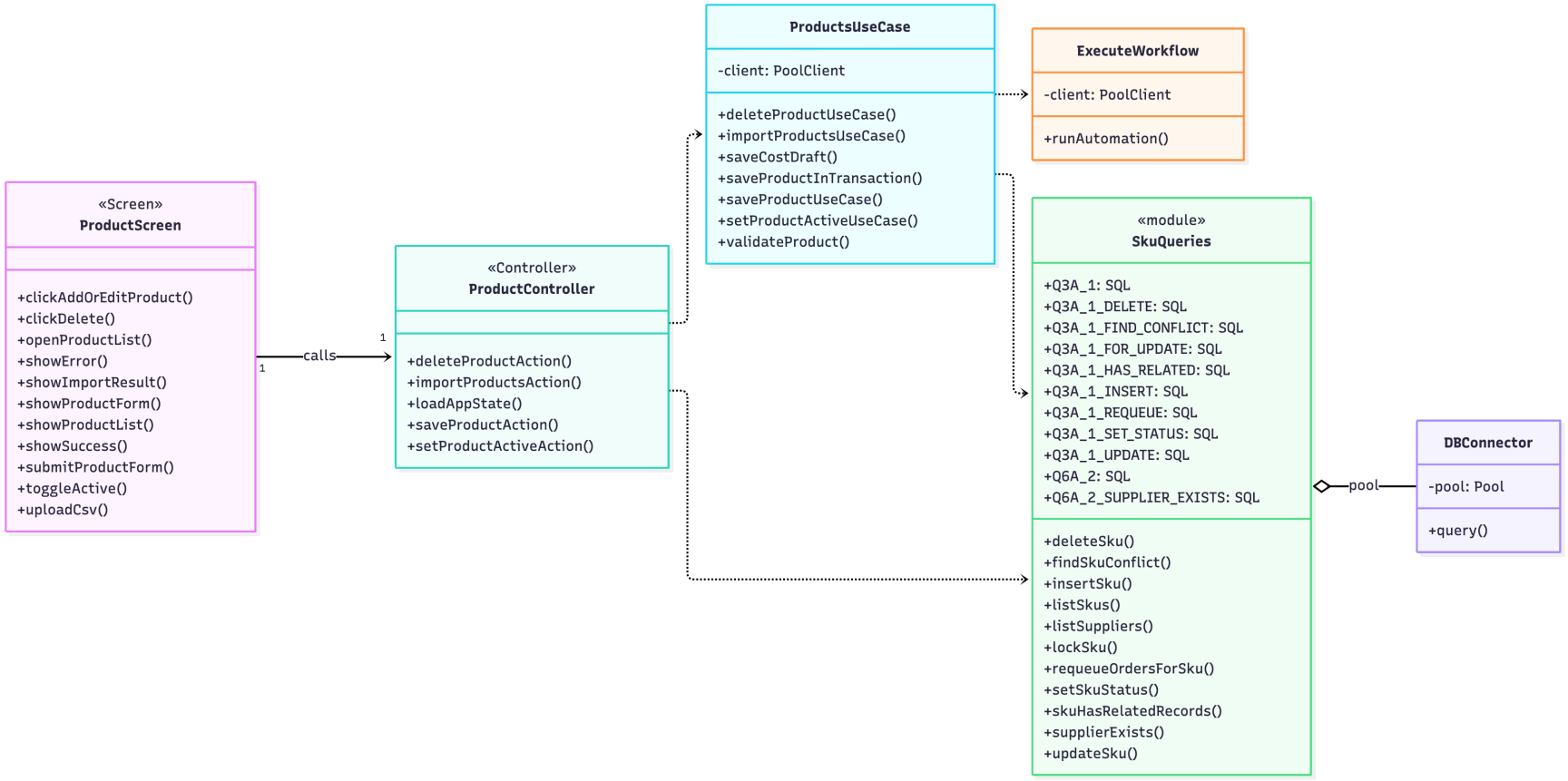
รูปที่ 2-94 Class Diagram กลุ่มเข้าสู่ระบบ (1/15)



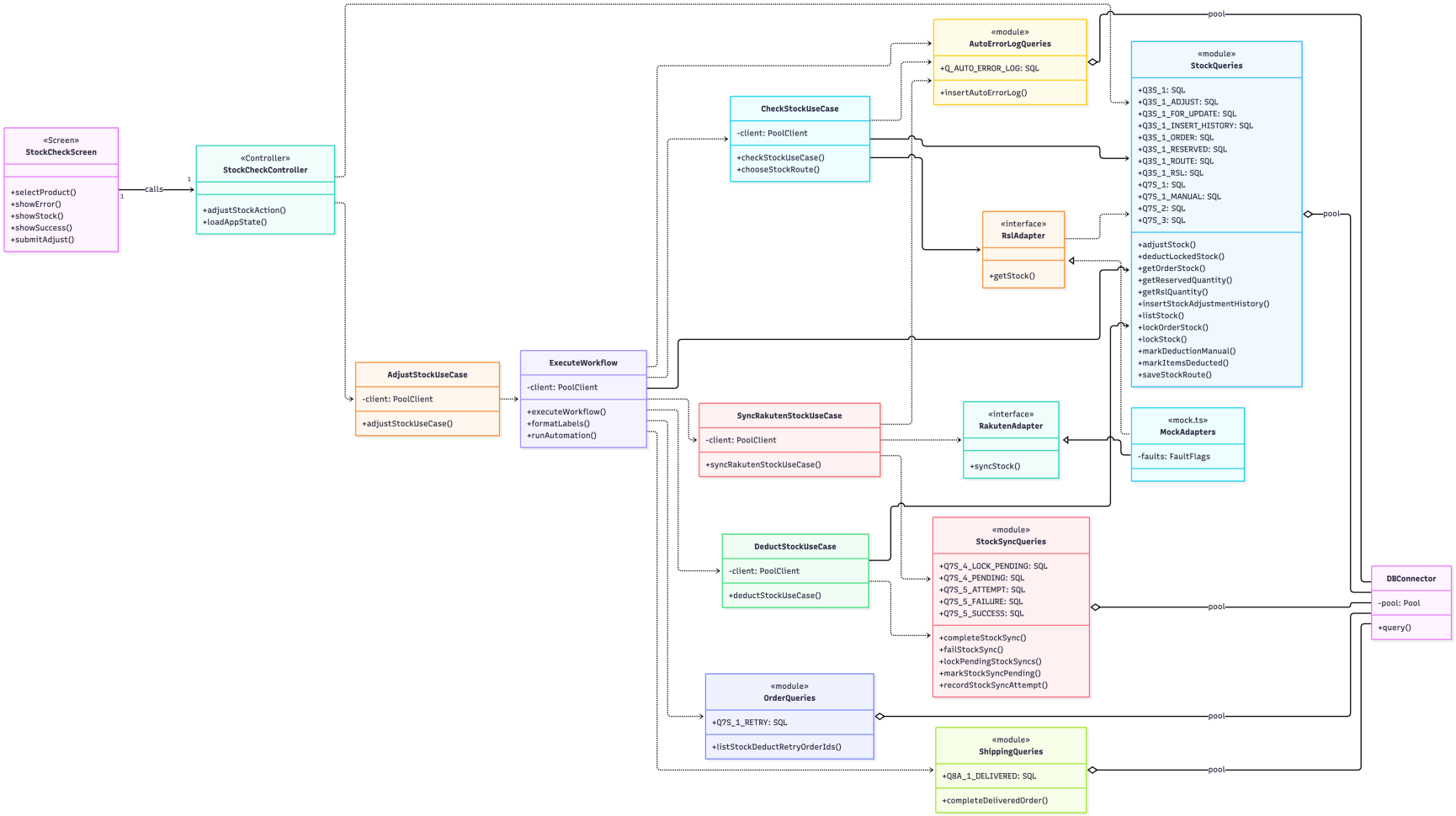
รูปที่ 2-95 Class Diagram กลุ่มรับคำสั่งซื้อและจับคู่ SKU (2/15)



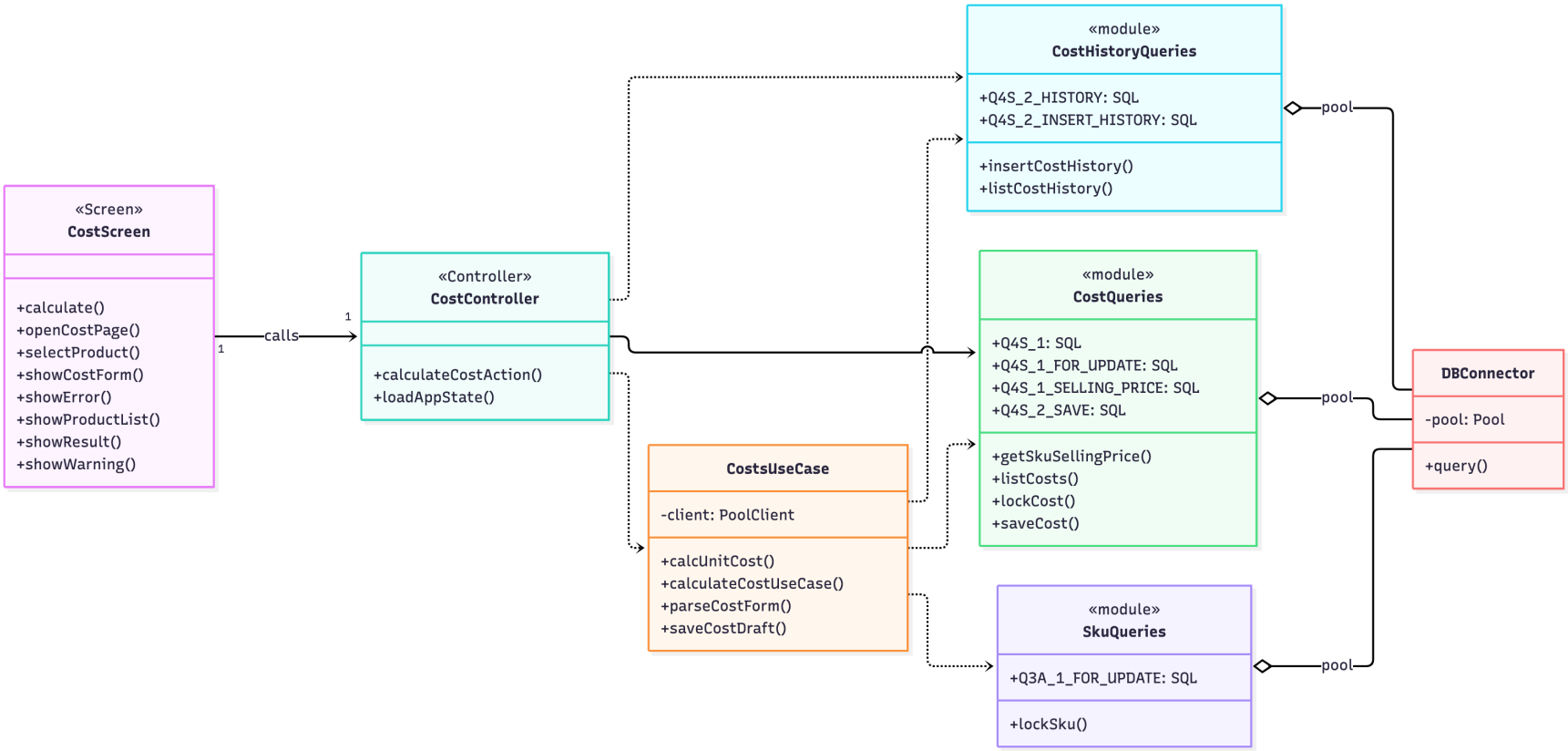
รูปที่ 2-96 Class Diagram กลุ่มตรวจสอบและยกเลิกคำสั่งซื้อ (3/15)



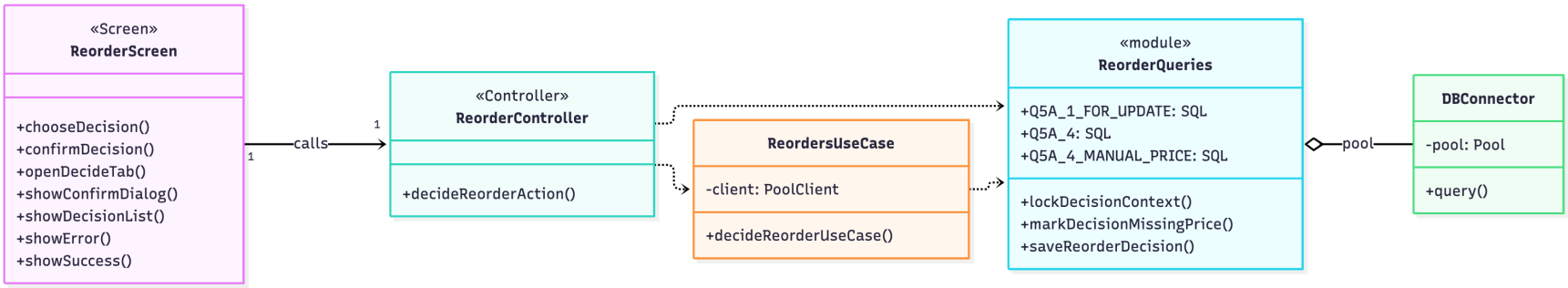
รูปที่ 2-97 Class Diagram กลุ่มจัดการสินค้า (4/15)



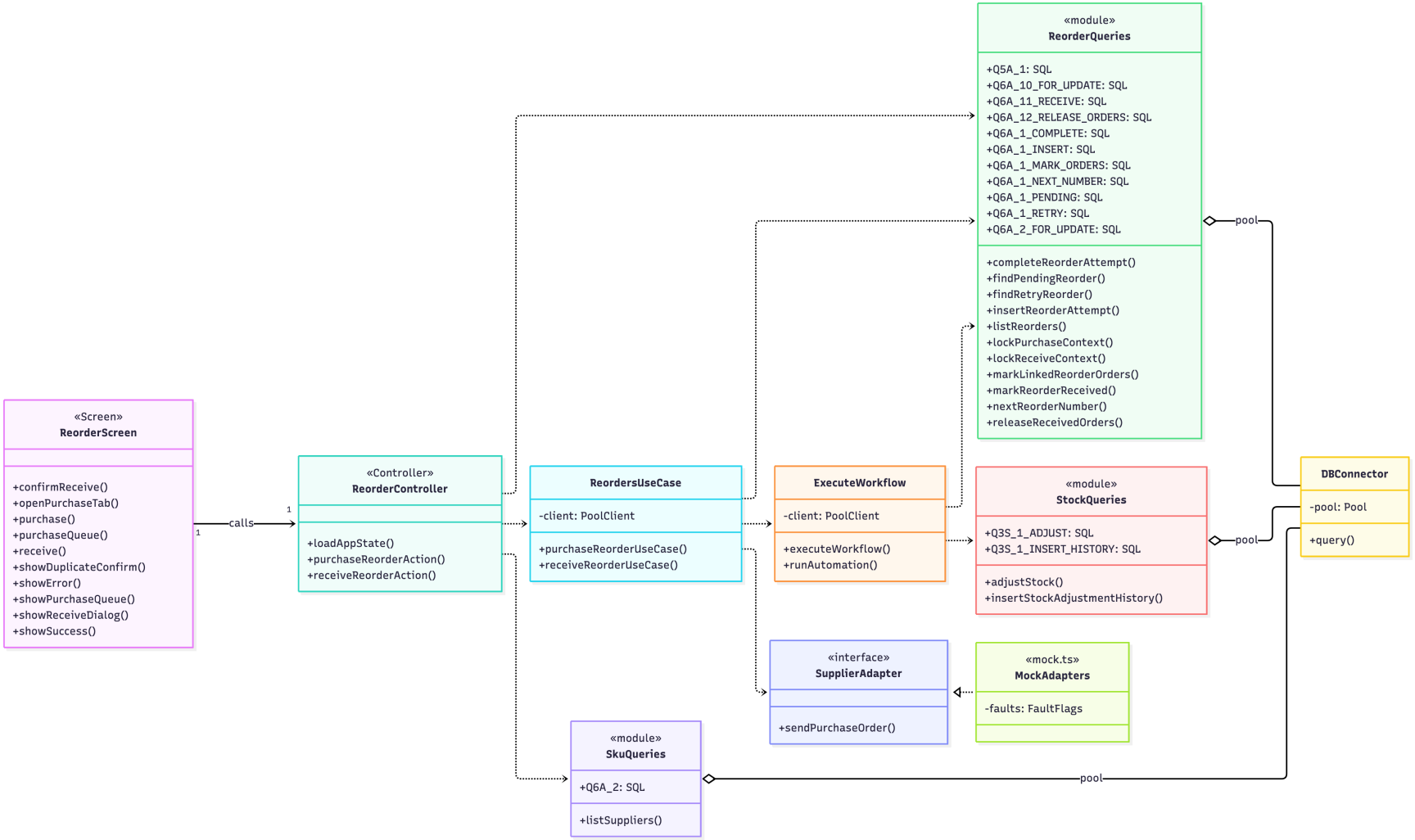
รูปที่ 2-98 Class Diagram กลุ่มตรวจและตัดสต๊อก (5/15)



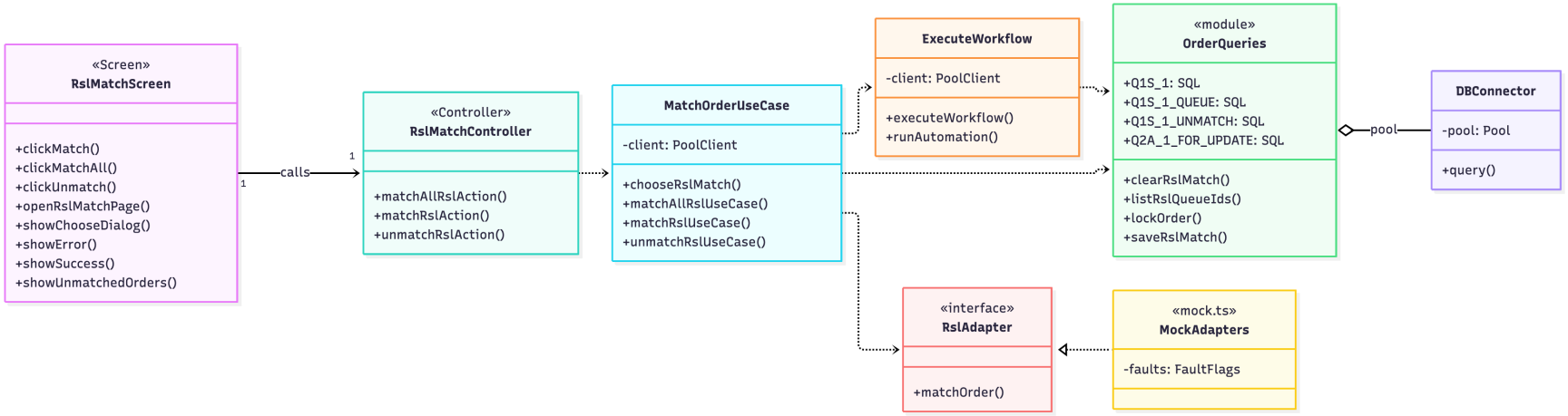
รูปที่ 2-99 Class Diagram กลุ่มคำนวณต้นทุน (6/15)



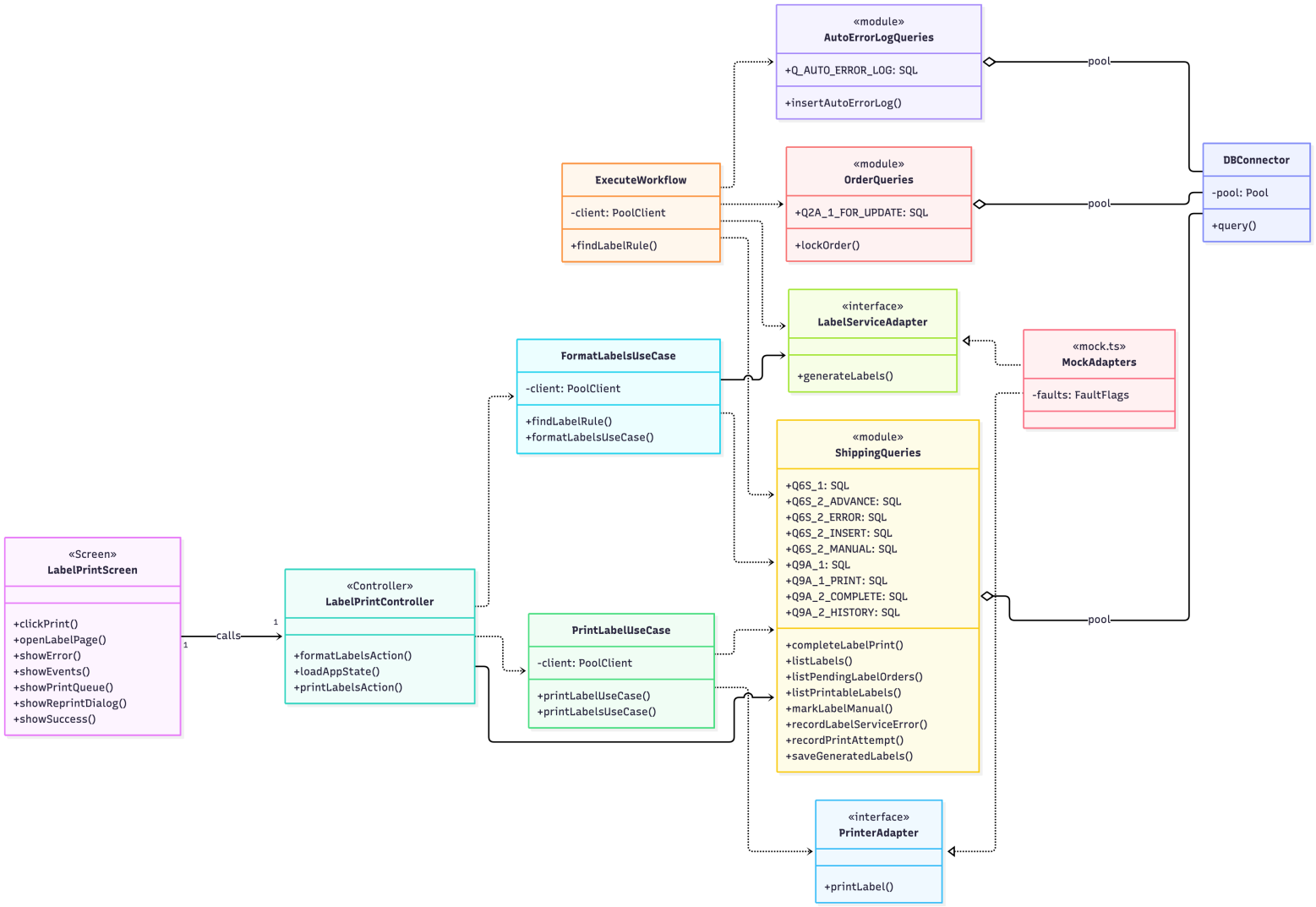
รูปที่ 2-100 Class Diagram กลุ่มตัดสินใจสั่งซื้อเติม stock (7/15)



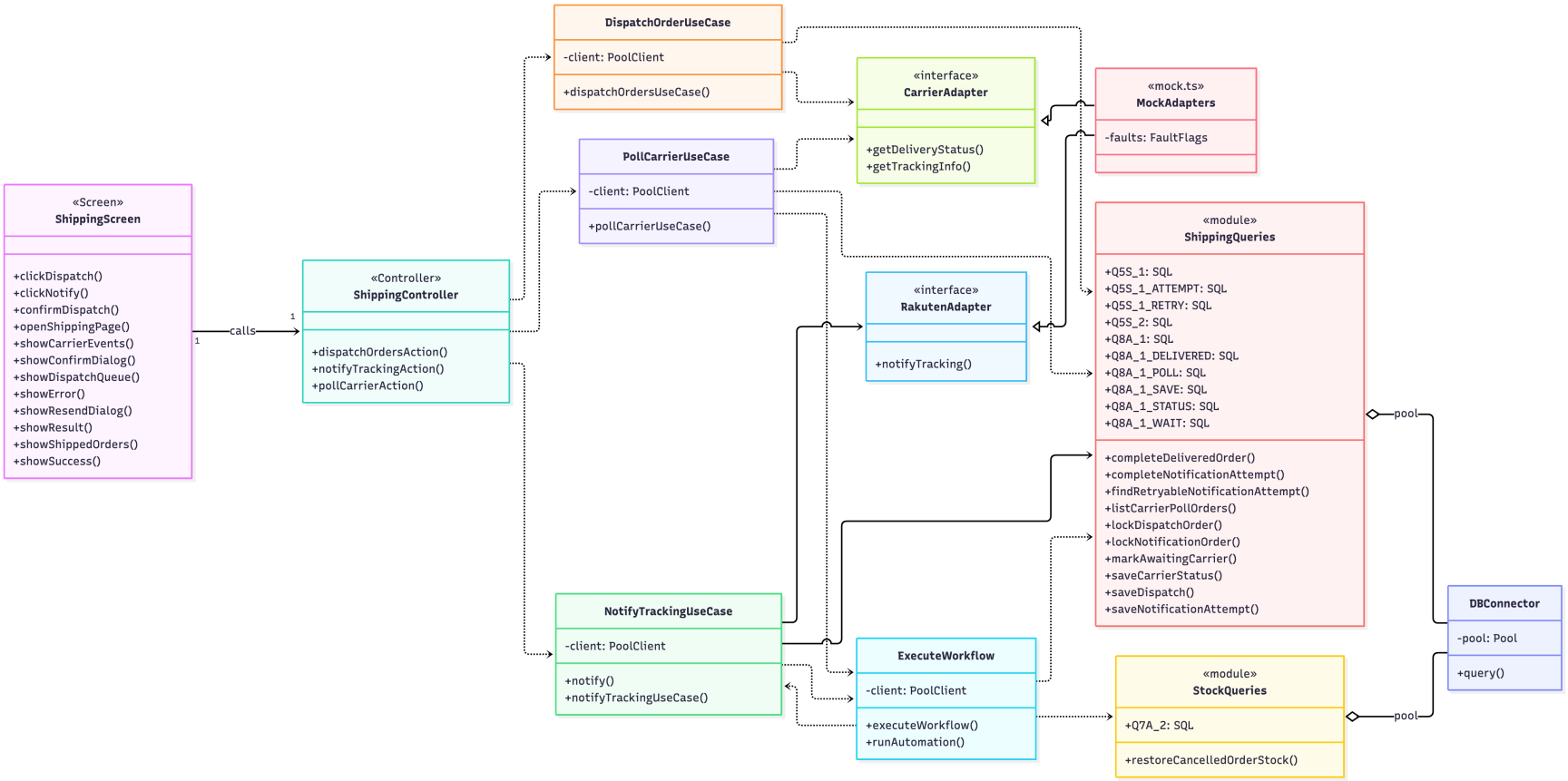
รูปที่ 2-101 Class Diagram กลุ่มสั่งซื้อเติม stock จาก supplier (8/15)



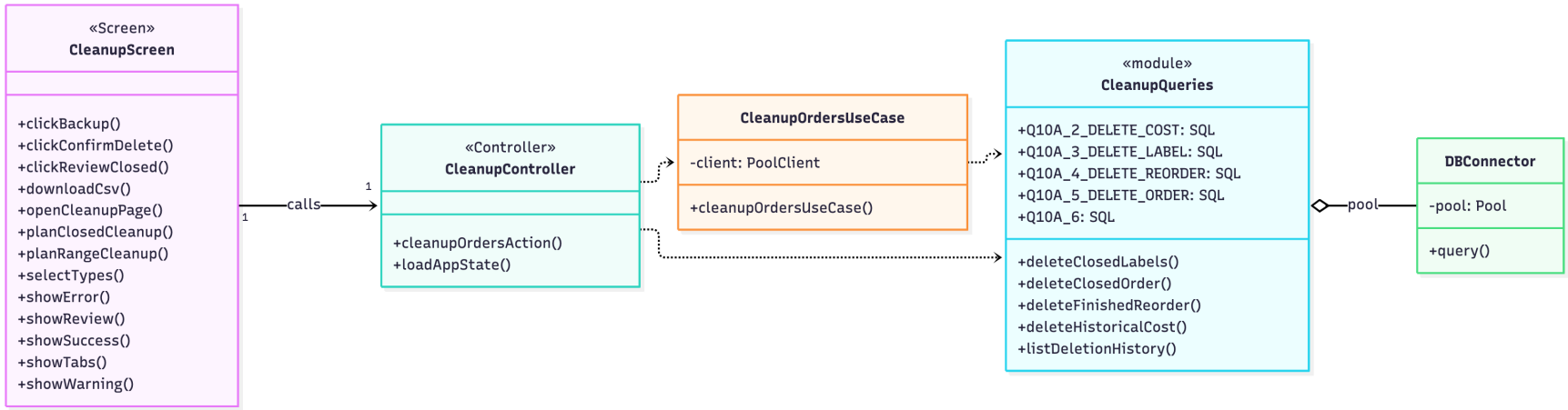
รูปที่ 2-102 Class Diagram กลุ่มส่ง Order เข้า RSL (9/15)



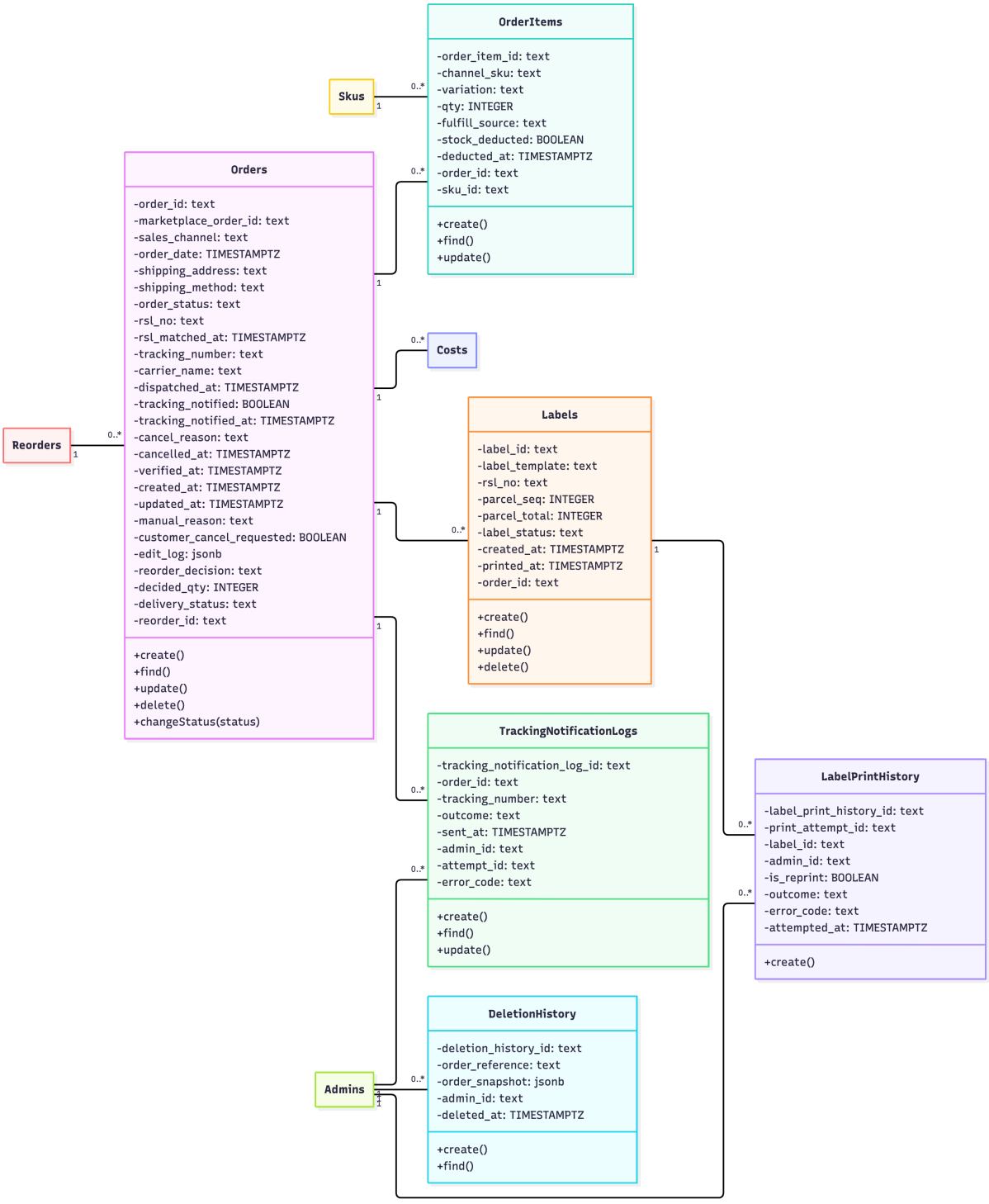
รูปที่ 2-103 Class Diagram กลุ่มจัดการใบปะ (10/15)



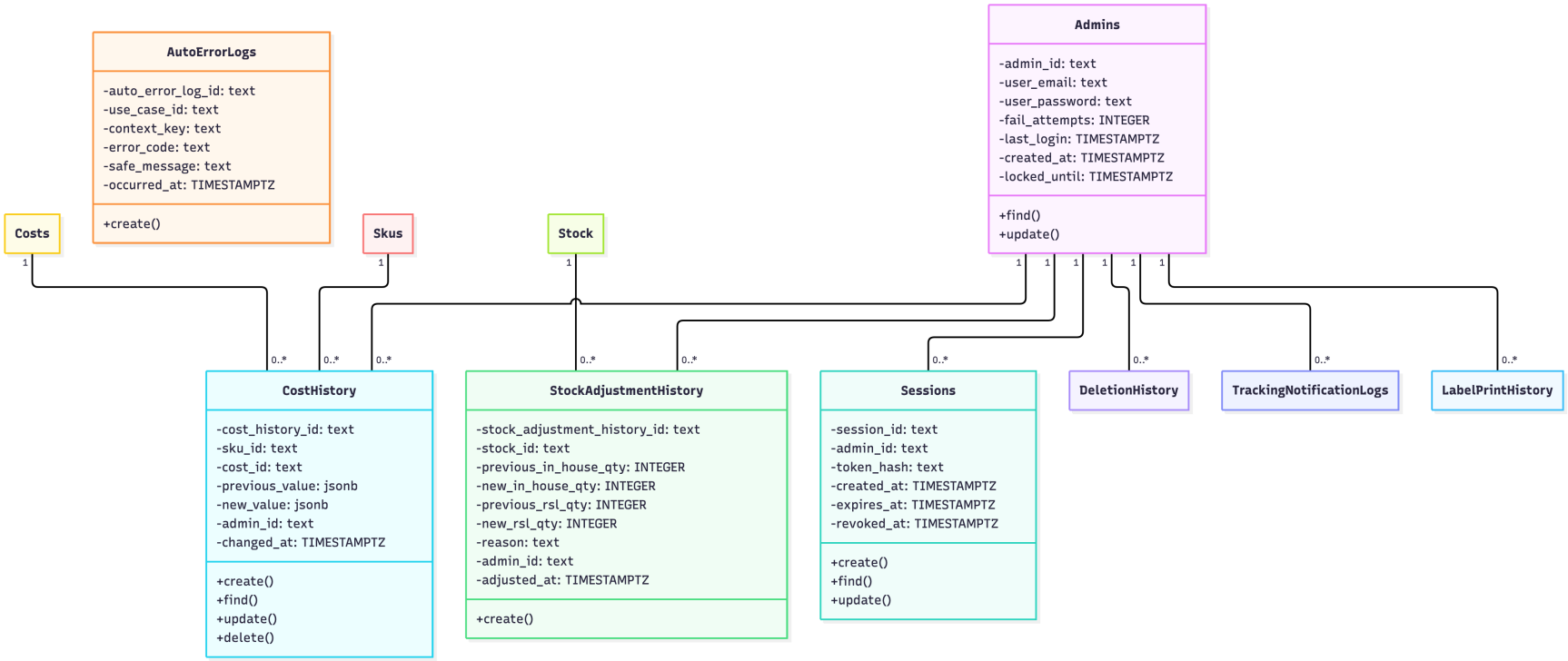
รูปที่ 2-104 Class Diagram กลุ่มจัดส่งและแจ้งเลขติดตาม (11/15)



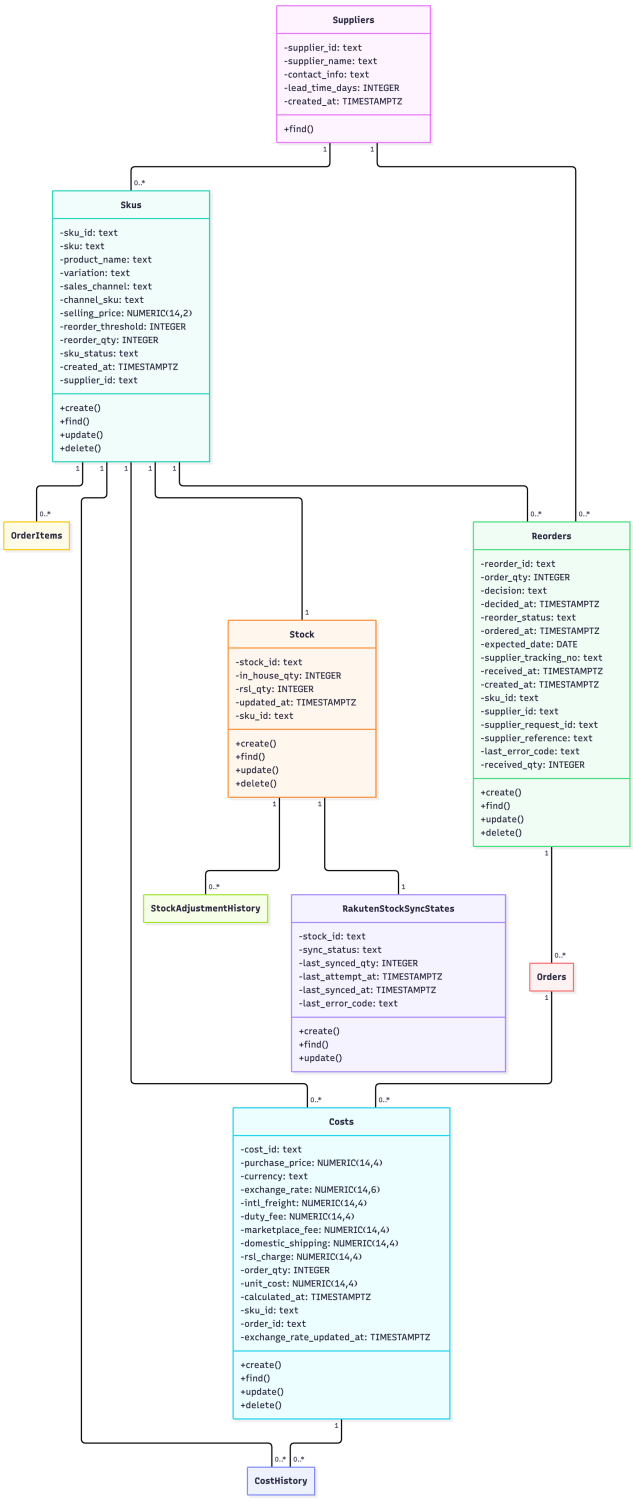
รูปที่ 2-105 Class Diagram กลุ่มลบข้อมูลเก่า (12/15)



รูปที่ 2-106 Class Diagram กลุ่มคลาสข้อมูลคำสั่งซื้อและการจัดส่ง (Entity) (13/15)



รูปที่ 2-107 Class Diagram กลุ่มคลาสข้อมูลผู้ดูแลระบบและประวัติ (Entity) (14/15)



รูปที่ 2-108 Class Diagram กลุ่มคลาสข้อมูลสินค้าและสต๊อก (Entity) (15/15)

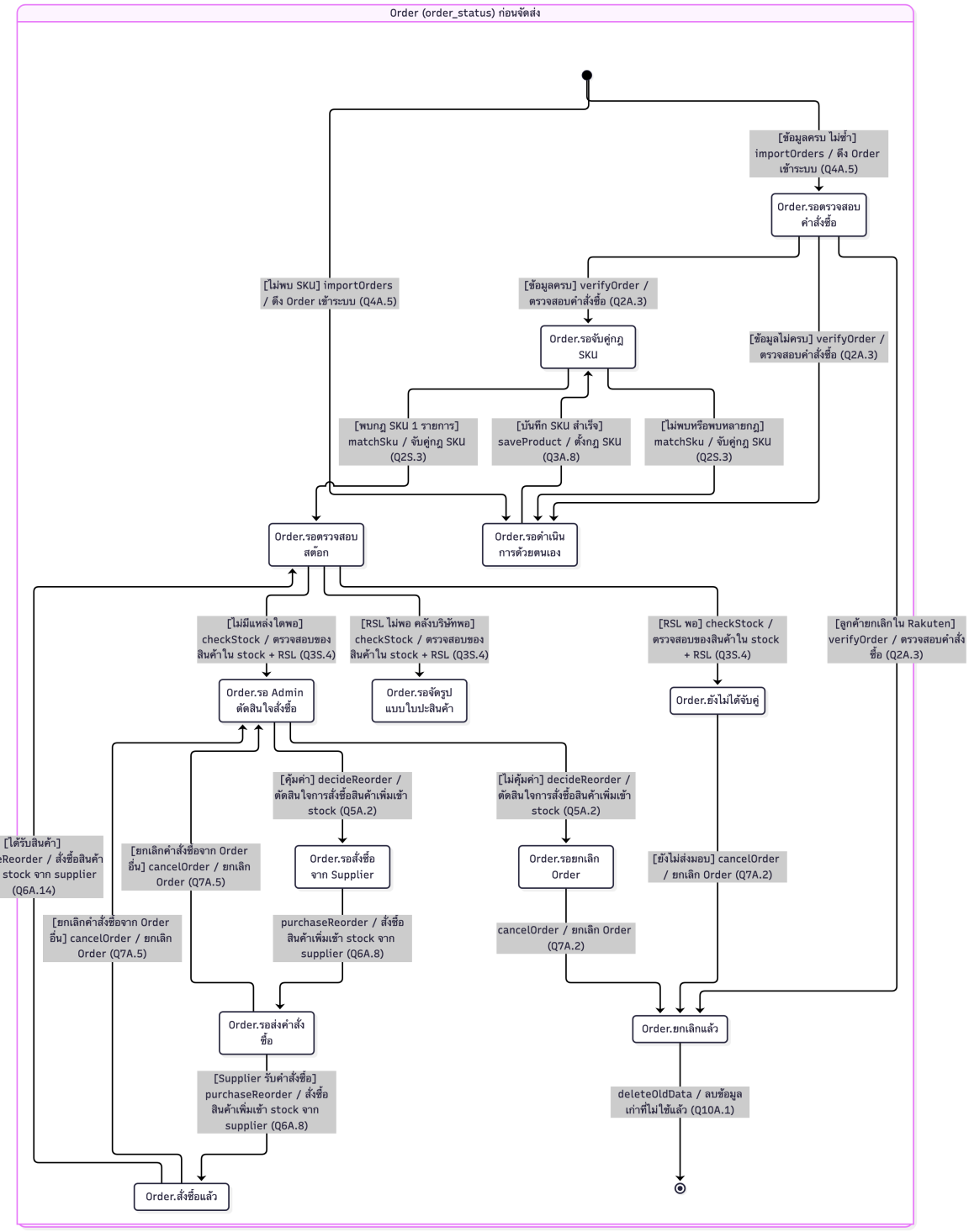
2.19 State Diagram

State Diagram แสดงสถานะของ Order.order_status ตั้งแต่ Order ถูกดึงเข้าระบบจนจัดส่งสำเร็จหรือถูกยกเลิก ทุกเส้นเขียนในรูปแบบ [เงื่อนไข] event / Use Case และทุกครั้งที่สถานะเปลี่ยน ระบบบันทึกลงฐานข้อมูลด้วยคำสั่ง UPDATE ตามเลข query ที่กำกับบนเส้น

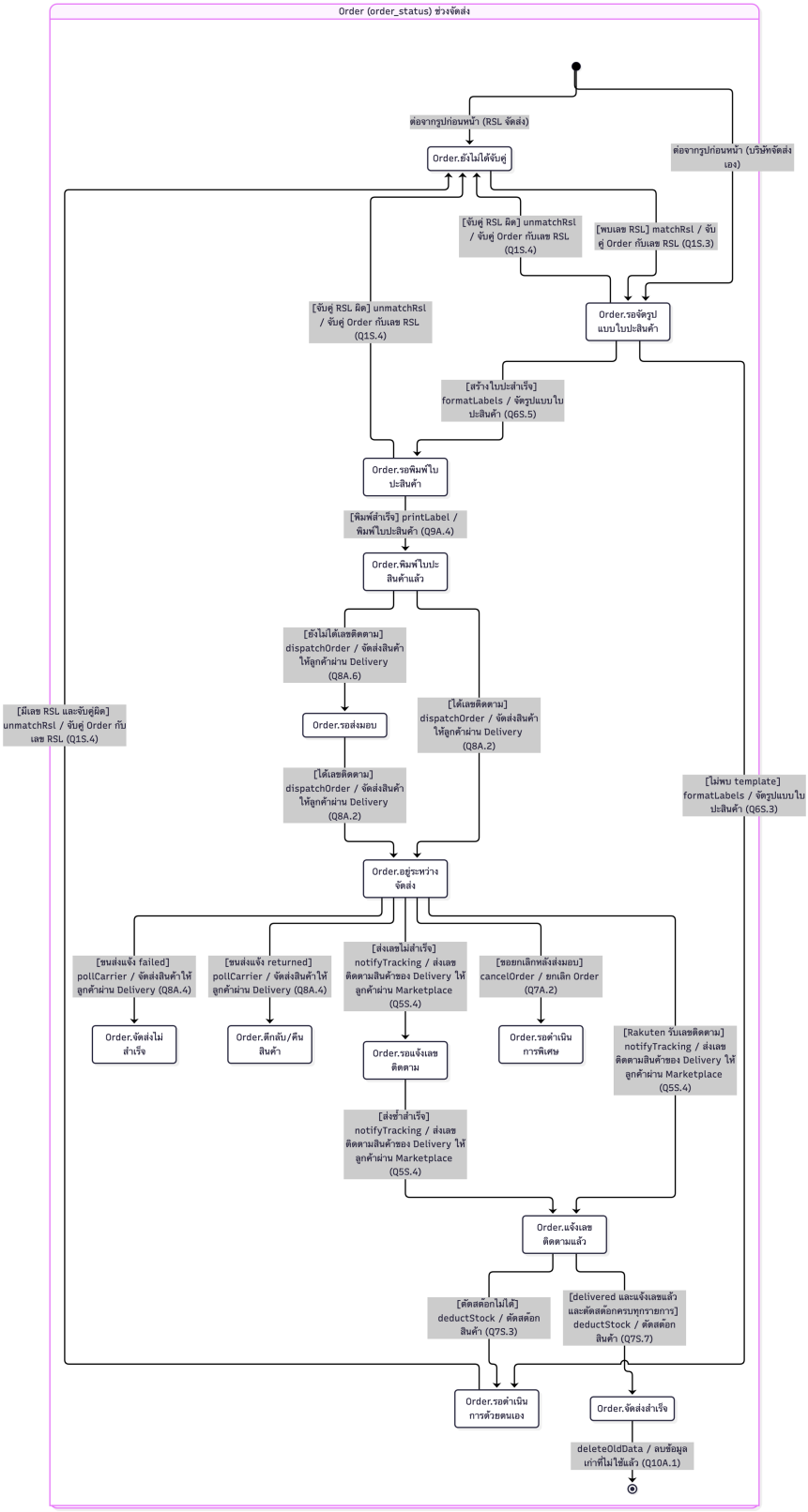
Order เป็น จัดส่งสำเร็จ ได้เมื่อครบ 3 เงื่อนไขเท่านั้น คือ ขนส่งแจ้ง delivered, แจ้งเลขติดตามให้ Rakuten แล้ว และตัดสต๊อกครบทุกรายการแล้ว คำสั่งที่ตรวจเงื่อนไขนี้มี 3 ข้อ คือ Q7S.7 (หลังตัดสต๊อก), Q5S.4 (หลังแจ้งเลขติดตาม) และ Q8A.4 (หลังดึงสถานะขนส่ง) รูปแสดงเส้นหลักของ 7S สถานะ รอดำเนินการด้วยตนเอง มี manual_reason บอกสาเหตุ (เช่น incomplete, sku-rule, sku-unregistered, price, rsl, template, stock-deduct) และกลับเข้ากระบวนการเมื่อ Admin แก้ข้อมูลแล้ว

Order ที่ สั่งซื้อแล้ว ออกจากสถานะนี้ได้ 2 ทาง คือ Admin บันทึกรับสินค้าใน 6A (Q6A.14) ทำให้ Order กลับไป รอตรวจสอบสต๊อก แล้ว 3S ตรวจสต๊อกใหม่ทันที หรือ Admin ยกเลิก Order อื่นที่ผูกกับคำสั่งซื้อเดียวกันพร้อมยกเลิกคำสั่งซื้อใน 7A (Q7A.5) ทำให้ Order กลับไป รอ Admin ตัดสินใจสั่งซื้อ คำสั่งซื้อเติมสต๊อกมีสถานะของตัวเอง (reorders.reorder_status) คือ รอส่งคำสั่งซื้อ, สั่งซื้อแล้ว, ได้รับสินค้าแล้ว และ ยกเลิกแล้ว (แสดงในรูปถัดไป)

นอกจาก order_status ยังมีคอลัมน์สถานะอีก 5 คอลัมน์ที่โค้ดอัปเดต จึงมี State Diagram แยกตามคอลัมน์ ใช้รูปแบบเดียวกัน คือ [เงื่อนไข] event / Use Case (เลข query)

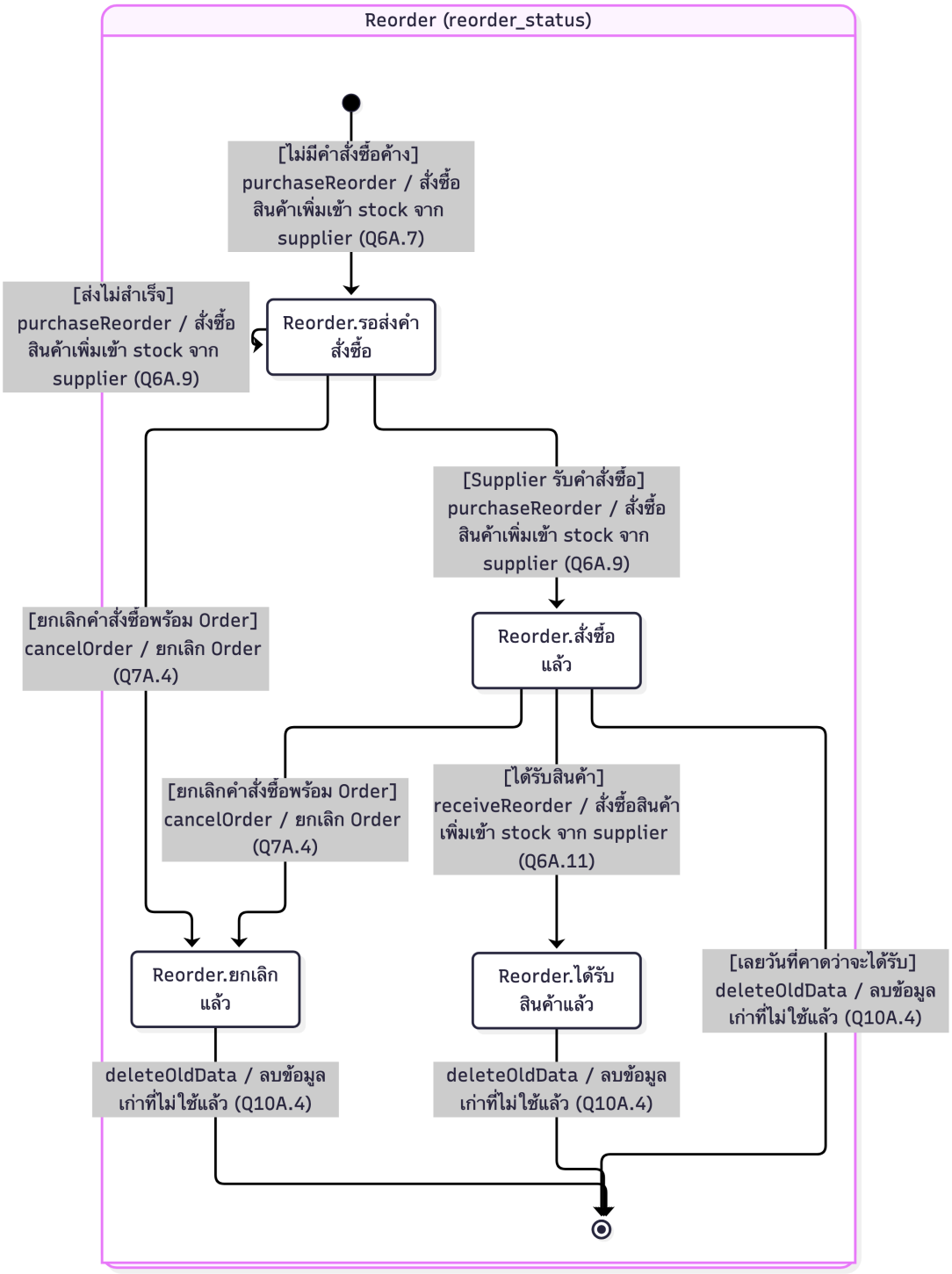


รูปที่ 2-109 State Diagram ของ Order (order_status) ก่อนจัดส่ง (1/2)



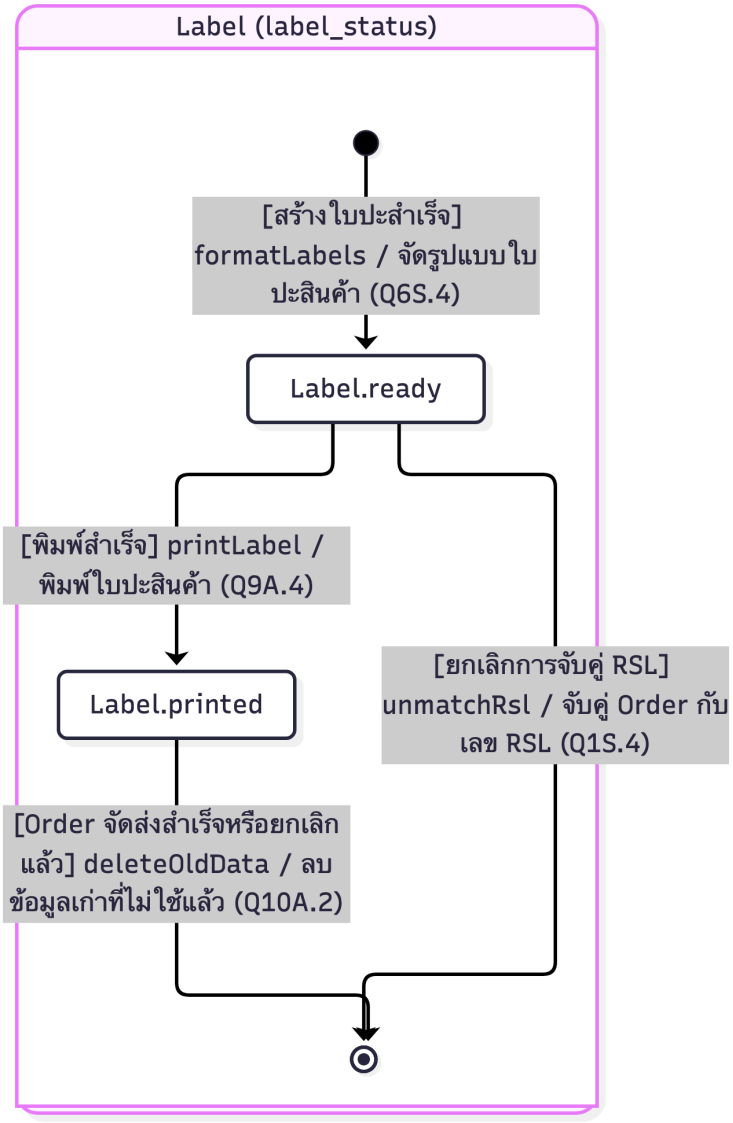
รูปที่ 2-110 State Diagram ของ Order (order_status) ช่วงจัดส่ง (2/2)

คำสั่งซื้อเติมสต๊อกถูกสร้างเป็น รอส่งคำสั่งซื้อ (Q6A.7) ก่อนส่งให้ Supplier หาก Supplier รับคำสั่งซื้อจะเป็น สั่งซื้อแล้ว แต่ถ้าส่งไม่สำเร็จจะคงสถานะเดิมเพื่อส่งซ้ำด้วย request id เดิม (Q6A.9) เมื่อรับสินค้าเป็น ได้รับสินค้าแล้ว (Q6A.11) และถ้า Admin ยกเลิก Order พร้อมยกเลิกคำสั่งซื้อจะเป็น ยกเลิกแล้ว (Q7A.4) ข้อมูลที่จบแล้วหรือเลยวันที่คาดว่าจะได้รับถูกลบได้ใน 10A (Q10A.4)



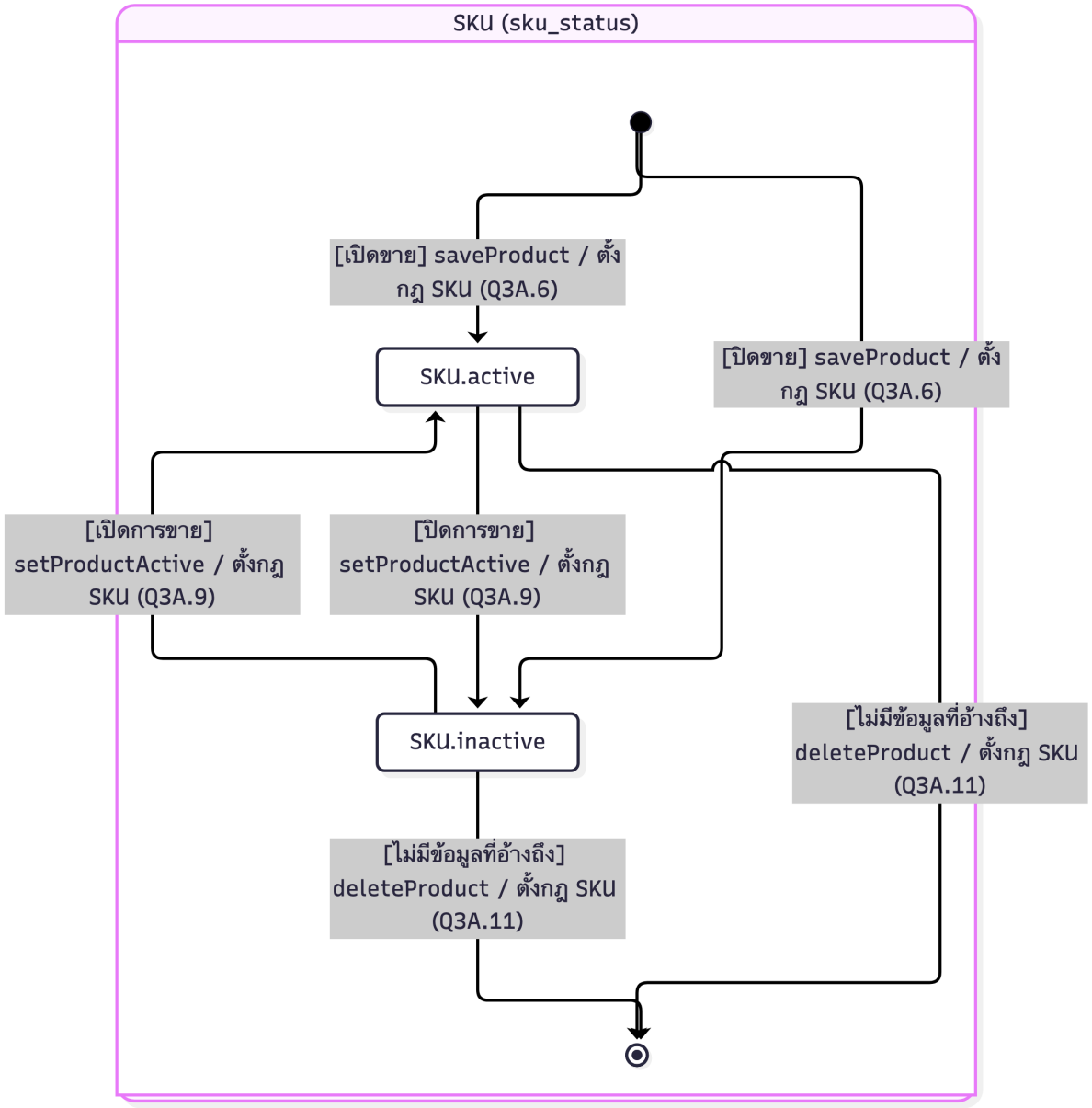
รูปที่ 2-111 State Diagram ของ Reorder (reorder_status)

ใบปะสินค้าถูกสร้างเป็น ready ใน 6S (Q6S.4) และเป็น printed เมื่อพิมพ์สำเร็จใน 9A (Q9A.4) การพิมพ์ซ้ำไม่เปลี่ยนสถานะ แต่บันทึกลง label_print_history ใบปะที่ยังไม่พิมพ์ถูกลบเมื่อยกเลิกการจับคู่ RSL (Q1S.4) เพื่อสร้างใหม่ และใบปะของ Order ที่จบแล้วถูกลบใน 10A (Q10A.2)



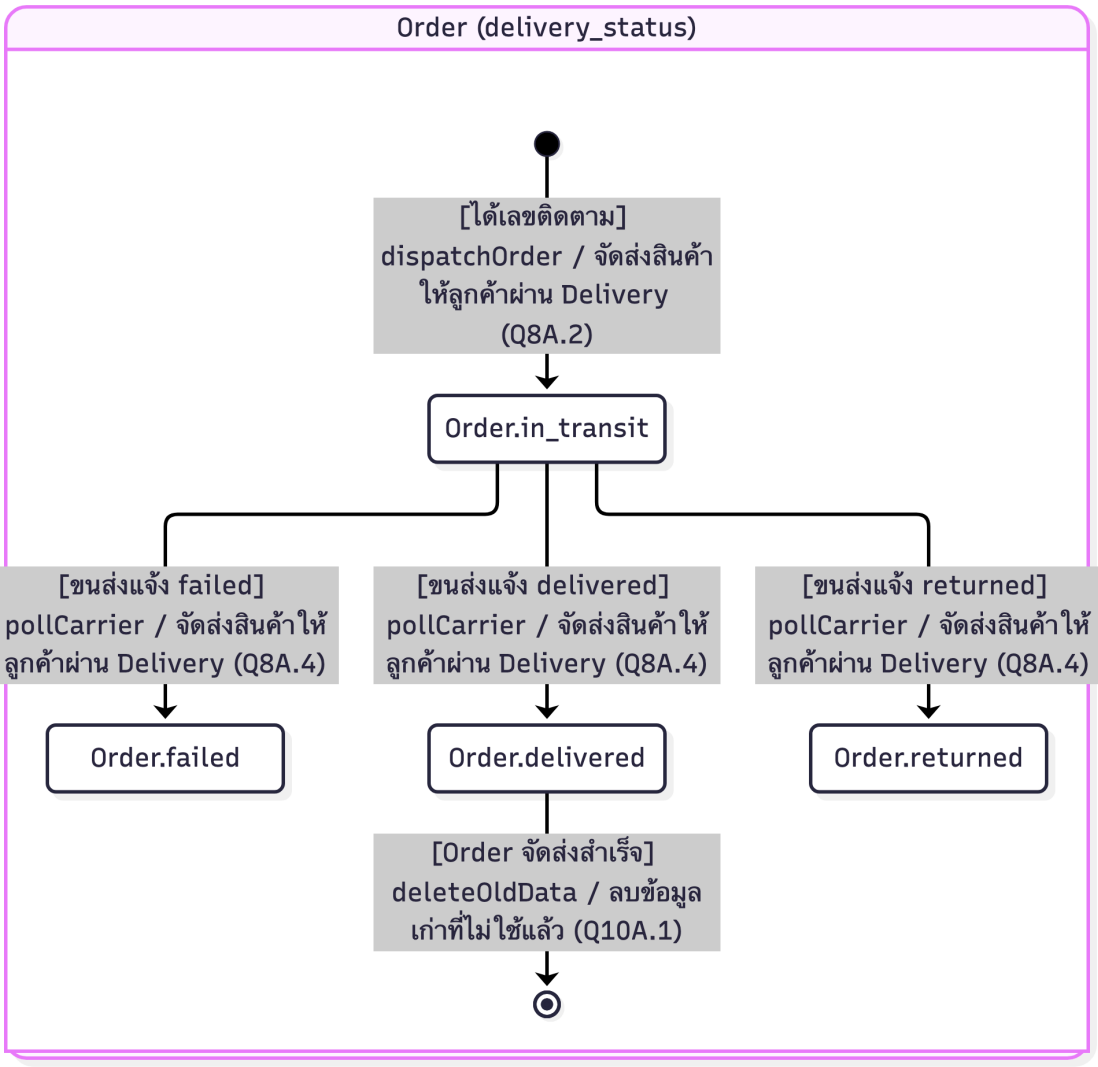
รูปที่ 2-112 State Diagram ของ Label (label_status)

SKU ถูกสร้างเป็น active หรือ inactive ตามที่ Admin เลือกใน 3A (Q3A.6) และสลับสถานะได้ (Q3A.9) SKU ที่เป็น inactive จะไม่ถูกใช้จับคู่ใน 2S SKU ถูกลบได้เมื่อไม่มีข้อมูลอื่นอ้างถึง (Q3A.11)



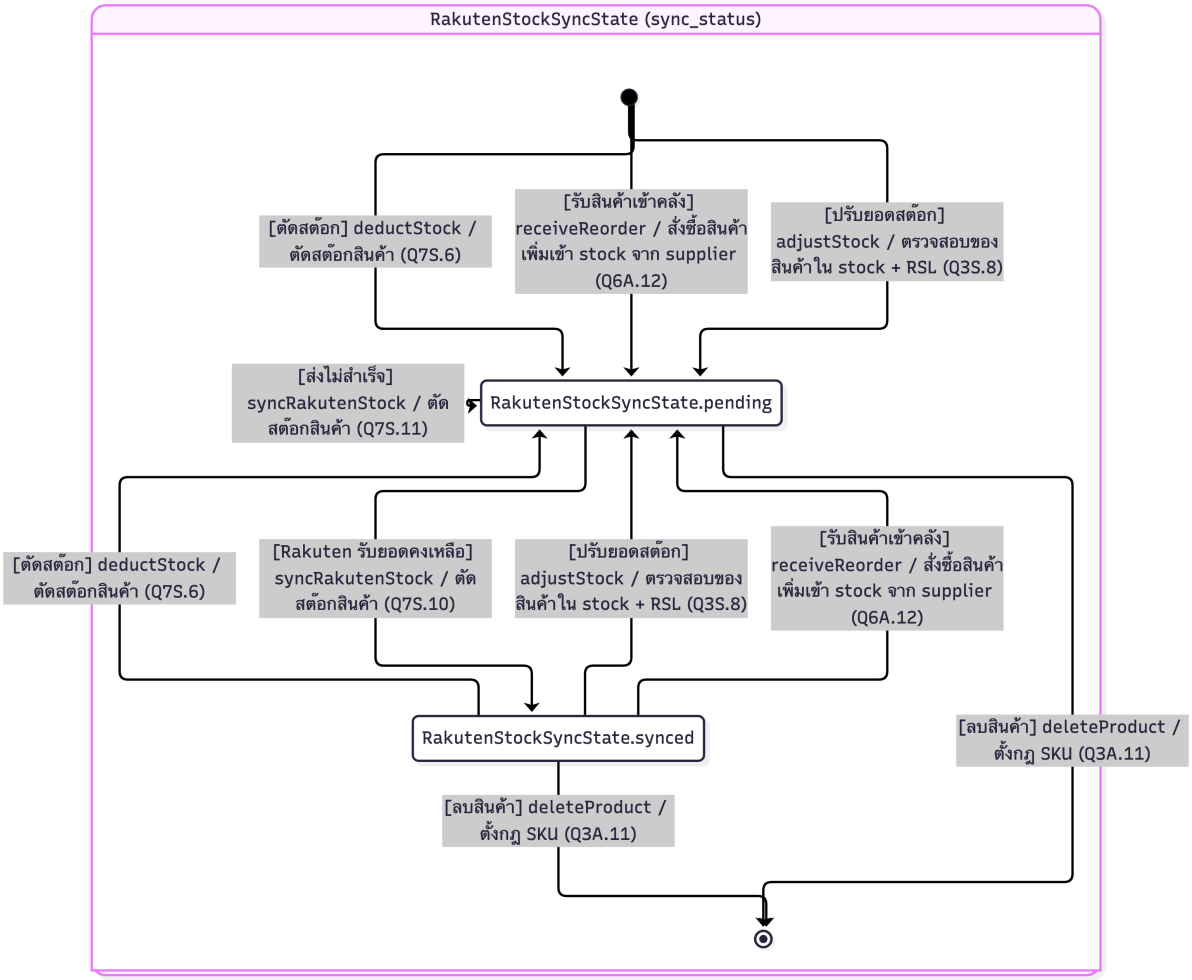
รูปที่ 2-113 State Diagram ของ SKU (sku_status)

delivery_status เป็นค่าว่างจนกว่า Order จะได้เลขติดตาม จากนั้นเป็น in_transit (Q8A.2) และเปลี่ยนเป็น delivered, failed หรือ returned ตามสถานะที่ดึงจากผู้ขนส่ง (Q8A.4) ค่า delivered เป็นเงื่อนไขหนึ่งของ order_status จัดส่งสำเร็จ ส่วน failed และ returned ทำให้ order_status เป็น จัดส่งไม่สำเร็จ และ ตีกลับ/คืนสินค้า ซึ่งรอ Admin จัดการต่อ



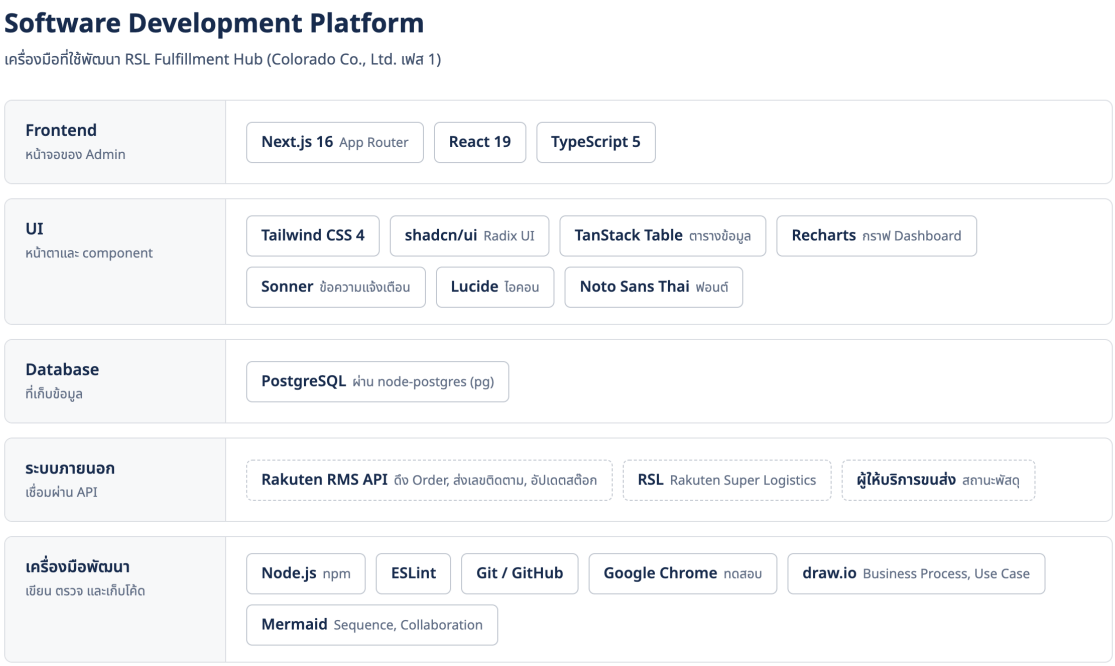
รูปที่ 2-114 State Diagram ของ Order (delivery_status)

ทุกครั้งที่ยอดสต๊อกเปลี่ยน (3S ปรับยอด Q3S.8, 6A รับสินค้า Q6A.12, 7S ตัดสต๊อก Q7S.6) สถานะซิงก์ของแถวสต๊อกนั้นเป็น pending ท้าย runAutomation ระบบส่งยอดคงเหลือไป Rakuten Ichiba หากสำเร็จเป็น synced (Q7S.10) หากไม่สำเร็จคงเป็น pending พร้อม error code เพื่อส่งซ้ำรอบถัดไป (Q7S.11) แถวนี้ถูกลบตามแถวสต๊อกเมื่อลบสินค้า (Q3A.11)



รูปที่ 2-115 State Diagram ของ RakutenStockSyncState (sync_status)

2.20 Software Development Platform



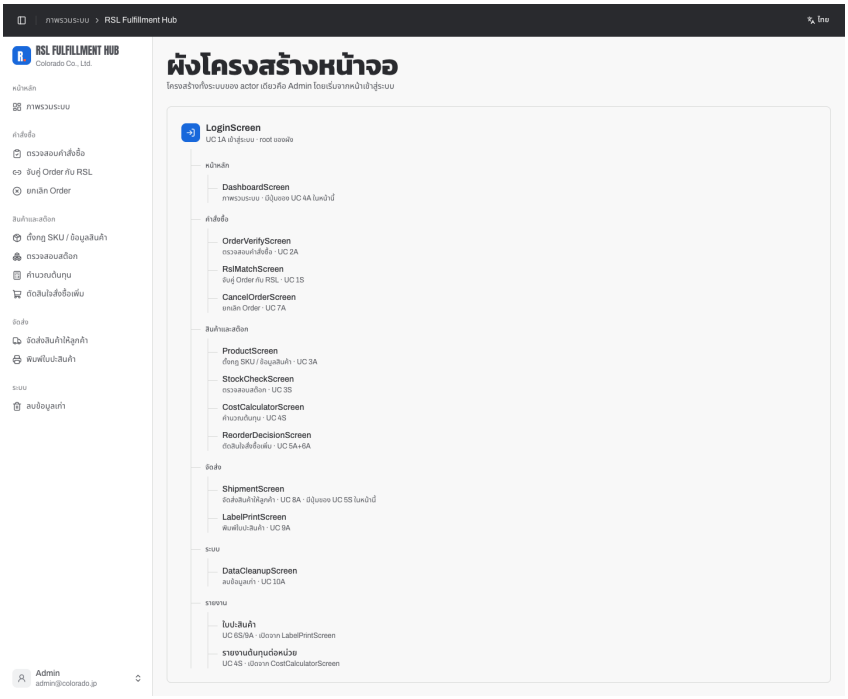
รูปที่ 2-116 เครื่องมือที่ใช้พัฒนาระบบ (Stack of Tools)

ตารางที่ 2-73 เครื่องมือที่ใช้พัฒนาระบบ

ส่วน	เครื่องมือ	ใช้ทำอะไร
Frontend	Next.js 16 (App Router), React 19, TypeScript 5	สร้างหน้าจอทั้งหมดของ Admin
UI	Tailwind CSS 4, shadcn/ui (Radix UI), TanStack Table, Recharts, Sonner, Lucide, ฟอนต์ Noto Sans Thai	หน้าตา ตาราง กราฟ ข้อความแจ้งเตือน และไอคอน
Database	PostgreSQL ผ่าน node-postgres (pg)	ฐานข้อมูลของระบบตามโครงสร้างใน 2.17 ทดสอบ SQL ทุกข้อด้วย PGlite (PostgreSQL) ใน 2.23 เว็บต้นแบบใน 2.24 อ่านและเขียน PostgreSQL จริงทุกหน้าผ่าน Server Action (actions.ts) ⟶ use-case ⟶ โมดูล queries ⟶ db.ts ข้อมูลตั้งต้นสร้างด้วย npm run db:reset

ส่วน	เครื่องมือ	ใช้ทำอะไร
ระบบภายนอก	Rakuten RMS API, RSL, ผู้ให้บริการขนส่ง, Supplier (ผ่าน adapter ใน src/lib/adapters ปัจจุบันเป็น mock)	ดึง Order, ส่งเลขติดตาม, อัปเดตสต๊อก, จับคู่เลข RSL, ดึงสถานะพัสดุ
เครื่องมือพัฒนา	Node.js (npm), ESLint, Git/GitHub, Google Chrome, draw.io, Mermaid	เขียนและตรวจโค้ด เก็บโค้ด ทดสอบ และวาด diagram

2.21 Site Map / UI Structure



รูปที่ 2-117 โครงสร้างหน้าจอของระบบ (Site Map)

ผู้ใช้ทุกคนเข้าผ่านหน้าเดียวกัน จึงมี tree เดียว root คือ LoginScreen

หลังเข้าสู่ระบบจะเห็นเมนูด้านซ้ายแบ่งเป็น 5 กลุ่ม

```
LoginScreen (1A)
└── DashboardScreen (ปุ่มนำเข้า Order ของ 4A)
    ├── คำสั่งซื้อ
    │   ├── OrderVerifyScreen (2A)
    │   ├── RslMatchScreen (1S)
    │   └── CancelOrderScreen (7A)
    ├── สินค้าและสต๊อก
    │   ├── ProductScreen (3A)
    │   ├── StockCheckScreen (3S)
    │   ├── CostScreen (4S) ── รายงานต้นทุนต่อหน่วย
    │   └── ReorderScreen (5A + 6A)
    ├── จัดส่ง
    │   ├── ShippingScreen (8A, ปุ่มส่งเลขติดตามของ 5S)
    │   └── LabelPrintScreen (9A) ── ใบปะสินค้า
    └── ระบบ
        └── CleanupScreen (10A)
```

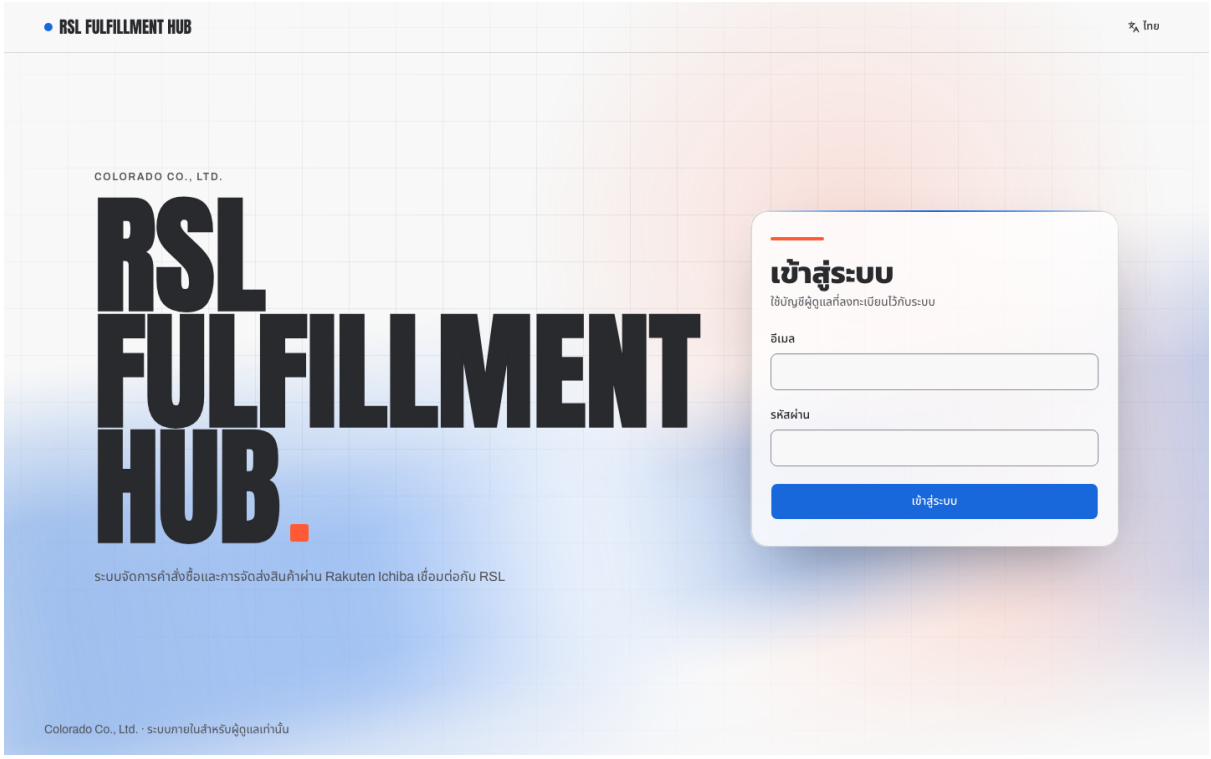
UC ที่ระบบทำเอง (2S, 6S, 7S) ไม่มีหน้าจอของตัวเอง ผลจะขึ้นเป็นข้อความแจ้งเตือนในหน้าที่ Super Admin เปิดอยู่

2.21.1 User Interface Design

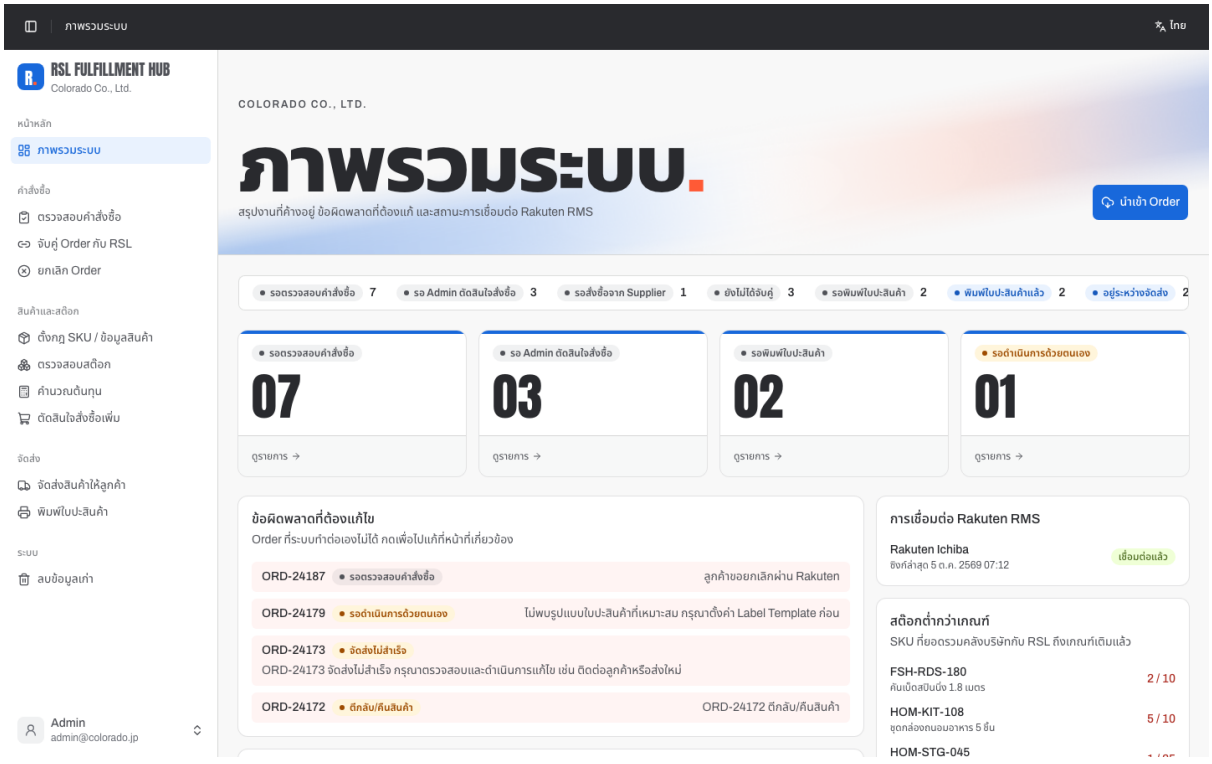
ชื่อหน้าจอตรงกับชื่อ Screen object ใน Sequence Diagram

ตารางที่ 2-74 หน้าจอของระบบและ Use Case ที่เกี่ยวข้อง

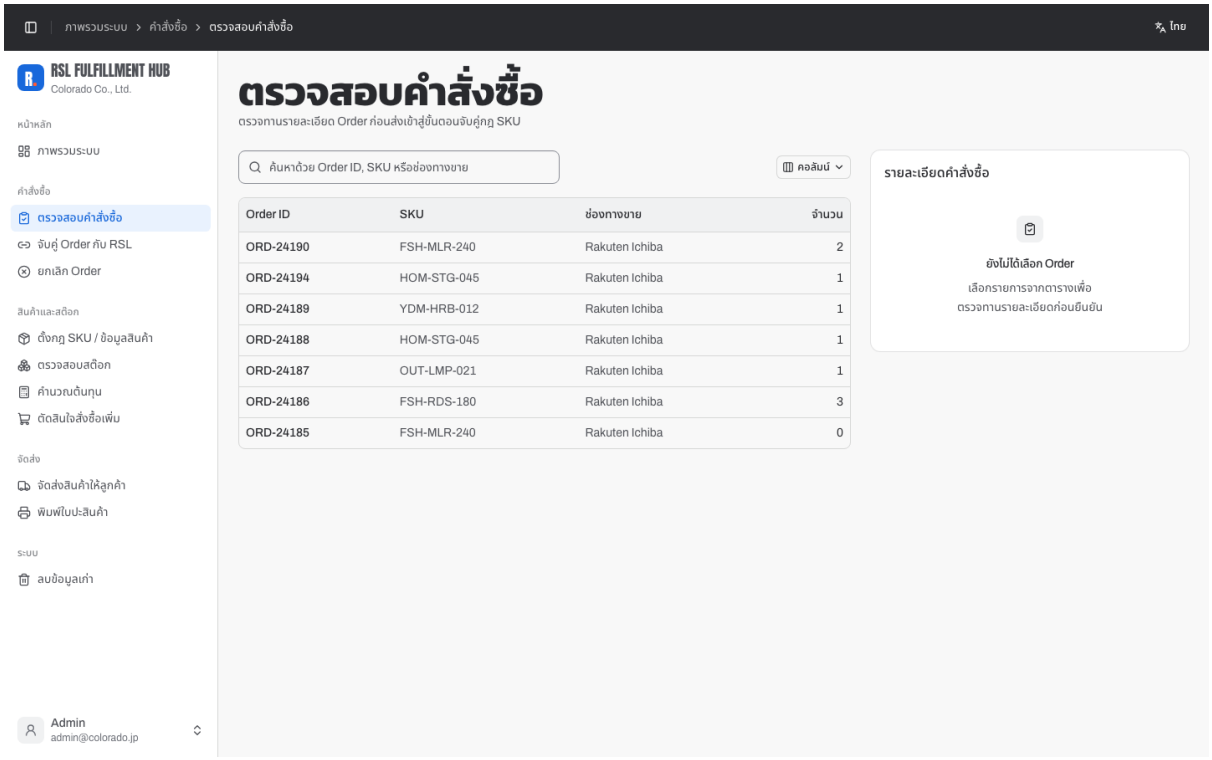
หน้าจอ	UC
LoginScreen	1A เข้าสู่ระบบ
DashboardScreen	4A ดึง Order เข้าระบบ (ปุ่มนำเข้า Order)
OrderVerifyScreen	2A ตรวจสอบคำสั่งซื้อ
RslMatchScreen	1S จับคู่ Order กับเลข RSL
ProductScreen	3A ตั้งกฎ SKU
StockCheckScreen	3S ตรวจสอบของสินค้าใน stock + RSL
CostScreen	4S คำนวณ Cost ในการสั่งสินค้าเติม stock เอง
ReorderScreen	5A ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock, 6A สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier
ReorderScreen (รับสินค้า)	6A สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier — บันทึกการรับสินค้าจาก Supplier
CancelOrderScreen	7A ยกเลิก Order
ShippingScreen	8A จัดส่งสินค้าให้ลูกค้าผ่าน Delivery, 5S ส่งเลขติดตาม
LabelPrintScreen	9A พิมพ์ใบปะสินค้า
CleanupScreen	10A ลบข้อมูลเก่าที่ไม่ใช้แล้ว



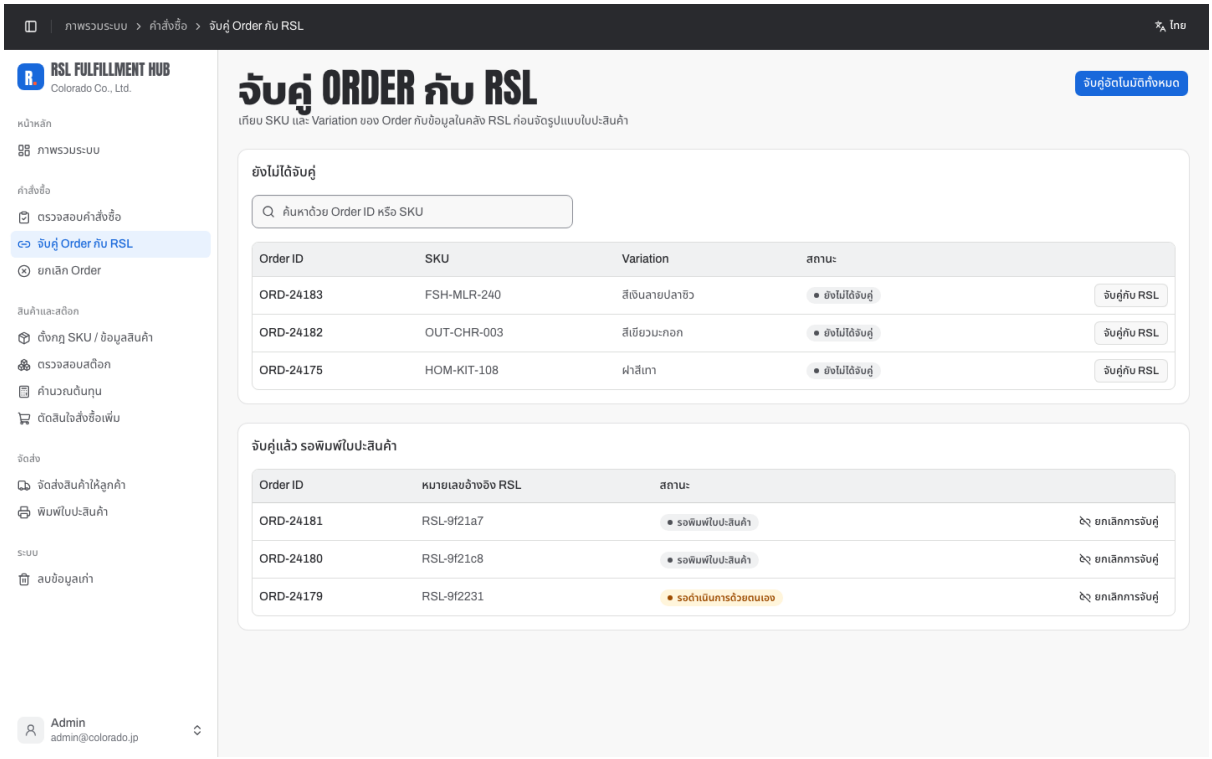
รูปที่ 2-118 LoginScreen (1A เข้าสู่ระบบ)



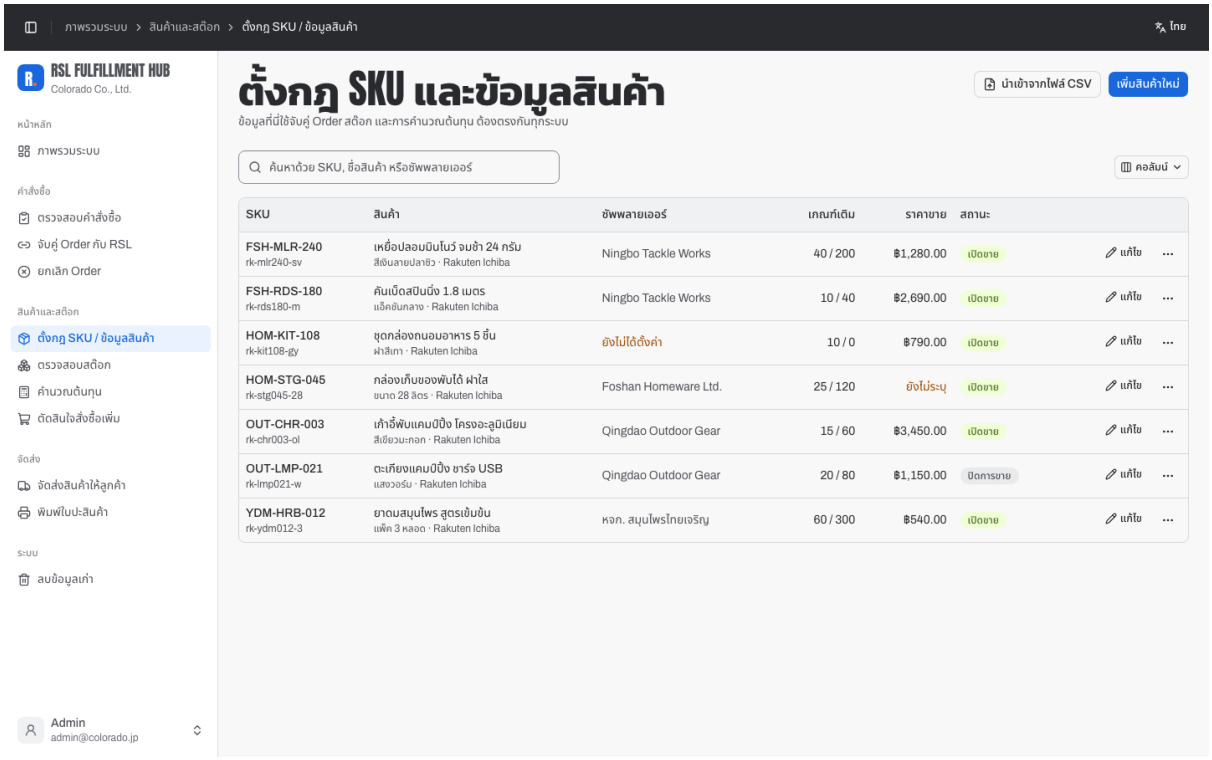
รูปที่ 2-119 DashboardScreen (4A ดึง Order เข้าระบบ (ปุ่มนำเข้า Order))



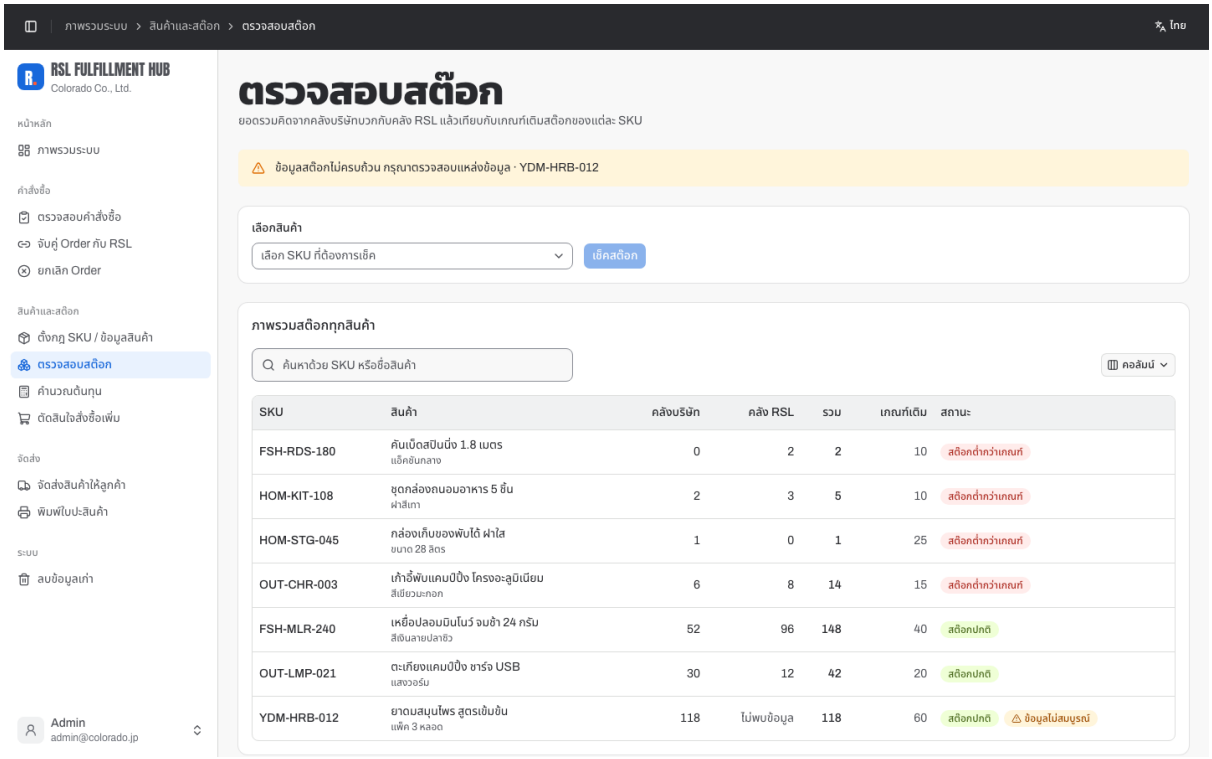
รูปที่ 2-120 OrderVerifyScreen (2A ตรวจสอบคำสั่งซื้อ)



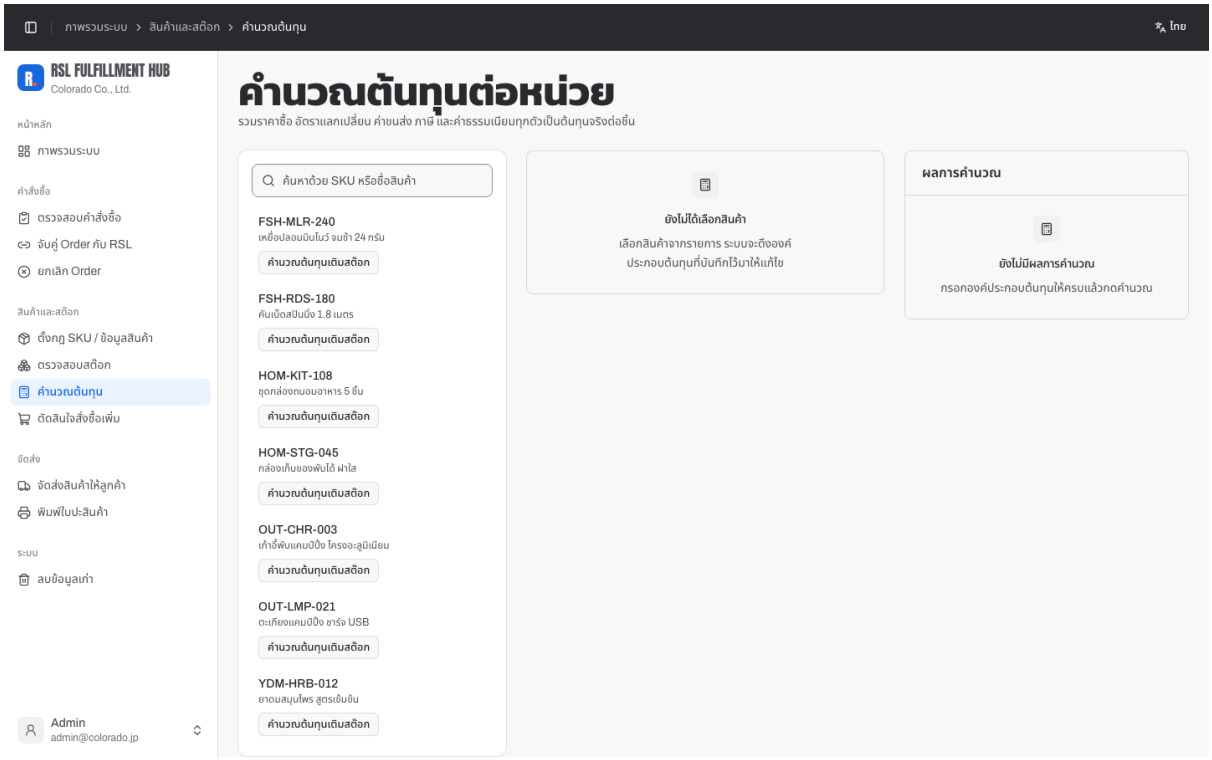
รูปที่ 2-121 RslMatchScreen (1S จับคู่ Order กับเลข RSL)



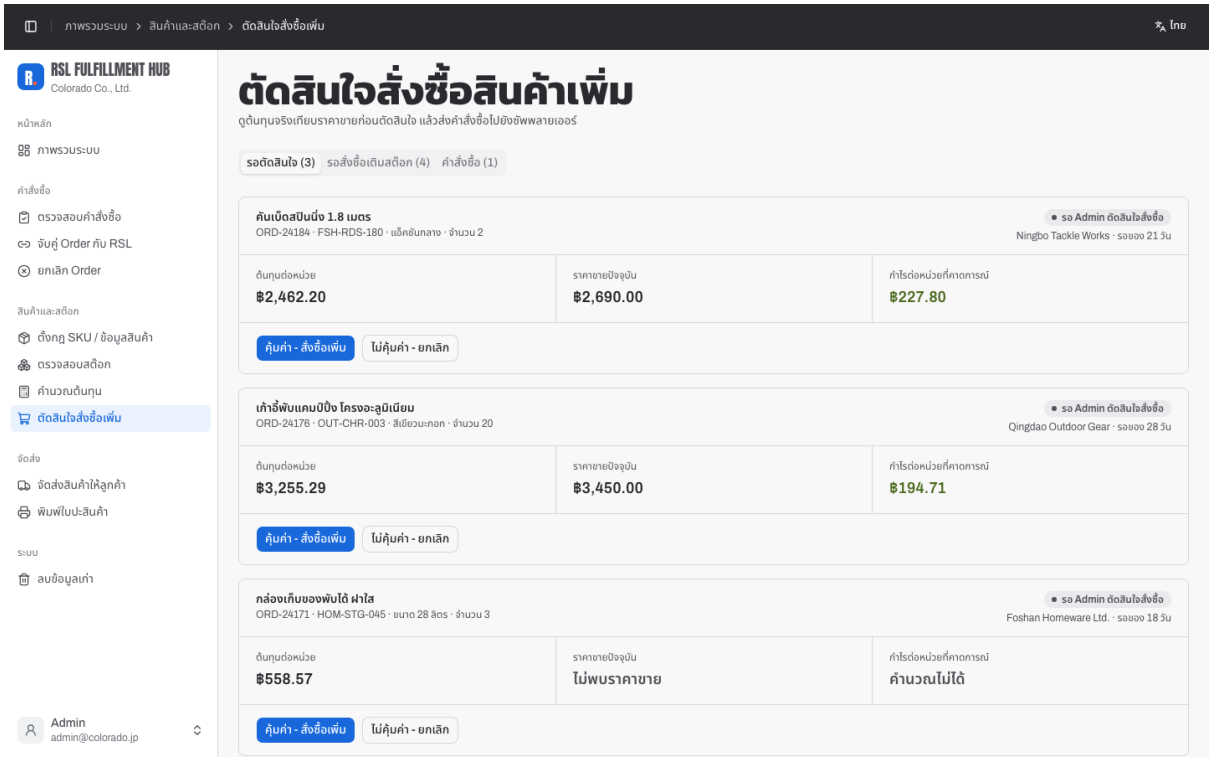
รูปที่ 2-122 ProductScreen (3A ตั้งกฎ SKU)



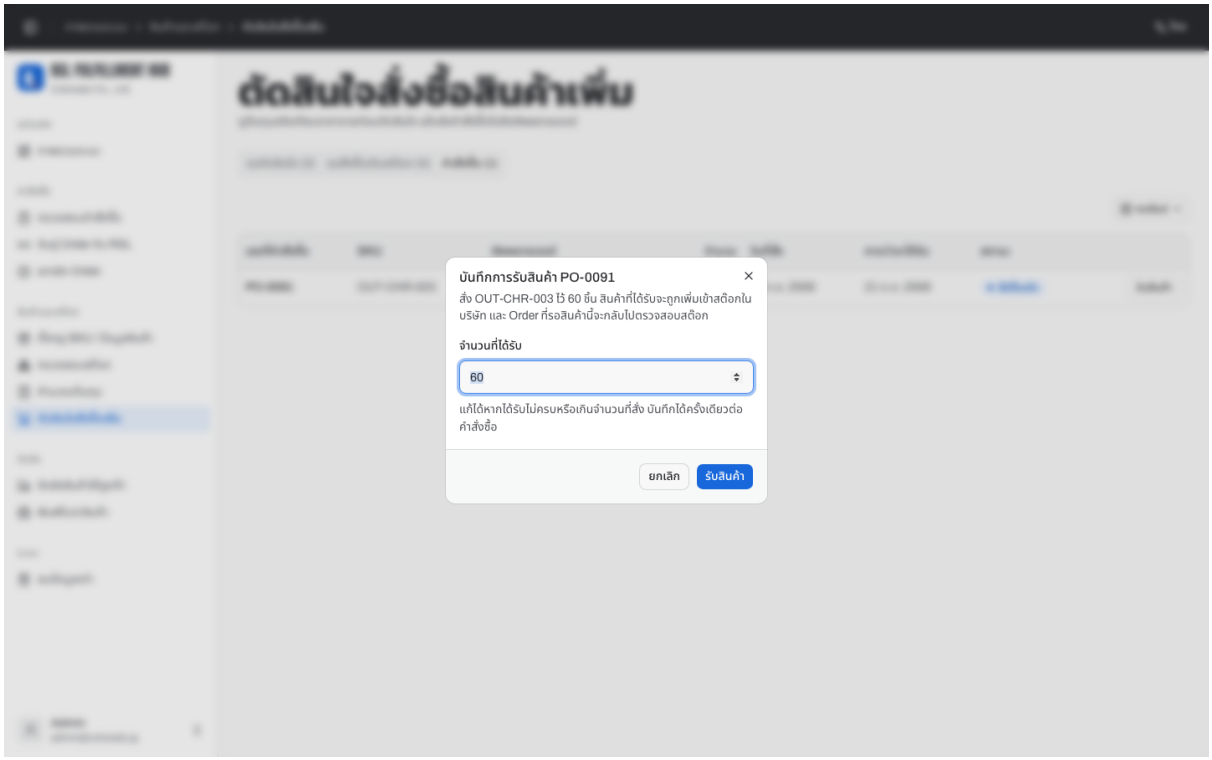
รูปที่ 2-123 StockCheckScreen (3S ตรวจสอบของสินค้าใน stock + RSL)



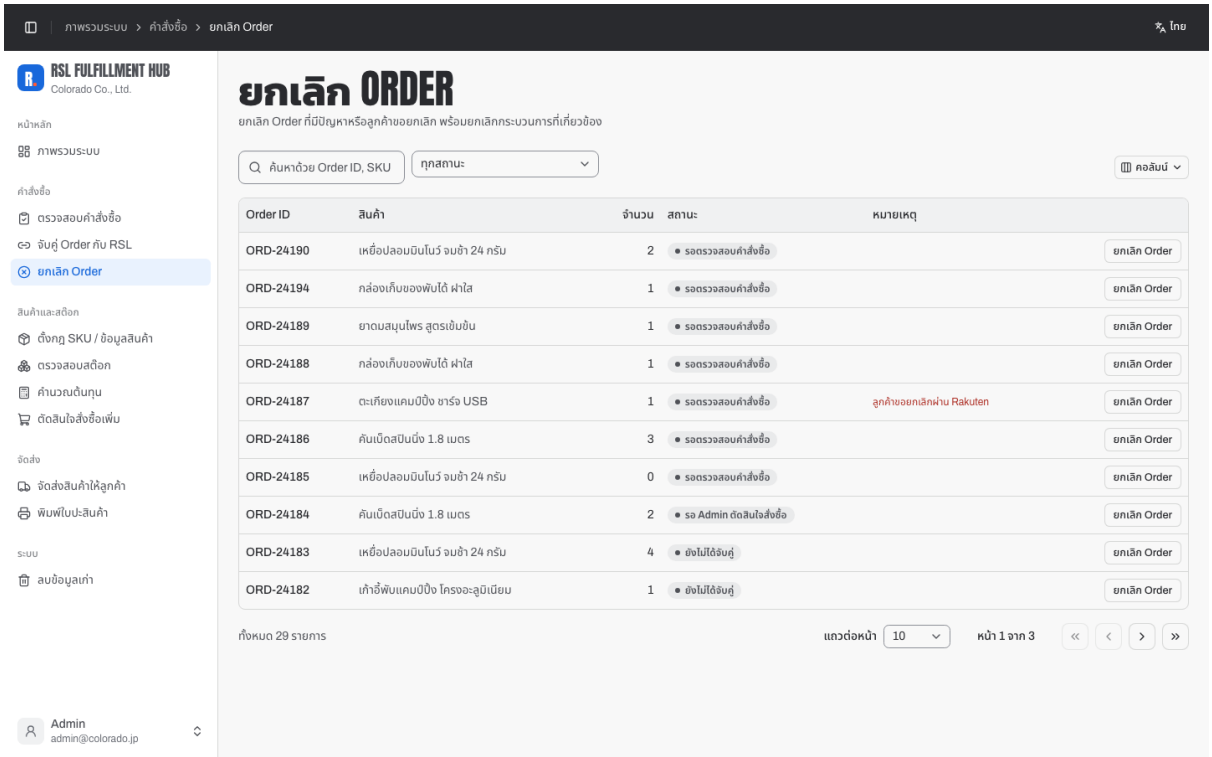
รูปที่ 2-124 CostScreen (4S คำนวณ Cost ในการสั่งสินค้าเติม stock เอง)



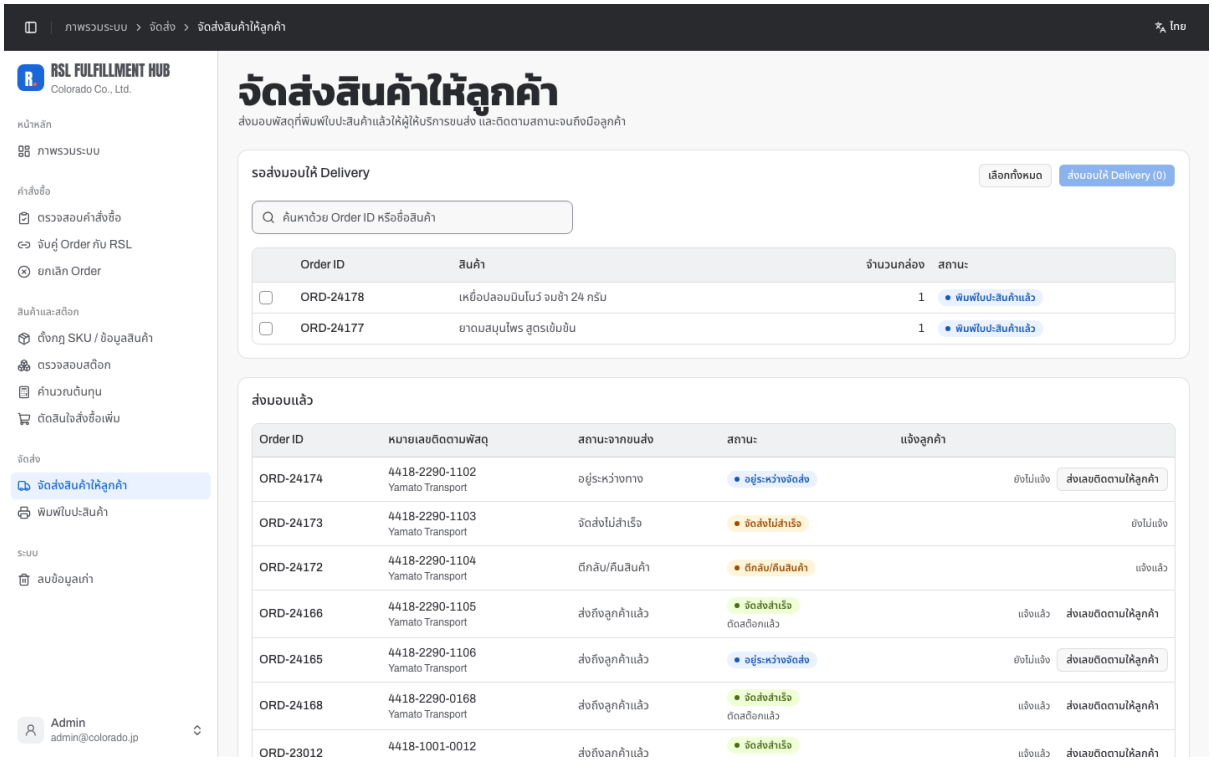
รูปที่ 2-125 ReorderScreen (5A ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock, 6A สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier)



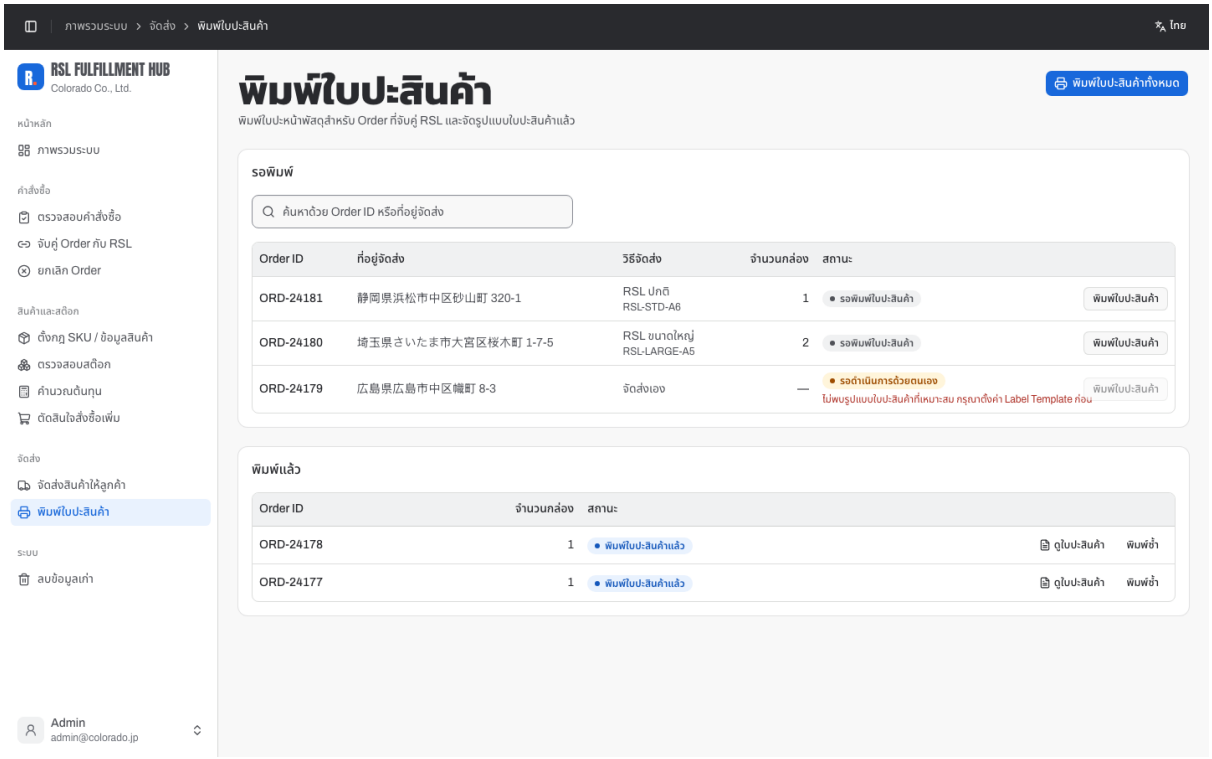
รูปที่ 2-126 ReorderScreen (รับสินค้า) (6A สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier — บันทึกการรับสินค้าจาก Supplier)



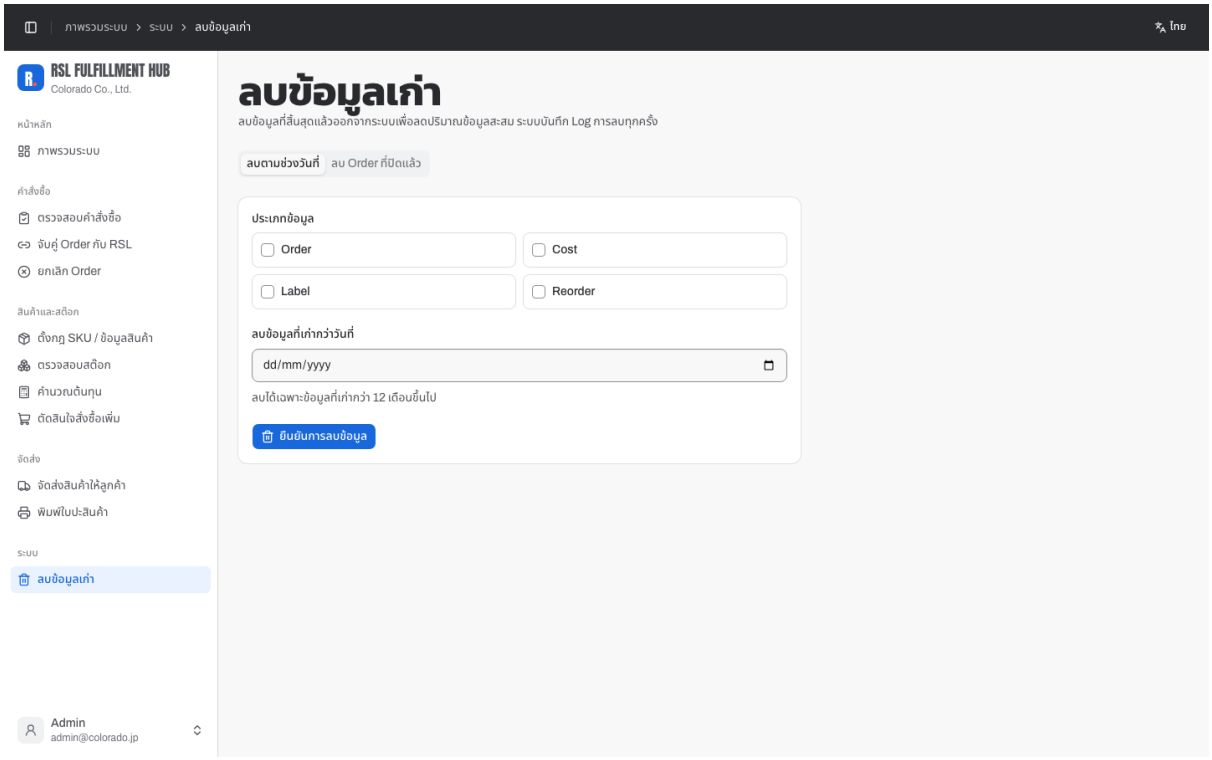
รูปที่ 2-127 CancelOrderScreen (7A ยกเลิก Order)



รูปที่ 2-128 ShippingScreen (8A จัดส่งสินค้าให้ลูกค้าผ่าน Delivery, 5S ส่งเลขติดตาม)



รูปที่ 2-129 LabelPrintScreen (9A พิมพ์ใบปะสินค้า)

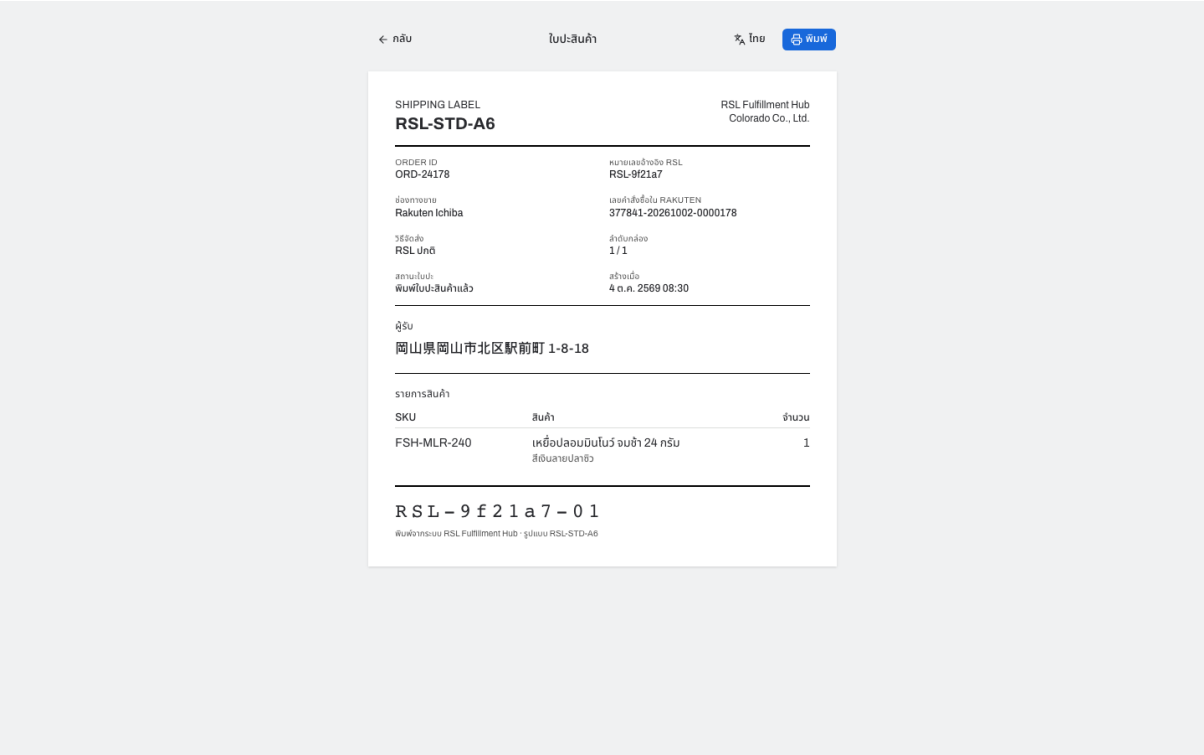


รูปที่ 2-130 CleanupScreen (10A ลบข้อมูลเก่าที่ไม่ใช้แล้ว)

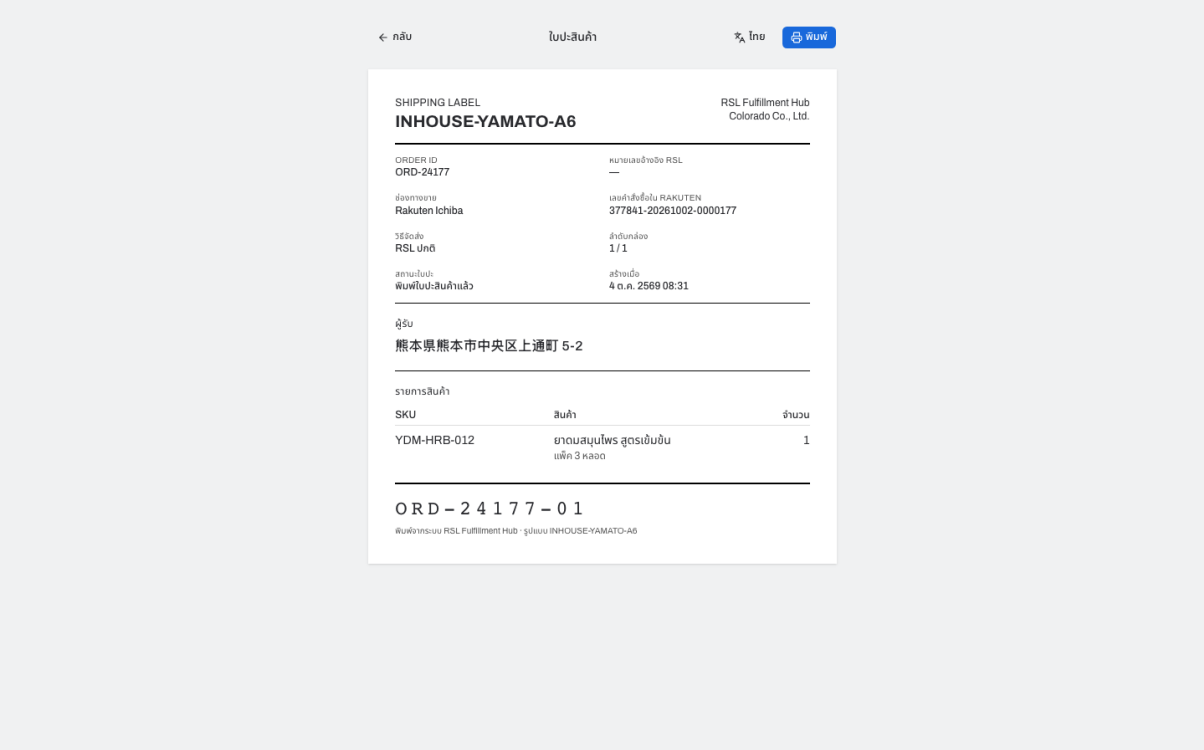
2.22 Output Design (Report)

ระบบมี 2 รายงาน ทั้งคู่เป็นหน้าสำหรับพิมพ์ที่เปิดจากหน้าจอเดิม ตอนพิมพ์จะซ่อนเมนูและปุ่มออก

2.22.1 ใบปะสินค้า (Shipping Label)



รูปที่ 2-131 ใบปะสินค้าของ ORD-24178 (RSL จัดส่ง, RSL-STD-A6)



รูปที่ 2-132 ใบปะสินค้าของ ORD-24177 (บริษัทจัดส่งเอง, INHOUSE-YAMATO-A6)

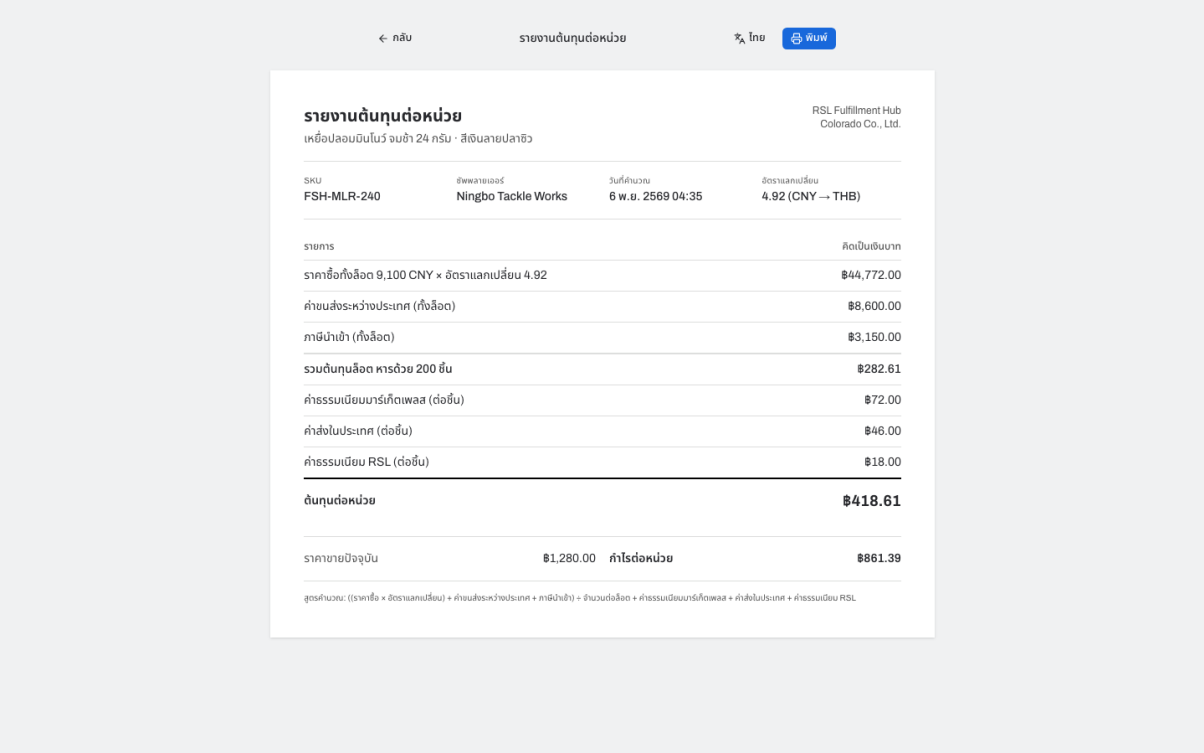
ใบปะสินค้าสร้างโดย Use Case 6S จัดรูปแบบใบปะสินค้า และพิมพ์ผ่าน Use Case 9A จากหน้า LabelPrintScreen ข้อมูลมาจาก Q6S.4 (ตาราง labels) และ Q9A.2 (labels ร่วมกับ orders) รูปแบบใบปะขึ้นกับแหล่งที่จัดส่งที่ 3S เลือก Order ที่ RSL จัดส่งพิมพ์เลข RSL และใช้เลข RSL เป็นรหัสสแกน ส่วน Order ที่บริษัทจัดส่งเองจากคลังบริษัทใช้รูปแบบ INHOUSE-YAMATO-A6 ช่องเลข RSL แสดง "—" และใช้ [Order ID]-[ลำดับกล่อง] เป็นรหัสสแกน

ตารางที่ 2-75 โครงสร้างของใบปะสินค้า

ส่วนของรายงาน	ข้อมูล	ที่มา
หัวรายงาน	รูปแบบใบปะ (`label_template`) RSL-STD-A6, RSL-LARGE-A5 หรือ INHOUSE-YAMATO-A6 ตามแหล่งที่จัดส่ง	Q6S.4 (template จากกฎใน src/mock/delivery.ts)
ข้อมูล Order	Order ID, เลข RSL (`labels.rsl_no`, Order ที่บริษัทจัดส่งเองแสดง "—"), ช่องทางขาย, เลขคำสั่งซื้อใน Rakuten, วิธีจัดส่ง	`orders`, 1S
ลำดับกล่อง	`parcel_seq` / `parcel_total` เช่น 1/1 หรือ 1/2	Q6S.4
สถานะ	`label_status` และวันเวลาที่สร้าง (`created_at`)	Q6S.4, Q9A.4

ส่วนของรายงาน	ข้อมูล	ที่มา
ผู้รับ	ที่อยู่จัดส่ง	`orders.shipping_address`
รายการสินค้า	SKU, ชื่อสินค้า, Variation, จำนวน	`orders`
ท้ายใบ	เลขอ้างอิงสำหรับสแกน (เลข RSL + ลำดับกล่อง หรือ Order ID + ลำดับกล่อง ถ้าบริษัทจัดส่งเอง)	Q6S.4

2.22.2 รายงานต้นทุนต่อหน่วย (Unit Cost Report)



รูปที่ 2-133 รายงานต้นทุนต่อหน่วยของ FSH-MLR-240

รายงานต้นทุนต่อหน่วยมาจาก Use Case 4S คำนวณ Cost ในการสั่งสินค้าเติม stock เอง เปิดจากหน้า CostScreen (/products/cost) คำนวณด้วยฟังก์ชัน calcUnitCost แล้วบันทึกประวัติด้วย Q4S.6 และบันทึกค่าล่าสุดลงตาราง costs ด้วย Q4S.5 เมื่อเลือกบันทึกค่า

```
Unit Cost = ((purchase_price × exchange_rate) + intl_freight +
duty_fee) / order_qty
            + marketplace_fee + domestic_shipping + rsl_charge
```

ตารางที่ 2-76 โครงสร้างของรายงานต้นทุนต่อหน่วย

ส่วนของรายงาน	ข้อมูล
หัวรายงาน	SKU, ชื่อสินค้า, วันที่คำนวณ, อัตราแลกเปลี่ยนที่ใช้
รายการต้นทุน	ราคาซื้อทั้งล็อต, อัตราแลกเปลี่ยน, ค่าขนส่งระหว่างประเทศ, ภาษีนำเข้า, จำนวนที่สั่ง, ค่าธรรมเนียม Marketplace, ค่าส่งในประเทศ, ค่าธรรมเนียม RSL แยกบรรทัด
สรุป	ต้นทุนต่อหน่วย เทียบกับราคาขาย
ประวัติ	ผลการคำนวณครั้งก่อนจาก cost log

2.23 SQL Statement ใน Use Case Description

SQL ทุกคำสั่งถูกรันกับฐานข้อมูลตัวอย่าง (PostgreSQL) ตารางแสดงเลข query, Use Case, คำสั่ง SQL, ค่าที่ส่งเข้าไป และผลลัพธ์ คำสั่ง INSERT, UPDATE และ DELETE แสดงข้อมูลหลังรันคำสั่ง ทุกคำสั่งรันแยกกันจากข้อมูลตั้งต้นชุดเดียวกัน SQL ทุกข้อเป็นคำสั่งเดียวกับค่าคงที่ใน src/lib/queries ของเว็บ

2.23.1 Use Case 1A: เข้าสู่ระบบ

ตารางที่ 2-77 SQL Statement และผลการทำงานของ Use Case 1A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q1A.1	`SELECT admin_id, user_email, user_password, fail_attempts, locked_until, last_login FROM admins WHERE user_email = $1`	admin@colorado.jp	admin_id=U001, user_email=admin@colorado.jp, user_password=scrypt$16384$8$1$yFu3Dhgi6N9Az8G5pC-K5g$2vNbQ3dPbBotWW7GtOH7okBthgVVwKV9synZ7JfHMjp8fRmqUFR_Ncu6EQrmmoJ1-vaToeuO2VoeUplGpM7inw, fail_attempts=0, locked_until=NULL, last_login=NULL
Q1A.2	`UPDATE admins SET fail_attempts = 0, locked_until = NULL WHERE admin_id = $1`	U002	แก้ไข 1 แถว หลังรัน: admin_id=U002, user_email=locked@colorado.jp, fail_attempts=0, locked_until=NULL
Q1A.3	`UPDATE admins SET fail_attempts = fail_attempts + 1, locked_until = CASE WHEN fail_attempts + 1 >= $3 THEN $2::timestamptz + make_interval(mins => $4::integer) ELSE locked_until END WHERE admin_id = $1 RETURNING fail_attempts, locked_until`	U001, 2026-11-06 09:00:00+07, 5, 15	แก้ไข 1 แถว หลังรัน: admin_id=U001, user_email=admin@colorado.jp, fail_attempts=5, locked_until=2026-11-06 02:15
Q1A.4	`UPDATE admins SET fail_attempts = 0, locked_until = NULL, last_login = $2 WHERE admin_id = $1`	U001, 2026-11-06 09:00:00+07	แก้ไข 1 แถว หลังรัน: admin_id=U001, fail_attempts=0, last_login=2026-11-06 02:00
Q1A.5	`INSERT INTO sessions ( session_id, admin_id, token_hash, created_at, expires_at, revoked_at ) VALUES ($1, $2, $3, $4, $5, NULL)`	SES-0003, U001, sha256:9c1d, 2026-11-06 09:00:00+07, 2026-11-06 21:00:00+07	แก้ไข 1 แถว หลังรัน: session_id=SES-0001, admin_id=U001, created_at=2026-10-04 23:30, expires_at=2026-10-05 11:30, revoked_at=NULL session_id=SES-0002, admin_id=U001, created_at=2026-10-03 01:10, expires_at=2026-10-03 13:10, revoked_at=2026-10-03 09:00 session_id=SES-0003, admin_id=U001, created_at=2026-11-06 02:00, expires_at=2026-11-06 14:00, revoked_at=NULL

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q1A.6	`SELECT s.session_id, s.admin_id, a.user_email, s.created_at, s.expires_at FROM sessions AS s INNER JOIN admins AS a ON a.admin_id = s.admin_id WHERE s.token_hash = $1 AND s.revoked_at IS NULL AND s.expires_at > $2`	sha256:7f3c9a1e5b2d4c8f0a6e9b3d1c7f5a2e, 2026-10-05 08:00:00+00	session_id=SES-0001, admin_id=U001, user_email=admin@colorado.jp, created_at=2026-10-04 23:30, expires_at=2026-10-05 11:30
Q1A.7	`UPDATE sessions SET revoked_at = $2 WHERE token_hash = $1 AND revoked_at IS NULL`	sha256:7f3c9a1e5b2d4c8f0a6e9b3d1c7f5a2e, 2026-10-05 09:00:00+00	แก้ไข 1 แถว หลังรัน: session_id=SES-0001, revoked_at=2026-10-05 09:00 session_id=SES-0002, revoked_at=2026-10-03 09:00

2.23.2 Use Case 2A: ตรวจสอบคำสั่งซื้อ

ตารางที่ 2-78 SQL Statement และผลการทำงานของ Use Case 2A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q2A.1	`SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_date, o.shipping_address, o.shipping_method, o.order_status, o.rsl_no, o.tracking_number, o.carrier_name, o.dispatched_at, o.delivery_status, o.tracking_notified, o.tracking_notified_at, o.cancel_reason, o.cancelled_at, o.verified_at, o.manual_reason, o.customer_cancel_requested, o.edit_log, o.reorder_decision, o.decided_qty, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.deducted_at, item.sku_id, sku.sku, sku.product_name, sku.selling_price, o.reorder_id, label.label_template, label.parcel_total, label.label_printed_at, sync.sync_status, notification.notify_log FROM orders o LEFT JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN stock stock_row ON stock_row.sku_id = sku.sku_id LEFT JOIN rakuten_stock_sync_states sync ON sync.stock_id = stock_row.stock_id LEFT JOIN LATERAL (SELECT MAX(l.label_template) AS label_template, MAX(l.parcel_total) AS parcel_total, MAX(l.printed_at) AS label_printed_at FROM labels l WHERE l.order_id = o.order_id) label ON TRUE LEFT JOIN LATERAL (SELECT jsonb_agg(jsonb_build_object('by', COALESCE(t.admin_id, 'system'), 'at', t.sent_at) ORDER BY t.sent_at) AS notify_log FROM tracking_notification_logs t WHERE t.order_id = o.order_id) notification ON TRUE ORDER BY o.order_date DESC, o.order_id DESC`	-	order_id=ORD-24190, marketplace_order_id=377841-20261005-0000190, sales_channel=Rakuten Ichiba, order_date=2026-10-05 00:12, shipping_address=東京都杉並区高円寺南 3-14-7 サンハイツ 402, shipping_method=RSL ปกติ, order_status=รอตรวจสอบคำสั่งซื้อ, rsl_no=NULL, tracking_number=NULL, carrier_name=NULL, dispatched_at=NULL, delivery_status=NULL, tracking_notified=false, tracking_notified_at=NULL, cancel_reason=NULL, cancelled_at=NULL, verified_at=NULL, manual_reason=NULL, customer_cancel_requested=false, edit_log={}, reorder_decision=NULL, decided_qty=NULL, channel_sku=rk-mlr240-sv, variation=สีเงินลายปลาชิว, qty=2, fulfill_source=rsl, stock_deducted=false, deducted_at=NULL, sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, product_name=เหยื่อปลอมมินโนว์ จมช้า 24 กรัม, selling_price=1280.00, reorder_id=NULL, label_template=NULL, parcel_total=NULL, label_printed_at=NULL, sync_status=synced, notify_log=NULL

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
			order_id=ORD-24194, marketplace_order_id=377841-20261005-0000194, sales_channel=Rakuten Ichiba, order_date=2026-10-04 23:47, shipping_address=長野県松本市中央 2-5-11, shipping_method=จัดส่งเอง, order_status=รอตรวจสอบคำสั่งซื้อ, rsl_no=NULL, tracking_number=NULL, carrier_name=NULL, dispatched_at=NULL, delivery_status=NULL, tracking_notified=false, tracking_notified_at=NULL, cancel_reason=NULL, cancelled_at=NULL, verified_at=NULL, manual_reason=NULL, customer_cancel_requested=false, edit_log={}, reorder_decision=NULL, decided_qty=NULL, channel_sku=rk-stg045-28, variation=ขนาด 28 ลิตร, qty=1, fulfill_source=rsl, stock_deducted=false, deducted_at=NULL, sku_id=SKU-HOM-STG-045, sku=HOM-STG-045, product_name=กล่องเก็บของพับได้ ฝาใส, selling_price=NULL, reorder_id=NULL, label_template=NULL, parcel_total=NULL, label_printed_at=NULL, sync_status=synced, notify_log=NULL order_id=ORD-24189, marketplace_order_id=377841-20261005-0000189, sales_channel=Rakuten Ichiba, order_date=2026-10-04 23:47, shipping_address=大阪府吹田市江坂町 1-23-5, shipping_method=RSL ปกติ, order_status=รอตรวจสอบคำสั่งซื้อ, rsl_no=NULL, tracking_number=NULL, carrier_name=NULL, dispatched_at=NULL, delivery_status=NULL, tracking_notified=false, tracking_notified_at=NULL, cancel_reason=NULL, cancelled_at=NULL, verified_at=NULL, manual_reason=NULL, customer_cancel_requested=false, edit_log={}, reorder_decision=NULL, decided_qty=NULL, channel_sku=rk-

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
			ydm012-3-new, variation=แพ็ค 3 หลอด, qty=1, fulfill_source=rsl, stock_deducted=false, deducted_at=NULL, sku_id=SKU-YDM-HRB-012, sku=YDM-HRB-012, product_name=ยาดมสมุนไพร สูตรเข้มข้น, selling_price=540.00, reorder_id=NULL, label_template=NULL, parcel_total=NULL, label_printed_at=NULL, sync_status=synced, notify_log=NULL order_id=ORD-24188, marketplace_order_id=377841-20261005-0000188, sales_channel=Rakuten Ichiba, order_date=2026-10-04 23:02, shipping_address=, shipping_method=RSL ปกติ, order_status=รอตรวจสอบคำสั่งซื้อ, rsl_no=NULL, tracking_number=NULL, carrier_name=NULL, dispatched_at=NULL, delivery_status=NULL, tracking_notified=false, tracking_notified_at=NULL, cancel_reason=NULL, cancelled_at=NULL, verified_at=NULL, manual_reason=NULL, customer_cancel_requested=false, edit_log={}, reorder_decision=NULL, decided_qty=NULL, channel_sku=rk-stg045-28, variation=ขนาด 28 ลิตร, qty=1, fulfill_source=rsl, stock_deducted=false, deducted_at=NULL, sku_id=SKU-HOM-STG-045, sku=HOM-STG-045, product_name=กล่องเก็บของพับได้ ฝาใส, selling_price=NULL, reorder_id=NULL, label_template=NULL, parcel_total=NULL, label_printed_at=NULL, sync_status=synced, notify_log=NULL (ทั้งหมด 31 แถว)
Q2A.2	`SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id,`	ORD-24190	order_id=ORD-24190, marketplace_order_id=377841-20261005-0000190, sales_channel=Rakuten Ichiba, order_status=รอตรวจสอบคำสั่งซื้อ, manual_reason=NULL, shipping_address=東京都杉並区

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
	reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o		高円寺南 3-14-7 サンハイツ 402, shipping_method=RSL ปกติ, rsl_no=NULL, cancelled_at=NULL, customer_cancel_requested=false, channel_sku=rk-mlr240-sv, variation=สีเงินลายปลาชิว, qty=2, fulfill_source=rsl, stock_deducted=false, sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, label_printed=false, reorder_id=NULL, reorder_status=NULL
Q2A.3	WITH updated_item AS (UPDATE order_items SET channel_sku = $2, variation = $3, qty = $4 WHERE order_id = $1) UPDATE orders SET shipping_address = $5, shipping_method = $6, order_status = $7, manual_reason = $8, customer_cancel_requested = $9, verified_at = $10, edit_log = CASE WHEN $11::boolean THEN edit_log || jsonb_build_array(jsonb_build_object('field', 'shipping_method', 'by', $12::text, 'at', $10::timestamptz)) ELSE edit_log END, cancel_reason = $13, cancelled_at = $14, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1	ORD-24190, rk-mlr240-sv, สีเงินลายปลาชิว, 2, 東京都杉並区高円寺南 3-14-7 サンハイツ 402, RSL ขนาดใหญ่, รอจับคู่กฎ SKU, NULL, false, 2026-10-05 10:00:00+09, true, U001, NULL, NULL	แก้ไข 1 แถว หลังรัน: order_id=ORD-24190, order_status=รอจับคู่กฎ SKU, shipping_method=RSL ขนาดใหญ่, verified_at=2026-10-05 01:00, edit_log={[object Object]}

2.23.3 Use Case 3A: ตั้งกฎ SKU

ตารางที่ 2-79 SQL Statement และผลการทำงานของ Use Case 3A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q3A.1	SELECT sku, product_name, variation, sales_channel, channel_sku, supplier_id, reorder_threshold, reorder_qty, selling_price, sku_status FROM skus ORDER BY sku	-	sku=FSH-MLR-240, product_name=เหยื่อปลอมมินโนว์ จมช้า 24 กรัม, variation=สีเงินลายปลาชิว, sales_channel=Rakuten Ichiba, channel_sku=rk-mlr240-sv, supplier_id=SUP-01, reorder_threshold=40, reorder_qty=200, selling_price=1280.00, sku_status=active sku=FSH-RDS-180, product_name=คันเบ็ดสปินนิ่ง 1.8 เมตร, variation=แอ็คชันกลาง, sales_channel=Rakuten Ichiba, channel_sku=rk-rds180-m, supplier_id=SUP-01, reorder_threshold=10, reorder_qty=40, selling_price=2690.00, sku_status=active sku=HOM-KIT-108, product_name=ชุดกล่องถนอมอาหาร 5 ชิ้น, variation=ฝาสีเทา, sales_channel=Rakuten Ichiba, channel_sku=rk-kit108-gy, supplier_id=NULL, reorder_threshold=10, reorder_qty=0, selling_price=790.00, sku_status=active

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
			sku=HOM-STG-045, product_name=กล่องเก็บของพับได้ ฝาใส, variation=ขนาด 28 ลิตร, sales_channel=Rakuten Ichiba, channel_sku=rk-stg045-28, supplier_id=SUP-04, reorder_threshold=25, reorder_qty=120, selling_price=NULL, sku_status=active (ทั้งหมด 7 แถว)
Q3A.2	`SELECT supplier_id, supplier_name, contact_info, lead_time_days FROM suppliers ORDER BY supplier_name, supplier_id`	-	supplier_id=SUP-04, supplier_name=Foshan Homeware Ltd., contact_info=trade@foshan-home.example.cn, lead_time_days=18 supplier_id=SUP-01, supplier_name=Ningbo Tackle Works, contact_info=order@ningbo-tackle.example.cn, lead_time_days=21 supplier_id=SUP-03, supplier_name=Qingdao Outdoor Gear, contact_info=po@qingdao-outdoor.example.cn, lead_time_days=28 supplier_id=SUP-02, supplier_name=หจก. สมุนไพรไทยเจริญ, contact_info=sales@thaiherb.example.co.th, lead_time_days=10
Q3A.3	`SELECT sku_id, sku, channel_sku FROM skus WHERE sku = $1 FOR UPDATE`	FSH-MLR-240	sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, channel_sku=rk-mlr240-sv
Q3A.4	`SELECT sku_id, sku, channel_sku FROM skus WHERE (sku = $1 OR channel_sku = $2) AND ($3::text IS NULL OR sku_id <> $3) LIMIT 1`	FSH-MLR-240, rk-mlr240-sv, NULL	sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, channel_sku=rk-mlr240-sv
Q3A.5	`SELECT EXISTS ( SELECT 1 FROM suppliers WHERE supplier_id = $1 ) AS found`	SUP-01	found=true
Q3A.6	`WITH inserted_sku AS (INSERT INTO skus (sku_id, sku, product_name, variation, sales_channel, channel_sku, selling_price, reorder_threshold, reorder_qty, sku_status, supplier_id) VALUES ($1, $2, $3, $4, 'Rakuten Ichiba', $5, $6, $7, $8, $9, $10) RETURNING sku_id) INSERT INTO stock (stock_id, in_house_qty, rsl_qty, updated_at, sku_id) SELECT $11, 0, 0, CURRENT_TIMESTAMP, sku_id FROM inserted_sku`	SKU-FSH-TRL-090, FSH-TRL-090, เหยื่อยางหนอน 9 ซม., สีส้มเรืองแสง, rk-trl090-or, 680, 30, 150, active, SUP-01, STK-FSH-TRL-090	แก้ไข 1 แถว หลังรัน: sku_id=SKU-FSH-TRL-090, sku=FSH-TRL-090, channel_sku=rk-trl090-or, sku_status=active, stock_id=STK-FSH-TRL-090, in_house_qty=0, rsl_qty=0
Q3A.7	`UPDATE skus SET sku = $2, product_name = $3, variation = $4, channel_sku = $5, selling_price = $6, reorder_threshold = $7, reorder_qty = $8, sku_status = $9, supplier_id = $10 WHERE sku_id = $1`	SKU-FSH-MLR-240, FSH-MLR-240, เหยื่อปลอมมินโนว์ จมช้า 24 กรัม, สีเงินลายปลาชิว, rk-mlr240-sv, 1290, 50, 240, active, SUP-01	แก้ไข 1 แถว หลังรัน: sku=FSH-MLR-240, reorder_threshold=50, reorder_qty=240, selling_price=1290.00
Q3A.8	`WITH registered_items AS ( UPDATE order_items SET sku_id = $2 WHERE sku_id IS NULL AND (channel_sku = $1 OR channel_sku = $4) RETURNING order_id ) UPDATE orders o SET order_status = CASE WHEN o.manual_reason = 'sku-rule' THEN 'รอจับคู่กฎ SKU' WHEN o.manual_reason =`	rk-mlr240-sv-b, SKU-FSH-MLR-240, true, FSH-MLR-240	แก้ไข 1 แถว หลังรัน: order_id=ORD-24185, order_status=รอตรวจสอบคำสั่งซื้อ,

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
	'sku-unregistered' THEN 'รอตรวจสอบคำสั่งซื้อ' ELSE 'รอ Admin ตัดสินใจสั่งซื้อ' END, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP FROM order_items item WHERE item.order_id = o.order_id AND o.order_status = 'รอดำเนินการด้วยตนเอง' AND ((o.manual_reason = 'sku-rule' AND item.channel_sku = $1) OR (o.manual_reason = 'sku-unregistered' AND item.order_id IN (SELECT order_id FROM registered_items)) OR (o.manual_reason = 'price' AND item.sku_id = $2 AND $3::boolean)) RETURNING o.order_id		manual_reason=NULL, sku_id=SKU-FSH-MLR-240
Q3A.9	UPDATE skus SET sku_status = $2 WHERE sku_id = $1	SKU-OUT-LMP-021, active	แก้ไข 1 แถว หลังรัน: sku=OUT-LMP-021, sku_status=active
Q3A.10	SELECT EXISTS (SELECT 1 FROM order_items WHERE sku_id = $1 UNION ALL SELECT 1 FROM reorders WHERE sku_id = $1) AS has_related	SKU-FSH-MLR-240	has_related=true
Q3A.11	WITH removed_history AS (DELETE FROM cost_history WHERE sku_id = $1), removed_costs AS (DELETE FROM costs WHERE sku_id = $1), removed_stock AS (DELETE FROM stock WHERE sku_id = $1) DELETE FROM skus WHERE sku_id = $1	SKU-TMP-001	แก้ไข 1 แถว หลังรัน: remaining=0

2.23.4 Use Case 4A: ดึง Order เข้าระบบ

ตารางที่ 2-80 SQL Statement และผลการทำงานของ Use Case 4A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q4A.1	SELECT order_status, COUNT(*)::integer AS count FROM orders GROUP BY order_status ORDER BY order_status	-	order_status=จัดส่งสำเร็จ, count=2 order_status=พิมพ์ใบปะสินค้าแล้ว, count=2 order_status=ยกเลิกแล้ว, count=2 order_status=ยังไม่ได้จับคู่, count=3 (ทั้งหมด 13 แถว)
Q4A.2	SELECT TO_CHAR(order_date::date, 'YYYY-MM-DD') AS date, COUNT(*)::integer AS count FROM orders WHERE order_date >= (SELECT MAX(order_date)::date - INTERVAL '6 days' FROM orders) GROUP BY order_date::date ORDER BY order_date::date	-	date=2026-09-29, count=1 date=2026-09-30, count=5 date=2026-10-01, count=2 date=2026-10-02, count=3 (ทั้งหมด 7 แถว)
Q4A.3	SELECT order_id, order_date FROM orders ORDER BY order_date DESC, order_id DESC LIMIT 6	-	order_id=ORD-24190, order_date=2026-10-05 00:12 order_id=ORD-24194, order_date=2026-10-04 23:47 order_id=ORD-24189, order_date=2026-10-04 23:47 order_id=ORD-24188, order_date=2026-10-04 23:02 (ทั้งหมด 6 แถว)

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q4A.4	`SELECT sku_id, sku, product_name, variation FROM skus WHERE channel_sku = $1 AND sales_channel = $2 AND sku_status = 'active' ORDER BY sku_id`	rk-ydm012-3, Rakuten Ichiba	sku_id=SKU-YDM-HRB-012, sku=YDM-HRB-012, product_name=ยาดมสมุนไพรสูตรเข้มข้น, variation=แพ็ค 3 หลอด
Q4A.5	`WITH inserted_order AS (INSERT INTO orders (order_id, marketplace_order_id, sales_channel, order_date, shipping_address, shipping_method, order_status, manual_reason, created_at, updated_at) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, CURRENT_TIMESTAMP, CURRENT_TIMESTAMP) ON CONFLICT (marketplace_order_id) DO NOTHING RETURNING order_id), inserted_item AS (INSERT INTO order_items (order_item_id, channel_sku, variation, qty, fulfill_source, order_id, sku_id) SELECT $9, $10, $11, $12, $13, order_id, $14 FROM inserted_order RETURNING order_id) SELECT order_id FROM inserted_order`	ORD-3F9A1C2B7D10, 377841-20261005-0000191, Rakuten Ichiba, 2026-10-05 10:02:00+09, 東京都千代田区丸の内 1-9-1, RSL ปกติ, รอตรวจสอบคำสั่งซื้อ, NULL, ORD-3F9A1C2B7D10-ITEM-1, rk-ydm012-3, แพ็ค 3 หลอด, 3, rsl, SKU-YDM-HRB-012	แก้ไข 0 แถว หลังรัน: order_id=ORD-3F9A1C2B7D10, marketplace_order_id=377841-20261005-0000191, order_status=รอตรวจสอบคำสั่งซื้อ, channel_sku=rk-ydm012-3, qty=3, sku_id=SKU-YDM-HRB-012
Q4A.6	`UPDATE orders SET customer_cancel_requested = TRUE, updated_at = CURRENT_TIMESTAMP WHERE marketplace_order_id = $1 AND order_status NOT IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') RETURNING order_id, order_status`	377841-20261004-0000183	แก้ไข 1 แถว หลังรัน: order_id=ORD-24183, order_status=ยังไม่ได้จับคู่, customer_cancel_requested=true

2.23.5 Use Case 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock

ตารางที่ 2-81 SQL Statement และผลการทำงานของ Use Case 5A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q5A.1	`SELECT o.order_id, o.order_status, o.manual_reason, item.sku_id, sku.sku, sku.selling_price, COALESCE(cost.unit_cost, NULLIF(history.new_value ->> 'unit_cost', '')::numeric) AS unit_cost FROM orders o INNER JOIN LATERAL (SELECT * FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN LATERAL (SELECT unit_cost FROM costs WHERE sku_id = item.sku_id ORDER BY calculated_at DESC NULLS LAST, cost_id DESC LIMIT 1) cost ON TRUE LEFT JOIN LATERAL (SELECT new_value FROM cost_history WHERE sku_id = item.sku_id AND new_value ->> 'unit_cost' IS NOT NULL ORDER BY changed_at DESC, cost_history_id DESC LIMIT 1) history ON TRUE WHERE o.order_id = $1 FOR UPDATE OF o`	ORD-24184	order_id=ORD-24184, order_status=รอ Admin ตัดสินใจสั่งซื้อ, manual_reason=NULL, sku_id=SKU-FSH-RDS-180, sku=FSH-RDS-180, selling_price=2690.00, unit_cost=2462.2000
Q5A.2	`UPDATE orders SET reorder_decision = $2, decided_qty = CASE WHEN $2 = 'คุ้มค่า' THEN $3::integer ELSE NULL END, order_status = $4, manual_reason = $5, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1`	ORD-24184, คุ้มค่า, 40, รอสั่งซื้อจาก Supplier, NULL	แก้ไข 1 แถว หลังรัน: order_id=ORD-24184, order_status=รอสั่งซื้อจาก Supplier, manual_reason=NULL, reorder_decision=คุ้มค่า, decided_qty=40
Q5A.3	`UPDATE orders SET order_status = 'รอดำเนินการด้วยตนเอง', manual_reason = 'price', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1`	ORD-24171	แก้ไข 1 แถว

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
			หลังรัน: order_id=ORD-24171, order_status=รอดำเนินการด้วยตนเอง, manual_reason=price

2.23.6 Use Case 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier

ตารางที่ 2-82 SQL Statement และผลการทำงานของ Use Case 6A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q6A.1	`SELECT reorders.reorder_id, sku.sku, reorders.supplier_id, reorders.order_qty, COALESCE(reorders.ordered_at, reorders.created_at) AS order_date, reorders.expected_date, reorders.reorder_status, reorders.received_at, reorders.received_qty FROM reorders INNER JOIN skus sku ON sku.sku_id = reorders.sku_id ORDER BY COALESCE(reorders.ordered_at, reorders.created_at) DESC, reorders.reorder_id DESC`	-	reorder_id=PO-0091, sku=OUT-CHR-003, supplier_id=SUP-03, order_qty=60, order_date=2026-09-24 01:00, expected_date=2026-10-22 00:00, reorder_status=สั่งซื้อแล้ว, received_at=NULL, received_qty=NULL
Q6A.2	`SELECT supplier_id, supplier_name, contact_info, lead_time_days FROM suppliers ORDER BY supplier_name, supplier_id`	-	supplier_id=SUP-04, supplier_name=Foshan Homeware Ltd., contact_info=trade@foshan-home.example.cn, lead_time_days=18 supplier_id=SUP-01, supplier_name=Ningbo Tackle Works, contact_info=order@ningbo-tackle.example.cn, lead_time_days=21 supplier_id=SUP-03, supplier_name=Qingdao Outdoor Gear, contact_info=po@qingdao-outdoor.example.cn, lead_time_days=28 supplier_id=SUP-02, supplier_name=หจก. สมุนไพรไทยเจริญ, contact_info=sales@thaiherb.example.co.th, lead_time_days=10
Q6A.3	`SELECT sku.sku_id, sku.sku, sku.product_name, sku.variation, supplier.supplier_id, supplier.supplier_name, supplier.lead_time_days FROM skus sku LEFT JOIN suppliers supplier ON supplier.supplier_id = sku.supplier_id WHERE sku.sku = $1 FOR UPDATE OF sku`	FSH-RDS-180	sku_id=SKU-FSH-RDS-180, sku=FSH-RDS-180, product_name=คันเบ็ดสปินนิ่ง 1.8 เมตร, variation=แอ็คชันกลาง, supplier_id=SUP-01, supplier_name=Ningbo Tackle Works, lead_time_days=21
Q6A.4	`SELECT reorder_id, order_qty, supplier_id, supplier_request_id, expected_date, created_at FROM reorders WHERE sku_id = $1 AND reorder_status = 'สั่งซื้อแล้ว' AND expected_date >= CURRENT_DATE - 1 ORDER BY ordered_at DESC NULLS LAST, created_at DESC LIMIT 1`	SKU-OUT-CHR-003	reorder_id=PO-0091, order_qty=60, supplier_id=SUP-03, supplier_request_id=NULL, expected_date=2026-11-17 00:00, created_at=2026-09-24 01:00

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q6A.5	SELECT reorder_id, order_qty, supplier_id, supplier_request_id, expected_date, created_at FROM reorders WHERE sku_id = $1 AND reorder_status = 'รอส่งคำสั่งซื้อ' ORDER BY created_at DESC LIMIT 1	SKU-OUT-CHR-003	reorder_id=PO-0091, order_qty=60, supplier_id=SUP-03, supplier_request_id=SUPPLIER-req-0091, expected_date=2026-10-22 00:00, created_at=2026-09-24 01:00
Q6A.6	SELECT nextval('reorder_no_seq') AS reorder_no	-	reorder_no=92
Q6A.7	INSERT INTO reorders (reorder_id, order_qty, reorder_status, expected_date, created_at, sku_id, supplier_id, supplier_request_id) VALUES ($1, $2, 'รอส่งคำสั่งซื้อ', $3, CURRENT_TIMESTAMP, $4, $5, $6)	PO-0092, 40, 2026-11-27, SKU-FSH-RDS-180, SUP-01, SUPPLIER-req-0092	แก้ไข 1 แถว หลังรัน: reorder_id=PO-0092, order_qty=40, reorder_status=รอส่งคำสั่งซื้อ, expected_date=2026-11-27 00:00, sku_id=SKU-FSH-RDS-180, supplier_request_id=SUPPLIER-req-0092
Q6A.8	UPDATE orders o SET order_status = $2, manual_reason = $3, reorder_id = COALESCE($4, o.reorder_id), updated_at = CURRENT_TIMESTAMP WHERE EXISTS (SELECT 1 FROM order_items item WHERE item.order_id = o.order_id AND item.sku_id = $1) AND (o.order_status IN ('รอสั่งซื้อจาก Supplier', 'รอส่งคำสั่งซื้อ') OR (o.order_status = 'รอดำเนินการด้วยตนเอง' AND o.manual_reason = 'supplier'))	SKU-FSH-RDS-180, รอส่งคำสั่งซื้อ, NULL, PO-0092	แก้ไข 1 แถว หลังรัน: order_id=ORD-24184, order_status=รอส่งคำสั่งซื้อ, manual_reason=NULL, reorder_id=PO-0092
Q6A.9	UPDATE reorders SET reorder_status = $2, ordered_at = CASE WHEN $2 = 'สั่งซื้อแล้ว' THEN CURRENT_TIMESTAMP ELSE ordered_at END, supplier_reference = $3, last_error_code = $4 WHERE reorder_id = $1 AND supplier_request_id = $5 AND ($2 = 'สั่งซื้อแล้ว' OR reorder_status <> 'สั่งซื้อแล้ว') RETURNING reorder_id	PO-0091, สั่งซื้อแล้ว, MOCK-SUPPLIER-req-0091, NULL, SUPPLIER-req-0091	แก้ไข 1 แถว หลังรัน: reorder_id=PO-0091, reorder_status=สั่งซื้อแล้ว, ordered_at=2026-11-07 17:08, supplier_reference=MOCK-SUPPLIER-req-0091, last_error_code=NULL
Q6A.10	SELECT r.reorder_id, r.reorder_status, r.order_qty, sku.sku, stock.stock_id, stock.in_house_qty, stock.rsl_qty FROM reorders r INNER JOIN skus sku ON sku.sku_id = r.sku_id INNER JOIN stock ON stock.sku_id = r.sku_id WHERE r.reorder_id = $1 FOR UPDATE OF r, stock	PO-0091	reorder_id=PO-0091, reorder_status=สั่งซื้อแล้ว, order_qty=60, sku=OUT-CHR-003, stock_id=STK-OUT-CHR-003, in_house_qty=6, rsl_qty=8
Q6A.11	UPDATE reorders SET reorder_status = 'ได้รับสินค้าแล้ว', received_at = CURRENT_TIMESTAMP, received_qty = $2 WHERE reorder_id = $1 AND reorder_status = 'สั่งซื้อแล้ว' RETURNING reorder_id	PO-0091, 60	แก้ไข 1 แถว หลังรัน: reorder_id=PO-0091, reorder_status=ได้รับสินค้าแล้ว, received_at=2026-11-07 17:08, received_qty=60
Q6A.12	WITH adjusted AS (UPDATE stock SET in_house_qty = $2, rsl_qty = $3, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 RETURNING stock_id) INSERT INTO rakuten_stock_sync_states (stock_id, sync_status) SELECT stock_id, 'pending' FROM adjusted ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL	STK-OUT-CHR-003, 66, 8	แก้ไข 1 แถว หลังรัน: stock_id=STK-OUT-CHR-003, in_house_qty=66, rsl_qty=8, sync_status=pending
Q6A.13	INSERT INTO stock_adjustment_history (stock_adjustment_history_id, stock_id, previous_in_house_qty, new_in_house_qty, previous_rsl_qty, new_rsl_qty, reason, admin_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)	SAH-0102, STK-OUT-CHR-003, 6, 66, 8, 8, รับสินค้าจาก Supplier ตามคำสั่งซื้อ PO-0091, U001	แก้ไข 1 แถว หลังรัน: stock_adjustment_history_id=SAH-0102, stock_id=STK-OUT-CHR-003, previous_in_house_qty=6, new_in_house_qty=66, reason=รับสินค้าจาก Supplier ตามคำสั่งซื้อ PO-0091, admin_id=U001

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q6A.14	`UPDATE orders SET order_status = 'รอตรวจสอบสต๊อก', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE reorder_id = $1 AND order_status = 'สั่งซื้อแล้ว' RETURNING order_id`	PO-0091	แก้ไข 2 แถว หลังรัน: order_id=ORD-24195, order_status=รอตรวจสอบสต๊อก, reorder_id=PO-0091, reorder_decision=คุ้มค่า, decided_qty=60 order_id=ORD-24196, order_status=รอตรวจสอบสต๊อก, reorder_id=PO-0091, reorder_decision=คุ้มค่า, decided_qty=60

2.23.7 Use Case 7A: ยกเลิก Order

ตารางที่ 2-83 SQL Statement และผลการทำงานของ Use Case 7A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q7A.1	`SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o`	ORD-24183	order_id=ORD-24183, marketplace_order_id=377841-20261004-0000183, sales_channel=Rakuten Ichiba, order_status=ยังไม่ได้จับคู่, manual_reason=NULL, shipping_address=神奈川県横浜市港北区新横浜 2-5-14, shipping_method=RSL ปกติ, rsl_no=NULL, cancelled_at=NULL, customer_cancel_requested=false, channel_sku=rk-mlr240-sv, variation=สีเงินลายปลาชิว, qty=4, fulfill_source=rsl, stock_deducted=false, sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, label_printed=false, reorder_id=NULL, reorder_status=NULL
Q7A.2	`UPDATE orders SET order_status = $2, cancel_reason = $3, cancelled_at = $4, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1`	ORD-24183, ยกเลิกแล้ว, ลูกค้าขอยกเลิกผ่าน Rakuten, 2026-10-05 10:00:00+09	แก้ไข 1 แถว หลังรัน: order_id=ORD-24183, order_status=ยกเลิกแล้ว, manual_reason=NULL, cancel_reason=ลูกค้าขอยกเลิกผ่าน Rakuten, cancelled_at=2026-10-05 01:00
Q7A.3	`WITH restored AS (UPDATE stock SET in_house_qty = in_house_qty + CASE WHEN $3 = 'in_house' THEN $4 ELSE 0 END, rsl_qty = CASE WHEN $3 = 'rsl' THEN COALESCE(rsl_qty, 0) + $4 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE sku_id = $2 AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = TRUE) RETURNING stock_id) UPDATE order_items SET stock_deducted = FALSE, deducted_at = NULL WHERE order_id = $1 AND EXISTS (SELECT 1 FROM restored) RETURNING qty`	ORD-24172, SKU-FSH-RDS-180, rsl, 1	แก้ไข 1 แถว หลังรัน: stock_id=STK-FSH-RDS-180, in_house_qty=0, rsl_qty=2, stock_deducted=false

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q7A.4	`UPDATE reorders SET reorder_status = 'ยกเลิกแล้ว' WHERE reorder_id = $1 AND reorder_status IN ('รอส่งค่าสั่งซื้อ', 'สั่งซื้อแล้ว') RETURNING reorder_id`	PO-0091	แก้ไข 1 แถว หลังรัน: reorder_id=PO-0091, reorder_status=ยกเลิกแล้ว
Q7A.5	`UPDATE orders SET order_status = 'รอ Admin ตัดสินใจสั่งซื้อ', manual_reason = NULL, reorder_id = NULL, reorder_decision = NULL, decided_qty = NULL, updated_at = CURRENT_TIMESTAMP WHERE reorder_id = $1 AND order_id <> $2 AND order_status IN ('รอส่งค่าสั่งซื้อ', 'สั่งซื้อแล้ว')`	PO-0091, ORD-24195	แก้ไข 1 แถว หลังรัน: order_id=ORD-24195, order_status=สั่งซื้อแล้ว, reorder_id=PO-0091, reorder_decision=คุ้มค่า, decided_qty=60 order_id=ORD-24196, order_status=รอ Admin ตัดสินใจสั่งซื้อ, reorder_id=NULL, reorder_decision=NULL, decided_qty=NULL

2.23.8 Use Case 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery

ตารางที่ 2-84 SQL Statement และผลการทำงานของ Use Case 8A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q8A.1	`SELECT o.order_id, o.order_status, o.tracking_number, o.carrier_name, o.delivery_status, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id) AND NOT EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NULL) AS labels_printed FROM orders o WHERE o.order_id = $1 FOR UPDATE OF o`	ORD-24178	order_id=ORD-24178, order_status=พิมพ์ใบปะสินค้าแล้ว, tracking_number=NULL, carrier_name=NULL, delivery_status=NULL, labels_printed=true
Q8A.2	`UPDATE orders SET tracking_number = $2, carrier_name = $3, dispatched_at = CURRENT_TIMESTAMP, delivery_status = 'in_transit', tracking_notified = FALSE, tracking_notified_at = NULL, order_status = 'อยู่ระหว่างจัดส่ง', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status IN ('พิมพ์ใบปะสินค้าแล้ว', 'รอส่งมอบ')`	ORD-24178, 4418-2291-4178, Yamato Transport	แก้ไข 1 แถว หลังรัน: order_id=ORD-24178, order_status=อยู่ระหว่างจัดส่ง, tracking_number=4418-2291-4178, carrier_name=Yamato Transport, delivery_status=in_transit, tracking_notified=false
Q8A.3	`SELECT o.order_id, o.order_status, o.tracking_number, o.delivery_status, o.tracking_notified, item.stock_deducted, item.sku_id, item.fulfill_source, item.qty FROM orders o INNER JOIN LATERAL (SELECT * FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE WHERE o.tracking_number IS NOT NULL AND o.order_status IN ('อยู่ระหว่างจัดส่ง', 'รอแจ้งเลขติดตาม', 'แจ้งเลขติดตามแล้ว', 'จัดส่งสำเร็จ', 'รอดำเนินการพิเศษ') ORDER BY o.order_id`	-	order_id=ORD-23012, order_status=จัดส่งสำเร็จ, tracking_number=4418-1001-0012, delivery_status=delivered, tracking_notified=true, stock_deducted=true, sku_id=SKU-FSH-MLR-240, fulfill_source=rsl, qty=2 order_id=ORD-24165, order_status=อยู่ระหว่างจัดส่ง, tracking_number=4418-2290-1106, delivery_status=in_transit, tracking_notified=false, stock_deducted=false, sku_id=SKU-OUT-LMP-021, fulfill_source=rsl, qty=1 order_id=ORD-24166, order_status=แจ้งเลขติดตามแล้ว, tracking_number=4418-2290-1105, delivery_status=in_transit,

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
			tracking_notified=true, stock_deducted=true, sku_id=SKU-OUT-LMP-021, fulfill_source=rsl, qty=1 order_id=ORD-24168, order_status=จัดส่งสำเร็จ, tracking_number=4418-2290-0168, delivery_status=delivered, tracking_notified=true, stock_deducted=true, sku_id=SKU-YDM-HRB-012, fulfill_source=in_house, qty=2 (ทั้งหมด 7 แถว)
Q8A.4	`UPDATE orders SET delivery_status = $2, order_status = CASE WHEN $2 = 'failed' THEN 'จัดส่งไม่สำเร็จ' WHEN $2 = 'returned' THEN 'ตีกลับ/คืนสินค้า' WHEN $2 = 'delivered' AND tracking_notified = TRUE AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE) THEN 'จัดส่งสำเร็จ' ELSE order_status END, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND (delivery_status IS DISTINCT FROM 'delivered' OR $2 <> 'in_transit')`	ORD-24174, failed	แก้ไข 1 แถว หลังรัน: order_id=ORD-24174, order_status=จัดส่งไม่สำเร็จ, delivery_status=failed
Q8A.5	`UPDATE orders SET order_status = 'จัดส่งสำเร็จ', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'แจ้งเลขติดตามแล้ว' AND tracking_notified = TRUE AND delivery_status = 'delivered' AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE)`	ORD-24172	แก้ไข 1 แถว หลังรัน: order_id=ORD-24172, order_status=จัดส่งสำเร็จ, delivery_status=delivered, tracking_notified=true
Q8A.6	`UPDATE orders SET order_status = 'รอส่งมอบ', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'พิมพ์ใบปะสินค้าแล้ว'`	ORD-24177	แก้ไข 1 แถว หลังรัน: order_id=ORD-24177, order_status=รอส่งมอบ
Q8A.7	`WITH restored AS (UPDATE stock SET in_house_qty = in_house_qty + CASE WHEN $3 = 'in_house' THEN $4 ELSE 0 END, rsl_qty = CASE WHEN $3 = 'rsl' THEN COALESCE(rsl_qty, 0) + $4 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE sku_id = $2 AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = TRUE) RETURNING stock_id) UPDATE order_items SET stock_deducted = FALSE, deducted_at = NULL WHERE order_id = $1 AND EXISTS (SELECT 1 FROM restored) RETURNING qty`	ORD-24172, SKU-FSH-RDS-180, rsl, 1	แก้ไข 1 แถว หลังรัน: sku_id=SKU-FSH-RDS-180, in_house_qty=0, rsl_qty=2, stock_deducted=false

2.23.9 Use Case 9A: พิมพ์ใบปะสินค้า

ตารางที่ 2-85 SQL Statement และผลการทำงานของ Use Case 9A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q9A.1	`SELECT order_id, MAX(label_template) AS label_template, MAX(parcel_total)::integer AS parcel_total, MAX(printed_at) AS printed_at FROM labels GROUP BY order_id ORDER BY order_id`	-	order_id=ORD-23012, label_template=RSL-STD-A6, parcel_total=1, printed_at=NULL

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
			order_id=ORD-24165, label_template=RSL-STD-A6, parcel_total=1, printed_at=2026-09-30 08:00 order_id=ORD-24166, label_template=RSL-STD-A6, parcel_total=1, printed_at=2026-09-30 08:00 order_id=ORD-24168, label_template=INHOUSE-YAMATO-A6, parcel_total=1, printed_at=NULL (ทั้งหมด 11 แถว)
Q9A.2	`SELECT l.label_id, l.order_id, l.label_template, l.rsl_no, l.parcel_seq, l.parcel_total, o.order_status, o.shipping_address, o.shipping_method FROM labels l INNER JOIN orders o ON o.order_id = l.order_id WHERE l.order_id = $1 ORDER BY l.parcel_seq`	ORD-24181	label_id=ORD-24181-LABEL-1, order_id=ORD-24181, label_template=RSL-STD-A6, rsl_no=RSL-9f21a7, parcel_seq=1, parcel_total=1, order_status=รอพิมพ์ใบปะสินค้า, shipping_address=静岡県浜松市中区砂山町 320-1, shipping_method=RSL ปกติ
Q9A.3	`INSERT INTO label_print_history ( label_print_history_id, print_attempt_id, label_id, admin_id, is_reprint, outcome, error_code ) VALUES ($1, $2, $3, $4, $5, $6, $7) ON CONFLICT (print_attempt_id, label_id) DO NOTHING`	LPH-T1, PRINT-T1, ORD-24181-LABEL-1, U001, false, printed, NULL	แก้ไข 1 แถว หลังรัน: label_print_history_id=LPH-T1, print_attempt_id=PRINT-T1, label_id=ORD-24181-LABEL-1, admin_id=U001, is_reprint=false, outcome=printed
Q9A.4	`WITH updated_labels AS ( UPDATE labels SET label_status = 'printed', printed_at = CURRENT_TIMESTAMP WHERE order_id = $1 ) UPDATE orders SET order_status = 'พิมพ์ใบปะสินค้าแล้ว', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอพิมพ์ใบปะสินค้า'`	ORD-24181	แก้ไข 1 แถว หลังรัน: label_id=ORD-24181-LABEL-1, label_status=printed, printed_at=2026-11-07 17:08, order_status=พิมพ์ใบปะสินค้าแล้ว

2.23.10 Use Case 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว (Order เก่าที่เสร็จสิ้น / Cost / Label / Reorder)

ตารางที่ 2-86 SQL Statement และผลการทำงานของ Use Case 10A

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q10A.1	`WITH eligible AS MATERIALIZED (SELECT o.order_id, jsonb_build_object('order', to_jsonb(o), 'items', COALESCE((SELECT jsonb_agg(to_jsonb(i)) FROM order_items i WHERE i.order_id = o.order_id), '[]'::jsonb), 'labels', COALESCE((SELECT jsonb_agg(to_jsonb(l)) FROM labels l WHERE l.order_id = o.order_id), '[]'::jsonb)) AS snapshot FROM orders o WHERE o.order_id = $1 AND o.order_status IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') FOR UPDATE), logged AS (INSERT INTO deletion_history (deletion_history_id, order_reference,`	ORD-23012, DEL-0101, U001	แก้ไข 0 แถว หลังรัน: deletion_history_id=DEL-0101, order_reference=ORD-23012, admin_id=U001, order_left=0, labels_left=0

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
	order_snapshot, admin_id) SELECT $2, order_id, snapshot, $3 FROM eligible RETURNING order_reference), detached_cost_history AS (UPDATE cost_history SET cost_id = NULL WHERE cost_id IN (SELECT cost_id FROM costs WHERE order_id IN (SELECT order_reference FROM logged)) RETURNING cost_history_id), deleted_costs AS (DELETE FROM costs WHERE order_id IN (SELECT order_reference FROM logged) AND (SELECT COUNT() FROM detached_cost_history) >= 0 RETURNING cost_id), deleted AS (DELETE FROM orders WHERE order_id IN (SELECT order_reference FROM logged) AND (SELECT COUNT() FROM deleted_costs) >= 0 RETURNING order_id) SELECT order_id FROM deleted		
Q10A.2	DELETE FROM labels l USING orders o WHERE l.order_id = $1 AND o.order_id = l.order_id AND o.order_status IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว') RETURNING l.label_id	ORD-24168	แก้ไข 1 แถว หลังรัน: labels_left=0
Q10A.3	WITH sku_row AS (SELECT sku_id FROM skus WHERE sku = $1), deleted_history AS (DELETE FROM cost_history h USING sku_row s WHERE h.sku_id = s.sku_id AND h.changed_at = $2::timestamptz RETURNING h.cost_history_id) SELECT 'history' AS kind FROM deleted_history	FSH-MLR-240, 2026-10-02 04:20:00+00	แก้ไข 0 แถว หลังรัน: cost_history_id=CH-0002, sku_id=SKU-OUT-LMP-021, changed_at=2026-10-03 05:00
Q10A.4	DELETE FROM reorders r WHERE r.reorder_id = $1 AND (r.reorder_status IN ('ยกเลิกแล้ว', 'ได้รับสินค้าแล้ว') OR (r.reorder_status = 'สั่งซื้อแล้ว' AND r.expected_date < CURRENT_DATE)) AND NOT EXISTS (SELECT 1 FROM orders o WHERE o.reorder_id = r.reorder_id AND o.order_status NOT IN ('จัดส่งสำเร็จ', 'ยกเลิกแล้ว')) RETURNING r.reorder_id	PO-0091	แก้ไข 0 แถว หลังรัน: reorders_left=1
Q10A.5	SELECT order_reference, admin_id, deleted_at FROM deletion_history ORDER BY deleted_at DESC	-	order_reference=ORD-22950, admin_id=U001, deleted_at=2026-10-01 02:00 order_reference=ORD-22951, admin_id=U001, deleted_at=2026-10-01 02:00

2.23.11 Use Case 1S: จับคู่ Order กับเลข RSL

ตารางที่ 2-87 SQL Statement และผลการทำงานของ Use Case 1S

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q1S.1	SELECT o.order_id FROM orders o WHERE (o.order_status = 'ยังไม่ได้จับคู่' OR (o.order_status = 'รอดำเนินการด้วยตนเอง' AND o.manual_reason = 'rsl')) AND EXISTS (SELECT 1 FROM order_items oi WHERE oi.order_id = o.order_id AND oi.fulfill_source = 'rsl') ORDER BY o.order_date, o.order_id	-	order_id=ORD-24175 order_id=ORD-24182 order_id=ORD-24183
Q1S.2	SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested,	ORD-24183	order_id=ORD-24183, marketplace_order_id=377841-20261004-0000183,

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
	item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o		sales_channel=Rakuten Ichiba, order_status=ยังไม่ได้จับคู่, manual_reason=NULL, shipping_address=神奈川県横浜市港北区新横浜 2-5-14, shipping_method=RSL ปกติ, rsl_no=NULL, cancelled_at=NULL, customer_cancel_requested=false, channel_sku=rk-mlr240-sv, variation=สีเงินลายปลาซิว, qty=4, fulfill_source=rsl, stock_deducted=false, sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, label_printed=false, reorder_id=NULL, reorder_status=NULL
Q1S.3	UPDATE orders SET rsl_no = $2, rsl_matched_at = CASE WHEN $2::text IS NULL THEN NULL ELSE CURRENT_TIMESTAMP END, order_status = $3, manual_reason = $4, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1	ORD-24183, RSL-9f2240, รอจัดรูปแบบใบปะสินค้า, NULL	แก้ไข 1 แถว หลังรัน: order_id=ORD-24183, order_status=รอจัดรูปแบบใบปะสินค้า, rsl_no=RSL-9f2240, rsl_matched_at=2026-11-07 17:08, manual_reason=NULL
Q1S.4	WITH eligible AS (SELECT order_id FROM orders WHERE order_id = $1 AND rsl_no IS NOT NULL AND (order_status IN ('รอจัดรูปแบบใบปะสินค้า', 'รอพิมพ์ใบปะสินค้า') OR (order_status = 'รอดำเนินการด้วยตนเอง' AND rsl_no IS NOT NULL))), removed_labels AS (DELETE FROM labels WHERE order_id IN (SELECT order_id FROM eligible)) UPDATE orders SET rsl_no = NULL, rsl_matched_at = NULL, order_status = 'ยังไม่ได้จับคู่', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id IN (SELECT order_id FROM eligible) RETURNING order_id	ORD-24181	แก้ไข 1 แถว หลังรัน: order_id=ORD-24181, order_status=ยังไม่ได้จับคู่, rsl_no=NULL, labels=0

2.23.12 Use Case 2S: จับคู่กฎ SKU

ตารางที่ 2-88 SQL Statement และผลการทำงานของ Use Case 2S

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q2S.1	SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT	ORD-24190	order_id=ORD-24190, marketplace_order_id=377841-20261005-0000190, sales_channel=Rakuten Ichiba, order_status=รอตรวจสอบคำสั่งซื้อ, manual_reason=NULL, shipping_address=東京都杉並区高円寺南 3-14-7 サンハイツ402, shipping_method=RSL ปกติ, rsl_no=NULL, cancelled_at=NULL, customer_cancel_requested=false,

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
	`JOIN reorders reorder ON reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o`		channel_sku=rk-mlr240-sv, variation=สีเงินลายปลาชิว, qty=2, fulfill_source=rsl, stock_deducted=false, sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, label_printed=false, reorder_id=NULL, reorder_status=NULL
Q2S.2	`SELECT sku_id, sku, product_name, variation FROM skus WHERE channel_sku = $1 AND sales_channel = $2 AND sku_status = 'active' ORDER BY sku_id`	rk-mlr240-sv, Rakuten Ichiba	sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, product_name=เหยื่อปลอมมินโนว์ จมช้า 24 กรัม, variation=สีเงินลายปลาชิว
Q2S.3	`WITH matched_item AS ( UPDATE order_items SET sku_id = $2 WHERE order_id = $1 ) UPDATE orders SET order_status = $3, manual_reason = $4, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจับคู่กฎ SKU'`	ORD-24190, SKU-FSH-MLR-240, รอตรวจสอบสต๊อก, NULL	แก้ไข 1 แถว หลังรัน: order_id=ORD-24190, order_status=รอตรวจสอบสต๊อก, manual_reason=NULL, sku_id=SKU-FSH-MLR-240, fulfill_source=rsl, stock_deducted=false
Q2S.4	`INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, $2, $3, $4, $5)`	AEL-0101, 2S, ORD-24190, external_unavailable, externalUnavailable	แก้ไข 1 แถว หลังรัน: auto_error_log_id=AEL-0001, use_case_id=2S, context_key=ORD-24185, error_code=external_unavailable auto_error_log_id=AEL-0101, use_case_id=2S, context_key=ORD-24190, error_code=external_unavailable
Q2S.5	`SELECT order_id FROM orders WHERE order_status = 'รอจับคู่กฎ SKU' ORDER BY order_date, order_id LIMIT 100 FOR UPDATE SKIP LOCKED`	-	order_id=ORD-24185 order_id=ORD-24190

2.23.13 Use Case 3S: ตรวจสอบของสินค้าใน stock + RSL

ตารางที่ 2-89 SQL Statement และผลการทำงานของ Use Case 3S

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q3S.1	`SELECT stock.stock_id, stock.sku_id, stock.in_house_qty, stock.rsl_qty FROM stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1 FOR UPDATE OF stock`	FSH-MLR-240	stock_id=STK-FSH-MLR-240, sku_id=SKU-FSH-MLR-240, in_house_qty=52, rsl_qty=96
Q3S.2	`SELECT stock.rsl_qty FROM stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1`	FSH-MLR-240	rsl_qty=96
Q3S.3	`SELECT COALESCE(SUM(item.qty) FILTER (WHERE item.fulfill_source = 'rsl'), 0) AS rsl_reserved, COALESCE(SUM(item.qty) FILTER (WHERE item.fulfill_source = 'in_house'), 0) AS in_house_reserved FROM order_items item INNER JOIN orders ON orders.order_id = item.order_id WHERE item.sku_id = $1 AND item.stock_deducted = FALSE AND orders.order_status = ANY($2::text[]) AND orders.order_id <> $3`	SKU-FSH-MLR-240, {ยังไม่ได้จับคู่,รอจัดรูปแบบใบปะสินค้า,รอพิมพ์ใบปะสินค้า,พิมพ์ใบปะสินค้าแล้ว,รอส่งมอบ,อยู่ระหว่างจัดส่ง,รอแจ้งเลขติดตาม,แจ้งเลขติดตามแล้ว}, ORD-24190	rsl_reserved=10, in_house_reserved=0

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q3S.4	`WITH latest_stock AS (UPDATE stock SET rsl_qty = COALESCE($2, rsl_qty), updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1), item_source AS (UPDATE order_items SET fulfill_source = COALESCE($5, fulfill_source) WHERE order_id = $3 AND EXISTS (SELECT 1 FROM orders WHERE order_id = $3 AND order_status = 'รอตรวจสอบสต๊อก')) UPDATE orders SET order_status = $4, manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $3 AND order_status = 'รอตรวจสอบสต๊อก'`	STK-FSH-MLR-240, 96, ORD-24190, ยังไม่ได้จับคู่, rsl	แก้ไข 1 แถว หลังรัน: order_id=ORD-24190, order_status=ยังไม่ได้จับคู่, manual_reason=NULL, sku_id=SKU-FSH-MLR-240, fulfill_source=rsl, stock_deducted=false
Q3S.5	`INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, $2, $3, $4, $5)`	AEL-0102, 3S, ORD-24186, external_unavailable, externalUnavailable	แก้ไข 1 แถว หลังรัน: auto_error_log_id=AEL-0003, use_case_id=3S, context_key=ORD-24176, error_code=stock_missing auto_error_log_id=AEL-0102, use_case_id=3S, context_key=ORD-24186, error_code=external_unavailable
Q3S.6	`SELECT sku.sku, stock.in_house_qty, stock.rsl_qty, stock.updated_at FROM stock INNER JOIN skus sku ON sku.sku_id = stock.sku_id ORDER BY sku.sku`	-	sku=FSH-MLR-240, in_house_qty=52, rsl_qty=96, updated_at=2026-10-04 21:00 sku=FSH-RDS-180, in_house_qty=0, rsl_qty=1, updated_at=2026-10-04 21:00 sku=HOM-KIT-108, in_house_qty=2, rsl_qty=3, updated_at=2026-10-04 21:00 sku=HOM-STG-045, in_house_qty=1, rsl_qty=0, updated_at=2026-10-04 21:00 (ทั้งหมด 7 แถว)
Q3S.7	`SELECT stock.stock_id, stock.sku_id, stock.in_house_qty, stock.rsl_qty FROM stock INNER JOIN skus ON skus.sku_id = stock.sku_id WHERE skus.sku = $1 FOR UPDATE OF stock`	FSH-MLR-240	stock_id=STK-FSH-MLR-240, sku_id=SKU-FSH-MLR-240, in_house_qty=52, rsl_qty=96
Q3S.8	`WITH adjusted AS (UPDATE stock SET in_house_qty = $2, rsl_qty = $3, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 RETURNING stock_id) INSERT INTO rakuten_stock_sync_states (stock_id, sync_status) SELECT stock_id, 'pending' FROM adjusted ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL`	STK-FSH-MLR-240, 50, 96	แก้ไข 1 แถว หลังรัน: stock_id=STK-FSH-MLR-240, in_house_qty=50, rsl_qty=96, sync_status=pending
Q3S.9	`INSERT INTO stock_adjustment_history ( stock_adjustment_history_id, stock_id, previous_in_house_qty, new_in_house_qty, previous_rsl_qty, new_rsl_qty, reason, admin_id ) VALUES ($1, $2, $3, $4, $5, $6, $7, $8)`	SAH-0101, STK-FSH-MLR-240, 52, 50, 96, 96, ตรวจนับสินค้าจริงประจำเดือน, U001	แก้ไข 1 แถว หลังรัน: stock_adjustment_history_id=SAH-0101, stock_id=STK-FSH-MLR-240, previous_in_house_qty=52, new_in_house_qty=50, reason=ตรวจนับสินค้าจริงประจำเดือน, admin_id=U001

2.23.14 Use Case 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง

ตารางที่ 2-90 SQL Statement และผลการทำงานของ Use Case 4S

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q4S.1	`SELECT sku.sku, costs.purchase_price, costs.currency, costs.exchange_rate, costs.exchange_rate_updated_at, costs.intl_freight, costs.duty_fee, costs.marketplace_fee, costs.domestic_shipping, costs.rsl_charge, costs.order_qty, costs.unit_cost, costs.calculated_at FROM costs INNER JOIN skus sku ON sku.sku_id = costs.sku_id ORDER BY sku.sku, costs.calculated_at NULLS LAST, costs.cost_id`	-	sku=FSH-MLR-240, purchase_price=9100.0000, currency=CNY, exchange_rate=4.920000, exchange_rate_updated_at=2026-11-06 01:07, intl_freight=8600.0000, duty_fee=3150.0000, marketplace_fee=72.0000, domestic_shipping=46.0000, rsl_charge=18.0000, order_qty=200, unit_cost=418.6100, calculated_at=2026-11-06 03:07 sku=FSH-RDS-180, purchase_price=16400.0000, currency=CNY, exchange_rate=4.920000, exchange_rate_updated_at=2026-11-06 01:07, intl_freight=5400.0000, duty_fee=2080.0000, marketplace_fee=148.0000, domestic_shipping=92.0000, rsl_charge=18.0000, order_qty=40, unit_cost=2462.2000, calculated_at=2026-11-06 03:07 sku=HOM-KIT-108, purchase_price=3000.0000, currency=CNY, exchange_rate=4.920000, exchange_rate_updated_at=2026-11-06 01:07, intl_freight=3000.0000, duty_fee=900.0000, marketplace_fee=60.0000, domestic_shipping=45.0000, rsl_charge=18.0000, order_qty=20, unit_cost=1056.0000, calculated_at=2026-11-06 03:07 sku=HOM-STG-045, purchase_price=8400.0000, currency=CNY, exchange_rate=4.920000, exchange_rate_updated_at=2026-11-06 01:07, intl_freight=6800.0000, duty_fee=2100.0000, marketplace_fee=64.0000, domestic_shipping=58.0000, rsl_charge=18.0000, order_qty=120, unit_cost=558.5667, calculated_at=2026-11-06 03:07 (ทั้งหมด 7 แถว)

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q4S.2	SELECT sku.sku, history.new_value, history.admin_id, history.changed_at FROM cost_history history INNER JOIN skus sku ON sku.sku_id = history.sku_id ORDER BY history.changed_at, history.cost_history_id	-	sku=FSH-MLR-240, new_value=[object Object], admin_id=U001, changed_at=2026-10-02 04:20 sku=OUT-LMP-021, new_value=[object Object], admin_id=U001, changed_at=2026-10-03 05:00
Q4S.3	SELECT sku_id, sku, channel_sku FROM skus WHERE sku = $1 FOR UPDATE	FSH-MLR-240	sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, channel_sku=rk-mlr240-sv
Q4S.4	SELECT costs.cost_id, to_jsonb(costs) AS snapshot, costs.exchange_rate, costs.exchange_rate_updated_at FROM costs WHERE costs.sku_id = $1 ORDER BY costs.calculated_at DESC NULLS LAST, costs.cost_id LIMIT 1 FOR UPDATE	SKU-FSH-MLR-240	cost_id=COST-FSH-MLR-240, snapshot=[object Object], exchange_rate=4.920000, exchange_rate_updated_at=2026-11-06 01:07
Q4S.5	INSERT INTO costs (cost_id, purchase_price, currency, exchange_rate, exchange_rate_updated_at, intl_freight, duty_fee, marketplace_fee, domestic_shipping, rsl_charge, order_qty, unit_cost, calculated_at, sku_id) VALUES ($1, $2, $3, $4, $5, $6, $7, $8, $9, $10, $11, $12, CURRENT_TIMESTAMP, $13) ON CONFLICT (cost_id) DO UPDATE SET purchase_price = EXCLUDED.purchase_price, currency = EXCLUDED.currency, exchange_rate = EXCLUDED.exchange_rate, exchange_rate_updated_at = EXCLUDED.exchange_rate_updated_at, intl_freight = EXCLUDED.intl_freight, duty_fee = EXCLUDED.duty_fee, marketplace_fee = EXCLUDED.marketplace_fee, domestic_shipping = EXCLUDED.domestic_shipping, rsl_charge = EXCLUDED.rsl_charge, order_qty = EXCLUDED.order_qty, unit_cost = EXCLUDED.unit_cost, calculated_at = CURRENT_TIMESTAMP RETURNING cost_id	COST-FSH-MLR-240, 9100, CNY, 4.92, 2026-11-05 20:18:24+00, 8600, 3150, 72, 46, 18, 200, 418.61, SKU-FSH-MLR-240	แก้ไข 1 แถว หลังรัน: cost_id=COST-FSH-MLR-240, unit_cost=418.6100, unit_cost_by_formula=418.61, calculated_at=2026-11-07 17:08
Q4S.6	INSERT INTO cost_history (cost_history_id, sku_id, cost_id, previous_value, new_value, admin_id) VALUES ($1, $2, $3, $4::jsonb, $5::jsonb, $6)	COST-HISTORY-T1, SKU-FSH-MLR-240, COST-FSH-MLR-240, {"unit_cost": 418.61}, {"unit_cost": 418.61}, U001	แก้ไข 1 แถว หลังรัน: cost_history_id=COST-HISTORY-T1, sku_id=SKU-FSH-MLR-240, cost_id=COST-FSH-MLR-240, new_value=[object Object], admin_id=U001
Q4S.7	SELECT selling_price FROM skus WHERE sku_id = $1	SKU-FSH-MLR-240	selling_price=1280.00

2.23.15 Use Case 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace

ตารางที่ 2-91 SQL Statement และผลการทำงานของ Use Case 5S

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q5S.1	SELECT order_id, order_status, marketplace_order_id, tracking_number, carrier_name, tracking_notified, delivery_status FROM orders WHERE order_id = $1 FOR UPDATE	ORD-24174	order_id=ORD-24174, order_status=อยู่ระหว่างจัดส่ง, marketplace_order_id=377841-20261001-0000174,

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
			tracking_number=4418-2290-1102, carrier_name=Yamato Transport, tracking_notified=false, delivery_status=in_transit
Q5S.2	`SELECT attempt_id FROM tracking_notification_logs WHERE order_id = $1 AND tracking_number = $2 AND outcome IN ('pending', 'failed') ORDER BY sent_at DESC LIMIT 1`	ORD-24174, 4418-2290-1102	attempt_id=NOTIFY-T0
Q5S.3	`INSERT INTO tracking_notification_logs (tracking_notification_log_id, attempt_id, order_id, tracking_number, outcome, sent_at, admin_id, error_code) VALUES ($1, $2, $3, $4, 'pending', CURRENT_TIMESTAMP, $5, NULL) ON CONFLICT (attempt_id) DO UPDATE SET outcome = 'pending', sent_at = CURRENT_TIMESTAMP, admin_id = EXCLUDED.admin_id, error_code = NULL`	TNL-T1, NOTIFY-T1, ORD-24174, 4418-2290-1102, U001	แก้ไข 1 แถว หลังรัน: tracking_notification_log_id=TNL-T1, attempt_id=NOTIFY-T1, order_id=ORD-24174, outcome=pending, admin_id=U001
Q5S.4	`WITH finished AS (UPDATE tracking_notification_logs SET outcome = $2, error_code = $3, sent_at = CURRENT_TIMESTAMP WHERE attempt_id = $1 RETURNING order_id) UPDATE orders o SET tracking_notified = CASE WHEN $2 = 'sent' THEN TRUE ELSE tracking_notified END, tracking_notified_at = CASE WHEN $2 = 'sent' THEN CURRENT_TIMESTAMP ELSE tracking_notified_at END, order_status = CASE WHEN $2 <> 'sent' AND tracking_notified = FALSE THEN 'รอแจ้งเลขติดตาม' WHEN $2 = 'sent' AND delivery_status = 'delivered' AND EXISTS (SELECT 1 FROM order_items WHERE order_id = o.order_id) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = o.order_id AND stock_deducted = FALSE) THEN 'จัดส่งสำเร็จ' WHEN $2 = 'sent' AND tracking_notified = FALSE THEN 'แจ้งเลขติดตามแล้ว' ELSE order_status END, updated_at = CURRENT_TIMESTAMP FROM finished WHERE o.order_id = finished.order_id`	NOTIFY-T0, sent, NULL	แก้ไข 1 แถว หลังรัน: order_id=ORD-24174, order_status=แจ้งเลขติดตามแล้ว, tracking_notified=true, tracking_notified_at=2026-11-07 17:08

2.23.16 Use Case 6S: จัดรูปแบบใบปะสินค้า

ตารางที่ 2-92 SQL Statement และผลการทำงานของ Use Case 6S

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q6S.1	`SELECT o.order_id, o.marketplace_order_id, o.sales_channel, o.order_status, o.manual_reason, o.shipping_address, o.shipping_method, o.rsl_no, o.cancelled_at, o.customer_cancel_requested, item.channel_sku, item.variation, item.qty, item.fulfill_source, item.stock_deducted, item.sku_id, sku.sku, EXISTS (SELECT 1 FROM labels l WHERE l.order_id = o.order_id AND l.printed_at IS NOT NULL) AS label_printed, reorder.reorder_id, reorder.reorder_status FROM orders o INNER JOIN LATERAL (SELECT oi.* FROM order_items oi WHERE oi.order_id = o.order_id ORDER BY oi.order_item_id LIMIT 1) item ON TRUE LEFT JOIN skus sku ON sku.sku_id = item.sku_id LEFT JOIN reorders reorder ON`	ORD-24183	order_id=ORD-24183, marketplace_order_id=377841-20261004-0000183, sales_channel=Rakuten Ichiba, order_status=ยังไม่ได้จับคู่, manual_reason=NULL, shipping_address=神奈川県横浜市港北区新横浜 2-5-14, shipping_method=RSL ปกติ, rsl_no=NULL, cancelled_at=NULL, customer_cancel_requested=false, channel_sku=rk-mlr240-sv, variation=สี

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
	`reorder.reorder_id = o.reorder_id WHERE o.order_id = $1 FOR UPDATE OF o`		เงินลายปลาซิว, qty=4, fulfill_source=rsl, stock_deducted=false, sku_id=SKU-FSH-MLR-240, sku=FSH-MLR-240, label_printed=false, reorder_id=NULL, reorder_status=NULL
Q6S.2	`SELECT o.order_id, o.sales_channel, o.rsl_no, o.shipping_address, o.shipping_method, item.qty, item.fulfill_source, o.order_status FROM orders o INNER JOIN LATERAL (SELECT qty, fulfill_source FROM order_items WHERE order_id = o.order_id ORDER BY order_item_id LIMIT 1) item ON TRUE WHERE o.order_status = 'รอจัดรูปแบบใบปะสินค้า' ORDER BY o.order_date, o.order_id`	-	order_id=ORD-24183, sales_channel=Rakuten Ichiba, rsl_no=RSL-9f2240, shipping_address=神奈川県横浜市港北区新横浜 2-5-14, shipping_method=RSL ปกติ, qty=4, fulfill_source=rsl, order_status=รอจัดรูปแบบใบปะสินค้า
Q6S.3	`UPDATE orders SET order_status = 'รอดำเนินการด้วยตนเอง', manual_reason = $2, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจัดรูปแบบใบปะสินค้า'`	ORD-24183, template	แก้ไข 1 แถว หลังรัน: order_id=ORD-24183, order_status=รอดำเนินการด้วยตนเอง, manual_reason=template
Q6S.4	`INSERT INTO labels (label_id, label_template, rsl_no, parcel_seq, parcel_total, label_status, created_at, order_id) SELECT $1, $2, $3, $4, $5, 'ready', CURRENT_TIMESTAMP, o.order_id FROM orders o WHERE o.order_id = $6 AND o.order_status = 'รอจัดรูปแบบใบปะสินค้า' ON CONFLICT (order_id, parcel_seq) DO NOTHING`	ORD-24183-LABEL-1, RSL-STD-A6, RSL-9f2240, 1, 1, ORD-24183	แก้ไข 1 แถว หลังรัน: label_id=ORD-24183-LABEL-1, label_template=RSL-STD-A6, rsl_no=RSL-9f2240, parcel_seq=1, parcel_total=1, label_status=ready
Q6S.5	`UPDATE orders SET order_status = 'รอพิมพ์ใบปะสินค้า', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'รอจัดรูปแบบใบปะสินค้า' AND EXISTS (SELECT 1 FROM labels WHERE order_id = $1)`	ORD-24183	แก้ไข 1 แถว หลังรัน: order_id=ORD-24183, order_status=รอพิมพ์ใบปะสินค้า
Q6S.6	`INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, '6S', $2, $3, 'Label service could not generate labels')`	AEL-T1, ORD-24183, invalid_response	แก้ไข 1 แถว หลังรัน: use_case_id=6S, context_key=ORD-24183, error_code=invalid_response, safe_message=Label service could not generate labels
Q6S.7	`INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, $2, $3, $4, $5)`	AEL-T2, 6S, ORD-24183, external_unavailable, externalUnavailable	แก้ไข 1 แถว หลังรัน: use_case_id=6S, context_key=ORD-24183, error_code=external_unavailable, safe_message=externalUnavailable

2.23.17 Use Case 7S: ตัดสต๊อกสินค้า

ตารางที่ 2-93 SQL Statement และผลการทำงานของ Use Case 7S

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
Q7S.1	`SELECT o.order_id FROM orders o WHERE o.order_status = 'แจ้งเลขติดตามแล้ว' AND ( EXISTS ( SELECT 1 FROM order_items item WHERE item.order_id = o.order_id AND`	-	order_id=ORD-24174

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
	item.stock_deducted = FALSE) OR (o.delivery_status = 'delivered' AND o.tracking_notified = TRUE)) ORDER BY o.order_date, o.order_id LIMIT 100 FOR UPDATE OF o SKIP LOCKED		
Q7S.2	SELECT item.order_item_id, item.sku_id, item.channel_sku, item.fulfill_source, item.qty, item.stock_deducted, stock.stock_id, stock.in_house_qty, stock.rsl_qty FROM order_items item INNER JOIN stock ON stock.sku_id = item.sku_id WHERE item.order_id = $1 ORDER BY stock.stock_id, item.order_item_id FOR UPDATE OF item, stock	ORD-24174	order_item_id=ORD-24174-ITEM-1, sku_id=SKU-FSH-MLR-240, channel_sku=rk-mlr240-sv, fulfill_source=rsl, qty=3, stock_deducted=false, stock_id=STK-FSH-MLR-240, in_house_qty=52, rsl_qty=96
Q7S.3	UPDATE orders SET order_status = 'รอดำเนินการด้วยตนเอง', manual_reason = 'stock-deduct', updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'แจ้งเลขติดตามแล้ว'	ORD-24174	แก้ไข 1 แถว หลังรัน: order_id=ORD-24174, order_status=รอดำเนินการด้วยตนเอง, manual_reason=stock-deduct, sku_id=SKU-FSH-MLR-240, fulfill_source=rsl, stock_deducted=false
Q7S.4	UPDATE stock SET in_house_qty = CASE WHEN $2 = 'in_house' THEN in_house_qty - $3 ELSE in_house_qty END, rsl_qty = CASE WHEN $2 = 'rsl' THEN rsl_qty - $3 ELSE rsl_qty END, updated_at = CURRENT_TIMESTAMP WHERE stock_id = $1 AND CASE WHEN $2 = 'in_house' THEN in_house_qty >= $3 ELSE rsl_qty IS NOT NULL AND rsl_qty >= $3 END RETURNING stock_id, sku_id, in_house_qty, rsl_qty	STK-FSH-MLR-240, rsl, 3	แก้ไข 1 แถว หลังรัน: stock_id=STK-FSH-MLR-240, in_house_qty=52, rsl_qty=93
Q7S.5	UPDATE order_items SET stock_deducted = TRUE, deducted_at = CURRENT_TIMESTAMP WHERE order_item_id = ANY($1::text[]) AND stock_deducted = FALSE	{ORD-24174-ITEM-1}	แก้ไข 1 แถว หลังรัน: order_item_id=ORD-24174-ITEM-1, stock_deducted=true, deducted_at=2026-11-07 17:08
Q7S.6	INSERT INTO rakuten_stock_sync_states (stock_id, sync_status, last_error_code) VALUES ($1, 'pending', NULL) ON CONFLICT (stock_id) DO UPDATE SET sync_status = 'pending', last_error_code = NULL	STK-FSH-MLR-240	แก้ไข 1 แถว หลังรัน: stock_id=STK-FSH-MLR-240, sync_status=pending, last_synced_qty=148
Q7S.7	UPDATE orders SET order_status = 'จัดส่งสำเร็จ', manual_reason = NULL, updated_at = CURRENT_TIMESTAMP WHERE order_id = $1 AND order_status = 'แจ้งเลขติดตามแล้ว' AND tracking_notified = TRUE AND delivery_status = 'delivered' AND EXISTS (SELECT 1 FROM order_items WHERE order_id = $1) AND NOT EXISTS (SELECT 1 FROM order_items WHERE order_id = $1 AND stock_deducted = FALSE)	ORD-24174	แก้ไข 1 แถว หลังรัน: order_id=ORD-24174, order_status=จัดส่งสำเร็จ, delivery_status=delivered, tracking_notified=true
Q7S.8	SELECT sync.stock_id, sku.channel_sku, (stock.in_house_qty + COALESCE(stock.rsl_qty, 0)) AS available_qty FROM rakuten_stock_sync_states sync INNER JOIN stock ON stock.stock_id = sync.stock_id INNER JOIN skus sku ON sku.sku_id = stock.sku_id WHERE sync.sync_status = 'pending' AND ($1::text[] IS NULL OR sync.stock_id = ANY($1::text[])) ORDER BY sync.stock_id FOR UPDATE OF sync, stock	{STK-FSH-MLR-240}	stock_id=STK-FSH-MLR-240, channel_sku=rk-mlr240-sv, available_qty=148
Q7S.9	UPDATE rakuten_stock_sync_states SET last_attempt_at = CURRENT_TIMESTAMP WHERE stock_id = $1 AND sync_status = 'pending'	STK-FSH-MLR-240	แก้ไข 1 แถว

Query	SQL Command	ค่าที่ส่ง	ผลการทำงาน
			หลังรัน: stock_id=STK-FSH-MLR-240, sync_status=pending, last_attempt_at=2026-11-07 17:08
Q7S.10	`UPDATE rakuten_stock_sync_states sync SET sync_status = 'synced', last_synced_qty = $2, last_synced_at = CURRENT_TIMESTAMP, last_error_code = NULL FROM stock WHERE sync.stock_id = $1 AND stock.stock_id = sync.stock_id AND stock.in_house_qty + COALESCE(stock.rsl_qty, 0) = $2`	STK-FSH-MLR-240, 148	แก้ไข 1 แถว หลังรัน: stock_id=STK-FSH-MLR-240, sync_status=synced, last_synced_qty=148, last_synced_at=2026-11-07 17:08
Q7S.11	`UPDATE rakuten_stock_sync_states SET sync_status = 'pending', last_error_code = $2 WHERE stock_id = $1`	STK-FSH-MLR-240, unavailable	แก้ไข 1 แถว หลังรัน: stock_id=STK-FSH-MLR-240, sync_status=pending, last_error_code=unavailable
Q7S.12	`INSERT INTO auto_error_logs ( auto_error_log_id, use_case_id, context_key, error_code, safe_message ) VALUES ($1, $2, $3, $4, $5)`	AEL-0103, 7S, STK-FSH-MLR-240, external_unavailable, externalUnavailable	แก้ไข 1 แถว หลังรัน: auto_error_log_id=AEL-0103, use_case_id=7S, context_key=STK-FSH-MLR-240, error_code=external_unavailable

2.24 การทดสอบโปรแกรมทำงานตาม Use Case Description

ทดสอบบนเว็บจริงด้วยข้อมูลตัวอย่าง เริ่มจากข้อมูลตั้งต้นทุกครั้ง

กรณีระบบภายนอกล่ม ใช้หน้า `/dev` (ไม่อยู่ในเมนู) จำลองข้อผิดพลาดตาม "ทางเลือก" ของแต่ละ UC

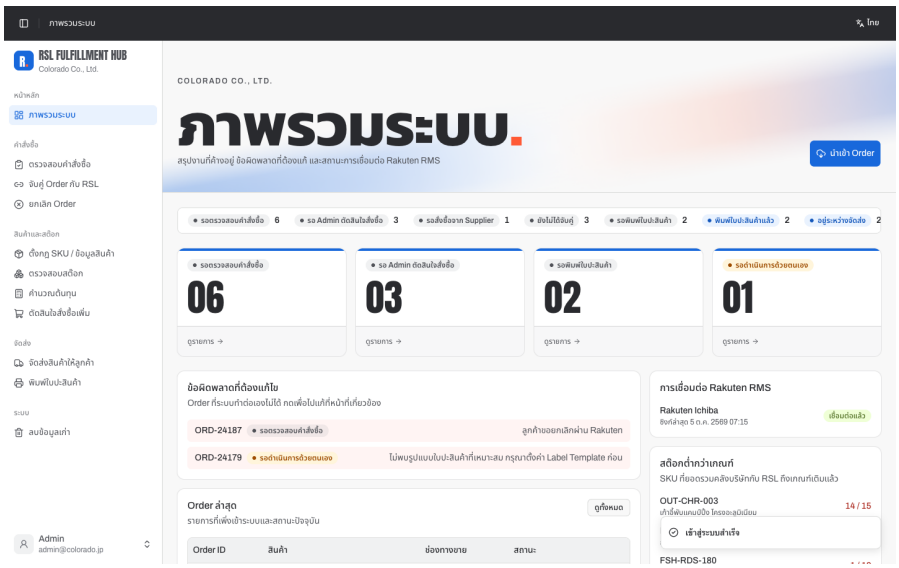
บัญชีทดสอบ: `admin@colorado.jp` และ `locked@colorado.jp` (บัญชีที่ถูกล็อก)

ข้อความที่คาดหวังคัดมาจาก Use Case Description คำต่อคำ ทุกกรณีผ่าน

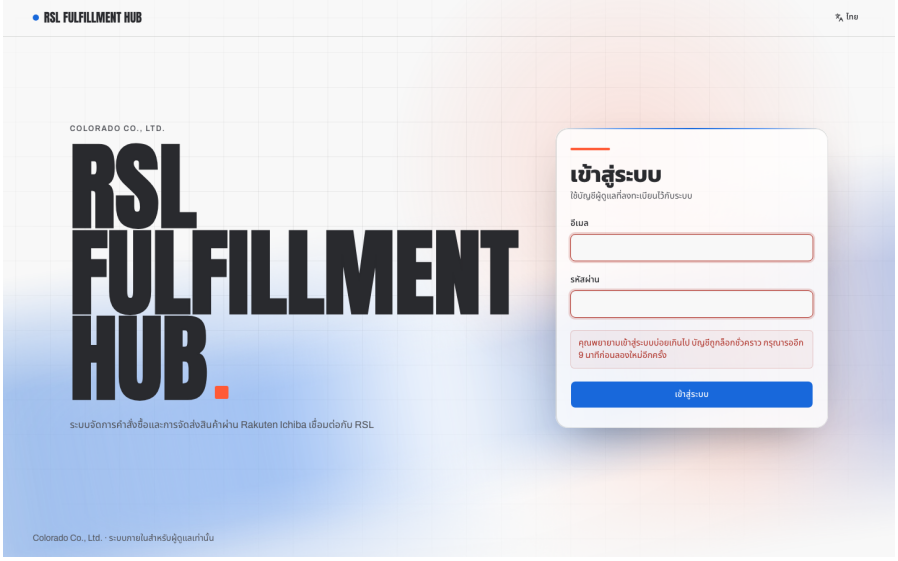
2.24.1 การทดสอบ Use Case 1A: เข้าสู่ระบบ

ตารางที่ 2-94 การทดสอบ Use Case 1A: เข้าสู่ระบบ

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
กรอก `admin@colorado.jp` ถูกต้อง	"เข้าสู่ระบบสำเร็จ" แล้วไปหน้า Dashboard	ผ่าน
ใช้บัญชีที่เข้าผิดครบ 5 ครั้ง (`locked@colorado.jp`)	"คุณพยายามเข้าสู่ระบบบ่อยเกินไป บัญชีถูกล็อกชั่วคราว กรุณารออีก N นาทีก่อนลองใหม่อีกครั้ง" (ปลดล็อกเองเมื่อครบ 15 นาที)	ผ่าน



รูปที่ 2-134 ผลการทดสอบ Use Case 1A กรณีปกติ

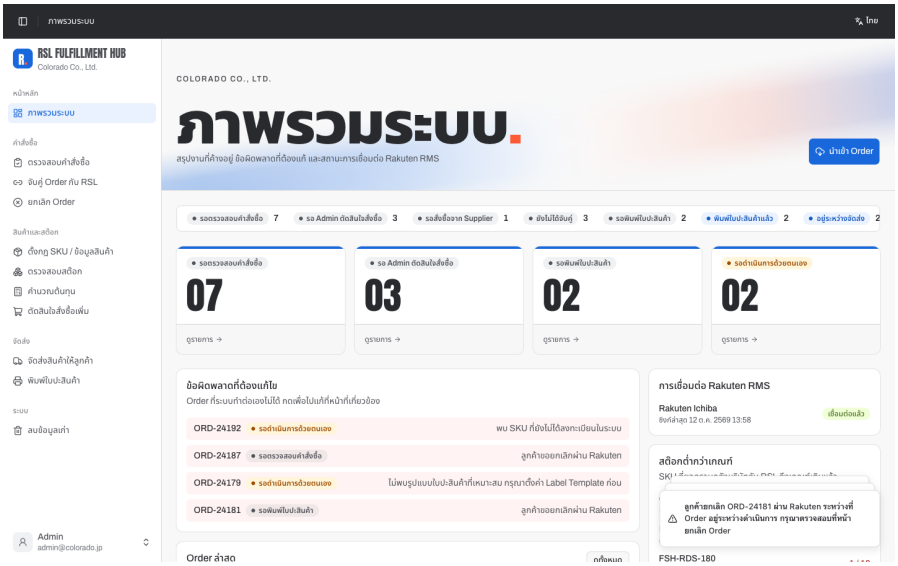


รูปที่ 2-135 ผลการทดสอบ Use Case 1A กรณีผิดพลาด

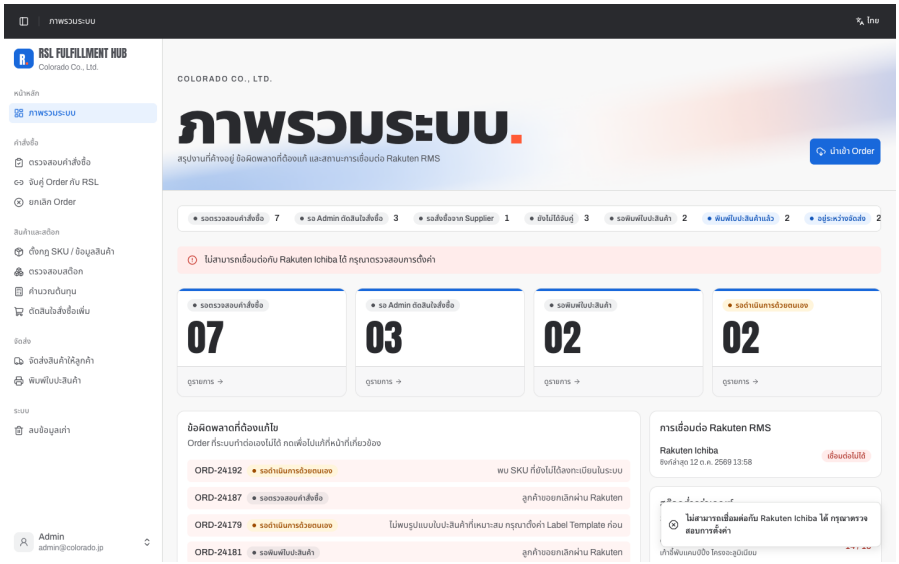
2.24.2 การทดสอบ Use Case 4A: ดึง Order เข้าระบบ

ตารางที่ 2-95 การทดสอบ Use Case 4A: ดึง Order เข้าระบบ

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
กดนำเข้า Order	"นำเข้า Order สำเร็จ 2 รายการ" และแจ้ง Order ซ้ำ / ไม่ครบ / SKU ที่ยังไม่ลงทะเบียน / ลูกค้าขอยกเลิก	ผ่าน
Rakuten RMS เชื่อมต่อไม่ได้	"ไม่สามารถเชื่อมต่อกับ Rakuten Ichiba ได้ กรุณาตรวจสอบการตั้งค่า"	ผ่าน



รูปที่ 2-136 ผลการทดสอบ Use Case 4A กรณีปกติ

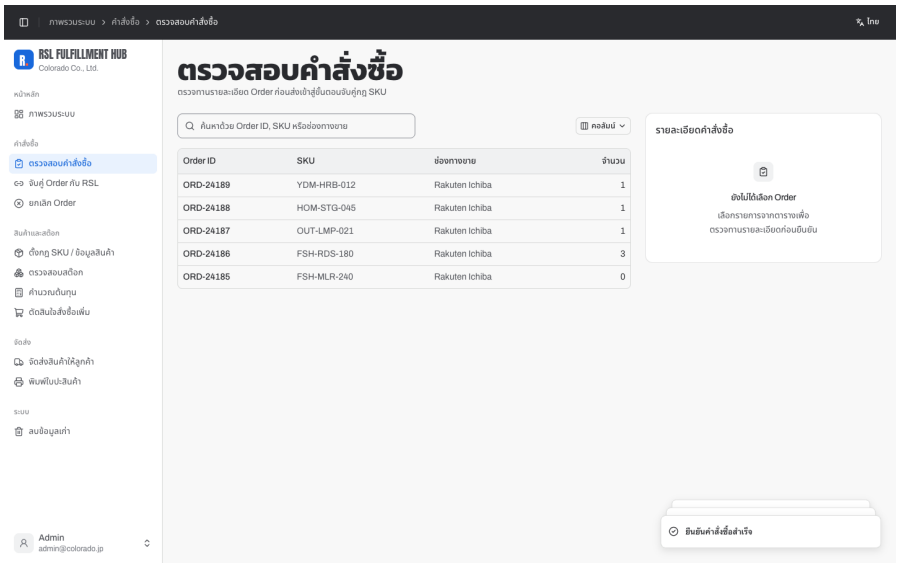


รูปที่ 2-137 ผลการทดสอบ Use Case 4A กรณีผิดพลาด

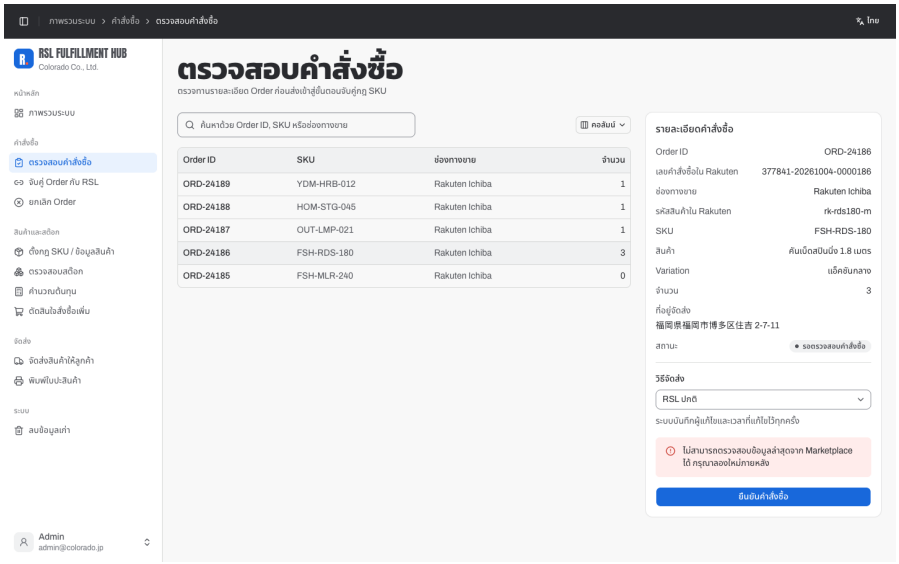
2.24.3 การทดสอบ Use Case 2A: ตรวจสอบคำสั่งซื้อ

ตารางที่ 2-96 การทดสอบ Use Case 2A: ตรวจสอบคำสั่งซื้อ

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ยืนยัน ORD-24190	"ยืนยันคำสั่งซื้อสำเร็จ"	ผ่าน
Marketplace ตอบกลับไม่ได้	"ไม่สามารถตรวจสอบข้อมูลล่าสุดจาก Marketplace ได้ กรุณาลองใหม่ภายหลัง"	ผ่าน



รูปที่ 2-138 ผลการทดสอบ Use Case 2A กรณีปกติ

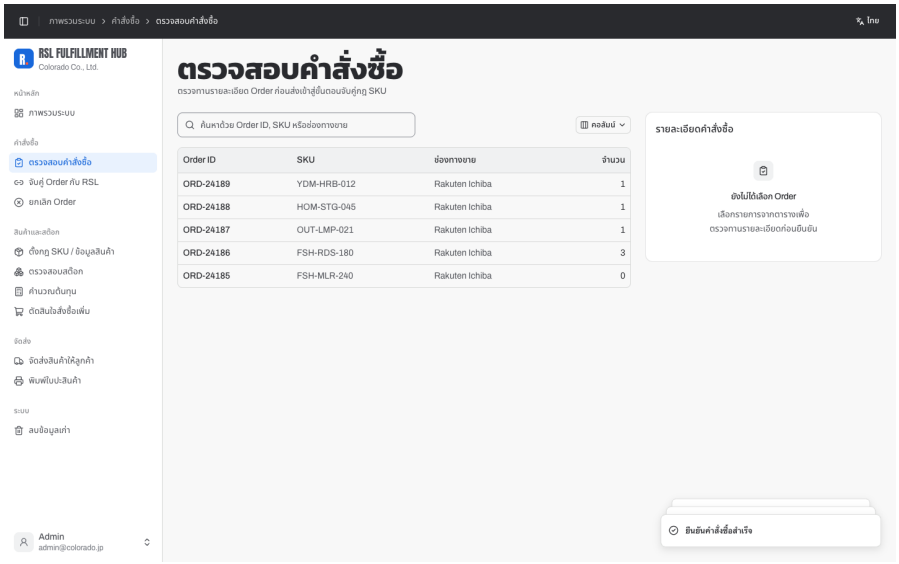


รูปที่ 2-139 ผลการทดสอบ Use Case 2A กรณีผิดพลาด

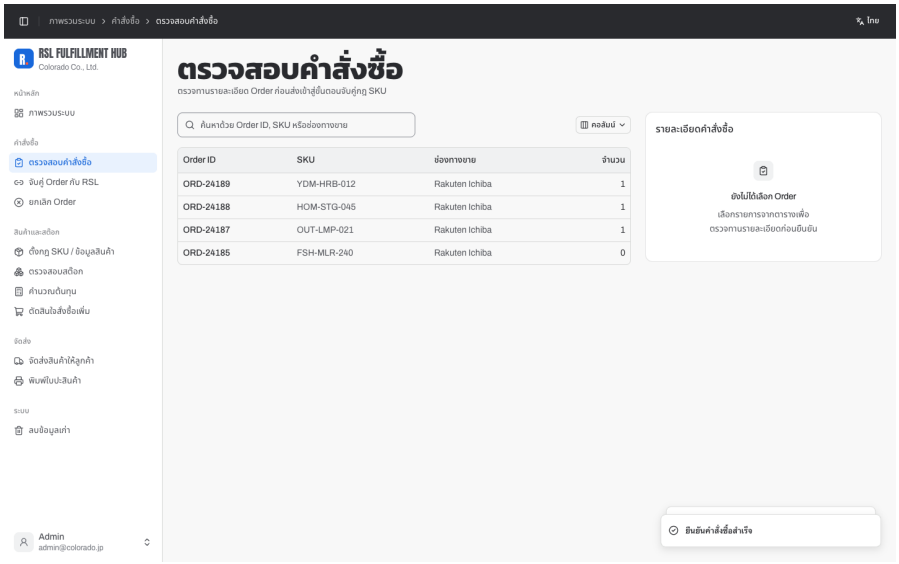
2.24.4 การทดสอบ Use Case 2S: จับคู่กฎ SKU

ตารางที่ 2-97 การทดสอบ Use Case 2S: จับคู่กฎ SKU

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ต่อจาก 2A (ระบบทำเอง)	"จับคู่กฎ SKU สำเร็จ 1 รายการ"	ผ่าน
อ่านข้อมูลกฎ SKU ไม่ได้	"ไม่สามารถอ่านข้อมูลกฎ SKU ได้ กรุณาลองใหม่ภายหลัง"	ผ่าน



รูปที่ 2-140 ผลการทดสอบ Use Case 2S กรณีปกติ

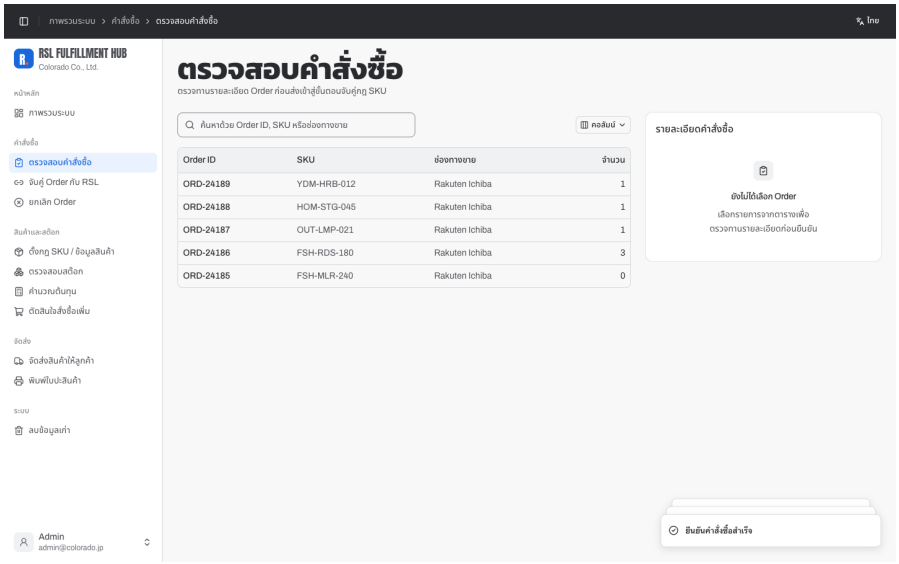


รูปที่ 2-141 ผลการทดสอบ Use Case 2S กรณีผิดพลาด

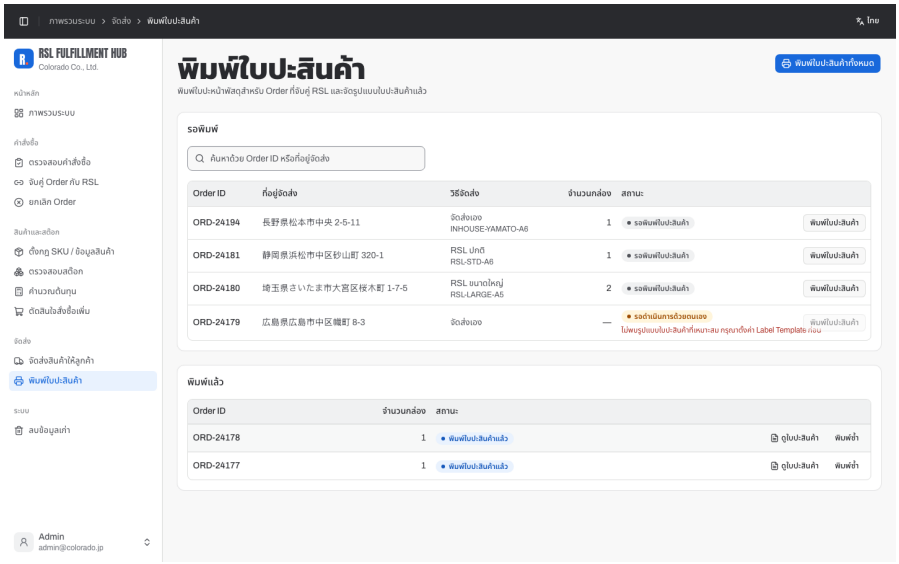
2.24.5 การทดสอบ Use Case 3S: ตรวจสอบของสินค้าใน stock + RSL

ตารางที่ 2-98 การทดสอบ Use Case 3S: ตรวจสอบของสินค้าใน stock + RSL

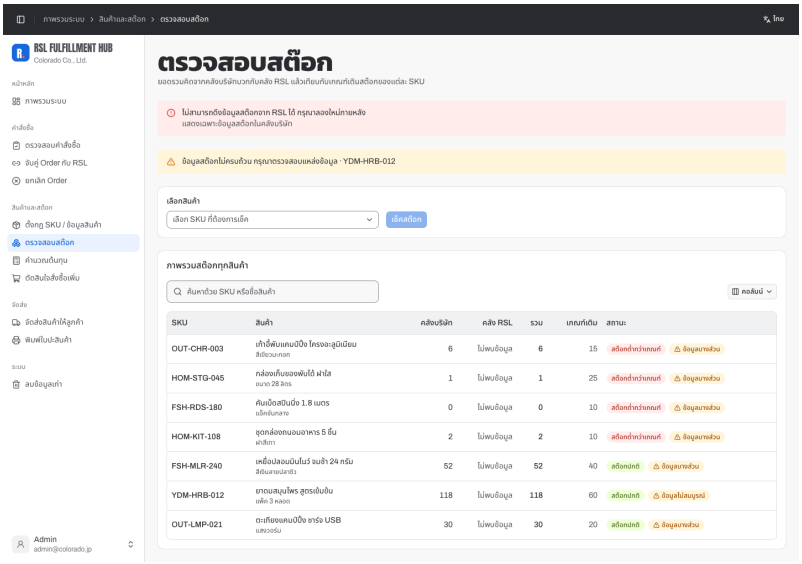
กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ต่อจาก 2S (ระบบทำเอง)	"ORD-24190 สต๊อกเพียงพอ ส่งต่อไปจับคู่กับ RSL"	ผ่าน
ยืนยัน ORD-24194 (RSL 0 ชิ้น, คลังบริษัท 1 ชิ้น)	บริษัทจัดส่งเอง ข้าม 1S ไป 6S ทันที ORD-24194 อยู่ในหน้าพิมพ์ใบปะ รูปแบบ INHOUSE-YAMATO-A6	ผ่าน
RSL เชื่อมต่อไม่ได้	"ไม่สามารถดึงข้อมูลสต๊อกจาก RSL ได้ กรุณาลองใหม่ภายหลัง" แสดงเฉพาะสต๊อกในคลังบริษัท	ผ่าน



รูปที่ 2-142 ผลการทดสอบ Use Case 3S กรณีปกติ



รูปที่ 2-143 ผลการทดสอบ Use Case 3S กรณีบริษัทจัดส่งเอง (in-house)

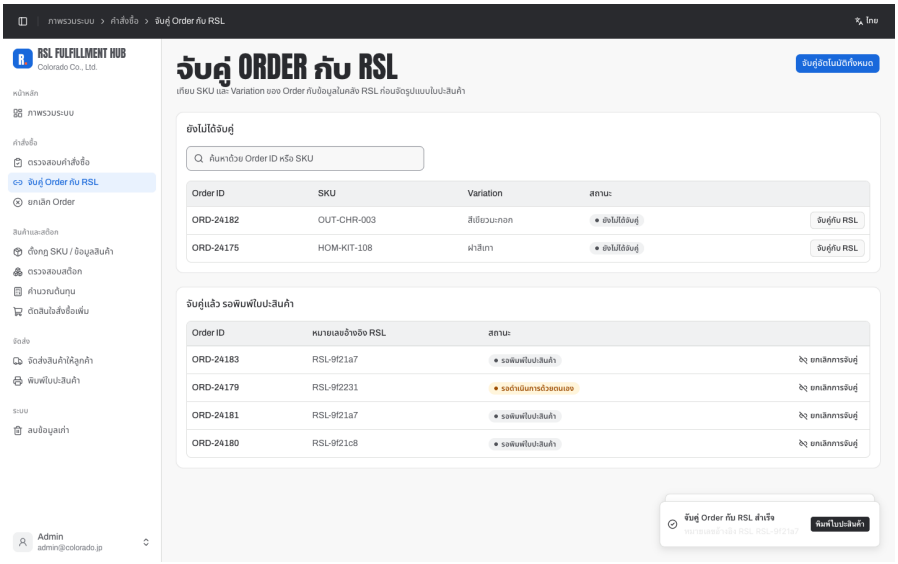


รูปที่ 2-144 ผลการทดสอบ Use Case 3S กรณีผิดพลาด

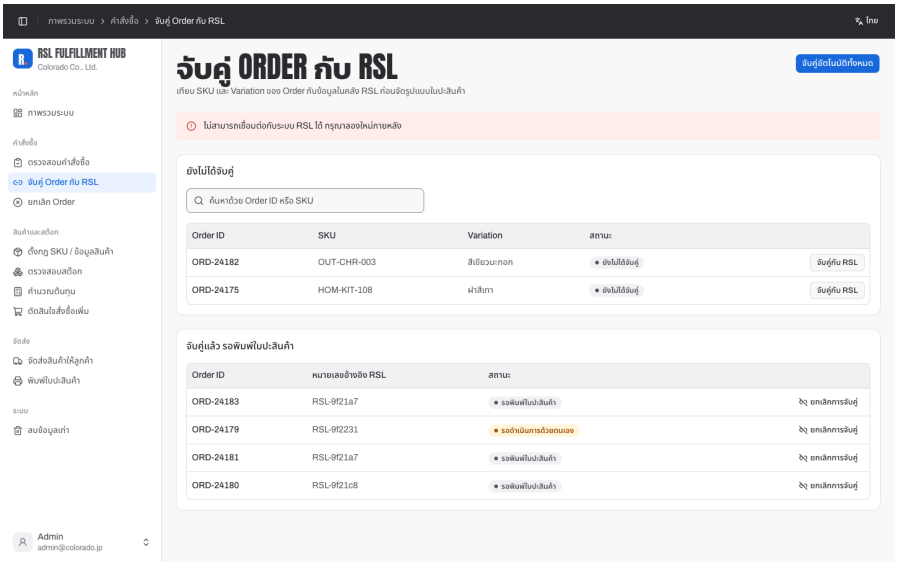
2.24.6 การทดสอบ Use Case 1S: จับคู่ Order กับเลข RSL

ตารางที่ 2-99 การทดสอบ Use Case 1S: จับคู่ Order กับเลข RSL

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
จับคู่ ORD-24183	"จับคู่ Order กับ RSL สำเร็จ" พร้อมเลข RSL	ผ่าน
RSL เชื่อมต่อไม่ได้	"ไม่สามารถเชื่อมต่อกับระบบ RSL ได้ กรุณาลองใหม่ภายหลัง"	ผ่าน



รูปที่ 2-145 ผลการทดสอบ Use Case 1S กรณีปกติ

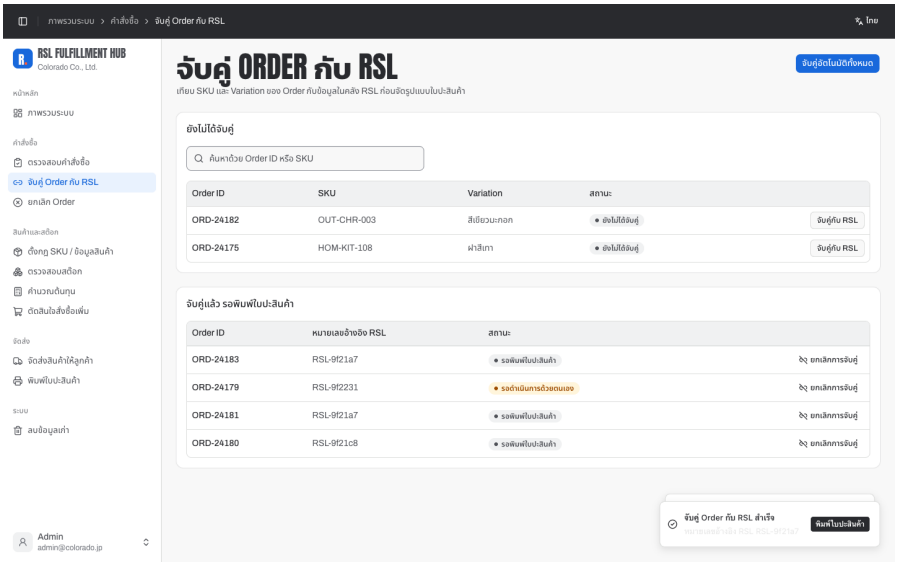


รูปที่ 2-146 ผลการทดสอบ Use Case 1S กรณีผิดพลาด

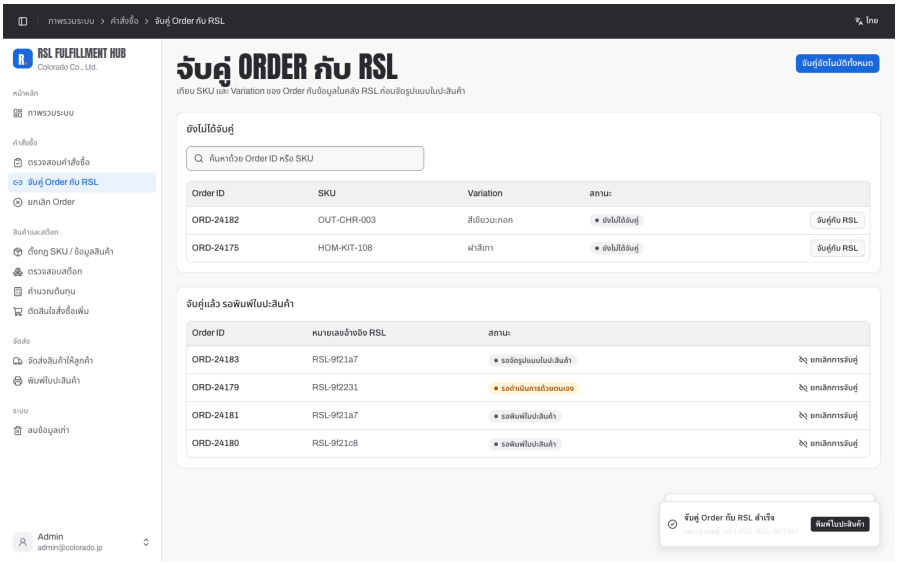
2.24.7 การทดสอบ Use Case 6S: จัดรูปแบบใบปะสินค้า

ตารางที่ 2-100 การทดสอบ Use Case 6S: จัดรูปแบบใบปะสินค้า

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ต่อจาก 1S (ระบบทำเอง)	"สร้างใบปะสินค้าสำเร็จ 1 ใบ"	ผ่าน
บริการสร้าง Label ล่ม	"ไม่สามารถสร้างใบปะสินค้าได้ กรุณาลองใหม่ภายหลัง"	ผ่าน



รูปที่ 2-147 ผลการทดสอบ Use Case 6S กรณีปกติ

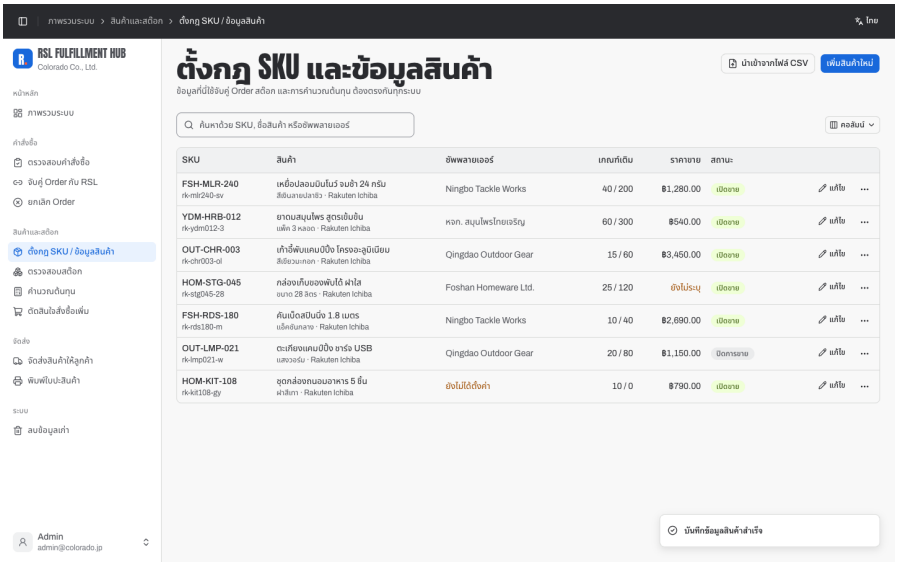


รูปที่ 2-148 ผลการทดสอบ Use Case 6S กรณีผิดพลาด

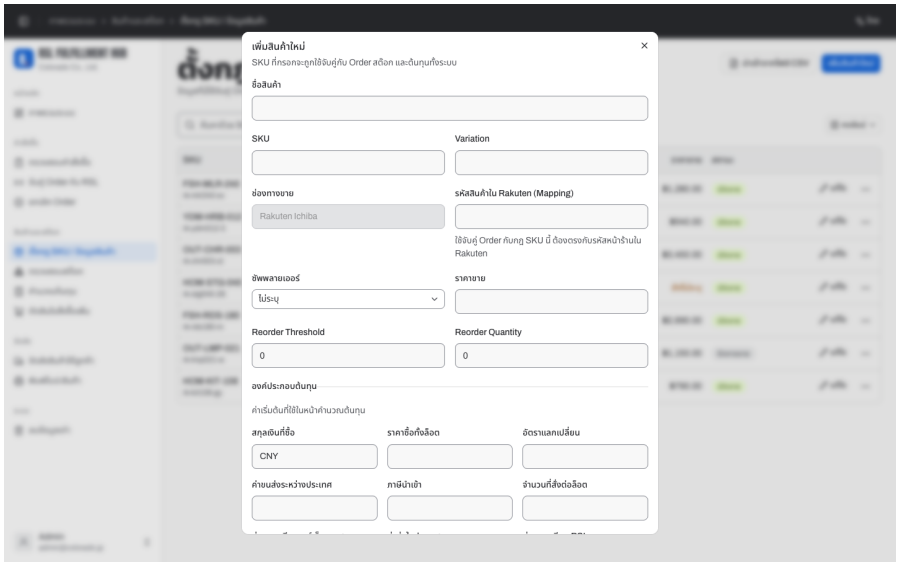
2.24.8 การทดสอบ Use Case 3A: ตั้งกฎ SKU

ตารางที่ 2-101 การทดสอบ Use Case 3A: ตั้งกฎ SKU

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
แก้ไขสินค้าแล้วบันทึก	"บันทึกข้อมูลสินค้าสำเร็จ"	ผ่าน
เพิ่มสินค้าโดยไม่กรอกข้อมูล	"กรุณากรอกข้อมูลให้ครบถ้วน"	ผ่าน



รูปที่ 2-149 ผลการทดสอบ Use Case 3A กรณีปกติ

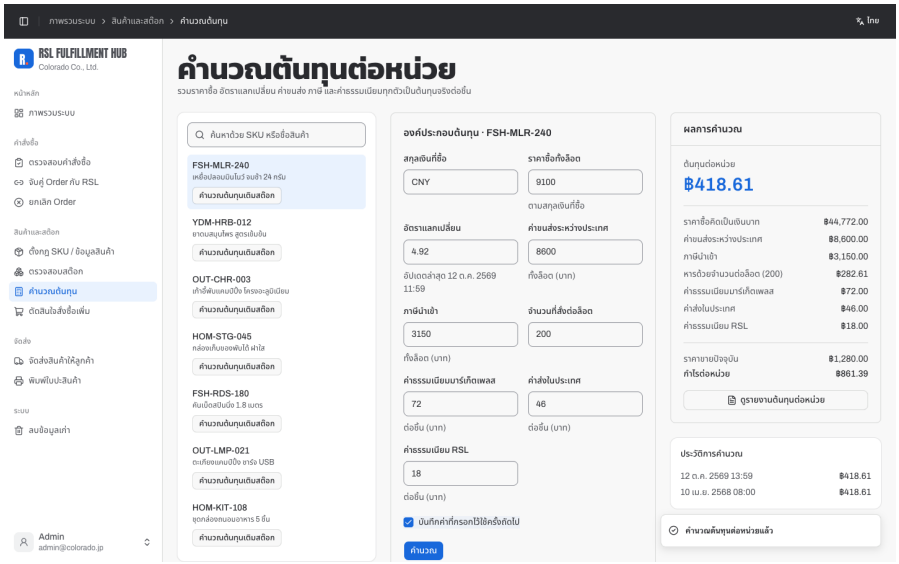


รูปที่ 2-150 ผลการทดสอบ Use Case 3A กรณีผิดพลาด

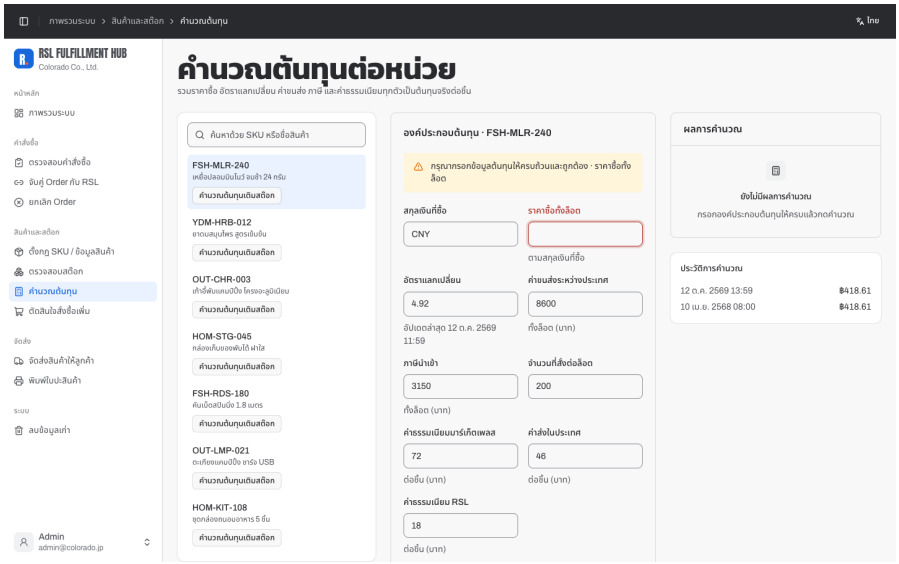
2.24.9 การทดสอบ Use Case 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง

ตารางที่ 2-102 การทดสอบ Use Case 4S: คำนวณ Cost ในการสั่งสินค้าเติม stock เอง

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
คำนวณด้วยข้อมูลครบ	"คำนวณต้นทุนต่อหน่วยแล้ว"	ผ่าน
ลบราคาซื้อออกแล้วคำนวณ	"กรุณากรอกข้อมูลต้นทุนให้ครบถ้วนและถูกต้อง"	ผ่าน



รูปที่ 2-151 ผลการทดสอบ Use Case 4S กรณีปกติ

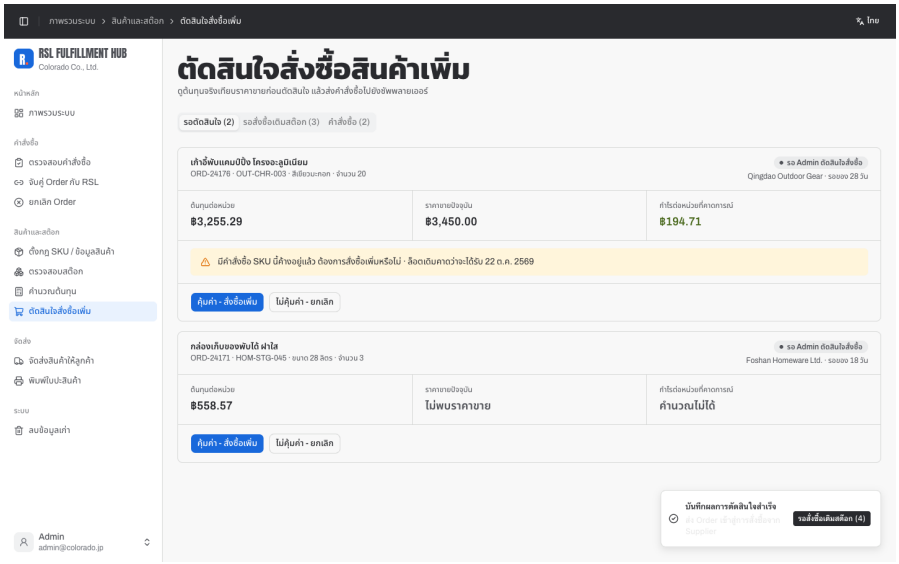


รูปที่ 2-152 ผลการทดสอบ Use Case 4S กรณีผิดพลาด

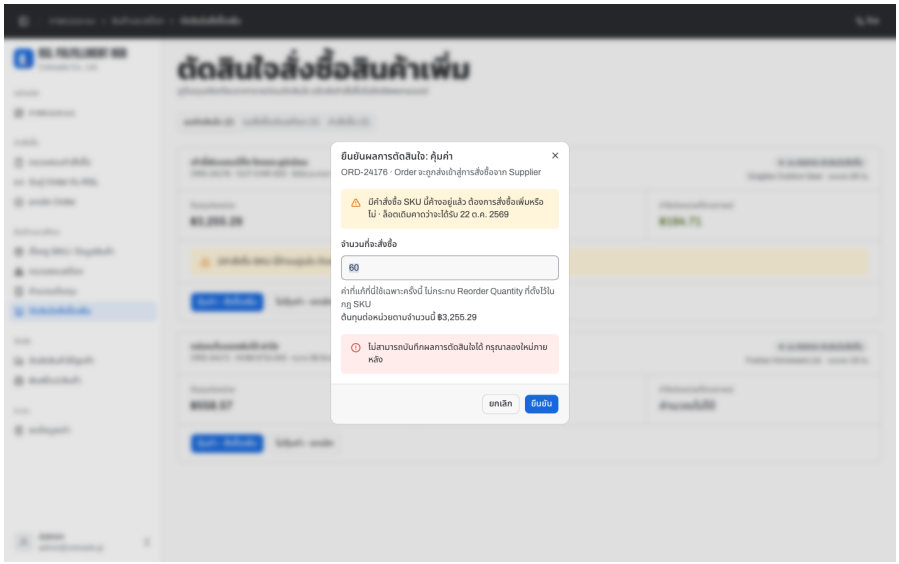
2.24.10 การทดสอบ Use Case 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock

ตารางที่ 2-103 การทดสอบ Use Case 5A: ตัดสินใจการสั่งซื้อสินค้าเพิ่มเข้า stock

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
เลือก "คุ้มค่า" ของ ORD-24184	"บันทึกผลการตัดสินใจสำเร็จ"	ผ่าน
ฐานข้อมูลบันทึกไม่ได้	"ไม่สามารถบันทึกผลการตัดสินใจได้ กรุณาลองใหม่ภายหลัง"	ผ่าน



รูปที่ 2-153 ผลการทดสอบ Use Case 5A กรณีปกติ

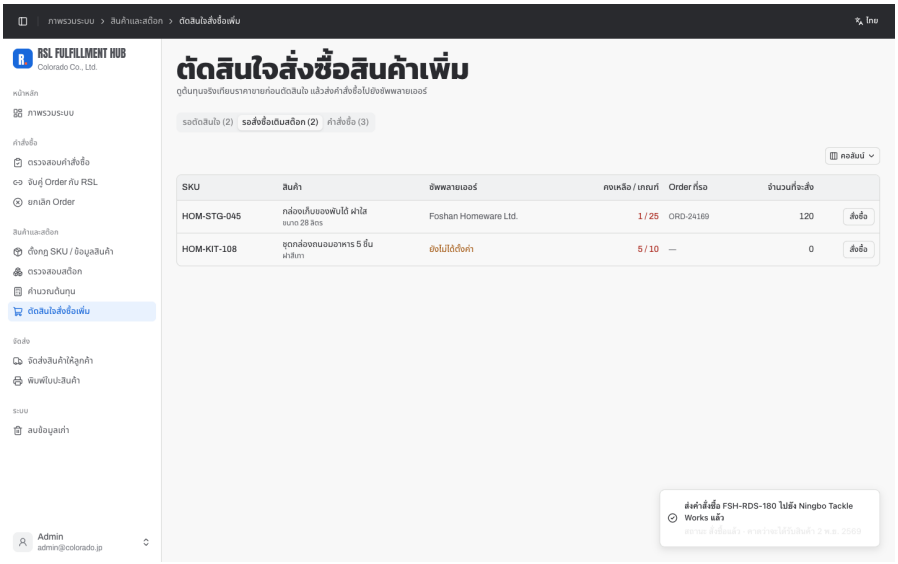


รูปที่ 2-154 ผลการทดสอบ Use Case 5A กรณีผิดพลาด

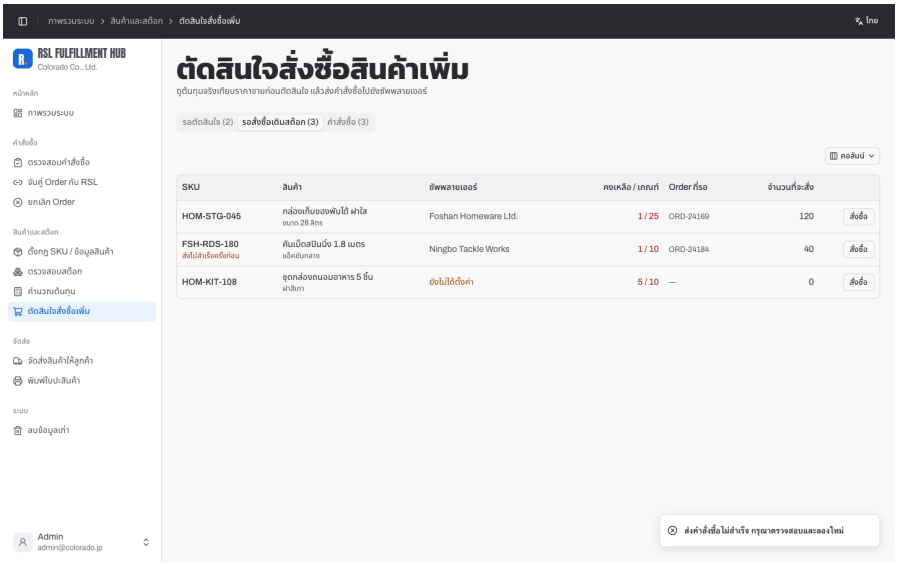
2.24.11 การทดสอบ Use Case 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier

ตารางที่ 2-104 การทดสอบ Use Case 6A: สั่งซื้อสินค้าเพิ่มเข้า stock จาก supplier

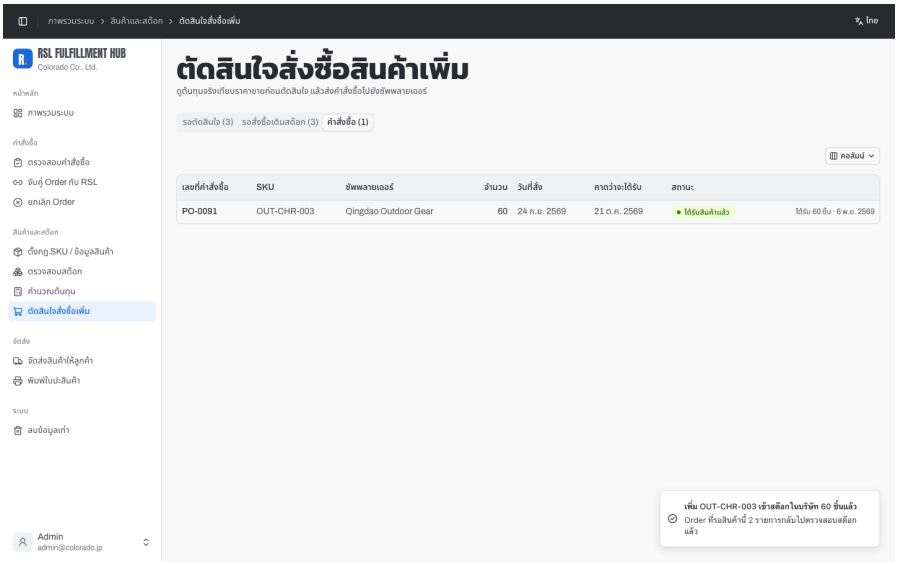
กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
สั่งซื้อ FSH-RDS-180	สถานะเป็น "สั่งซื้อแล้ว" พร้อมวันที่คาดว่าจะได้รับสินค้า	ผ่าน
ส่งคำสั่งซื้อไปยัง Supplier ไม่ได้	"ส่งคำสั่งซื้อไม่สำเร็จ กรุณาตรวจสอบและลองใหม่" คงสถานะ "รอส่งคำสั่งซื้อ"	ผ่าน
รับสินค้า PO-0091 จำนวน 60 ชิ้น	"เพิ่ม OUT-CHR-003 เข้าสต๊อกในบริษัท 60 ชิ้นแล้ว" คำสั่งซื้อเป็น "ได้รับสินค้าแล้ว" และ Order ที่ผูกไว้ 2 รายการกลับไปตรวจสอบสต๊อก	ผ่าน



รูปที่ 2-155 ผลการทดสอบ Use Case 6A กรณีปกติ



รูปที่ 2-156 ผลการทดสอบ Use Case 6A กรณีผิดพลาด

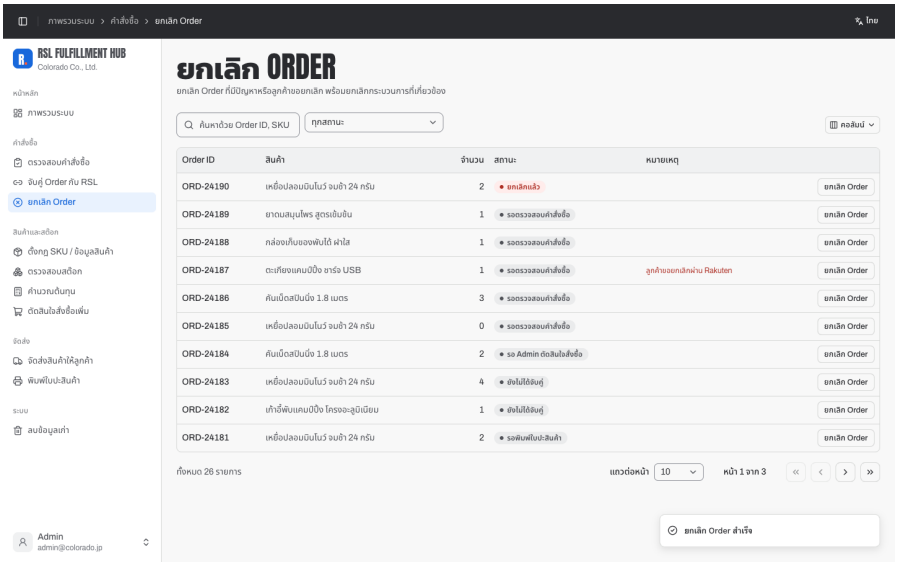


รูปที่ 2-157 ผลการทดสอบ Use Case 6A กรณีปกติ

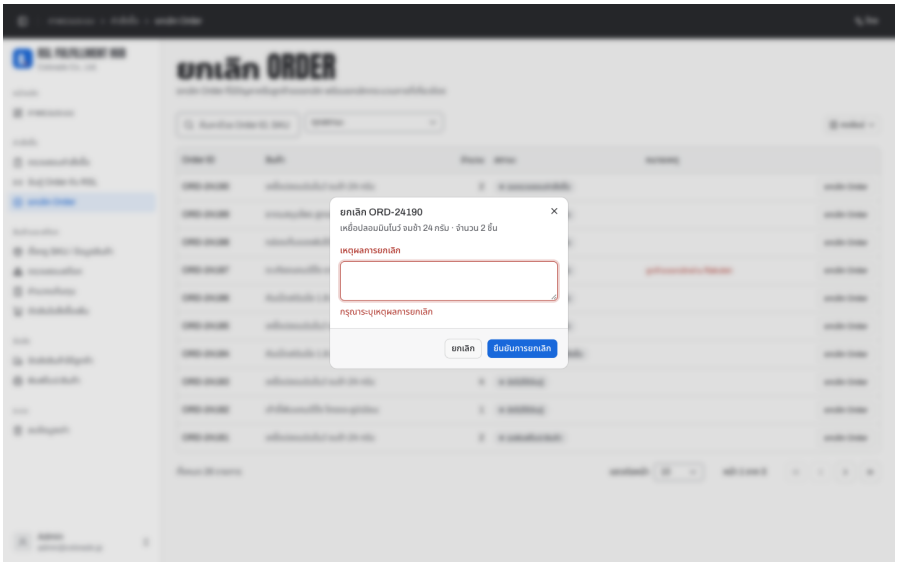
2.24.12 การทดสอบ Use Case 7A: ยกเลิก Order

ตารางที่ 2-105 การทดสอบ Use Case 7A: ยกเลิก Order

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ใส่เหตุผลแล้วยืนยัน	"ยกเลิก Order สำเร็จ"	ผ่าน
ไม่ใส่เหตุผล	"กรุณาระบุเหตุผลการยกเลิก"	ผ่าน



รูปที่ 2-158 ผลการทดสอบ Use Case 7A กรณีปกติ

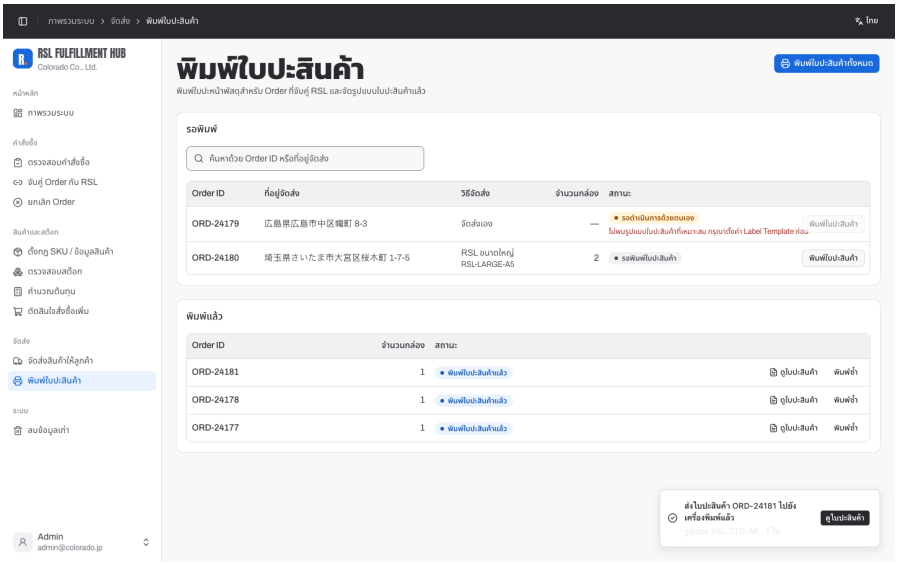


รูปที่ 2-159 ผลการทดสอบ Use Case 7A กรณีผิดพลาด

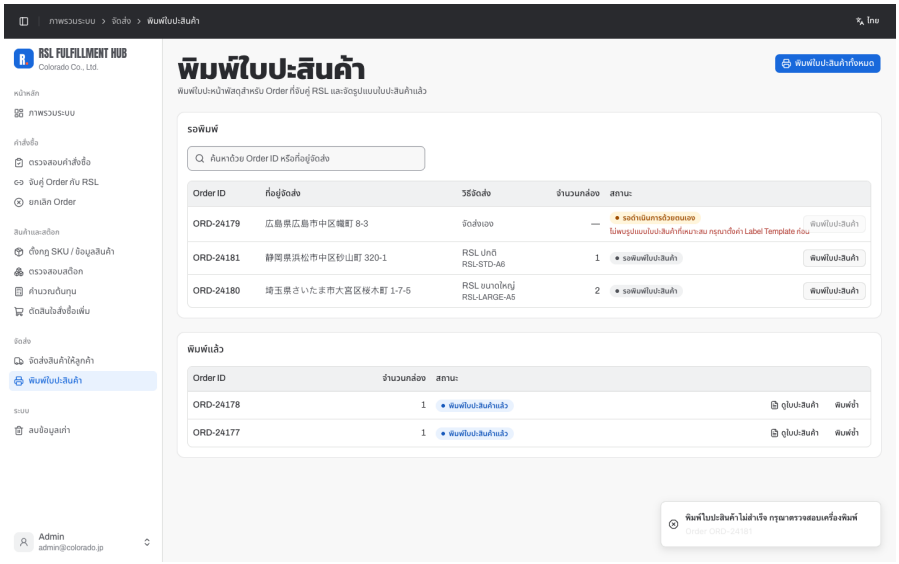
2.24.13 การทดสอบ Use Case 9A: พิมพ์ใบปะสินค้า

ตารางที่ 2-106 การทดสอบ Use Case 9A: พิมพ์ใบปะสินค้า

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
พิมพ์ ORD-24181	ส่งใบปะไปยังเครื่องพิมพ์ สถานะเป็น "พิมพ์ใบปะสินค้าแล้ว"	ผ่าน
เครื่องพิมพ์ไม่พร้อม	"พิมพ์ใบปะสินค้าไม่สำเร็จ กรุณาตรวจสอบเครื่องพิมพ์"	ผ่าน



รูปที่ 2-160 ผลการทดสอบ Use Case 9A กรณีปกติ

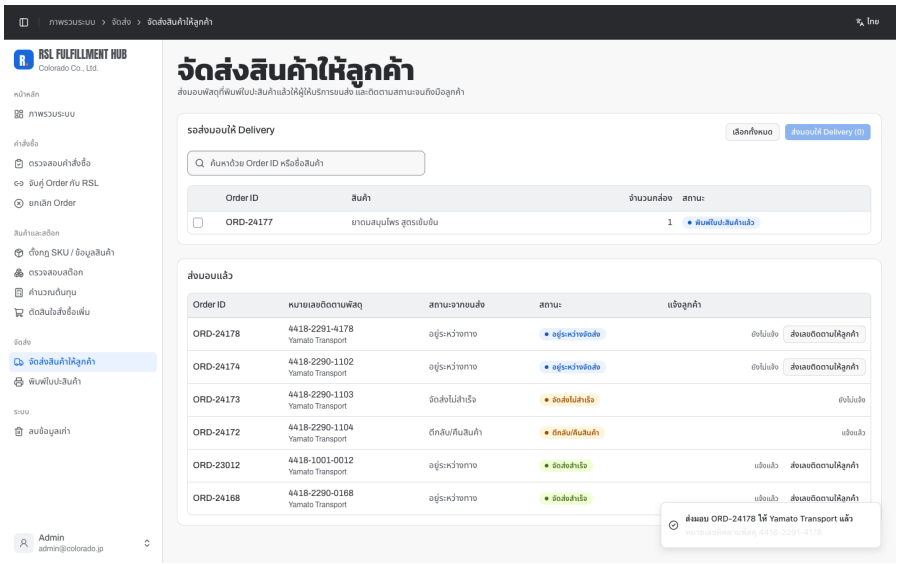


รูปที่ 2-161 ผลการทดสอบ Use Case 9A กรณีผิดพลาด

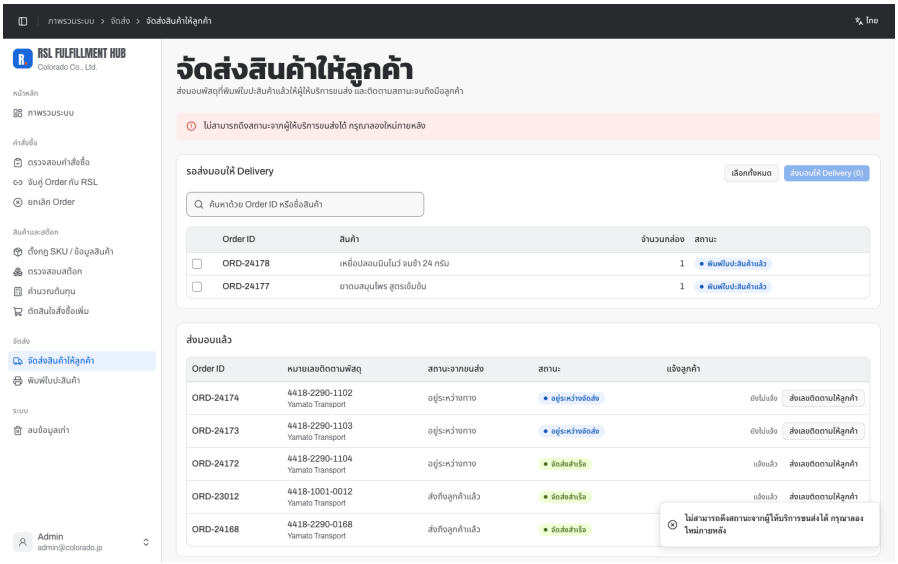
2.24.14 การทดสอบ Use Case 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery

ตารางที่ 2-107 การทดสอบ Use Case 8A: จัดส่งสินค้าให้ลูกค้าผ่าน Delivery

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ส่งมอบ ORD-24178	ได้หมายเลขติดตามพัสดุ สถานะเป็น "อยู่ระหว่างจัดส่ง"	ผ่าน
ดึงสถานะจากขนส่งไม่ได้	"ไม่สามารถดึงสถานะจากผู้ให้บริการขนส่งได้ กรุณาลองใหม่ภายหลัง"	ผ่าน



รูปที่ 2-162 ผลการทดสอบ Use Case 8A กรณีปกติ

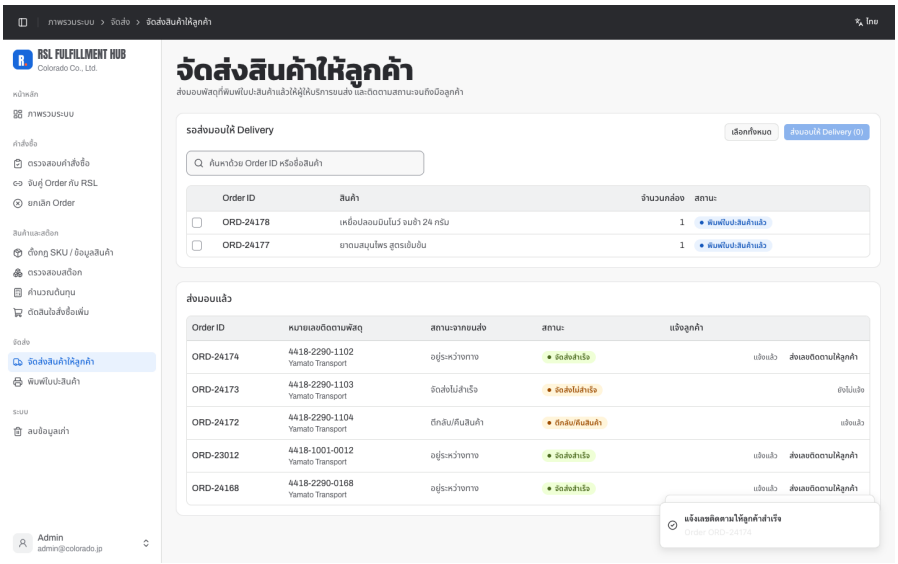


รูปที่ 2-163 ผลการทดสอบ Use Case 8A กรณีผิดพลาด

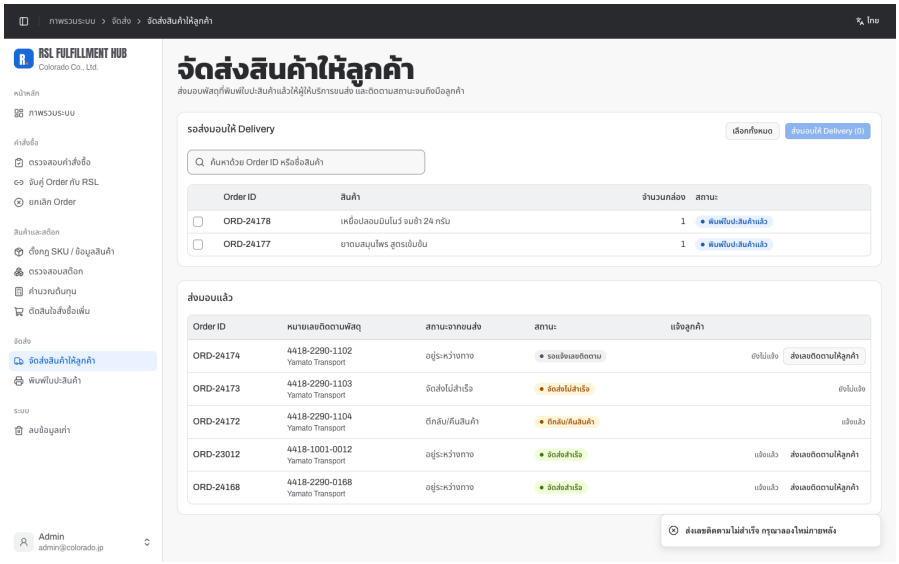
2.24.15 การทดสอบ Use Case 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace

ตารางที่ 2-108 การทดสอบ Use Case 5S: ส่งเลขติดตามสินค้าของ Delivery ให้ลูกค้าผ่าน Marketplace

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ส่งเลขติดตามของ ORD-24174	"แจ้งเลขติดตามให้ลูกค้าสำเร็จ"	ผ่าน
Rakuten รับเลขติดตามไม่ได้	"ส่งเลขติดตามไม่สำเร็จ กรุณาลองใหม่ภายหลัง"	ผ่าน



รูปที่ 2-164 ผลการทดสอบ Use Case 5S กรณีปกติ

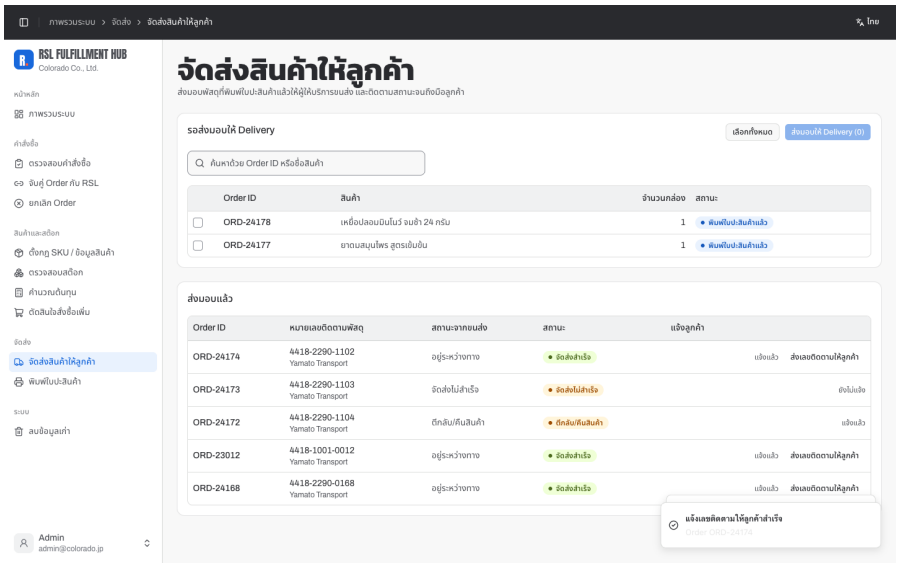


รูปที่ 2-165 ผลการทดสอบ Use Case 5S กรณีผิดพลาด

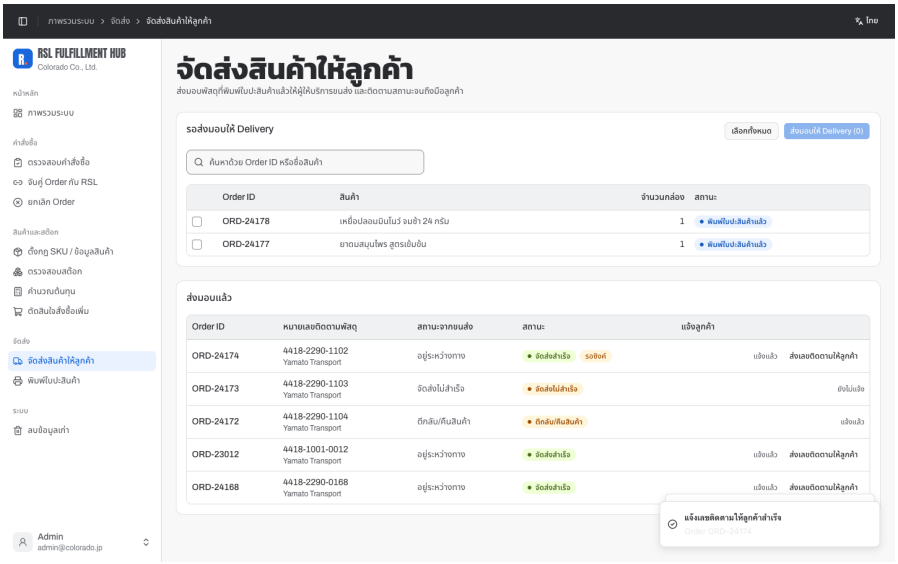
2.24.16 การทดสอบ Use Case 7S: ตัดสต๊อกสินค้า

ตารางที่ 2-109 การทดสอบ Use Case 7S: ตัดสต๊อกสินค้า

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ต่อจาก 5S (ระบบทำเอง)	"ตัดสต๊อกและอัปเดตจำนวนสินค้าสำเร็จ"	ผ่าน
อัปเดตจำนวนใน Rakuten ไม่ได้	"ไม่สามารถอัปเดตจำนวนสินค้าใน Rakuten Ichiba ได้ กรุณาลองใหม่ภายหลัง" ตัดสต๊อกในระบบไว้ก่อน	ผ่าน



รูปที่ 2-166 ผลการทดสอบ Use Case 7S กรณีปกติ

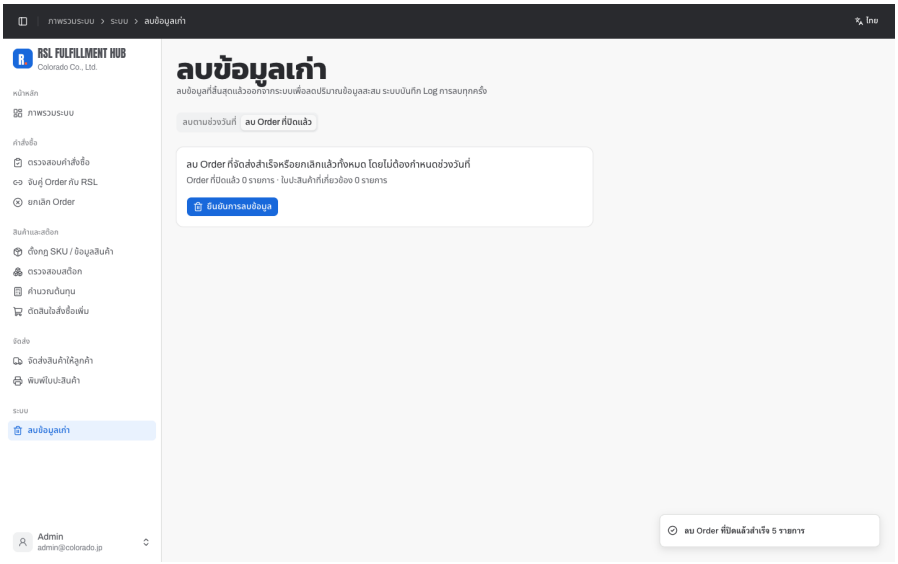


รูปที่ 2-167 ผลการทดสอบ Use Case 7S กรณีผิดพลาด

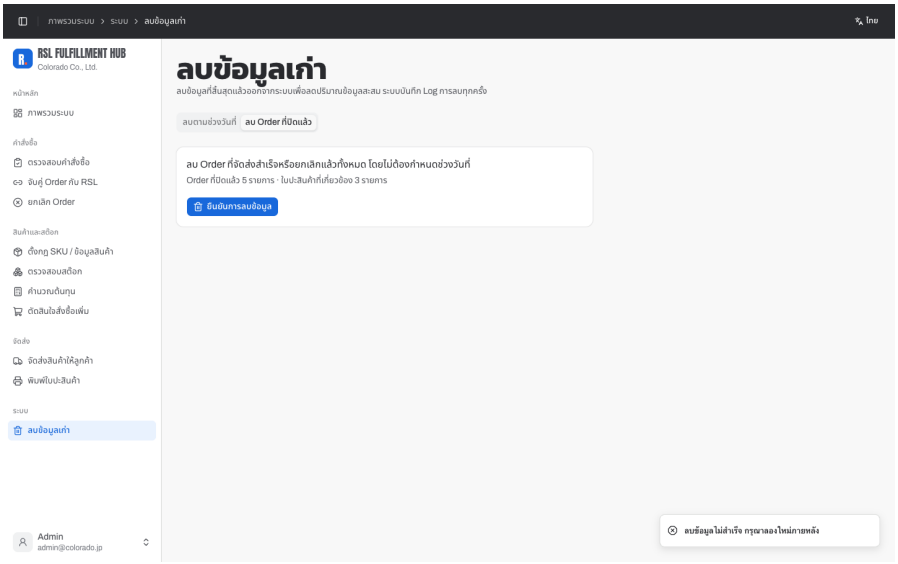
2.24.17 การทดสอบ Use Case 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว

ตารางที่ 2-110 การทดสอบ Use Case 10A: ลบข้อมูลเก่าที่ไม่ใช้แล้ว

กรณีทดสอบ	ผลที่คาดหวัง (ข้อความจาก UC)	ผล
ลบ Order ที่ปิดแล้ว	"ลบ Order ที่ปิดแล้วสำเร็จ 5 รายการ"	ผ่าน
ฐานข้อมูลขัดข้องระหว่างลบ	"ลบข้อมูลไม่สำเร็จ กรุณาลองใหม่ภายหลัง"	ผ่าน



รูปที่ 2-168 ผลการทดสอบ Use Case 10A กรณีปกติ

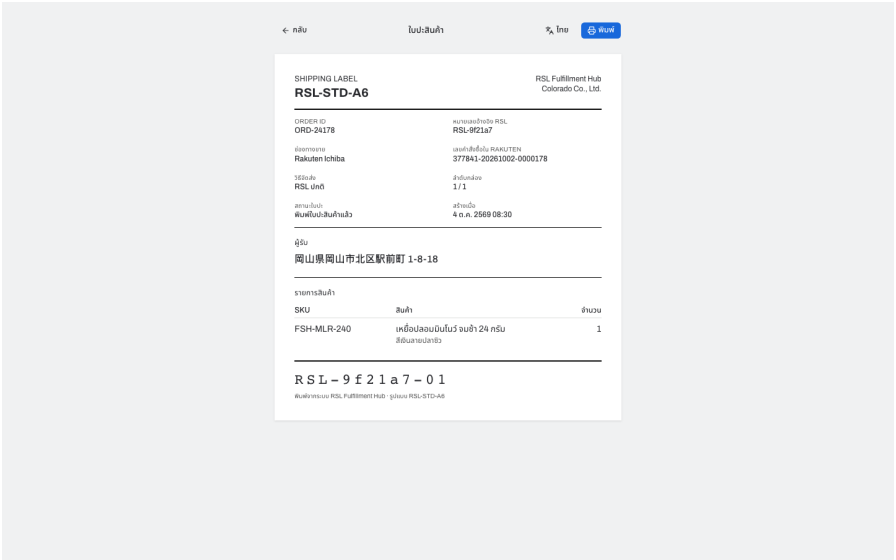


รูปที่ 2-169 ผลการทดสอบ Use Case 10A กรณีผิดพลาด

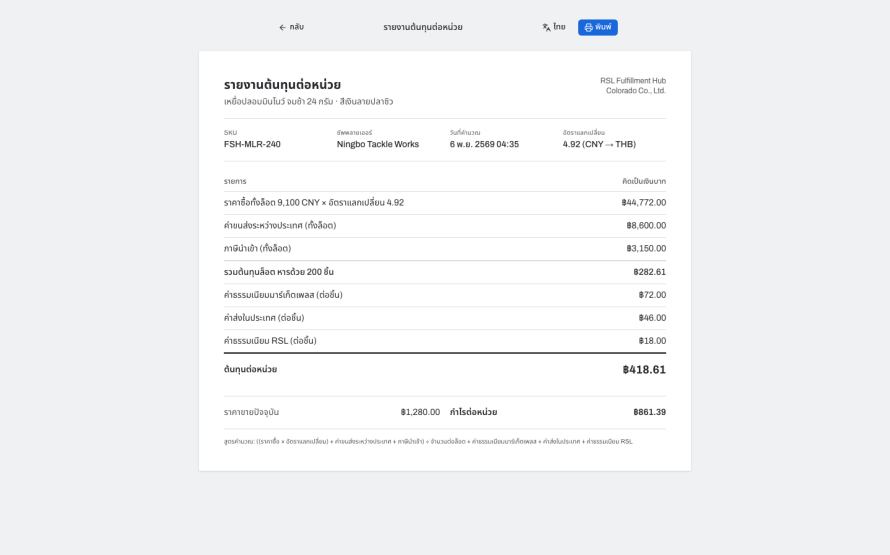
2.24.18 การทดสอบการออกรายงาน

ตารางที่ 2-111 การทดสอบการออกรายงาน

รายงาน	Use Case	วิธีทดสอบ	ผล
ใบปะสินค้า	6S, 9A	จับคู่ ORD-24178 กับ RSL แล้วเปิดหน้าพิมพ์ใบปะ ข้อมูลบนใบปะต้องตรงกับ Order และเลข RSL	ผ่าน
รายงานต้นทุนต่อหน่วย	4S	คำนวณต้นทุนของ FSH-MLR-240 แล้วเปิดรายงาน ต้นทุนต่อหน่วยต้องตรงกับสูตรใน 4S	ผ่าน



รูปที่ 2-170 ผลการออกรายงานใบปะสินค้า



รูปที่ 2-171 ผลการออกรายงานต้นทุนต่อหน่วย

2.25 หน้าที่รับผิดชอบ

หัวหน้ากลุ่ม: นายธนภัทร พูลโภคา หมู่ 200

ตารางแสดงการแบ่งความรับผิดชอบงานตามรายการที่กำหนด งานใดที่ทำหลายคน หัวหน้าทีมระบุผู้รับผิดชอบหลัก

ตารางที่ 2-112 จำนวนงานของสมาชิกแต่ละคน

สมาชิก	sec	รหัส	รวมงาน (ชิ้น)
1. นายธนภัทร พูลโภคา	200	6710450911	16
2. นายอชิระ ภู่รุ่งเรือง	200	6710451488	17
3. นางสาวปิยากร ผลพานิช	200	6710451054	20

ตารางที่ 2-113 หน้าที่รับผิดชอบ

ที่	รายการ	วัตถุประสงค์	ขนาด	หน่วย	การตรวจสอบ	ผู้รับผิดชอบ
0	ปก	มีรูป รหัส ชื่อ นามสกุล ของสมาชิกทุกคน			หากไม่มีจะหัก 5 คะแนน	ปิยากร
1	สารบัญ	ใช้ในการหาเอกสารอื่นๆ	1	ชุด	มีสารบัญที่ระบุหน้าเอกสารที่ส่งทุกรายการ	ปิยากร
2	รูป	อ้างอิงรูป	1	ชุด	ให้มีเลขรูปอยู่ พร้อมระบุว่าเป็นรูปอะไร	ปิยากร
3	ตาราง	อ้างอิงตาราง	1	ชุด	ให้มีเลขตาราง พร้อมระบุว่าเป็นตารางอะไร	ปิยากร
4	Abstract	อธิบายงานทั้งหมดสั้นๆ	1	หน้า	ทั้งภาษาไทย และ ภาษาอังกฤษ	ปิยากร
5	บทนำ	แนะนำหน่วยงาน ที่มาที่ไปของหน่วยงาน	0.5	หน้า	ไม่ใช้ภาษาพูด เขียนจับใจความได้ มีการอ้างอิงถึงหน่วยงานที่จะไปพัฒนาระบบให้ ว่าประสบปัญหาอย่างไร	ปิยากร
6	ปัญหา	ประเด็นปัญหาที่ต้องการจะแก้ไขโดยใช้ Application	0.5	หน้า	แสดงข้อมูล อภิปราย ถึงปัญหาอย่างชัดเจน แต่ไม่ใช่การบอกว่าระบบมีปัญหาเนื่องจากไม่ได้ใช้ระบบคอมพิวเตอร์ โดยควรจะมีตัวเลขสนับสนุน เช่น ทำงานช้า ใช้เวลา 2 ชั่วโมงในการตรวจสอบยอดคลังสินค้า	ปิยากร
7	Business Process เดิม	Business Process	1	รูป	สัญญลักษณ์ถูกต้อง ชื่อต้องเป็น verb อ่านแล้วต้องชัดเจน เช่น "ตรวจสอบ" อ่านแล้วไม่เข้าใจว่าตรวจสอบอะไร ควรแก้ไขเป็น "ตรวจสอบสินค้าถึงจุดสั่งซื้อ" และ สวยงานดูสบายตา มีช่องไฟที่ดี และแสดง logic จากบนลงล่าง และ ซ้ายไปขวา และ มีเลข Process	ธนภัทร, อชิระ
8	คำอธิบาย Business Process เดิม	อธิบาย Business Process	0.5	หน้า	อธิบายการทำงานของ Business Process ตั้งแต่เริ่มจบครบทุกขั้นตอน โดยระบุ ใครทำอะไร ได้ผลอะไร	ธนภัทร, อชิระ

ที่	รายการ	วัตถุประสงค์	ขนาด	หน่วย	การตรวจสอบ	ผู้รับผิดชอบ
9	ปัญหา	อธิบายให้ทราบถึงปัญหาหลักๆของระบบ อาจจะมีการคำนวณให้เข้าใจถึงขนาดของปัญหาว่าเสียหายเท่าไหร่ หรือใช้เวลาไปเท่าไหร่ในการทำงาน	0.5	หน้า	เป็นปัญหาที่เกิดขึ้นกับระบบเดิม และ สามารถแก้ไขได้โดยระบบใหม่ด้วยคอมพิวเตอร์ ทั้งนี้ห้ามระบุว่าปัญหาคือการไม่ใช้คอมพิวเตอร์ ในการทำงาน และ ปัญหานี้ต้องวัดและตรวจสอบได้ว่าจะหายไปหรือลดลงเมื่อใช้ระบบคอมพิวเตอร์	ธนภัทร, อชิระ
10	Business Process ใหม่	Business Process ใหม่ (รูปต้องชี้ให้เห็นว่าปรับปรุงที่ processes ใด)	1	รูป	สัญญลักษณ์ถูกต้อง ชื่อต้องเป็น verb อ่านแล้วต้องชัดเจน อ่านแล้วไม่เข้าใจว่าทำอะไร และ สวยงานดูสบายตา มีช่องไฟที่ดี ไส้นัดกันน้อยที่สุดและแสดง logic จากบนลงล่าง และ ซ้ายไปขวา และ มีเลข Process	ธนภัทร, อชิระ
11	คำอธิบาย Business Process ใหม่	เขียนบรรยายว่าระบบใหม่มีปรับปรุงอะไรบ้าง	0.5	หน้า	อธิบายการทำงานของ Business Process ตั้งแต่เริ่มจบครบทุกขั้นตอน โดยระบุ ใครทำอะไร ได้ผลอะไร และให้ระบุว่า Process ใดเป็น manual และกระบวนการใดที่แก้ไขปัญหาในข้อปัญหา	ธนภัทร, อชิระ
12	Business Process ที่สัมพันธ์กับ use case	แสดงการวิเคราะห์เพื่อแปลงเป็น use case	1	รูป	แสดง use case บน business process ทั้ง uc ที่เป็น computerและ manual	ธนภัทร, อชิระ
13	ตารางจับคู่ระหว่าง BP & UC	แสดงความสัมพันธ์ของ UC และ BP	1	tables	ทุก UC ต้องมาจาก BP	ปิยากร
14	Use case diagram ก่อนปรับปรุง	รูป UC ที่ได้จากการวิเคราะห์จาก BP	5	UC	แสดง actor ตาม swim lane ชื่อต้องตรงกับรูป Business Process มีเลขกำกับ uc	ปิยากร
15	CRUD Tabe	ตารางแสดงความสัมพันธ์ระหว่าง UC/Etities	1	tables	แสดง uc ทั้งหมด และ Entity ที่สำคัญๆ ในระบบจาก การวิเคราะห์ noun ในกระบวนการทำงาน	ธนภัทร
16	คำอธิบายประกอบตาราง CRUD	คำอธิบายการวิเคราะห์ตาราง CRUD	0.5	หน้า	คำอธิบายแสดงการวิเคราะห์ และ ชี้ประเด็นให้ได้ว่าจากการวิเคราะห์แล้วยังขาด uc อะไร	ธนภัทร, อชิระ
17	Use case diagram ที่ทบทวนปรับปรุงแล้ว	UC diagram ที่แก้ไขเพื่อมเติมจากการวิเคราะห์ CRUD Table	1	รูป	uc ที่เพิ่มเติมจากการวิเคราะห์	ปิยากร
18	CRUD Table ทบทวน	ตารางแสดงความสัมพันธ์ระหว่าง UC/Etities ที่ทบทวนแล้ว พร้อมคำอธิบาย	1	tables	ตารางใหม่ที่ได้เพิ่ม uc และ CRUD ในระบบ	อชิระ
19	Use case description	แสดง logic การทำงานของแต่ละ uc	5	UC	แสดง actor ตาม uc #แสดง logic ของ system/actor สอดคล้องกับ user interface #query แสดงใน ERD ได้อย่างถูกต้อง การจัดการเนื้อหา ควรจะให้ uc description หนุ่ง ตามด้วย sequence diagram และ การเรียงลำดับให้ เรียน uc desc แต่ละอัน ตามด้วย sequence และ collaboration และไม่ควรจะแสดงแยก uc desc จนครบแล้วจึงค่อยไป diagram อื่น เพราะจะทำให้ดูยาก	ธนภัทร, อชิระ
20	ER Diagram	และข้อมูลที่ใช้ในระบบ	5	entities	ชื่อ ตรงกับ CRUD, มี PK/FK, relationship (ต้องไม่ซ้ำ), cardinality (ไม่ควรมี m:m ถ้ามี	ธนภัทร, อชิระ

ที่	รายการ	วัตถุประสงค์	ขนาด	หน่วย	การตรวจสอบ	ผู้รับผิดชอบ
					จะต้องแสดง ERD ที่เป็น logicsl schema อีกรูปหนึ่ง), total/particl participation	
21	ER Diagram ที่มี query	แสดงความสัมพันธ์ระหว่าง uc กับ ERD ว่าquery กระทำกับ entity ใด	1	รูป	แสดง query บน erd โดย ERD แสดงเฉพาะ entity-pk-fk เท่านั้น ยกเว้นกรณี updated Query ให้แสดง attribute(s) ที่ถูก update ด้วย	ธนภัทร, อชิระ
22	Query table	แสดง queries ทั้งหมดและ uc ของแต่ละ query	1	tables	แสดง query id, uc id, uc name, sql command	อชิระ
23	Table structure	แสดง data dictionary ของทุก table	5	tables	กลุ่มใดที่มีตารางน้อยก็จะถูกลดทอนตามสัดส่วน โดยประกอบไปด้วย attribute name, description, PK/FK, Not null ?, data type (size), range of data หรือการตรวจสอบความถูกต้อง, ตัวอย่างข้อมูล อย่างน้อย 5 รายการ	ธนภัทร, อชิระ
24	Sequence Diagram	แสดง logic การทำงานของแต่ละ uc ในรูปแบบลำดับเวลาการทำงาน	10	diagram	แสดงการทำงานร่วมกันของ object(calss) ต่างๆ โดย obj ที่ติดกับ actor ต้องเป็น user interface ห้ามใช้ชื่อว่า view และการเชื่อมต่อกับ db ควรใช้ database connect และแสดงให้เห็นจริงๆว่าโปรแกรมใช้	ปิยากร
25	Collaboration diagram	แสดง logic การทำงานของแต่ละ uc ในรูปแบบ messaging	10	diagram	แปลงจาก sequence diagram, object ต้องตรงกับ sequence diagram, ต้องมีเลขกำกับ message	ปิยากร
26	Class diagram	แสดงโครงสร้างของคลาสในระบบ	10	classes	Obj ทุกรายการในทุก sequence diagram จะต้องถูก abstraction เป็นคลาส ในรูปนี้และกำหนดความสัมพันธ์ของคลาสเป็น super/sub, aggregation, dependency และ association แต่ละคลาสมี attribute และ method	ปิยากร
27	State Diagram (ถ้ามี)	แสดง obj.state ที่สำคัญๆของระบบ	1	diagram	ต้องกำหนดให้ state เป็น class.state โดยระบุ class การเปลี่ยน จะกำหนดโดย [condition]event/action โดย action จะต้องเป็นชื่อ uc และเมื่อมีการเปลี่ยนค่า state จะต้องปรับปรุงใน db โดยคำสั่ง SQL	ธนภัทร, อชิระ
29	SW development platform	แสดง platform ของเครื่องมือต่างๆที่ใช้	1	รูป	แสดง รูปของ เครื่องมือต่างๆที่ใช้ในการพัฒนา (Stack of tools)	ปิยากร
30	Site map/UI structure	แสดงโครงสร้างของ application	8	screen	แสดงเป็น tree โดย root คือ login และให้แสดง tree ของแต่ละ ผู้ใช้	ปิยากร
31	User interface design	แสดงหน้าจอการทำงานของ actor	8	screen	แสดง screen ทั้งหมด โดยให้กำหนดชื่อของแต่ละ screen ตาม uc name	ปิยากร
32	Output design (report)	แสดงรายงานทั้งหมดในระบบ	2	reports	แสดงโครงสร้างของ report และ ตัวอย่างของรายงานที่ได้บันทึกข้อมูลผ่าน uc ต่างๆ	ปิยากร
33	SQL Statement ใน UC description	แสดง SQL command ทั้งหมดในระบบ	10	statements	แสดง Query number, uc id, sql command ไม่ควรใช้ AI generate เพราะจะซับซ้อนเกินไป โดยให้ทดสอบว่าทุก sql statement สามารถ รันได้	ธนภัทร, อชิระ

ที่	รายการ	วัตถุประสงค์	ขนาด	หน่วย	การตรวจสอบ	ผู้รับผิดชอบ
34	การทดสอบโปรแกรมทำงานตาม UC description	แสดงผลการทดสอบโปรแกรมต่างๆ	8	tables	แสดง uc id, uc name, capture screen แสดงการบันทึกและ การออกรายงานทั้งหมด	ธนภัทร, อชิระ, ปิยากร
35	ความพร้อม	แสดงความพร้อมของการนำเสนอ	1	งาน	มีสไลด์ (มีเลขหน้าสไลด์), รายงาน (pdf, word) และโปรแกรมพร้อมทำงาน และสมาชิกทุกคน ออนไลน์ และเปิดกล้องได้ หากไม่เปิดกล้องจะหัก 50%	ธนภัทร, อชิระ, ปิยากร
36	คะแนนการทำงาน	แสดงการแบ่งงานและผลการทำงาน	1	งาน	มีคะแนนการทำงานของทีมงาน โดยหัวหน้าตาม รูปแบบที่กำหนด	ปิยากร

2.25.1 คะแนนการทำงานของทีม

หัวหน้าทีม (นายธนภัทร พูลโภคา) เป็นผู้ให้คะแนนการทำงานของสมาชิกแต่ละคน คะแนนเต็ม 5

ตารางที่ 2-114 คะแนนการทำงานของทีม

สมาชิก	คะแนนที่ได้ (เต็ม 5)	เหตุผลที่ได้คะแนน
1. นายธนภัทร พูลโภคา	5	มีส่วนร่วมในการทำ Diagram ในส่วนของ Business Flow (As-Is & To-Be), ER Diagram และมีส่วนร่วมในการทำ CRUD Table (As-Is), Usecase Description และมีส่วนร่วมในการทำในส่วนของ Backend และ Database ของแอปพลิเคชัน
2. นายอชิระ ภู่รุ่งเรือง	5	มีส่วนร่วมในการทำ Diagram ในส่วนของ Business Flow (As-Is & To-Be), ER Diagram และมีส่วนร่วมในการทำ CRUD Table (To-Be), Query Table, Usecase Description และมีส่วนร่วมในการทำในส่วนของ Backend และ Database ของแอปพลิเคชัน
3. นางสาวปิยากร ผลพานิช	5	เป็นผู้ประสานงานหลักติดต่อและสอบถามกับเจ้าของกิจการ และเป็นผู้จัดทำรายงานหลัก มีส่วนร่วมในการทำ Diagram ในส่วนของ Usecase Diagram, Sequence Diagram, Collaboration Diagram และรับผิดชอบในส่วนของ UX/UI และ Frontend ของแอปพลิเคชัน